

AI: Intelligent systems with LangChain and LangGraph

Agentic AI
Volume 3
Release 1



Alessandro Ciambrone

**Intelligent systems with
LangChain and LangGraph
Agentic AI
Volume 3
Release 1**

CONTENTS

1	Tools and Tool-Using Agents	16
1.1	Tools as APIs: contracts, schemas, determinism, idempotency	17
1.1.1	A tool is not a helper function	17
1.1.2	The contract boundary	17
1.1.3	Schemas are operational constraints	18
1.1.4	Idempotency is what keeps retries from becoming incidents	21
1.1.5	Design rules that survive contact with production	23
1.2	LangChain tool abstractions (functions, Runnables, structured tools).....	25
1.2.1	Common tool authoring options.....	25
1.2.2	Plain functions as tools	25
1.2.3	Runnables as tools	27
1.2.4	Structured tools	28
1.2.5	Where all three abstractions meet	29
1.2.6	How to choose.....	29
1.3	Built-in tool categories (search, DB, HTTP, code, math).....	30
1.3.1	Search tools	30
1.3.2	Database tools	31
1.3.3	HTTP and API tools	34
1.3.4	Code tools	36
1.3.5	Math and calculator tools	38
1.3.6	Choosing the right family in real systems.....	39
1.4	Robust custom tools: validation, retries, timeouts, circuit breakers.....	40
1.4.1	Robustness starts at the tool boundary	40
1.4.2	Validation is schema plus runtime policy	40
1.4.3	Config, runtime, and ToolRuntime	40
1.4.4	Retries belong to transient failures.....	41
1.4.5	Timeouts are not optional.....	42
1.4.6	Circuit breakers are a different control	42
1.4.7	A complete example	43
1.4.8	Practical rules	48
1.5	Tool error handling: recoverable vs fatal; user-fixable vs system-fixable	50
1.5.1	The four classes of tool failure	50

- 1.5.2 Example: a trading agent that classifies failures at the tool boundary 52
 - 1.5.3 Practical rules 58
- 1.6 ReAct-style workflows and standard tool-calling patterns 60
 - 1.6.1 ReAct is the workflow on top of tool calling 60
 - 1.6.2 The canonical tool-calling loop 61
 - 1.6.3 Implementing the loop directly in LangChain 61
 - 1.6.4 Letting the runtime own the loop 63
 - 1.6.5 A LangGraph version of the same loop 64
 - 1.6.6 Practical guidance 68
- 1.7 The `create_agent()` component in depth..... 70
 - 1.7.1 The execution contract..... 70
 - 1.7.2 Reading the parameter surface correctly..... 71
 - 1.7.3 A fuller support assistant with `create_agent()` 71
 - 1.7.4 Middleware is where the real customization happens 74
 - 1.7.5 A custom middleware stack for a support assistant..... 76
 - 1.7.6 A production-control stack for a high-stakes refund assistant 81
 - 1.7.7 The middleware catalog 85
 - 1.7.8 State, runtime context, store, and checkpoints 91
 - 1.7.9 Structured output with `create_agent()` 93
 - 1.7.10 Streaming 95
 - 1.7.11 Where `create_agent()` stops being the right abstraction 97
- 1.8 Summary..... 98
- 2 Controlled Generation, ToolNode, and MCP 100
 - 2.1 Controlled generation & function calling: what guarantees you really get..... 101
 - 2.1.1 The guarantee ladder: from text to schemas to tool calls..... 101
 - 2.1.2 Structured output: the strongest guarantee for final answers..... 103
 - 2.1.3 Function calling: a proposal for action, not execution authority..... 104
 - 2.1.4 ToolNode: the execution boundary made explicit 107
 - 2.2 ToolNode mechanics in LangGraph 109
 - 2.2.1 From a tool call proposal to an executable graph step..... 109
 - 2.2.2 Tool results are messages before they are anything else..... 111
 - 2.2.3 State updates follow graph rules, not special tool rules 112
 - 2.2.4 Updating state from inside a tool..... 113
 - 2.2.5 Runtime injection inside tools..... 117

- 2.2.6 Multiple tool calls..... 120
 - 2.2.7 Error handling in `ToolNode` 121
 - 2.2.8 A short note about MCP in this execution model..... 123
- 2.3 MCP (Model Context Protocol): standardized tool interfaces and transport 124
 - 2.3.1 MCP and provider ecosystems: compatibility is real, sameness is not 125
 - 2.3.2 The interface MCP standardizes 125
 - 2.3.3 Transport in practice: local and remote servers..... 126
 - 2.3.4 `MultiServerMCPClient`: the LangChain bridge..... 126
 - 2.3.5 Loading capabilities: tools, resources, and prompts..... 131
 - 2.3.6 Runtime behavior: sessions, structured output, multimodal output, and callbacks 132
 - 2.3.7 Structured output, multimodal output, and callbacks..... 132
 - 2.3.8 Control behavior: interceptors, authorization, and state updates 133
 - 2.3.9 LangGraph can also be published through MCP 133
 - 2.3.10 What guarantee you really get from this layer 136
- 2.4 Security: tool permissioning, allowlists, sandboxing, secrets handling 137
 - 2.4.1 Security starts where controlled generation stops..... 137
 - 2.4.2 Tool permissioning begins with what the model is allowed to see..... 137
 - 2.4.3 Allowlists 140
 - 2.4.4 Sandboxing 141
 - 2.4.5 Secrets handling 142
- 2.5 Summary..... 144
- 3 Agents Planning, Routing, and Parrallelization 146
 - 3.1 Plan-and-solve and decomposition strategies (when planning helps vs harms).. 147
 - 3.1.1 Reactive loops versus upfront planning..... 147
 - 3.1.2 Planner-Executor-Solver as a mental model..... 148
 - 3.1.3 Fixed decomposition, dynamic decomposition, and lighter planning 149
 - 3.1.4 Structured handoffs are part of the design, not an afterthought..... 150
 - 3.1.5 When planning helps..... 150
 - 3.1.6 When planning hurts..... 151
 - 3.1.7 A practical rule for choosing decomposition..... 152
 - 3.1.8 LangGraph implementation of Planner-Executor-Solver 153
 - 3.2 Routing Strategies: Rule-Based, LLM-Based, Classifier-Based, Embedding-Based, and Hybrid 157

3.2.1	Routing as graph control flow	158
3.2.2	Rule-based routing	159
3.2.3	LLM-based routing.....	162
3.2.4	Classifier-based routing.....	164
3.2.5	Embedding-based semantic routing	165
3.2.6	Hybrid routing	169
3.2.7	Choosing the right routing architecture.....	173
3.3	Parallelization (fan-out/fan-in): map-reduce, voting, consensus, aggregation.....	176
3.3.1	Parallelization as workflow control	176
3.3.2	Map-reduce	178
3.3.3	Voting	183
3.3.4	Consensus	186
3.3.5	Peer-evaluated selection	186
3.3.6	Aggregation and fan-in contracts	191
3.3.7	Choosing the right parallel pattern.....	192
3.4	Integrating Routing And Parallelization with RAG AND Tools	194
3.4.1	The control-flow order.....	194
3.4.2	Where RAG fits in the integrated workflow	195
3.4.3	Where tools fit	195
3.4.4	A worked support example	196
3.4.5	How the graph executes and where synchronization matters	201
3.4.6	Design rules that keep the pattern useful.....	202
3.4.7	When a simpler design is better.....	203
3.5	Cost/latency control: tiered models, caching, early exits	204
3.5.1	Cost and latency are workflow problems, not just provider settings.....	204
3.5.2	Tiered models: spend intelligence where it matters	204
3.5.3	Caching: reuse work at the right boundary.....	206
3.5.4	Early exits: stop as soon as the answer is already known.....	208
3.5.5	Putting the three levers together.....	211
3.5.6	Common mistakes.....	212
3.6	Summary.....	213
4	Reflection, Reasoning, and Adaptation.....	215
4.1	Reflection and Verification Loops: Self-Critique, Verifier Patterns, and Fail-Closed Design	217

4.1.1	Reflection as a controlled workflow.....	217
4.1.2	Self-critique: useful, but limited.....	217
4.1.3	Verifier patterns and the assurance hierarchy.....	220
4.1.4	Claim-level verification	222
4.1.5	Process-level verification	224
4.1.6	Bounded loops and fail-closed outcomes.....	229
4.1.7	Integrated example: a bounded support-policy reflection loop.....	231
4.1.8	Extending verification to actions: human approval.....	235
4.1.9	Extending reflection across runs: validated memory	236
4.1.10	Implementation and evaluation in LangChain, LangGraph, and LangSmith 238	
4.1.11	Legacy evaluator classes and current equivalents	239
4.1.12	When reflection helps, and when it hurts.....	239
4.2	Search-based reasoning: tree-of-thought, best-of-N, MCTS-style variants.....	241
4.2.1	From single-pass reasoning to explicit search	241
4.2.2	Best-of-N: sample, score, and select	241
4.2.3	Tree-of-Thought as branching and pruning.....	246
4.2.4	Search control is the real engineering problem	254
4.2.5	Bounded sequential search with <code>RemainingSteps</code>	254
4.2.6	MCTS-style variants	259
4.2.7	Search in LangGraph: one graph, several search regimes	259
4.2.8	When search helps, and when it hurts	260
4.2.9	A practical escalation path.....	261
4.3	Using Explicit Feedback for Controlled Improvement.....	262
4.3.1	Ratings, labels, and human review are not the same thing.....	262
4.3.2	A simple architecture you can implement yourself.....	263
4.3.3	A minimal feedback pipeline.....	263
4.3.4	Pairwise review for comparing versions.....	266
4.3.5	Where LangGraph and LangSmith fit.....	267
4.3.6	Using reviewed feedback to guide application decisions	268
4.3.7	Practical design rules.....	274
4.4	Learning from implicit signals (clicks, conversions, trading outcomes)	275
4.4.1	Learning from behavior rather than labels.....	275
4.4.2	Clicks, conversions, and trading outcomes are different kinds of signals.....	275

- 4.4.3 What the Lang ecosystem gives you today 276
 - 4.4.4 A practical architecture for implicit-signal learning 277
 - 4.4.5 A working example 278
- 4.5 Implementing Runtime Adaptation: Context, Model Context, Retrieval Policy, and Graph Control..... 283
 - 4.5.1 Runtime context: supply trusted per-invocation inputs 284
 - 4.5.2 Model context: adapt instructions and capabilities 291
 - 4.5.3 Retrieval policy: adapt how evidence is gathered 291
 - 4.5.4 Graph control: branch, pause, resume, and recover 299
 - 4.5.5 Design rules that keep adaptation useful..... 300
- 4.6 Avoiding Degradation: Reward Hacking, Drift, and Overfitting to Evaluation Sets 302
 - 4.6.1 Why reflection systems still degrade 302
 - 4.6.2 Reward hacking: when the judge becomes the target 302
 - 4.6.3 Drift: when yesterday’s good behavior no longer fits today’s system 303
 - 4.6.4 Overfitting to eval sets: when benchmark gains stop meaning real gains..... 303
 - 4.6.5 A layered anti-degradation loop..... 304
 - 4.6.6 A bounded, fail-closed verifier workflow in LangGraph 305
 - 4.6.7 A small evaluation harness..... 308
 - 4.6.8 Practical design rules..... 315
- 4.7 Summary..... 316
- 5 Multi-Agent Collaboration..... 318
 - 5.1 Multi-agent patterns: manager/worker, specialists, debate, escalation..... 319
 - 5.1.1 Manager/worker: one agent coordinates, others execute..... 319
 - 5.1.2 Shared setup used by the examples in this chapter 320
 - 5.1.3 Manager/Worker 323
 - 5.1.4 Specialists: narrower agents with clearer responsibilities..... 325
 - 5.1.5 Adding lightweight escalation to the supervisor 327
 - 5.1.6 Debate and structured review 328
 - 5.1.7 Escalation 330
 - 5.1.8 Choosing the right pattern..... 332
 - 5.1.9 Practical design rules..... 332
 - 5.2 Communication Mechanisms: Handoffs, Shared Message Lists, Shared State Schemas, and A2A 334
 - 5.2.1 Communication is part of the architecture..... 334

5.2.2	Handoffs: transferring control without losing context	334
5.2.3	Shared message lists: agents work through one shared conversation.....	337
5.2.4	Shared state schemas: making collaboration explicit.....	341
5.2.5	A2A: communication beyond one process.....	344
5.2.6	Choosing the right communication mechanism.....	354
5.3	Multi-agent graphs in LangGraph: coordination, synchronization, stopping rules	356
5.3.1	A multi-agent graph is still one graph	356
5.3.2	Coordination means deciding who handles the next step	356
5.3.3	Synchronization is not thread management; it is graph semantics	357
5.3.4	Stopping rules should be designed, not improvised	358
5.3.5	Worked example: a coordinated support graph.....	359
5.4	Summary.....	368

About the author

Alessandro Ciambrone is a Chief Architect and Cloud Lead with over a decade of experience delivering large-scale digital transformations across banking, pensions, insurance, automotive, and the public sector. He has designed and led enterprise and solution architectures for complex organizations, focusing on platforms that must be scalable, resilient, secure, and cost-effective.

Across cloud, hybrid, and on-premises environments, Alessandro has built and modernized systems using serverless architectures, container-based APIs, microservices, and event-driven architectures. His work consistently balances delivery speed with the realities of production engineering: availability, observability, governance, compliance, and operational simplicity.

Alessandro is recognized for deep hands-on expertise across the major cloud ecosystems—Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure—and for translating architectural strategy into pragmatic implementation plans that teams can execute.

Over the past 18 months, he has focused heavily on applied AI engineering, experimenting with the Lang* ecosystem (LangChain, LangGraph, and LangSmith) to build agent-based and LLM-powered applications that can operate reliably in real workflows. That practical exploration—what works, what breaks, and how to design for traceability and control—directly informs the patterns and guidance in this book.

When he isn't designing systems, Alessandro teaches and writes, with a clear goal: help engineers turn complex technology into dependable, maintainable solutions they can ship with confidence. (alexciambrone@gmail.com)

Preface

Large language models become significantly more powerful when they can do more than generate text. Once a model can query databases, call APIs, use external tools, execute workflows, or trigger business operations, the engineering problem changes. The question is no longer only whether the model can produce a good answer. It is whether the system can control what the model is allowed to do, validate its decisions, recover from failures, and prevent incorrect actions from becoming real-world incidents.

This volume is about building those systems.

The central principle throughout the book is that agentic does not mean uncontrolled. An agent can choose actions, use tools, plan, route work, evaluate results, adapt its behaviour, and collaborate with other agents. But those capabilities must operate inside boundaries defined by ordinary software engineering. Permissions, validation, state, persistence, retries, timeouts, approval points, stopping conditions, and security remain responsibilities of the application runtime. The practical focus is the Lang ecosystem. LangChain provides the abstractions used to connect models with tools and build agent runtimes. LangGraph makes state, control flow, persistence, branching, parallel execution, and coordination explicit. LangSmith supports the evaluation and feedback processes needed to understand whether changes actually improve the system. Together, these components provide a practical foundation for moving from simple tool-calling assistants to more sophisticated agentic architectures.

A recurring theme is that additional autonomy should always have a reason. A deterministic rule is preferable when the decision is deterministic. A single agent is preferable when one agent can perform the work clearly. Planning, parallelization, reflection, adaptation, and multi-agent collaboration are valuable only when they solve a real limitation of the simpler design. More agents and more reasoning steps do not automatically produce a better system; they also introduce additional latency, cost, state, coordination, and failure modes. The examples continue to use customer support, e-commerce, and financial market analysis. These domains make the consequences of agent decisions visible. Looking up an order is different from issuing a refund. Retrieving a market price is different from placing a trade. Recommending a product is different from changing an order. Reusing these scenarios makes it possible to show how increasingly capable agentic patterns can be introduced without losing sight of the business operation being controlled.

This is a practical book for software engineers. You do not need to be a machine-learning researcher, but you do need to approach agentic AI as a software architecture problem rather than as an exercise in prompt engineering. Models will remain probabilistic. Reliable systems come from deciding where that uncertainty is acceptable and surrounding it with explicit contracts, observable execution, verification, and control. The objective of this volume is therefore not to build the most autonomous system possible. It is to show how to build the simplest agentic system that can perform the required work reliably, safely, and visibly.

What this book covers

This book is organised into two parts, designed to take you from controlled tool use and agent execution to more advanced orchestration, reasoning, adaptation, and multi-agent collaboration with the Lang* ecosystem

Part 1 — Tools and Agents

Chapter 1: Tools and Tool-Using Agents

You learn how tools turn an LLM from a text-generation component into a system that can interact with real software. The chapter covers tool contracts, schemas, validation, idempotency, retries, timeouts, circuit breakers, and error handling, before moving into ReAct-style tool-calling workflows and LangChain's `create_agent()` runtime. You also learn how middleware, runtime context, state, stores, checkpoints, structured output, streaming, and human approval provide the controls needed to operate tool-using agents safely.

Chapter 2: Controlled Generation, ToolNode, and MCP

This chapter explains how structured output and function calling constrain what a model can produce, while keeping execution authority in the application runtime. You learn how LangGraph's `ToolNode` executes tool calls, returns tool results, updates graph state, injects runtime information, handles multiple calls, and manages errors. The chapter then introduces the Model Context Protocol (MCP) as a standard interface for exposing tools, resources, and prompts across local and remote services, before examining permissioning, allowlists, sandboxing, authorization, and secrets management.

Part 2 — Agentic Workflows, Adaptation, and Collaboration

Chapter 3: Agents Planning, Routing, and Parallelization

You move beyond the basic tool-calling loop into richer control-flow patterns. The chapter compares reactive execution with plan-and-solve and other decomposition strategies, then examines rule-based, LLM-based, classifier-based, embedding-based, and hybrid routing. You also learn how to use parallel fan-out and fan-in for map-reduce, voting, consensus, peer evaluation, and aggregation. The chapter brings these patterns together with RAG and tools and shows how tiered models, caching, early exits, and explicit budgets can control cost and latency.

Chapter 4: Reflection, Reasoning, and Adaptation

This chapter explores how an agent can evaluate results and change direction without turning improvement into an uncontrolled loop. You learn about self-critique, verifier patterns, claim-level and process-level verification, bounded reflection, and fail-closed outcomes. The chapter also covers search-based reasoning through Best-of-N, Tree-of-Thought, and MCTS-style approaches, followed by explicit and implicit feedback, runtime adaptation of context, models, retrieval policy, and graph control. It concludes by examining reward hacking, drift, and evaluation-set overfitting so that improvements remain measurable and safe.

Chapter 5: Multi-Agent Collaboration

You learn when dividing responsibilities among several agents is more useful than continuing to expand a single agent. The chapter covers manager/worker architectures, specialist agents, structured debate, review, and escalation, followed by the main communication mechanisms used between agents: handoffs, shared message histories, shared state schemas, and Agent-to-Agent (A2A) communication. Finally, you build coordinated multi-agent workflows in LangGraph and examine routing, synchronization, shared-state updates, stopping rules, execution budgets, checkpoints, interrupts, and human escalation.

Code examples and repository

All code examples and supporting resources for this book are available in the companion GitHub repository:

[https://github.com/alexciambrone/Intelligent Systems with LangChain and LangGraph](https://github.com/alexciambrone/Intelligent_Systems_with_LangChain_and_LangGraph)

The repository is structured to mirror the book's progression, so you can follow along chapter by chapter, run the examples locally, and adapt them into your own projects.

Part 1 – Tools and Agents

Chapter 1: Tools and Tool-Using Agents

Chapter 2: Controlled Generation, ToolNode, and MCP

1 TOOLS AND TOOL-USING AGENTS

Tools are the point where an LLM application stops being only a text-generation system and starts interacting with real software. The moment a model can search the web, query a database, call an API, calculate a value, create a refund, or initiate another business operation, tool use becomes an application-design problem rather than simply a prompting technique. In this chapter, you will learn how to design tool-using agents around explicit contracts: what the model is allowed to request, what the runtime actually executes, and where validation, authorization, idempotency, retries, timeouts, and error handling belong. The recurring examples use customer support, e-commerce, and trading because these domains make the distinction between information and action particularly clear. Looking up an order or market quote may be relatively harmless; issuing a refund, creating a return, or submitting a trade changes real system state and therefore requires stronger controls. The chapter starts with the tool boundary itself and explains why tools should be treated as governed APIs with clear schemas rather than as helper functions exposed to a model. It then moves through LangChain's main tool abstractions, common built-in tool families, robust custom-tool design, and practical strategies for handling failures. From there, you will build the standard model-tool-observation loop that underlies modern tool calling and ReAct-style agents, first explicitly and then through LangGraph and LangChain's higher-level `create_agent()` runtime.

The final sections move from basic tool execution to production-oriented agent operation. You will see how runtime context, state, stores, checkpoints, middleware, human review, structured output, and streaming fit around the same fundamental tool loop, and when that abstraction should give way to an explicit LangGraph workflow. The objective throughout the chapter is not to give the model as many capabilities as possible. It is to expose the smallest, useful set of capabilities while keeping execution, trusted context, side effects, recovery, and control firmly in the application runtime.

1.1 TOOLS AS APIs: CONTRACTS, SCHEMAS, DETERMINISM, IDEMPOTENCY

1.1.1 A tool is not a helper function

The easiest way to misunderstand tools is to think of them as a convenience feature for the model. In production systems, a tool is an action interface exposed to the model through a contract. The model sees the tool contract, typically a name, description, and argument schema. Depending on the model provider and application configuration, the model may choose whether to call a tool, or the application may constrain or force tool use. When a tool is selected, the model emits a structured tool-call request. The actual execution belongs to a controlled runtime. In ordinary tool-calling flows, the model does not directly run your Python function or directly reach into your database. It proposes an action. The execution layer then decides whether that proposal is valid, authorized, and safe to execute. In many LangChain or LangGraph applications, that layer is your application code. In other designs, it may be a client runtime, an MCP server, or a provider-managed server-side tool. Once you look at tools this way, many design choices become less mysterious. A tool may wrap a local function, an HTTP API, a database lookup, a search backend, a workflow step, a code-execution sandbox, or a capability exposed by an MCP server. In all of those cases, the tool contract is the boundary between probabilistic model reasoning and controlled software execution.

Some platforms also offer built-in server-side tools, where the provider executes the tool rather than your application. The core design issue remains the same: the moment a model can trigger real behavior, you are designing an action interface, not merely writing a helper function. That is why tools matter so much in agentic systems. A support assistant that only generates text can explain your refund policy. A support assistant with tools can look up an order, check delivery status, verify the refund window against policy and order data, and then request a refund through an order service if the application permits that action. The second system is no longer just generating language. It is orchestrating capabilities through explicit contracts.

1.1.2 The contract boundary

A good tool contract is small, explicit, and stable. At minimum it needs a stable name, a description that tells the model when the tool should be used, an input schema that defines the allowed arguments, and an output shape that the rest of the system can rely on. The name is the identifier the model will emit when it proposes a call. The description teaches the model the intended job of the tool. The input schema constrains what the model is allowed to send. The output shape matters just as much, because the tool result has to come back into the workflow in a form the rest of the system can understand and trust. When this contract is vague, the model has too much room to guess. When it is explicit, the runtime has a clearer and safer execution boundary.

This is also where it helps to separate query tools from command tools. A query tool answers questions about the world as it currently is: order status, current inventory, latest quote, customer tier. A command tool changes the world: create a return request, apply a discount, place an order, cancel a shipment. This distinction affects runtime design. Read-only operations need precise arguments, predictable outputs, and appropriate access control. Side-

effecting operations also need authorization, audit trails, confirmation rules, idempotency, and protection against duplicate execution. If a retry repeats a read operation, the result may be inefficient or stale. If a retry repeats a command operation, the system may create a duplicate refund, order, cancellation, or trade.

Not every useful piece of information should be exposed as a tool. A return policy, product specification, pricing table, or account profile may be better represented as retrieved data, runtime context, graph state, or a resource. A retrieval tool can still be useful when the model needs to decide whether to fetch information on demand. However, static or already-available information should not be wrapped as a tool only because the agent framework supports tools. A clean contract boundary keeps executable capabilities separate from passive context. This makes the system easier to understand, secure, test, and operate because the runtime has a clear distinction between information the model can use and actions the system may execute.

1.1.3 Schemas are operational constraints

In tool-using systems, the schema turns a model-facing instruction into a runtime-checkable contract. It describes the arguments the model is expected to provide and gives the application a concrete structure to validate before execution. In LangChain, there are several ways to define tools, but two common approaches are enough to illustrate the design choice. For simple tools, LangChain can infer an input schema from the Python function signature and type hints. The tool description can come from the function docstring or from an explicit description. For stricter control, you can define the schema separately, commonly with a Pydantic model passed through `args_schema`. Schema inference is convenient. An explicit schema is usually better when the tool needs field descriptions, validation constraints, enums, examples, or a more precise machine-readable contract. The model is guided by the contract exposed to it, not by the internal implementation details of the Python function. A vague schema leaves more room for incomplete or ambiguous tool calls. A precise schema gives the model clearer guidance and gives the runtime stronger validation leverage.

A useful tool schema does three things. It narrows the accepted input space, explains the meaning of each field, and makes required and optional arguments explicit. The first example uses schema inference. LangChain uses the function signature, type hints, and available tool description to construct the tool schema automatically.

```
from __future__ import annotations
import json
from langchain.tools import tool

ORDERS = {
    "ORD-100": {
        "status": "delivered",
        "amount": 129.00,
        "currency": "EUR",
        "refundable": True
    }
}

@tool
def lookup_order_simple(order_id: str, include_history: bool = False) -> dict:
    # Return normalized order status data for a single order.
    order = ORDERS[order_id]
    return {
        "order id": order_id,
        "status": order["status"],
        "amount": order["amount"],
        "currency": order["currency"],
```



```

        "refundable": order["refundable"],
        "include_history": include_history
    }

if __name__ == "__main__":
    schema = lookup_order_simple.args_schema.model_json_schema()
    print("=== INFERRED SCHEMA ===")
    print(json.dumps(schema, indent = 2))
    print()
    print("=== TOOL RESULT ===")
    print(lookup_order_simple.invoke({
        "order_id": "ORD-100",
        "include_history": True
    }))

```

vol3_ch_01\src\lookup_order.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.lookup_order

=== INFERRED SCHEMA ===
{
  "description": "Return normalized order status data for a single order.",
  "properties": {
    "order_id": {
      "title": "Order Id",
      "type": "string"
    },
    "include_history": {
      "default": false,
      "title": "Include History",
      "type": "boolean"
    }
  },
  "required": [
    "order_id"
  ],
  "title": "lookup_order_simple",
  "type": "object"
}

=== TOOL RESULT ===
{'order_id': 'ORD-100', 'status': 'delivered', 'amount': 129.0, 'currency': 'EUR',
'refundable': True, 'include_history': True}

```

This version is compact and convenient, but the contract is limited to what can be derived from the function definition and tool description. The field types and default values are inferred, and required fields are identified, but there are no field-level descriptions, pattern constraints, enum values, examples, or custom validation rules. The second example uses an explicit Pydantic schema. Here the input contract is declared separately from the tool function, which gives the application tighter control over what is accepted at runtime and gives the model a clearer description of the expected arguments.

```

from __future__ import annotations
import json
from enum import Enum
from pydantic import BaseModel, Field
from langchain.tools import tool

ORDERS = {
    "ORD-100": {
        "status": "delivered",
        "amount": 129.00,
        "currency": "EUR",
        "refundable": True
    }
}

class Channel(str, Enum):
    WEB = "web"
    MOBILE = "mobile"
    STORE = "store"

class OrderLookupInput(BaseModel):

```

```

order_id: str = Field(
    ...,
    description = "Customer order identifier such as ORD-100",
    pattern = r"ORD-\d{3,}$",
    examples = ["ORD-100"]
)
include_history: bool = Field(
    default = False,
    description = "Whether to include recent order events"
)
max_events: int = Field(
    default = 5,
    ge = 1,
    le = 20,
    description = "Maximum number of order events to return"
)
channel: Channel = Field(
    ...,
    description = "Sales channel where the order was placed"
)

@tool(args_schema = OrderLookupInput)
def lookup_order(
    order_id: str,
    include_history: bool = False,
    max_events: int = 5,
    channel: Channel = Channel.WEB
) -> dict:
    # Return normalized order status data for a single order.
    order = ORDERS[order_id]
    return {
        "order_id": order_id,
        "status": order["status"],
        "amount": order["amount"],
        "currency": order["currency"],
        "refundable": order["refundable"],
        "include_history": include_history,
        "max_events": max_events,
        "channel": channel.value
    }

if __name__ == "__main__":
    schema = lookup_order.args_schema.model_json_schema()
    print("=== EXPLICIT PYDANTIC SCHEMA ===")
    print(json.dumps(schema, indent = 2))
    print()
    print("=== TOOL RESULT ===")
    print(lookup_order.invoke({
        "order_id": "ORD-100",
        "include history": True,
        "max_events": 3,
        "channel": "web"
    }))

```

vol3_ch_01\src\lookup_order_pydantic.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.lookup_order_pydantic

=== EXPLICIT PYDANTIC SCHEMA ===
{
  "$defs": {
    "Channel": {
      "enum": [
        "web",
        "mobile",
        "store"
      ],
      "title": "Channel",
      "type": "string"
    }
  },
  "properties": {
    "order_id": {
      "description": "Customer order identifier such as ORD-100",
      "examples": [
        "ORD-100"
      ],
      "pattern": "^ORD-\\d{3,}$",

```

```

    "title": "Order Id",
    "type": "string"
  },
  "include_history": {
    "default": false,
    "description": "Whether to include recent order events",
    "title": "Include History",
    "type": "boolean"
  },
  "max_events": {
    "default": 5,
    "description": "Maximum number of order events to return",
    "maximum": 20,
    "minimum": 1,
    "title": "Max Events",
    "type": "integer"
  },
  "channel": {
    "$ref": "#/$defs/Channel",
    "description": "Sales channel where the order was placed"
  }
},
"required": [
  "order_id",
  "channel"
],
"title": "OrderLookupInput",
"type": "object"
}

=== TOOL RESULT ===
{'order_id': 'ORD-100', 'status': 'delivered', 'amount': 129.0, 'currency': 'EUR',
'refundable': True, 'include_history': True, 'max_events': 3, 'channel': 'web'}

```

The contrast is the important point. In the inferred version, the schema is derived from the function. In the explicit version, the schema is a separate contract object. That separation lets you define field descriptions, regex constraints, examples, enum values, numeric limits, and validation rules. Some of these rules are visible in the generated JSON Schema. Others may run only during runtime validation. Both are useful, but they serve different purposes: the model-facing schema guides tool-call generation, while runtime validation decides whether the call is acceptable. The examples are intentionally small so that the boundary is easy to see. In both cases, the input schema is machine-readable and inspectable, and the return value is structured rather than free-form text. The explicit Pydantic version is usually the better choice when the tool affects business logic, customer data, orders, refunds, market actions, or any workflow where invalid arguments would be costly. This leads to an important design rule: include in the model-visible tool schema only the inputs that the model is allowed to choose. Authenticated user identifiers, tenant identifiers, ACL scopes, trace identifiers, and session metadata should come from trusted runtime context, not from model-generated arguments. The model can propose a tool call, but the application remains responsible for deciding whether the call is valid, authorized, and safe to execute. Inferred schemas are appropriate for demos and simple read-only tools. Explicit schemas are preferable when correctness, validation, authorization, auditability, or business impact matter.

1.1.4 Idempotency is what keeps retries from becoming incidents

Idempotency is one of those ideas that sounds dry until the first duplicate refund, duplicate order, or duplicate trade arrives and starts chewing through your weekend. In plain terms, an idempotent operation is safe to run more than once because repeated execution has the same effect as a single successful execution. Modern distributed-system practice treats mutating operations as retry-safe only when they are idempotent. Payment APIs commonly use

idempotency keys for this reason: the same logical request can be retried without performing the same business operation twice. This becomes even more important in LangGraph because durable execution changes the failure model. When a graph is checkpointed and later resumed, execution does not necessarily continue from the exact Python line where it stopped. A failed or interrupted unit of work may run again from a safe checkpoint boundary. The design consequence is simple: side-effecting operations must be idempotent, and non-deterministic or external API work must be isolated behind a boundary that can tolerate re-execution. In the Functional API, that usually means placing API calls inside task functions and making those tasks idempotent. In graph-based workflows, it means designing side-effecting nodes so that retry or replay cannot create duplicate refunds, orders, notifications, or trades. A good mental model is to split tools into reads and writes. Read tools are usually easy to retry. Checking an order status or reading a quote can be repeated because they are idempotent by nature. Write tools are different. A refund request, discount application, shipment cancellation, or trade placement should be idempotent by contract. That usually means one of two things: either the tool accepts an explicit idempotency token, or the backend derives a stable business key that uniquely identifies the intended action and returns the previously recorded result when the same action is requested again. The following example shows the pattern:

```
from __future__ import annotations
from enum import Enum
from pydantic import BaseModel, Field
from langchain.tools import tool

ORDERS = {
    "ORD-100": {
        "status": "delivered",
        "refundable_amount": 129.00
    }
}

REFUNDS: dict[str, dict] = {}

class RefundReason(str, Enum):
    DAMAGED = "damaged"
    WRONG_ITEM = "wrong_item"
    LATE_DELIVERY = "late_delivery"

class RefundInput(BaseModel):
    order_id: str = Field(..., description="Order identifier such as ORD-100")
    reason: RefundReason = Field(..., description="Why the customer is asking for a refund")
    amount: float = Field(..., gt = 0, description="Refund amount in euros")

@tool(
    args_schema=RefundInput,
    description="Create a refund request or return the existing one for the same business action."
)
def request_refund(order_id: str, reason: RefundReason, amount: float) -> dict:
    order = ORDERS[order_id]

    if amount > order["refundable_amount"]:
        return {
            "ok": False,
            "error": "amount_exceeds_refundable_amount"
        }

    # Domain-level idempotency key.
    # In a real system, store this record in durable storage.
    idempotency_key = f"refund:{order_id}:{reason.value}:{amount:.2f}"

    if idempotency_key in REFUNDS:
        return {
            "ok": True,
            "replayed": True,
            **REFUNDS[idempotency_key]
```

```

    }

    refund = {
        "refund_id": f"rf_{len(REFUNDS) + 1:04d}",
        "order_id": order_id,
        "reason": reason.value,
        "amount": amount,
        "status": "accepted"
    }
    REFUNDS[idempotency_key] = refund

    return {
        "ok": True,
        "replayed": False,
        **refund
    }

if __name__ == "__main__":
    first = request_refund.invoke(
        {"order_id": "ORD-100", "reason": "damaged", "amount": 129.0}
    )
    second = request_refund.invoke(
        {"order_id": "ORD-100", "reason": "damaged", "amount": 129.0}
    )

    print(first)
    print(second)

```

vol3_ch_01\src\idempotency.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.lookup_order
{'ok': True, 'replayed': False, 'refund_id': 'rf_0001', 'order_id': 'ORD-100', 'reason': 'damaged', 'amount': 129.0, 'status': 'accepted'}
{'ok': True, 'replayed': True, 'refund_id': 'rf_0001', 'order_id': 'ORD-100', 'reason': 'damaged', 'amount': 129.0, 'status': 'accepted'}

```

The important part is not the tiny in-memory store. The important part is the contract decision. The second call does not create a second refund request. It returns the same business result for the same business action. In a real system, the idempotency record would live in durable storage and would usually be coupled to audit metadata, timestamps, and operator traceability. But the core rule remains the same: a side-effecting tool must be safe under retry and replay.

1.1.5 Design rules that survive contact with production

A good way to close this section is to turn the discussion into a small set of rules that remain useful after the demo code is gone. Tools are easy to make look elegant in examples. They become much less elegant once they cross trust boundaries, touch external systems, and start being retried, resumed, audited, and operated by other people.

- First, design the contract before you think about the decorator or framework abstraction. Decide what the tool is for, what inputs it accepts, what shape it returns, and which part of the system is responsible for actually performing the work. The decorator is just packaging. The real design work is deciding the boundary between model choice and application control.
- Second, expose only the inputs the model is genuinely allowed to choose. Apply the contract boundary introduced above: business arguments belong in the schema; trusted identity, tenant, permission, trace, and session values belong in runtime-controlled context. This keeps the callable surface smaller and avoids asking the model to supply values it must never control.
- Third, keep query tools and command tools separate. A query tool reads information. A command tool changes state. The distinction has already been established; the operational

consequence is that commands need stronger controls because retries, resumes, or repeated calls can otherwise duplicate real-world side effects.

- Fourth, make schemas narrow and explicit. When the schema is vague, the model has to guess. When the schema is explicit, the runtime has a much better chance of rejecting bad input before anything important happens. Pydantic is useful here because it can generate JSON Schema and lets you define descriptions, enums, examples, patterns, and numeric constraints as part of the input contract. OpenAI's Structured Outputs can further constrain model output to a supplied JSON Schema when strict mode is used, but that does not remove the need for application-side validation. Provider-side constraints help; they do not absolve your runtime of responsibility. The bouncer still has to check IDs at the door.
- Fifth, push determinism into the tool and workflow layers rather than expecting it from the model alone. Model settings can improve consistency, but predictable behavior comes from explicit code paths, stable schemas, controlled side effects, and workflow logic that is clear about what happens next.
- Sixth, treat idempotency as a first-class property of any tool that changes external state. In agent systems, a call may be repeated because of a timeout, transient failure, workflow resume, or retry policy. If the operation is not idempotent, one intended action can become several actual actions. Tools that submit orders, issue refunds, create tickets, or send notifications must therefore be replay-safe by design.

Taken together, these rules produce a healthier view of tools. A tool is a governed interface between model decisions and system behavior. The contract defines the boundary, the schema constrains the call, the workflow controls execution, and idempotency protects state-changing operations from repeated mutation.

1.2 LANGCHAIN TOOL ABSTRACTIONS (FUNCTIONS, RUNNABLES, STRUCTURED TOOLS)

LangChain provides several ways to define application tools. These options exist because real systems do not always start from the same kind of code. Sometimes the tool is a plain Python function. Sometimes the capability already exists as a Runnable or LCEL pipeline. Sometimes the input contract is important enough to define explicitly before wiring it to execution logic. The authoring abstractions differ in how the tool is created, but they all produce the same kind of model-facing boundary introduced in section 1.1: a named capability with a description, input schema, and runtime-controlled execution path.

1.2.1 Common tool authoring options

The simplest entry point is a Python function or coroutine. The `@tool` decorator converts it into a LangChain tool. By default, the function name becomes the tool name, the docstring becomes the tool description, and the type hints define the input schema. These defaults can be overridden when the tool needs a clearer name, a more precise description, or an explicit schema. This is usually the right starting point when the capability is already implemented as direct application logic. A second option is to start from a Runnable. In LangChain, a Runnable is a unit of work that can be invoked, batched, streamed, transformed, and composed. This matters because a tool does not have to wrap only a single leaf function. It can expose an existing pipeline, such as a normalization step, classifier, retrieval chain, or LCEL sequence, provided the pipeline has a clear input boundary.

`Runnable.as_tool()` creates a `BaseTool` from a Runnable. Where possible, LangChain infers the tool schema from the Runnable input schema. If the Runnable input type is too generic, the tool definition should provide schema information explicitly, for example with an argument schema or argument types.

A third option is `StructuredTool`. This is useful when you want to construct the tool explicitly and keep close control over its name, description, function, coroutine, and argument schema. `StructuredTool` is especially relevant for tools with multiple named inputs or a contract that should be reviewed and maintained as part of the application design. `StructuredTool.from_function()` can build a tool from a synchronous function or coroutine. It can infer a schema from the function signature when appropriate, and it also accepts an explicit `args_schema` when the contract should be defined directly. The practical distinction is about authoring style, not runtime semantics. Use `@tool` when the capability is naturally expressed as a Python function. Use `Runnable.as_tool()` when the capability already exists as a Runnable or LCEL pipeline. Use `StructuredTool` when explicit construction gives you better control over the tool contract and execution configuration. In all three cases, the result is a tool interface that can be described to the model and invoked by the runtime.

1.2.2 Plain functions as tools

Plain function tools are the best starting point because they keep the boundary visible. When you define a function with `@tool`, you are not only wrapping code. You are declaring a contract that the model will see. That is why the docs require type hints that define the schema.

It is also why the docstring matters because it becomes part of the model-facing description that helps the model decide when the tool should be used. A vague docstring produces a vague tool. A precise docstring produces a tool the model can choose more reliably. In practice, function tools work best when the capability is already small, deterministic, and naturally described by a normal function signature. An order lookup, a refund-window calculation, or a small policy checker are all good examples. The code stays ordinary Python, and the tool wrapper simply makes that capability visible to a model.

```
from __future__ import annotations
import json
from langchain.tools import tool

ORDERS = {
    "ORD-100": {"status": "delivered", "days_since_delivery": 12, "amount": 129.00},
    "ORD-101": {"status": "processing", "days_since_delivery": 0, "amount": 59.00}
}

@tool
def lookup_order(order_id: str) -> dict:
    """Return basic order facts for a single order."""
    order = ORDERS[order_id]
    return {
        "order_id": order_id,
        "status": order["status"],
        "days_since_delivery": order["days_since_delivery"],
        "amount": order["amount"]
    }

if __name__ == "__main__":
    print("=== TOOL SCHEMA ===")
    print(json.dumps(lookup_order.args_schema.model_json_schema(), indent = 2))
    print()
    print("=== TOOL RESULT ===")
    print(lookup_order.invoke({"order_id": "ORD-100"}))
```

vol3_ch_01\scr\tool.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.tool

=== TOOL SCHEMA ===
{
  "description": "Return basic order facts for a single order.",
  "properties": {
    "order id": {
      "title": "Order Id",
      "type": "string"
    }
  },
  "required": [
    "order_id"
  ],
  "title": "lookup_order",
  "type": "object"
}

=== TOOL RESULT ===
{'order_id': 'ORD-100', 'status': 'delivered', 'days_since_delivery': 12, 'amount': 129.0}
```

The model sees a tool called `lookup_order`, a description taken from the docstring, and a schema derived from the function signature. The runtime can invoke the same tool directly, which makes testing much easier than trying to test tool calling only through a live model loop. The same contract boundary still applies: only model-choosable business inputs should appear in the schema. Runtime-managed values can be injected through the framework's runtime mechanisms without becoming model-generated arguments.

1.2.3 Runnables as tools

Runnable-based tools are useful when the capability you want to expose is already built as a LangChain pipeline rather than as a single Python function. A normal tool often wraps one operation, such as calling an API, searching a database, or running a utility function. A Runnable-based tool can expose a reusable workflow. For example, an internal support-policy check might already combine several steps: normalize the user request, classify the policy area, retrieve the relevant policy text, and return a structured result. If that workflow is already implemented as a Runnable, you can expose it as a tool instead of rewriting it as a separate function. LangChain provides `as_tool()` for this purpose. It converts a Runnable into a `BaseTool`. When the Runnable has a clear input type, LangChain can often infer the tool schema automatically. When the Runnable accepts a generic dictionary or the inferred schema is not precise enough, provide an explicit `args_schema` or `arg_types`.

```
from typing_extensions import TypedDict
from langchain_core.runnables import RunnableLambda

class ReturnCheckInput(TypedDict):
    days_since_delivery: int
    item_opened: bool
    damaged: bool

def normalize(data: ReturnCheckInput) -> ReturnCheckInput:
    return {
        "days_since_delivery": int(data["days_since_delivery"]),
        "item_opened": bool(data["item_opened"]),
        "damaged": bool(data["damaged"])
    }

def evaluate(data: ReturnCheckInput) -> dict:
    eligible = (
        data["damaged"]
        or (data["days_since_delivery"] <= 30 and not data["item_opened"])
    )
    return {
        "eligible": eligible,
        "reason": (
            "damaged item"
            if data["damaged"]
            else "within 30 days and unopened"
            if eligible
            else "outside standard policy"
        )
    }

return_policy_pipeline = RunnableLambda(normalize) | RunnableLambda(evaluate)

return_policy_tool = return_policy_pipeline.as_tool(
    name="evaluate_return_policy",
    description="Evaluate whether a return appears eligible under the standard policy.",
    arg_types={
        "days_since_delivery": int,
        "item_opened": bool,
        "damaged": bool
    }
)

if __name__ == "__main__":
    result = return_policy_tool.invoke(
        {"days_since_delivery": 12, "item_opened": False, "damaged": False}
    )
    print(result)
```

vol3_ch_01\src\as_tool.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.as_tool

return_policy_tool = return_policy_pipeline.as_tool(
{'eligible': True, 'reason': 'within 30 days and unopened'}
```

This is the core value of Runnable-backed tools. You are not duplicating logic just to satisfy the tool system. You are promoting an existing executable pipeline into the tool layer. The moment its input boundary is clear enough, it can become a tool. That makes the Runnable abstraction especially useful in systems where the same capability must be reusable both inside ordinary application code and inside an agent loop.

1.2.4 Structured tools

Structured tools are useful when a tool needs several named inputs rather than one simple value. This is common in business applications. A product search tool may need a category, maximum price, currency, and stock preference. A trading analysis tool may need a symbol, date range, and interval. A refund tool may need an order ID, reason code, and package condition. In these cases, the input is not just a single string. It is a small object with fields that must be clearly defined. `StructuredTool` lets you define that contract explicitly. This makes the tool easier for the model to call correctly and easier for the runtime to validate. The schema can describe which fields are required, which values are allowed, and what each field means. `StructuredTool.from_function()` can create a structured tool from a synchronous or asynchronous function. It can infer a schema from the function signature, or you can provide an explicit `args_schema` when you need more control. Use a structured tool when the arguments are part of the design of the operation. If the fields represent business rules, validation rules, or safety controls, they should be visible in the tool contract rather than hidden inside the function body.

```
from __future__ import annotations
from typing import Literal
from pydantic import BaseModel, Field
from langchain_core.tools import StructuredTool

CATALOG = [
    {"sku": "LAP-100", "category": "laptop", "price": 999.0, "in_stock": True},
    {"sku": "LAP-200", "category": "laptop", "price": 1499.0, "in_stock": False},
    {"sku": "MON-100", "category": "monitor", "price": 299.0, "in_stock": True}
]

class ProductSearchInput(BaseModel):
    category: Literal["laptop", "monitor"] = Field(description="Product category")
    max_price: float | None = Field(default=None, description="Maximum price limit")
    in_stock_only: bool = Field(default=True, description="Return only in-stock items")
    limit: int = Field(default=5, ge=1, le=20, description="Maximum number of results")

def search_catalog(
    category: str,
    max_price: float | None = None,
    in_stock_only: bool = True,
    limit: int = 5
) -> list[dict]:
    results = []
    for item in CATALOG:
        if item["category"] != category:
            continue
        if max_price is not None and item["price"] > max_price:
            continue
        if in_stock_only and not item["in_stock"]:
            continue
        results.append(item)
    return results[:limit]

catalog_search_tool = StructuredTool.from_function(
    func = search_catalog,
    name = "catalog_search",
    description = "Search the e-commerce catalog using structured filters.",
    args_schema = ProductSearchInput
)
```

```
if __name__ == "__main__":
    print(
        catalog_search_tool.invoke(
            {
                "category": "laptop",
                "max_price": 1200,
                "in_stock_only": True,
                "limit": 3
            }
        )
    )
```

vol3_ch_01\scr\structured_tool.py

The gain here is not just nicer code. The gain is a clearer contract for the model and a clearer interface for the rest of the application. Required fields, enums, descriptions, and numeric constraints all become part of the schema rather than hidden assumptions inside Python code. That is why structured tools are such a good fit for tool inputs that are closer to business objects than to casual strings.

1.2.5 Where all three abstractions meet

The three authoring styles look different in code, but they converge on the same runtime shape. A tool can start as a Python function, a Runnable, or a StructuredTool. Once exposed, it becomes a named capability with a description, input schema, and invocation path. In a simple flow, the application executes the selected tool and returns the result to the model. In an agent flow, the same pattern is wrapped in a loop: the runtime inspects model output, executes requested tools, records tool results, and calls the model again. The tool remains a focused unit of capability; the agent runtime manages the control flow around it. This is why tool definitions should stay small, precise, and limited to the arguments the model is allowed to choose.

1.2.6 How to choose

Choose the tool abstraction based on where the capability already exists and how explicit the input contract needs to be.

- Use a plain function tool when the capability is already implemented as a simple Python function or coroutine. This is the best starting point for direct operations with a small input surface, such as looking up an order, calculating a refund estimate, or checking product availability.
- Use a Runnable-backed tool when the capability already exists as a LangChain workflow. This is useful when the logic is already implemented as an LCEL chain, retrieval pipeline, classifier, normalizer, or other reusable Runnable. In this case, `Runnable.as_tool()` lets you expose the existing pipeline without rewriting it as a separate function.
- Use `StructuredTool` when explicit construction gives you better control over the tool contract. This is useful when the tool has several named inputs, when the schema should be reviewed as part of the application design, or when validation rules are important. A refund tool, trading query, or product search tool usually benefits from this style because the fields are part of the business contract.

1.3 BUILT-IN TOOL CATEGORIES (SEARCH, DB, HTTP, CODE, MATH)

Once you understand tools as contracts, the next practical question is much less philosophical and much more architectural: what kind of tool should the agent use for this job? The integrations catalog is broader than the five families in this section and changes over time. It includes tools and toolkits for search, code execution, web browsing, databases, productivity systems, finance, and integration platforms. That broader catalog matters, but the five families in this section are the ones you will reach for most often when turning a model from a text generator into a system that can fetch facts, query structured data, call external services, compute deterministically, and manipulate information without guessing.

A useful way to think about these families is that they are not interchangeable. They all expose callable surfaces to a model, but they solve different truth problems. Search tools help when the truth lives outside your system and may have changed recently. Database tools help when the truth lives in structured systems of record. HTTP tools help when the truth or action surface is behind an external API. Code tools help when the task is procedural or computational. Math tools help when precision matters more than eloquence. If you pick the wrong family, the agent can still produce a fluent answer, but it will do so by leaning on the wrong evidence boundary. That is how clever-looking systems become unreliable in wonderfully annoying ways.

1.3.1 Search tools

Search tools retrieve current external information from the web when the answer cannot safely come from the model's training data alone. The specific provider list changes over time, so treat search as a capability family rather than a fixed brand inventory. The stable design point is the job description: search tools are appropriate when freshness, public web evidence, and source metadata matter. In a support assistant, that might mean checking a carrier status page. In an e-commerce assistant, it might mean looking up a manufacturer announcement or public recall notice. In a trading assistant, it often means fetching recent market-moving information before generating an explanation. Search outputs vary by provider, but they commonly include source-oriented evidence such as URL, title, snippet, page content, published date, images, or a direct answer. They are not the same thing as retrieval over your own private indexed corpus. That related design problem belongs to the retrieval and RAG side of the stack, not to the web-search category. Please install ddgs library before running this example

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m pip install -U ddgs
```

```
from langchain_community.tools import DuckDuckGoSearchRun
search = DuckDuckGoSearchRun()
result = search.invoke("latest ECB interest rate decision")
print(result)
```

vol3_ch_01\src\search_tool.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.search_tool
```

```
ECB's interest rate decision is due out today and the bank is widely expected to remain on hold keeping the refinancing rate at 4.50%. Currently EUR OIS imply a probability of 94%
```

```
for such a scenario to materialise. Major European stock markets opened mixed on Friday following the European Central Bank's (ECB) latest interest rate decision. On the data front, Germany reported a 1% increase in industrial output for January, surpassing the 0.5% growth anticipated by analysts. The EUR/USD pair rose after the latest ECB interest rate decision. The bank left its deposit facility rate at -0.50% and interest rate at zero. The EUR/USD pair tilted higher after the latest ECB interest rate decision. The 4H chart... At 10:15 p.m. Sydney time, we get the ECB interest rate decision, deposit facility rate, marginal lending rate, and the monetary policy statement. The key event follows 30 minutes later: the press conference with Christine Lagarde. Asian and European stock markets mostly fell Thursday before the European Central Bank's latest interest rate decision, after hotter-than-expected US inflation dented hopes for Federal Reserve rate reduction any time soon...
```

This is a good fit when the assistant needs fresh public information and you want the model to consume search results rather than improvise from stale internal knowledge. It is a poor fit when the truth already exists inside your own structured systems or private knowledge base. In that case, using search is like sending an intern to wander the internet because your own filing cabinet felt too straightforward.

1.3.2 Database tools

Database tools are the family for structured truth. In the current integrations catalog, Database is a first-class category and includes toolkits such as SQLDatabase Toolkit, Spark SQL Toolkit, Cassandra Database Toolkit, MCP Toolbox, Drasi Toolkit, and Stardog. The category exists because many business answers are not “document answers” at all. They are database answers. An order exists or it does not. A shipment is delivered or it is not. A customer has a plan tier, an account balance, a transaction history, or a portfolio position. These are not things you want the model to approximate semantically. They are things you want it to query exactly.

Before exposing SQL tools, configure the database user as if the model will eventually generate a bad query: read-only where possible, restricted schema access, explicit row or tenant boundaries, query limits, and no privileged DDL/DML permissions unless the workflow is specifically designed for controlled writes. The relational SQL flow is especially instructive because it shows what a serious database tool loop looks like. The agent is expected to inspect the available tables, inspect the schema of the relevant tables, formulate a syntactically correct query, double-check that query, execute it, and then answer from the results. Even in a small example, one practical concern appears immediately: an agent should not be allowed to discover or query the whole database just because the connection happens to point there. The wrapper can help reduce visibility. SQLDatabase supports `include_tables`, `ignore_tables`, and `view_support`, which makes it possible to expose only a narrow set of tables or views to the tool layer. That matters because the model’s query-planning loop begins by inspecting what is available. If you reduce what is visible, you reduce what the model can reason over in the first place. But this is only part of the story. The real security boundary is not the toolkit. It is the database connection and the rights attached to it. For SQLite, this usually means treating the problem as one of exposure and connection mode rather than role management. SQLite does not implement GRANT and REVOKE in the way a server database does, so you normally control access by exposing only safe views and by opening the database in read-only mode. That works well for assistants because SQLite views are read-only unless you explicitly add INSTEAD OF triggers, and SQLite URI mode supports opening the database with `mode=ro`. In other words, if the assistant only needs to read order status, do not point it at the whole schema with full write access and then hope prompt

instructions will compensate. Expose a narrow projection of the data and make the connection itself read-only.

```
from pathlib import Path
import sqlite3
from langchain_openai import ChatOpenAI
from langchain_community.utilities import SQLiteDatabase
from langchain_community.agent_toolkits import SQLiteDatabaseToolkit
from .config import OPENAI_API_KEY

db_path = Path("shop.db")

# Build the demo database
conn = sqlite3.connect(db_path)
conn.executescript("""
DROP TABLE IF EXISTS orders;

CREATE TABLE orders (
    order_id TEXT PRIMARY KEY,
    customer_name TEXT NOT NULL,
    total_amount REAL NOT NULL,
    status TEXT NOT NULL
);

INSERT INTO orders VALUES
    ('ORD-100', 'Alice', 129.00, 'delivered'),
    ('ORD-101', 'Bob', 42.50, 'processing'),
    ('ORD-102', 'Carla', 310.00, 'delivered');

-- Expose only a safe, read-only projection for the agent.
DROP VIEW IF EXISTS agent_orders;
CREATE VIEW agent_orders AS
SELECT
    order_id,
    total_amount,
    status
FROM orders;
""")
conn.commit()
conn.close()

# Open SQLite in read-only mode.
# SQLAlchemy documents SQLite URI mode=ro with uri=true.
readonly_uri = f"sqlite:///file:{db_path}?mode=ro&uri=true"

# Only expose the view, not the base table.
db = SQLiteDatabase.from_uri(
    readonly_uri,
    include_tables = ["agent_orders"],
    view_support = True
)

model = ChatOpenAI(model = "gpt-4.1-mini", temperature = 0)
toolkit = SQLiteDatabaseToolkit(db = db, llm = model)
tools = {tool.name: tool for tool in toolkit.get_tools()}

print("Tables:", tools["sql_db_list_tables"].invoke(""))
print("Schema:", tools["sql_db_schema"].invoke("agent_orders"))

print(
    "Top order:",
    tools["sql_db_query"].invoke(
        "SELECT order_id, total_amount, status "
        "FROM agent_orders "
        "ORDER BY total_amount DESC "
        "LIMIT 1;"
    )
)

# This should fail because the connection is read-only.
try:
    print(
        tools["sql_db_query"].invoke(
            "UPDATE orders SET status = 'refunded' WHERE order_id = 'ORD-100';"
        )
    )
except Exception as e:
    print("Write blocked:", e)
```

vol3_ch_01\scr\db_tool.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.db_tool

Tables: agent_orders
Schema:
CREATE TABLE agent_orders (
  order_id TEXT,
  total_amount REAL,
  status TEXT
)

/*
3 rows from agent_orders table:
order_id    total_amount    status
ORD-100 129.0    delivered
ORD-101 42.5     processing
ORD-102 310.0    delivered
*/
Top order: [('ORD-102', 310.0, 'delivered')]
Error: (sqlite3.OperationalError) attempt to write a readonly database
[SQL: UPDATE orders SET status = 'refunded' WHERE order_id = 'ORD-100';]
(Background on this error at: https://sqlalche.me/e/20/e3q8)
```

First, the agent does not see the base table at all; it only sees the view that you chose to expose. Second, the view omits `customer_name`, so the model cannot accidentally retrieve data it was never meant to access. Third, the connection is opened in read-only mode, so even a badly generated UPDATE, DELETE, or INSERT cannot succeed. The point is not merely to prevent damage. It is to keep the tool aligned with the business question. A support assistant that needs to read order status should be given a read surface, not a general-purpose SQL interface. The same principle carries over to production databases, even though the mechanism is different. In PostgreSQL, MySQL, or similar systems, the usual pattern is to create a dedicated database role for the assistant, grant it only the minimum rights it needs, and expose only approved tables or views. Read-oriented assistants normally get SELECT on a restricted surface. Write operations, when they are truly required, should be separated and handled with much tighter controls than a generic free-form SQL tool. Once you think about database tools this way, the category becomes much easier to reason about: the model may help decide which structured question to ask, but the system still decides what part of structured truth it is allowed to touch.

1.3.2.1 How to “teach” the model the schema

You usually do not “teach” the model the schema in a permanent sense. You ground it with schema context at runtime. In practice there are three common patterns.

- For a small database, you can inject the schema directly into the prompt. `SQLDatabase` exposes `get_context()` and `get_table_info()`, and it also supports `sample_rows_in_table_info`, `indexes_in_table_info`, and `custom_table_info` so the context can include not just column names but also sample rows, index details, and your own business descriptions.
- For a normal SQL agent, the preferred pattern is on-demand inspection. The documented SQL-agent flow is to list the tables, fetch the schema for the relevant tables, generate a query, check it, then execute it. In other words, the model does not need the whole schema stuffed into the initial prompt if it has tools like `sql_db_list_tables` and `sql_db_schema` available.
- For a large enterprise schema, dumping everything up front is a bad idea because it wastes context and increases errors. Use progressive disclosure: expose lightweight descriptions first, then load only the relevant schema when needed.


```

db = SQLiteDatabase.from_uri(
    readonly_uri,
    include_tables = ["agent_orders"],
    view_support = True,
    sample_rows_in_table_info = 2,
    custom_table_info = {
        "agent_orders": """
Business meaning:
- One row per order visible to the support assistant.
- order_id is the business identifier.
- total_amount is the order total in EUR.
- status is one of: processing, delivered.
- The assistant must never assume customer identity is available here.
"""
    }
)

```

1.3.2.2 SQLiteDatabaseToolkit

In the current SQLiteDatabaseToolkit, the practical SQL-agent surface is built around four tools: `sql_db_query`, `sql_db_schema`, `sql_db_list_tables`, and `sql_db_query_checker`. These map to query execution, schema inspection, table listing, and query checking before execution. Avoid making the chapter depend on the exact return order of `get_tools()`; the stable concept is the tool roles, not their list position. Here is the practical list:

- `sql_db_query`: Execute a SQL query against the database and return the result, or an error if the query is invalid. Its input schema is a single field named `query: str`.
- `sql_db_schema`: Return the schema and sample rows for one or more tables. Its input schema is a single field named `table_names: str`, passed as a comma-separated list such as "orders, customers".
- `sql_db_list_tables`: List the usable tables visible through the SQLiteDatabase wrapper. Its input is an empty string. The implementation joins `db.get_usable_table_names()`, so this reflects whatever you exposed through `include_tables`, `ignore_tables`, and similar wrapper settings.
- `sql_db_query_checker`: Use an LLM to check a SQL query before execution. Its input schema is again a single field named `query: str`. The current SQL agent tutorial explicitly recommends using this tool before `sql_db_query`.

1.3.3 HTTP and API tools

HTTP and API tools let an agent call external services through request-style interfaces. LangChain's integrations catalog does not treat "HTTP" as a stable top-level design family; generic request access appears through request-oriented tools and toolkits, while many production integrations are exposed as purpose-built toolkits for specific services. The practical design rule is therefore simple: prefer a dedicated integration when one exists for the target system and use a generic request toolkit only when you intentionally need a flexible HTTP bridge. Permissions, reachable hosts, HTTP verbs, headers, authentication, and response handling should be narrowed as much as possible, because a generic HTTP tool can otherwise become a broad action surface.

```

from langchain_community.agent_toolkits.openapi.toolkit import RequestsToolkit
from langchain_community.utilities.requests import TextRequestsWrapper

toolkit = RequestsToolkit(
    requests_wrapper = TextRequestsWrapper(headers = {}),
    allow_dangerous_requests = True
)

```



```

)

tools = {tool.name: tool for tool in toolkit.get_tools()}

response = tools["requests_get"].invoke(
    {"url": "https://jsonplaceholder.typicode.com/posts?_limit=2"}
)

print(response)

```

vol3_ch_01\scr\http_tool.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.http_tool

[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
  }
]

```

The two important constructor arguments are:

- `requests_wrapper`: This is the object that actually performs the HTTP calls. The `TextRequestsWrapper` is a lightweight wrapper around the `requests` library with `async` support (`JsonRequestsWrapper` and `GenericRequestsWrapper` are also available). Available attributes are: `headers`, `aiosession`, `auth`, `response_content_type='text'`, and `verify=True`. This means you can control request headers, `async` session handling, authentication, response formatting, and TLS certificate verification through the wrapper rather than through the toolkit itself.
- `allow_dangerous_requests`: This is the safety gate. The toolkit emphasizes that these tools can make real outbound HTTP requests, so LangChain requires an explicit opt-in before enabling them. Without that opt-in, the toolkit is intentionally harder to use because it would otherwise expose a broad network-capable action surface to the model.

The most important thing to remember is that the toolkit decides which HTTP-verb tools exist, while the wrapper decides how those requests are made.

The tools exposed by `toolkit.get_tools()` are the five HTTP verb tools: `requests_get`, `requests_post`, `requests_patch`, `requests_put`, and `requests_delete`. The toolkit reference states that it contains tools for GET, POST, PATCH, PUT, and DELETE requests:

- `requests_get`: This is the simplest one. It is used when you need specific content from a website, and the input should be a URL string. So this is valid:
- `requests_post`: Input should be a JSON string with two keys: "url" and "data". The "url" value is the target endpoint, and "data" is a dictionary of key-value pairs to send as the JSON body.
- `requests_delete`: This one takes a specific URL and returns the text response of the DELETE request.

- `requests_patch` and `requests_put`: Used for making PATCH and PUT requests. The exact input schema is exposed at runtime through each tool's `args_schema`, so the safest way to present them in code is to inspect the schema before use, then invoke them with the required structure.

1.3.4 Code tools

Code tools are useful when the assistant needs to execute procedural logic rather than retrieve a fact or call a business API. They are a good fit for data analysis, file transformation, report generation, ad hoc validation, and multi-step calculations that are easier to express as code than as a single formula. They are different from simple math tools. A math tool should normally expose a narrow deterministic operation, such as calculating a discount or dividing two numbers. A code tool gives the assistant a more general execution surface. That makes it more powerful, but also more dangerous.

The security rule is strict: never treat arbitrary model-generated code as safe. A code tool can read files, write files, consume CPU, make network calls, or damage the host environment if it is not isolated. In production, code execution should run in a sandbox with resource limits, network controls, filesystem restrictions, and audit logging. The example below is a local teaching example: it adds basic validation and a timeout, but it is not a production sandbox. A realistic use case is an e-commerce assistant that needs to analyze a small set of order records. The model can decide that a short Python analysis is useful, but the runtime still controls how the code is executed.

```
from __future__ import annotations
import ast
import subprocess
import sys
import tempfile
import textwrap
from pathlib import Path
from langchain.tools import tool
from pydantic import BaseModel, Field

ALLOWED_IMPORTS = {
    "csv",
    "decimal",
    "json",
    "math",
    "statistics"
}

BLOCKED_CALLS = {
    "__import__",
    "compile",
    "eval",
    "exec",
    "globals",
    "input",
    "locals",
    "open",
    "vars"
}

class PythonAnalysisInput(BaseModel):
    code: str = Field(
        ...,
        description=(
            "Small Python script to execute. "
            "The script must print the final result."
        )
    )
    timeout_seconds: int = Field(
        default = 3,
        ge = 1,
```

```

        le = 10,
        description = "Maximum execution time in seconds."
    )

def validate_code(code: str) -> str | None:
    try:
        tree = ast.parse(code)
    except SyntaxError as exc:
        return f"Rejected: invalid Python syntax ({exc.msg})."

    for node in ast.walk(tree):
        if isinstance(node, ast.Import):
            for alias in node.names:
                root_module = alias.name.split(".")[0]
                if root_module not in ALLOWED_IMPORTS:
                    return f"Rejected: import '{root_module}' is not allowed."

        if isinstance(node, ast.ImportFrom):
            root_module = (node.module or "").split(".")[0]
            if root_module not in ALLOWED_IMPORTS:
                return f"Rejected: import '{root_module}' is not allowed."

        if isinstance(node, ast.Call):
            if isinstance(node.func, ast.Name) and node.func.id in BLOCKED_CALLS:
                return f"Rejected: call to '{node.func.id}' is not allowed."

        if isinstance(node, ast.Attribute):
            if node.attr.startswith("_"):
                return "Rejected: access to private Python attributes is not allowed."

    return None

@tool(args_schema = PythonAnalysisInput)
def run_python_analysis(code: str, timeout_seconds: int = 3) -> str:
    """Run a small Python analysis script and return what it prints."""

    validation_error = validate_code(code)
    if validation_error:
        return validation_error

    with tempfile.TemporaryDirectory() as temp_dir:
        script_path = Path(temp_dir) / "analysis.py"
        script_path.write_text(code, encoding = "utf-8")

        try:
            completed = subprocess.run(
                [sys.executable, str(script_path)],
                cwd = temp_dir,
                capture_output = True,
                text = True,
                timeout = timeout_seconds,
                check = False
            )
        except subprocess.TimeoutExpired:
            return "ERROR: code execution timed out."

        if completed.returncode != 0:
            return f"ERROR:\n{completed.stderr.strip()}"

        output = completed.stdout.strip()
        return output if output else "(no output printed)"

if __name__ == "__main__":
    analysis_code = """
from statistics import mean
import json

orders = [
    {"order_id": "ORD-100", "amount": 129.00, "status": "delivered"},
    {"order_id": "ORD-101", "amount": 42.50, "status": "processing"},
    {"order_id": "ORD-102", "amount": 310.00, "status": "delivered"},
]

delivered_amounts = [
    order["amount"]
    for order in orders
    if order["status"] == "delivered"
]

report = {
    "delivered_orders": len(delivered_amounts),
    "average_delivered_value": round(mean(delivered_amounts), 2),
    "total_delivered_value": round(sum(delivered_amounts), 2),
}
    """
    result = run_python_analysis(analysis_code)
    print(result)

```

```
print(json.dumps(report, indent=2))
"""
    result = run_python_analysis.invoke(
        {
            "code": textwrap.dedent(analysis_code),
            "timeout_seconds": 3
        }
    )
    print(result)
```

vol3_ch_01\src\code_tool.py

```
(.env) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.code_tool
{
  "delivered_orders": 2,
  "average_delivered_value": 219.5,
  "total_delivered_value": 439.0
}
```

This example shows the difference between code execution and ordinary arithmetic. The tool is not only adding or dividing numbers. It runs a small analysis program: it filters records, computes aggregate values, builds a report, and returns structured output. The important point is not the size of the example. The important point is the boundary. The model may request code execution, but the application controls whether the code is accepted, where it runs, how long it can run, and what result is returned. In production, this boundary should be enforced by a real sandbox or provider-managed code interpreter, not by prompt instructions alone.

1.3.5 Math and calculator tools

Math is slightly different from the other tool families. Search, database, code and HTTP tools are usually about reaching outward to retrieve information or call external systems. Math tools are about refusing to guess when the task requires an exact result. In practice, arithmetic is not a separate kind of intelligence problem. It is a reliability problem. When the assistant needs to compute a prorated refund, a gross margin percentage, a discount amount, or the percentage change between two prices, the goal is not a plausible answer. The goal is the correct answer. This is why the modern pattern is to treat arithmetic as an ordinary deterministic tool inside the same tool-calling loop as everything else. Earlier approaches included a dedicated `LLMMathChain`, but that class is now deprecated. The direction is clear, instead of routing mathematical work through a special-purpose chain, you define small calculation tools and let the model call them in the normal way. That keeps arithmetic in the same execution model as the rest of the system and removes the need for a separate mental model just for simple calculations.

That design is important for a deeper reason. A language model can often produce arithmetic that looks convincing while still being wrong. The error may be small, but in real systems small numerical errors are rarely harmless. A support assistant may refund the wrong amount. A trading assistant may report the wrong percentage move or exposure. In all of these cases, the fix is to move the calculation out of free-form text generation and into a tool that returns one deterministic result. Use the model to decide when a calculation is needed and which inputs should be sent to the calculator. Use the tool to do the calculation itself. The model remains responsible for interpreting the user's request and explaining the result in natural language,

but it should not be trusted to improvise the number when exactness matters. A minimal calculator surface is therefore just a few ordinary tools:

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two integers."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """Divide a by b."""
    return a / b

print(add.invoke({"a": 3, "b": 4}))
print(multiply.invoke({"a": 6, "b": 7}))
print(divide.invoke({"a": 22, "b": 7}))
```

vol3_ch_01\scr\math_tool.py

1.3.6 Choosing the right family in real systems

The practical decision rule across all five families is simple: use the tool that best matches where the reliable answer should come from.

Use search tools when the assistant needs fresh public information from the web. Use database tools when the answer lives in structured business data. Use HTTP or a dedicated API toolkit when the system must call an external service. Use code tools when the task requires procedural logic, data transformation, file processing, or multi-step analysis. Use calculator-style tools when the task is exact arithmetic and the operation is narrow enough to express as a simple deterministic function.

The support assistant scenario makes the distinction clear. “What is your returns policy?” may be answered from policy text. “Was order ORD-100 delivered?” is a database question. “Can you create the return label?” is often an HTTP or API question. “Can you summarize these returned-order records by reason code and average refund value?” is a code-execution question. “What is the prorated reimbursement after applying a 15% restocking fee?” is a calculator question.

The goal is not to give the model the most powerful tool available. The goal is to give it the narrowest tool that can answer the question correctly. Search retrieves external evidence. Database tools query structured records. HTTP tools call services. Code tools execute controlled procedures. Calculator tools compute exact numeric results. Choosing the right family keeps the assistant grounded in the correct evidence boundary and reduces the chance that it fills gaps with plausible but unsupported text.

1.4 ROBUST CUSTOM TOOLS: VALIDATION, RETRIES, TIMEOUTS, CIRCUIT BREAKERS

1.4.1 Robustness starts at the tool boundary

A robust custom tool should be understood first as a boundary. The model-facing side is the contract introduced earlier; the execution side is ordinary application code running under runtime control. Validation, retries, timeouts, and circuit breakers are not optional extras added around a tool after the fact. They are the mechanisms that protect this boundary and keep tool execution predictable when model output, networks, or downstream services behave imperfectly.

In LangChain and LangGraph, the relevant execution pieces are `ToolRuntime` and `ToolNode`. `ToolRuntime` gives a tool access to runtime-managed resources such as context, state, stores, streaming support, and services. `ToolNode` provides the standard execution path for running tools inside a graph. The design point is that robustness is not mainly about clever prompting. It is about how execution is constrained and supervised: narrow contracts, runtime-owned trusted values, targeted retries, explicit timeouts, and dependency protection when a downstream system is unhealthy.

Seen this way, robustness begins before the tool runs and continues throughout execution. The question is never just “can the model call this function?” The real question is “what controls ensure that this function behaves safely and predictably when the model, the network, or a downstream service behaves imperfectly?” That is the level at which strong tool design operates.

1.4.2 Validation is schema plus runtime policy

Validation begins with the schema, because the schema narrows the model-visible input space. LangChain tools can use Pydantic models or JSON Schema to define required fields, descriptions, enums, patterns, and numeric bounds. But schema validation is only the first layer. Robust tools validate twice: first the shape of the payload, then the business meaning of the request. A refund tool does not only need a syntactically valid order identifier. It must also check whether the order belongs to the authenticated user, whether the item is still within the refund window, whether the requested amount is allowed, and whether the operation is permitted for the current tenant and session. Those checks depend on trusted runtime context, not on additional model-generated arguments. So schema validation answers “is this payload well formed?” Runtime validation answers “is this action allowed, meaningful, and safe right now?” Robust tools need both answers before they do anything real.

1.4.3 Config, runtime, and ToolRuntime

The contract boundary now becomes concrete in the execution API. A model should choose only the business inputs it is genuinely allowed to decide. Everything else should come from the execution environment. This is where the distinction between config, runtime, and `ToolRuntime` becomes important.

In LangChain, config and runtime are not ordinary parameter names. They are reserved for the framework’s execution model. `ToolRuntime` is the mechanism that gives a tool access to

trusted runtime-managed data without exposing that data in the public tool schema. That matters because many of the values a tool needs are not values the model should invent. A tool may need the authenticated user, the current tenant, access to a store, a service client, a stream writer, or the current tool call identifier. Those are execution concerns, not model choices. This is the same boundary applied to custom tools. If a support tool needs an `order_id`, the model can provide that. The authenticated user, database connection, permission flags, tenant context, and service clients must come from trusted application state.

`ToolRuntime` is how this trusted information enters the tool. It is injected automatically by the runtime and kept outside the schema presented to the model. From inside the tool, it provides access to the short-term state for the current run, immutable context supplied for that invocation, long-term store access when configured, streaming support for progress updates, execution configuration, and the current tool call ID. The tool can therefore use the information it needs without asking the model to supply it.

This distinction becomes even clearer when looking at graph execution. At the graph level, a node can work with three separate inputs: state, config, and runtime. state is the mutable working data that flows through the graph as execution progresses. config carries execution-related metadata, such as thread identifiers, tracing information, or callback settings. runtime carries trusted context and supporting services needed while the graph is running. Inside a tool, the same idea is condensed into `ToolRuntime`, which gives the tool a single injected object through which these runtime-managed capabilities can be reached.

The practical rule is simple. Use the schema for model-choosable business arguments, runtime context for trusted application values, and config for execution metadata. When that separation is respected, tools are easier to reason about and easier to secure. A small graph-node example makes the distinction concrete:

```
from dataclasses import dataclass
from typing_extensions import TypedDict
from langchain_core.runnables import RunnableConfig
from langgraph.runtime import Runtime

class State(TypedDict):
    order_id: str
    result: dict

@dataclass
class Context:
    user_id: str

def load_order(state: State, config: RunnableConfig, runtime: Runtime[Context]) -> dict:
    thread_id = config["configurable"]["thread_id"]
    user_id = runtime.context.user_id

    # Use thread_id for execution/session concerns.
    # Use user_id for trusted business context.
    return {
        "result": {
            "thread_id": thread_id,
            "user_id": user_id,
            "order_id": state["order_id"],
        }
    }
```

1.4.4 Retries belong to transient failures

Retries are useful, but only when they are aimed at the right class of failure. LangChain now provides `ToolRetryMiddleware` as a first-class prebuilt middleware for tool calls. It

supports targeted tool lists, exception filtering, exponential backoff, bounded delay growth, jitter, and configurable behavior when retries are exhausted. At the graph level, LangGraph also lets you attach a `RetryPolicy` to a node, which is often the right place to put retry logic when the failure belongs to a wider workflow step rather than to one tool in isolation. This gives you two different retry scopes. Tool-level retries are appropriate when a single tool wraps a flaky external dependency such as an HTTP API, search backend, or carrier gateway. Node-level retries are appropriate when the whole graph step is the unit that may temporarily fail, for example a database read or documentation lookup performed inside a dedicated node. The distinction matters because the smallest retry scope that solves the problem is usually the safest one.

The real danger is not retrying too little. It is retrying the wrong thing. Retries are appropriate for transient failures, but they are safe for mutating operations only when those operations are idempotent. Durable workflows add the same warning from another angle: resumed execution may repeat work after failure, so side effects should be isolated and designed to tolerate replay, often through idempotency keys or verification of existing results. That is why query tools and command tools should be treated differently. A read-oriented tool that checks order status or fetches a public quote is often safe to retry. A mutating tool that creates a refund, sends a notification, or places a trade must be idempotent before it is considered safe under retry. Otherwise the retry mechanism stops being a resilience feature and becomes a source of duplicate business operations.

1.4.5 Timeouts are not optional

Retries are only half of the story. A tool that can wait forever is not robust, because the retry policy never gets a chance to act if the original call never returns. Current LangChain prebuilt middleware includes retry and tool-call limits, but timeout enforcement is still something you usually apply in the underlying client or in your own wrapper layer around the tool call. That is not a weakness in the framework, it is the correct place to solve the problem, because different clients expose different timeout models. For asynchronous Python work, the standard library gives you direct timeout control through `asyncio.timeout()` and `asyncio.wait_for()`. When the deadline expires, a `TimeoutError` is raised, which means your code can treat timeouts as ordinary exceptions for retry or reporting purposes. For synchronous HTTP clients, the situation is even more blunt. `Requests` does not time out by default unless you explicitly set a timeout. Without one, your code can hang for minutes or longer. `Requests` also distinguishes connection timeouts from read timeouts, and its own exception definitions note that `ConnectTimeout` is safe to retry.

A strong rule follows from this. Every network-facing custom tool should enforce a timeout at the client boundary. If the tool is asynchronous, it should also bound the overall operation with an execution timeout. Only after that boundary exists does retry policy start to make sense. Without timeouts, “retrying” is mostly a theoretical concept that never gets to leave the whiteboard.

1.4.6 Circuit breakers are a different control

Retries and circuit breakers are related, but they solve different problems. Retry assumes an operation may succeed if attempted again, while the circuit breaker prevents repeated

attempts against an operation that is likely to fail. The two patterns can be combined but retry logic should stop when the circuit breaker indicates the problem is not transient. This matters because a failing downstream dependency can be harmed by enthusiastic retries just as easily as it can be helped by cautious ones. A circuit breaker watches for repeated failures, opens when a threshold is crossed, rejects new calls for a cooldown period, and then allows a limited probe again. That behavior is especially useful when a tool fronts an unstable remote dependency such as a shipping API, market data gateway, or partner service.

In the current LangChain prebuilt middleware set, the documented resilience controls include tool retry and tool call limits, but not a generic prebuilt circuit-breaker middleware. That means true circuit-breaker behavior is usually implemented either in the service wrapper used by the tool or in custom middleware using wrap-style hooks such as `wrap_tool_call`. LangChain's custom middleware model is explicitly designed for that kind of control-flow logic: a wrap-style hook can call the handler zero times, once, or multiple times, which is exactly the kind of control a breaker or retry wrapper needs. `ToolCallLimitMiddleware` is still worth knowing here, but for a different reason. It is not a circuit breaker in the resilience-pattern sense. It is an operational guardrail. It can limit calls globally or per tool, per run or per conversation thread, and thread-level limits can persist across invocations when a checkpoint is present. That makes it excellent for controlling runaway agent loops, expensive APIs, or over-eager web search, even though it does not model open, half-open, and closed breaker states.

1.4.7 A complete example

The following example uses the support and e-commerce scenario to put the pieces together. It shows four ideas at once. First, model-visible inputs are explicit Pydantic schemas. Second, trusted values such as `user_id` and service clients come from `ToolRuntime` rather than from the schema. Third, the mutating refund tool is idempotent through a stable business key stored in the runtime store. Fourth, the carrier-status tool uses an explicit timeout and a small circuit-breaker wrapper, while LangChain's `ToolRetryMiddleware` is used only for transient failures on that read-oriented tool. The overall pattern follows the current LangChain and LangGraph runtime model closely.

```
from __future__ import annotations
import asyncio
from dataclasses import dataclass, field
from enum import Enum
from time import monotonic
from typing import Any
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.runnables import RunnableConfig
from langgraph.store.memory import InMemoryStore
from langchain.agents import create_agent
from langchain.agents.middleware import ToolRetryMiddleware
from .config import OPENAI_API_KEY

ORDERS = {
    "ORD-100": {
        "user_id": "user-123",
        "status": "delivered",
        "tracking_number": "TRK-100",
        "refundable_amount": 59.00
    },
    "ORD-101": {
        "user_id": "user-123",
```

```

        "status": "processing",
        "tracking_number": "TRK-101",
        "refundable_amount": 0.00
    }
}

class OrderLookupInput(BaseModel):
    order_id: str = Field(
        ...,
        description="Order identifier such as ORD-100",
        pattern=r"^[ORD-]{3,}$"
    )

class RefundReason(str, Enum):
    DAMAGED = "damaged"
    WRONG_ITEM = "wrong_item"
    LATE_DELIVERY = "late_delivery"

class RefundInput(BaseModel):
    order_id: str = Field(
        ...,
        description="Order identifier such as ORD-100",
        pattern=r"^[ORD-]{3,}$"
    )
    reason: RefundReason = Field(
        ...,
        description="Reason for the refund request"
    )
    amount: float = Field(
        ...,
        gt=0,
        description="Refund amount in euros"
    )

class CarrierLookupInput(BaseModel):
    tracking_number: str = Field(
        ...,
        description="Tracking identifier such as TRK-100",
        pattern=r"^[TRK-[A-Z0-9-]]+$"
    )

class CircuitOpenError(RuntimeError):
    pass

class FakeCarrierClient:
    async def lookup(self, tracking_number: str) -> dict:
        if tracking_number == "TRK-TIMEOUT":
            await asyncio.sleep(5)
            return {"carrier_status": "unknown"}

        if tracking_number == "TRK-FAIL":
            raise ConnectionError("carrier gateway unavailable")

        await asyncio.sleep(0.2)
        return {
            "carrier_status": "in_transit",
            "carrier": "AcmeExpress"
        }

class AsyncCircuitBreaker:
    def __init__(self, failure_threshold: int = 3, reset_after_seconds: float = 30.0):
        self.failure_threshold = failure_threshold
        self.reset_after_seconds = reset_after_seconds
        self.failure_count = 0
        self.opened_at: float | None = None

    def is_open(self) -> bool:
        if self.opened_at is None:
            return False
        if monotonic() - self.opened_at >= self.reset_after_seconds:
            self.failure_count = 0
            self.opened_at = None
            return False
        return True

    async def call(self, func, *args, **kwargs):
        if self.is_open():
            raise CircuitOpenError("carrier service temporarily unavailable")
        try:
            result = await func(*args, **kwargs)
        except (ConnectionError, TimeoutError):
            self.failure_count += 1

```

```

        if self.failure_count >= self.failure_threshold:
            self.opened_at = monotonic()
            raise
        else:
            self.failure_count = 0
            self.opened_at = None
            return result

@dataclass
class SupportContext:
    user_id: str
    carrier_client: FakeCarrierClient
    carrier_breaker: AsyncCircuitBreaker
    # Teaching-only switch: records refund actions for which the example has
    # already simulated a lost response after the side effect was committed.
    refund_timeout_after_commit_once: set[str] = field(default_factory = set)

    async def query_carrier_with_timeout(
        client: FakeCarrierClient,
        tracking_number: str,
        timeout_seconds: float
    ) -> dict:
        async with asyncio.timeout(timeout_seconds):
            return await client.lookup(tracking_number)

    @tool(args_schema=OrderLookupInput)
    def lookup_order(order_id: str, config: RunnableConfig) -> dict:
        """Look up one order for the authenticated user."""
        ctx: SupportContext = config["configurable"]["support_context"]
        order = ORDERS.get(order_id)

        if order is None:
            return {"ok": False, "error": "order_not_found"}

        if order["user_id"] != ctx.user_id:
            return {"ok": False, "error": "forbidden"}

        return {
            "ok": True,
            "order_id": order_id,
            "status": order["status"],
            "tracking_number": order["tracking_number"],
            "refundable_amount": order["refundable_amount"]
        }

    @tool(args_schema=RefundInput)
    def request_refund(
        order_id: str,
        reason: RefundReason,
        amount: float,
        config: RunnableConfig
    ) -> dict:
        """Create a refund request, or return the existing result for the same business
        action."""
        ctx: SupportContext = config["configurable"]["support_context"]
        store: InMemoryStore = config["configurable"]["store"]

        order = ORDERS.get(order_id)

        if order is None:
            return {"ok": False, "error": "order_not_found"}

        if order["user_id"] != ctx.user_id:
            return {"ok": False, "error": "forbidden"}

        if amount > order["refundable_amount"]:
            return {"ok": False, "error": "amount_exceeds_refundable_amount"}

        if store is None:
            raise RuntimeError("A store is required for idempotent refund handling")

        # Stable business key for retry safety.
        idempotency_key = f"refund:{order_id}:{reason.value}:{amount:.2f}"
        namespace = ("refunds", ctx.user_id)

        existing = store.get(namespace, idempotency_key)
        if existing is not None:
            return {"ok": True, "replayed": True, **existing.value}

        refund = {
            "refund_id": f"rf_{order_id.lower()}_{reason.value}",
            "order_id": order_id,
            "reason": reason.value,
            "amount": amount,

```

```

        "status": "accepted"
    }

    store.put(namespace, idempotency_key, refund)

    # Teaching-only failure injection. This simulates a common distributed-system
    # ambiguity: the refund was accepted and stored, but the caller receives a
    # timeout before it sees the response. ToolRetryMiddleware retries the tool
    # call. Because the idempotency key is stable, the second call finds the
    # existing refund above and returns it as a replay instead of creating a
    # duplicate refund.
    if idempotency_key not in ctx.refund_timeout_after_commit_once:
        ctx.refund_timeout_after_commit_once.add(idempotency_key)
        raise TimeoutError("refund gateway timed out after accepting the request")

    return {"ok": True, "replayed": False, **refund}

@tool(args_schema=CarrierLookupInput)
async def fetch_carrier_status(
    tracking_number: str,
    config: RunnableConfig
) -> dict:
    """Get carrier status with an execution timeout and a circuit breaker.

    Generic retry behavior is intentionally not implemented in this tool.
    Transient ConnectionError and TimeoutError exceptions are allowed to
    propagate so ToolRetryMiddleware can retry the failed tool call.
    """
    ctx: SupportContext = config["configurable"]["support context"]
    client = ctx.carrier_client
    breaker = ctx.carrier_breaker

    try:
        result = await breaker.call(
            query_carrier_with_timeout,
            client,
            tracking_number,
            2.0
        )
    except CircuitOpenError as exc:
        # The circuit breaker is already open, so this is not a transient
        # carrier call worth retrying immediately. Return a controlled
        # domain-level failure to the agent.
        return {"ok": False, "error": str(exc)}

    return {
        "ok": True,
        "tracking_number": tracking_number,
        **result
    }

def retry_failure_message(error: Exception) -> str:
    """Return the final tool message after ToolRetryMiddleware exhausts retries."""
    return f"tool failed after retry attempts: {error}"

async def main() -> None:
    store = InMemoryStore()

    context = SupportContext(
        user_id = "user-123",
        carrier_client = FakeCarrierClient(),
        carrier_breaker = AsyncCircuitBreaker(
            failure_threshold = 3,
            reset_after_seconds = 30.0
        )
    )

    # create_agent builds the LangGraph-backed agent runtime.
    # ToolRetryMiddleware retries two tools in this teaching example:
    # - fetch_carrier_status, which calls a network-like dependency.
    # - request_refund, which is side-effecting but protected by a stable
    #   idempotency key, so retry can safely replay the same business action.
    agent = create_agent(
        model = ChatOpenAI(model = "gpt-4o-mini", temperature = 0),
        tools = [lookup_order, request_refund, fetch_carrier_status],
        middleware = [
            ToolRetryMiddleware(
                max_retries = 2,
                initial_delay = 0.5,
                backoff_factor = 2.0,
                max_delay = 5.0,
                jitter = False,
            )
        ]
    )

```

```

        tools = ["fetch_carrier_status", "request_refund"],
        retry_on = (ConnectionError, TimeoutError),
        on_failure = retry_failure_message
    )
    1,
    store = store
)
result = await agent.ainvoke(
    {
        "messages": [
            {
                "role": "user",
                "content": (
                    "Check order ORD-100, tell me the order status and carrier "
                    "status, then tell me whether it can be refunded. If it can, "
                    "request a refund for 59 euros because it arrived damaged."
                )
            }
        ],
        "config": {
            "configurable": {
                "support_context": context,
                "store": store  # passed explicitly so request_refund can use it
            }
        }
    }
)
print(result["messages"][-1].content)
if __name__ == "__main__":
    asyncio.run(main())

```

vol3_ch_01\scr\robust_custom_tool.py

This example applies the same boundary in a complete support workflow. Each tool exposes a narrow Pydantic schema: `OrderLookupInput` accepts an `order_id`, `RefundInput` accepts an `order_id`, a constrained refund reason, and a positive amount, and `CarrierLookupInput` accepts a tracking number with a defined format. Trusted values are supplied by the execution environment. The support context contains the authenticated `user_id`, carrier client, and circuit breaker; the store is supplied by the runtime and used for idempotent refund handling. The model can request an order lookup or refund, but it cannot control identity, service clients, circuit-breaker state, or idempotency storage.

The `lookup_order` tool demonstrates this boundary in the simplest form. The model provides an order ID. The tool retrieves the trusted support context from the runtime configuration, loads the order, and compares the order owner with the authenticated user. If the order does not belong to that user, the tool returns a forbidden result. Authorization is therefore enforced by application logic, not inferred from the model's request. The `request_refund` tool is the command tool in the example. It changes business state, so it must be safe under repeated execution. The tool first validates that the order exists, belongs to the authenticated user, and has enough refundable value. It then builds a stable idempotency key from the order ID, refund reason, and amount. That key defines what counts as the same logical refund request. Before creating a new refund, the tool checks the store using that idempotency key. If a matching result already exists, the tool returns the existing refund with `replayed` set to `True`. If no result exists, it creates the refund and stores the result under the same key. This means that if the same logical refund request is executed again, the system returns the stored result instead of creating a duplicate refund.

The example deliberately simulates a difficult distributed-systems failure. After the refund has been stored, the tool raises a `TimeoutError` once. This represents the case where the side

effect was committed, but the caller did not receive the response. The agent middleware then retries the tool call. On the retry, the idempotency key is the same, so the tool finds the stored refund and returns it as a replay. This is the key lesson: retrying a command tool is acceptable only when the command is designed to be replay-safe. The `fetch_carrier_status` tool shows a different reliability pattern. It is a read-oriented tool that depends on an external carrier service. The carrier call is wrapped in `asyncio.timeout(2.0)`, so a slow downstream service becomes a bounded failure instead of blocking the agent indefinitely. The same call is also routed through an `AsyncCircuitBreaker`. The breaker tracks repeated `ConnectionError` and `TimeoutError` failures. After the configured threshold is reached, it opens and temporarily rejects new carrier calls. The retry logic for both the carrier lookup and the refund command is configured outside the tools, in `ToolRetryMiddleware`. The middleware is limited to `fetch_carrier_status` and `request_refund`, retries only `ConnectionError` and `TimeoutError`, and uses bounded exponential backoff. This keeps retry policy out of the tool bodies and makes retry behavior visible at the agent configuration level.

The two tools use retry for different reasons. The carrier lookup is retried because network-like read operations often fail transiently. The refund operation is retried because the example intentionally makes it idempotent before enabling retry. The retry policy is therefore not applied blindly. Retry handles short-lived failures; the circuit breaker handles persistent dependency failure by failing fast after repeated errors. Taken together, the example demonstrates the operational boundary in practice: schema-constrained requests, runtime-supplied trusted values, timeout and circuit-breaker protection for read-oriented external calls, and idempotency for mutating operations.

1.4.8 Practical rules

A few rules survive contact with production better than any particular API surface.

- First, treat schema validation and runtime validation as different jobs. The schema decides whether the payload is well formed. Runtime policy decides whether the action is allowed, meaningful, and safe. Both must pass before execution.
- Second, keep model-visible arguments narrow. Apply the same contract boundary consistently: schema for model-choosable inputs, `ToolRuntime` for trusted execution values.
- Third, retry only transient failures, and make mutating tools idempotent before you retry them. Exponential backoff and jitter are useful. Duplicate side effects are not.
- Fourth, enforce timeouts explicitly at the client or wrapper boundary. A tool that can block forever is not robust, no matter how elegant its schema looks.
- Fifth, do not confuse circuit breakers with retries. Retries are for short-lived faults. Circuit breakers are for persistent faults where failing fast is safer than trying again immediately. LangChain gives you strong extension points and useful operational guardrails, but the actual breaker logic is still something you usually implement yourself.

When these rules are followed, custom tools stop being fragile little appendages hanging off a prompt and become what they should have been from the beginning: small, explicit, and resilient application interfaces that a model can use without being trusted to govern.

1.5 TOOL ERROR HANDLING: RECOVERABLE VS FATAL; USER-FIXABLE VS SYSTEM-FIXABLE

Once the tool boundary is explicit and protected with validation and runtime controls, the next design question is what should happen when a tool still fails. That question matters because not every failure deserves the same response. Some failures should be retried automatically. Some should be shown back to the model so it can reformulate the call or ask for missing information. Some should pause the workflow and wait for a person. Some should end the current run immediately and surface as real defects. Treating all of these cases as one generic “tool error” produces confused systems that either retry nonsense, hide bugs, or keep bothering users for problems the runtime should have fixed by itself. A practical way to make this manageable is to classify failures along two axes. The first axis is recoverable versus fatal. A recoverable failure is one after which the workflow can still continue in a controlled way. A fatal failure is one that should stop the current path and surface clearly. The second axis is user-fixable versus system-fixable. A user-fixable failure needs missing information, approval, or a change in the user’s context. A system-fixable failure can be handled by runtime logic such as retry, fallback, validation-and-repair, or by exposing the failure back into the tool loop so the model can make another attempt.

1.5.1 The four classes of tool failure

1.5.1.1 *Recoverable and system-fixable*

Some failures can be handled by the system itself. They do not require a human operator, a support agent, or a developer to intervene while the workflow is running. The runtime can either try again safely, use a fallback, or give the model enough information to correct the next step. There are two common cases.

The first case is transient infrastructure failure. This includes network glitches, temporary upstream outages, rate limits, and timeouts. These failures may disappear if the same operation is tried again after a short delay. For this type of failure, the usual controls are bounded retry, backoff, and fallback. LangGraph supports this pattern with node-level retry through `RetryPolicy`. LangChain supports it for tool calls through `ToolRetryMiddleware`. The important constraint is that retry must be bounded. The runtime should retry only when the failure is likely to be temporary, and the operation is safe to repeat. A read-only carrier-status lookup may be safe to retry. A refund request is safe to retry only if it has been designed to be idempotent.

The second case is model-visible repair. Here the problem is not a temporary infrastructure failure. The tool call may be valid as a piece of JSON, but wrong for the business operation. For example, the model may generate a SQL query that is syntactically valid but fails when executed, request a refund amount that is higher than the permitted amount, or look up a product using the wrong catalog identifier. In this situation, repeating the same tool call without any change is not useful. The better approach is to return a structured error back into the agent loop. The model can then use that error to change the next tool call or ask the user for clarification. LangChain supports this pattern at more than one level. At the agent layer, `@wrap_tool_call` middleware can catch a tool failure and return a `ToolMessage` linked to

the original `tool_call_id`. This keeps the failed tool call visible in the conversation flow and gives the model a chance to recover. At the tool layer, `BaseTool` and `StructuredTool` provide `handle_tool_error` and `handle_validation_error`. These options let selected exceptions be converted into tool-visible messages instead of stopping the workflow immediately.

The design rule is simple: retry temporary failures when retry is safe; return structured errors when the model needs to repair its request. Both cases are recoverable, but they are recovered in different ways.

1.5.1.2 *Recoverable and user-fixable*

This class includes failures where the system is still in a healthy state but cannot continue until a user or reviewer provides something the runtime does not have. The most common examples are missing identifiers, ambiguous intent, unclear date ranges, missing account context, or approvals for sensitive actions. A support assistant may need an order ID before it can look up a shipment. An e-commerce assistant may need the user to choose among several near-matching products. A trading assistant may need confirmation before placing a real order. In all of these cases, the error should not be retried and should not be “fixed” by making up data. The workflow should pause and ask. LangGraph’s `interrupt()` mechanism exists precisely for this kind of situation. It pauses execution, persists the graph state through the checkpoint, and resumes later when external input is available. LangChain’s `HumanInTheLoopMiddleware` builds on the same idea for agent tool calls, with explicit review decisions such as approve, edit, and reject. This matters because many failures are not really technical failures at all. They are precondition failures. The tool cannot do good work until the missing business fact, approval, or correction arrives from outside the model-runtime loop.

1.5.1.3 *Fatal and system-fixable*

Some failures should stop the current run immediately, even though they can be fixed by the engineering team. These failures are not caused by the user and should not be treated as normal conversation errors. They usually mean that the application, tool contract, or orchestration layer is misconfigured. Examples include a broken tool implementation, an invalid tool schema, an incorrect message protocol, a missing execution hook for dynamically registered tools, or another defect in the agent runtime. In these cases, the system should not try to hide the problem by returning a friendly message to the model. The safer response is to stop the run, log the failure, and fix the application code or configuration.

One example is runtime-registered tools. If tools are discovered or added during execution, the agent must know two things: which tools should be exposed to the model, and how those tools should be executed if the model calls them. The model-facing part is handled when the runtime adds the tool definition to the model request. The execution part must also be handled. If the runtime exposes a tool to the model but does not provide a matching execution path, the agent may receive a valid tool call that it cannot run. That is not a user error. It is an orchestration defect.

A second example is the tool-calling message protocol. When an assistant message requests tool calls, the runtime must return one matching tool result for each `tool_call_id`. Missing

tool results, duplicate tool results, or tool messages that do not correspond to a previous tool call break the protocol. In a custom tool loop, this kind of error should be allowed to fail clearly. It should be logged and fixed in code rather than converted into a normal tool response. This category is important because automatic recovery can make real defects harder to diagnose. Retrying a broken schema will not make the schema valid. Asking the model to recover from a missing execution hook will not create the missing hook. Passing a protocol violation back into the conversation may only produce more confusing behavior. These failures need engineering correction, not model repair. Prevention is often more important than recovery. Code execution is the clearest example. If an agent can run code, shell commands, or file operations, the main control should be containment. The execution environment should be sandboxed, inspected, isolated from the host system, and restricted by resource, network, and file-system controls. The goal is to prevent unsafe behavior from reaching the host environment, not to recover after the damage has already started. The design rule is simple: if the failure means the runtime cannot safely continue, stop the current run. Log enough detail for diagnosis, surface an operational error to the application, and fix the defect in code, configuration, or infrastructure.

1.5.1.4 Fatal and user-fixable

This is the smallest category, but it is still useful. It includes cases where the system is behaving correctly, yet the current run cannot proceed unless the user changes something outside the current tool call. The classic examples are missing authentication, insufficient permissions, unsupported account state, market closed, insufficient funds, or a policy rule that forbids the requested action under the current conditions. Retrying these cases is nonsense. The right behavior is to stop the current path, explain what must change, and only resume once the external condition has changed.

A good system tries to prevent some of these failures earlier by reducing tool exposure. LangChain's dynamic tool selection guidance explicitly supports filtering tools based on authentication state, permissions, feature flags, or conversation stage. That means a delete tool, a private search tool, or a trade-placement tool does not need to be visible to the model when the user is not authenticated or not allowed to use it. Preventing a forbidden tool call is usually cleaner than letting the model discover the refusal at runtime.

1.5.2 Example: a trading agent that classifies failures at the tool boundary

A trading assistant is a good place to make tool error handling concrete, because the difference between failure classes is hard to ignore. A market-data lookup tool is read-oriented and usually safe to retry when the problem is a timeout or a short-lived network failure. An order-placement tool is different. It is a mutating operation with financial consequences, so it should not be treated like a harmless quote request. Some failures should come back to the model as correctable tool feedback. Some should pause for approval before the tool runs. Some should stop the current path and surface clearly because the system is broken or the current business conditions make the request impossible. The example below defines two tools and gives them different recovery policies. `get_realtime_quote` is a read tool. If the quote service times out, the runtime retries it automatically. `place_order` is a write tool. It always requires human approval before execution, and it is not part of the automatic retry policy. Business-level failures such as unknown symbols, market closed, or insufficient buying power are

classified explicitly in middleware so the agent can respond differently to each one. Unexpected exceptions are left alone so they fail visibly instead of being wrapped in cheerful nonsense. LangChain's current middleware model is designed for exactly this sort of interception around tool calls, and human-in-the-loop approval is a first-class pattern for sensitive financial actions.

```

from __future__ import annotations
import json
from enum import Enum
from typing import Any
from pydantic import BaseModel, Field, field_validator, model_validator
from langchain.agents import create_agent
from langchain.agents.middleware import (
    HumanInTheLoopMiddleware,
    ToolRetryMiddleware,
    wrap_tool_call)
from langchain.messages import AIMessage, ToolMessage
from langchain.tools import tool
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

# -----
# Fake broker and market state used by the tools
# -----
PRICE_FEED = {
    "AAPL": 192.35,
    "MSFT": 412.20,
    "NVDA": 118.80
}

ACCOUNT = {
    "account_id": "acct-001",
    "buying_power": 2500.00,
    "trading_enabled": True
}

MARKET_STATE = {
    "is_open": False # Set to True to allow successful placement
}

QUOTE_ATTEMPTS: dict[str, int] = {}

# -----
# Business exceptions used for classification
# -----
class CorrectableTradeInputError(Exception):
    """The request is understandable, but the user can correct it."""

class TradeBlockedError(Exception):
    """The request is valid, but current business conditions block execution."""

# -----
# Tool schemas
# -----
class Side(str, Enum):
    BUY = "buy"
    SELL = "sell"

class OrderType(str, Enum):
    MARKET = "market"
    LIMIT = "limit"

class QuoteInput(BaseModel):
    symbol: str = Field(
        ...,
        description = "Ticker symbol in uppercase, for example AAPL or MSFT."
    )

    @field_validator("symbol")
    @classmethod
    def normalize_symbol(cls, value: str) -> str:
        value = value.strip().upper()
        if not value.isalpha():
            raise ValueError("symbol must contain only letters")
        return value

class PlaceOrderInput(BaseModel):
    symbol: str = Field(

```

```

        ...,
        description = "Ticker symbol in uppercase, for example AAPL or MSFT."
    )
    side: Side = Field(
        ...,
        description = "Whether to buy or sell."
    )
    quantity: int = Field(
        ...,
        gt = 0,
        description = "Number of shares to trade. Must be greater than zero."
    )
    order_type: OrderType = Field(
        default = OrderType.MARKET,
        description = "Use market for immediate execution or limit with a limit price."
    )
    limit_price: float | None = Field(
        default = None,
        gt = 0,
        description = "Required only for limit orders."
    )

    @field_validator("symbol")
    @classmethod
    def normalize_symbol(cls, value: str) -> str:
        value = value.strip().upper()
        if not value.isalpha():
            raise ValueError("symbol must contain only letters")
        return value

    @model_validator(mode = "after")
    def validate_price_rules(self) -> "PlaceOrderInput":
        if self.order_type == OrderType.LIMIT and self.limit_price is None:
            raise ValueError("limit_price is required for limit orders")
        if self.order_type == OrderType.MARKET and self.limit_price is not None:
            raise ValueError("limit_price is only valid for limit orders")
        return self

# -----
# Tools
# -----
@tool(args_schema=QuoteInput)
def get_realtime_quote(symbol: str) -> str:
    """Return the latest market price for a symbol."""

    attempts = QUOTE_ATTEMPTS.get(symbol, 0) + 1
    QUOTE_ATTEMPTS[symbol] = attempts

    # Diagnostic line so retries are visible in the console output.
    print(f"[tool:get_realtime_quote] attempt={attempts} symbol={symbol}")

    # Simulate a transient infrastructure failure that succeeds on retry.
    if symbol == "AAPL" and attempts < 3:
        raise TimeoutError("quote service timed out")

    # This is not a transient fault. The user or model must correct the request.
    if symbol not in PRICE_FEED:
        raise CorrectableTradeInputError(f"Unknown symbol '{symbol}'")

    return json.dumps(
        {
            "symbol": symbol,
            "price": PRICE_FEED[symbol],
            "currency": "USD",
            "source": "simulated-feed"
        }
    )

@tool(args_schema=PlaceOrderInput)
def place_order(
    symbol: str,
    side: Side,
    quantity: int,
    order_type: OrderType = OrderType.MARKET,
    limit_price: float | None = None
) -> str:
    """Place a stock order through the broker."""

    print(
        "[tool:place_order] "
        f"symbol={symbol} side={side.value} quantity={quantity} "
        f"order_type={order_type.value} limit_price={limit_price}"
    )

```

```

    )

    if symbol not in PRICE_FEED:
        raise CorrectableTradeInputError(f"Unknown symbol '{symbol}'")

    if not ACCOUNT["trading_enabled"]:
        raise TradeBlockedError("Trading is not enabled for this account.")

    if not MARKET_STATE["is_open"]:
        raise TradeBlockedError("Market is currently closed.")

    reference_price = (
        PRICE_FEED[symbol] if limit_price is None else float(limit_price)
    )
    estimated_notional = reference_price * quantity

    if side == Side.BUY and estimated_notional > ACCOUNT["buying_power"]:
        raise TradeBlockedError("Insufficient buying power for this order.")

    order_id = f"ord-{symbol.lower()}-{quantity}-{side.value}"

    return json.dumps(
        {
            "status": "accepted",
            "order_id": order_id,
            "symbol": symbol,
            "side": side.value,
            "quantity": quantity,
            "order type": order_type.value,
            "limit_price": limit_price
        }
    )

# -----
# Middleware that classifies non-transient tool failures
# -----
@wrap_tool_call
def classify_tool_errors(request, handler):
    try:
        return handler(request)

    except CorrectableTradeInputError as e:
        return ToolMessage(
            content=(
                f"Correctable trading input problem: {e}. "
                "Ask for corrected order details or choose a different symbol."
            ),
            tool_call_id=request.tool_call["id"]
        )

    except TradeBlockedError as e:
        return ToolMessage(
            content=(
                f"Trading request blocked: {e}. "
                "Do not retry automatically. Explain the condition to the user."
            ),
            tool_call_id=request.tool_call["id"]
        )

    # Any other exception is treated as an unexpected system problem
    # and is allowed to bubble up.

# -----
# Agent
# -----
agent = create_agent(
    model = "openai:gpt-4.1-mini",
    tools = [get_realtime_quote, place_order],
    system_prompt = (
        "You are a trading assistant. "
        "Use get_realtime_quote for price lookup. "
        "Use place_order only when the user explicitly asks to trade. "
        "Never retry place_order automatically. "
        "If a tool returns a correctable input problem, ask for a correction. "
        "If a tool returns a blocked-trade message, explain the reason and stop."
    ),
    middleware=[
        classify_tool_errors,
        ToolRetryMiddleware(
            tools = ["get_realtime_quote"],
            retry_on = (TimeoutError, ConnectionError),
            max_retries = 3,

```

```

        initial_delay = 0.5,
        backoff_factor = 2.0
    ),
    HumanInTheLoopMiddleware(
        interrupt_on={
            "place_order": {"allowed_decisions": ["approve", "edit", "reject"]},
            "get_realtime_quote": False
        }
    )
]
checkpointer = InMemorySaver()
)

# -----
# Pretty trace helpers
# -----
.
.
.

# -----
# Demo
# -----
if __name__ == "__main__":
    quote_config = {"configurable": {"thread_id": "quote-thread"}}
    trade_config = {"configurable": {"thread_id": "trade-thread"}}

    # Reset quote attempts so the retry behavior is easy to observe every run.
    QUOTE_ATTEMPTS.clear()

    # 1) Read-oriented tool with transient failure: retried automatically
    trace_run(
        "1) Quote lookup with retry",
        {
            "messages": [
                {
                    "role": "user",
                    "content": "What is the latest price for AAPL?"
                }
            ]
        },
        quote_config
    )

    # 2) High-stakes mutating tool: paused for approval before execution
    trace_run(
        "2) Trade request before approval",
        {
            "messages": [
                {
                    "role": "user",
                    "content": "Place a market order to buy 20 shares of AAPL."
                }
            ]
        },
        trade_config
    )

    # 3) Resume after approval. With MARKET_STATE['is_open'] = False,
    # the tool executes and returns a blocked-trade message.
    trace_run(
        "3) Resume after approval",
        Command(resume={"decisions": [{"type": "approve"}]}),
        trade_config
    )

```

vol3_ch_01\src\tool_error_handling.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.tool_error_handling
=====
1) Quote lookup with retry
=====

--- step 1 ---
[model] tool call -> get_realtime_quote({"symbol": "AAPL"})

--- step 2 ---
[HumanInTheLoopMiddleware.after_model] None
[tool:get_realtime_quote] attempt=1 symbol=AAPL

```



```
[tool:get_realtime_quote] attempt=2 symbol=AAPL
[tool:get_realtime_quote] attempt=3 symbol=AAPL

--- step 3 ---
[tools:get_realtime_quote] result -> {"symbol": "AAPL", "price": 192.35, "currency":
"USD", "source": "simulated-feed"}

--- step 4 ---
[model] final answer -> The latest price for AAPL is $192.35.

--- step 5 ---
[HumanInTheLoopMiddleware.after_model] None

=====
2) Trade request before approval
=====

--- step 1 ---
[model] tool call -> place_order({"symbol": "AAPL", "side": "buy", "quantity": 20,
"order_type": "market"})

--- step 2 ---
[interrupt] approval required
  request 1: <unknown>({"symbol": "AAPL", "side": "buy", "quantity": 20, "order_type":
"market"})
    description: Tool execution requires approval

Tool: place_order
Args: {'symbol': 'AAPL', 'side': 'buy', 'quantity': 20, 'order_type': 'market'}
  review 1: allowed_decisions=['approve', 'edit', 'reject']

=====
3) Resume after approval
=====

--- step 1 ---
[model] tool call -> place_order({"symbol": "AAPL", "side": "buy", "quantity": 20,
"order_type": "market"})
[tool:place_order] symbol=AAPL side=buy quantity=20 order_type=market limit_price=None

--- step 2 ---
[tools] result -> Trading request blocked: Market is currently closed.. Do not retry
automatically. Explain the condition to the user.

--- step 3 ---
[model] final answer -> The market is currently closed, so I am unable to place the order
to buy 20 shares of AAPL at this time. You can try placing the order again once the
market opens. Is there anything else I can assist you with?

--- step 4 ---
[HumanInTheLoopMiddleware.after_model] None
```

The example works because each tool failure is handled according to what kind of failure it is, rather than according to one giant `except Exception` reflex. The quote tool demonstrates recoverable, system-fixable failure. A temporary timeout is the sort of thing the runtime can often resolve by itself, so the retry policy sits close to the tool call. It is attached only to `get_realtime_quote`, not to every tool in the agent, because retry is appropriate only where repetition is safe and useful. LangChain's built-in tool retry middleware is explicitly meant for transient external failures and supports retry scoping by tool name plus exception filtering.

The same quote tool also shows a second class of failure. If the symbol does not exist, that is not an infrastructure problem. Repeating the same bad request will not cause the market data service to discover a new company out of pity. The tool raises `CorrectableTradeInputError`, and the `wrap_tool_call` middleware turns that into a `ToolMessage` that the model can understand. This is the key move for recoverable, user-fixable failures. The workflow continues, but it continues by asking for corrected input rather

than by retrying. The order-placement tool shows why write tools need stricter handling. Before the tool even runs, the agent pauses for approval because placing a trade is a high-stakes action. Human-in-the-loop middleware is designed for this exact case: it inspects the model's proposed tool call, interrupts execution when a configured tool is selected, and resumes only after an explicit decision such as approve, edit, or reject. That is not mere ceremony. It is the runtime acknowledging that some actions should cross a human checkpoint before they become real side effects.

After approval, the order tool still may fail, and those failures are not all alike. `TradeBlockedError` represents conditions such as market closed or insufficient buying power. These are not transient technical glitches. They are real business constraints. The runtime returns them to the model as structured tool feedback, but the content explicitly says not to retry automatically. That makes them fatal for the current execution path even though the surrounding agent can still explain the failure cleanly. In other words, the conversation continues, but the attempted trade does not. This is the right shape for failures that require a state change outside the current tool call before progress is possible. LangGraph's error taxonomy treats these as a separate class from transient and model-recoverable failures, and LangChain's human-in-the-loop guidance frames financial transactions as a canonical approval case rather than something to repeat automatically.

The last class is the one the code does not try to prettify. Anything that is not a known business exception is allowed to bubble up. That includes tool bugs, broken assumptions, bad runtime wiring, and protocol mistakes. This is the category that maps to fatal, system-fixable failure. It is "system-fixable" because the engineering team can fix it in code or configuration, but it is "fatal" for the current run because the agent no longer has a trustworthy recovery path. LangChain's current guidance is quite direct here: custom tool-error middleware is for failures you intentionally want to expose back to the model, while the tool-calling protocol itself still has hard requirements such as providing exactly one matching `ToolMessage` for each `tool_call_id`. Contract violations are not a clever place for optimism. They are defects.

1.5.3 Practical rules

First, decide who fixes the failure before deciding how to code the handler. Retry is a system answer. Interrupt is a human answer. Bubbling the exception is a developer answer. Ending the current run with a clear refusal is a business-policy answer. When those answers are mixed up, the workflow becomes noisy and unreliable.

Second, separate transient failures from logical failures. A timeout, rate limit, or short-lived upstream outage may deserve retry. Market closed, insufficient funds, forbidden access, and unsupported actions do not. They are not waiting to become true if you click harder.

Third, let the model see only the failures it can meaningfully act on. Recoverable business errors can be turned into tool-visible outputs so the loop can repair itself. Bugs, schema defects, and protocol violations should not be repackaged as if they were ordinary tool results.

Fourth, keep sensitive tools out of reach when the user or context is not ready for them. Filtering tools by authentication state, permissions, feature flags, or conversation stage reduces the number of fatal user-fixable failures you ever need to explain. The cleanest refusal is often the tool call that never became available in the first place.

Fifth, keep all recovery paths bounded. Retry counts should be finite. Repair loops should have attempt limits. Human review should resume from checkpoints, not from a fresh run that forgets where it was. Tool call limits should stop runaway loops before they become operational incidents. Reliability is not only about recovering from failure. It is also about knowing when to stop trying.

A tool-using agent becomes trustworthy when error handling stops being an afterthought and becomes part of the design of the tool boundary itself. The important question is never merely whether a tool failed. The important question is who should fix the failure, and whether the current run should continue at all. Once that decision is explicit, the rest of the implementation becomes much less mysterious and much more dependable.

1.6 REACT-STYLE WORKFLOWS AND STANDARD TOOL-CALLING PATTERNS

Once tools are defined as explicit contracts, the next question is how they are actually used at runtime. The answer is simple in shape, even if the surrounding framework can make it look grander than it is. The model receives the conversation and the available tools, decides whether it can answer directly or needs external help, emits one or more structured tool calls, and then waits while the runtime executes those calls and returns the results. If the new information is enough, the model answers. If not, it can ask for more tools and continue the loop. That basic pattern appears across modern provider APIs and frameworks. The names vary, the JSON varies, the shiny diagrams vary, but the beast is the same. This is also why tool calling matters so much. A raw model can generate plausible language, but it cannot by itself fetch current data, query a private system of record, run exact calculations reliably, or trigger real operations in external systems. Tool calling extends the model with those capabilities at runtime. In practice, “tool calling” and “function calling” are often treated as the same idea. A function is simply one common, schema-driven form of a tool. It is useful to separate this from retrieval over an indexed corpus. In tool calling, the model invokes a capability such as search, database lookup, code execution, or an API call. In retrieval-based systems, the runtime fetches passages from an indexed knowledge base and places them back into the model’s context. Both approaches ground answers, but they are not the same mechanism. One is action-oriented. The other is retrieval-oriented.

1.6.1 ReAct is the workflow on top of tool calling

ReAct, short for Reason and Act, gave this pattern its clearest representation. In the original formulation, the model alternates between reasoning, taking an action, observing the result, and reasoning again. The value of the pattern is not the presence of dramatic labels such as Thought, Action, Observation, and Final Answer. The value is the feedback loop. The model does not have to decide everything up front. It can react to what actually comes back from the world. This was an important step in the evolution from “model that thinks about a task” to “system that can pursue a task.” A support assistant that needs to decide whether a refund is possible may first need to look up an order, then inspect the delivery date, then evaluate the policy, and only then answer. A trading assistant may need to fetch a quote before it can explain whether a planned order is reasonable. An e-commerce assistant may need to search the catalog, check stock, and then answer with only the available options. In all of these cases, the next step depends on what the previous step revealed. That is classic ReAct territory. The logic of ReAct is still central, but the mechanics are now usually expressed through structured tool-call objects and structured tool results. In other words, ReAct survived, but the wire format grew up. You should design around tool calls and tool results, not around parsing decorative traces that look clever until they break in production. It is also worth distinguishing ReAct from planning-first designs. In a ReAct loop, decision-making happens step by step after each observation. In planning-first systems, more of the route is decided up front and tools are then used to execute the plan. Both patterns can be useful, but they solve different control problems. ReAct is the right mental model when each tool result may change what should happen next.

1.6.2 The canonical tool-calling loop

At runtime, the standard loop is usually just this:

1. Expose a set of tools to the model.
2. Invoke the model with the current messages.
3. If the model returns no tool calls, stop and use the answer.
4. If the model returns tool calls, execute each call, append the results as tool-result messages, and invoke the model again.

That is the whole mechanism. The sophistication comes from the tools, validation, and control logic around the loop, not from prompt text alone. Two practical details matter immediately. First, tool calls are a list, not a single slot. Even when your examples use one tool at a time, your runtime should assume the model may request more than one. Second, every tool result must be paired back to the original tool-call identifier. That identifier is how the runtime knows which observation belongs to which action. If those pairings are missing or malformed, the loop is broken. The standard loop is broader than any one framework. Some systems execute custom tools on your infrastructure. Some providers also offer built-in server-side tools such as search, file access, code execution, or similar capabilities. Conceptually, the pattern is unchanged. The model still asks for a capability, an execution layer still performs the work, and the result still comes back into the interaction. The main difference is where that execution boundary lives and who owns it operationally.

1.6.3 Implementing the loop directly in LangChain

The lowest-friction way to understand the pattern is to implement it once by hand. The example below uses a support scenario with two tools. The first tool looks up an order. The second evaluates whether the order is still within the standard refund window. The model is responsible for deciding when to use the tools. The runtime is responsible for executing them and feeding the results back.

```
from __future__ import annotations
import json
import os
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage, ToolMessage
from langchain.tools import tool
from .config import OPENAI_API_KEY

ORDERS = {
    "ORD-100": {
        "status": "delivered",
        "days_since_delivery": 12,
        "item_opened": False,
        "amount": 129.00
    },
    "ORD-101": {
        "status": "delivered",
        "days_since_delivery": 41,
        "item_opened": True,
        "amount": 59.00
    }
}

@tool
def lookup_order(order_id: str) -> str:
    """Return normalized facts for a single order."""
    order = ORDERS[order_id]
    return json.dumps(
        {
            "order_id": order_id,
            "status": order["status"],
            "days_since_delivery": order["days_since_delivery"],
```

```

        "item_opened": order["item_opened"],
        "amount": order["amount"]
    }
)

@tool
def evaluate_refund_policy(days_since_delivery: int, item_opened: bool) -> str:
    """Evaluate whether a delivered item is still eligible for the standard refund
    policy."""
    eligible = days_since_delivery <= 30 and not item_opened
    return json.dumps(
        {
            "eligible": eligible,
            "reason": (
                "within 30 days and unopened"
                if eligible
                else "outside the standard refund window"
            )
        }
    )

def main() -> None:
    model = init_chat_model(
        "gpt-4.1-mini",
        model_provider = "openai",
        temperature = 0
    )

    tools = [lookup_order, evaluate_refund_policy]
    tools_by_name = {tool.name: tool for tool in tools}

    # Bind the tool contracts to the model.
    model_with_tools = model.bind_tools(tools)

    messages = [
        HumanMessage(
            content=(
                "A customer asks whether order ORD-100 is still refundable. "
                "Check the order first, then answer."
            )
        )
    ]

    while True:
        ai_msg = model_with_tools.invoke(messages)
        messages.append(ai_msg)

        if not ai_msg.tool_calls:
            print("\nFINAL ANSWER\n")
            print(ai_msg.content)
            break

        for tool_call in ai_msg.tool_calls:
            print(f"\nTOOL CALL -> {tool_call['name']}({tool_call['args']})")

            tool = tools_by_name[tool_call["name"]]
            result = tool.invoke(tool_call["args"])

            print(f"TOOL RESULT -> {result}")

            messages.append(
                ToolMessage(
                    Content = result,
                    tool_call_id = tool_call["id"]
                )
            )

if __name__ == "__main__":
    main()

```

vol3_ch_01\src\react_langchain.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.react_langchain
TOOL CALL -> lookup_order({'order_id': 'ORD-100'})
TOOL RESULT -> {"order_id": "ORD-100", "status": "delivered", "days_since_delivery": 12,
"item_opened": false, "amount": 129.0}

TOOL CALL -> evaluate_refund_policy({'days_since_delivery': 12, 'item_opened': False})
TOOL RESULT -> {"eligible": true, "reason": "within 30 days and unopened"}

FINAL ANSWER

```

```
The order ORD-100 is still refundable as it was delivered 12 days ago and the item has not been opened. It meets the standard refund policy criteria. If you would like to proceed with the refund, please let me know!
```

The code binds two business tools to the chat model and then drives a full ReAct-style execution loop in application code rather than hiding that loop behind an agent wrapper. That choice is important because it makes the control flow explicit. The model is allowed to decide when a tool is needed, but the runtime remains responsible for executing the tool, capturing the result, and feeding that result back into the conversation. The tool binding happens when the model is wrapped with `bind_tools(tools)`. At that point, the model is no longer operating as a plain text generator. It is given a set of callable capabilities, each with a name, description, and argument schema derived from the tool definitions. In this example, that means the model can choose between looking up order facts and evaluating refund eligibility. The binding step is what turns those Python functions into actions the model can request during inference.

The message list is then initialized with a user request that deliberately leaves some work unresolved. The customer wants to know whether an order is refundable, but the answer depends on order data and policy evaluation. That setup encourages the model to use the tools rather than guessing. The `while True` loop is the core execution cycle. Each iteration sends the accumulated conversation to the bound model. The returned `ai_msg` may contain either a final natural-language answer or one or more structured tool calls. If no tool calls are present, the loop ends because the model has reached a stopping point and produced its answer. If tool calls are present, the runtime iterates through them one by one. It resolves the requested tool by name, invokes it with the supplied arguments, and captures the result. That result is not printed only for debugging convenience; it is also appended back into the message history as a `ToolMessage` linked to the original `tool_call_id`. That link is essential because it preserves the causal connection between the model's requested action and the returned observation. Once those tool results are added, the next loop iteration gives the model the updated state it needs to continue reasoning and either request another tool or answer the user.

1.6.4 Letting the runtime own the loop

Hand-writing the loop is excellent for learning and for cases where you need very tight control. In ordinary application code, however, you usually want the framework to manage the repetitive plumbing. In current LangChain, the high-level entry point for that is `create_agent()`, which builds a production-ready agent runtime on top of LangGraph. Under the hood, it is still the same agent loop: call the model, let it choose tools, execute them, and stop when no more tool calls are needed. A minimal version looks like this:

```
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-4.1-mini", model_provider = "openai", temperature = 0)
agent = create_agent(
    model = model,
    tools = [lookup_order, evaluate_refund_policy]
)

result = agent.invoke(
    {
        "messages": [
            ("user", "Can order ORD-100 still be refunded?")
```

```

    }
    )
    print(result["messages"][-1].content)

```

Older LangChain material often used `load_tools(...)`, `initialize_agent(...)`, `create_tool_calling_agent`, and `AgentExecutor`, with ReAct-style agent types such as `ZERO_SHOT_REACT_DESCRIPTION`. Those older examples are useful historically because they teach the same pattern clearly. Current LangChain guidance centres on `create_agent()` or on explicit LangGraph graphs, but the underlying workflow has not changed. The runtime is still managing a ReAct-like loop over tool calls and observations.

When you need the loop to be explicit rather than hidden inside a convenience wrapper, LangGraph expresses it directly as state and control flow. One node calls the model. Another node executes the requested tools. A conditional edge decides whether to go back to the model or to stop. That is not a different architecture. It is the same architecture with the moving parts made visible. This is exactly why LangGraph is useful: not because it invents a new kind of agent, but because it lets you represent the existing loop as an explicit graph that you can inspect, extend, and control. Across frameworks, this is the main difference you will keep seeing. Some frameworks hide more orchestration behind a high-level agent object. Others expose the state machine more directly. Some provider APIs support built-in tools on the server side. Others expect you to execute every tool yourself. Those are implementation differences, not conceptual differences. The contract remains the same: define capabilities clearly, let the model request them in structured form, execute them outside the model, and continue the interaction with the returned results.

1.6.5 A LangGraph version of the same loop

The same workflow becomes even clearer when it is written as an explicit graph. Instead of relying on a high-level agent wrapper, we define the state, add one node that calls the model, add one node that executes tools, and then use a conditional edge to decide whether the graph should continue or stop. This makes the ReAct-style loop visible in code. The model step decides whether another action is needed. The tool step performs the requested action and returns the observation. The graph then either loops back to the model or ends when no more tool calls are present. That is the same architecture as the manual loop but now represented as stateful control flow. The example below uses the same support scenario as the previous section. This is deliberate. When the business problem stays the same, the difference between the orchestration styles becomes much easier to see.

```

from __future__ import annotations
import json
import os
from typing import Annotated, Any, TypedDict
from langchain.chat_models import init_chat_model
from langchain.messages import AIMessage, ToolMessage
from langchain.tools import tool
from langgraph.graph import END, START, StateGraph
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from .config import OPENAI_API_KEY

ORDERS = {
    "ORD-100": {
        "status": "delivered",
        "days_since_delivery": 12,
        "item_opened": False,

```



```

        "amount": 129.00
    },
    "ORD-101": {
        "status": "delivered",
        "days_since_delivery": 41,
        "item_opened": True,
        "amount": 59.00
    }
}

@tool
def lookup_order(order_id: str) -> str:
    """Return normalized facts for a single order."""
    order = ORDERS[order_id]
    return json.dumps(
        {
            "order_id": order_id,
            "status": order["status"],
            "days_since_delivery": order["days_since_delivery"],
            "item_opened": order["item_opened"],
            "amount": order["amount"]
        }
    )

@tool
def evaluate_refund_policy(days_since_delivery: int, item_opened: bool) -> str:
    """Evaluate whether a delivered item is still eligible for the standard refund
    policy."""
    eligible = days_since_delivery <= 30 and not item_opened
    return json.dumps(
        {
            "eligible": eligible,
            "reason": (
                "within 30 days and unopened"
                if eligible
                else "outside the standard refund window"
            )
        }
    )

class SupportState(TypedDict):
    messages: Annotated[list, add_messages]

SYSTEM_PROMPT = """You are a customer support assistant.
Rules:
1. When you decide to use a tool, include one short visible sentence explaining the next
step.
2. Keep that sentence concise and factual.
3. After tool results arrive, explain the conclusion clearly.
4. Do not expose private reasoning. Only provide brief action summaries."""

def _json_dumps(value: Any) -> str:
    return json.dumps(value, ensure_ascii = False, default = str)

def _content_to_text(content: Any) -> str:
    if content is None:
        return ""
    if isinstance(content, str):
        return content
    if isinstance(content, list):
        parts: list[str] = []
        for item in content:
            if isinstance(item, str):
                parts.append(item)
            elif isinstance(item, dict):
                if item.get("type") == "text" and "text" in item:
                    parts.append(str(item["text"]))
                elif "text" in item:
                    parts.append(str(item["text"]))
                else:
                    parts.append(_json_dumps(item))
            else:
                parts.append(str(item))
        return " ".join(p for p in parts if p)
    if isinstance(content, dict):
        return _json_dumps(content)
    return str(content)

def _short(text: str, limit: int = 240) -> str:
    text = " ".join(text.split()).strip()
    if len(text) <= limit:

```



```

        return text
    return text[: limit - 3] + "..."

def _print_ai_message(msg: AIMessage) -> str | None:
    final_answer = None

    text = _short(_content_to_text(msg.content))
    if text:
        print(f"[assistant] {text}")

    tool_calls = getattr(msg, "tool_calls", None) or []
    for call in tool_calls:
        name = call.get("name", "<unknown>")
        args = call.get("args", {})
        print(f"[assistant -> tool] {name} ({_json_dumps(args)})")

    if text and not tool_calls:
        final_answer = text

    return final_answer

def _print_tool_message(msg: ToolMessage) -> None:
    name = getattr(msg, "name", None) or "tool"
    text = _short(_content_to_text(msg.content))
    print(f"[{name}] {text}")

def main() -> None:
    tools = [lookup_order, evaluate_refund_policy]

    model = init_chat_model(
        "gpt-4.1-mini",
        model_provider = "openai",
        temperature = 0
    )

    model_with_tools = model.bind_tools(tools)
    tool_node = ToolNode(tools)

    def call_model(state: SupportState) -> SupportState:
        response = model_with_tools.invoke(
            [
                ("system", SYSTEM_PROMPT),
                *state["messages"]
            ]
        )
        return {"messages": [response]}

    def route_after_model(state: SupportState) -> str:
        last_message = state["messages"][-1]
        return "tools" if getattr(last_message, "tool_calls", None) else "end"

    graph_builder = StateGraph(SupportState)
    graph_builder.add_node("call_model", call_model)
    graph_builder.add_node("tools", tool_node)

    graph_builder.add_edge(START, "call_model")
    graph_builder.add_conditional_edges(
        "call_model",
        route_after_model,
        {
            "tools": "tools",
            "end": END
        }
    )
    graph_builder.add_edge("tools", "call_model")

    graph = graph_builder.compile()

    input_state = {
        "messages": [
            (
                "user",
                "A customer asks whether order ORD-100 is still refundable. "
                "Check the order first, then answer."
            )
        ]
    }

    print("\nTRACE\n")

    step = 0
    final_answer: str | None = None

    for chunk in graph.stream(input_state, stream_mode = "updates"):
        for node_name, update in chunk.items():

```

```

    step += 1
    print(f"--- step {step}: {node_name} ---")
    messages = update.get("messages", [])
    for msg in messages:
        if isinstance(msg, AIMessage):
            maybe_final = _print_ai_message(msg)
            if maybe_final:
                final_answer = maybe_final
        elif isinstance(msg, ToolMessage):
            print(tool_message(msg))
        else:
            text = _short(_content_to_text(getattr(msg, "content", msg)))
            print(f"[{node_name}] {text}")

    print()

    print("FINAL ANSWER\n")
    print(final_answer or "<no final answer captured>")

if __name__ == "__main__":
    main()

```

vol3_ch_01\scr\react_langgraph.py

```

(.venv) C:\Users\alexc\11m-app\vol3_ch_01> python -m src.react_langgraph

TRACE

--- step 1: call model ---
[assistant -> tool] lookup_order({"order_id": "ORD-100"})

--- step 2: tools ---
[lookup_order] {"order_id": "ORD-100", "status": "delivered", "days_since_delivery": 12, "item_opened": false, "amount": 129.0}

--- step 3: call model ---
[assistant -> tool] evaluate_refund_policy({"days_since_delivery": 12, "item_opened": false})

--- step 4: tools ---
[evaluate_refund_policy] {"eligible": true, "reason": "within 30 days and unopened"}

--- step 5: call model ---
[assistant] The order ORD-100 is still refundable because it was delivered 12 days ago and the item has not been opened. It meets the standard refund policy criteria.

FINAL ANSWER

The order ORD-100 is still refundable because it was delivered 12 days ago and the item has not been opened. It meets the standard refund policy criteria.

```

The code binds the order tools directly to the chat model and then places that tool-aware model inside a small LangGraph loop. That combination is the core of the execution flow. The model is given the ability to request `lookup_order` and `evaluate_refund_policy`, but it never executes them itself. Binding the tools is what exposes their names, descriptions, and argument schemas to the model so it can choose them as part of its response. The important point in the code is that the model is not asked to answer in one shot. It is asked to participate in an iterative cycle where tool use is allowed and expected when needed. The `call_model` node is the reasoning step of the loop. It invokes the bound model with the system prompt plus the accumulated conversation state. Because the state uses `add_messages`, each new model response is appended rather than replacing previous messages. This matters because every tool result becomes part of the conversation history that the next model call can see. The model therefore reasons over a growing record of user input, prior assistant messages, and tool outputs.

The routing function controls whether the graph should continue or stop. It inspects the last message produced by the model and checks whether it contains any `tool_calls`. If tool calls

are present, execution moves to the `tools` node. If not, the graph ends. This is the actual ReAct loop in operational form: model step, tool step, model step again, repeating until the model no longer requests external actions. The `ToolNode` is the execution step. It reads the requested tool calls from the last AI message, runs the matching tool functions with the provided arguments, and returns the results as tool messages. Those tool messages are then appended to state and fed back into the next `call_model` invocation. That handoff is what allows the model to move from “I need information” to “I now have the information, so I can conclude.”

The streaming block makes this loop visible. Rather than only printing the final answer, it prints each model action and each tool result as the graph advances, which lets you observe the reasoning-action cycle as it unfolds.

There are two reasons this style matters. First, it makes the control flow explicit. In a larger system, you can inspect the graph and understand exactly where model calls happen, where tools run, and where the stop condition is evaluated. Second, it scales naturally when you need more control. The moment you want checkpoints, interrupts, custom routing, or additional state fields, the graph form is often easier to extend than a hand-written while loop or a high-level agent wrapper. LangGraph is not replacing the tool-calling pattern here. It is making the pattern concrete and operable.

1.6.6 Practical guidance

The first rule is to identify the type of problem correctly. If the task depends on current information, private business data, exact calculation, or an external action, it should usually be handled with tool calling. Prompt instructions alone are not enough in these cases. The model may produce a plausible answer, but plausibility is not the same as access to the correct information or the ability to perform the required action.

The second rule is to keep the control boundary clear. The model can decide that a tool may be useful and can request a tool call with arguments. The runtime decides whether that request is valid, authorized, safe, and executable. Exposing a tool to the model does not mean giving the model direct control over the system. Execution remains an application responsibility.

The third rule is to separate the workflow pattern from the visible text format. ReAct is not only a prompt style where reasoning and actions are written as text. It is a workflow pattern: the system reasons over the current state, acts through tools, observes the result, and continues until it has enough information to stop. Modern systems may expose short reasoning traces, hide reasoning entirely, or return only structured tool-call objects. The important part is the loop, not the formatting.

The fourth rule is to apply the same pattern in multi-turn conversations. Memory does not replace the tool-calling loop. It changes the context available to the next model step. A conversational support assistant still follows the same basic structure: inspect the current state, decide whether a tool is needed, execute the tool through the runtime, observe the result, and continue. The difference is that the next tool decision can depend on the latest user message and on relevant earlier conversation state.

The fifth rule is to test the loop as a loop. Because the runtime protocol is explicit, not every test needs to call a live model. You can simulate or record model tool-call outputs, pass them

through the tool-execution path, and verify that the system appends the correct tool results, handles errors correctly, and reaches the expected final state. This is more useful than testing only the final text response, because many agent failures happen in the intermediate steps.

Seen this way, standard tool calling and ReAct are not competing ideas. Tool calling is the execution protocol: the model requests a tool call, and the runtime executes it under application control. ReAct is the workflow pattern that uses this protocol to solve tasks step by step. LangChain provides convenient ways to bind tools and run agents. LangGraph lets you make the loop explicit as state and control flow. The foundation is the same in both cases: the model does not solve the whole task from internal knowledge alone. It progresses by combining reasoning with controlled access to external capabilities.

1.7 THE `CREATE_AGENT()` COMPONENT IN DEPTH

By this point, the basic tool-calling loop should already feel familiar. A model receives messages and tool definitions, decides whether a tool is needed, gets the tool result back, and continues until it can answer. `create_agent()` matters because it packages that loop as a production-ready construction primitive. It is not a reasoning algorithm by itself, and it is not just a convenience wrapper around `bind_tools()`. It is an agent factory that assembles a model, a tool interface, an operating prompt, and a graph runtime into one executable object. In current LangChain, it is the standard high-level entry point for building agents, and it supersedes the older `langgraph.prebuilt.create_react_agent` path.

That design choice is important because it explains both the strengths and the limits of the abstraction. `create_agent()` is high level, but it is not opaque. The object it returns follows the LangGraph Graph API, which is why methods such as `invoke()` and `stream()` work naturally, and why features such as persistence, interrupts, human review, and streaming do not feel bolted on afterward. A useful visualisation is to see `create_agent()` as four layers packaged together: the standard model-and-tools loop, the LangGraph runtime that executes it, a middleware system that shapes behavior, and a persistence/runtime layer for state, context, and memory.

1.7.1 The execution contract

At runtime, `create_agent()` follows a simple and stable execution pattern. The runtime sends the current conversation state to the model together with the tools the model is allowed to use, the system instructions, and any structured-output constraints. The model then has two basic choices. It can produce a final answer, or it can request one or more tool calls. When tool calls are requested, the runtime executes them, converts the results into messages the model can read, adds those messages back into state, and continues the loop. The run ends when the model returns an answer instead of another tool call, or when the runtime reaches its iteration limit. The important point is that `create_agent()` does not invent a different agent pattern. It gives that pattern a well-defined runtime with clear boundaries and predictable behavior.

The way you invoke the agent matches that design. You do not pass a raw prompt and wait for a single string to come back. You pass a state update, most often a `messages` list containing the latest user message, and the runtime returns the updated agent state. That returned state usually includes the full message history after the run, not just the final assistant reply. If structured output is enabled, the validated result is also returned separately in `structured_response`. This is a small but important distinction. The return value is state, not just text, because the runtime is managing a multi-step process rather than a one-shot completion.

Tools are explicit at the same boundary. The `tools` argument can contain normal LangChain tool objects, ordinary Python callables that expose a clear signature and description, and supported provider-native tool descriptors. In all cases, the runtime turns them into a tool surface the model can reason over and call. That makes the tool boundary inspectable instead of implicit. If you pass no tools at all, the runtime still works, but the graph effectively becomes a single model step with no tool-execution phase. The code is valid, but it is no longer a tool-

using agent in the practical sense, because there is nothing available for the model to call beyond the model itself.

1.7.2 Reading the parameter surface correctly

The easiest way to understand the `create_agent()` signature is not to memorize every parameter, but to group them by responsibility. First, there is the reasoning surface: `model`, `tools`, `system_prompt`, and optionally `name`. These define what the agent is. The model can be passed as a model identifier string or as a fully configured chat-model object when you need direct control over parameters such as temperature, token limits, timeouts, or provider-specific settings. The system prompt can be a plain string or a `SystemMessage`, which is useful when you need richer content blocks or provider-specific features such as prompt caching. The `name` parameter is more important than it first appears, because it becomes the node identifier when the agent is used as a subgraph in a larger multi-agent system. Current guidance recommends snake_case, and avoiding spaces or special characters, because some providers reject them.

Second, there is the control surface: `middleware`. This is where most serious customization now lives. Rather than giving you a huge number of specialized agent classes, it gives you one standard agent factory and a middleware system that can modify prompts, swap models, filter tools, apply retries, enforce limits, redact PII, summarize history, and interrupt execution for approval. In practice, this is one of the most important things to understand about modern `create_agent()`: the agent factory builds the loop, but middleware governs its behavior.

Third, there is the data surface: `state_schema`, `context_schema`, `store`, and `checkpointers`. These define what data the agent can see, what survives across steps, and what survives across turns. `state_schema` customizes short-term agent state. `context_schema` defines per-invocation runtime dependencies such as user IDs or service handles. `store` exposes long-term memory across sessions. `checkpointers` persists thread state step by step so the run can be resumed. This is the layer that turns `create_agent()` from a neat loop into something operationally useful.

Fourth, there is the integration surface: `response_format`, `streaming`, and operational controls such as `interrupt_before`, `interrupt_after`, and `debug`. These are the knobs that matter when the agent has to live inside a real application, not just inside a notebook. Structured output changes the final return contract. Streaming changes the user experience. Interrupt and debug hooks make the runtime inspectable and controllable.

1.7.3 A fuller support assistant with `create_agent()`

A very small example is useful for showing that an agent can call a tool, but it often hides the real value of `create_agent()`. The more important thing to see is that `create_agent()` is the point where a model, a tool surface, a system instruction, and an agent identity are assembled into one executable runtime. The tools themselves are still ordinary Python functions. The prompt is still just an instruction. The model is still just a model. What changes is that the runtime now owns the loop that decides when to call a tool, how to pass the tool result back to the model, and when the run is complete. That is the feature we want to make visible in the example. In the support scenario below, the user asks a simple business question, but the answer depends on several pieces of information. The agent must retrieve the order

facts, retrieve the policy facts, combine them, and then turn the result into a clear support response. This is a good example of `create_agent()` because the orchestration is real enough to matter, but still small enough to read in one pass.

```
from typing import Any
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_core.messages import AIMessage, HumanMessage, ToolMessage

ORDERS = {
    "ORD-100": {
        "status": "delivered",
        "days_since_delivery": 12,
        "item_opened": False,
        "amount": 129.00
    },
    "ORD-101": {
        "status": "delivered",
        "days_since_delivery": 41,
        "item_opened": True,
        "amount": 59.00
    }
}

REFUND_POLICY = {
    "max_days": 30,
    "opened_items_allowed": False,
    "return_shipping_paid_by_customer": True
}

@tool
def lookup_order(order_id: str) -> dict[str, Any]:
    """Return the order facts needed for a refund decision."""
    order = ORDERS.get(order_id)
    if order is None:
        return {"found": False, "order_id": order_id}
    return {"found": True, "order_id": order_id, **order}

@tool
def get_refund_policy() -> dict[str, Any]:
    """Return the store's standard refund policy."""
    return REFUND_POLICY

@tool
def evaluate_refund(
    days_since_delivery: int,
    opened: bool,
    max_days: int,
    opened_items_allowed: bool
) -> dict[str, Any]:
    """Evaluate refund eligibility from order facts and policy facts."""
    eligible = (
        days_since_delivery <= max_days
        and (opened_items_allowed or not opened)
    )

    if eligible:
        reason = "The order is within the refund window and meets the item-condition rule."
    elif days_since_delivery > max_days:
        reason = "The order is outside the refund window."
    else:
        reason = "Opened items are not eligible under the standard policy."

    return {
        "eligible": eligible,
        "reason": reason
    }

agent = create_agent(
    model = "openai:gpt-5",
    tools = [lookup_order, get_refund_policy, evaluate_refund],
    system_prompt = (
        "You are a customer support assistant for an online store. "
        "When a user asks about refund eligibility, first gather the order facts, "
        "then retrieve the refund policy, then evaluate eligibility, and finally "
        "answer in plain language. If the order is not eligible, explain why clearly. "
        "If the order is eligible, explain the next step."
    )
)
```

```

    ),
    name = "support_refund_agent"
)

result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Can order ORD-100 be refunded? Please explain the reason."
            }
        ]
    }
)

print("FINAL ANSWER")
print(result["messages"][-1].content)
print()

print("MESSAGE TRACE")
for message in result["messages"]:
    if isinstance(message, HumanMessage):
        print(f"USER: {message.content}")
    elif isinstance(message, AIMessage) and message.tool_calls:
        print("ASSISTANT REQUESTED TOOLS:")
        for call in message.tool_calls:
            print(f"  - {call['name']}({call['args']})")
    elif isinstance(message, ToolMessage):
        print(f"TOOL RESULT [{message.name}]: {message.content}")
    elif isinstance(message, AIMessage):
        print(f"ASSISTANT: {message.content}")

```

vol3_ch_01\src\create_agent.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.create_agent

FINAL ANSWER
Yes, order ORD-100 is eligible for a refund.

Reason:
- Delivered 12 days ago, which is within the 30-day refund window.
- Item is unopened, and the policy does not allow refunds for opened items.

Next step:
- You can start a return. Per our policy, return shipping is paid by the customer. Would you like me to initiate the return for order ORD-100?

MESSAGE TRACE
USER: Can order ORD-100 be refunded? Please explain the reason.
ASSISTANT REQUESTED TOOLS:
  - lookup_order({'order_id': 'ORD-100'})
  - get_refund_policy({})
TOOL RESULT [lookup_order]: {"found": true, "order_id": "ORD-100", "days_since_delivery": 12, "opened": false, "amount": 129.0, "status": "delivered"}
TOOL RESULT [get_refund_policy]: {"max_days": 30, "opened_items_allowed": false, "return_shipping_paid_by_customer": true}
ASSISTANT REQUESTED TOOLS:
  - evaluate_refund({'days_since_delivery': 12, 'opened': False, 'max_days': 30, 'opened_items_allowed': False})
TOOL RESULT [evaluate_refund]: {"eligible": true, "reason": "The order is within the refund window and meets the item-condition rule."}
ASSISTANT: Yes, order ORD-100 is eligible for a refund.

Reason:
- Delivered 12 days ago, which is within the 30-day refund window.
- Item is unopened, and the policy does not allow refunds for opened items.

Next step:
- You can start a return. Per our policy, return shipping is paid by the customer. Would you like me to initiate the return for order ORD-100?

```

What the example makes visible is the shape of the runtime output. The call to `agent.invoke()` returns updated agent state rather than only a final answer, and the `messages` field preserves the sequence produced during execution. That sequence is useful because it shows the path from the user request to the final response: tool requests, tool results,

and the concluding assistant message all live in the same returned state. For an engineer, this is an important part of the contract. The agent is not just a text generator. It is a stateful runtime that records the interaction it has just completed. The example also shows why the tool surface is kept narrow. Each tool does one clear job and exposes a small contract. That keeps the model's choices legible and keeps business logic easier to reason about. In practice, this is usually a better fit for real applications than hiding everything behind one large business function, because application data, policies, and decisions are often separated across different services or modules. A tool-using agent works best when those boundaries remain visible.

The name parameter is worth keeping in view even in a small example. It does not affect the refund decision itself, but it gives the runtime a stable identity. That becomes useful when the same agent is reused as part of a larger graph or a multi-agent design, where clear runtime identities matter for composition and tracing.

1.7.4 Middleware is where the real customization happens

If `create_agent()` is the assembly point for the agent runtime, `middleware` is the place where that runtime is shaped. The current `middleware` surface is built around two hook families. Node-style hooks run at fixed points in the lifecycle: before the run starts, before each model call, after each model response, and after the run completes. Wrap-style hooks run around model calls or tool calls, so they can retry, short-circuit, replace inputs, replace outputs, or call the wrapped handler more than once. There is also a convenience layer for dynamic prompts, which is really a specialized way of changing the system instruction at call time. The practical consequence is that most advanced behavior is no longer expressed as a different agent class. It is expressed as policy around the standard loop.

That design is the reason `middleware` feels so central in modern LangChain. Dynamic model routing, prompt shaping, selective tool exposure, retries, limits, fallbacks, PII handling, summarization, and human approval all fit into the same extension surface. Some policies are best written as small custom hooks. Others are already packaged as built-in `middleware`. In practice, real systems usually combine both: a small amount of domain-specific custom `middleware` on top of a stack of reusable controls. The prompt remains important, but its role is narrower: it steers how the available operations are used; it does not replace tool contracts or runtime policy. The following example shows two common `middleware` patterns together: changing the system prompt from runtime context and changing the model based on conversation state.

```
from __future__ import annotations
import os
from dataclasses import dataclass
from typing import Callable
from langchain.agents import create_agent
from langchain.agents.middleware import (
    ModelRequest,
    ModelResponse,
    dynamic_prompt,
    wrap_model_call
)
from langchain_openai import ChatOpenAI
from .config import OPENAI_API_KEY
from .react_langgraph import lookup_order, evaluate_refund_policy

basic_model = ChatOpenAI(model = "gpt-4.1-mini", temperature = 0)
advanced_model = ChatOpenAI(model = "gpt-4.1", temperature = 0)

@dataclass
```

```

class Context:
    user_role: str

@wrap_model_call
def select_model(request: ModelRequest, handler):
    message_count = len(request.state["messages"])
    chosen = advanced_model if message_count >= 4 else basic_model
    print(f"[middleware:model] message_count={message_count} -> {chosen.model_name}")
    return handler(request.override(model = chosen))

@dynamic_prompt
def role_prompt(request: ModelRequest) -> str:
    base = (
        "You are a customer support assistant. "
        "Before any tool call, write one short visible sentence explaining the next step"
        "Do not expose hidden reasoning. Use concise action summaries only."
    )

    message_count = len(request.state["messages"])

    if message_count >= 4:
        prompt = base + "Now synthesize the final answer using the tool results only."
    else:
        prompt = base + "First decide which tool to call next."

    print("\n[middleware:prompt] effective prompt:")
    print(prompt)
    print()
    return prompt

agent = create_agent(
    model = basic_model,
    tools = [lookup_order, evaluate_refund_policy],
    middleware = [select_model, role_prompt],
    context_schema = Context
)

def print_trace(user_text: str, role: str = "expert") -> None:
    print("\nTRACE\n")

    step = 0
    final_answer = None

    for chunk in agent.stream(
        {
            "messages": [
                {
                    "role": "user",
                    "content": user_text
                }
            ]
        },
        context = Context(user_role = role),
        stream_mode = "updates"
    ):
        for node_name, update in chunk.items():
            step += 1
            print(f"--- step {step}: {node_name} ---")

            messages = update.get("messages", [])
            if not messages:
                print(update)
                print()
                continue

            for msg in messages:
                tool_calls = getattr(msg, "tool_calls", None)

                if tool_calls:
                    text = getattr(msg, "content", "")
                    if text:
                        print(f"[assistant] {text}")

                    for tc in tool_calls:
                        print(f"[assistant -> tool] {tc['name']} ({tc['args']})")

                else:
                    content = getattr(msg, "content", "")
                    name = msg.__class__.__name__

                    if name == "ToolMessage":
                        print(f"[tool] {content}")
                    else:
                        print(f"[assistant] {content}")

```

```

        final_answer = content

        print()

        print("FINAL ANSWER\n")
        print(final_answer or "<no final answer captured>")

if __name__ == "__main__":
    print_trace(
        "Explain the refund decision for ORD-100. First check the order, then evaluate refund eligibility.",
        role = "expert"
    )

```

vol3_ch_01\src\agent_dynamic_prompt.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.agent_dynamic_prompt

TRACE

[middleware:model] message_count=1 -> gpt-4.1-mini

[middleware:prompt] effective prompt:
You are a customer support assistant. Before any tool call, write one short visible sentence explaining the next step. Do not expose hidden reasoning. Use concise action summaries only. First decide which tool to call next.

--- step 1: model ---
[assistant] I will first retrieve the details for order ORD-100 and then check the refund eligibility.
[assistant -> tool] lookup_order({'order_id': 'ORD-100'})
[assistant -> tool] evaluate_refund_policy({'days_since_delivery': 0, 'item_opened': False})

--- step 2: tools ---
[tool] {"order_id": "ORD-100", "status": "delivered", "days_since_delivery": 12, "item_opened": false, "amount": 129.0}

--- step 3: tools ---
[tool] {"eligible": true, "reason": "within 30 days and unopened"}

[middleware:model] message_count=4 -> gpt-4.1

[middleware:prompt] effective prompt:
You are a customer support assistant. Before any tool call, write one short visible sentence explaining the next step. Do not expose hidden reasoning. Use concise action summaries only. Now synthesize the final answer using the tool results only.

--- step 4: model ---
[assistant] Order ORD-100 was delivered 12 days ago and the item is unopened. According to the refund policy, it is eligible for a refund because it is within 30 days of delivery and the item has not been opened.

FINAL ANSWER
Order ORD-100 was delivered 12 days ago and the item is unopened. According to the refund policy, it is eligible for a refund because it is within 30 days of delivery and the item has not been opened.

```

The code uses middleware to adapt both the model and the system instruction at runtime without changing the agent definition. `select_model()` inspects the current message history and upgrades from the smaller model to the stronger one once the conversation reaches four messages. This keeps simple turns cheap while reserving more capable inference for later stages, when the interaction is likely to require synthesis rather than straightforward tool selection. `role_prompt()` also reads the current state and rewrites the effective system prompt accordingly. Early in the run it pushes the model toward choosing the next tool; later it shifts the model toward composing the final answer from tool results.

1.7.5 A custom middleware stack for a support assistant

This example stays in the support domain and focuses on custom middleware. It shows the full shape of the middleware surface without introducing persistence or human approval

yet. The agent uses all of the following in one coherent flow: lifecycle hooks, dynamic prompt generation, dynamic model selection, dynamic tool filtering, early termination, and tool-call interception.

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Callable
from typing_extensions import NotRequired

from langchain.agents import create_agent
from langchain.agents.middleware import (
    AgentMiddleware,
    AgentState,
    ExtendedModelResponse,
    ModelRequest,
    ModelResponse,
    dynamic_prompt,
    hook_config
)
from langchain.messages import AIMessage, ToolMessage
from langchain.tools import tool
from langchain.tools.tool_node import ToolCallRequest
from langgraph.types import Command
from langchain_openai import ChatOpenAI
from .config import OPENAI_API_KEY

ORDERS = {
    .
    .
}

REFUND_POLICY = {
    "max_days": 30,
    "opened_items_allowed": False
}

@tool
def lookup_order(order_id: str) -> dict[str, Any]:
    """Return order facts needed for a refund decision."""
    order = ORDERS.get(order_id)
    if order is None:
        return {"found": False, "order_id": order_id}
    return {"found": True, "order_id": order_id, **order}

@tool
def get_refund_policy() -> dict[str, Any]:
    """Return the current standard refund policy."""
    return REFUND_POLICY

@tool
def evaluate_refund(
    days_since_delivery: int,
    opened: bool,
    max_days: int,
    opened_items_allowed: bool
) -> dict[str, Any]:
    """Evaluate refund eligibility from order facts and policy facts."""
    eligible = (
        days_since_delivery <= max_days
        and (opened_items_allowed or not opened)
    )

    if eligible:
        reason = "The order is within the refund window and meets the item-condition rule."
    elif days_since_delivery > max_days:
        reason = "The order is outside the refund window."
    else:
        reason = "Opened items are not eligible under the standard policy."

    return {
        "eligible": eligible,
        "reason": reason
    }

@tool
def create_return_label(order_id: str) -> str:
    """Create a return label for an order that is eligible for return processing."""

```

```

        return f"Return label created for {order_id}"

basic_model = ChatOpenAI(
    model = "gpt-4.1-mini",
    temperature = 0
)

advanced_model = ChatOpenAI(
    model = "gpt-4.1",
    temperature = 0
)

@dataclass
class SupportContext:
    user_tier: str # "standard" or "premium"
    audience: str # "end_user" or "expert"

class SupportState(AgentState):
    model_call_count: NotRequired[int]
    selected_model: NotRequired[str]
    started: NotRequired[bool]
    finished: NotRequired[bool]

@dynamic_prompt
def support_prompt(request: ModelRequest) -> str:
    audience = request.runtime.context.audience

    if audience == "expert":
        style = "Be concise, technical, and explicit about the rule that was applied."
    else:
        style = "Explain the result in plain language and avoid jargon."

    return (
        "You are a customer support assistant. "
        "For refund questions, first gather order facts, then gather policy facts, "
        "then evaluate eligibility, then explain the result clearly. "
        "Before each tool call, write one short visible sentence describing the next "
        "step. "
        "Do not expose hidden reasoning. Use concise action summaries only. "
        f"{style}"
    )

class SupportOperationsMiddleware(AgentMiddleware[SupportState]):
    state_schema = SupportState

    def before_agent(self, state: SupportState, runtime) -> dict[str, Any] | None:
        return {
            "started": True,
            "model_call_count": 0,
            "finished": False
        }

    @hook_config(can_jump_to=["end"])
    def before_model(self, state: SupportState, runtime) -> dict[str, Any] | None:
        # A normal successful path may need:
        # 1) lookup_order
        # 2) get_refund_policy
        # 3) evaluate_refund
        # 4) create return label
        # 5) final answer
        # Give it one extra turn of headroom.
        if state.get("model_call_count", 0) >= 6:
            return {
                "messages": [
                    AIMessage(
                        content=(
                            "I could not complete this request within the allowed "
                            "reasoning budget. Please hand it to a human support agent."
                        )
                    )
                ],
                "jump_to": "end"
            }
        return None

    def after_model(self, state: SupportState, runtime) -> dict[str, Any] | None:
        return {"model_call_count": state.get("model_call_count", 0) + 1}

    def after_agent(self, state: SupportState, runtime) -> dict[str, Any] | None:
        return {"finished": True}

    def wrap_model_call(
        self,

```

```

        request: ModelRequest,
        handler: Callable[[ModelRequest], ModelResponse]
    ) -> ExtendedModelResponse:
        if len(request.messages) > 8:
            selected_model = advanced_model
            selected_label = "gpt-4.1"
        else:
            selected_model = basic_model
            selected_label = "gpt-4.1-mini"

        if request.runtime.context.user_tier == "premium":
            visible_tools = list(request.tools)
        else:
            visible_tools = [
                tool for tool in request.tools
                if tool.name != "create_return_label"
            ]

        response = handler(
            request.override(
                model = selected_model,
                tools = visible_tools
            )
        )

        return ExtendedModelResponse(
            model_response = response,
            command = Command(update={"selected_model": selected_label})
        )

def wrap_tool_call(
    self,
    request: ToolCallRequest,
    handler: Callable[[ToolCallRequest], ToolMessage | Command]
) -> ToolMessage | Command:
    try:
        return handler(request)
    except Exception as exc:
        return ToolMessage(
            content = f"Tool {request.tool_call['name']} failed: {exc}",
            tool_call_id = request.tool_call["id"],
            name = request.tool_call["name"]
        )

agent = create_agent(
    model = basic_model,
    tools = [lookup_order, get_refund_policy, evaluate_refund, create_return_label],
    middleware = [
        SupportOperationsMiddleware(),
        support_prompt
    ],
    context_schema = SupportContext,
    name = "support_refund_agent"
)

def print_trace(result: dict[str, Any]) -> None:
    .
    .
    .

def main() -> None:
    inputs = {
        "messages": [
            {
                "role": "user",
                "content": (
                    "Can order ORD-100 be refunded? "
                    "If it is eligible, create a return label."
                )
            }
        ]
    }

    context = SupportContext(
        user_tier = "premium",
        audience = "end_user"
    )

    result = agent.invoke(inputs, context = context)
    print_trace(result)

if __name__ == "__main__":

```

```
main()
```

```
vol3_ch_01\scr\custom_middleware.py
```

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.custom_middleware

TRACE

--- step 1: assistant ---
[assistant] I will first check the order details and the refund policy to determine if
the order ORD-100 is eligible for a refund.
[assistant -> tool] lookup_order({'order id': 'ORD-100'})
[assistant -> tool] get_refund_policy({})
--- step 2: tool ---
[tool:lookup_order] {"found": true, "order_id": "ORD-100", "days_since_delivery": 12,
"opened": false, "amount": 129.0, "status": "delivered"}
--- step 3: tool ---
[tool:get refund policy] {"max days": 30, "opened items allowed": false}
--- step 4: assistant ---
[assistant -> tool] evaluate_refund({'days_since_delivery': 12, 'opened': False,
'max_days': 30, 'opened_items_allowed': False})
--- step 5: tool ---
[tool:evaluate_refund] {"eligible": true, "reason": "The order is within the refund
window and meets the item-condition rule."}
--- step 6: assistant ---
[assistant] The order ORD-100 is eligible for a refund because it was delivered 12 days
ago, which is within the 30-day refund window, and the item has not been opened. I will
now create a return label for you.
[assistant -> tool] create_return_label({'order_id': 'ORD-100'})
--- step 7: tool ---
[tool:create_return_label] Return label created for ORD-100
--- step 8: assistant ---
[assistant] Your order ORD-100 is eligible for a refund. A return label has been created
for you to send the item back. If you need further assistance, please let me know!

FINAL

Your order ORD-100 is eligible for a refund. A return label has been created for you to
send the item back. If you need further assistance, please let me know!
selected_model = gpt-4.1-mini
model_call_count = 4
started = True finished = True
```

SupportOperationsMiddleware is the operational control layer for the agent. It does not answer the refund question itself. Its job is to govern how the agent runs, how much work it is allowed to do, which model it should use, which tools it may expose, and how failures are handled. This makes the example useful because every middleware element has a clearly defined responsibility and those responsibilities map cleanly onto the execution lifecycle.

The class begins by declaring SupportState as its state schema. That gives the middleware a place to persist run-level facts such as whether execution has started, whether it has finished, how many model turns have been consumed, and which model was selected. Those fields are not business data. They are operational facts about the run itself, which is exactly the kind of information middleware should manage. The lifecycle hooks define that execution envelope. `before_agent()` initializes the run in a predictable state by marking it as started, resetting the model-call counter, and marking it as not yet finished. `after_agent()` closes the loop by marking the run as finished. Between those two boundaries, `before_model()` and `after_model()` manage the model budget. `after_model()` increments the counter after each model response, while `before_model()` checks whether the run has already consumed too many turns. If the budget has been exhausted, it injects a clear assistant message and jumps directly to the end of the graph. That is an important production safeguard. A refund flow may legitimately require several reasoning steps, but the runtime should still fail in a bounded and explainable way rather than looping indefinitely. All four of these are node-style hooks,

so they work by returning state updates at predictable points in execution. That makes them the right fit for bookkeeping, validation, and simple control decisions.

The wrap hooks do a different kind of work. `wrap_model_call()` changes what the model sees at the point of execution. It selects between the smaller and larger model based on conversation length, which keeps shorter runs cheaper while reserving the stronger model for more demanding interactions. In the same method, it filters the visible tool set according to customer tier. Premium users can see `create_return_label`, while standard users cannot. The wrapped call then executes with that adjusted model and tool surface, and the middleware records the chosen model back into agent state through an `ExtendedModelResponse` and `Command`. This is a good example of why wrap hooks matter: they do not just observe execution; they can reshape it.

`wrap_tool_call()` applies the same idea at the tool boundary. Instead of letting a raw exception terminate the run, it converts a failing tool call into a `ToolMessage`. That means the failure is turned into structured feedback that the model can read and potentially recover from. This is the practical difference between the two hook families. Node hooks act at fixed lifecycle points. Wrap hooks intervene around a call boundary and can alter the path of execution itself. The dynamic prompt belongs to the same overall design even though its syntax is simpler. It is still call-time customization. The prompt changes from runtime context, specifically the audience type, rather than from a second agent definition or branching application code. That keeps the design easier to reason about. There is still one agent runtime, but its behavior is shaped at execution time through middleware. For expert users, the prompt asks for concise and technical explanations. For end users, it asks for plain language. The underlying tools and workflow remain the same.

One subtle but important point is that tool filtering here is dynamic, but the tools are still pre-registered up front. That is the common pattern when all candidate tools are known at startup and middleware only needs to decide which subset should be visible for a particular call. It keeps the model's working tool surface smaller, reduces confusion, and lets the application enforce permissions without rewriting the rest of the agent. Taken together, the example shows middleware in its most useful form. The agent remains a standard `create_agent()` runtime, but the middleware turns it into a controlled operational component: bounded, traceable, context-sensitive, permission-aware, and resilient in the face of tool failures.

1.7.6 A production-control stack for a high-stakes refund assistant

The second example shows the other side of middleware: reusable operational controls. Here the assistant can issue store credit, which is a write operation and therefore a higher-risk action than a read-only lookup. The middleware stack combines long-thread summarization, PII protection, model fallback, call budgets, and human approval before the store-credit tool is allowed to execute.

```
from __future__ import annotations
import os
import uuid
from typing import Any
from langchain.agents import create_agent
from langchain.agents.middleware import (
    HumanInTheLoopMiddleware,
    ModelCallLimitMiddleware,
    ModelFallbackMiddleware,
```



```

    PIIMiddleware,
    SummarizationMiddleware,
    ToolCallLimitMiddleware,
)
from langchain.messages import AIMessage, ToolMessage
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from .config import OPENAI_API_KEY

ORDERS = {
    .
    .
}

@tool
def lookup_order(order_id: str) -> dict[str, Any]:
    """Return order information needed for support actions."""
    order = ORDERS.get(order_id)
    if order is None:
        return {"found": False, "order_id": order_id}
    return {"found": True, "order_id": order_id, **order}

@tool
def issue_store_credit(order_id: str, amount: float, reason: str) -> str:
    """Issue store credit for a customer order."""
    return f"Store credit of {amount:.2f} issued for {order_id}. Reason: {reason}"

primary_model = ChatOpenAI(
    model = "gpt-4.1",
    temperature = 0
)

fallback_model = ChatOpenAI(
    model = "gpt-4.1-mini",
    temperature = 0
)

agent = create_agent(
    model = primary_model,
    tools = [lookup_order, issue_store_credit],
    system_prompt = (
        "You are a customer support assistant. "
        "Always look up the order before taking any action. "
        "If the user explicitly asks for store credit and the order exists, "
        "use the issue_store_credit tool instead of only describing the action. "
        "After the action is completed, explain clearly what happened."
    ),
    middleware = [
        PIIMiddleware("email", strategy = "redact", apply_to_input = True),
        PIIMiddleware("credit_card", strategy = "mask", apply_to_input = True),
        SummarizationMiddleware(
            model = fallback_model,
            trigger = ("messages", 6),
            keep = ("messages", 4)
        ),
        ModelFallbackMiddleware(fallback_model),
        ModelCallLimitMiddleware(
            thread_limit = 20,
            run_limit = 6,
            exit_behavior = "end"
        ),
        ToolCallLimitMiddleware(
            thread_limit = 10,
            run_limit = 4
        ),
        ToolCallLimitMiddleware(
            tool_name = "issue_store_credit",
            thread_limit = 1,
            run_limit = 1
        ),
        HumanInTheLoopMiddleware(
            interrupt_on = {
                "issue_store_credit": {
                    "allowed_decisions": ["approve", "edit", "reject"]
                },
                "lookup_order": False
            }
        )
    ],
    checkpoint = InMemorySaver(),

```

```

    name = "support_store_credit_agent"
)

def print_trace(result: dict[str, Any]) -> None:
    .
    .

def _get_interrupts(state: Any) -> list[Any]:
    if state is None:
        return []

    if isinstance(state, dict):
        return state.get("interrupts", []) or []

    interrupts = getattr(state, "interrupts", None)
    if interrupts is not None:
        return interrupts

    values = getattr(state, "values", None)
    if isinstance(values, dict):
        return values.get("interrupts", []) or []

    return []

def _get_result_values(result: Any) -> dict[str, Any]:
    if result is None:
        return {}

    if isinstance(result, dict):
        if "values" in result and isinstance(result["values"], dict):
            return result["values"]
        return result

    value = getattr(result, "value", None)
    if isinstance(value, dict):
        return value

    values = getattr(result, "values", None)
    if isinstance(values, dict):
        return values

    return {}

def main() -> None:
    config = {
        "configurable": {
            "thread_id": f"refund-thread-{uuid.uuid4()}"
        }
    }

    # Warm-up turns so the thread has enough history for summarization.
    for user_message in [
        "My order arrived last week.",
        "The item is still sealed.",
        "I would prefer store credit instead of a replacement."
    ]:
        agent.invoke(
            {
                "messages": [
                    {
                        "role": "user",
                        "content": user_message
                    }
                ]
            },
            config = config,
            version = "v2"
        )

    inputs = {
        "messages": [
            {
                "role": "user",
                "content": (
                    "My email is alex@example.com and my card is 4111 1111 1111 1111. "
                    "Please issue store credit for order ORD-100."
                )
            }
        ]
    }

    # Start the run. If HITL pauses correctly, the pending interrupt will be
    # stored in the thread state.
    agent.invoke(

```

```

        inputs,
        config = config,
        version = "v2"
    )

    state = agent.get_state(config)
    interrupts = _get_interrupts(state)

    print("INTERRUPTS\n")
    if not interrupts:
        print("No pending interrupts.")
        return

    for idx, interrupt in enumerate(interrupts, start = 1):
        print(f"--- interrupt {idx} ---")
        print(interrupt)
        print()

    # Approve the paused action and resume the same thread.
    approved = agent.invoke(
        Command(resume = {"decisions": [{"type": "approve"}]}),
        config = config,
        version = "v2"
    )

    print_trace(_get_result_values(approved))

if __name__ == "__main__":
    main()

```

vol3_ch_01\src\standard_middleware.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.standard_middleware

INTERRUPTS

--- interrupt 1 ---
Interrupt(value={'action_requests': [{'name': 'issue_store_credit', 'args': {'order_id': 'ORD-100', 'amount': 129.0, 'reason': 'Customer requested store credit for sealed item.'}, 'description': "Tool execution requires approval\n\nTool: issue_store_credit\nArgs: {'order_id': 'ORD-100', 'amount': 129.0, 'reason': 'Customer requested store credit for sealed item.'}"}], 'review_configs': [{'action_name': 'issue_store_credit', 'allowed_decisions': ['approve', 'edit', 'reject']}], id='0e4a208ad7c3a43a190879c8dde6275d'})

TRACE

--- step 1: assistant ---
[assistant -> tool] lookup_order({'order_id': 'ORD-100'})
--- step 2: tool ---
[tool:lookup_order] {"found": true, "order_id": "ORD-100", "amount": 129.0, "status": "delivered"}
--- step 3: assistant ---
[assistant -> tool] issue_store_credit({'order_id': 'ORD-100', 'amount': 129.0, 'reason': 'Customer requested store credit for sealed item.'})
--- step 4: tool ---
[tool:issue_store_credit] Store credit of 129.00 issued for ORD-100. Reason: Customer requested store credit for sealed item.
--- step 5: assistant ---
[assistant] Store credit in the amount of $129.00 has been issued for your order ORD-100, as you requested for the sealed item. You can now use this store credit on your next purchase with us.

If you have any further questions or need assistance with anything else, please let me know!

FINAL

Store credit in the amount of $129.00 has been issued for your order ORD-100, as you requested for the sealed item. You can now use this store credit on your next purchase with us.

If you have any further questions or need assistance with anything else, please let me know!
message_count = 6

```

This stack is built from prepackaged middleware, but the logic is still easy to read because each layer addresses a different operational concern. The PII middleware sanitizes sensitive values in user input before they flow deeper into the run. The summarization middleware compresses older conversation history when the thread grows. The fallback middleware gives the agent a backup model if the primary model fails. The call-limit middleware bounds model and tool usage so the agent cannot spin indefinitely or keep calling expensive operations without restraint. None of those policies belong inside the business tools themselves. Middleware is the correct place because the policies apply across the runtime, not inside one domain function.

The human-in-the-loop layer is the clearest example of why middleware is the right abstraction. The agent is allowed to inspect the order without approval, but store credit requires review. When a human-in-the-loop interrupt is reached, execution pauses and the caller receives an interrupt payload describing the pending decision. The application records or displays that decision request, then resumes the same thread with `Command(resume=...)` once the reviewer has approved, rejected, or edited the action. Because the graph is checkpointed, the pause and resume happen inside the same persisted run rather than through ad hoc application state.

This is the pattern that makes middleware feel practical rather than ornamental. The assistant still follows the same model-and-tools loop, but the runtime is now constrained, sanitized, resilient, and reviewable. That is exactly the kind of control surface real systems need once the agent begins to touch sensitive data or execute business actions.

1.7.7 The middleware catalog

Once the basic middleware model is clear, the next practical question is not how hooks work, but which middleware you actually reach for. LangChain provides built-in middleware for common production concerns such as summarization, human review, call limits, fallback models, PII handling, tool selection, retries, context editing, provider-side tool search, filesystem/file search access, shell tools, and subagents. The catalog changes over time, so treat this section as a design map rather than a fixed inventory. A useful way to understand the middleware surface is to group it by responsibility rather than memorize class names alphabetically. Some middleware control context size and memory pressure. Some enforce safety and operational limits. Some improve resilience when models or tools fail. Some add higher-level planning, filesystem-backed working context, or delegation behavior. That grouping is much closer to how you will choose middleware in real systems. You rarely start from the class name. You start from a concrete problem: the thread is getting too long, a tool is too risky, the model may fail, the tool set is too large, or the task needs coordination across several steps.

1.7.7.1 *Middleware for context size, memory pressure, and working context*

`SummarizationMiddleware` is the built-in answer to long conversation history. Its job is to preserve the interaction while compressing older content before the context window becomes a problem. It is a good fit for multi-turn support sessions, research assistants, or any agent that accumulates enough history to threaten token limits. Importantly, it is not the same thing as deleting old messages. The intent is to keep the conversation usable while replacing raw older

history with a shorter summary and preserving a configurable amount of recent detail. Control parameters are as follows:

- `model`: This is the model used to write the summary, not the main agent model unless you choose to make it the same. It can be a model identifier string or a `BaseChatModel` instance. In practice this is the main cost/latency knob for summarization quality versus overhead.
- `trigger`: This controls when summarization runs. It can be expressed as a single threshold, a list of thresholds using OR logic, or a composed trigger clause that combines thresholds with AND logic. Thresholds can be based on token count, message count, or a fraction of the model context window. The important design point is not the exact syntax but the policy: summarization should occur only when the retained conversation context is becoming too large or too noisy for the next model call. Common threshold forms are:
 - `fraction`: a fraction of the model context window, for example `("fraction", 0.8)`
 - `tokens`: an absolute token threshold, for example `("tokens", 4000)`
 - `messages`: a message-count threshold, for example `("messages", 50)`

If `trigger` is omitted, summarization does not run automatically.

- `keep`: This controls how much recent context is preserved after summarization. It must be exactly one `ContextSize` tuple, using one of the same three units:
 - `("fraction", x)`
 - `("tokens", n)`
 - `("messages", n)`

The default is keeping the most recent 20 messages. This is the main continuity knob: too small and you lose conversational precision; too large and you recover less context budget.

- `token_counter`: This is a custom function for counting tokens in the message history. The docs describe the default as approximate / character-based token counting. You override this when you want the middleware to make decisions using a more accurate tokenizer for your chosen model family.
- `summary_prompt`: This is the prompt template used to generate the summary. If you override it, the template must contain a `{messages}` placeholder where the conversation history will be inserted. This is the main behavior-shaping knob for summary style, level of detail, and what gets preserved.
- `trim_tokens_to_summarize`: This sets the maximum amount of history sent into the summarization call itself. Before summarization, LangChain trims the candidate messages to fit this limit. Default is 4000 tokens; passing `None` disables that trimming. This is the main guardrail for summarization-call cost and latency.

`ContextEditingMiddleware` addresses a narrower but very common problem: tool results can be much larger than the natural-language conversation around them. Search results, SQL results, retrieved passages, and generated artifacts can quickly dominate the prompt. Context editing lets the runtime trim or clear older tool-use content while keeping the recent results that still matter. The middleware is configured with one or more edit rules and a token-counting method. A common rule is `ClearToolUsesEdit`, which decides when older tool-call content should be removed or replaced. The middleware owns the editing pipeline; the

edit rule owns the specific cleanup policy. Common `ClearToolUsesEdit` parameters are as follows:

- `trigger`: Token count threshold that activates the edit. (Default: 100000). Once the conversation grows past this size, the middleware starts clearing older tool-use content. In practice, this is the point at which the runtime decides that old tool outputs are now expensive enough to prune.
- `clear_at_least`: Minimum number of tokens to reclaim when the edit runs. (Default: 0). This controls how aggressively the cleanup should reduce the context. When set to 0, the edit is allowed to clear as much as needed to get the context back under control, rather than stopping after a small fixed reduction.
- `Keep`: Number of most recent tool results to preserve. (Default: 3). These newest tool outputs are treated as protected context and will not be removed. The idea is that the most recent tool interactions are usually the ones the model still needs in order to continue the current reasoning or answer correctly.
- `clear_tool_inputs`: Whether to also clear the originating tool-call arguments on the AI message. (Default: `False`). By default, the runtime removes old tool outputs but leaves the original call arguments visible, which preserves some trace of what the model asked the tool to do. When set to `True`, those arguments are also stripped and replaced with empty objects, which saves more space but removes some execution history from the prompt.
- `exclude_tools`: List of tool names to exclude from clearing. (Default: empty sequence `()`). Any tool named here is protected from pruning, even when other tool results are removed. This is useful for tools whose outputs remain important for the rest of the conversation, such as identity lookup, policy retrieval, or other results that the model may need to refer back to repeatedly.
- `Placeholder`: Replacement text inserted where cleared tool output used to be. (Default: `"[cleared]"`). The middleware does not silently delete the old message without a trace; it replaces the removed content with a marker. That helps preserve the conversational structure by showing that a tool result existed there, even though the full payload has been removed from context.

`FilesystemFileSearchMiddleware` adds filesystem search capability in the form of file-oriented lookup tools such as glob-style and grep-style search. This is useful when the agent needs to inspect a working directory, locate candidate files, or search within files without you having to wire those search tools by hand. It is less about long-term memory and more about practical navigation over a local file space. Control parameters are as follows:

- `root_path`: This sets the root directory the middleware is allowed to search, and all file operations are relative to that path.
- `use_ripgrep`: This tells it to use `ripgrep` for content search when available; if not available, it falls back to a Python regex implementation.
- `max_file_size_mb`: Files larger than this size are skipped during search.

`FilesystemMiddleware`, from Deep Agents, goes further. It gives the agent a filesystem-backed working context that can hold short-term and long-term material outside the main conversation buffer. This is especially helpful when an agent produces or consumes variable-

length intermediate artifacts and you do not want those artifacts to live directly in the prompt. It changes the context strategy from “keep everything in messages” to “keep messages concise and store heavier working material in files.” That is a different mode of operation from summarization or context editing, but it solves a related scaling problem.

1.7.7.2 *Middleware for safety, governance, and execution limits*

`HumanInTheLoopMiddleware` is the clearest guardrail in the catalog. It pauses execution before selected tool calls and waits for a human decision. The runtime can approve the call as-is, edit it, or reject it with feedback. This is the right pattern for high-stakes actions such as financial operations, data modification, email sending, or other side-effecting business actions. Because the run must pause safely and resume later, this middleware depends on checkpointed state. Control parameters are as follow:

- `interrupt_on`: Is the per-tool approval policy. It maps tool names to one of three behaviors: `True` means the tool call must pause for review, and the reviewer can, by default, approve, edit, or reject; `False` means the tool runs without interruption; and an `InterruptOnConfig` means you want a custom policy for that specific tool. Any tool not listed in `interrupt_on` is auto-approved, so in practice you usually include only higher-risk tools such as write, delete, payment, email, or scheduling actions.
- `description_prefix`: Is the default text used to label the pending approval request. It affects how the interrupted action is presented to the human reviewer, not whether the action is interrupted. If a tool has its own description in `InterruptOnConfig`, that tool-specific description overrides the prefix. The default value is "Tool execution requires approval".
- Inside `InterruptOnConfig`, `allowed_decisions` restricts what the reviewer is allowed to do for that tool. The supported values are `approve`, `edit`, and `reject`, and you can expose all of them or only a subset. `description` provides custom review text, either as a fixed string or as a callable that builds the text dynamically. Current docs also show `description` explicitly, while the human-in-the-loop guide documents `allowed_decisions`; if you allow edits, the edited action must still be supplied in the expected tool-name-and-args form when execution resumes.

`PIIMiddleware` addresses sensitive data handling. Its role is to detect and then block, redact, mask, or hash personally identifiable information depending on how you configure it. The important architectural point is that this policy sits around the runtime, not inside individual tools. That lets you apply one consistent rule to user input, model output, or tool results instead of scattering PII logic across the rest of the system. Control parameters are as follows:

- `pii_type`: The PII category to detect. Built-in values documented today are `email`, `credit_card`, `ip`, `mac_address`, and `url`, but you can also supply a custom type name. The first positional argument in examples is this value, for example `PIIMiddleware("email", ...)`.
- `strategy`: How the middleware handles matches. Supported values are:
 - `block` → raise an exception
 - `redact` → replace with `[REDACTED_{PII_TYPE}]`
 - `mask` → partially obscure the value
 - `hash` → replace with a deterministic hash

- `detector`: Optional custom detector. If omitted, LangChain uses the built-in detector for the selected `pii_type`. For custom detection, there are three supported forms:
 - `regex string`
 - `compiled regex`
 - `custom function` returning a list of matches with text, start, and end fields
- `apply_to_input`: Whether to inspect user messages before the model call.
- `apply_to_output`: Whether to inspect AI/model messages after the model call.
- `apply_to_tool_results`: Whether to inspect tool-result messages after tool execution.

`ModelCallLimitMiddleware` and `ToolCallLimitMiddleware` are budget controls. The first bounds how many times the runtime is allowed to call the model. The second bounds how many tool calls are allowed, either globally or for a specific tool. Both matter because runaway agents are not only a quality problem. They are also a cost and operational stability problem. The model limit is useful when you want to stop excessive reasoning loops. The tool limit is useful when you want to constrain web search, expensive APIs, or any tool that should not be called repeatedly without restraint.

`ShellToolMiddleware` sits in a more specialized category. It exposes a persistent shell session to the agent. That can be useful for engineering or automation workflows, but it also creates one of the sharpest capability surfaces in the middleware catalog. It should therefore be treated as a deliberately exposed execution environment, not as a casual convenience. In practice, you would only use it in tightly controlled environments where command execution is part of the intended design and where workspace and execution policy are carefully bounded.

1.7.7.3 *Middleware for resilience, fallback, and testing*

`ModelFallbackMiddleware` is the straightforward resilience layer for model availability and redundancy. When the primary model fails, the runtime can fall back to one or more alternatives. That is useful both for reliability and for cost strategy, since the fallback does not have to be equivalent in price or provider. The main idea is not sophistication. It is continuity: the runtime should have a second path when the first model is unavailable or unsuitable. Control parameters are as follow:

- `first_model`: the first fallback model to try when the primary model fails.
- `*additional_models`: any further fallback models, tried in the order you pass them.

`ToolRetryMiddleware` and `ModelRetryMiddleware` handle transient failure by retrying tool calls or model calls with bounded backoff behavior. These are not universal “fix everything” switches. They make sense only where retry is actually appropriate. A flaky external lookup may deserve retry. A business-rule refusal or a mutating operation without replay safety does not. The existence of separate retry middleware for tools and models is useful because those are different failure surfaces and often need different retry policies. Common control parameters are:

- `backoff_factor`: multiplier for exponential backoff.
- `initial_delay`: delay before the first retry, in seconds.
- `max_delay`: upper bound on retry delay growth.

- `jitter`: whether to add random jitter, documented as $\pm 25\%$, to avoid synchronized retry spikes.

Parameters specific to `ToolRetryMiddleware`:

- `max_retries`: how many retry attempts after the initial tool call. Default 2.
- `tools`: optional allowlist of tools to which retry applies. You can pass tool names or `BaseTool` instances; if `None`, it applies to all tools.
- `retry_on`: either a tuple of exception types or a predicate callable taking an exception and returning `True/False`. Default is all Exception.
- `on_failure`: what happens after retries are exhausted. The current retry middleware API uses `continue` to return an error message into the agent loop and `error` to re-raise the exception and stop execution. A custom callable can format the error message. Some documentation and older examples may still show the older names `return_message` and `raise`; treat those as compatibility aliases and prefer to `continue` and `error` in new code.

Controls parameters specific to `ModelRetryMiddleware`:

- `max_retries`: retry attempts after the initial model call. Default 2.
- `retry_on`: tuple of exception types or predicate callable. Default is all Exception.
- `on_failure`: `continue`, `error`, or a custom callable. `Continue` returns an `AIMessage` carrying error details, `error` re-raises.

`LLMToolEmulator` serves a different purpose. Instead of executing real tools, it lets the runtime emulate tool execution with a model. This is primarily useful in testing, development, and simulation scenarios where you want to exercise the agent loop without granting access to real external side effects. It is best understood as a harness or stand-in, not as a substitute for deterministic production tools.

1.7.7.4 *Middleware for planning, selection, and delegation*

`ToDoListMiddleware` adds explicit task planning and tracking behavior. Rather than expecting the agent to hold a multi-step plan only inside transient model state, it equips the runtime with a to-do mechanism and accompanying guidance so the agent can plan and track progress across a more complex task. This becomes useful when the agent has to coordinate several tools or stages and when progress visibility matters as much as the final answer.

`LLMToolSelectorMiddleware` is aimed at large tool surfaces. Instead of handing every available tool directly to the main model call, it uses a model to reduce the candidate set first. This is useful when the full tool catalog is large enough to create prompt bloat or tool-selection confusion. Conceptually, it acts as a pre-selection stage that narrows the visible tool surface before the main reasoning step.

`SubAgentMiddleware`, from *Deep Agents*, extends this idea from tool selection to delegation. It allows the main agent to spawn subagents and hand off work to more specialized runtimes. This is not just a bigger tool list. It is a different orchestration move: instead of selecting one tool, the main agent can isolate a subtask in another agent context and receive a concise result back. That is especially valuable when the main problem benefits from context isolation or specialist behavior rather than from one flat agent loop with many tools.

1.7.7.5 *The custom middleware surface*

The built-in catalog covers many common needs, but it is not meant to cover every policy you will ever want. That is why the middleware surface also includes a custom API. At the broadest level, you can provide a full `AgentMiddleware` subclass when you want several related hooks to live together, especially if they share custom state. This is the right choice when the middleware is substantial enough to deserve its own runtime component. For lighter-weight customization, the decorator-based hooks are often simpler. `@before_agent` and `@after_agent` are lifecycle hooks that run once per invocation. `@before_model` and `@after_model` run around each model step and are good places for validation, logging, counters, or early termination logic. These hooks are node-style hooks, which means they operate at fixed points in the lifecycle and typically return state updates rather than replacing the call path itself. `@wrap_model_call` and `@wrap_tool_call` are the control hooks. They run around the actual model or tool boundary and can therefore do more than observe. They can retry, replace, filter, short-circuit, or enrich the call. This is where dynamic model selection, dynamic tool filtering, custom retries, and tool-error interception usually belong. The wrap hooks are the right fit whenever the policy has to change what is executed rather than merely record what happened. `@dynamic_prompt` is the most specialized decorator in the set. Its job is narrower but extremely useful: change the system instruction at call time from state or runtime context. In practice, it is often the cleanest way to adapt tone, audience, operating rules, or step-specific behavior without branching into several near-duplicate agent definitions. Although the syntax is simple, it belongs in the same middleware mental model as the other hooks because it is still runtime shaping of the standard loop.

1.7.7.6 *How to choose without turning the agent into a patchwork*

A good working rule is to use built-in middleware whenever the problem is generic and operational. Summarization, human approval, limits, fallback, retry, and PII handling are standard enough that the built-in components are usually the right starting point. Reach for custom middleware when the policy is specific to your business rules, your runtime context, or your notion of what the agent should see at a particular step. Reach for the Deep Agents middleware only when you genuinely need filesystem-backed working context or true subagent delegation rather than a larger ordinary tool set.

Seen this way, the middleware surface is not just a list of classes. It is a set of control layers around one standard runtime. Some layers control memory pressure, some enforce safety, some improve resilience, some add planning structure, and some let you write your own policies. That is why middleware occupies such a central place in `create_agent()`: it is the mechanism that turns a generic tool-using loop into an agent runtime shaped for a real application.

1.7.8 **State, runtime context, store, and checkpoints**

Short-term state is the working memory of the current thread. By default, LangChain agents use `AgentState`, which includes the messages sequence. You can extend that state when the agent needs additional fields. In current LangChain v1 agent patterns, use the documented `state_schema` extension points for short-term agent state and `context_schema` for typed runtime context. Keep the distinction clear: state is part of the agent's evolving execution data, while context carries trusted runtime information such as user identity, request metadata,

service handles, or configuration that should not be exposed as model-generated tool arguments. Define state in middleware when the state belongs conceptually to that middleware and use `state_schema` on `create_agent()` when tools or middleware need direct access to extra thread state.

Runtime context is different. Context is static for a single invocation. It carries the trusted values discussed earlier: authenticated user IDs, permissions, service handles, and request-specific metadata. In current v1-style code, new applications are expected to pass such data through the `context=` argument rather than overloading `config["configurable"]`, although the older pattern still exists for backward compatibility.

Long-term memory lives in a store. This is shared across sessions and threads, and tools reach it through `ToolRuntime`. The store is not the same thing as the checkpointer. The store is for durable application memory. The checkpointer is for thread-scoped execution state and recovery. That distinction matters a great deal in production design. The example below shows the three layers working together. The context identifies the current user. The store holds durable customer data. The checkpointer preserves thread state between turns.

```
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.tools import tool, ToolRuntime
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.store.memory import InMemoryStore
from .config import OPENAI_API_KEY

store = InMemoryStore()
store.put(("users",), "user_123", {"preferred_channel": "sms"})

@dataclass
class Context:
    user_id: str

@tool
def get_customer_profile(runtime: ToolRuntime[Context]) -> str:
    """Look up durable customer preferences."""
    user_info = runtime.store.get(("users",), runtime.context.user_id)
    return str(user_info.value) if user_info else "Unknown user"

agent = create_agent(
    model = "openai:gpt-5-mini",
    tools = [get_customer_profile],
    store = store,
    context_schema = Context,
    checkpointer = InMemorySaver()
)

config = {"configurable": {"thread id": "support-thread-1"}}
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "What contact channel does this customer prefer?"
            }
        ]
    },
    config = config,
    context = Context(user_id = "user_123")
)

print(result["messages"][-1].content)
```

vol3_ch_01\src\middleware_store.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.middleware_store
The customer's preferred contact channel is SMS (text message).
```

This code wires a store-backed agent so customer-specific data lives outside the model and can be retrieved through the runtime at execution time. The important design choice is that the agent is created with a shared store and a typed `context_schema`, which means tool execution can access durable application state and request-scoped identity without exposing those control values as model-chosen arguments. Before the agent is created, the store is seeded with a user record under the "users" namespace. That matters because the tool does not infer or guess who the customer is. Instead, `get_customer_profile` reads the current user identity from `runtime.context.user_id` and uses `runtime.store.get(...)` to fetch the corresponding profile. In other words, the model can decide to call the tool, but the runtime decides which customer record is actually in scope. When `create_agent(...)` receives both `store=store` and `context_schema=Context`, it binds the agent to this execution model: tools can read shared durable state, and invocations must supply context in the expected typed shape. The `checkpoint` complements this by preserving thread state separately from the store, so conversation progress and long-lived business data remain distinct concerns. At invocation time, `thread_id` identifies the conversation, while `Context(user_id="user_123")` identifies the customer whose stored preference the tool should read.

1.7.8.1 *Extra short-term state,*

When an agent needs a small amount of extra short-term state, you define it by extending `AgentState` and passing the new type as `state_schema` to `create_agent()`. In the example below, `customer_id` is required for every run, while `current_step` is optional and can be added only when the workflow needs it. This gives the agent a place to keep operational data such as which customer is currently in scope and which stage of the workflow is in progress.

```
from typing_extensions import NotRequired
from langchain.agents import AgentState

class SupportState(AgentState):
    customer_id: str
    current_step: NotRequired[str]
```

You then pass this schema to the agent:

```
agent = create_agent(
    model = ...,
    tools = [...],
    state_schema = SupportState
)
```

The key point is that these fields become part of the agent's runtime state for the thread, alongside the message history already managed by `AgentState`. Tools and middleware can read that state during execution, which is often cleaner than forcing the model to restate the same values in each turn. Keep this separate from runtime context: state evolves during the thread, while context is supplied by the application for the invocation and is the right place for trusted dependencies, user identity, service handles, request metadata, and configuration.

1.7.9 Structured output with `create_agent()`

Structured output is one of the areas where `create_agent()` does more than simply run the tool loop. The `response_format` parameter lets the agent return validated data instead of

only natural-language text. It can accept an explicit `ProviderStrategy`, an explicit `ToolStrategy`, a schema type, or `None`. When a schema is passed directly, LangChain can select the appropriate strategy: provider-native structured output where the model supports it, otherwise tool-based structured output. The final parsed object is returned under `structured_response` in the final agent state. This matters for two reasons. First, it changes the application boundary. The caller no longer has to scrape facts out of free-form prose. Second, it keeps the agent loop intact. You do not need a separate parsing pipeline after the agent finishes. The final response contract is part of the agent itself. One practical caveat still matters: if the agent uses tools and structured output at the same time, the selected model must support both capabilities together. A support example makes the idea concrete:

```
from __future__ import annotations
from typing import Any
from pydantic import BaseModel, Field
from langchain.agents import create_agent
from langchain.tools import tool
from .config import OPENAI_API_KEY

ORDERS = {
    .
    .
}

REFUND_POLICY = {
    .
    .
}

class RefundDecision(BaseModel):
    order_id: str = Field(description = "Order identifier")
    eligible: bool = Field(description = "Whether the order is refundable")
    reason: str = Field(description = "Short business explanation")

@tool
def lookup_order(order_id: str) -> dict[str, Any]:
    """Return order facts needed for a refund decision."""
    order = ORDERS.get(order_id)
    if order is None:
        return {"found": False, "order_id": order_id}
    return {"found": True, "order_id": order_id, **order}

@tool
def evaluate_refund(
    days_since_delivery: int,
    opened: bool,
    max_days: int,
    opened_items_allowed: bool,
    order_id: str
) -> dict[str, Any]:
    """Evaluate refund eligibility from order facts and policy facts."""
    eligible = (
        days_since_delivery <= max_days
        and (opened_items_allowed or not opened)
    )

    if eligible:
        reason = "The order is within the refund window and meets the item-condition rule."
    elif days_since_delivery > max_days:
        reason = "The order is outside the refund window."
    else:
        reason = "Opened items are not eligible under the standard policy."

    return {
        "order_id": order_id,
        "eligible": eligible,
        "reason": reason
    }

agent = create_agent(
    model = "openai:gpt-5",
    tools = [lookup_order, evaluate_refund],
    response_format = RefundDecision,
    system_prompt = (
        "You are a refund decision assistant. "
```

```

        "Always look up the order first. "
        "Then evaluate refund eligibility using this policy: "
        f"max_days={REFUND_POLICY['max_days']}, "
        f"opened items allowed={REFUND_POLICY['opened items allowed']}. "
        "Return the final answer only as structured output."
    )
)
def main() -> None:
    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": "Decide whether ORD-100 is refundable."
                }
            ]
        }
    )
    structured = result["structured_response"]

    print("STRUCTURED RESPONSE")
    print(structured)
    print()

    print("AS DICT")
    print(structured.model_dump())

if __name__ == "__main__":
    main()

```

vol3_ch_01\src\middleware_response.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.middleware_response

STRUCTURED RESPONSE
order_id='ORD-100' eligible=True reason='The order is within the refund window and meets
the item-condition rule.'

AS DICT
{'order_id': 'ORD-100', 'eligible': True, 'reason': 'The order is within the refund
window and meets the item-condition rule.'}

```

This is the kind of pattern that becomes extremely useful when an agent is part of a wider system rather than a chat window. A user interface may still display natural language, but downstream code can consume the structured decision directly.

1.7.10 Streaming

Because `create_agent()` returns a graph-based runtime, it supports the same streaming model as LangGraph. That means you are not restricted to waiting for one final answer. You can stream state updates, full state values, token chunks, custom progress signals, checkpoint events, task events, and detailed debug traces. In practice, the most useful pair for interactive applications is usually updates plus messages: state progress and token-level text in the same flow. A minimal streaming loop looks like this:

```

from langchain.agents import create_agent
from langchain.tools import tool
from .config import OPENAI_API_KEY

ORDERS = {
    .
    .
}

@tool
def lookup_order(order_id: str) -> str:
    """Return the order details needed to assess refund eligibility."""
    order = ORDERS.get(order_id)
    if not order:
        return f"Order {order_id} was not found."
    return (

```

```

        f"Order {order_id}: status={order['status']}, "
        f"days_since_delivery={order['days_since_delivery']}, "
        f"item_opened={order['item_opened']}, "
        f"amount={order['amount']}"
    )
    agent = create_agent(
        model = "openai:gpt-5-mini",
        tools = [lookup_order],
        system_prompt = (
            "You are a customer support assistant. "
            "Use the tool when order data is needed. "
            "Answer clearly and briefly."
        )
    )
    for mode, data in agent.stream(
        {
            "messages": [
                {"role": "user", "content": "Can order ORD-100 be refunded?"}
            ],
        },
        stream_mode = ["updates", "messages"],
        version = "v2"
    ):
        if mode == "updates":
            print("\nSTATE UPDATE:")
            print(data)

        elif mode == "messages":
            message_chunk, metadata = data
            if getattr(message_chunk, "content", None):
                print(message_chunk.content, end = "", flush = True)

```

vol3_ch_01\scr\agent_stream.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_01> python -m src.agent_stream

STATE UPDATE:
{'model': {'messages': [AIMessage(content='', additional_kwargs={},
response_metadata={'finish_reason': 'tool calls', 'model_name': 'gpt-5-mini-2025-08-07',
'service_tier': 'default', 'model_provider': 'openai'}, id='lc_run--019cec84-b710-7832-84fe-cfdbb75fd8f3', tool_calls=[{'name': 'lookup_order', 'args': {'order_id': 'ORD-100'},
'id': 'call_kmo8rZ5Jaj2zAsALCsZhmlDP', 'type': 'tool_call'}], invalid_tool_calls=[],
usage_metadata={'input_tokens': 161, 'output_tokens': 26, 'total_tokens': 187,
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}})]}}
Order ORD-100: status=delivered, days_since_delivery=12, item_opened=False, amount=129.0
STATE UPDATE:
{'tools': {'messages': [ToolMessage(content='Order ORD-100: status=delivered,
days_since_delivery=12, item_opened=False, amount=129.0', name='lookup_order',
id='aa807f45-3d44-436b-9526-fc397ac44a81',
tool_call_id='call_kmo8rZ5Jaj2zAsALCsZhmlDP')]}
Yes - it appears eligible. ORD-100 was delivered 12 days ago and the item is marked
unopened. The order total is $129.00.

Would you like me to start the return and refund now? If so, I can generate a return
label and initiate the refund to your original payment method.
STATE UPDATE:
{'model': {'messages': [AIMessage(content='Yes - it appears eligible. ORD-100 was
delivered 12 days ago and the item is marked unopened. The order total is
$129.00.\n\nWould you like me to start the return and refund now? If so, I can generate a
return label and initiate the refund to your original payment method.',
additional_kwargs={}, response_metadata={'finish_reason': 'stop', 'model_name': 'gpt-5-
mini-2025-08-07', 'service_tier': 'default', 'model_provider': 'openai'}, id='lc_run--
019cec84-be7e-7a30-8529-355745b5039c', tool_calls=[], invalid_tool_calls=[],
usage_metadata={'input_tokens': 218, 'output_tokens': 583, 'total_tokens': 801,
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 512}})]}}

```

This code uses streaming to expose the agent's execution as it happens rather than hiding the tool loop behind a single final response. The important design choice is the call to `agent.stream(...)` with `stream_mode=["updates", "messages"]`. That tells the runtime to emit two different signal types in the same run: progress events from the agent workflow and incremental text generated by the model. The user asks whether an order can

be refunded, but the model does not answer immediately. Because the prompt instructs it to use tools when order data is needed, the agent first calls `lookup_order`. That intermediate work appears through the "updates" branch. Printing those updates makes the internal progress visible: the reader can see when the model requests the tool and when the tool result returns to the runtime. The "messages" branch handles a different concern. Here, the code receives `message_chunk` objects and prints their `content` as soon as each chunk arrives. This is what makes the final answer appear progressively on screen instead of waiting for the whole sentence or paragraph to finish. The `flush=True` call matters because it forces immediate console output, preserving the live feel of the stream.

1.7.11 Where `create_agent()` stops being the right abstraction

The best use case for `create_agent()` is a standard model-and-tools loop that still needs serious operational features: middleware, persistence, runtime context, structured output, interrupts, and streaming. Many support assistants, e-commerce assistants, internal copilots, and light workflow agents fit there well. The abstraction becomes less ideal when the workflow itself is no longer standard. If you need bespoke branching, explicit parallel paths, custom state channels, nontrivial subgraph composition, or orchestration that should not look like a normal tool loop, raw LangGraph is the better fit. There is also an upper boundary. If the problem requires heavier planning, coding-agent behavior, filesystem-backed working context, or specialized higher-level agent architecture, use a more specialized abstraction. The practical rule is simple: reach for `create_agent()` when the workflow is fundamentally a configurable tool-using agent loop; step down to raw LangGraph when the graph itself is the product; step up to specialized abstractions when the agent no longer behaves like a standard tool-using runtime.

1.8 SUMMARY

This chapter moved tool use from a convenient LLM feature to its proper place in system architecture: a controlled boundary between probabilistic model decisions and deterministic software execution. A model may decide that an action is useful and generate a structured request for it, but the application runtime remains responsible for deciding whether that request is valid, authorized, safe, and executable. That distinction is the foundation of reliable tool-using agents.

We started by treating tools as APIs with explicit contracts. Names and descriptions help the model select the right capability, while schemas constrain the arguments it can provide. Explicit schemas become increasingly important as business impact increases because they allow field descriptions, enums, patterns, ranges, and other validation rules to become part of the contract. Just as importantly, trusted values such as authenticated identities, tenant identifiers, permissions, service clients, and execution metadata should come from runtime-controlled context rather than from model-generated arguments. Query tools and command tools also need to be distinguished because commands change external state and therefore require stronger controls, particularly authorization, auditing, and idempotency. The chapter then examined the main ways LangChain exposes tools. A normal Python function can become a tool through `@tool`, an existing Runnable or LCEL pipeline can be exposed through `as_tool()`, and `StructuredTool` provides more explicit control over the contract and implementation. These authoring styles differ, but they converge on the same runtime abstraction: a named capability with a description, an input schema, and a controlled invocation path. The correct abstraction therefore depends mainly on where the capability already exists and how explicit its contract needs to be. Built-in tool families were examined according to the source of truth they provide. Search tools are appropriate for fresh public information, database tools for structured systems of record, HTTP tools for external services and APIs, code-execution tools for procedural or analytical tasks, and calculator-style tools for narrow deterministic computation. The important design principle is not to expose the most powerful tool available, but the narrowest capability that can solve the task correctly.

Custom tools require another layer of engineering. Schema validation checks whether an input is structurally valid, while runtime validation determines whether the requested operation is meaningful and permitted in the current business context. Retries should be reserved for failures that are genuinely transient and safe to repeat. Timeouts prevent external calls from blocking indefinitely, while circuit breakers can protect unhealthy downstream dependencies from repeated traffic. Side-effecting operations must also be designed for retries and replay through idempotency so that a timeout, resume, or duplicate invocation cannot accidentally create multiple refunds, orders, notifications, or trades. Error handling follows the same principle of explicit ownership. Recoverable system failures may be retried or handled through a fallback. Some business errors can be returned to the model so it can reformulate the call. Missing information or approval may require human intervention. Programming defects and unsafe conditions should fail clearly rather than being disguised as ordinary tool results. The useful question is therefore not simply whether a tool failed, but who can correct the failure and whether execution should continue. All retry, repair, and agent loops should remain bounded. With those foundations in place, the chapter introduced the standard tool-

calling loop and its relationship to ReAct. The model reasons over the current state, requests one or more actions, the runtime executes those actions, the results are returned as observations, and the model decides what to do next. Modern implementations normally exchange structured tool calls and tool results instead of parsing textual `Thought`, `Action`, and `Observation` traces, but the underlying feedback loop is the same. You implemented that loop directly, then saw how LangGraph makes the model step, tool-execution step, state, and stop condition explicit through nodes and edges.

Finally, the chapter showed how `create_agent()` packages the same fundamental loop into LangChain's higher-level agent runtime. The important capability is not merely that it saves boilerplate. It provides a place to combine tools with runtime context, custom state, persistent checkpoints, long-term stores, middleware, human-in-the-loop controls, call limits, retry and fallback policies, context management, structured responses, and streaming. These facilities allow operational policies to surround the agent without contaminating individual business tools with unrelated concerns. The central design lesson is therefore straightforward: the model proposes; the runtime governs and executes. Good tool-using systems keep that boundary visible. They expose narrow capabilities, validate before execution, protect side effects, classify failures deliberately, preserve state where necessary, and make important operations inspectable and controllable. Use `create_agent()` when the problem is fundamentally a configurable model-and-tools loop. Move to explicit LangGraph when the workflow itself requires bespoke branching, parallel paths, specialized state, or custom orchestration. Tool use becomes production-ready not when the model is given more autonomy, but when the surrounding runtime gives that autonomy precise boundaries.

2 CONTROLLED GENERATION, TOOLNODE, AND MCP

This chapter is about the boundary between probabilistic model output and deterministic software execution. A model can produce text, structured data, or a tool-call request, but each form carries a different kind of guarantee. Production systems become safer when those guarantees are named precisely. JSON mode gives a serialization boundary. Structured output gives a schema boundary. Function calling gives an action-request boundary. `ToolNode` gives an execution boundary inside LangGraph. MCP gives an interoperability boundary between hosts and capability providers.

The chapter follows that progression from output shape to executable capability. First, you will see how controlled generation turns vague model output into JSON or schema-constrained data. Then you will see why function calling should be treated as a structured proposal for action, not as execution authority. From there, the focus moves into LangGraph: `ToolNode` is the point where a proposed tool call becomes actual program execution and where tool results return to the graph as messages. Finally, MCP shows how the same capability boundary can extend beyond the local process, allowing tools, resources, and prompts to be exposed and invoked through a standard protocol.

The central lesson is that controlled generation improves structure, but real reliability still comes from validation, state management, authorization, observability, and disciplined execution boundaries.

2.1 CONTROLLED GENERATION & FUNCTION CALLING:

WHAT GUARANTEES YOU REALLY GET

In agentic systems, controlled generation does not mean the model has become deterministic, omniscient, or intrinsically trustworthy. It means the application has narrowed the range of outputs the model is allowed to produce. Sometimes that means forcing the output into a specific JSON shape. Sometimes it means requiring the model to emit a tool call instead of plain prose. Sometimes it means constraining the model to one tool, or to one tool from a small, allowed set. These controls are extremely useful because they replace vague parsing and hopeful prompting with explicit contracts. They do not, however, guarantee that the content is correct in business terms, factually grounded, or safe to execute without validation.

That distinction is the most important one in this chapter. The strongest guarantees in this space are mostly structural. A constrained result may be valid JSON and still contain the wrong values. A schema-valid object may still be based on stale data. A perfectly formed tool call may still ask for the wrong action. Once this is clear, the rest of the landscape becomes easier to understand. Structured output solves output-shape problems. Function calling solves action-shape problems. `ToolNode` solves execution-boundary problems inside a graph. MCP solves interoperability problems between hosts and capability providers. Each layer is valuable, but each layer guarantees something different.

2.1.1 The guarantee ladder: from text to schemas to tool calls

It helps to think of controlled generation as a ladder of contracts, where each step reduces ambiguity at the boundary between the model and the rest of the application. At the lowest step is ordinary text generation. The model can answer in any form it finds plausible: a paragraph, a list, a mixed explanation, or a half-structured reply that only looks machine-readable at first glance. That flexibility is useful when the output is meant only for a human reader, but it becomes fragile as soon as another part of the system needs to consume the result. A UI renderer, a validator, or a downstream service does not want “something that looks about right.” It needs a stable shape. With plain text, the burden of interpretation falls entirely on the application, which usually means extra parsing logic, edge cases, and failure modes that are difficult to predict.

One step higher is JSON mode. This is already a meaningful improvement because the model is no longer free to reply in arbitrary prose. The output must be syntactically valid JSON. That removes a whole class of formatting problems, but it still leaves an important gap. Valid JSON is not the same thing as the right JSON. The model may still return fields with the wrong names, omit required keys, use the wrong types, or include extra structure the application does not expect. In other words, JSON mode gives you a serialization guarantee, not a full contract for application data. That is why it can be perfectly adequate for lightweight use cases and still too weak for systems that depend on exact fields, exact types, and explicit constraints.

Above JSON mode is schema-constrained structured output. Here the goal is no longer simply to make the response parseable. The goal is to make it conform to a declared schema. That is a much stronger promise, because the application is no longer asking, “Did I get JSON?” but “Did I get an object that matches the contract I asked for?” In LangChain, this distinction

appears in two places. At the agent layer, a schema passed as the response format can be handled either through provider-native structured output or through a tool-based strategy, and LangChain can choose the strategy when a schema type is supplied directly. At the model-wrapper layer, you can be more explicit by selecting a method such as JSON schema, function calling, or JSON mode when the integration supports it. The provider-native route is generally the strongest option when the selected model supports it, because enforcement happens closer to generation rather than only after text has been produced.

This is the point where the contract becomes genuinely useful for production software. A schema can say not only that a field exists, but also what type it must have, which values are allowed, which fields are required, and which output objects should be rejected. That does not make the answer correct in a business sense, but it does make the output predictable enough to pass safely into deterministic code. The difference matters. A well-typed object can still express the wrong decision, but at least the application is no longer guessing what the model meant.

Function calling belongs beside structured output rather than above it, because it solves a different problem. Structured output is about constraining the shape of an answer. Function calling is about constraining the shape of an action proposal. The model is not being asked to return the final business object. It is being asked to produce a request to invoke some external capability, together with arguments that fit a declared schema. That is a powerful form of control because it turns vague intent into a machine-usable action request. Even so, the guarantee is still limited to form. A function call can be perfectly well formed and still be the wrong thing to do. The model may choose the wrong tool, provide unhelpful arguments, or request an action that should not be executed in the current business context. So function calling improves control over how an action is proposed, but it does not transfer execution authority or judgment from the application to the model.

That is why this ladder should be read as a progression in contract strength, not as a progression toward correctness. Plain text gives almost no contract. JSON mode gives a formatting contract. Schema-constrained output gives a data contract. Function calling gives an action-request contract. Each step is stronger than the one below it, but none of them, by itself, guarantees that the content is true, the interpretation is sound, or the proposed action is wise. Those guarantees must still come from validation, policy, and the rest of the system around the model. The same distinction can be summarized as follows:

Layer	What it constrains	What it does not guarantee
JSON mode	JSON syntax	Correct fields, types, or business meaning
Structured output	Output schema	Correct interpretation or domain truth
Function calling	Tool name and argument shape	Authorization, safety, or execution validity
ToolNode	Graph-side tool execution	Planning, policy, or business approval
MCP	Capability discovery and invocation protocol	Correct tool design, trust, or runtime behavior

2.1.2 Structured output: the strongest guarantee for final answers

When the application needs a final answer in a predictable machine-readable form, structured output is usually the best fit. This is especially true for classification, extraction, routing decisions, and UI payload assembly. In those cases, the application is not asking the model to perform an external action. It is asking the model to commit to a typed result. A recurring e-commerce pattern makes this concrete. Suppose the user writes, “Show me waterproof hiking jackets under 180 euros, dark colours only, and not for winter extremes.” A chat-style answer is not the most useful first output. What the application often needs first is a typed search intent object: category, max price, excluded conditions, and colour preferences. That object can then drive catalog retrieval and filtering without brittle string parsing. With LangChain’s model wrappers, the structured-output call can be made directly on the model. On supported OpenAI-backed models, `method="json_schema"` engages provider-native structured output rather than falling back to tool calling. On unsupported models, choose an alternative method or use a higher-level agent strategy that can fall back to tool-based structured output.

```
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from .config import OPENAI_API_KEY

class CatalogIntent(BaseModel):
    category: str = Field(description = "Primary catalog category")
    max_price_eur: int = Field(description = "Maximum acceptable price in euros")
    colors: list[str] = Field(description = "Preferred colors")
    must_be_waterproof: bool = Field(description = "Whether waterproofing is required")
    exclude_use_case: str = Field(description = "Conditions the product should not target")

llm = ChatOpenAI(model = "gpt-4.1")

intent_llm = llm.with_structured_output(
    CatalogIntent,
    method = "json_schema"
)

intent = intent_llm.invoke(
    "Show me waterproof hiking jackets under 180 euros, dark colors only, "
    "and not for winter extremes."
)

print(intent.model_dump())
```

vol3_ch_02\src\structured_output.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_02> python -m src.structured_output
{'category': 'hiking jackets', 'max_price_eur': 180, 'colors': ['dark'],
'must_be_waterproof': True, 'exclude_use_case': 'winter extremes'}
```

The important line in this example is the creation of `intent_llm`:

```
llm.with_structured_output(CatalogIntent, method = "json_schema")
```

This call does not simply add a formatting instruction to the prompt. It creates a new runnable model wrapper whose output contract is defined by the `CatalogIntent` Pydantic class. LangChain reads the fields, types, and field descriptions from the Pydantic model and converts them into a schema that the model provider can use to constrain the response. In this case, `category` must be a string, `max_price_eur` must be an integer, `colors` must be a list of strings and `must_be_waterproof` must be a boolean. The field descriptions are also useful: they tell the model what each field means, so they should be written as part of the contract, not as decorative comments. The `method = "json_schema"` argument selects OpenAI’s provider-native structured-output mechanism. Instead of asking the model to

produce ordinary prose and then hoping that a parser can recover a valid object, the request is sent with a JSON schema derived from `CatalogIntent`. The provider then constrains the generated answer to match that schema. LangChain receives the structured response, parses it, and validates it against the Pydantic model. Because the schema was supplied as a Pydantic class, the value returned by `intent_llm.invoke(...)` is a `CatalogIntent` instance, not a plain string. That is why the example can call `intent.model_dump()` and obtain a normal Python dictionary ready for application code.

The result is a clean boundary between model interpretation and deterministic execution. The model is responsible for turning the user's natural-language request into a typed intent object. The application is responsible for using that object to query the catalog, apply filters, enforce business rules, and render the user interface. No downstream component has to scrape constraints out of a chat response or search for phrases such as "under 180 euros" in generated text. The guarantee is structural, not absolute semantic truth. The provider can enforce that the response has the required fields and compatible types, and Pydantic can validate the returned object. That does not prove that the model interpreted the user perfectly. A schema-valid value such as `max_price_eur = 180` is reliable as structure; it is not automatically reliable as understanding. For ambiguous, high-value, or safety-sensitive decisions, the typed output should still be checked by deterministic validation rules, user confirmation, or domain-specific business logic. This distinction also explains why provider-native structured output is usually preferable when the model supports it. With `method = "json_schema"`, schema enforcement happens at the provider level. Other structured-output strategies, such as tool-calling-based output or JSON-mode prompting, remain useful for portability, but they do not all provide the same enforcement semantics. The practical rule is to use provider-native structured output when available; use tool-based or parser-based alternatives when you need compatibility across models; in all cases, treat the schema as the interface between probabilistic interpretation and deterministic application code.

2.1.3 Function calling: a proposal for action, not execution authority

Function calling is often described casually as if the model "uses a tool." That wording is convenient, but it hides the real architecture. The model does not execute your business logic. It emits a structured request that names a tool and provides arguments. Your application, runtime, or agent loop then decides whether to execute that request, how to execute it, and what to do with the result. That distinction matters as soon as a tool can affect external state. In a trading assistant, a function call such as `screen_equities` is read-oriented: it asks the application to retrieve or compute data. A function call such as `submit_order` is different: it proposes an operation with real consequences. Even if the proposed arguments match the declared schema, the order should not be placed automatically. The runtime still needs business validation, permission checks, risk controls, account checks, market-hours checks, audit logging, and execution policy. Function calling therefore creates a contract boundary. The model may convert a natural-language request into a machine-readable tool-call request. The application remains responsible for deciding whether that request is valid, authorized, safe, and executable.

```
from pydantic import BaseModel, Field
from typing import Literal
from langchain_openai import ChatOpenAI
```

```

class ScreenEquities(BaseModel):
    """Screen equities that match a simple strategy filter."""
    sector: str = Field(description = "Sector to filter on")
    min_market_cap_usd_billion: int = Field(
        description = "Minimum market cap in billions"
    )
    style: Literal["value", "growth", "quality"] = Field(
        description = "Investment style"
    )

class SubmitOrder(BaseModel):
    """Submit a proposed equity order for later validation and execution."""
    symbol: str = Field(description = "Ticker symbol")
    side: Literal["buy", "sell"] = Field(description = "Order side")
    quantity: int = Field(description = "Number of shares")
    order_type: Literal["market", "limit"] = Field(description = "Order type")
    limit_price_usd: float | None = Field(
        description = "Limit price in USD. Use null for market orders."
    )

llm = ChatOpenAI(model = "gpt-4.1")
tools = [ScreenEquities, SubmitOrder]

def show(label: str, message) -> None:
    print(f"\n--- {label} ---")
    print("content:", repr(message.content))
    print("tool_calls:", message.tool_calls)

auto_llm = llm.bind_tools(
    tools,
    strict = True,
    tool_choice = "auto",
    parallel_tool_calls = False
)

show(
    "auto: tool call when useful",
    auto_llm.invoke("Find quality technology stocks above 20 billion market cap.")
)

show(
    "auto: direct answer when no tool is needed",
    auto_llm.invoke("Explain market capitalization in one sentence.")
)

no_tool_llm = llm.bind_tools(
    tools,
    strict = True,
    tool_choice = "none"
)

show(
    "none: tools disabled for this call",
    no_tool_llm.invoke("Find quality technology stocks above 20 billion market cap.")
)

required_llm = llm.bind_tools(
    tools,
    strict = True,
    tool_choice = "required",
    parallel_tool_calls = False
)

show(
    "required: at least one tool call",
    required_llm.invoke(
        "Find quality technology stocks above 20 billion market cap."
    )
)

forced_screen_llm = llm.bind_tools(
    tools,
    strict = True,
    tool_choice = "ScreenEquities",
    parallel_tool_calls = False
)

show(
    "forced: specific tool",
    forced_screen_llm.invoke(
        "Find quality technology stocks above 20 billion market cap."
    )
)

```

```
)
)
```

vol3_ch_02\scr\bind_tool.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_02> python -m src.bind_tool

--- auto: tool call when useful ---
content: ''
tool_calls: [{'name': 'ScreenEquities', 'args': {'sector': 'Technology',
'min_market_cap_usd_billion': 20, 'style': 'quality'}, 'id': 'call_...', 'type':
'tool_call'}]

--- auto: direct answer when no tool is needed ---
content: 'Market capitalization is the total market value of a company's outstanding
shares, calculated by multiplying the share price by the number of shares outstanding.'
tool_calls: []

--- none: tools disabled for this call ---
content: 'I can describe how to screen for quality technology stocks above a given
market-cap threshold, but tool use is disabled for this call.'
tool_calls: []

--- required: at least one tool call ---
content: ''
tool_calls: [{'name': 'ScreenEquities', 'args': {'sector': 'Technology',
'min_market_cap_usd_billion': 20, 'style': 'quality'}, 'id': 'call_...', 'type':
'tool_call'}]

--- forced: specific tool ---
content: ''
tool_calls: [{'name': 'ScreenEquities', 'args': {'sector': 'Technology',
'min_market_cap_usd_billion': 20, 'style': 'quality'}, 'id': 'call_...', 'type':
'tool_call'}]
```

When the model emits a tool call, the response is an `AIMessage` with entries in `tool_calls`. Each entry contains the tool name, structured arguments, a call identifier, and a type. What comes back is not screened data, not a submitted order, and not a final recommendation. It is a structured request for the surrounding system to do something. The `bind_tools` call is the first important mechanism. It makes tool schemas available to the model. In this example, the schemas come from two Pydantic classes. `ScreenEquities` describes a read-oriented screening request. `SubmitOrder` describes a side-effecting trading request. LangChain converts those Pydantic models into provider-compatible tool definitions and sends them with the model request. The model can then respond with a tool-call request instead of ordinary text. The second mechanism is `tool_choice`. This controls how much freedom the model has in the current turn. With `tool_choice="auto"`, the model may either answer directly or emit a tool call. This is the most common setting for general assistants because the model can decide whether a tool is useful. In the first auto call, the user asks for a stock screen, so the model emits a `ScreenEquities` request. In the second auto call, the user asks for a general explanation, so the model can answer directly, and `tool_calls` is empty.

With `tool_choice="none"`, tools are disabled for that call. The model must respond directly, even though tool schemas were bound. This is useful when the application wants an explanatory answer only, when the current step must not perform retrieval or external work, or when a safety policy has temporarily disabled tools. With a force-any setting such as `tool_choice="any"`, or with `tool_choice="required"` on integrations that support that value, the model must emit at least one tool call while still choosing from the available tools. This is useful when the application already knows that the next step must be represented as a tool request but does not want to force a particular tool by name. The exact accepted values

are provider- and integration-specific, so treat the chapter's values as OpenAI/LangChain examples rather than universal syntax. With a named tool such as `tool_choice="ScreenEquities"`, the model must target that specific tool. This is useful when the surrounding workflow has already selected the next operation and only wants the model to fill in the tool arguments. For example, a routing step might have already determined that the request belongs to equity screening, so the next call can force `ScreenEquities`.

The third mechanism is `strict=True`, when the selected model integration supports it. This is about schema adherence, not business correctness. It asks the provider or integration to enforce stricter conformance between the generated arguments and the declared tool schema. In this example, a `ScreenEquities` call must provide values compatible with `sector`, `min_market_cap_usd_billion`, and `style`. The `style` value must be one of "value", "growth", or "quality". A `SubmitOrder` call must provide values compatible with `symbol`, `side`, `quantity`, `order_type`, and `limit_price_usd`. This is much stronger than asking the model to write a JSON-looking paragraph and parsing it afterwards. However, it is still only a structural guarantee. A schema-valid `SubmitOrder` request can still be wrong. It may exceed the user's risk limit, refer to an account the user cannot trade from, violate market rules, fail suitability checks, duplicate an earlier request, or require human approval. Schema validity only means that the arguments match the expected shape. It does not mean that the operation should be executed.

This is why function calling should be treated as a proposal boundary. The model proposes a tool name and arguments. The runtime validates the arguments, checks policy, executes the tool if allowed, records what happened, and feeds the result back into the conversation or graph. For read-only tools, this may be a simple validation and execution step. For side-effecting tools, it should be a controlled workflow with authorization, idempotency, audit logging, and, where appropriate, explicit confirmation. A useful production rule is to classify tools by consequence. Query tools such as `screen_equities`, `get_order_status`, or `search_catalog` can usually be executed after ordinary validation and authorization. Command tools such as `submit_order`, `request_refund`, `cancel_subscription`, or `change_address` need stronger controls because they mutate state or create obligations. The model can request both kinds of tools through the same function-calling interface, but the runtime should not treat them the same way. Function calling gives the model a structured way to ask for action. It does not give the model authority to act.

2.1.4 ToolNode: the execution boundary made explicit

Once a workflow includes tools, the system needs a clear place where a model's requested action is turned into actual program execution. That handoff is more important than it first appears. Without an explicit execution boundary, tool handling tends to leak into model nodes, routing logic, and custom helper code, until the graph becomes harder to understand and harder to trust. `ToolNode` exists to prevent that. In a LangGraph workflow, it is the prebuilt node responsible for executing tool calls that have already been proposed elsewhere in the graph. It can run tools, handle configured error behavior, support parallel tool execution, and provide tools with runtime access to the current execution context. That makes `ToolNode` an execution component, not a reasoning component. Its job begins after a prior step has already produced a tool call. In other words, `ToolNode` is not the part of the system

that decides whether a tool is needed, which tool is appropriate, or whether the overall plan is sensible. It sits on the other side of that decision. Its purpose is to take an already proposed action and execute it in a consistent, framework-supported way. This separation is one of the main reasons graph-based systems remain maintainable as they grow: the part that proposes actions and the part that carries them out do not have to be mixed together.

This separation also clarifies what `ToolNode` really guarantees. It guarantees that tool execution happens through a reusable node designed for that purpose, rather than through scattered glue code hidden across the workflow. It also gives tools a standard runtime surface, so a tool can access injected context or other runtime utilities without relying on globals or ad hoc wiring. That is a real architectural benefit, especially in larger graphs where multiple branches may lead to tool execution and where consistency matters as much as correctness.

At the same time, it is important not to attribute stronger guarantees to `ToolNode` than it actually provides. `ToolNode` does not validate the semantic or business correctness of a tool call. It does not know whether the model should have asked for the tool in the first place. It does not decide whether the result is meaningful, complete, or safe to act on. If a side-effecting tool should require approval, if a parameter should be checked against business rules, or if a workflow should stop instead of executing a risky action, those controls must be implemented elsewhere in the system. `ToolNode` gives you a disciplined execution boundary. It does not replace planning, validation, or governance.

2.2 TOOLNODE MECHANICS IN LANGGRAPH

2.2.1 From a tool call proposal to an executable graph step

Once the model has decided to call a tool, the main question is no longer whether a tool is needed. The question is how that proposed action becomes a normal graph step that can read state, execute application code, and return something the workflow can use in the next step. In LangGraph, that handoff is handled by `ToolNode`. In the standard pattern, the node receives the current graph state, looks at the message history, finds the most recent `AIMessage`, reads any `tool_calls` attached to it, executes the requested tools, and returns the corresponding tool results back into the graph. This is the point where a model-generated intention stops being a suggestion and becomes real program execution. The control flow is simpler than it first appears. A model node writes an `AIMessage` into state. A conditional edge then checks that last assistant message. If it contains tool calls, execution is routed to the tools node. If it does not, the graph can end or continue along a different path. Seen this way, the loop is not mysterious at all. It is just message-driven flow control: the model produces a message, the graph inspects that message, tools run only when the message asks for them, and the updated state is then passed back into the next step.

`ToolNode` is also more flexible than many introductory examples suggest. The most common input is a state object with a message field, but that is not the only form it can accept. It can also operate on a plain list of messages, provided the last relevant message is an `AIMessage` containing tool calls. It can even accept tool-call structures directly in programmatic scenarios. In practice, this flexibility is useful because it makes testing much easier. You can exercise tool execution behaviour without having to run a full end-to-end agent loop every time. A graph is still the normal production setting, but the node itself is not tightly coupled to only one representation of input. There is another small but important detail here. Although most examples use a state key named `messages`, that name is not hard-coded into the overall design. `ToolNode` supports a configurable `messages_key`, which allows the same execution logic to work with graphs that store their conversation history under a different field name. That may sound minor, but it reflects a broader design choice: the node is built to work with the graph's state model rather than forcing every graph into exactly the same shape.

`ToolNode` is still an ordinary graph node. It receives input from state and returns updates that LangGraph merges according to the reducers defined for each state key. For message histories, this usually means appending tool-result messages rather than replacing the conversation. The following small support example isolates that execution handoff without involving a live model:

```
from langchain_core.messages import AIMessage, ToolMessage
from langchain_core.tools import BaseTool, tool

ORDERS = {
    "ORD-100": {
        "status": "delivered",
        "days_since_delivery": 12,
        "amount": 129.0
    }
}

@tool
def lookup_order(order_id: str) -> str:
    """Return a short description of an order."""
```

```

order = ORDERS.get(order_id)
if not order:
    return f"Order {order_id} was not found."

return (
    f"Order {order_id}: status={order['status']}, "
    f"days_since_delivery={order['days_since_delivery']}, "
    f"amount={order['amount']}"
)

def invoke_tools_direct(state: dict, tools: list[BaseTool]) -> dict:
    messages = state.get("messages", [])
    if not messages:
        raise ValueError("State must contain at least one message.")

    last = messages[-1]
    if not isinstance(last, AIMessage):
        raise TypeError("The last message must be an AIMessage.")

    tool_calls = last.tool_calls or []
    tool_map = {tool.name: tool for tool in tools}

    tool_messages = []
    for call in tool_calls:
        name = call["name"]
        args = call.get("args", {})
        tool_call_id = call["id"]

        if name not in tool_map:
            raise ValueError(f"Unknown tool: {name}")

        result = tool_map[name].invoke(args)

        tool_messages.append(
            ToolMessage(
                content = str(result),
                name = name,
                tool_call_id = tool_call_id
            )
        )

    return {"messages": tool_messages}

state = {
    "messages": [
        AIMessage(
            content = "",
            tool_calls = [
                {
                    "name": "lookup_order",
                    "args": {"order_id": "ORD-100"},
                    "id": "call_lookup_1",
                    "type": "tool call"
                }
            ]
        )
    ]
}

update = invoke_tools_direct(state, [lookup_order])

for message in update["messages"]:
    print(type(message).__name__)
    print("tool_call_id:", message.tool_call_id)
    print("content:", message.content)

```

vol3_ch_02\src\tool_node.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_02> python -m src.tool_node
ToolMessage
tool_call_id: call_lookup_1
content: Order ORD-100: status=delivered, days_since_delivery=12, amount=129.0

```

This code isolates the execution side of tool calling. The assistant message already contains structured tool-call metadata. `invoke_tools_direct` resolves the requested tool by name, invokes it with the supplied arguments, and returns a `ToolMessage` with the same `tool_call_id`. The important point is not the order lookup itself; it is that the function

returns a graph-compatible message update rather than rebuilding the conversation from scratch.

2.2.2 Tool results are messages before they are anything else

When a model asks to use a tool, the result must be added back into the conversation as a message. `ToolNode` executes the requested tools and writes their results into graph state as `ToolMessage` objects, so the next model call can see which tool request has been satisfied. For example, imagine a customer support assistant asks to call a tool named `lookup_order`:

```
Assistant message:
Tool call: lookup_order(order_id="ORD-123")
```

After the tool runs, the result must be added to the conversation as a tool-result message:

```
ToolMessage:
tool_call_id="call_abc123"
content="Order ORD-123 was shipped on 12 June and is expected to arrive on 15 June."
```

On the next model call, the model does not magically “know” the result. It sees the result because it is now part of the message history. This also applies when the tool returns structured data. If the tool returns a dictionary, such as:

```
{
  "order_id": "ORD-123",
  "status": "shipped",
  "expected_delivery": "2026-06-15"
}
```

the value is serialized into a tool-result message. The model can then inspect the fields from the message history instead of trying to infer them from informal prose. This is why `tool_call_id` is important. A tool result is not just a random piece of data added later. It is the answer to one specific tool call requested by one specific assistant message. The `tool_call_id` connects the tool result to the original tool request. The required sequence is therefore:

1. `UserMessage`
"Where is my order ORD-123?"
2. `AIMessage`
The model requests a tool call:
`lookup_order(order_id="ORD-123")`
3. `ToolMessage`
The tool result is added:
"Order ORD-123 was shipped on 12 June..."
4. `AIMessage`
The model uses the tool result to answer the user:
"Your order was shipped on 12 June and should arrive on 15 June."

If the conversation contains an assistant message with a tool call, the next model-facing history must include the matching tool result before the model continues. Otherwise, the message history is incomplete. LangGraph can treat that history as malformed because the model asked for a tool, but the corresponding tool answer is missing. This is a common source of bugs when

developers handle tools manually. A developer may call the tool in Python, store the result in a local variable, and then call the model again with a normal prompt. The problem is that the tool result was never added to the message history as a `ToolMessage`. The application may have the result, but the model-facing conversation does not. `ToolNode` avoids this mistake. It executes the requested tools and returns state updates in the same form used by the rest of the graph: messages. In practical terms, `ToolNode` keeps the tool-calling loop consistent:

model asks for a tool

→ `ToolNode` executes the tool

→ `ToolNode` adds the tool result as a `ToolMessage`

→ the model reads the `ToolMessage` and continues

2.2.3 State updates follow graph rules, not special tool rules

Tool execution in LangGraph is not a separate execution model sitting outside the graph. It follows the same rules as every other node. A node receives the current state, performs some work, and returns an update rather than a complete replacement of the whole application state. LangGraph then applies that update according to the schema and the reducer defined for each state key. Execution proceeds in discrete super-steps: the active nodes for that step run, their updates are collected, and those updates are merged into the shared state before the next step begins. `ToolNode` fits into that mechanism exactly like any other node. Its output is a normal state update that becomes part of the next graph state. That point matters because it explains why tool execution feels predictable when the graph is designed well. The tool step does not secretly mutate the workflow behind the scenes. It returns just the keys it wants to update, and the graph merges those keys with the existing state. If a node returns `{"foo": 2}`, only `foo` changes. If it returns `{"messages": [...]}`, then only the message channel is updated. This is the same model whether the node is an assistant node, a data-processing node, or a `ToolNode`. Once you see that, tool execution stops looking like a special agent feature and starts looking like ordinary graph state transition.

For graphs that store conversation history in a `messages` field, the way that field is updated is critical. A tool result should normally be added to the existing conversation. It should not replace the whole conversation history. For example, after a customer support assistant calls `lookup_order`, the tool result should be added after the model's tool call. The previous user message, assistant message, and tool call must remain in the history. This is why message-centric LangGraph state usually defines a reducer for the `messages` field. A reducer tells LangGraph how to merge a node's update into the existing state. Without a reducer, the default behaviour is overwrite. That means a node returning this:

```
{
  "messages": [tool_message]
}
```

would replace the entire existing message list with only `tool_message`. The graph would lose the previous conversation. The intended behaviour is different:

existing messages

+ new tool message

= updated conversation history

In practice, the usual reducer for message state is `add_messages`. `add_messages` is better than simple list concatenation. It does not just append every incoming message blindly. When a message is new, it adds it to the history. When a message already exists, it can update the existing message by using message identity. This matters because message histories are not only extended during normal execution. They may also be modified during replay, debugging, inspection, or human-in-the-loop editing. A naive reducer, such as plain list concatenation, can create duplicate or incorrect histories in those situations. For example, if a human edits an existing message during a review step, simple concatenation may add the edited message as a second copy instead of replacing the original one. `add_messages` preserves the structure of the conversation more reliably:

new message

→ append it

existing message with same identity

→ update the existing entry

The key idea is simple: in message-centric graphs, the `messages` field represents the running conversation. Each node usually contributes new messages to that conversation. The `add_messages` reducer ensures those contributions are merged into the history safely instead of accidentally overwriting it or duplicating existing entries.

This is also why the prebuilt `MessagesState` is useful. It gives you a standard state shape for graphs that revolve around conversation history, with a `messages` field already configured to use `add_messages`. When you need more than messages alone, you can extend that state with additional fields and keep the same merge behaviour for the message channel. The result is a graph where tool results, assistant messages, and user messages all flow through the same state model instead of being handled through separate conventions. A compact schema example makes the idea concrete:

```
import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class SupportState(TypedDict):
    messages: Annotated[list, add_messages]
    audit_log: Annotated[list[str], operator.add]
```

Here the message history uses `add_messages`, while `audit_log` uses list concatenation through `operator.add`. The distinction matters because reducers are how you define merge semantics. `ToolNode` can execute more than one tool call in the same turn, and if multiple tools update the same field, you need a deterministic rule for combining those updates. Message history already has a purpose-built reducer. Custom state fields need the same level of care.

2.2.4 Updating state from inside a tool

A normal tool return is enough when the tool's job is only to produce data for the model to read on the next step. That works well for lookups, calculations, and other operations whose

only output is the result itself. It is not enough when the tool must also change the graph's working state, for example by storing a loaded customer profile, marking that a capability has been enabled, or recording information that later steps must read from state rather than recover from message text. In those cases, the tool should return `Command(update=...)`. That tells the runtime that the tool is not merely reporting an observation; it is requesting a state update that must be merged into the graph like any other node output. This distinction matters because tool output and graph state are not the same thing. A plain return value becomes tool-result content. A `Command` changes state directly. That is what allows a tool to populate custom fields, append to structured state, or otherwise make information available to later nodes without forcing the model to reconstruct that information from natural-language tool output. In practice, this is the step that turns tool execution from a simple request-response mechanism into part of the graph's state machine. There is one rule here that is easy to overlook but extremely important. When a tool was called from an assistant message that contains a tool call, the resulting update must still preserve a valid message history. That means the `Command` update must include the message-history field, and that message list must contain a `ToolMessage` tied to the current `tool_call_id`. The state change and the tool reply serve different purposes, and both are required. The state update makes the new data available to the workflow. The `ToolMessage` completes the outstanding tool request, so the next model step receives a valid conversation history. Without that matching tool-result message, the history is malformed.

A good way to think about it is this: when a tool updates state, it is doing two jobs at once. First, it writes something the application needs to keep. Second, it answers the model's request in the message channel the model expects. Those two actions usually happen together in the same `Command(update=...)`. The example below shows that full pattern in a small support workflow. The assistant's first pass emits a tool call; the tool writes to state and returns the matching `ToolMessage`; the second pass can then use the newly written state directly.

```
from typing import Annotated
from typing_extensions import TypedDict
from langchain.messages import AIMessage, HumanMessage, ToolMessage
from langchain.tools import tool, ToolRuntime
from langgraph.graph import StateGraph, START
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.types import Command
from .config import OPENAI_API_KEY

CUSTOMERS = {
    "alex@example.com": {"tier": "gold", "refund_window_days": 30}
}

class SupportState(TypedDict):
    messages: Annotated[list, add_messages]
    customer_email: str
    customer_profile: dict | None

@tool
def load_customer_profile(runtime: ToolRuntime) -> Command:
    """Load the current customer's profile into graph state."""
    email = runtime.state["customer_email"]
    profile = CUSTOMERS.get(email, {"tier": "standard", "refund_window_days": 14})

    return Command(
        update = {
            "customer_profile": profile,
            "messages": [
                ToolMessage(
                    content = (
                        f"Loaded customer profile: "
```

```

        f"tier={profile['tier']}", "
        f"refund_window_days={profile['refund_window_days']}"
    ),
    tool call id = runtime.tool call id
)
    ]
}
)

def assistant(state: SupportState):
    if state.get("customer profile") is None:
        return {
            "messages": [
                AIMessage(
                    content = "",
                    tool_calls = [
                        {
                            "name": "load_customer_profile",
                            "args": {},
                            "id": "call_profile_1",
                            "type": "tool_call"
                        }
                    ]
                )
            ]
        }

    profile = state["customer_profile"]
    return {
        "messages": [
            AIMessage(
                content = (
                    f"The customer profile is now in state. "
                    f"Tier={profile['tier']}; "
                    f"refund window={profile['refund_window_days']} days."
                )
            )
        ]
    }

builder = StateGraph(SupportState)
builder.add_node("assistant", assistant)
builder.add_node("tools", ToolNode([load_customer_profile]))

builder.add_edge(START, "assistant")
builder.add_conditional_edges("assistant", tools_condition)
builder.add_edge("tools", "assistant")

graph = builder.compile()

result = graph.invoke(
    {
        "messages": [HumanMessage(content = "Can I still ask for a refund?")],
        "customer_email": "alex@example.com",
        "customer_profile": None
    }
)

print("\nRUN TRACE\n")

for i, message in enumerate(result["messages"], start = 1):
    label = type(message).__name__

    if label == "HumanMessage":
        print(f"[{i}] User")
        print(f"    {message.content}")

    elif label == "AIMessage":
        print(f"[{i}] Assistant")
        if getattr(message, "tool_calls", None):
            print("    Requested tool call(s):")
            for tc in message.tool_calls:
                print(
                    f"        - {tc['name']} (args={tc['args']}, id={tc['id']})"
                )
            if message.content:
                print(f"    {message.content}")

    elif label == "ToolMessage":
        print(f"[{i}] Tool result")
        print(f"    {message.content}")

    else:

```

```

        print(f" [{i}] {label}")
        print(f"      {message.content}")

    print()

print("FINAL STATE")
print(f"  customer_email    : {result['customer_email']}")
print(f"  customer_profile    : {result['customer_profile']}")

```

vol3_ch_02\scr\tool_node_state_update.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_02> python -m src.tool_node_state_update
RUN TRACE

[1] User
    Can I still ask for a refund?

[2] Assistant
    Requested tool call(s):
    - load_customer_profile(args={}, id=call_profile_1)

[3] Tool result
    Loaded customer profile: tier=gold, refund_window_days=30

[4] Assistant
    The customer profile is now in state. Tier=gold; refund window=30 days.

FINAL STATE
  customer_email    : alex@example.com
  customer_profile  : {'tier': 'gold', 'refund_window_days': 30}

```

This code shows how a tool changes the LangGraph state in a controlled way and how that change affects the next step of execution. The important action happens inside `load_customer_profile`. Instead of returning plain tool output, it returns a `Command` with an update payload. That update writes the resolved customer profile into the `customer_profile` field, which means the tool is not only reporting information but changing the workflow's working state. That state change is the turning point of the graph. At the beginning, assistant checks whether `customer_profile` is missing. When it is `None`, the assistant does not try to answer the refund question directly. It emits a tool call for `load_customer_profile`. `ToolNode` executes that request, and the tool reads the trusted value `customer_email` from `runtime.state`, derives the correct profile, and writes it back into graph state. This is significant because the profile becomes structured application data, not just text embedded in a message.

The `ToolMessage` included in the `Command` update is equally important. It satisfies the outstanding tool call by attaching the result to the current `tool_call_id`, while the state update makes the loaded profile available to later nodes. These two effects happen together: the message completes the conversational contract, and the state mutation changes what the graph knows. On the second pass, assistant sees that `customer_profile` is now present, so its behavior changes. It stops requesting the tool and produces a normal assistant response based on the updated state. The trace shows this transition through the sequence of messages: the first assistant message requests `load_customer_profile`, the tool result confirms that the profile was loaded, and the second assistant message uses the updated state to report the customer tier and refund window. The final state then confirms that `customer_profile` now contains the structured profile data.

2.2.5 Runtime injection inside tools

Stateful tool execution becomes much easier to manage when tools do not have to discover their environment through globals, thread-local variables, or custom plumbing. In LangGraph, that problem is handled through runtime injection. When a tool declares a parameter such as `runtime: ToolRuntime`, the runtime supplies that object automatically at execution time, and that parameter is not exposed as part of the model-visible tool schema. This matters because it preserves a clean boundary between two different kinds of input. The model sees only the arguments it is allowed to choose. The tool, however, still receives trusted execution context from the application. That separation makes tool interfaces easier to reason about and prevents system-controlled values from being turned into model-controlled arguments by accident. The most important part of this injected runtime is that it gives the tool access to the current execution environment in a structured way. `runtime.state` exposes the current short-term graph state, including the message history and any custom fields the graph carries for the current run. This is what allows a tool to read information that was already gathered earlier in the workflow without having to pass that information explicitly through every tool schema. `runtime.context` provides invocation-scoped context that is meant to stay stable during the run, such as user identity, session-level settings, or other application data that should not be rewritten by the model. `runtime.store` provides access to longer-lived storage when the application has configured one, so a tool can work with persistent data without mixing that concern into the model-facing arguments. `runtime.stream_writer` allows the tool to emit progress updates while it is still running, which is useful when execution may take long enough that the application should surface intermediate feedback.

Another small but important field is `runtime.tool_call_id`. A tool execution does not happen in isolation; it is the fulfilment of a specific tool call requested by the assistant. The runtime exposes that identifier so the tool can produce output that remains tied to the original request. This is especially important when a tool needs to participate in stateful updates rather than simply return plain text. At a lower level, LangGraph also supports more explicit injection patterns for state and store, but `ToolRuntime` is usually the clearest entry point because it brings the execution context together in one object instead of scattering it across several separate parameters. This runtime model is one of the reasons `ToolNode` scales better than hand-written glue code. The tool does not need private knowledge of where state lives, how context is passed around, or how execution metadata is threaded through the system. Those concerns are handled by the runtime contract. The result is a cleaner design in which model-controlled inputs stay narrow, runtime-controlled data stays trusted, and tools can still participate fully in the stateful flow of the graph.

```
from __future__ import annotations
import json
from dataclasses import dataclass
from langchain.messages import AIMessage, HumanMessage, ToolMessage
from langchain.tools import ToolRuntime, tool
from langgraph.graph import END, START, MessagesState, StateGraph
from langgraph.prebuilt import ToolNode
from langgraph.store.memory import InMemoryStore

# ---- 1) Graph state: short-term state for the current run ----
# MessagesState already gives us the standard "messages" list.
# We extend it with one custom field to show runtime.state reading both.
class State(MessagesState):
```



```

ticket_id: str

# ---- 2) Invocation-scoped context: stable for a single run -----
@dataclass
class Context:
    user_id: str
    plan: str

# ---- 3) Tool that uses all four runtime surfaces -----
@tool
def inspect_runtime(note: str, runtime: ToolRuntime[Context, State]) -> str:
    """Inspect runtime.state, runtime.context, runtime.store, and
    runtime.stream_writer."""
    # A) runtime.state -> current graph state for this run
    messages = runtime.state["messages"]
    ticket_id = runtime.state["ticket_id"]

    last_user_message = next(
        (m.content for m in reversed(messages) if isinstance(m, HumanMessage)),
        None
    )

    # B) runtime.context -> stable invocation-scoped data
    user_id = runtime.context.user_id
    plan = runtime.context.plan

    # C) runtime.stream_writer -> emit progress while the tool is still running
    if runtime.stream_writer:
        runtime.stream_writer(
            {
                "phase": "start",
                "message_count": len(messages),
                "ticket_id": ticket_id
            }
        )

    # D) runtime.store -> longer-lived storage, separate from per-run state
    previous_note = None
    if runtime.store:
        namespace = ("users", user_id)
        item = runtime.store.get(namespace, "last_note")
        previous_note = item.value if item else None

        if runtime.stream_writer:
            runtime.stream_writer(
                {
                    "phase": "loaded-store",
                    "previous_note": previous_note
                }
            )

        runtime.store.put(
            namespace,
            "last_note",
            {"ticket_id": ticket_id, "note": note}
        )

    if runtime.stream_writer:
        runtime.stream_writer({"phase": "done", "saved note": note})

    # Return a JSON string so the result is easy to inspect in the ToolMessage
    return json.dumps(
        {
            "from_state": {
                "ticket_id": ticket_id,
                "message_count": len(messages),
                "last_user_message": last_user_message
            },
            "from_context": {
                "user_id": user_id,
                "plan": plan
            },
            "from_store": {
                "previous_note": previous_note,
                "new_note": note
            }
        }
    )

# ---- 4) Build the graph -----
# The graph has one node: ToolNode.
# We seed the graph with an AIMessage that already contains a tool call.

```

```

store = InMemoryStore()

builder = StateGraph(State, context_schema = Context)
builder.add_node("tools", ToolNode([inspect_runtime]))
builder.add_edge(START, "tools")
builder.add_edge("tools", END)

graph = builder.compile(store = store)

# ---- 5) Helper to create an input state -----
def make_input(ticket_id: str, note: str) -> State:
    return {
        "ticket_id": ticket_id,
        "messages": [
            HumanMessage(content = "Please inspect the runtime."),
            AIMessage(
                Content = "",
                tool_calls = [
                    {
                        "name": "inspect_runtime",
                        "args": {"note": note},
                        "id": "call_1",
                        "type": "tool_call"
                    }
                ]
            )
        ]
    }

# ---- 6) Run twice to show the difference between state and store -----
# State is per run. Store survives across runs because the same store instance
# is reused.
def run_once(ticket_id: str, note: str) -> None:
    print(f"\n=== RUN ticket id={ticket_id!r}, note={note!r} ===")
    final_state = None

    for mode, data in graph.stream(
        make_input(ticket_id = ticket_id, note = note),
        context = Context(user_id = "user-123", plan = "gold"),
        stream_mode = ["custom", "values"],
        version = "v2"
    ):
        if mode == "custom":
            print("CUSTOM:", data)
        elif mode == "values":
            final_state = data

    assert final_state is not None

    # The tool result is appended as a ToolMessage by ToolNode
    tool_msg = final_state["messages"][-1]
    assert isinstance(tool_msg, ToolMessage)

    print("TOOL RESULT:")
    print(json.dumps(json.loads(tool_msg.content), indent = 2))

if __name__ == "__main__":
    run_once(ticket_id = "T-100", note = "first run")
    run_once(ticket_id = "T-200", note = "second run")

```

vol3_ch_02\src\tool_node_state.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_02> python -m src.tool_node_state

=== RUN ticket id='T-100', note='first run' ===
CUSTOM: {'phase': 'start', 'message_count': 2, 'ticket_id': 'T-100'}
CUSTOM: {'phase': 'loaded-store', 'previous_note': None}
CUSTOM: {'phase': 'done', 'saved_note': 'first run'}
TOOL RESULT:
{
  "from state": {
    "ticket_id": "T-100",
    "message_count": 2,
    "last_user_message": "Please inspect the runtime."
  },
  "from context": {
    "user_id": "user-123",
    "plan": "gold"
  },
  "from store": {
    "previous_note": null,

```

```

    "new_note": "first run"
  }
}

=== RUN ticket_id='T-200', note='second run' ===
CUSTOM: {'phase': 'start', 'message_count': 2, 'ticket_id': 'T-200'}
CUSTOM: {'phase': 'loaded-store', 'previous_note': {'ticket_id': 'T-100', 'note': 'first run'}}
CUSTOM: {'phase': 'done', 'saved_note': 'second run'}
TOOL RESULT:
{
  "from_state": {
    "ticket_id": "T-200",
    "message_count": 2,
    "last_user_message": "Please inspect the runtime."
  },
  "from_context": {
    "user_id": "user-123",
    "plan": "gold"
  },
  "from_store": {
    "previous_note": {
      "ticket_id": "T-100",
      "note": "first run"
    },
    "new_note": "second run"
  }
}

```

2.2.6 Multiple tool calls

A single assistant message can request more than one tool call. For example, a customer support assistant may decide that it needs to do three things before answering the user:

1. Look up the order status.
2. Check the refund policy.
3. Retrieve the customer's support history.

These are separate tool calls, but they can all come from the same assistant message. `ToolNode` can execute those tool calls concurrently when they are independent. In synchronous execution, it dispatches the calls through an executor. In asynchronous execution, it awaits them together, for example with `asyncio.gather`. This improves throughput when the tools spend most of their time waiting on external systems such as APIs, databases, search services, or internal business platforms. Instead of waiting for one tool to finish before starting the next, the graph can run independent tools in parallel and collect the results. The important point is not only performance. It also changes how you must think about state. Once multiple tools can run at the same time, multiple tools can also return state updates at roughly the same time. That means overlapping updates are not an unusual edge case. They are part of the normal execution model. For example:

Tool A adds an audit event.

Tool B adds another audit event.

Tool C adds a tool result message.

The graph must know how to merge those updates into the existing state. In LangGraph, each state field has its own merge rule. That merge rule is called a reducer. If you do not define a reducer for a field, the default behaviour is overwrite. Overwrite is acceptable for fields where only one value should survive. For example, if a node sets a final status field to `"complete"`, overwriting may be exactly what you want. It is unsafe for fields that can receive updates from multiple parallel tool calls. For example, assume two tools both return audit events:

```
{"audit_events": ["checked_order_status"]}
```

and:

```
{"audit_events": ["checked_refund_policy"]}
```

If `audit_events` uses the default overwrite behaviour, one update may replace the other. The final state may contain only one audit event, even though both tools ran successfully. The correct design is to define a reducer that merges the lists:

existing audit events

+ audit events from Tool A

+ audit events from Tool B

= complete audit trail

The same rule applies to any custom state field that can be updated by parallel tools. If two tools can add warnings, merge the warnings. If two tools can add retrieved documents, merge the documents. If two tools can write the same scalar field, define which value wins. For example, you may choose the latest timestamp, the highest severity, the first successful result, or an explicit conflict state. What you should not do is leave the behaviour accidental. If two parallel tools can update the same field, overwrite is not a neutral default. It is an implicit decision that one update may erase another. The `messages` field already has a deliberate merge rule. Message-centric graphs usually use the `add_messages` reducer. That reducer appends new messages and can update existing messages by identity when needed. As a result, tool-result messages from multiple tool calls can be accumulated into the conversation history in a controlled way. Custom state fields do not automatically get the same protection. If parallel tool calls can update them, you must give those fields explicit reducers. The key idea is simple: multiple tool calls make parallel state updates normal. `ToolNode` can execute the tools, but the graph state still needs clear merge rules. Reducer design is therefore part of correctness, not a later cleanup step.

2.2.7 Error handling in ToolNode

`ToolNode` does not treat every tool failure in the same way, because different failures mean different things. The first kind of failure happens before the tool can run correctly. This is usually caused by invalid arguments from the model. For example, the model may ask to call an order lookup tool but provide the wrong field name, the wrong data type, or a required value may be missing. In this case, the tool itself has not really failed. The model has made an invalid tool request. By default, `ToolNode` treats this kind of validation problem as something the model may be able to fix. Instead of stopping the whole graph immediately, it turns the validation failure into a tool-result message that explains what was wrong. The model can then read that message, correct the arguments, and try the tool call again. This is useful because many argument errors are recoverable inside the normal agent loop. The model asked for the right kind of operation but expressed the request incorrectly. Returning a clear error message gives the model a chance to repair its own tool call.

The second kind of failure happens after a tool has been selected and invoked, but execution fails while the tool is running. The database may be unavailable, an external API may time out, a downstream service may return an error, or the tool implementation may raise an unexpected exception. These are execution failures, not argument-validation failures. Current `ToolNode` behavior is deliberately configurable. By default, current `ToolNode` behavior is designed to return invocation-time argument errors as tool-result messages while allowing other execution failures to propagate unless broader handling is configured. This distinction matters because a malformed model request and a broken backend service are not the same problem. The first may be recoverable by the model. The second often requires retries, fallback logic, escalation, or human review. Because this behavior is configurable and can vary by LangGraph version, production code should set `handle_tool_errors` deliberately rather than relying on the default.

`ToolNode` also needs to preserve the structure of tool-calling conversations. When an error is handled, the failure is returned as a `ToolMessage` connected to the original tool request through the same `tool_call_id` and marked as an error. The tool call did not succeed, but the message history remains structurally valid. That matters because, if an assistant message contains tool calls, the next model-facing history must contain matching tool-result messages before the model continues. A handled tool error lets the model see that the tool call completed with an error result and decide what to do next: correct the arguments, call a different tool, ask the user for missing information, or explain that the operation cannot be completed. When broader error handling is required, configure `handle_tool_errors` explicitly: catch all ordinary exceptions, catch selected exception types, return a fixed safe message, use a custom handler, or disable handling so exceptions propagate immediately. For argument-validation failures, the error returned to the model should focus on values the model controlled, not on runtime-injected values such as graph state, stores, runtime objects, or application context.

An unhandled exception is different. If the tool raises an exception and the graph stops before `ToolNode` can write the matching tool-result message, the conversation history is left incomplete. It contains an assistant message with a tool call, but no corresponding tool result. LangGraph can treat that as an invalid chat history, because the model-facing message sequence has an unanswered tool request. The practical rule is simple: once the model asks for a tool, the next model-facing history must include a matching tool result. That result may describe success, or it may describe a handled error. What matters is that the tool request has a corresponding response. The design question is therefore not whether all tool errors should be hidden. They should not. The real design question is which errors should become model-visible tool results, and which errors should stop the graph. Argument errors are often useful to return to the model because the model may be able to correct them. Infrastructure failures, programming bugs, sensitive errors, and business-critical failures may be better handled through retries, fallback paths, interrupts, or human review. A small configuration example makes the options concrete:

```
from langgraph.prebuilt import ToolNode

# Default: convert invocation-time argument errors into tool results,
# but re-raise other execution failures
tool_node = ToolNode(tools)

# Catch all ordinary exceptions and return them as ToolMessages
```

```
tool_node = ToolNode(tools, handle_tool_errors = True)

# Catch only specific exception types
tool_node = ToolNode(tools, handle_tool_errors = (ValueError, TypeError))

# Return the same custom text for every handled error
tool_node = ToolNode(tools, handle_tool_errors = "Tool execution failed")

# Map an exception to custom text
def handle_errors(e: ValueError) -> str:
    return "Invalid input provided"

tool_node = ToolNode(tools, handle_tool_errors = handle_errors)
```

The design goal is not to hide failure. It is to decide, case by case, whether failure should become part of the conversation as a tool result or whether it should stop the run immediately. That choice affects both recovery behavior and message-history integrity, so it should be made deliberately rather than left as an accidental side effect of whatever exception happened to be raised.

2.2.8 A short note about MCP in this execution model

At execution time, a tool loaded through MCP reaches the graph as an ordinary LangChain tool. Discovery, transport, remote connectivity, and protocol negotiation have already been handled by the MCP client and adapter. ToolNode therefore follows the same graph rules for local and MCP-backed tools: ordinary returns become ToolMessage objects, Command returns can update graph state, and any concurrent updates are merged through the reducers defined for the state. This keeps MCP connectivity separate from graph execution mechanics.

2.3 MCP (MODEL CONTEXT PROTOCOL): STANDARDIZED TOOL INTERFACES AND TRANSPORT

When an application only needs a small and fixed set of local tools, direct tool binding is usually the simpler design. The tools are known in advance, they live inside the same application, and the developer can wire them directly into the agent. In that case, adding another protocol layer may not provide enough value. MCP becomes useful when the situation is different. It is designed for cases where tools and capabilities live outside the host application, need to be discovered at runtime, or must be reused by several different AI applications. Instead of every application integrating each tool through custom SDK code, MCP provides a standard way for applications and external capability providers to communicate. At that point, the main problem is no longer just whether the model can request a tool call. LangChain and model providers already support tool calling. The larger problem is how many different capabilities can be exposed to many different hosts through one consistent contract. That is the problem MCP addresses. MCP standardizes the connection between an AI application and external capability servers. It defines what the two sides can say to each other, how they describe available capabilities, how they negotiate what is supported, and how messages are exchanged during execution. The architecture has three main roles.

- The host is the AI application. This could be a chatbot, an IDE assistant, a desktop AI application, or an agent runtime.
- The client is the component created by the host to communicate with one MCP server. A host can create multiple clients if it needs to connect to multiple servers.
- The server exposes capabilities. These capabilities may include tools, resources, prompts, or other context that the host can use.

Each client maintains a dedicated connection to one server. This gives the system a clean boundary: the host does not need bespoke integration code for every external capability. Instead, it communicates through the MCP protocol. MCP also separates the protocol into two concerns.

- The data layer defines the message model and the shared semantics. It covers the JSON-RPC structure, lifecycle events, capability negotiation, and the core primitives exchanged between clients and servers.
- The transport layer defines how those messages physically move between the two sides. The same protocol concepts can therefore be carried over different communication mechanisms, depending on how the host and server are deployed.

The practical value is standardization. Direct tool binding is still a good choice when tools are local, fixed, and application-specific. MCP is more appropriate when tools are external, shared, dynamic, or intended to be reused across multiple hosts. It replaces one-off integration glue with a predictable protocol boundary between AI applications and the systems that provide capabilities.

MCP should therefore be read as an interoperability layer, not as a reasoning layer. It standardizes the boundary where capabilities are described, discovered, and invoked; planning, routing, validation, and execution policy remain application concerns.

2.3.1 MCP and provider ecosystems: compatibility is real, sameness is not

MCP matters because it is becoming a shared way to expose capabilities across different AI applications and frameworks. An MCP server can publish tools, resources, and prompts in a standard form, and client-side libraries can load those capabilities into agent frameworks instead of forcing every integration to be rewritten from scratch. That is the practical value of MCP: it turns external capability access from a collection of bespoke adapters into something closer to a reusable interface layer.

Compatibility, however, is not sameness. Two systems may both support MCP and still behave differently once execution begins. One framework may load MCP capabilities as local-looking tools. Another may require an approval flow before a tool is invoked. One host may allow direct model-controlled calls, while another may enforce stricter policy checks around the same capability. MCP helps these systems speak a common language about capability description, discovery, and invocation. It does not force them to share the same runtime behavior, execution policy, or user experience. The gain is real, but specific: a more uniform integration surface, not a guarantee that every provider and every agent runtime behaves identically.

2.3.2 The interface MCP standardizes

Once the protocol boundary is in place, three kinds of capability matter most: tools, resources, and prompts. Earlier in the chapter we already treated these as different concepts. Here the key point is that MCP gives each of them a standard interface. Tools are executable operations. Resources are readable context objects addressed by URIs. Prompts are reusable prompt templates that can be retrieved and parameterized. The host does not need a different integration style for each one. It speaks the same protocol family to all of them. For tools specifically, MCP standardizes both discovery and invocation. A client can discover available tools through `tools/list`, which returns metadata such as name, title, description, and input schema. It can invoke a tool through `tools/call`, passing structured arguments rather than scraping meaning out of prose. The current tools specification also allows optional `outputSchema`, structured results in `structuredContent`, and richer content payloads such as text, images, audio, resource links, and embedded resources. In practice, this means the protocol gives you a standard shape for tool contracts and tool results, even when the underlying systems are completely different.

That is a strong improvement in interface discipline, but it remains an integration guarantee. The stronger discussion of what MCP does and does not guarantee belongs later in the section, after the reader has seen tools, resources, prompts, transports, sessions, and adapters in practice. MCP is broader than remote tool execution. A server does not only expose capabilities to the host; in some cases, it can also ask the host to provide support during the interaction. For example, a server may ask the host to run a model call, request additional input from the user, or receive logging information about what is happening. These are client-side capabilities because they are provided by the host side of the connection, not by the MCP server itself. This

matters because MCP is not just a standardized catalog of callable functions. It also defines how an external capability provider and an AI host can cooperate during a workflow. The server may expose tools, resources, and prompts, but it may also depend on the host for model access, user interaction, or operational feedback. In that sense, MCP standardizes a two-way protocol between the AI application and external capability servers, not merely a one-way mechanism for calling remote tools.

2.3.3 Transport in practice: local and remote servers

At transport level, the current MCP specification centres on two standard mechanisms: `stdio` and `Streamable HTTP`. `stdio` is the local case. The client launches the MCP server as a subprocess, sends JSON-RPC messages to standard input, and receives responses on standard output. `Streamable HTTP` is the remote case. It is the transport to think about when the MCP server runs as a networked service rather than as a local helper process. Older SSE-oriented examples still exist in parts of the ecosystem, but the current specification's standard remote transport story is `Streamable HTTP`. That transport choice is not cosmetic. A local `stdio` server is often the fastest and simplest option when the capability belongs on the same machine as the host application. A remote HTTP server is the right fit when the capability must be shared across machines, users, or deployments, or when it fronts a service that already lives behind a network boundary. The protocol is the same family in both cases, but the operational characteristics are not. Local processes reduce network overhead. Remote servers make reuse and centralized administration easier.

The authorization model follows the same separation. For HTTP-based transports, the MCP authorization specification defines an OAuth 2.1-based transport-level approach and requires discovery through Protected Resource Metadata when authorization is supported. For `stdio`, the same specification explicitly says not to use the HTTP authorization flow and instead retrieve credentials from the environment. That does not yet answer the larger application question of which tool should be exposed to which model or user, but it does clarify where transport authentication belongs.

The tools specification is equally careful about approval. It allows tools to be model-controlled, but it recommends that applications make tool exposure visible to users and keep a human in the loop for operations that deserve confirmation. That is the right boundary to keep in mind. MCP standardizes the invocation channel. Approval policy still belongs to the host application.

2.3.4 `MultiServerMCPClient`: the LangChain bridge

Inside LangChain, the main adapter for this layer is `MultiServerMCPClient`, provided by `langchain-mcp-adapters`. Its job is to connect to one or more MCP servers and turn their capabilities into LangChain-native objects. That matters because LangChain agents and LangGraph workflows do not want to reason about JSON-RPC packets or transport sessions directly. They want tools, messages, and other framework-native types. `MultiServerMCPClient` is the bridge that performs that adaptation. A practical support-assistant example makes the role of the client concrete. Imagine that order lookups are exposed by a local MCP server running on the same machine as the application, while shipment status comes from a remote MCP server owned by another team. The agent should not need two

different integration models. It should see one set of tools. Start with `mcp_orders_server`, a local MCP server that uses the `stdio` transport. The example assumes the MCP server dependencies have already been installed in the project environment.

```
(.venv) C:\Users\alexc\llm-app> python -m pip install mcp
(.venv) C:\Users\alexc\llm-app> python -m pip install langchain-mcp-adapters
```

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("Orders")

ORDERS = {
    "ORD-100": {
        "customer": "Alice Carter",
        "status": "paid",
        "shipping_id": "SHP-9001",
        "items": [
            {"sku": "SKU-CHAIR-01", "name": "Office Chair", "qty": 1}
        ],
        "priority": "normal"
    },
    "ORD-200": {
        "customer": "Bob Smith",
        "status": "paid",
        "shipping_id": "SHP-9002",
        "items": [
            {"sku": "SKU-DESK-01", "name": "Standing Desk", "qty": 1}
        ],
        "priority": "high"
    }
}

@mcp.tool()
def get_order(order_id: str) -> dict:
    """Return order details for a given order ID."""
    order = ORDERS.get(order_id)
    if not order:
        return {
            "found": False,
            "order_id": order_id,
            "message": f"Order {order_id} was not found."
        }
    return {
        "found": True,
        "order id": order_id,
        **order
    }

if __name__ == "__main__":
    # stdio transport for local subprocess communication
    mcp.run(transport = "stdio")
```

vol3_ch_02\scr\mcp_orders_server.py

This code turns the order lookup capability into a local MCP server that can be launched as a subprocess and spoken to over standard input and output. The important design choice is at the bottom: running with `transport="stdio"` makes the server suitable for host applications that want a simple local integration boundary without exposing an HTTP endpoint. The `get_order` function is registered as an MCP tool, so the host can discover and invoke it through the protocol rather than through direct Python imports. The return payload is deliberately structured, which makes downstream tool consumers easier to implement and less dependent on ad hoc text parsing.

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("Shipping")

SHIPMENTS = {
    "SHP-9001": {
```

```

        "carrier": "DHL",
        "status": "in_transit",
        "last_event": "Departed local sorting facility",
        "eta": "2026-03-19",
        "delivery_attempted": False,
        "lost_risk": "low"
    },
    "SHP-9002": {
        "carrier": "UPS",
        "status": "exception",
        "last_event": "No movement for 5 days; parcel location unknown",
        "eta": None,
        "delivery_attempted": False,
        "lost_risk": "high"
    }
}

@mcp.tool()
def get_shipping_status(shipping_id: str) -> dict:
    """Return shipping status for a given shipment ID."""
    shipment = SHIPMENTS.get(shipping_id)
    if not shipment:
        return {
            "found": False,
            "shipping_id": shipping_id,
            "message": f"Shipment {shipping_id} was not found."
        }

    return {
        "found": True,
        "shipping_id": shipping_id,
        **shipment
    }

@mcp.tool()
def should_escalate_shipping_case(
    status: str,
    delivery_attempted: bool,
    lost_risk: str
) -> dict:
    """Return a simple support escalation recommendation."""
    escalate = False
    reason = "No escalation needed."

    if status == "exception":
        escalate = True
        reason = "Shipment is in exception state."
    elif lost_risk == "high":
        escalate = True
        reason = "Shipment has high lost-risk."
    elif delivery_attempted and status != "delivered":
        escalate = True
        reason = "Delivery was attempted but shipment is not delivered."

    return {
        "escalate": escalate,
        "reason": reason
    }

if __name__ == "__main__":
    mcp.run(transport="streamable-http")

```

vol3_ch_02\scr\mcp_shipping_server.py

The HTTP-facing server is the Shipping MCP service. It exposes shipping capabilities as named MCP tools so a remote client can call them through a standard endpoint instead of importing application code directly. One tool retrieves shipment state from the in-memory store and returns a consistent structured payload, including a not-found branch that avoids ambiguous failures. The second tool encodes a small escalation policy behind a server-side interface, so clients request a recommendation rather than reimplementing the logic themselves. The final `mcp.run(transport="streamable-http")` line is what makes this service network-accessible as an MCP server rather than a local subprocess tool provider.

```
import asyncio
```

```

import os
import sys
from pathlib import Path
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_mcp_adapters.client import MultiServerMCPClient
from .config import OPENAI_API_KEY

def extract_text_messages(result: dict) -> None:
    """Print final message contents in a readable way."""
    print("\n=== AGENT MESSAGES ===\n")
    for i, message in enumerate(result["messages"], start = 1):
        message_type = getattr(message, "type", message.__class__.__name__)
        print(f"[{i}] {message_type}")

        content = getattr(message, "content", None)
        print(content)

        tool_calls = getattr(message, "tool_calls", None)
        if tool_calls:
            print("tool_calls =", tool_calls)

        print()

async def main() -> None:
    base_dir = Path(__file__).resolve().parent
    orders_server = str((base_dir / "mcp_orders_server.py").resolve())

    client = MultiServerMCPClient(
        {
            "orders": {
                "transport": "stdio",
                "command": sys.executable,
                "args": [orders_server]
            },
            "shipping": {
                "transport": "http",
                "url": "http://127.0.0.1:8000/mcp",
                # Optional:
                # "headers": {"Authorization": "Bearer YOUR_TOKEN"},
            }
        }
    )

    tools = await client.get_tools()
    agent = create_agent("openai:gpt-4.1", tools)
    result = await agent.ainvoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": (
                        "Customer ORD-100 says the parcel has not arrived. "
                        "Check the order and shipping status, then tell me "
                        "whether support should escalate."
                    )
                }
            ]
        }
    )

    extract_text_messages(result)

if __name__ == "__main__":
    asyncio.run(main())

```

vol3_ch_02\scr\mcp_app.py

Now open a new terminal window and start the shipping server:

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_02> python -m src.mcp_shipping_server

INFO:      Started server process [22068]
INFO:      Waiting for application startup.
[03/17/26 08:47:06] INFO      StreamableHTTP session manager started
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

```

Now run the mcp application:

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_02> python -m src.mcp_app

.
.
.
=== AGENT MESSAGES ===

[1] human
Customer ORD-100 says the parcel has not arrived. Check the order and shipping status,
then tell me whether support should escalate.

[2] ai
tool_calls = [{'name': 'get_order', 'args': {'order_id': 'ORD-100'}, 'id':
'call_VajN9l5crW8JNu4dPOGpnDjt', 'type': 'tool_call'}]

[3] tool
[{'type': 'text', 'text': '{\n  "found": true,\n  "order_id": "ORD-100",\n  "customer":
"Alice Carter",\n  "status": "paid",\n  "shipping_id": "SHP-9001",\n  "items":
{\n    {\n      "sku": "SKU-CHAIR-01",\n      "name": "Office Chair",\n      "qty":
1\n    }\n  },\n  "priority": "normal"\n}', 'id': 'lc_4ae4f143-ed5c-45e4-8e39-
4c6fc16e5a46'}]

[4] ai
tool_calls = [{'name': 'get_shipping_status', 'args': {'shipping_id': 'SHP-9001'}, 'id':
'call_yi8LOeZ5NrOXFL8ZQnF67ukS', 'type': 'tool_call'}]

[5] tool
[{'type': 'text', 'text': '{\n  "found": true,\n  "shipping_id": "SHP-
9001",\n  "carrier": "DHL",\n  "status": "in_transit",\n  "last_event": "Departed local
sorting facility",\n  "eta": "2026-03-19",\n  "delivery_attempted":
false,\n  "lost_risk": "low"\n}', 'id': 'lc_164f00f4-fc8f-4729-8573-a5d35eb5b3c5'}]

[6] ai
tool_calls = [{'name': 'should_escalate_shipping_case', 'args': {'status': 'in_transit',
'delivery_attempted': False, 'lost_risk': 'low'}, 'id': 'call_TosQdpLHmso2NACFHVAXi7JB',
'type': 'tool_call'}]

[7] tool
[{'type': 'text', 'text': '{\n  "escalate": false,\n  "reason": "No escalation
needed."\n}', 'id': 'lc_eab11f66-3599-497e-bdb0-4ccb49b14244'}]

[8] ai
Order ORD-100 for Alice Carter is paid and currently in transit with DHL. The latest
status is "Departed local sorting facility," and the estimated arrival is 2026-03-19. No
delivery attempt has been made yet, and the risk of the parcel being lost is low.

Support does not need to escalate this case at this time. The parcel appears to be on its
way as expected.
```

MultiServerMCPClient is used here as the single integration point that turns multiple MCP servers, running under different transports, into one coherent tool surface for an agent. The client is initialized with a named server map. The “orders” entry is configured for the stdio transport, spawning a local MCP server as a subprocess using the current Python interpreter and the resolved path to mcp_orders_server.py. This matters because it lets the application treat a local, version-controlled capability as a tool provider without embedding that logic directly in the agent runtime. The “shipping” entry is configured for HTTP, pointing at an MCP endpoint (/mcp) on localhost. Keeping these side-by-side demonstrates the key design choice: transport differences are pushed to the boundary, while the agent consumes a uniform tool API.

await client.get_tools() is the pivotal step: it discovers/loads the exposed tools from all configured servers and returns them as a single list ready for agent binding. From that point on, the agent is created with those tools and can plan across them in one run—e.g., calling an order lookup tool from the stdio server and a shipment status tool from the HTTP server—

without the application manually routing requests. In other words, `MultiServerMCPClient` centralizes tool discovery, process/network wiring, and tool adaptation so the agent invocation stays clean and transport-agnostic.

```
[StructuredTool(
  name='get_order',
  description='Return order details for a given order ID.',
  args_schema={'properties': {'order_id': {'title': 'Order Id', 'type': 'string'}},
                'required': ['order_id'], 'title': 'get_orderArguments', 'type':
                'object'},
  response_format='content_and_artifact', coroutine=<function convert_mcp_tool_to_
    langchain_tool.<locals>.call_tool at 0x000001E247ECEDE0>),

StructuredTool(
  name='get_shipping_status',
  description='Return shipping status for a given shipment ID.',
  args_schema={'properties': {'shipping_id': {'title': 'Shipping Id', 'type':
    'string'}}, 'required': ['shipping_id'], 'title':
    'get_shipping_statusArguments', 'type': 'object'},
  response_format='content_and_artifact', coroutine=<function convert_mcp_tool_to_
    langchain_tool.<locals>.call_tool at 0x000001E247E8F2E0>),

StructuredTool(
  name='should_escalate_shipping_case', description='Return a simple support escalation
    recommendation.',
  args_schema={'properties': {'status': {'title': 'Status', 'type': 'string'},
    'delivery_attempted': {'title': 'Delivery Attempted', 'type':
    'boolean'}, 'lost_risk': {'title': 'Lost Risk', 'type': 'string'}},
    'required': ['status', 'delivery_attempted', 'lost_risk'], 'title':
    'should_escalate_shipping_caseArguments', 'type': 'object'},
  response_format='content_and_artifact', coroutine=<function convert_mcp_tool_to_
    langchain_tool.<locals>.call_tool at 0x000001E247EFDC60>)]
```

2.3.5 Loading capabilities: tools, resources, and prompts

A common mistake is to treat MCP as “just a way to load tools”. In practice, the LangChain MCP client exposes the other MCP primitives too, so you can bring different capability types into a workflow without flattening them into tools.

- `client.get_tools()` discovers executable actions across the configured MCP servers and returns LangChain tool objects that an agent can call. That is the right channel when you expect the model to request an operation and receive a computed result.
- `client.get_resources("server_name", ...)` reads resources from a specific MCP server and returns LangChain Blob objects. That is the right channel for readable context such as Markdown, JSON, CSV, policy files, or binary content that the application controls.
- `client.get_prompt("server_name", "prompt_name", ...)` retrieves a named server-defined prompt and returns LangChain message objects. That is the right channel for reusable interaction frames, not executable work.

In the same support-assistant scenario, a refund policy is naturally a resource because it should be read and referenced, while an escalation template is naturally a prompt because it should shape how the assistant writes the escalation.

```
# Load a policy document exposed as an MCP resource
policy_blobs = await client.get_resources(
    "orders",
    uris = ["file:///policies/refunds.md"]
)

# Load a reusable prompt template exposed by the server
escalation_prompt = await client.get_prompt(
    "orders",
    "escalate_missing_delivery",
    arguments = {"channel": "chat", "priority": "normal"}
)
```


This separation keeps the workflow honest: read via resources, steer via prompts, act via tools. MCP gives you one protocol family without flattening those distinctions.

2.3.6 Runtime behavior: sessions, structured output, multimodal output, and callbacks

A subtle but high-impact behavior in `MultiServerMCPClient` is that it is stateless by default. In practical terms, each tool invocation is executed in its own short-lived MCP `ClientSession`: a session is opened, the tool runs, and the session is closed. This “one call, one session” model is often the right default because it keeps execution isolated, avoids accidental cross-talk between requests, and works well when the server treats every call as independent. That default breaks down when the server’s correctness depends on continuity across calls. If the server accumulates state (conversation context, workflow progress, cached resources, temporary handles, or any other session-scoped data) then creating a fresh session per tool call means the server never sees a consistent thread of interaction. The agent may issue a sequence of logically related tool calls, but the server experiences them as unrelated, reset interactions.

To support continuity-sensitive servers, `MultiServerMCPClient` exposes explicit session control via `client.session("server_name")`. Opening a persistent `ClientSession` lets you bind tool discovery and subsequent tool executions to the same logical session, so state is preserved across calls. One common trap is assuming `stdio` implies shared session state because a subprocess can remain alive. Process lifetime is not the same thing as session lifetime. Even with a long-running subprocess, the default behavior still creates a new logical MCP session per tool call unless you explicitly manage the session.

```

:
:
:
async with client.session("orders") as session:
    tools = await load_mcp_tools(session)
    agent = create_agent("openai:gpt-4.1", tools)

    result = await agent.ainvoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": (
                        "Continue the return flow we already started for "
                        "order ORD-100 and add the note: box slightly damaged."
                    )
                }
            ]
        }
    )
:
:

```

vol3_ch_02\scr\mcp_app_session.py

2.3.7 Structured output, multimodal output, and callbacks

When MCP integration moves past simple “call a tool, get back a string” behavior, the adapter layer takes on a more important role. It is not just discovering tools. It is also translating richer MCP responses into forms the rest of the LangChain runtime can use consistently. If an MCP tool returns `structuredContent`, that structured payload is preserved and attached to the resulting `ToolMessage` as an artifact. This is important because the result is no longer

reduced to plain text. The application can still access the structured data after the tool call completes, which makes downstream handling more reliable and less dependent on brittle text parsing. The same applies to multimodal results. If a tool returns mixed content, such as text together with images, the adapter converts that output into LangChain's standard content-block format. That conversion matters because it gives the rest of the application a uniform way to process the result, regardless of which model provider or MCP server produced it. The practical benefit is that richer tool outputs remain usable inside the agent runtime instead of being flattened into a less informative text response.

The callback layer extends this model further. `MultiServerMCPClient` can receive progress updates and log messages from MCP servers while work is in progress. It can also respond to elicitation requests, where a server pauses execution and asks the client for additional structured input before continuing. That makes the interaction model more dynamic than ordinary function calling. A tool execution can become a multi-step exchange rather than a single request-response operation.

2.3.8 Control behavior: interceptors, authorization, and state updates

Interceptors are the control-behavior layer around MCP tool execution. They wrap tool calls in sequence, so each interceptor can examine the incoming request before the tool runs and inspect or transform the result afterwards. They can also stop execution before the tool is called at all. In practice, this gives the host application a place to enforce authorization, approval rules, rate limits, tenant scope, input normalization, output normalization, and runtime-aware behavior around MCP-backed tools. In the current adapter surface, interceptors can inspect runtime context, modify requests, return `ToolMessage` responses, and return `Command` objects that participate in graph execution by updating state or affecting control flow. That is the point where MCP integration stops looking like a thin RPC bridge and starts behaving like a first-class part of the agent runtime.

2.3.9 LangGraph can also be published through MCP

Up to this point, the chapter has looked at LangChain and LangGraph as MCP clients: they connect to MCP servers, discover capabilities, and use those capabilities as tools. The same architecture can also work in the opposite direction. A LangGraph deployment can itself be exposed through MCP, so that other MCP clients can discover and invoke it. In that setup, LangGraph Agent Server exposes an MCP endpoint at `/mcp` over `Streamable HTTP`. The deployed agent is presented outward as an MCP tool. Its published name and description become the tool metadata that clients see, and its declared input schema becomes the MCP input schema for that tool. In other words, the graph is no longer the consumer of external MCP capabilities. It becomes the capability being published. That reversal is architecturally important because it shows that MCP is not limited to connecting agents to outside tools. It can also be used to make an agent or graph available to other systems through the same protocol surface. This is also the point where schema discipline matters again. When a graph is published through MCP, the exposed contract should stay narrow and intentional. The client should see a clear input and output shape for the capability being offered, not the graph's full internal state or message history. That keeps the external interface stable, understandable, and easier to use safely.

At the moment, the `/mcp` endpoint provided by Agent Server is stateless, so each request is handled independently. Authentication is aligned with the rest of the LangGraph API, rather than defined as a separate MCP-specific mechanism. Please note, to run this example you need to install:

```
(.venv) C:\Users\alexc\llm-app> python -m pip install -U "langgraph-cli[inmem]"
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class InputState(TypedDict):
    question: str

class OutputState(TypedDict):
    answer: str

class OverallState(InputState, OutputState):
    pass

def answer_node(state: InputState) -> dict:
    question = state["question"]
    return {
        "answer": f"You asked: {question}. This response came from a LangGraph app
                    exposed through MCP."
    }

builder = StateGraph(
    OverallState,
    input_schema = InputState,
    output_schema = OutputState
)

builder.add_node("answer_node", answer_node)
builder.add_edge(START, "answer_node")
builder.add_edge("answer_node", END)

graph = builder.compile()
```

vol3_ch_02\src\langgraph_agent.py

The graph state is configured to separate the external contract from the graph's full internal state. `InputState` defines the only field the caller must provide, while `OutputState` defines the only field the graph promises to return. `OverallState` combines both so the graph can carry input and produced output in one typed state object during execution. This matters because `StateGraph(...)` is being told exactly how to interpret state at each boundary: `input_schema=InputState` constrains what enters the graph, and `output_schema=OutputState` constrains what leaves it. The result is a narrow, explicit state model that keeps execution predictable, prevents accidental state exposure, and makes the graph easier to validate and publish safely.

```
{
  "dependencies": [".",],
  "graphs": {
    "faq_agent": {
      "path": "./src/langgraph_agent.py:graph",
      "description": "Answers one question with a short text response."
    }
  }
}
```

Vol3_ch_02\langgraph.json

This file defines the deployable surface of the application. The `dependencies` entry tells the runtime which local package or project directory must be available when loading the graph, so the server can resolve the referenced module correctly. The `graphs` section then publishes a named graph, `faq_agent`, by pointing to the compiled graph object at

`./src/langgraph_agent.py:graph`. That mapping is the important part: it turns an internal Python object into a registered application entry point. The description adds human-readable metadata, which is useful when the graph is exposed outward, for example through discovery surfaces such as MCP. Now start the MCP server with:

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_02> langgraph dev

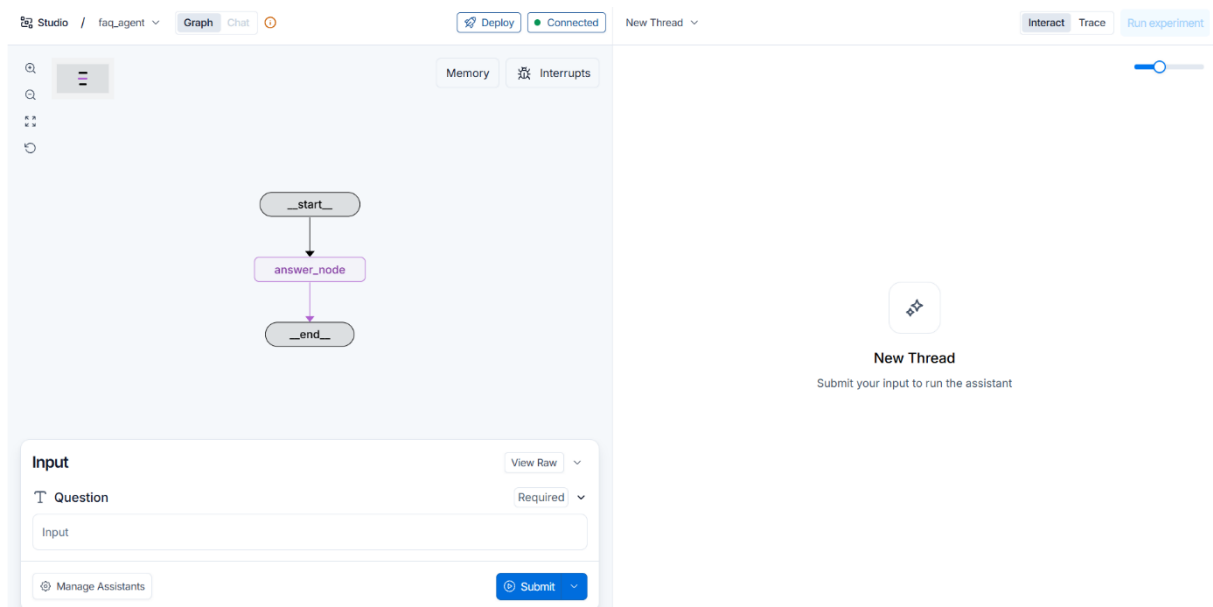
INFO:langgraph_api.cli:
    Welcome to

LANGGRAPH

- 🚀 API: http://127.0.0.1:2024
- 🎮 Studio UI: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
- 📖 API Docs: http://127.0.0.1:2024/docs

This in-memory server is designed for development and testing.
For production use, please use LangSmith Deployment.

2026-03-17T13:33:58.202507Z [info      ] Starting dev persistence flush loop
[langgraph_runtime_inmem._persistence] api_variant=local_dev langgraph_api_version=0.7.76
thread_name=MainThread
2026-03-17T13:33:58.225611Z [info      ] Using
langgraph_runtime_inmem [langgraph_runtime] api_variant=local_dev
langgraph_api_version=0.7.76 thread_name=MainThread
2026-03-17T13:33:58.252086Z [info      ] Using auth of
type=noop [langgraph_api.auth.middleware] api_variant=local_dev
langgraph_api version=0.7.76 thread_name=MainThread
2026-03-17T13:33:58.263004Z [info      ] Starting In-Memory runtime with langgraph-
api=0.7.76 and in-memory runtime=0.26.0 [langgraph_runtime_inmem.lifespan]
api_variant=local_dev langgraph_api_version=0.7.76 langgraph_runtime_inmem_version=0.26.0
thread name=asyncio 0 version=0.7.76
2026-03-17T13:33:58.263200Z [info      ] 1 change detected [watchfiles.main]
api_variant=local_dev langgraph_api_version=0.7.76 thread_name=Main
.
.
```



```
import asyncio
from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

SERVER_URL = "http://localhost:2024/mcp"

async def main() -> None:
    async with streamablehttp_client(url = SERVER_URL) as (read, write, _):
```

```

async with ClientSession(read, write) as session:
    await session.initialize()

    tools_result = await session.list_tools()
    print("\n=== TOOLS ===")
    for tool in tools_result.tools:
        print(f"name={tool.name}")
        print(f"description={tool.description}")
        print(f"inputSchema={tool.inputSchema}")
        print()

    result = await session.call_tool(
        "faq_agent",
        {"question": "What are you?"}
    )

    print("\n=== TOOL RESULT ===")
    for item in result.content:
        print(item)

if __name__ == "__main__":
    asyncio.run(main())

```

vol3_ch_02\scr\langgraph_client.py

```

(.venv) C:\Users\alexc\l1m-app\vol3_ch_02> python -m src.langgraph_client

=== TOOLS ===
name=faq_agent
description=Answers one question with a short text response.
inputSchema={'properties': {'question': {'title': 'Question', 'type': 'string'}},
'required': ['question'], 'title': 'InputState', 'type': 'object'}

=== TOOL RESULT ===
type='text' text="{ 'answer': 'You asked: What are you?. This response came from a
LangGraph app exposed through MCP.'}" annotations=None meta=None

```

This MCP client opens a `Streamable` HTTP connection to the server's `/mcp` endpoint and then creates a session on top of that transport. The first meaningful step is `initialize()`, which establishes the protocol session before any capability can be used. It then calls `list_tools()` to inspect what the server exposes, printing each tool's name, description, and input schema. That inspection is important because it confirms the published contract the client can rely on. The client then invokes the `faq_agent` tool with a minimal structured payload. Finally, it iterates over `result.content`, showing the returned MCP content blocks rather than assuming a plain-text response.

2.3.10 What guarantee you really get from this layer

The practical guarantee of MCP is now easier to state precisely. It gives you standardized capability interfaces and standardized transport. It gives you a common way to describe tools, resources, and prompts, to discover them, and to invoke them across local and remote boundaries. In LangChain, `MultiServerMCPClient` turns that protocol surface into ordinary tools, resources, prompts, callbacks, and interceptors that the rest of the framework can use without speaking JSON-RPC directly. What it does not give you is semantic correctness, business validity, or universal sameness across hosts. Two different MCP-capable systems may both expose compatible tools and still differ in approval flow, execution policy, state handling, or user experience. MCP makes capability integration more portable. It does not make every runtime identical, and it does not remove the need for validation and policy in the host application. That is the correct way to place it in the larger stack: standardized interface, standardized transport, no guarantee of semantic or business correctness.

2.4 SECURITY: TOOL PERMISSIONING, ALLOWLISTS, SANDBOXING, SECRETS HANDLING

2.4.1 Security starts where controlled generation stops

Controlled generation helps you constrain shape. It can force a schema, tighten tool arguments, and make a model emit a tool call instead of prose. That is valuable, but it is not yet a security model. A structurally valid tool call can still target the wrong operation, use the wrong business context, or request an action the current user is not allowed to perform. In this stack, security lives in the runtime around the model: which capabilities are exposed, which arguments are hidden from the model, which calls require approval, which processes are isolated, and how credentials are supplied. LangChain's structured-output guidance is explicit that provider-native structured output improves reliability by enforcing schema, while tool calling constrains the action-request channel rather than validating the semantic or business correctness of the proposed action.

This is also the right place to separate MCP's three capability types, because they do not deserve the same permission model. In MCP, tools are model-controlled executable actions, resources are application-driven context objects identified by URI, and prompts are user-controlled templates intended for explicit selection. That distinction is more than taxonomy. If something should not be invoked automatically by the model, it probably should not be exposed as a tool in the first place. If it is just data to read, make it a resource. If it is a reusable interaction pattern, make it a prompt. A secure design starts by choosing the right primitive before you worry about code-level checks. A practical rule follows from that. An agent that can reach everything is not flexible; it is under-specified. The smallest safe tool surface is the one that exposes only the capabilities required for the current workflow, keeps high-risk actions separate from low-risk reads, and treats transport, identity, and approval as first-class concerns rather than afterthoughts. MCP itself standardizes discovery and invocation, but its own tools specification still requires servers to validate tool inputs, implement access controls, rate limit invocations, and sanitize outputs; on the client side it recommends confirmation for sensitive operations, validation of tool results, timeouts, and audit logging.

2.4.2 Tool permissioning begins with what the model is allowed to see

The first permission check happens before execution. It happens when you decide what tools to bind to a model or expose through an agent. If a support assistant only needs to read order status and create a refund request, it should not also receive tools for deleting files, running shell commands, editing inventory, or issuing arbitrary SQL. Once a tool is in the model's advertised tool set, the model is free to propose it. That is true for ordinary LangChain tools and for MCP-discovered tools alike. LangChain's tools are simply callable functions with explicit inputs and outputs passed to a chat model. The model decides when to invoke them and what arguments to provide. Security therefore starts with tool selection, not just tool validation. The second permission check is the schema itself. A tool schema is an allowlist of admissible arguments. Narrow schemas reduce ambiguity and reduce the attack surface. An `enum` is safer than an open string when only a few values are valid. A bounded integer is safer than "any number." A model should be asked to choose only the values it is genuinely allowed

to choose. Identity, tenant, plan, session, and backend authorization context usually do not belong in model-visible tool parameters at all. LangChain makes this separation explicit: runtime is a reserved parameter name for ToolRuntime, and ToolRuntime exposes state, immutable invocation context, long-term store access, the execution config, and the current tool-call identifier without turning those values into tool arguments the model can manipulate.

That design is one of the cleanest ways to implement permissioning. Let the model choose the business input it should choose, such as an order identifier or a refund reason. Let the runtime inject everything else, such as the authenticated user, role, tenant, or policy state. This keeps the tool contract honest: the model proposes an action within a narrow surface, while the runtime supplies the authority boundary.

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Dict
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langchain.tools import tool, ToolRuntime
from langgraph.checkpoint.memory import InMemorySaver
from .config import OPENAI_API_KEY

ORDERS: Dict[str, dict] = {
    "ORD-100": {"owner_id": "user-123", "status": "delivered", "amount": 129.0},
    "ORD-200": {"owner_id": "user-999", "status": "processing", "amount": 49.0}
}

AUTHORIZED_REFUND_ROLES = {"support_supervisor", "finance_ops"}

@dataclass
class SupportContext:
    user_id: str
    role: str

@tool
def lookup_order(order_id: str, runtime: ToolRuntime[SupportContext]) -> str:
    """Return the current user's order details."""
    order = ORDERS.get(order_id)
    if not order:
        return f"Order {order_id} was not found."

    # Permission check uses runtime context, not model-supplied arguments.
    if order["owner_id"] != runtime.context.user_id:
        return "Access denied: you can only inspect orders for the current user."

    return (
        f"order_id={order_id}, status={order['status']}, amount={order['amount']}"
    )

@tool
def submit_refund_request(
    order_id: str,
    reason: str,
    runtime: ToolRuntime[SupportContext]
) -> str:
    """Create a refund request for an eligible order."""
    order = ORDERS.get(order_id)
    if not order:
        return f"Order {order_id} was not found."

    if runtime.context.role not in AUTHORIZED_REFUND_ROLES:
        return "Refund submission denied: current role is not allowed to submit refunds."

    if order["owner_id"] != runtime.context.user_id:
        return "Refund submission denied: order does not belong to the current user."

    if order["status"] != "delivered":
        return "Refund submission denied: only delivered orders can enter refund review."

    # In a real system, call the refund service here.
    return f"Refund request created for {order_id} with reason='{reason}'."

agent = create_agent(
    model = "gpt-4.1",
    tools = [lookup_order, submit_refund_request],
    context_schema = SupportContext,
```



```

checkpointer = InMemorySaver(),
middleware = [
    HumanInTheLoopMiddleware(
        interrupt_on = {
            "lookup_order": False,
            "submit_refund_request": {
                "allowed_decisions": ["approve", "edit", "reject"]
            }
        }
    )
]
system_prompt = (
    "You are a customer support assistant. "
    "Use tools when needed, but do not invent permissions or policy outcomes."
)
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Please refund order ORD-100 because the customer changed
                    their mind."
            }
        ],
        context = SupportContext(user_id = "user-123", role = "support_supervisor"),
        config = {"configurable": {"thread_id": "support-case-001"}}
    )

print("\nMESSAGES:")
for m in result["messages"]:
    print(f"\n- {type(m).__name__}")
    if getattr(m, "content", None):
        print("  content:", m.content)
    if getattr(m, "tool_calls", None):
        print("  tool_calls:", m.tool_calls)
    if getattr(m, "name", None):
        print("  name:", m.name)

interrupts = result.get("__interrupt__", [])
if interrupts:
    print("\nINTERRUPT:")
    for intr in interrupts:
        print("  pending action:", intr.value["action_requests"][0]["name"])
        print("  args:", intr.value["action_requests"][0]["args"])
        print("  allowed decisions:",
            intr.value["review_configs"][0]["allowed_decisions"])

```

vol3_ch_02\scr\agent_security.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_02> python -m src.agent_security

MESSAGES:

- HumanMessage
  content: Please refund order ORD-100 because the customer changed their mind.

- AIMessage
  tool_calls: [{'name': 'lookup_order', 'args': {'order_id': 'ORD-100'}, 'id':
'call_p8nG3RsYJGEg15wxz2FWUBWV', 'type': 'tool_call'}]

- ToolMessage
  content: order_id=ORD-100, status=delivered, amount=129.0
  name: lookup_order

- AIMessage
  tool_calls: [{'name': 'submit_refund_request', 'args': {'order_id': 'ORD-100',
'reason': 'Customer changed their mind'}, 'id': 'call_ZXUvx1WpE9uwr92Xes046Jyx', 'type':
'tool_call'}]

INTERRUPT:
  pending action: submit_refund_request
  args: {'order_id': 'ORD-100', 'reason': 'Customer changed their mind'}
  allowed decisions: ['approve', 'edit', 'reject']

```

This example does three security-relevant things at once. It exposes only two tools. It keeps `user_id` and `role` outside the tool schema by reading them from `runtime.context`. And it treats the write-like operation as approval-worthy by pausing before execution. LangChain's human-in-the-loop middleware is built precisely for this pattern: it can interrupt selected tool calls, persist graph state through the checkpoint, and then resume with an approve, edit, or reject decision. That is a much safer pattern than asking the model to "be careful" in prose.

2.4.3 Allowlists

Allowlists are not a single list of approved tool names. In real systems, they exist at several distinct layers, and each layer answers a different question.

A discovery allowlist controls what the agent can even see: which tools, resources, prompts, or entire MCP servers are exposed to the model at all. An execution allowlist controls which of those visible capabilities may actually run for this caller, role, tenant, or session. An argument allowlist constrains which inputs are structurally acceptable through schemas, enums, bounds, and patterns. A transport or credential allowlist constrains what the underlying MCP connection, token, or server-side identity is permitted to reach. These layers should reinforce each other. If a sensitive capability is hidden early, the later layers have less work to do. If it must remain visible, the later layers must be strict enough to prevent accidental or unauthorized execution.

`ToolNode` centralizes tool execution inside the graph but allowlist policy should be enforced before and around that execution boundary. Tool visibility, caller authorization, tenant scope, and approval requirements should be decided through discovery rules, runtime checks, middleware, interceptors, or wrapped tool implementations. `ToolNode` gives the workflow a disciplined place to run tools; it should not be treated as the policy engine.

That distinction matters even more when the model can propose multiple tool calls in the same turn. `ToolNode` may execute those calls concurrently inside one graph node, so any state updates they return still need explicit reducer rules. Once you allow concurrent tool activity, reducer design stops being an implementation detail and becomes part of your correctness, security, and audit model. Fields such as audit trails, approval markers, counters, warning lists, retrieved records, and intermediate tool outcomes must have deterministic merge behavior. Otherwise, concurrent updates can silently overwrite one another, and the system may lose exactly the evidence it needs during incident analysis.

For MCP-based agents, per-user and per-role allowlists often belong at two boundaries. The first boundary is discovery, where the application hides tools, resources, prompts, or entire servers that the model should not consider for the current workflow. The second boundary is execution, where interceptors or wrapped tools re-check the caller's role, arguments, credentials, approval state, tenant scope, and workflow context before the remote capability runs. Current LangChain MCP adapters document interceptors as the place where you can access runtime context, including state, config, store, and tool-call metadata, while also modifying requests, adding headers, implementing retries, short-circuiting execution, returning `ToolMessage` responses, or returning `Command` objects that update state or redirect graph flow.

That makes interceptors the right enforcement point for checks such as “this tool is visible but not executable for this user,” “this argument set exceeds the caller’s scope,” or “this action must terminate early with a refusal instead of reaching the remote server.” In other words, the model may still reason over capabilities at a high level, but the final decision to let a remote action proceed should be enforced in code at the interceptor boundary, not entrusted to the model’s judgment. The practical rule is simple: apply allowlists as early as possible, and then re-check them at execution time. Hide what should not be discoverable. Validate what may be called. Enforce caller-specific authorization before the tool runs. Constrain transport credentials so that even a mistaken call cannot exceed its real scope. That is how allowlists stop being a decorative control and become a real containment mechanism.

```
ALLOWED_TOOLS_BY_ROLE = {
    "support_agent": {"lookup_order", "track_shipment"},
    "support_supervisor": {"lookup_order", "track_shipment", "submit_refund"}
}

async def enforce_tool_policy(request: MCPToolCallRequest, handler):
    runtime = request.runtime
    role = runtime.context.role
    allowed = ALLOWED_TOOLS_BY_ROLE.get(role, set())

    if request.name not in allowed:
        return ToolMessage(
            content = f"Tool '{request.name}' is not permitted for role '{role}'.",
            tool_call_id = runtime.tool_call_id
        )

    # Inject transport credentials at execution time instead of exposing
    # them as tool arguments visible to the model.
    request = request.override(
        headers = {"Authorization": f"Bearer {runtime.context.access_token}"}
    )

    return await handler(request)

async def main() -> None:
    client = MultiServerMCPClient(
        {
            "orders": {
                "transport": "stdio",
                "command": "python",
                "args": ["../orders_mcp_server.py"]
            },
            "shipping": {
                "transport": "http",
                "url": "https://shipping.example.com/mcp"
            }
        },
        tool_interceptors = [enforce_tool_policy]
    )

    tools = await client.get_tools()
```

This is a practical allowlist pattern for MCP. The model may discover several tools, but the interceptor still enforces a second gate using the caller’s runtime role. It also injects the bearer token at the transport layer instead of asking the model to carry it as part of the tool arguments. LangChain’s MCP documentation explicitly supports both static headers and dynamic request modification through interceptors, and it shows the same interceptor surface being used for authentication checks, state-dependent tool filtering, rate limiting, and logging.

2.4.4 Sandboxing

Sandboxing is what prevents “a tool can run” from turning into “the host can be damaged.” Some tools do more than read business data. They can run code, read or write files, start local processes, or make network requests. Once a tool has that kind of power, ordinary permission

checks are no longer enough. You also need isolation. That is what sandboxing does. It puts the tool inside a restricted environment so that, even if the tool is misused, the damage is contained. This matters especially for local MCP servers. A local MCP server is not just a harmless connector. It is code that runs on the same machine as the client. Without sandboxing and explicit user approval, it can become a path to arbitrary code execution. The MCP guidance is very clear on this point. If a client offers a simple way to install or run a local MCP server, it should show the exact command that will be executed, require explicit approval, and let the user cancel. It also recommends running local servers with minimal privileges, restricting file-system and network access by default, granting extra privileges only when they are truly needed, and using modern sandboxing mechanisms. When possible, it recommends `stdio` for local servers, because that limits exposure to the MCP client itself. If HTTP is used instead, stronger local restrictions are recommended.

This is the practical point: a local MCP server may have real access to your machine. It may be able to read local files, launch programs, or connect to external systems. If you would not run an unknown shell script directly on the host, you should not treat a local MCP server as safe simply because it uses MCP. The protocol standardizes how the client and server communicate. It does not remove the risk of what the server can do once it is running. The same lesson appears in LangChain guidance for agents that execute code. The advice is straightforward: block network access when it is not required, review sandbox outputs before acting on them, treat everything produced inside the sandbox as untrusted, and avoid putting secrets into the sandbox at all. If secrets absolutely must be available, the guidance recommends strong compensating controls such as human approval, narrowly scoped and short-lived credentials, restricted network access, and monitoring for unexpected outbound traffic. Even then, this is described as a risky workaround, not as the preferred design. Sandboxing is not an optional extra for later hardening. It is part of the basic design whenever a tool can run code, access the local machine, or communicate freely over the network. Controlled generation can help ensure that the model produces well-formed tool calls. It cannot stop an unsandboxed process from reading sensitive files, sending data out, or changing the host after you have allowed that process to run.

2.4.5 Secrets handling

Secrets handling becomes much cleaner once you adopt one discipline: anything the runtime already knows should stay out of the model-visible channel unless the model truly needs to reason about it. That means API keys, bearer tokens, service credentials, tenant identifiers, internal user IDs, and backend-only authorization data should not be ordinary tool arguments when the runtime can inject them through context, store access, or transport configuration. LangChain's tool runtime exists precisely so tools can read immutable invocation context and persistent store data without forcing those values into the tool schema. The MCP adapter layer extends the same pattern through interceptors, which can inject headers or other execution-time data right before the call leaves the client.

For HTTP-based MCP transports, the protocol's authorization story is transport-level and OAuth-based. The authorization specification defines OAuth 2.1-style flows for HTTP transports, requires Protected Resource Metadata for authorization-server discovery, and makes the client discover the server's authorization setup through standard OAuth metadata.

For `stdio`, the guidance is the opposite: do not use the HTTP authorization flow there; retrieve credentials from the environment instead. LangChain's MCP client reflects this design. It supports static headers, custom `httpx.Auth` implementations, and runtime header injection through interceptors. That gives you several ways to keep credentials out of prompts and out of model-chosen arguments.

The MCP security guidance also calls out one anti-pattern that deserves to be named plainly: token passthrough. In that anti-pattern, an MCP server accepts tokens from a client and forwards them to a downstream API without validating that the tokens were actually issued for that MCP server. The guidance forbids it because it bypasses security controls, weakens accountability, and breaks trust boundaries. The secure pattern is brokered access: the MCP server authenticates and authorizes as itself, or with tokens explicitly issued for it, rather than blindly proxying whatever credential the client happened to supply.

Least-privilege scope design is the other half of secrets handling. The MCP security guidance recommends a progressive scope model: start with a minimal initial scope set for low-risk discovery and read operations, then elevate only when a privileged operation is actually attempted. It explicitly warns against publishing all possible scopes, using wildcard scopes, bundling unrelated privileges, or treating "claimed scopes in the token" as a complete substitute for server-side authorization logic. That is the right mental model for agents. A token is not a permission strategy by itself. It is only one input into the permission strategy.

One final operational point matters here. Logs are necessary, but secret-bearing logs are an incident waiting to happen. The safer default is to log the tool call identifier, server name, tool name, decision, timing, and outcome, while redacting credentials and minimizing sensitive payloads. LangChain's tool runtime exposes a `tool_call_id` precisely so tool execution can be correlated cleanly, and the MCP tools guidance explicitly recommends audit logging and result validation. That is enough to reconstruct what happened without turning your observability stack into a second secret store.

2.5 SUMMARY

This chapter separated several guarantees that are often blurred together. JSON mode constrains syntax. Structured output constrains the shape of final answers. Function calling constrains the shape of action proposals. `ToolNode` provides a graph-side execution boundary for tool calls that have already been proposed. MCP standardizes how hosts discover and invoke external capabilities. These are useful guarantees, but they are mostly structural and architectural. They do not prove that an answer is true, that a tool call is safe, or that an operation should be approved.

The practical design rule is to place each control at the right boundary. Use structured output when the application needs typed data rather than prose. Treat function calls as requests, not authority. Let `ToolNode` centralize tool execution, while reducers, `ToolMessage` objects, and `tool_call_id` preserve valid graph state and valid message history. Use MCP when capabilities need to be exposed, discovered, or reused across host applications, but keep validation, approval, authorization, and business policy in the host system.

The chapter also showed why production reliability depends on the software around the model. Tool schemas should expose only model-choosable arguments. Trusted identity, tenant scope, credentials, stores, and runtime configuration should come from runtime context or transport configuration, not from the prompt. Side-effecting tools need stronger controls than read-only tools. Parallel tool execution requires deliberate reducer design. MCP-backed tools need allowlists, interceptors, sandboxing, least-privilege credentials, and audit logging. Controlled generation improves the interface. Reliable systems still come from explicit state, policy, validation, observability, and disciplined execution boundaries.

Part 2 - Agentic Design Patterns

Chapter 3: Planning, Routing, and Parallelization

Chapter 4: Reflection, Reasoning, and Adaptation

Chapter 5: Multi-Agent Collaboration

3 AGENTS PLANNING, ROUTING, AND PARRALLELIZATION

An agentic system becomes useful not merely when it can call a model or a tool, but when it can organize work, choose the appropriate execution path, and coordinate independent operations. Planning, routing, and parallelization are the control-flow patterns that make those decisions explicit. They determine whether a task should be decomposed before execution, whether the next step is fixed or selected from runtime state, and whether several branches can run concurrently before their results are combined. These patterns solve different problems and should not be collapsed into one generic idea of autonomy. Planning gives multi-step work an explicit structure and separates planning, execution, and synthesis responsibilities. Routing selects one or more destinations from the current request and workflow state. Parallelization reduces elapsed time by running genuinely independent work concurrently, but it also requires explicit synchronization, state-update semantics, failure policies, and fan-in rules. Used together, these patterns form the execution architecture of many practical LangChain and LangGraph applications.

The chapter begins with plan-and-solve and related decomposition strategies. It distinguishes reactive execution from upfront planning, fixed decomposition from runtime decomposition, and lightweight task planning from a dedicated planner-executor graph. It then examines structured handoffs and shows why planning is valuable only when the additional stages make a task easier to execute, inspect, or control. The second part turns to routing. It compares rule-based, LLM-based, supervised-classifier, embedding-based, and hybrid strategies, while keeping route selection separate from authorization and business-policy enforcement. The chapter then develops parallel fan-out and fan-in through map-reduce, voting, peer-evaluated selection, consensus, and structured aggregation. Finally, it combines planning, routing, retrieval, tools, and parallel execution in one workflow and closes with the cost and latency controls that keep the design practical: tiered models, caching, early exits, explicit budgets, and continuous evaluation.

3.1 PLAN-AND-SOLVE AND DECOMPOSITION STRATEGIES (WHEN PLANNING HELPS VS HARMS)

Planning is best understood as a way of reshaping work before execution begins. Instead of asking one model call to understand the request, choose the right strategy, decide which tools matter, gather evidence, and draft the final answer all at once, a planning-oriented design splits that burden into smaller parts. In LangGraph, this broader design space is usually discussed through workflow and agent patterns. Workflow patterns include prompt chaining for well-defined sequential steps, routing for branch selection, parallelization for independent branches, orchestrator-worker for input-dependent decomposition, and evaluator-optimizer loops for iterative refinement. Plan-and-solve belongs to the decomposition side of this landscape, but it should be described carefully. In its narrow research meaning, Plan-and-Solve is a prompting strategy. In application architecture, the same idea often appears as a plan-and-execute workflow or agent design. This section explains when that extra structure helps, when it becomes overhead, and how to choose the smallest decomposition strategy that makes the work easier to execute and inspect.

3.1.1 Reactive loops versus upfront planning

A useful contrast is between observation-dependent reasoning and upfront planning. In a reactive loop, the model is prompted with the current task state, proposes an action, receives a tool observation, and then uses the updated state to choose the next step. The path emerges one step at a time. Older LangChain material and examples often described this style through action agents and the legacy Zero-Shot ReAct setup, such as AgentType.ZERO_SHOT_REACT_DESCRIPTION. In this style, reasoning and action are tightly coupled to the latest observation. The system does not commit to a full plan first. It keeps revising its next move as new information arrives.

Plan-and-solve takes a different approach. It tries to make the structure of the work explicit before the solving phase begins. In an agent implementation, that solving phase may involve tool calls. In the narrow prompting pattern, it may simply involve solving each planned subtask in text. The narrow meaning of the pattern comes from the Plan-and-Solve Prompting paper. The method first asks the model to create a plan by dividing the task into smaller subtasks. It then asks the model to carry out those subtasks according to the plan. The paper targets a specific failure mode: zero-shot chain-of-thought prompting can miss necessary reasoning steps. It also discusses other common errors, including calculation errors and semantic misunderstanding errors. Plan-and-Solve is aimed mainly at missing-step failures, while PS+ adds more detailed instructions to improve calculation quality and the quality of the generated reasoning steps.

For that reason, plan-and-solve should not be used as a loose synonym for any multi-step agent. It is a decomposition-first strategy. Its value is that it makes the structure of the task visible before the system starts solving it. In practical terms, the difference is simple. Reactive reasoning is a good fit when the next action genuinely depends on fresh observations at each turn. Upfront planning is a good fit when the main structure of the work can be known early and made explicit. For a support assistant handling a standard refund request, the sequence is often stable: identify the order, gather the order facts, evaluate the refund policy, and draft the

reply. That is a good candidate for upfront planning because the major stages are known before execution starts.

3.1.2 Planner-Executor-Solver as a mental model

One useful way to implement a plan-and-solve or plan-and-execute design is to separate the system into three responsibilities: planning, execution, and final synthesis. First, a planner decides how the work should be broken down. It does not try to solve the whole problem immediately. Its job is to produce a sequence of clear, executable steps that make the task easier to run and inspect. Next, an executor carries out those steps. In some architectures, this executor is implemented as one or more worker nodes. Depending on the application, the execution layer may retrieve information, call tools, run deterministic checks, invoke smaller models, or perform other step-specific work. In a customer-support assistant, one execution step might retrieve the order record, another might retrieve the refund policy, and another might check whether the customer is still inside the return window. Finally, a solver, or synthesis step, returns to the broader task. It receives the original request, the plan, and the execution results, then produces the final answer or final decision. In the refund example, the synthesis step would not simply repeat the retrieved facts. It would combine the order details, policy result, and conversation context into a clear response for the customer.

This separation is useful because each part has a different responsibility and a different failure mode. Planning is about structure. Execution is about doing the planned work. Synthesis is about combining the results into a coherent outcome. When those responsibilities are kept separate, the system becomes easier to reason about. You can inspect the plan before execution starts, examine the input and output of each execution step, and see which evidence or intermediate results were available to the final synthesis step. That makes failures easier to diagnose. If the final answer is wrong, you can ask a precise question: was the plan incomplete, was a step executed incorrectly, was the evidence weak, or was the final synthesis poor? This is much better than debugging one large prompt where planning, retrieval, policy interpretation, and answer writing all happened inside the same opaque generation.

The basic execution pattern is straightforward, and that simplicity is one of its strengths. First, generate a plan. Second, execute the planned steps, usually one by one, unless the plan contains independent steps that can safely run in parallel. Third, synthesize the final answer from the original task, the plan, and the accumulated results. The value is not that this introduces a complicated reasoning theory. The value is that it gives the system an explicit execution shape that can be traced, tested, and constrained. This structure also creates a practical architectural boundary between planning, execution, and synthesis. Because those roles are separate, they do not have to use the same component. The planning stage may justify a stronger model when the task is complex, ambiguous, or safety-sensitive enough that a poor decomposition would cause downstream errors. The execution stage may use deterministic code, narrow tools, retrieval calls, or smaller models when the step is well-scoped enough to avoid a more expensive general-purpose model. The final synthesis stage may use a stronger model when the output requires careful consolidation or nuanced wording. For routine or compliance-sensitive outputs, a smaller model, a constrained structured-output call, or a deterministic template may be better. That flexibility is one of the main reasons this model is useful in real systems. It lets you spend model capacity where judgment is needed, use

deterministic code where precision is required, and keep the entire execution path visible enough to debug.

3.1.3 Fixed decomposition, dynamic decomposition, and lighter planning

Not every decomposition strategy solves the same problem. The first distinction to make is between work whose structure is already known and work whose structure has to be derived while the system is running. That difference determines whether a fixed sequence is enough or whether the workflow needs a more flexible orchestration shape. The simplest case is fixed decomposition. Here, the task can be broken into a stable sequence of smaller steps, and each step passes its result to the next one. This is the logic behind prompt chaining. It is decomposition, but it is not necessarily planning in the agentic sense because the path is already designed by the developer. In a fixed chain, the main design question is not “what should happen next?” but “how should we divide the work so that each step is easier and safer than the original request?” That makes fixed decomposition a strong fit for tasks where the sequence rarely changes and intermediate outputs can be validated cheaply. In real systems, this is often the most practical starting point because it gives you clarity without introducing much orchestration overhead. A customer-support assistant gives a simple example. If the task is to answer a standard refund question, the workflow may be stable: classify the request, retrieve the order, retrieve the refund policy, check eligibility, and draft the reply. There is no need for a model to invent a new process each time. The process is part of the application design.

A more dynamic case needs a different structure. Sometimes the right subtasks cannot be selected confidently before the input has been inspected. In that situation, a static chain can become brittle. It may include steps that are not needed, or miss steps that only become obvious after the request has been examined in more detail. This is where an orchestrator-worker workflow often becomes more appropriate. Instead of following one predefined chain, the orchestrator derives the subtasks needed for this case, dispatches them to worker nodes or worker executions, and then makes their outputs available for synthesis, often through shared graph state. The important idea is that decomposition is no longer fixed ahead of time. The workflow still has a designed orchestration pattern, but the specific subtasks are derived from the input at runtime. For example, an e-commerce assistant asked to prepare a product comparison may not know upfront how many products, criteria, or data sources are relevant. The orchestrator might dispatch one worker execution to retrieve product specifications, another to collect review summaries, another to check availability, and another to compare prices. A different request may need a different set of subtasks. The orchestration pattern is stable; the work assigned inside it is dynamic. That conceptual boundary is important. Plan-and-solve and plan-and-execute designs are strongest when enough of the task structure can be identified early to guide execution. Once the number, type, or order of subtasks depends heavily on the input, the problem shifts from “follow this predefined sequence” to “derive the right task structure for this case.” At that point, a dynamic orchestrator is usually a better fit than a rigid staged chain. There is also a useful middle ground between a single prompt and a fully custom graph. Some tasks are clearly multi-step but not complex enough to justify a dedicated planner-executor workflow. In those cases, lightweight planning can still add value, even if it is not a separate planning architecture.

A to-do style planning step can help the system break a larger objective into trackable tasks, keep progress visible, and avoid losing the thread across a long sequence of tool calls or intermediate steps. This does not create a fully separate planning architecture, but it still improves coordination by making the work explicit. A similar idea appears in hierarchical API interaction, especially with large OpenAPI-style specifications. When the action space is large, it is often inefficient to give one model call the full detailed specification for every endpoint and operation at once. A lightweight planning or routing layer can first select the relevant part of the API, and a narrower execution layer can then work only with the detailed information for that selected slice. In that design, planning is being used as an engineering tool to reduce confusion, control context size, and improve coherence over long tool-using sequences.

Seen together, these three forms create a useful spectrum. Fixed decomposition works when the sequence is stable. Dynamic decomposition works when the subtasks must be derived from the input. Lightweight planning helps when the task is clearly multi-step but not complex enough to justify a dedicated planner-executor graph. The next sections explain what makes these designs reliable in practice and when the extra structure is worth the cost.

3.1.4 Structured handoffs are part of the design, not an afterthought

A decomposition strategy only works if the handoff between stages is reliable. This is where many otherwise sensible designs fail. If the planner produces vague steps, or if worker nodes emit free-form text where the next stage expects precise fields, downstream stages inherit uncertainty they may not be able to resolve. The result is a workflow that looks well organized from the outside but has weak contracts between stages. That is why intermediate outputs should be treated as interface contracts in staged designs. When one stage hands off a parsed, schema-shaped result, the next stage receives a stable data shape rather than an ambiguous paragraph. In practice, that shape may come from schema-constrained JSON, a Pydantic model, a dataclass, or a TypedDict-style schema. The structure does not prove the content is correct, and some schema forms still require application-side validation, but structured handoffs make the content easier to check and use safely. A weak intermediate output does not stay isolated inside the step that produced it. The next step consumes it, and the error can spread through the rest of the workflow. For example, a refund workflow should not pass a paragraph such as “the customer probably qualifies.” It should pass fields such as `order_id`, `purchase_date`, `delivery_date`, `policy_version`, `eligibility_status`, `reason`, and `required_approval`. Those fields can be checked. A vague sentence cannot be checked with the same reliability.

3.1.5 When planning helps

Planning is most useful when the task is genuinely multi-step and each smaller step is easier to execute, validate, or correct than the original request. If one model call must understand the request, choose a strategy, decide which tools to use, gather information, apply rules, and write the final answer, the call is carrying several different responsibilities at once. Planning can improve reliability by separating those responsibilities. The workflow may take more time because it involves several steps instead of one, but each step becomes narrower, easier to validate, and easier to correct. In practical terms, planning helps most when the system’s main failure mode is that it skips necessary steps or jumps too quickly to an answer.

Planning can also help when the system has a large action surface: many tools, many possible branches, or many business rules that could apply. At that point, the problem is not only whether the model can reason well. The workflow must also narrow the task to the relevant steps and tools. A planning stage can reduce that decision space before execution begins. Instead of asking the system to handle everything at once, the planner proposes the likely steps, and the execution layer carries out the relevant work after any required validation or policy checks. In long-running tasks, even a lightweight task list can provide value because it helps the system track progress and stay aligned with the original objective. In that sense, planning is not only about improving reasoning. It is also about giving the system a clearer operational structure.

Planning is especially helpful when you need observability and control. An explicit plan can create checkpoints that do not exist in a one-shot answer, provided the system records the plan, intermediate inputs, intermediate outputs, and final synthesis. You can inspect the planned task structure, the evidence collected, the intermediate decisions made by the workflow, and the inputs used to produce the final result. That makes evaluation easier and debugging much faster. In a support assistant, there is a big difference between saying “the model gave a poor answer” and saying, “the planner failed to include the policy-evaluation step.” The first is vague. The second points to a concrete defect in the workflow. This is one of the strongest practical reasons to use planning in production systems. A planned workflow can make the internal process more visible. That visibility helps you measure quality, isolate failures, and improve the system step by step. When the task is complex enough to justify that structure, planning often becomes the simplest way to make the workflow understandable, testable, and safe to operate.

3.1.6 When planning hurts

Planning becomes a problem when the structure you add is heavier than the task itself. A multi-step design can improve quality, but it usually adds latency, orchestration complexity, and more places where something can fail. It may also increase token usage, although careful designs can sometimes offset that by using smaller models or deterministic code for execution steps. That trade-off only makes sense when the task is difficult enough, risky enough, or important enough to justify the extra structure. A good default is to start with the simplest design that can solve the problem reliably, unless the task has known risk, audit, or control requirements that demand a more explicit workflow from the beginning. Extra orchestration becomes worthwhile when the simpler baseline fails in clear, repeatable ways. Fixed planning also hurts when the task does not have a stable shape. A fixed plan works well when the same kinds of steps are usually needed in roughly the same order. Some requests do not behave like that. Their structure changes from case to case, and the right path cannot be selected confidently before the input has been inspected or some observations have been collected. In those situations, a static plan can become brittle: it may force unnecessary work, miss an important branch, or commit too early to the wrong sequence. At that point, a routed workflow, dynamic orchestrator-worker pattern, re-planning loop, or reactive loop may fit better. It is also important not to expect planning to fix every reasoning failure. The original Plan-and-Solve prompting work was designed mainly to reduce missing-step errors in zero-shot chain-of-thought reasoning. That is a real benefit, but it does not remove other failure

modes. PS+ adds more detailed instructions to improve calculation quality and reasoning-step quality, but planning still does not guarantee that the model has understood the task correctly. A model can still misunderstand the user’s intent, misread domain language, or choose the wrong interpretation of the problem before planning even begins. When that happens, the resulting plan may look well formed, but it is solving the wrong problem. The value of structure, in that case, is that it can make the mistake easier to see earlier. The structure does not eliminate the misunderstanding itself. Finally, planning becomes overhead when the job is simple, direct, or highly conversational. Some requests do not need a planner, an executor, and a synthesis stage. They need a fast and competent response. A short support reply, a direct catalog lookup, or a simple order-status question often falls into that category: the user asks one thing, the system retrieves or computes one answer, and there is little benefit in planning several stages. For tasks like these, adding an explicit planning layer usually slows the system down more than it improves the result. Planning earns its keep on tasks that are genuinely multi-step, tool-heavy, risk-sensitive, or reasoning-intensive. Outside that range, it can add ceremony rather than value.

3.1.7 A practical rule for choosing decomposition

The practical rule is simple: use the smallest amount of decomposition that makes the task easier to execute, easier to inspect, and safer to trust. Use a fixed decomposed workflow when the sequence is stable. Use dynamic decomposition when the required subtasks, branches, or tools genuinely depend on the input. Use lightweight planning when the task is multi-step but not complex enough to justify a dedicated planner-executor graph. Use a reactive loop when the next step depends on fresh observations. Use deterministic code for rule-based, mechanical, or safety-critical work. The same rule applies across the recurring scenarios in this book. A support assistant benefits from decomposition when it must gather facts, evaluate policy, and draft a response. An e-commerce assistant benefits when it must clarify intent, retrieve candidates, compare options, and recommend a product. A trading assistant benefits when it must collect market facts, retrieve relevant news, check constraints, and consolidate a final view. The final test is whether the extra structure reduces ambiguity. If it does, decomposition is helping. If it only stands between a simple question and a simple answer, remove it.

Strategy	Use when	Avoid when
Fixed decomposition	The sequence is stable and intermediate outputs can be checked	The task shape changes significantly by input
Dynamic decomposition	The required subtasks depend on the input	A simple fixed chain is enough
Lightweight planning	The task is multi-step but does not justify a full planner-executor graph	The request is simple or conversational
Reactive loop	The next step depends on fresh observations	The task can be planned safely upfront
Deterministic step	The work is rule-based, mechanical, or safety-critical	The task requires language interpretation or synthesis

3.1.8 LangGraph implementation of Planner-Executor-Solver

The following example uses the support-assistant scenario because it shows the value of decomposition without drifting into the later routing and parallelization chapters. The graph has three explicit roles. The planner creates a structured plan. The executor executes one step at a time and stores structured evidence. The solver writes the final answer using only the accumulated state. That keeps the handoffs explicit and avoids the “free-form paragraph into free-form paragraph” failure mode that makes many staged workflows unreliable. The implementation uses current LangGraph-style stateful orchestration rather than older PlanAndExecute helpers, but the underlying Planner-Worker-Solver idea is the same.

```
from __future__ import annotations
import operator
import re
from typing import Annotated, Literal
from pydantic import BaseModel, Field
from typing_extensions import TypedDict
from langchain.messages import HumanMessage, SystemMessage
from langchain_openai import ChatOpenAI
from langgraph.graph import START, END, StateGraph

# Sample private data for the support assistant.
ORDERS = {
    .
    .
    .
}

class PlanStep(BaseModel):
    step_type: Literal[
        "ask for order id",
        "extract_order_id",
        "lookup_order",
        "evaluate_policy"
    ]
    description: str

class Plan(BaseModel):
    steps: list[PlanStep] = Field(description = "Ordered workflow steps")

class EvidenceItem(BaseModel):
    step_type: str
    data: dict
    note: str

class SupportState(TypedDict):
    user_request: str
    plan: list[dict]
    current_step: int
    order_id: str | None
    policy_decision: str | None
    final_answer: str | None
    evidence: Annotated[list[dict], operator.add]

planner_llm = ChatOpenAI(model = "gpt-4.1", temperature = 0).with_structured_output(Plan)
solver_llm = ChatOpenAI(model = "gpt-4.1", temperature = 0)

def planner(state: SupportState) -> dict:
    result = planner_llm.invoke(
        [
            SystemMessage(
                content=(
                    "You are the planner in a customer-support workflow. "
                    "If the request does not contain an order ID like ORD-100, "
                    "return a single step: ask_for_order_id. "
                    "Otherwise return these steps in order: "
                    "extract_order_id, lookup_order, evaluate_policy."
                )
            )
        ]
    )
    HumanMessage(content = state["user_request"])
    return {
        "plan": [step.model_dump() for step in result.steps],
        "current_step": 0,
        "evidence": []
    }
```

```

    }

def executor(state: SupportState) -> dict:
    step = state["plan"][state["current_step"]]
    step_type = step["step_type"]
    updates = {"current_step": state["current_step"] + 1}

    if step_type == "ask_for_order_id":
        item = EvidenceItem(
            step_type = step_type,
            data = {},
            note = "Missing order ID; user clarification required."
        )
        return {
            **updates,
            "evidence": [item.model_dump()],
            "final_answer": (
                "I can help with that, but I need the order ID first. "
                "Please send it in the form ORD-123."
            )
        }

    if step_type == "extract_order_id":
        match = re.search(r"ORD-\d{3,}", state["user_request"])
        order_id = match.group(0) if match else None
        item = EvidenceItem(
            step_type = step_type,
            data = {"order_id": order_id},
            note = "Extracted order identifier from the user request."
        )
        return {
            **updates,
            "order_id": order_id,
            "evidence": [item.model_dump()]
        }

    if step_type == "lookup_order":
        order = ORDERS.get(state["order_id"])
        item = EvidenceItem(
            step_type = step_type,
            data = {"order": order},
            note = "Loaded order facts from the order store."
        )
        return {
            **updates,
            "evidence": [item.model_dump()]
        }

    if step_type == "evaluate_policy":
        order = None
        for ev in state["evidence"]:
            if ev["step_type"] == "lookup_order":
                order = ev["data"]["order"]

        if order is None:
            decision = "Order not found."
        elif order["status"] != "delivered":
            decision = "Refund cannot start until delivery is confirmed."
        elif order["days_since_delivery"] > 30:
            decision = "Outside the 30-day damaged-item refund window."
        elif not order["damaged"]:
            decision = "Use the normal returns path instead of damaged-item handling."
        else:
            decision = "Eligible for damaged-item refund review."

        item = EvidenceItem(
            step_type = step_type,
            data = {"policy_decision": decision},
            note = "Applied the damaged-item refund policy to the order facts."
        )
        return {
            **updates,
            "policy_decision": decision,
            "evidence": [item.model_dump()]
        }

    raise ValueError(f"Unknown step type: {step_type}")

def should_continue(state: SupportState) -> str:
    if state.get("final_answer"):
        return "solver"
    if state["current_step"] < len(state["plan"]):

```

```

        return "worker"
    return "solver"

def solver(state: SupportState) -> dict:
    if state.get("final_answer"):
        return {}

    msg = solver_llm.invoke(
        [
            SystemMessage(
                content=(
                    "You are the solver in a customer-support workflow. "
                    "Write a concise customer reply using only the structured evidence. "
                    "Do not invent policy details."
                )
            ),
            HumanMessage(
                content=(
                    f"User request: {state['user_request']}\n"
                    f"Plan: {state['plan']}\n"
                    f"Evidence: {state['evidence']}\n"
                    f"Policy decision: {state['policy_decision']}"
                )
            )
        ]
    )
    return {"final_answer": msg.content}

builder = StateGraph(SupportState)
builder.add_node("planner", planner)
builder.add_node("worker", executor)
builder.add_node("solver", solver)

builder.add_edge(START, "planner")
builder.add_edge("planner", "worker")
builder.add_conditional_edges(
    "worker",
    should_continue,
    {
        "worker": "worker",
        "solver": "solver"
    }
)
builder.add_edge("solver", END)

graph = builder.compile()

if __name__ == "__main__":
    result = graph.invoke(
        {
            "user_request": "My order ORD-100 arrived damaged yesterday. Can I get a refund?",
            "plan": [],
            "current_step": 0,
            "order_id": None,
            "policy_decision": None,
            "final_answer": None,
            "evidence": []
        }
    )

    print("PLAN:")
    for step in result["plan"]:
        print(step)

    print("\nEVIDENCE:")
    for item in result["evidence"]:
        print(item)

    print("\nFINAL ANSWER:")
    print(result["final_answer"])

```

vol_ch_03\src\plan_and_solve.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.plan_and_solve

PLAN:
{'step_type': 'extract_order_id', 'description': "Extract the order ID from the customer's message."}
{'step_type': 'lookup_order', 'description': 'Look up the order details using the extracted order ID.'}

```

```
{'step_type': 'evaluate_policy', 'description': 'Evaluate the refund policy based on the
order details and the reported issue.')}

EVIDENCE:
{'step_type': 'extract_order_id', 'data': {'order_id': 'ORD-100'}, 'note': 'Extracted
order identifier from the user request.')}
{'step_type': 'lookup_order', 'data': {'order': {'status': 'delivered',
'days_since_delivery': 8, 'damaged': True, 'opened': True, 'amount': 129.0}}, 'note':
'Loaded order facts from the order store.')}
{'step_type': 'evaluate_policy', 'data': {'policy_decision': 'Eligible for damaged-item
refund review.'}, 'note': 'Applied the damaged-item refund policy to the order facts.')}

FINAL ANSWER:
Thank you for letting us know about the damaged item in your order ORD-100. Your order is
eligible for a damaged-item refund review. We will begin processing your request and
follow up with the next steps.
```

This code implements plan-execute-solve as a deliberately staged workflow rather than a single all-at-once agent call. The important design choice is that each stage writes its result into shared graph state, so later stages operate on explicit artifacts instead of reconstructing intent from free-form text. The planner is responsible for turning the user request into an ordered execution path. It does not answer the customer. It classifies the request into a small, controlled set of step types and returns them as structured plan data. That matters because the rest of the graph can now execute against a known sequence instead of improvising its next move at every turn. The plan also makes the workflow inspectable: you can see exactly what the system believes must happen before any work begins.

Execution is handled by the `worker` node, which processes one planned step at a time using `current_step` as the cursor. Each branch performs a narrow responsibility and appends a structured `EvidenceItem` into state. That evidence list is the core execution contract. Instead of passing loosely worded intermediate text downstream, the worker records concrete outputs such as the extracted order id, retrieved order data, or policy decision, each with a note explaining what happened. The policy evaluation step is especially important because it shows where deterministic business rules belong: once the order facts are available, the decision is made in code rather than delegated back to the model. The control flow is equally deliberate. `should_continue` loops back to the worker while there are remaining steps, but it can also short-circuit to the solver when execution has already produced a final answer, such as when the order id is missing. This prevents unnecessary work.

The solver is isolated to one task: produce the final customer-facing reply from the original request, the plan, and the accumulated evidence. Because it is constrained to structured state, synthesis happens after the facts have been gathered and checked, which keeps the final response grounded and easier to trust.

3.2 ROUTING STRATEGIES: RULE-BASED, LLM-BASED, CLASSIFIER-BASED, EMBEDDING-BASED, AND HYBRID

Prompt chaining is useful when a task can be divided into a known sequence of dependent steps, where each step uses the result of the previous one. Routing addresses a different problem. It is useful when the next step depends on the current request, the workflow state, or the result of an earlier operation. A routed workflow does not always follow the same path. It evaluates the available information and selects the most appropriate next step from a set of available destinations. That destination may be a workflow branch, one or more specialists, a clarification step, a human-review step, or an escalation path. The available node types and control rules are designed in advance, although the selected path, the number of dispatched tasks, and the data passed to them may be determined at runtime.

A routing result may be represented as a label, score, ranking, or structured value, but the application interprets it as a control-flow decision. If that decision is wrong, every downstream component may behave correctly within its own responsibilities while the overall result remains incorrect because the request entered the wrong branch. Good routing therefore allows downstream branches to remain narrow and specialized. A customer-support assistant, for example, may route billing questions to a billing workflow, technical problems to a troubleshooting workflow, and sensitive cases to a human escalation path. Each branch can then use instructions, tools, validation rules, and evaluation criteria suited to that category of request. Routing should be implemented as an explicit and testable part of the workflow rather than hidden inside a general-purpose prompt. The system should make it possible to observe which route was selected, which signals influenced the decision, and what happened after dispatch. In LangChain and LangGraph applications, routing may select the initial workflow, choose a branch after an intermediate result, or determine whether execution should continue, stop, request clarification, or require human review.

A router should not be confused with a supervisor. A router normally makes a focused dispatch decision: it determines where the current request or subtask should go. A supervisor has a broader role. It maintains coordination context and may repeatedly decide which specialist should act next over a multi-step interaction. Routing is therefore appropriate when the available destinations and hand-off conditions are reasonably well defined. Supervisory orchestration is more suitable when coordination must continue throughout a longer process.

Before choosing a routing method, define the route taxonomy: the set of available destinations and the rules that distinguish their responsibilities. Each route should correspond to a meaningful downstream difference, such as different instructions, tools, data sources, validation rules, authorization boundaries, or operational ownership. Routes that substantially overlap or operate at inconsistent levels of abstraction are difficult to classify reliably regardless of the routing technology used. The taxonomy should also define how unsupported, ambiguous, and multi-intent requests are handled. A more sophisticated router cannot compensate for destinations whose responsibilities are unclear.

3.2.1 Routing as graph control flow

At the graph level, LangGraph provides several ways to express routing. Static flow is represented with normal edges created with `add_edge(...)`. Branching decisions can be expressed with conditional edges or, when routing is performed inside a node, with `Command`. If the first node must be selected dynamically, add conditional edges from the virtual `START` node with `add_conditional_edges(START, routing_function)`. The routing function examines the initial graph state and returns the name of the node, or nodes, that should run first. When routing and a state update should be produced by the same node function, return `Command(update=..., goto=...)`. Use a separate conditional edge when the node returns its state updates and routing is handled by a distinct routing function. A conditional routing function may return one node name or several node names. When it returns several names, those nodes are scheduled in parallel in the next graph superstep and receive the same input-state snapshot. Their returned state updates are then combined according to the reducers defined for the affected state fields. When dynamically selected destinations require different input states, return `Send` objects from the conditional routing function instead. One important rule applies when using `Command`: it does not replace static outgoing edges already defined for the same node. If a node returns `Command(goto="x")` and also has a static edge to "y", both "x" and "y" will run. A node should therefore use either `Command` or static outgoing edges for routing, not both. Conditional edges, `Command`, and `Send` determine how LangGraph applies a routing decision. They do not determine how the application should choose the route. The following sections examine rule-based, LLM-based, classifier-based, embedding-based, and hybrid decision strategies.

The following support example uses a conditional entry point. Instead of always starting with the same application node, the routing function examines the initial graph state and selects `order_help`, `policy_help`, or `clarify` as the first node.

```
from typing_extensions import TypedDict, Literal
from langgraph.graph import StateGraph, START, END

class SupportState(TypedDict):
    user_message: str
    is_authenticated: bool
    has_order_id: bool
    response: str

def choose_entry(state: SupportState) -> Literal["order_help", "policy_help",
                                                "clarify"]:
    text = state["user_message"].lower()

    if state["is_authenticated"] and state["has_order_id"]:
        return "order_help"

    if any(token in text for token in ["refund", "return", "exchange", "warranty"]):
        return "policy_help"

    return "clarify"

def order_help(state: SupportState):
    return {"response": "Starting in order_help"}

def policy_help(state: SupportState):
    return {"response": "Starting in policy_help"}

def clarify(state: SupportState):
    return {"response": "Starting in clarify"}

builder = StateGraph(SupportState)
builder.add_node("order_help", order_help)
builder.add_node("policy_help", policy_help)
builder.add_node("clarify", clarify)

# Conditional entry point: the graph starts in different places
```

```
# depending on the current state.
builder.add_conditional_edges(START, choose_entry)

builder.add_edge("order_help", END)
builder.add_edge("policy_help", END)
builder.add_edge("clarify", END)

graph = builder.compile()
```

vol3_ch_03\scr\conditional_entry.py

This example makes an important architectural point: not every routing decision requires an LLM. The routing function uses authentication status, the presence of an order identifier, and keywords in the user's message to choose the first application node. When application state and deterministic rules are sufficient, using them is usually preferable to introducing a model call because the decision remains predictable and easy to test. Avoiding an unnecessary model call also usually reduces latency and model cost. Routing to an order workflow does not by itself authorize access to order data. The destination workflow must still enforce ownership and permission checks.

3.2.2 Rule-based routing

Rule-based routing uses explicit application logic to select a destination. When both its inputs and rules are deterministic, the same input state produces the same routing decision. The rules may use `if/else` branches, lookup tables, pattern or keyword checks, structured request fields, authentication status, user role, product family, locale, or thresholds applied to an existing score. A threshold rule is deterministic given its input score, although the score itself may have been produced by a probabilistic model. When the rules use local data and remain small enough to understand, rule-based routing is usually fast, predictable, inexpensive, and easy to test. Its main limitation is coverage. Rules handle the situations their designers anticipated. As requests become more varied, ambiguous, or linguistically complex, the rule set may grow difficult to maintain and may misroute inputs that do not match the expected patterns. Rule-based routing is usually the best starting point when the possible destinations are stable and the routing decision can be derived reliably from graph state, request fields, verified metadata, or a small set of unambiguous patterns. For example, a trading application may receive an explicit request type such as `price_quote` or `research_report`. It does not need an LLM to distinguish between those routes because the required intent is already present in the request. In such cases, rule-based routing is not merely a baseline; it is the appropriate strategy. In the following example, the selected route is stored explicitly in graph state rather than being used only as a temporary control-flow result. This allows downstream nodes to inspect and reuse the original decision without repeating the routing logic or inferring the route from the request again. The `route_history` field makes the routing decision visible to downstream nodes during execution. This is useful for debugging, testing, and explaining why a particular handler was selected.

```
from typing_extensions import TypedDict, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

class EcommerceState(TypedDict):
    user_message: str
    route: str
    route_history: list[str]
    response: str

def route_request(
```



```

state: EcommerceState,
) -> Command[Literal["product_search", "returns_policy", "order_help", "clarify"]]:
    text = state["user_message"].lower()

    if any(token in text for token in ["recommend", "suggest", "looking for", "best laptop"]):
        chosen = "product_search"
        return Command(
            update = {
                "route": chosen,
                "route_history": state["route_history"] + [f"route request -> {chosen}"]
            },
            goto = chosen
        )

    if any(token in text for token in ["refund", "return", "exchange"]):
        chosen = "returns_policy"
        return Command(
            update = {
                "route": chosen,
                "route_history": state["route_history"] + [f"route_request -> {chosen}"]
            },
            goto = chosen
        )

    if any(token in text for token in ["where is my order", "track", "delivery", "cancel order"]):
        chosen = "order_help"
        return Command(
            update = {
                "route": chosen,
                "route_history": state["route_history"] + [f"route_request -> {chosen}"]
            },
            goto = chosen
        )

    chosen = "clarify"
    return Command(
        update = {
            "route": chosen,
            "route_history": state["route_history"] + [f"route_request -> {chosen}"]
        },
        goto = chosen
    )

def product_search(state: EcommerceState):
    return {
        "response": (
            f"Routed to '{state['route']}' because the request looks like product discovery. "
            f"Audit trail: {state['route_history']}"
        )
    }

def returns_policy(state: EcommerceState):
    return {
        "response": (
            f"Routed to '{state['route']}' because the request mentions refund/return/exchange. "
            f"Audit trail: {state['route_history']}"
        )
    }

def order_help(state: EcommerceState):
    return {
        "response": (
            f"Routed to '{state['route']}' because the request looks like order tracking or delivery support. "
            f"Audit trail: {state['route_history']}"
        )
    }

def clarify(state: EcommerceState):
    return {
        "response": (
            f"Routed to '{state['route']}' because the intent was ambiguous. "
            f"Audit trail: {state['route_history']}"
        )
    }

builder = StateGraph(EcommerceState)
builder.add_node("route_request", route_request)

```

```

builder.add_node("product_search", product_search)
builder.add_node("returns_policy", returns_policy)
builder.add_node("order_help", order_help)
builder.add_node("clarify", clarify)

builder.add_edge(START, "route_request")
builder.add_edge("product_search", END)
builder.add_edge("returns_policy", END)
builder.add_edge("order_help", END)
builder.add_edge("clarify", END)

graph = builder.compile()

if __name__ == "__main__":
    examples = [
        "Can you recommend the best laptop for a student?",
        "I want a refund for the headphones I bought last week.",
        "Where is my order? The delivery is late.",
        "I need help.",
    ]

    for text in examples:
        result = graph.invoke(
            {
                "user_message": text,
                "route": "",
                "route_history": [],
                "response": ""
            }
        )

        print("\n=== RUN ===")
        print("User message :", result["user_message"])
        print("Final route   :", result["route"])
        print("Route history:", result["route_history"])
        print("Response      :", result["response"])

```

vol3_ch_03\src\rule_based_routing.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.rule_based_routing

=== RUN ===
User message : Can you recommend the best laptop for a student?
Final route   : product_search
Route history: ['route_request -> product_search']
Response      : Routed to 'product_search' because the request looks like product
discovery. Audit trail: ['route_request -> product_search']

=== RUN ===
User message : I want a refund for the headphones I bought last week.
Final route   : returns_policy
Route history: ['route_request -> returns_policy']
Response      : Routed to 'returns_policy' because the request mentions
refund/return/exchange. Audit trail: ['route_request -> returns_policy']

=== RUN ===
User message : Where is my order? The delivery is late.
Final route   : order_help
Route history: ['route_request -> order_help']
Response      : Routed to 'order_help' because the request looks like order tracking or
delivery support. Audit trail: ['route_request -> order_help']

=== RUN ===
User message : I need help.
Final route   : clarify
Route history: ['route_request -> clarify']
Response      : Routed to 'clarify' because the intent was ambiguous. Audit trail:
['route_request -> clarify']

```

The keyword list is only a simple demonstration. The important part is how the routing responsibility is represented in the graph. Given the same input state, `route_request` applies the same ordered rules, records the selected route in graph state, and routes execution to the corresponding handler. The order of the rules matters. If a request matches more than one condition, the first matching branch wins because the function returns immediately. This behaviour is deterministic, but it does not guarantee that the selected route is the best

interpretation of a multi-intent request. The `route_history` list demonstrates how later nodes can inspect the routing decision during the graph run. By itself, it is not a durable audit log. Persisted graph state and tracing are useful for operational reconstruction, but compliance-grade auditing normally requires a dedicated, durable application audit store with appropriate retention, access-control, and integrity protections. Using `Command(update=..., goto=...)` is appropriate here because the routing decision and its related state update belong to the same operation. Keeping them together makes the relationship between the selected route and the state update explicit. Rule-based routing becomes less reliable when predefined signals are not sufficient to distinguish the available routes. Consider the request, “I tried to change my delivery address, then I got charged twice and now my item is still delayed.” It contains several possible intents: an address change, a duplicate charge, and a delayed delivery. A single keyword route may ignore part of the request or select whichever rule matches first. At this point, the system may ask for clarification, route to multiple handlers, decompose the request into separate tasks, apply a defined priority rule, or use an LLM-based, classifier-based, or embedding-based routing method. These approaches can handle richer language than simple keyword rules, but each introduces its own uncertainty and still requires validation, fallback behaviour, and explicit handling for ambiguous or multi-intent requests.

3.2.3 LLM-based routing

LLM-based routing uses a language model to predict one or more route labels from a predefined route taxonomy. The model receives the current request together with route descriptions, relevant workflow state, and any instructions that define how ambiguous or unsupported cases should be handled. Embedding-based routing, which ranks destinations by vector similarity rather than generating a route decision, is treated separately later in the chapter. In a single-label router, the model is instructed to select exactly one route from a predefined set and return the decision in a constrained format. This approach can use the broader linguistic context of a request instead of relying only on exact keywords or fixed patterns. It is useful when the same intent can be expressed through varied, indirect, or linguistically complex language. For example, a support router may distinguish a refund request from a delivery complaint or a suspected fraud report even when the user uses unfamiliar wording. The model cannot reliably recover information that the request does not contain. These examples use closed-set routing: the valid destinations are predefined. Real traffic will inevitably include requests that are unsupported, outside the known route set, or too ambiguous to classify safely. The route taxonomy should therefore include an abstention path such as clarification, safe fallback, or human escalation. Abstention means that the router deliberately declines to force a low-quality match when the available evidence is insufficient. Compared with local rule-based routing, LLM-based routing normally adds model latency and usage cost. It also requires ongoing evaluation and maintenance as route definitions, prompts, models, and real user traffic change. A common implementation defines a schema containing the allowed routes and invokes the model with structured output so that application code receives typed, machine-readable data rather than unconstrained text. The example below stores the route label in graph state and dispatches execution through a conditional edge. A graph can instead return `Command` when the routing decision and its associated state updates belong in the same node function.

```

from typing_extensions import TypedDict, Literal
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
from .config import OPENAI_API_KEY

llm = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class RouteDecision(BaseModel):
    route: Literal["product_search", "returns_policy", "order_help", "clarify"] = Field(
        description = "Best next step for this e-commerce request.")

router_llm = llm.with_structured_output(RouteDecision)

class EcommerceState(TypedDict):
    user_message: str
    route: str
    response: str

def choose_route(state: EcommerceState):
    decision = router_llm.invoke(
        [
            {
                "role": "system",
                "content": (
                    "Route the request to exactly one branch. "
                    "Use product_search for shopping or recommendation requests. "
                    "Use returns_policy for returns, refunds, or exchanges. "
                    "Use order_help for order tracking, cancellation, or delivery issues"
                    "Use clarify if the request is too ambiguous."
                )
            },
            {"role": "user", "content": state["user_message"]}
        ]
    )
    return {"route": decision.route}

def next_node(state: EcommerceState) -> str:
    return state["route"]

def product_search(state: EcommerceState):
    return {"response": "Route selected: product_search"}

def returns_policy(state: EcommerceState):
    return {"response": "Route selected: returns_policy"}

def order_help(state: EcommerceState):
    return {"response": "Route selected: order_help"}

def clarify(state: EcommerceState):
    return {"response": "Route selected: clarify"}

builder = StateGraph(EcommerceState)
builder.add_node("choose_route", choose_route)
builder.add_node("product_search", product_search)
builder.add_node("returns_policy", returns_policy)
builder.add_node("order_help", order_help)
builder.add_node("clarify", clarify)

builder.add_edge(START, "choose_route")
builder.add_conditional_edges(
    "choose_route",
    next_node,
    {
        "product_search": "product_search",
        "returns_policy": "returns_policy",
        "order_help": "order_help",
        "clarify": "clarify"
    }
)

builder.add_edge("product_search", END)
builder.add_edge("returns_policy", END)
builder.add_edge("order_help", END)
builder.add_edge("clarify", END)

graph = builder.compile()

if __name__ == "__main__":
    examples = [
        "Can you recommend a good laptop for university?",
        "I want a refund for my headphones.",
        "Where is my order? It should have arrived yesterday.",
    ]

```

```

    "I need help.",
]
for text in examples:
    result = graph.invoke(
        {
            "user_message": text,
            "route": "",
            "response": ""
        }
    )
    print("\n=== RUN ===")
    print("User message:", result["user_message"])
    print("Chosen route:", result["route"])
    print("Response      :", result["response"])

```

vol3_ch_03\scr\model_routing.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.model_routing

=== RUN ===
User message: Can you recommend a good laptop for university?
Chosen route: product_search
Response      : Route selected: product_search

=== RUN ===
User message: I want a refund for my headphones.
Chosen route: returns_policy
Response      : Route selected: returns_policy

=== RUN ===
User message: Where is my order? It should have arrived yesterday.
Chosen route: order_help
Response      : Route selected: order_help

=== RUN ===
User message: I need help.
Chosen route: clarify
Response      : Route selected: clarify

```

In this example, the routing taxonomy is represented in two complementary places. The `Literal` type restricts the values accepted by the application, while the system instruction explains the intended meaning of each route to the model. These definitions must remain consistent when routes are added, removed, or renamed. When structured generation and schema validation succeed, an unsupported route label cannot be accepted as a valid routing result. The application must still handle provider failures, invocation failures, and validation errors. A structurally valid result may also remain semantically wrong: the model could return `order_help` in the required format even when `returns_policy` would have been the better destination. Routing quality must therefore be evaluated with representative requests and expected route labels. This router also requires exactly one route. That is appropriate for the simple examples shown here. A request containing several independent problems may instead require clarification, decomposition into separate subtasks, or a multi-label or multi-destination router.

3.2.4 Classifier-based routing

Classifier-based routing uses a supervised machine-learning model trained specifically to predict route labels. During training, the model learns from labelled examples such as support messages assigned to `order_help`, `returns_policy`, or `clarify`. At runtime, it applies the learned decision function to each new request and returns one or more class predictions.

This is different from LLM-based routing. An LLM normally receives route descriptions and instructions at inference time and generates a structured decision. A supervised classifier

learns its route boundaries during training and does not need the full route catalogue to be described in every runtime prompt. Depending on the model and deployment architecture, this can provide lower and more predictable latency, stable outputs, and lower per-request inference cost. A classifier is a strong choice when the route taxonomy is stable, representative labelled data is available, and the expected request distribution is understood. It is particularly useful for high-volume classification problems where low variance and operational predictability matter more than the flexibility of a general-purpose language model.

Its limitations are primarily data and lifecycle related. Poor labels, class imbalance, sparse examples for rare routes, and inconsistent annotation rules can produce misleading decision boundaries. Changes in user language, products, policies, or route definitions may also create concept drift, requiring new labelled examples and retraining. If the classifier exposes class probabilities, those values should not automatically be treated as calibrated confidence estimates. Thresholds and abstention rules must be validated against representative evaluation data. A production classifier should therefore be evaluated per route rather than only through overall accuracy. Precision, recall, confusion matrices, calibration quality, abstention behaviour, and performance on minority classes are often more informative than one aggregate score. Once the classifier has produced an accepted route label, LangGraph can apply that decision through a conditional edge or through `Command`, using the same control-flow mechanisms introduced earlier.

3.2.5 Embedding-based semantic routing

Semantic routing represents the incoming request and the available routes as embeddings, then compares those vectors with a similarity function. It avoids asking a generative chat model to choose a route, but it still performs model inference when it embeds each incoming request. The application embeds the query and compares the resulting vector with vectors created from route descriptions, representative examples, or both. The route with the highest similarity score becomes the leading candidate. This is similar to semantic retrieval. The router uses a learned representation of textual similarity instead of relying only on exact words or fixed keyword patterns. This allows differently worded requests to match the same route, although similarity does not guarantee that the inferred intent is correct. Semantic routing is useful when users express the same intent in many different ways and each route can be represented by clear descriptions or representative examples. For example, it can map varied support requests to refunds, exchanges, delayed delivery, or account access. In e-commerce, it can distinguish product discovery from policy and order-related questions. In a trading assistant, it can provide broad dispatch among macro analysis, company research, portfolio explanation, and screening. Compared with invoking a generative chat model for every routing decision, semantic routing can reduce model cost and latency, particularly when route embeddings are precomputed and similarity search runs locally. The actual difference depends on the embedding provider, network calls, caching, route-catalogue size, and search implementation. The method also has important limitations. If two routes have similar representations, the router may select the wrong destination even when both matches appear plausible. Routing quality degrades when route descriptions are vague, overlapping, too short, or unrepresentative of real requests. A semantic routing decision must not be treated as an authorization decision. Tenant access, user permissions, data scope, locale restrictions,

version constraints, and approval requirements must be enforced through deterministic application controls. Semantic similarity may identify a relevant destination, but it cannot grant permission to read data or execute operations. The following example applies semantic routing to four specialist branches in a trading assistant.

```
from dataclasses import dataclass
from math import sqrt
from typing_extensions import TypedDict, Literal
from langchain.embeddings import init_embeddings
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command
from .config import OPENAI_API_KEY

@dataclass(frozen = True)
class RouteProfile:
    name: str
    description: str

ROUTES = [
    RouteProfile(
        name = "macro_analysis",
        description = "Questions about inflation, interest rates, central banks,
                      currencies, or broad market trends."),
    RouteProfile(
        name = "company_research",
        description = "Questions about a single company, earnings, fundamentals,
                      guidance, or valuation."),
    RouteProfile(
        name = "screening",
        description = "Requests to filter or rank stocks by market cap, sector, factor,
                      dividend yield, or other criteria."),
    RouteProfile(
        name = "portfolio_explanation",
        description = "Questions about positions, allocations, diversification,
                      drawdowns, or why a portfolio behaved a certain way.")
]

# Create the embedding model once.
embeddings = init_embeddings("openai:text-embedding-3-small")

# Precompute route vectors once.
route_vectors = embeddings.embed_documents([route.description for route in ROUTES])

class RoutingState(TypedDict):
    user_message: str
    route: str
    score: float
    ranking: list[tuple[str, float]]
    response: str

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = sqrt(sum(x * x for x in a))
    norm_b = sqrt(sum(x * x for x in b))
    return dot / (norm_a * norm_b)

def semantic_router(
    state: RoutingState
) -> Command:
    Literal["macro_analysis", "company_research", "screening",
           "portfolio_explanation", "clarify"]
1:
    query = state["user_message"]
    query_vector = embeddings.embed_query(query)

    ranking = sorted(
        [
            (route.name, cosine_similarity(query_vector, vector))
            for route, vector in zip(ROUTES, route_vectors)
        ],
        key = lambda item: item[1],
        reverse = True
    )

    best_route, best_score = ranking[0]
    min_score = 0.35

    if best_score < min_score:
        chosen = "clarify"
```



```

else:
    chosen = best_route

return Command(
    update={
        "route": chosen,
        "score": best_score,
        "ranking": ranking
    },
    goto = chosen
)

def macro_analysis(state: RoutingState):
    .
    .

def company_research(state: RoutingState):
    .
    .

def screening(state: RoutingState):
    .
    .

def portfolio_explanation(state: RoutingState):
    .
    .

def clarify(state: RoutingState):
    .
    .

builder = StateGraph(RoutingState)

builder.add_node("semantic_router", semantic_router)
builder.add_node("macro_analysis", macro_analysis)
builder.add_node("company_research", company_research)
builder.add_node("screening", screening)
builder.add_node("portfolio_explanation", portfolio_explanation)
builder.add_node("clarify", clarify)

builder.add_edge(START, "semantic_router")
builder.add_edge("macro_analysis", END)
builder.add_edge("company_research", END)
builder.add_edge("screening", END)
builder.add_edge("portfolio_explanation", END)
builder.add_edge("clarify", END)

graph = builder.compile()

if __name__ == "__main__":
    examples = [
        "How would lower interest rates affect the euro and European equities?",
        "Can you analyze Nvidia's latest earnings and valuation?",
        "Find large-cap healthcare stocks with strong dividend yield.",
        "Why did my portfolio fall more than the market this month?",
        "Hello there"
    ]

    for query in examples:
        result = graph.invoke(
            {
                "user_message": query,
                "route": "",
                "score": 0.0,
                "ranking": [],
                "response": ""
            }
        )

        print("\n=== QUERY ===")
        print(result["user_message"])
        print(f"Chosen route: {result['route']}")
        print(f"Top score : {result['score']:.4f}")
        print("Top matches :")
        for name, score in result["ranking"]:
            print(f" - {name:22} {score:.4f}")
        print("Response : ", result["response"])

```

vol3_ch_03\src\semantic_routing.py

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.semantic_routing

```

=== QUERY ===
How would lower interest rates affect the euro and European equities?
Chosen route: macro_analysis
Top score   : 0.4148
Top matches :
- macro_analysis      0.4148
- company_research    0.2799
- portfolio_explanation 0.2714
- screening           0.2039
Response    : Routed to 'macro_analysis' with score 0.4148. This looks like a macro
question about rates, inflation, currencies, or broad markets.

=== QUERY ===
Can you analyze Nvidia's latest earnings and valuation?
Chosen route: company_research
Top score   : 0.5156
Top matches :
- company_research    0.5156
- portfolio_explanation 0.2899
- screening           0.2823
- macro_analysis      0.2513
Response    : Routed to 'company_research' with score 0.5156. This looks like single-
company analysis: earnings, valuation, guidance, or fundamentals.

=== QUERY ===
Find large-cap healthcare stocks with strong dividend yield.
Chosen route: screening
Top score   : 0.4223
Top matches :
- screening           0.4223
- company_research    0.3171
- portfolio_explanation 0.2384
- macro_analysis      0.1843
Response    : Routed to 'screening' with score 0.4223. This looks like a stock filtering
or ranking request.

=== QUERY ===
Why did my portfolio fall more than the market this month?
Chosen route: portfolio_explanation
Top score   : 0.4551
Top matches :
- portfolio_explanation 0.4551
- screening           0.3201
- company_research    0.3075
- macro_analysis      0.2982
Response    : Routed to 'portfolio_explanation' with score 0.4551. This looks like a
portfolio-behavior or allocation explanation request.

=== QUERY ===
Hello there
Chosen route: clarify
Top score   : 0.1819
Top matches :
- company_research    0.1819
- portfolio_explanation 0.1807
- macro_analysis      0.1753
- screening           0.0988
Response    : Top score was only 0.1819, so the graph routed to 'clarify'. The request is
too ambiguous to classify confidently.

```

The route vectors are computed when the module is initialized and reused for subsequent requests handled by that process, avoiding repeated embedding of an unchanged route catalogue. In production, cache or persist these vectors and regenerate them whenever the route descriptions, representative examples, text-preprocessing logic, or embedding model changes. The full ranking supports debugging and evaluation, but similarity scores are not probabilities or calibrated confidence values. The example therefore applies a minimum-score fallback and selects clarify when the leading score is below 0.35. That threshold is illustrative and must be validated against representative labelled requests for the selected embedding model and route catalogue. The leading score alone is insufficient: a request may have a relatively high top score while the second-best route is almost equally similar. Production

routing should therefore consider both absolute relevance and the margin between the leading candidates, with weak or ambiguous matches sent to clarification. The selected route, top score, and ranking are stored in graph state, while Command directs execution to the selected branch.

3.2.6 Hybrid routing

Hybrid routing is an application-level strategy that combines two or more routing methods in the same decision pipeline. It is not a separate LangGraph control-flow primitive. A hybrid router may combine deterministic rules, embedding similarity, a supervised classifier, an LLM, or other application signals. The purpose is to assign each stage a limited responsibility instead of expecting one method to handle every routing decision. A common design begins with deterministic policy and eligibility checks. These checks use authoritative application information to determine which destinations are permitted. Examples include tenant scope, verified permissions, allowed product domains, mandatory locale restrictions, and other rules that define the set of eligible routes. Prohibited destinations should be removed before similarity-based, classifier-based, or generative routing begins. The next stage can use semantic similarity to rank the remaining routes and produce a smaller shortlist. This may reduce the number of candidates that need more expensive analysis, although its cost and latency depend on the embedding service, caching strategy, route-catalogue size, and search implementation. If the leading candidates remain difficult to distinguish, an LLM arbiter can make a final selection from the shortlist using structured output. The schema restricts the result to the permitted route names, but it does not guarantee that the selected route is correct. This layered design is useful when the workflow contains both authoritative constraints and natural-language ambiguity. Deterministic controls make eligibility rules explicit and testable. Semantic similarity narrows the candidate set. The LLM is invoked only for cases that the earlier stages do not resolve clearly. Hybrid routing is justified when evaluation demonstrates that different stages solve distinct parts of the decision more effectively than one method alone. The following example implements this pipeline:

deterministic eligibility checks → semantic shortlist → early exit for a decisive match → LLM arbitration for ambiguous matches → selected branch

```
from dataclasses import dataclass
from math import sqrt
from typing_extensions import TypedDict, Literal
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langchain.embeddings import init_embeddings
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command
from .config import OPENAI_API_KEY

@dataclass(frozen = True)
class RouteProfile:
    name: str
    description: str

ROUTES = [
    .
    .
]

embeddings = init_embeddings("openai:text-embedding-3-small")
route_vectors = embeddings.embed_documents([route.description for route in ROUTES])

arbiter = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class FinalDecision(BaseModel):
    route: Literal["returns_policy", "order_help", "product_qa",
```

```

        "recommendations", "clarify"
    ] = Field(description = "Final route selected from the shortlist.")
arbiter_llm = arbiter.with_structured_output(FinalDecision)

class HybridRoutingState(TypedDict):
    user_message: str
    user_role: str
    has_order_id: bool
    route: str
    route_reason: str
    shortlist: list[tuple[str, float]]
    response: str

def cosine_similarity(a: list[float], b: list[float]) -> float:
    .
    .

def semantic_top_k(query: str, k: int = 2) -> list[tuple[str, float]]:
    query_vector = embeddings.embed_query(query)
    scored = sorted(
        [
            (route.name, cosine_similarity(query_vector, vector))
            for route, vector in zip(ROUTES, route_vectors)
        ],
        key = lambda item: item[1],
        reverse = True
    )
    return scored[:k]

def hybrid_router(
    state: HybridRoutingState
) -> Command[
    Literal["returns policy", "order help", "product qa", "recommendations", "clarify"]]:
    text = state["user_message"].lower()
    user_role = state["user_role"]
    has_order_id = state["has_order_id"]

    # 1) Deterministic gates first.
    if user_role == "admin":
        return Command(
            update = {
                "route": "clarify",
                "route_reason": "Deterministic gate: admin requests require manual
                                clarification.",
                "shortlist": []
            },
            goto = "clarify"
        )

    if has_order_id and any(token in text for token in ["where is my order", "track",
                                                         "delivery"]):
        return Command(
            update = {
                "route": "order_help",
                "route_reason": "Deterministic gate: order-id present and message is
                                about tracking/delivery.",
                "shortlist": []
            },
            goto = "order_help"
        )

    # 2) Semantic narrowing.
    shortlist = semantic_top_k(state["user message"], k = 2)

    # Strong winner: stop early.
    if len(shortlist) == 1 or (shortlist[0][1] - shortlist[1][1]) >= 0.12:
        chosen = shortlist[0][0]
        return Command(
            update = {
                "route": chosen,
                "route_reason": (
                    f"Semantic winner: {chosen} won decisively with margin "
                    f"{shortlist[0][1] - shortlist[1][1]:.4f}."
                    if len(shortlist) > 1
                    else f"Semantic winner: only one shortlist result, chose {chosen}."
                ),
                "shortlist": shortlist
            },
            goto = chosen
        )

```

```

# 3) LLM arbitration only for ambiguous cases.
options = ", ".join(name for name, _ in shortlist)
decision = arbiter_llm.invoke(
    [
        {
            "role": "system",
            "content": (
                f"Choose exactly one route from: {options}. "
                "Prefer the route that best matches the user's main intent. "
                "Return only the route label."
            )
        },
        {"role": "user", "content": state["user_message"]}
    ]
)

return Command(
    update = {
        "route": decision.route,
        "route_reason": f"LLM arbitration: semantic shortlist was ambiguous, final choice from [{options}] was {decision.route}.",
        "shortlist": shortlist
    },
    goto = decision.route
)

def returns_policy(state: HybridRoutingState):
    .
    .

def order_help(state: HybridRoutingState):
    .
    .

def product_qa(state: HybridRoutingState):
    .
    .

def recommendations(state: HybridRoutingState):
    .
    .

def clarify(state: HybridRoutingState):
    .
    .

builder = StateGraph(HybridRoutingState)

builder.add_node("hybrid_router", hybrid_router)
builder.add_node("returns_policy", returns_policy)
builder.add_node("order_help", order_help)
builder.add_node("product_qa", product_qa)
builder.add_node("recommendations", recommendations)
builder.add_node("clarify", clarify)

builder.add_edge(START, "hybrid_router")
builder.add_edge("returns_policy", END)
builder.add_edge("order_help", END)
builder.add_edge("product_qa", END)
builder.add_edge("recommendations", END)
builder.add_edge("clarify", END)

graph = builder.compile()

if __name__ == "__main__":
    examples = [
        {
            "user_message": "Where is my order ORD-100? The delivery is late.",
            "user_role": "customer",
            "has_order_id": True
        },
        {
            "user_message": "Can I get a refund for the shoes I bought last week?",
            "user_role": "customer",
            "has_order_id": False
        },
        {
            "user_message": "Does this monitor work with a MacBook Air?",
            "user_role": "customer",
            "has_order_id": False
        },
        {
            "user_message": "Can you suggest a good laptop for travel and office work?",

```

```

        "user_role": "customer",
        "has_order_id": False
    },
    {
        "user_message": "Please review this request.",
        "user_role": "admin",
        "has_order_id": False
    }
]

for example in examples:
    result = graph.invoke(
        {
            "user_message": example["user_message"],
            "user_role": example["user_role"],
            "has_order_id": example["has_order_id"],
            "route": "",
            "route_reason": "",
            "shortlist": [],
            "response": ""
        }
    )

    print("\n=== RUN ===")
    print("User message :", result["user_message"])
    print("User role      :", result["user_role"])
    print("Has order id   :", result["has_order_id"])
    print("Chosen route   :", result["route"])
    print("Reason          :", result["route_reason"])
    print("Shortlist       :", result["shortlist"])
    print("Response        :", result["response"])

```

vol3_ch_03\src\hybrid_routing.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.hybrid_routing

=== RUN ===
User message : Where is my order ORD-100? The delivery is late.
User role    : customer
Has order id : True
Chosen route : order_help
Reason       : Deterministic gate: order-id present and message is about
tracking/delivery.
Shortlist    : []
Response     : Routed to 'order_help'. Reason: Deterministic gate: order-id present and
message is about tracking/delivery. This request belongs in the order tracking, delivery,
or cancellation flow.

=== RUN ===
User message : Can I get a refund for the shoes I bought last week?
User role    : customer
Has order id : False
Chosen route : returns_policy
Reason       : LLM arbitration: semantic shortlist was ambiguous, final choice from
[returns_policy, order_help] was returns_policy.
Shortlist    : [('returns_policy', 0.38706515484321236), ('order_help',
0.30923069218797766)]
Response     : Routed to 'returns_policy'. Reason: LLM arbitration: semantic shortlist
was ambiguous, final choice from [returns_policy, order_help] was returns_policy. This
request belongs in the returns/refunds/exchanges flow.

=== RUN ===
User message : Does this monitor work with a MacBook Air?
User role    : customer
Has order id : False
Chosen route : product_qa
Reason       : LLM arbitration: semantic shortlist was ambiguous, final choice from
[product_qa, returns_policy] was product_qa.
Shortlist    : [('product_qa', 0.22330242206850798), ('returns_policy',
0.13846664543723328)]
Response     : Routed to 'product_qa'. Reason: LLM arbitration: semantic shortlist was
ambiguous, final choice from [product_qa, returns_policy] was product_qa. This request is
primarily about product details or compatibility.

=== RUN ===
User message : Can you suggest a good laptop for travel and office work?
User role    : customer
Has order id : False

```

```
Chosen route : recommendations
Reason      : LLM arbitration: semantic shortlist was ambiguous, final choice from
[recommendations, product_qa] was recommendations.
Shortlist   : [('recommendations', 0.25375891540012735), ('product qa',
0.16283094090416667)]
Response    : Routed to 'recommendations'. Reason: LLM arbitration: semantic shortlist
was ambiguous, final choice from [recommendations, product_qa] was recommendations. This
request is best handled as a recommendation or shopping-advice flow.

=== RUN ===
User message : Please review this request.
User role    : admin
Has order id : False
Chosen route : clarify
Reason       : Deterministic gate: admin requests require manual clarification.
Shortlist    : []
Response     : Routed to 'clarify'. Reason: Deterministic gate: admin requests require
manual clarification. The request should be clarified before proceeding.
```

The example applies the stages in a deliberate order. Deterministic rules run first because eligibility and policy decisions should not depend on semantic similarity or an LLM response. In this demonstration, an administrator request is sent to clarify, while the combination of an order identifier and order-related language can route directly to `order_help`. These deterministic rules narrow or resolve the eligible route set. Each destination must still enforce its own authorization controls before reading sensitive data or executing protected operations. If no deterministic rule resolves the request, the semantic stage ranks the eligible routes and retains the two candidates with the highest cosine-similarity scores. This reduces the candidate set passed to the LLM, but it introduces an important dependency: the arbiter can select only from the shortlist. It cannot recover the correct route if semantic shortlisting has already removed it. The example applies the score-and-margin policy introduced in the semantic-routing section. It skips LLM arbitration only when the leading candidate is sufficiently relevant and sufficiently separated from the second candidate. The exact thresholds must be selected from representative labelled requests rather than copied from this demonstration. When the margin is below the threshold, the LLM arbiter selects one route from the two shortlisted candidates. As with the earlier LLM router, schema validation restricts the accepted labels but does not establish semantic correctness. This layered design can reduce the number of LLM calls, but it also creates irreversible failure points. A route excluded by the deterministic stage cannot be recovered by later probabilistic stages. A route omitted from the semantic shortlist cannot be selected by the LLM arbiter. The arbiter may also select the wrong route from an otherwise correct shortlist. Hybrid routing therefore provides additional control and cost-management options, but it does not eliminate routing errors. Its value must be demonstrated through both stage-level and end-to-end evaluation.

3.2.7 Choosing the right routing architecture

Choose the simplest routing strategy that satisfies the required quality, latency, cost, maintainability, and control targets. The routing architecture should follow the available decision signal rather than a preference for a particular AI technique.

Strategy	Best decision signal	Main advantage	Main limitation	Primary requirement
Rule-based	Explicit state, metadata, or stable business rules	Deterministic, inexpensive, and easy to test	Limited coverage of varied or ambiguous language	Clear and maintainable rules
LLM-based	Nuanced or indirect natural-language intent	Flexible interpretation of complex requests	Model latency, cost, and semantic variability	Clear route descriptions and evaluation data

Classifier-based	Stable intent classes represented by labelled examples	Predictable runtime behaviour and low variance	Training effort, class imbalance, and drift	Representative labelled dataset
Embedding-based	Semantic similarity between requests and route descriptions or examples	Efficient meaning-based matching	Overlapping routes and uncalibrated similarity scores	Distinct route representations
Hybrid	A combination of authoritative constraints and ambiguous language	Layered control and selective use of expensive models	Additional complexity and error propagation	Evidence that each stage adds measurable value

- Use rule-based routing when the decision can be derived reliably from explicit application state, structured request fields, verified metadata, or a small set of stable rules.
- Use LLM-based routing when the available routes are clearly defined but users express their intent through varied, indirect, or complex language that deterministic rules cannot classify reliably. Do not expect the model to recover information that the request does not contain. When evidence is insufficient or several routes are equally plausible, clarify, decompose, abstain, or escalate.
- Use classifier-based routing when the route taxonomy is stable, representative labelled data is available, and a dedicated model meets the required accuracy, latency, calibration, and operational targets.
- Use embedding-based semantic routing when routes can be represented by distinct descriptions or representative examples and differently worded requests should map to the same destination through vector similarity.
- Use hybrid routing only when evaluation demonstrates that separate methods solve distinct parts of the routing problem more effectively than one method alone. For example, deterministic rules may establish eligibility, semantic search may produce a high-recall shortlist, and an LLM may distinguish between the remaining candidates.

Place the router where the workflow first has enough trustworthy information to decide. Route at the entry point when the initial request and metadata are sufficient. Route later when validation, retrieval, service calls, or earlier workflow steps must first enrich graph state. Routing may also occur inside a specialist workflow when it must select among predefined tools, operations, or subgraphs.

Fixed sequencing, routing, and parallel execution are different control-flow patterns, but they can coexist in the same graph. Fixed sequencing gives each completed step one predetermined successor. Routing selects one or more destinations from runtime information. Parallel execution runs independent branches whose outputs can be combined safely afterward. A router may therefore select several specialists, dispatch them in parallel, and send their results to a synthesis node. Multi-destination routing and Send are discussed further in the parallelization section, where fan-out, reducers, and result synthesis are treated together.

Evaluate the router on representative labelled requests that include normal, ambiguous, unsupported, multi-intent, adversarial, and safety-critical cases. Measure per-route precision and recall, confusion matrices, abstention quality, and end-to-end task success. For classifier-based routers, also assess class imbalance and calibration. For embedding-based routers,

evaluate threshold and margin behaviour. For layered routers, measure shortlist recall and the accuracy of each early-exit rule. Latency, model usage, and operational cost should be measured alongside routing quality.

After deployment, monitor real traffic for drift as route definitions, prompts, models, embedding configurations, labelled data, and user behaviour change. Prefer the simplest routing architecture that continues to meet the required quality and control targets. Routing is not valuable because it makes a workflow appear more agentic. It is valuable when it places each request in the branch best equipped to handle it, while making uncertainty, fallbacks, and decision boundaries visible to the rest of the system.

3.3 PARALLELIZATION (FAN-OUT/FAN-IN): MAP-REDUCE, VOTING, CONSENSUS, AGGREGATION

Parallelization is the third major control-flow tool alongside fixed sequencing and routing. A fixed sequence determines the next step in advance. Routing decides which branch or branches should run from the current state. Parallelization describes how several selected branches execute independently and later rejoin. Routing and parallelization are therefore related but separate decisions: routing chooses the work, while parallelization controls how simultaneously eligible work is executed. This is the fan-out/fan-in model. Work fans out to several branches, those branches perform their tasks independently, and a later stage collects or combines their outputs. In LangGraph, nodes that are eligible at the same point run in the same super-step. A super-step is one logical execution round in which the eligible nodes run before their state updates become visible to the next round. Nodes in the same super-step read the state produced by the previous super-step, not the updates being produced by their parallel peers. This execution model makes parallelization a state-management problem as well as a performance technique. Parallel branches may read the same input-state snapshot and may contribute updates to shared state fields, but those updates must have unambiguous merge semantics. If one operation requires a value produced by another operation before it can begin, the two operations are sequential and belong in separate super-steps.

Parallelization is useful when the same operation must be applied to many independent items, when several specialists must evaluate the same input from different perspectives, or when several candidate answers must be generated before one is selected or synthesized. It can reduce elapsed time when branch execution overlaps, and the runtime and external services permit sufficient concurrency. It does not reduce the total amount of work. Its benefit may be limited by the slowest branch, orchestration overhead, provider rate limits, connection limits, or resource contention. It also introduces more state updates, branch failures, outputs, and timing behaviour to inspect during testing and observability.

3.3.1 Parallelization as workflow control

At the graph level, fan-out is usually expressed through either static or dynamic branching. With static branching, the graph structure is known when the application is built. One node connects to several predefined downstream nodes. For example, a support response might be sent to separate branches that check tone, policy compliance, and clarity. If these branches have the same length and finish in the same super-step, a shared downstream node runs after all of them complete. If the branches contain different numbers of steps, an ordinary downstream connection does not necessarily create a barrier that waits for the longest branch. In that case, the join node may need deferred execution so that it runs only after all pending branch work has completed. With dynamic branching, the number of worker invocations depends on runtime data. LangGraph's Send API supports this pattern. A routing function can examine the current state and return one Send object for each item that must be processed. Every Send identifies the destination node and the state fragment supplied to that worker invocation. This is a natural fit for chunk-based summarization, per-item analysis, order-line validation, and report generation in which the number of work units is not known until

execution begins. The fan-in side requires equally deliberate design. A state key may define a reducer that tells LangGraph how its existing value should be combined with incoming updates. Without a reducer, a key uses single-value replacement semantics. If several parallel branches write to the same unreduced key during one super-step, LangGraph cannot determine which value should win and raises `INVALID_CONCURRENT_GRAPH_UPDATE`.

For accumulator fields, a common pattern is to annotate a list with a reducer such as `operator.add` and have every branch return a one-item list. LangGraph then concatenates the branch contributions. This is appropriate when the workflow only needs to collect results and their arrival order has no business meaning. When order matters, every result should carry a stable sequence number or item identifier, and a downstream node should restore the required order explicitly. Reducers and aggregation nodes have different responsibilities. A reducer defines state-update semantics: it specifies how the existing value of a state key is combined with an incoming update. An aggregation node performs application logic over the resulting collection. It may validate results, calculate scores, select a winner, synthesize a response, detect disagreement, or decide whether another processing round is required. Separating state combination from business interpretation makes larger graphs easier to test and evolve. Reducers combine updates. Aggregators interpret the combined result. Parallel execution also creates a failure boundary. Updates produced during a parallel super-step are committed to graph state only when that super-step succeeds. If one branch raises an unhandled exception, the super-step fails and its combined state update is not committed. When a checkpoint is configured, the results of successful nodes in that failed super-step are saved and reused when execution resumes, so only the failed work needs to be retried. This transactional behaviour applies to LangGraph state, not to external systems. A database update, message publication, payment request, or API call that has already completed is not automatically reversed because another branch failed. Nodes that perform external side effects should therefore use idempotency keys, upserts, duplicate detection, transactional outbox patterns, or other safeguards that make retries safe. Node-level retry policies are usually best suited to transient or otherwise explicitly retrievable failures. Validation errors, authorization failures, invalid business requests, and deterministic application errors normally require correction, routing, or escalation rather than repetition. Concurrency should also be bounded so that the graph respects provider rate limits, connection pools, memory limits, and downstream service capacity.

Not every fan-out requires a graph-level design. When the requirement is simply to run several predefined operations on the same input and collect their outputs, LCEL's `RunnableParallel` is usually the lighter abstraction. It invokes a mapping of runnables concurrently, supplies the same input to every branch, and returns a mapping containing their outputs. This is often sufficient for local scoring, content audits, independent retrieval calls, or side-by-side generation. LangGraph is usually clearer when the workflow requires runtime worker creation, reducer-backed shared state, durable checkpoints, loops, conditional continuation, explicit failure paths, or graph-level synchronization. The remaining subsections examine the application-level patterns that can be built from these primitives: map-reduce, voting, consensus, and aggregation. These are workflow designs rather than separate

LangGraph node types. They are implemented with ordinary nodes, edges, state fields, reducers, dynamic dispatch, and, when necessary, iterative loops.

3.3.2 Map-reduce

Map-reduce is a common fan-out/fan-in pattern. It divides a task into smaller independent units, processes those units separately, and then combines the results. The map phase applies the same operation to several input items. The reduce phase takes the outputs produced by those operations and combines them into one final result. This pattern is widely used in distributed data processing, but it also fits many LLM workflows. A long document can be divided into chunks, a report into sections, and a list of companies into individual stock symbols. Each unit can be processed separately, and the resulting outputs can then be combined. A support conversation can also be divided into messages, but only when each message contains enough information to be understood on its own. If a message depends on earlier parts of the conversation, the worker must also receive the relevant previous context. Another option is to divide the conversation into larger groups of related messages instead of processing every message in isolation. Map-reduce works well when every worker performs the same type of task and when each worker can finish without using the results produced by the other workers. For example, every worker may summarize one document chunk or analyse one company. When these conditions are satisfied, the work can be distributed cleanly, and the results can be combined afterward.

Simple map-reduce is not enough when tasks depend on one another. If one task requires the output of another task, the dependency must be represented explicitly in the graph through separate nodes and sequential super-steps. A different control-flow pattern may be needed when a coordinator must create tasks, assign them to workers, inspect their results, or request additional work. This is usually described as an orchestrator-worker pattern. A consensus pattern may be needed when several participants must compare their conclusions and satisfy an explicit agreement rule. These patterns can still contain map-reduce stages for the parts of the workflow that remain independent. The reduce phase can also become too large for a single model call. This happens when many workers produce long intermediate results. Passing all those results directly to one reduce node may exceed the model's context window or leave too little space for the final answer. In this situation, the workflow can add intermediate reduction levels. It first divides the mapped results into smaller groups. Each group is reduced into a shorter intermediate result. Those shorter results are then passed to the final reduce step. This is often called a collapse step or hierarchical reduction. For very large inputs, the process can be repeated several times until the remaining content fits safely within the model's input budget. The workflow is still based on map-reduce, but it contains several levels of reduction rather than one.

The following example summarizes a customer-support conversation for an internal case file. Each message in this example is sufficiently clear to be summarized independently. LangGraph creates one worker invocation for each message. Every worker summarizes its assigned message and returns one structured result. The workflow then collects all the message summaries and passes them to a final reduce node. That node combines them into one structured case summary. The important part of the example is the control flow rather than the exact prompt text. The conditional routing function returns one `Send` object for each

support message. Each `Send` object identifies the worker node to invoke and the message-specific state to pass to it. Every worker returns a partial state update containing one summary. The `chunk_summaries` field uses a reducer, so LangGraph can combine the updates returned by all the parallel worker invocations. Parallel workers may finish in a different order from the original messages. For this reason, every result contains the original message index. Before constructing the final prompt, the reduce node sorts the summaries by that index. The workflow therefore reads the support messages, creates one worker invocation for each message, summarizes the messages in parallel, collects the results through the reducer, restores the original order, and combines the summaries into the final case report.

```
import operator
from typing import Annotated
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model
from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send
from .config import OPENAI_API_KEY

llm = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class ChunkSummary(TypedDict):
    index: int
    summary: str

class SupportThreadState(TypedDict):
    thread_messages: list[str]
    chunk_summaries: Annotated[list[ChunkSummary], operator.add]
    case_summary: str

class WorkerState(TypedDict):
    index: int
    message: str
    chunk_summaries: Annotated[list[ChunkSummary], operator.add]

def prepare(state: SupportThreadState) -> dict:
    print("\n[PREPARE]")
    print("input:", {"thread_messages": state["thread_messages"]})
    output = {}
    print("output:", output)
    return output

def dispatch_messages(state: SupportThreadState):
    sends = [
        Send("summarize message", {"index": i, "message": message})
        for i, message in enumerate(state["thread_messages"])
    ]

    print("\n[DISPATCH]")
    for i, message in enumerate(state["thread_messages"]):
        print(
            f"branch {i} input:",
            {"index": i, "message": message}
        )

    return sends

def summarize_message(state: WorkerState) -> dict:
    print(f"\n[MAP branch {state['index']}]")
    print("input:", {"index": state["index"], "message": state["message"]})

    result = llm.invoke(
        [
            SystemMessage(
                content = (
                    "Summarize this support-thread message for an internal case file. "
                    "Keep the summary factual, brief, and operational."
                )
            ),
            HumanMessage(content = state["message"])
        ]
    )

    output = {
        "chunk_summaries": [
```

```

        {
            "index": state["index"],
            "summary": result.content
        }
    ]
}

print("output:", output)
return output

def reduce_case(state: SupportThreadState) -> dict:
    print("\n[REDUCE]")
    print("input:", {"chunk_summaries": state["chunk_summaries"]})

    ordered = sorted(state["chunk_summaries"], key = lambda item: item["index"])
    summaries_text = "\n".join(
        f"{item['index'] + 1}. {item['summary']}" for item in ordered
    )

    print("ordered summaries text:")
    print(summaries_text)

    result = llm.invoke(
        [
            SystemMessage(
                content = (
                    "You are preparing an internal support case summary. "
                    "Combine the message-level summaries into one concise report with "
                    "three sections: customer problem, key facts, and "
                    "recommended_next_step."
                )
            ),
            HumanMessage(content = summaries_text)
        ]
    )

    output = {"case_summary": result.content}
    print("output:", output)
    return output

builder = StateGraph(SupportThreadState)
builder.add_node("prepare", prepare)
builder.add_node("summarize_message", summarize_message)
builder.add_node("reduce_case", reduce_case)

builder.add_edge(START, "prepare")
builder.add_conditional_edges("prepare", dispatch_messages, ["summarize_message"])
builder.add_edge("summarize_message", "reduce_case")
builder.add_edge("reduce_case", END)

graph = builder.compile()

initial_state = {
    "thread_messages": [
        "I changed my shipping address yesterday but the parcel still went to the old address.",
        "I was charged twice after I tried to update the order.",
        "I need to know whether the second charge will be refunded automatically."
    ]
}

print("\n=== STREAMED EXECUTION ===")
for event in graph.stream(initial_state, stream_mode = "updates"):
    print("\nEVENT:", event)

print("\n=== FINAL RESULT ===")
state = graph.invoke(initial_state)
print(state["case_summary"])

```

vol3_ch_03\src\map_reduce.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.map_reduce

=== STREAMED EXECUTION ===

[PREPARE]
input: {'thread_messages': ['I changed my shipping address yesterday but the parcel still went to the old address.', 'I was charged twice after I tried to update the order.', 'I need to know whether the second charge will be refunded automatically.']}
output: {}

[DISPATCH]

```



```

branch 0 input: {'index': 0, 'message': 'I changed my shipping address yesterday but the
parcel still went to the old address.'}
branch 1 input: {'index': 1, 'message': 'I was charged twice after I tried to update the
order.'}
branch 2 input: {'index': 2, 'message': 'I need to know whether the second charge will be
refunded automatically.'}

EVENT: {'prepare': None}

[MAP branch 0]
input: {'index': 0, 'message': 'I changed my shipping address yesterday but the parcel
still went to the old address.'}

[MAP branch 1]
input: {'index': 1, 'message': 'I was charged twice after I tried to update the order.'}

[MAP branch 2]
input: {'index': 2, 'message': 'I need to know whether the second charge will be refunded
automatically.'}
output: {'chunk_summaries': [{'index': 1, 'summary': 'User reported being charged twice
following an attempt to update their order.'}]}

EVENT: {'summarize_message': {'chunk_summaries': [{'index': 1, 'summary': 'User reported
being charged twice following an attempt to update their order.'}]}}
output: {'chunk_summaries': [{'index': 2, 'summary': 'User inquired if the second charge
will be refunded automatically.'}]}

EVENT: {'summarize_message': {'chunk_summaries': [{'index': 2, 'summary': 'User inquired
if the second charge will be refunded automatically.'}]}}
output: {'chunk_summaries': [{'index': 0, 'summary': 'User updated their shipping address
yesterday, but the parcel was still delivered to the old address.'}]}

EVENT: {'summarize_message': {'chunk_summaries': [{'index': 0, 'summary': 'User updated
their shipping address yesterday, but the parcel was still delivered to the old
address.'}]}}

[REDUCE]
input: {'chunk_summaries': [{'index': 0, 'summary': 'User updated their shipping address
yesterday, but the parcel was still delivered to the old address.'}, {'index': 1,
'summary': 'User reported being charged twice following an attempt to update their
order.'}, {'index': 2, 'summary': 'User inquired if the second charge will be refunded
automatically.'}]}
ordered summaries text:
1. User updated their shipping address yesterday, but the parcel was still delivered to
the old address.
2. User reported being charged twice following an attempt to update their order.
3. User inquired if the second charge will be refunded automatically.
output: {'case_summary': 'customer problem:\nThe user updated their shipping address, but
the parcel was delivered to the old address. Additionally, the user was charged twice
after attempting to update their order and is inquiring whether the second charge will be
refunded automatically.\n\nkey_facts:\n- Shipping address was updated by the user
yesterday.\n- Parcel was delivered to the old address despite the update.\n- User
experienced a double charge following the order update attempt.\n- User is seeking
confirmation about automatic refund of the duplicate
charge.\n\nrecommended_next_step:\nInvestigate the order and shipping system to determine
why the updated address was not applied and why the parcel was sent to the old address.
Verify the billing records to confirm the double charge and initiate a refund process if
applicable. Communicate with the user to confirm the refund status and provide guidance
on how the shipping issue will be resolved.'}

EVENT: {'reduce_case': {'case_summary': 'customer problem:\nThe user updated their
shipping address, but the parcel was delivered to the old address. Additionally, the user
was charged twice after attempting to update their order and is inquiring whether the
second charge will be refunded automatically.\n\nkey_facts:\n- Shipping address was
updated by the user yesterday.\n- Parcel was delivered to the old address despite the
update.\n- User experienced a double charge following the order update attempt.\n- User
is seeking confirmation about automatic refund of the duplicate
charge.\n\nrecommended_next_step:\nInvestigate the order and shipping system to determine
why the updated address was not applied and why the parcel was sent to the old address.
Verify the billing records to confirm the double charge and initiate a refund process if
applicable. Communicate with the user to confirm the refund status and provide guidance
on how the shipping issue will be resolved.'}}

=== FINAL RESULT ===

[PREPARE]

```

```

input: {'thread_messages': ['I changed my shipping address yesterday but the parcel still
went to the old address.', 'I was charged twice after I tried to update the order.', 'I
need to know whether the second charge will be refunded automatically.']}
output: {}

[DISPATCH]
branch 0 input: {'index': 0, 'message': 'I changed my shipping address yesterday but the
parcel still went to the old address.'}
branch 1 input: {'index': 1, 'message': 'I was charged twice after I tried to update the
order.'}
branch 2 input: {'index': 2, 'message': 'I need to know whether the second charge will be
refunded automatically.'}

[MAP branch 0]
input: {'index': 0, 'message': 'I changed my shipping address yesterday but the parcel
still went to the old address.'}

[MAP branch 1]

[MAP branch 2]input: {'index': 1, 'message': 'I was charged twice after I tried to update
the order.'}

input: {'index': 2, 'message': 'I need to know whether the second charge will be refunded
automatically.'}
output: {'chunk_summaries': [{'index': 0, 'summary': 'User updated shipping address
yesterday, but the parcel was still delivered to the old address.'}]}
output: {'chunk_summaries': [{'index': 2, 'summary': 'User inquired if the second charge
will be refunded automatically.'}]}
output: {'chunk_summaries': [{'index': 1, 'summary': 'User reported being charged twice
following an attempt to update their order.'}]}

[REDUCE]
input: {'chunk_summaries': [{'index': 0, 'summary': 'User updated shipping address
yesterday, but the parcel was still delivered to the old address.'}, {'index': 1,
'summary': 'User reported being charged twice following an attempt to update their
order.'}, {'index': 2, 'summary': 'User inquired if the second charge will be refunded
automatically.'}]}
ordered summaries text:
1. User updated shipping address yesterday, but the parcel was still delivered to the old
address.
2. User reported being charged twice following an attempt to update their order.
3. User inquired if the second charge will be refunded automatically.
output: {'case_summary': 'customer_problem:\n\nThe user updated their shipping address, but
the parcel was delivered to the old address. Additionally, the user was charged twice
after attempting to update their order and is inquiring whether the second charge will be
refunded automatically.\n\nkey_facts:\n- Shipping address was updated by the user
yesterday.\n- Parcel was delivered to the original, old address despite the update.\n-
User experienced a double charge related to the order update.\n- User is seeking
confirmation about the refund process for the duplicate
charge.\n\nrecommended next step:\nInvestigate the order and shipping system to determine
why the updated address was not applied and why the parcel was sent to the old address.
Verify the billing records to confirm the double charge and initiate a refund if
applicable. Communicate with the user to explain the situation, confirm the refund
process, and provide guidance on how to receive the parcel or arrange redelivery.'}
customer_problem:
The user updated their shipping address, but the parcel was delivered to the old address.
Additionally, the user was charged twice after attempting to update their order and is
inquiring whether the second charge will be refunded automatically.

key_facts:
- Shipping address was updated by the user yesterday.
- Parcel was delivered to the original, old address despite the update.
- User experienced a double charge related to the order update.
- User is seeking confirmation about the refund process for the duplicate charge.

recommended next step:
Investigate the order and shipping system to determine why the updated address was not
applied and why the parcel was sent to the old address. Verify the billing records to
confirm the double charge and initiate a refund if applicable. Communicate with the user
to explain the situation, confirm the refund process, and provide guidance on how to
receive the parcel or arrange redelivery.

```

The streamed events make one operational point visible: completion order and business order are different. In this run, the workers for messages 1 and 2 finish before the worker for message 0. The workflow must therefore not infer the original conversation sequence from the order in

which branch events appear. The explicit index gives each result a stable business position. The reduce node sorts on that value before constructing the synthesis prompt, so the final case summary follows the original conversation order regardless of branch timing. Because the worker invocations belong to the same mapped fan-out stage, the reduce node receives the completed collection rather than running once for each streamed partial result.

3.3.3 Voting

Voting can be implemented as a fan-out/fan-in pattern, but it differs from map-reduce in what the branches usually process. In a typical map-reduce workflow, each branch processes a different item or section of the input. In a voting workflow, several branches normally evaluate the same request or decision and cast choices within a shared answer space. The fan-in step then applies a defined voting rule, such as majority voting, weighted voting, approval voting, or rank aggregation. Voting is related to self-consistency, but the techniques are not identical. Self-consistency repeatedly samples the same reasoning task and aggregates the normalized final answers. A voting ensemble may instead use different models, prompts, roles, evidence sources, or review perspectives. Voting should also be distinguished from candidate selection. If branches produce complete open-ended answers and a separate model judges which answer is best, the workflow is using judge-based selection rather than voting. Similarly, adding numerical quality scores and choosing the highest-scoring candidate is score-and-select unless those scores are explicitly defined as weighted votes. Direct voting works best when every branch returns a value from the same compact answer space. Examples include classification labels, risk tiers, escalation levels, and accept/reject decisions. Raw free-form text cannot normally be tallied directly because semantically equivalent answers may use different wording. The workflow must first normalize or group equivalent answers, or it must use a separate ranking or selection mechanism.

A support-priority example makes the pattern concrete. The same ticket is sent to three reviewer branches in parallel. Each branch applies a different review lens and returns one structured label (normal, priority, or urgent) together with a short reason. The reasons are retained so that the decision can be inspected, but the resolver uses only the labels when calculating the result. The fan-in node selects the label with the unique highest vote count. If several labels have the same highest count, it applies a deterministic, risk-sensitive rule and selects the most severe label among those that are tied. This makes tie handling predictable, although it does not prove that the selected label is correct. The example does not imply that every decision requires three reviewers. It shows how parallel branches can collect several separately produced judgments before the workflow commits to a result. Static fan-out is appropriate because the reviewer branches are known when the graph is built. The reducer on the votes field combines the records returned by the parallel branches before the resolver runs.

```
import operator
from collections import Counter
from typing import Annotated, Literal
from pydantic import BaseModel, Field
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model
from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.graph import StateGraph, START, END
from .config import OPENAI_API_KEY

reviewer_llm = init_chat_model("openai:gpt-4.1-mini", temperature = 0.7)
```

```

class PriorityVote(BaseModel):
    label: Literal["normal", "priority", "urgent"] = Field(
        description = "Escalation priority for this support request."
    )
    reason: str = Field(description = "Brief reason for the priority choice.")

structured_reviewer = reviewer_llm.with_structured_output(PriorityVote)

class VoteRecord(TypedDict):
    reviewer: str
    label: str
    reason: str

class TicketState(TypedDict):
    ticket_text: str
    votes: Annotated[list[VoteRecord], operator.add]
    final_priority: str

def make_reviewer(reviewer_name: str, lens: str):
    def review(state: TicketState) -> dict:
        result = structured_reviewer.invoke(
            [
                SystemMessage(
                    content = (
                        "Classify the escalation priority of the ticket. "
                        "Return one label from: normal, priority, urgent. "
                        f"Use this review lens: {lens}"
                    )
                ),
                HumanMessage(content = state["ticket_text"])
            ]
        )
        return {
            "votes": [
                {
                    "reviewer": reviewer_name,
                    "label": result.label,
                    "reason": result.reason
                }
            ]
        }
    return review

def resolve_vote(state: TicketState) -> dict:
    counts = Counter(vote["label"] for vote in state["votes"])
    ranked = counts.most_common()

    if len(ranked) == 1 or ranked[0][1] > ranked[1][1]:
        winner = ranked[0][0]
    else:
        safety_order = {"normal": 0, "priority": 1, "urgent": 2}
        tied_labels = [label for label, count in ranked if count == ranked[0][1]]
        winner = max(tied_labels, key = lambda label: safety_order[label])

    return {"final_priority": winner}

builder = StateGraph(TicketState)
builder.add_node(
    "operations_reviewer",
    make_reviewer(
        "operations_reviewer",
        "Focus on delivery failure, refund urgency, and the risk of unresolved account impact."
    )
)
builder.add_node(
    "policy_reviewer",
    make_reviewer(
        "policy_reviewer",
        "Focus on whether the request suggests a policy exception, compliance issue, or manual intervention."
    )
)
builder.add_node(
    "customer_impact_reviewer",
    make_reviewer(
        "customer_impact_reviewer",
        "Focus on customer harm, frustration, repeated contact, and whether the issue is getting worse."
    )
)

```

```

builder.add_node("resolve_vote", resolve_vote)

builder.add_edge(START, "operations_reviewer")
builder.add_edge(START, "policy_reviewer")
builder.add_edge(START, "customer_impact_reviewer")

builder.add_edge("operations_reviewer", "resolve_vote")
builder.add_edge("policy_reviewer", "resolve_vote")
builder.add_edge("customer_impact_reviewer", "resolve_vote")
builder.add_edge("resolve_vote", END)

graph = builder.compile()

state = graph.invoke(
    {
        "ticket_text": (
            "My order was delivered to the wrong address, I have been charged twice, "
            "and I have already contacted support once without a refund."
        )
    }
)

print("\n=== REVIEWER VOTES ===")
for i, vote in enumerate(state["votes"], start = 1):
    print(f"\n[{i}] {vote['reviewer']}")
    print(f"    Label : {vote['label']}")
    print(f"    Reason: {vote['reason']}")

print("\n=== FINAL DECISION ===")
print(f"Priority: {state['final_priority']}")

```

vol3_ch_03\src\voting.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.voting

=== REVIEWER VOTES ===

[1] customer_impact_reviewer
Label : urgent
Reason: The customer is facing multiple issues including delivery to the wrong address,
double charging, and has already contacted support once without resolution, causing
significant frustration and harm.

[2] operations_reviewer
Label : urgent
Reason: The order was delivered to the wrong address, the customer was charged twice, and
there has been no refund despite prior contact, indicating a critical delivery failure
and urgent refund issue with potential unresolved account impact.

[3] policy_reviewer
Label : urgent
Reason: The request involves financial issues (double charge) and delivery errors,
requiring immediate manual intervention and potential policy exception to resolve
customer dissatisfaction.

=== FINAL DECISION ===
Priority: urgent

```

Because the reviewer branches run in parallel, the order of the records in `state["votes"]` is not guaranteed. The resolver depends only on the count for each label, so its result is independent of record order. Voting adds value only when the branches provide meaningful diversity. Repeating the same prompt with the same model and greedy or very low-variance decoding will usually produce highly similar judgments. The extra calls may then increase cost and latency without adding useful evidence. Different review lenses, prompts, models, tools, evidence sources, or sampling strategies can encourage diversity, but they do not make the resulting errors statistically independent. Agreement is a useful corroborating signal, not proof that the selected decision is correct. A voting workflow should therefore be evaluated against a single-reviewer baseline using representative labelled tickets, including ambiguous requests, minority classes, rare labels, disagreement cases, and ties.

3.3.4 Consensus

Consensus and voting are different decision protocols. The distinction is not simply whether the branches communicate. Participants may exchange arguments and still finish with an ordinary vote, while independently generated candidates may be passed directly to a selector without any discussion. In a voting workflow, the result is chosen by aggregating choices or preferences over a set of alternatives. In a consensus workflow, participants are given an opportunity to examine disagreement, revise or reconcile their positions, and accept a shared result under an explicit agreement rule. That rule may require unanimous acceptance or another application-defined threshold after the revision process. A one-pass majority tally remains voting; it does not become consensus merely because most participants chose the same answer. Consensus therefore often requires an iterative graph rather than one fan-out followed by one fan-in. Participants may inspect one another's proposals, identify conflicting assumptions, exchange critiques, revise their answers, and repeat the process until the agreement rule is satisfied. The workflow must also define a maximum number of rounds and a fallback for cases in which agreement is not reached. Closed-label classification usually does not need this machinery. Majority voting or another deterministic aggregation rule is normally simpler, cheaper, and easier to evaluate. Consensus becomes more relevant when different branches contribute complementary perspectives and the final result must reconcile them into one position that the participants explicitly accept.

For example, a trading workflow might include a macro analyst, a fundamentals analyst, and a risk analyst. A voting or scoring mechanism can select one analyst's recommendation. A consensus process goes further: the analysts examine the areas in which their assumptions conflict, revise their positions where appropriate, and produce or accept a shared recommendation. The system should preserve the drafts, critiques, revisions, and final agreement so that the reasoning behind the result can be inspected. Peer scoring, judge-model selection, and external verification are useful related mechanisms, but they are not consensus by themselves. Peer scoring ranks competing drafts. A judge model selects or synthesizes an answer. An external oracle verifies whether a candidate satisfies objective constraints. These mechanisms become part of a consensus workflow only when their results are fed back to the participants and used to establish an agreed shared position.

Consensus should earn its additional cost. Debate, critique, and revision can improve some tasks, but their value depends on the models, prompts, agreement protocol, task, and evaluation method. Every additional round increases model calls, token use, latency, coordination logic, and the number of possible failure paths. A practical production strategy is to begin with the simplest credible baseline, such as one model call, one reviewer, majority voting, or score-based selection. Add a consensus loop only when evaluation on representative cases shows that interaction and revision materially improve quality, reliability, or explainability. A complete consensus implementation would require at least four stages: initial proposals, exchange of critiques, revision or explicit acceptance, and an agreement check. The graph would repeat the review and revision stages until the agreement rule was satisfied or a maximum-round fallback was reached.

3.3.5 Peer-evaluated selection

Peer-evaluated selection is related to consensus because the branches inspect one another's work, but it has a different termination rule. The participants do not need to revise their

positions or accept a shared recommendation. Instead, they evaluate the competing candidates, and a selector chooses the candidate with the strongest accumulated score. This pattern is useful when open-ended candidates cannot be compared through ordinary majority voting, but the workflow does not justify a full consensus loop. It introduces inter-branch evaluation while remaining a one-pass selection process. The following LangGraph example uses two parallel stages. First, three analysts produce investment recommendations from different analytical perspectives. Second, each analyst scores the other drafts. A final selector totals the accepted scores and chooses the highest-scoring candidate.

```
from __future__ import annotations
import operator
from typing import Annotated, Literal
from pydantic import BaseModel, Field
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model
from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send
from .config import OPENAI_API_KEY

class DraftResult(BaseModel):
    action: Literal["buy", "hold", "sell"]
    thesis: str
    rationale: str

class ScoreItem(BaseModel):
    target: str
    score: int = Field(ge = 1, le = 10)
    reason: str

class PeerReviewResult(BaseModel):
    scores: list[ScoreItem]

draft_model = init_chat_model(
    "openai:gpt-4.1-mini",
    temperature = 0
).with_structured_output(DraftResult)

peer_review_model = init_chat_model(
    "openai:gpt-4.1-mini",
    temperature = 0
).with_structured_output(PeerReviewResult)

class DraftRecord(TypedDict):
    analyst: str
    action: str
    thesis: str
    rationale: str

class ScoreRecord(TypedDict):
    reviewer: str
    target: str
    score: int
    reason: str

class TradingState(TypedDict):
    market_brief: str
    drafts: Annotated[list[DraftRecord], operator.add]
    peer_scores: Annotated[list[ScoreRecord], operator.add]
    winning_analyst: str
    final_recommendation: str

class PeerScoringState(TypedDict):
    reviewer: str
    drafts: list[DraftRecord]
    peer_scores: Annotated[list[ScoreRecord], operator.add]

def make_analyst_node(analyst_name: str, lens: str):
    def analyst_node(state: TradingState) -> dict:
        result = draft_model.invoke(
            [
                SystemMessage(
                    content = (
                        "You are a trading analyst. "
                        f"Write one recommendation using this analytical lens: {lens}. "
                        "Return one action: buy, hold, or sell. "
                        "Keep the answer concise and practical."
                    )
                )
            ]
        )
        return {
            "analyst": analyst_name,
            "action": result.action,
            "thesis": result.thesis,
            "rationale": result.rationale
        }
```



```

        ),
        HumanMessage(content = state["market_brief"])
    ]
)
output = {
    "drafts": [
        {
            "analyst": analyst_name,
            "action": result.action,
            "thesis": result.thesis,
            "rationale": result.rationale
        }
    ]
}

print(f"[draft] {analyst_name}: {result.action} - {result.thesis}")
return output

return analyst_node

def prepare_peer_scoring(state: TradingState) -> dict:
    print("\n--- peer scoring ---")
    return {}

def dispatch_peer_scoring(state: TradingState):
    return [
        Send(
            "peer_score_worker",
            {
                "reviewer": draft["analyst"],
                "drafts": state["drafts"]
            }
        )
        for draft in state["drafts"]
    ]

def peer_score_worker(state: PeerScoringState) -> dict:
    reviewer = state["reviewer"]
    drafts = state["drafts"]

    reviewer_draft = next(d for d in drafts if d["analyst"] == reviewer)
    other_drafts = [d for d in drafts if d["analyst"] != reviewer]
    valid_targets = {d["analyst"] for d in other_drafts}

    other_drafts_text = "\n\n".join(
        [
            (
                f"Analyst: {d['analyst']}\n"
                f"Action: {d['action']}\n"
                f"Thesis: {d['thesis']}\n"
                f"Rationale: {d['rationale']}"
            )
            for d in other_drafts
        ]
    )

    result = peer_review_model.invoke(
        [
            SystemMessage(
                content = (
                    "You are participating in a lightweight consensus workflow. "
                    "Read your own draft and score the other drafts. "
                    "Score each other analyst from 1 to 10. "
                    "Judge them on clarity, evidence, risk awareness, and actionability"
                    "Do not score yourself."
                )
            ),
            HumanMessage(
                content = (
                    f"Your analyst name: {reviewer}\n\n"
                    f"Your draft:\n"
                    f"Action: {reviewer_draft['action']}\n"
                    f"Thesis: {reviewer_draft['thesis']}\n"
                    f"Rationale: {reviewer_draft['rationale']}\n\n"
                    f"Other drafts to score:\n{other_drafts_text}"
                )
            )
        ]
    )

```

```

peer_scores: list[ScoreRecord] = []
for item in result.scores:
    if item.target in valid_targets:
        peer_scores.append(
            {
                "reviewer": reviewer,
                "target": item.target,
                "score": item.score,
                "reason": item.reason
            }
        )
    print(f"[score] {reviewer} -> {item.target}: {item.score}/10")

return {"peer_scores": peer_scores}

def select_final_answer(state: TradingState) -> dict:
    totals = {draft["analyst"]: 0 for draft in state["drafts"]}

    for score in state["peer_scores"]:
        totals[score["target"]] += score["score"]

    print("\n--- totals ---")
    for analyst, total in totals.items():
        print(f"{analyst}: {total}")

    drafts_by_name = {draft["analyst"]: draft for draft in state["drafts"]}
    safety_order = {"sell": 2, "hold": 1, "buy": 0}

    winner = max(
        totals,
        key = lambda analyst: (
            totals[analyst],
            safety_order[drafts_by_name[analyst]["action"]]
        )
    )

    winning_draft = drafts_by_name[winner]

    print(f"\n[winner] {winner}: {winning_draft['action']} ({totals[winner]})")

    final_text = (
        f"Selected analyst: {winner}\n"
        f"Action: {winning_draft['action']}\n"
        f"Peer score total: {totals[winner]}"
    )

    return {
        "winning_analyst": winner,
        "final_recommendation": final_text
    }

builder = StateGraph(TradingState)

builder.add_node(
    "macro_analyst",
    make_analyst_node(
        "macro_analyst",
        "macro conditions, rates, inflation, and broad market context"
    )
)

builder.add_node(
    "fundamentals_analyst",
    make_analyst_node(
        "fundamentals_analyst",
        "revenue, margins, cash flow, balance sheet, and valuation"
    )
)

builder.add_node(
    "risk_analyst",
    make_analyst_node(
        "risk_analyst",
        "downside scenarios, volatility, uncertainty, and capital preservation"
    )
)

builder.add_node("prepare_peer_scoring", prepare_peer_scoring)
builder.add_node("peer_score_worker", peer_score_worker)
builder.add_node("select_final_answer", select_final_answer)

builder.add_edge(START, "macro_analyst")
builder.add_edge(START, "fundamentals_analyst")
builder.add_edge(START, "risk_analyst")

builder.add_edge("macro_analyst", "prepare_peer_scoring")

```

```

builder.add_edge("fundamentals_analyst", "prepare_peer_scoring")
builder.add_edge("risk_analyst", "prepare_peer_scoring")

builder.add_conditional_edges(
    "prepare_peer_scoring",
    dispatch_peer_scoring,
    [{"peer_score_worker"}]
)

builder.add_edge("peer_score_worker", "select_final_answer")
builder.add_edge("select_final_answer", END)

graph = builder.compile()

initial_state = {
    "market_brief": (
        "The company posted solid revenue growth, but margin pressure remains. "
        "Interest rates are still elevated, and sector volatility has increased. "
        "Debt is manageable, but guidance was cautious."
    )
}

state = graph.invoke(initial_state)

print("\n=== FINAL RESULT ===")
print(state["final_recommendation"])

```

ch_03\src\consensus_peer_scores.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.consensus_peer_scores

[draft] fundamentals_analyst: hold - Maintain position due to solid revenue growth offset
by margin pressures and cautious guidance amid sector volatility.
[draft] risk_analyst: hold - Maintain position due to solid revenue growth but margin
pressures and elevated interest rates increase risk.
[draft] macro_analyst: hold - Maintain a hold position due to solid revenue growth offset
by margin pressures and cautious guidance amid elevated interest rates and sector
volatility.

--- peer scoring ---
[score] risk_analyst -> fundamentals_analyst: 8/10
[score] risk_analyst -> macro_analyst: 9/10
[score] macro_analyst -> fundamentals_analyst: 8/10
[score] macro_analyst -> risk_analyst: 7/10
[score] fundamentals_analyst -> macro_analyst: 9/10
[score] fundamentals_analyst -> risk_analyst: 7/10

--- totals ---
fundamentals_analyst: 16
macro_analyst: 18
risk_analyst: 14

[winner] macro_analyst: hold (18)

=== FINAL RESULT ===
Selected analyst: macro_analyst
Action: hold
Peer score total: 18

```

The graph has two fan-out/fan-in stages. In the first stage, the analyst factory creates three branches that receive the same market brief but apply different analytical lenses. Each branch returns one draft, and the reducer on drafts collects those results. Their list order has no business meaning because parallel updates are not guaranteed to arrive in a fixed order.

The `prepare_peer_scoring` node marks the boundary between generation and review. `dispatch_peer_scoring` then uses `Send` to create one reviewer task for each analyst. Every reviewer receives the complete draft collection, retains its own draft as context, and scores only the other candidates. The `valid_targets` check rejects self-scores and unknown analyst names. A production implementation should also verify that every expected reviewer-target pair appears exactly once and that no duplicate scores are accepted.

The selector totals the accepted scores and chooses the highest-scoring draft. Its sell-before-hold-before-buy tie-breaker is deterministic, but it is only an illustrative policy. It is not a universal definition of financial safety. The workflow selects an existing candidate; it does not synthesize a new recommendation, verify the market claims, or establish agreement among the analysts. It should therefore be evaluated as a peer-scored selection mechanism and compared with simpler baselines for quality, stability, latency, and cost.

3.3.6 Aggregation and fan-in contracts

Aggregation is the general fan-in design problem: how should the workflow combine the outputs produced by several branches? Map-reduce, voting, peer evaluation, and consensus all require some form of aggregation, but they do not use the same merge rule.

It is useful to distinguish the mechanism that collects branch updates from the logic that produces the final result. A reducer combines updates written to a state field. A downstream aggregation node may then calculate, select, synthesize, validate, or structure the collected results. Treating these as one operation hides important design decisions.

Five aggregation forms appear frequently in graph-based workflows:

- **Collection or accumulation:** parallel branches contribute values to a shared state field. For a list-valued field, `operator.add` concatenates the current list with each returned list, so branches normally return one-item lists such as `[result]`. Use a custom reducer when the workflow must deduplicate values, preserve an explicit order, merge dictionaries, reject invalid updates, or limit the size of the accumulated collection.
- **Deterministic reduction:** the fan-in step calculates a result using ordinary code. Examples include counting labels, summing scores, computing an average, selecting a minimum or maximum, applying a threshold, or checking whether all branch results satisfy a rule. This is usually preferable to another model call when the merge rule is objective.
- **Synthesis:** an aggregation node reads several partial results and creates one combined answer, summary, report, or explanation. This is a common reduce step in map-reduce workflows, although map-reduce may also use deterministic reduction when its outputs are numerical or structured.
- **Selection or ranking:** the workflow retains the candidates as alternatives and chooses one or more of them instead of blending them. Selection may use deterministic scores, business rules, peer scores, a model-based judge, or an external verifier. Tree-of-Thought pruning and final-candidate selection belong in this category. A model-based judge can be useful for open-ended candidates, but it remains a fallible evaluation method rather than a guarantee that the best answer was selected.
- **Structured composition:** branch outputs are assembled into different parts of a final typed object. For example, one branch may produce a market summary, another a risk assessment, and another a recommendation. If the branches write to different state keys, each key can normally be updated separately. If several branches update the same dictionary-valued key in one parallel step, that key needs a reducer that defines how the dictionaries are merged.

Parallel execution does not improve the final result by itself. It only produces several branch outputs. The fan-in logic determines whether those outputs become a useful decision, a coherent answer, or a larger collection of unresolved errors. A separate aggregation node is valuable when fan-in requires calculation, selection, synthesis, validation, or conflict resolution. It gives that logic an explicit place in the graph, with its own rules, prompt, trace, and evaluation criteria. The workflow can then be inspected at two levels: what each branch

produced and how the final result was derived from those outputs. A dedicated node is not required when state combination is simple and contains no business decision. In that case, a reducer may be sufficient. The reducer should be associative so that updates can be combined in stages without changing their meaning. If the result must also be independent of parallel update order, the reducer should be commutative. When the merge is order-sensitive, every update must carry stable ordering information that a downstream node can use to normalize the result. List concatenation is therefore safe only when list order has no business meaning or when the required order is restored explicitly.

3.3.7 Choosing the right parallel pattern

The first design question is what each branch receives and what the fan-in stage must produce.

Pattern	What the branches receive	Fan-in result	Iterative	Best suited to
Map-reduce	Different independent items	Combined or reduced result	Usually no	Repeated processing over documents, records, chunks, or report sections
Voting	The same decision	Winning normalized choice	No	Labels, priorities, risk tiers, rankings, and accept/reject decisions
Score-and-select	Open-ended competing candidates	Highest-ranked candidate	No	Plans, drafts, recommendations, and explanations
Structured composition	Different responsibilities or fields	One combined typed object	No	Multi-part reports and specialist contributions
Consensus	Proposals plus peer feedback	Shared accepted position	Usually yes	Reconciliation of competing perspectives

Use map-reduce when a collection of independent items can be processed with the same operation and their partial results can later be combined. Use voting when branches choose from a common normalized answer space. Use score-and-select when the outputs are open-ended candidates that should remain separate. Use structured composition when each branch owns a different part of the final object. Use consensus only when participants must review disagreement, revise their positions, and satisfy an explicit agreement rule. For every pattern, define the fan-in contract before implementation. Specify which results are expected, how missing or failed branches are handled, whether duplicates are possible, whether ordering matters, which validation rules apply, and how the final output is produced. This prevents branch management, validation, ranking, and synthesis from being hidden inside one large prompt. Always compare the selected design with the simplest credible baseline. Depending on the problem, that baseline may be one model call, one reviewer, a deterministic rule, or majority voting. Additional branches are justified only when they produce a measurable improvement in quality, reliability, latency-adjusted value, or explainability. Finally, keep workflow-level parallelization separate from concurrency inside a node. Workflow-level parallelization makes branches, dependencies, state updates, failure paths, and aggregation rules visible in the graph. Internal concurrency—such as asynchronous API calls, provider-side batching, or parallel tool execution—remains an implementation detail of one node unless those operations are modelled as separate graph tasks. The two forms can be combined, but they solve different architectural problems. The central rule is simple: parallelize only work

that is genuinely independent, make every merge rule explicit, and choose the least complex fan-in mechanism that satisfies the task.

3.4 INTEGRATING ROUTING AND PARALLELIZATION WITH RAG AND TOOLS

At this point, the individual patterns introduced in the chapter need to be combined into one end-to-end workflow. Planning identifies the work that may be required. Routing decides which parts of that work are relevant to the current request. RAG supplies external evidence that can ground an answer. Tools retrieve data, perform deterministic operations, or, when permitted, change external systems. Parallelization reduces elapsed time by running independent operations concurrently. Orchestration coordinates these capabilities so that the system performs the necessary work in the correct order and combines the results safely.

Consider the request used later in the worked example: “My order ORD-100 arrived damaged. Can I get a refund, and how long will it take?” The request requires both current order information and policy evidence. A router can activate an order-lookup branch together with the relevant policy-retrieval branches. Because these operations do not depend on one another, they can run concurrently. A later fan-in step can then combine the order facts with the policy conditions and produce one coherent response. Routing and parallelization solve different control-flow problems. Routing selects the work that should run. Parallelization determines which of the selected operations can run concurrently. RAG and tools are also architectural roles rather than mutually exclusive implementation categories because a retriever, database query, or external service may itself be exposed as a tool. The following sections use the same support request to explain how these decisions are ordered, executed, synchronized, and combined in LangGraph.

3.4.1 The control-flow order

A useful way to structure the integrated workflow is in three stages: decide what work is needed, execute the selected work, and combine the results.

First, the workflow routes the request when it has enough reliable information to choose the correct path. Sometimes the decision can be made directly from the user request. In other cases, the system must first validate the input, authenticate the user, retrieve preliminary information, or call a service. The router should therefore be placed at the earliest point where the decision can be made safely. In LangGraph, a conditional edge can select one destination or return several destination names. When several names are returned, the selected nodes receive the same graph-state snapshot and run in parallel in the next super-step. A node can instead return `Command` when it must combine a state update with a routing decision. Use `Send` when the number of worker invocations is determined at runtime or when each worker must receive a different input state.

Second, the workflow runs independent operations in parallel and keeps dependent operations sequential. For example, a trading assistant can retrieve news, filings, and market prices concurrently. However, it cannot calculate an indicator until the required price data has been returned. When several parallel workers update the same state field, a reducer defines how those updates are combined. A reducer may append results to a list, add numeric values, or merge structured records. A later aggregation node then interprets the collected results by filtering, ranking, comparing, validating, or synthesizing them. Parallel read-only operations,

such as retrieval and data lookups, are generally easier to coordinate than state-changing operations. However, they still require authorization, timeouts, rate-limit handling, consistency rules, and explicit failure policies. Actions that modify external systems require additional controls because their effects may persist outside the graph. Authorization, ordering, human approval, idempotency, retries, and recovery must be handled explicitly because LangGraph does not automatically reverse an external action when another branch fails.

Third, the workflow combines the partial results into one answer, decision, or next action. This fan-in step must resolve duplication, conflicting evidence, missing information, and business rules. It should not merely concatenate the branch outputs. Validation can occur before generation, after generation, or at both points. The workflow may check whether the retrieved evidence is relevant and sufficient, then verify that the generated answer is supported by that evidence. If validation fails, the graph can retrieve more information, reformulate the query, regenerate the answer, request clarification, or escalate the case.

This produces a simple control-flow rule: route when enough information is available, run independent work concurrently, keep dependencies sequential, and explicitly validate and combine the results before returning an answer or performing an action.

3.4.2 Where RAG fits in the integrated workflow

In a routed workflow that supports several capabilities, retrieval does not need to run for every request. This differs from a dedicated two-step RAG pipeline, where retrieval deliberately precedes every generated answer. The decision should not depend on whether the model may already have encountered a fact during training. Model knowledge is neither authoritative nor directly inspectable. Retrieval is appropriate when the answer requires current, private, domain-specific, authoritative, or traceable information. The router can decide whether retrieval is needed and which source should be searched. A support request about return conditions should search the policy knowledge base. A request for the current status of an order should query the order-management system. A request that needs both can activate both paths. Scoping each branch to the relevant source can improve precision and avoid unnecessary retrieval work, model calls, latency, and cost. The selected branch remains responsible for its own retrieval strategy. A policy branch may rewrite the question before searching. A product branch may apply metadata filters for category, price, or availability. A research branch may decompose a complex request into several smaller searches. After retrieval, the branch may remove irrelevant or duplicate results, rerank candidates, or verify that the evidence is sufficient. In Agentic RAG, retrieval is commonly exposed through one or more tools, allowing the agent to decide when and how to retrieve, inspect the returned evidence, and retrieve again when necessary. More controlled hybrid designs can keep retrieval, evaluation, and retry decisions as explicit graph steps. In either design, retrieved evidence should be validated before it is used to generate the final answer.

3.4.3 Where tools fit

Tools connect the workflow to capabilities outside the model. They can retrieve current or private information, perform calculations, query databases, update business records, or trigger external processes. When the required operation is already known, a LangGraph node can

invoke the tool directly. In a custom model-driven tool loop, the model emits one or more tool calls and a `ToolNode` executes them. A `ToolNode` can execute multiple tool calls in parallel. The `tools_condition` helper examines the most recent `AIMessage` and, by default, routes to the tools node when tool calls are present or to `END` when they are absent. The most important operational distinction is between read tools and tools that change external systems. Read tools include order-status lookups, customer-record queries, market-data requests, document searches, and approved read-only database queries. Independent reads can often run concurrently, but they still require authorization, tenant isolation, privacy controls, timeout and rate-limit handling, and appropriate consistency rules. Write tools include cancelling an order, issuing a refund, updating a customer record, escalating a support case, or submitting a transaction. These operations require stronger controls because an incorrect, duplicated, or partially completed call can create real business consequences. A safe workflow normally gathers and validates the required evidence before enabling a write operation. It should then verify authorization, resource ownership, business rules, argument validity, idempotency requirements, and any required approval. LangGraph interrupts support approval workflows by pausing execution and returning control to the application. Interrupt-based workflows require a checkpoint and a stable thread identifier so that the same execution can later be resumed. The application, or higher-level human-in-the-loop middleware, is responsible for presenting the proposed operation and interpreting the reviewer's decision to approve, edit, or reject it. Routing does not replace these controls. A router selects the part of the workflow that should handle a request; it does not grant access to data or authorize an action. Authentication, permissions, tenant boundaries, business policies, and approval rules must be enforced by the application and tool layer.

3.4.4 A worked support example

The worked example implements the support request introduced at the beginning of the section: "My order ORD-100 arrived damaged. Can I get a refund, and how long will it take?" The assistant cannot answer the request from one source. It needs the current order details, the damaged-item policy, and the refund-timing policy. The router identifies the request as mixed and selects the required branches. One branch looks up the order, while the other branches retrieve the relevant policy evidence. Because these operations do not depend on one another, LangGraph can run them concurrently. The architectural requirement is that synthesis should run only after all required branches have completed. The graph shown below deliberately exposes a subtle synchronization issue that can occur when parallel branches contain different numbers of steps. A previous section traces the execution and explains how to enforce the intended fan-in behaviour. The example uses a small policy corpus indexed in a local Chroma vector store with OpenAI embeddings. The `search_policy` tool performs semantic similarity search within the policy branch selected by the router. This keeps the dataset small while still demonstrating vector-based retrieval with metadata filtering.

A production implementation could use a larger or remote vector store, hybrid retrieval, query rewriting, reranking, richer metadata filters, authorization, retries, durable persistence, monitoring, concurrency limits, and explicit failure-handling paths. These additions would

strengthen the implementation without changing the fundamental route, fan-out, and fan-in structure.

```

from __future__ import annotations
import operator
import re
from typing import Annotated, Literal
from pydantic import BaseModel, Field
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langchain_core.documents import Document
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send
from .config import OPENAI_API_KEY

ORDERS = {
    .
    .
    .
}

POLICY_CORPUS = {
    .
    .
    .
}

def _extract_order_id(text: str) -> str | None:
    match = re.search(r"\bORD-\d+\b", text.upper())
    return match.group(0) if match else None

@tool
def lookup_order(order_id: str) -> dict:
    """Return normalized order facts for a support workflow."""
    if order_id not in ORDERS:
        raise ValueError(f"Unknown order id: {order_id}")
    return {"order_id": order_id, **ORDERS[order_id]}

def build_policy_vectorstore() -> Chroma:
    docs: list[Document] = []

    for branch, items in POLICY_CORPUS.items():
        for i, item in enumerate(items):
            docs.append(
                Document(
                    page_content = item["text"],
                    metadata = {
                        "branch": branch,
                        "source": item["source"],
                        "doc_id": f"{branch}-{i}"
                    }
                )
            )

    embeddings = OpenAIEmbeddings(model = "text-embedding-3-small")
    vectorstore = Chroma(
        collection_name = "support_policy",
        embedding_function = embeddings,
        persist_directory = "./chroma_policy_db"
    )

    existing = vectorstore.get()
    existing_ids = existing.get("ids", []) if existing else []

    if not existing_ids:
        vectorstore.add_documents(
            documents = docs,
            ids = [doc.metadata["doc_id"] for doc in docs]
        )

    return vectorstore

POLICY_VECTORSTORE = build_policy_vectorstore()

@tool
def search_policy(
    branch: Literal["returns", "damaged_items", "refund_timing", "gift_returns"],
    query: str,
    top_k: int = 2
) -> list[dict]:

```

```

"""Semantic search over the policy corpus using a Chroma vector store."""
hits = POLICY_VECTORSTORE.similarity_search(
    query = query,
    k = top_k,
    filter = {"branch": branch}
)

if not hits:
    return []

return [
    {
        "branch": doc.metadata["branch"],
        "source": doc.metadata["source"],
        "text": doc.page_content
    }
    for doc in hits
]

class RouteDecision(BaseModel):
    route: Literal["policy_only", "order_only", "mixed", "escalate"]
    policy_branches: list[
        Literal["returns", "damaged_items", "refund_timing", "gift_returns"]
    ] = Field(default_factory = list)
    needs_order_lookup: bool = Field(default = False)
    reason: str

class PolicyEvidence(TypedDict):
    branch: str
    source: str
    text: str

class SupportState(TypedDict):
    user_query: str
    order_id: str | None
    route: str
    route_reason: str
    policy_branches: list[str]
    policy_evidence: Annotated[list[PolicyEvidence], operator.add]
    order_data: dict | None
    final_answer: str

class PolicyWorkerState(TypedDict):
    user_query: str
    policy_branch: Literal["returns", "damaged_items", "refund_timing", "gift_returns"]
    policy_evidence: Annotated[list[PolicyEvidence], operator.add]

llm = init_chat_model("openai:gpt-4.1-mini", temperature = 0)
router = llm.with_structured_output(RouteDecision)

def route_request(state: SupportState) -> dict:
    order_id = _extract_order_id(state["user_query"])

    decision = router.invoke(
        [
            SystemMessage(
                content = (
                    "Route support requests into one of four paths.\n"
                    "policy_only: policy interpretation with no order facts needed.\n"
                    "order_only: order facts only, with no policy reasoning needed.\n"
                    "mixed: both policy interpretation and order facts are required.\n"
                    "escalate: legal, safety, abusive, or clearly human-only cases.\n"
                    "Return only the branches actually needed."
                )
            ),
            HumanMessage(
                content = (
                    f"User request: {state['user_query']}\n"
                    f"Extracted order id: {order_id}"
                )
            )
        ]
    )

    return {
        "order_id": order_id,
        "route": decision.route,
        "route_reason": decision.reason,
        "policy_branches": list(decision.policy_branches)
    }

def choose_after_route(

```

```

state: SupportState
) -> Literal["policy_only", "order_only", "mixed", "escalate"]:
    return state["route"]

def mixed_entry(state: SupportState) -> dict:
    return {}

def policy_fanout(state: SupportState) -> dict:
    return {}

def send_policy_workers(state: SupportState) -> list[Send]:
    return [
        Send(
            "policy_worker",
            {
                "user_query": state["user_query"],
                "policy_branch": branch
            }
        )
        for branch in state["policy_branches"]
    ]

def policy_worker(state: PolicyWorkerState) -> dict:
    hits = search_policy.invoke(
        {
            "branch": state["policy_branch"],
            "query": state["user_query"],
            "top_k": 2
        }
    )

    evidence: list[PolicyEvidence] = [
        {
            "branch": hit["branch"],
            "source": hit["source"],
            "text": hit["text"]
        }
        for hit in hits
    ]

    return {"policy_evidence": evidence}

def lookup_order_node(state: SupportState) -> dict:
    if not state.get("order_id"):
        return {"order_data": None}
    return {"order_data": lookup_order.invoke({"order_id": state["order_id"]})}

def escalate_node(state: SupportState) -> dict:
    return {
        "final_answer": "This request should be escalated to a human support specialist."
    }

def synthesize_response(state: SupportState) -> dict:
    order_block = state.get("order_data") or {"order_id": None, "status": "unknown"}
    evidence_lines = "\n".join(
        f"- [{item['branch']}] {item['source']}: {item['text']}"
        for item in state.get("policy_evidence", [])
    )

    response = llm.invoke(
        [
            SystemMessage(
                content = (
                    "You are a customer support assistant. "
                    "Answer only from the provided order facts and policy evidence. "
                    "Do not invent policy. If the evidence is incomplete, say what is "
                    "known and what still requires review."
                )
            ),
            HumanMessage(
                content = (
                    f"User request:\n{state['user_query']}\n\n"
                    f"Route reason:\n{state['route_reason']}\n\n"
                    f"Order facts:\n{order_block}\n\n"
                    f"Policy evidence:\n{evidence_lines}\n\n"
                    "Write one grounded answer for the customer."
                )
            )
        ]
    )

    return {"final_answer": response.content}

```

```

builder = StateGraph(SupportState)

builder.add_node("route", route_request)
builder.add_node("mixed_entry", mixed_entry)
builder.add_node("policy_fanout", policy_fanout)
builder.add_node("policy_worker", policy_worker)
builder.add_node("lookup_order", lookup_order_node)
builder.add_node("escalate", escalate_node)
builder.add_node("synthesize", synthesize_response)

builder.add_edge(START, "route")
builder.add_conditional_edges(
    "route",
    choose_after_route,
    {
        "policy_only": "policy_fanout",
        "order_only": "lookup_order",
        "mixed": "mixed_entry",
        "escalate": "escalate"
    }
)

builder.add_edge("mixed_entry", "policy_fanout")
builder.add_edge("mixed_entry", "lookup_order")

builder.add_conditional_edges("policy_fanout", send_policy_workers, ["policy_worker"])

builder.add_edge("policy_worker", "synthesize")
builder.add_edge("lookup_order", "synthesize")
builder.add_edge("synthesize", END)
builder.add_edge("escalate", END)

graph = builder.compile()

if __name__ == "__main__":
    result = graph.invoke(
        {
            "user_query": (
                "My order ORD-100 arrived damaged. Can I get a refund, and how long "
                "will it take?"
            ),
            "policy_evidence": []
        }
    )

    print("\n=== FINAL ANSWER ===\n")
    print(result["final_answer"])

    print("\n=== ORDER DATA ===\n")
    print(result.get("order_data"))

    print("\n=== POLICY EVIDENCE ===\n")
    for item in result.get("policy_evidence", []):
        print(item)

```

vol3_ch_03\src\routing_tools_and_rag.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.routing_tools_and_rag

=== FINAL ANSWER ===
Your order ORD-100 was delivered 6 days ago and you have reported it as damaged, which
makes it eligible for a refund review. According to our policy, damaged items should be
reported as soon as possible, and since your damage has been confirmed, you may receive a
refund. Support may request photos or delivery evidence before finalizing the refund.
Once the refund is approved, it is usually issued within 5 business days. Please note
that the time for the refunded amount to appear in your account can vary depending on
your payment provider.

=== ORDER DATA ===
{'order_id': 'ORD-100', 'status': 'delivered', 'delivered_days_ago': 6,
'damaged_reported': True, 'eligible_for_refund_review': True, 'currency': 'EUR',
'amount': 129.0}

=== POLICY EVIDENCE ===
{'branch': 'damaged_items', 'source': 'damaged-items.md#reporting', 'text': 'Damaged
items should be reported as soon as possible. When damage is confirmed, the customer may
receive a replacement or refund.'}
{'branch': 'damaged_items', 'source': 'damaged-items.md#evidence', 'text': 'Support may
request photos or delivery evidence before confirming a damaged-item refund.'}
{'branch': 'refund_timing', 'source': 'refunds.md#timing', 'text': 'Approved refunds are
usually issued within 5 business days after review.'}

```

```
{'branch': 'refund_timing', 'source': 'refunds.md#payment-rail', 'text': 'The time for funds to appear can vary by payment provider after the refund has been issued.'}
```

3.4.5 How the graph executes and where synchronization matters

The graph separates the request into distinct responsibilities instead of asking one model call to perform routing, retrieval, tool use, and answer generation at the same time. First, `_extract_order_id` identifies the order number through deterministic code. The router then examines the request and decides whether it requires policy information, order information, both, or human escalation. For the example request, the route is mixed. The assistant needs the current facts for ORD-100 together with the damaged-item and refund-timing policies. Neither source is sufficient on its own. The `mixed_entry` node starts two paths. One path calls `lookup_order` to retrieve the order facts. The other enters `policy_fanout`, which creates one `policy_worker` invocation for each policy branch selected by the router. Each worker receives the original request and the specific policy branch it must search. `Send` is appropriate here because the number of policy searches is determined at runtime. A request may require one policy branch, several branches, or none. Each worker searches independently, and the reducer on `policy_evidence` combines the returned lists into one state field. Routing also makes retrieval more focused. The policy documents are stored in Chroma with metadata identifying their branch and source. The router first selects the relevant policy branches, and `search_policy` then performs semantic search only within those branches. A damaged-item request therefore does not search unrelated areas such as gift returns. Once all required evidence has been collected, the synthesis node should receive four distinct inputs: the original request, the reason for the selected route, the order facts, and the retrieved policy evidence. It can then produce one response grounded in both operational data and policy documentation. This separation makes failures easier to diagnose. If the answer is incorrect, the execution can be inspected stage by stage:

- Was the order identifier extracted correctly?
- Did the router select the correct route and policy branches?
- Did the order tool return the expected data?
- Did retrieval return the relevant policy text?
- Did synthesis use the available evidence correctly?

The graph as currently connected contains an important synchronization defect. The two mixed-request branches contain different numbers of steps. The order path goes directly from `lookup_order` to `synthesize`, while the policy path passes through `policy_fanout` and then through one or more policy workers. An ordinary downstream edge does not create a barrier that waits for the longest branch. As a result, `synthesize` can run after the shorter order path completes and then run again after the policy workers finish. The later invocation can overwrite the earlier answer with one based on more complete state, but the first invocation still performs an unnecessary model call using incomplete evidence. More importantly, the graph does not enforce the intended guarantee that synthesis must use every required input. The most direct correction for this example is to register the synthesis node with `defer=True`. A deferred node runs after the graph has completed its other pending tasks, making it suitable for final aggregation across branches of different lengths. A more explicit alternative is to maintain completion state and route to synthesis only after every required evidence-gathering operation has reported success or failure. This second approach is useful when the workflow needs different policies for partial failure. A failed branch may trigger a retry, use a fallback

source, produce a clearly labelled partial answer, fail the complete request, or escalate to a human. The aggregation step should never silently interpret a missing branch as successful evidence. Once synchronization is enforced, the workflow has a clear contract: classify the request, start the required evidence-gathering operations, collect their results, evaluate whether the evidence is complete, and generate one final answer only when the configured fan-in conditions have been satisfied. Query rewriting, reranking, richer metadata filters, and answer validation can then be added without changing the fundamental structure.

3.4.6 Design rules that keep the pattern useful

Keep sequencing, routing, and fan-out conceptually separate. Use a fixed sequence when the next step is predetermined, routing when the next path depends on runtime information, and fan-out when several selected operations can run independently. A graph can contain all three. At the orchestration level, it remains a workflow when its possible paths are defined by application code, even if individual nodes contain model-driven or agentic behaviour. Routing and parallel branches do not by themselves make the complete graph an agent. Parallelize only genuinely independent work. Use static branches when the destinations are known when the graph is built and each branch can receive the same shared state. Use `Send` when runtime data determines the number of worker invocations or when each worker needs different input. Parallel execution can reduce elapsed time, especially for I/O-bound operations, but it does not reduce the total amount of work. Dynamic parallelism must also be bounded. A runtime-generated worker list can exceed model-provider quotas, database connection limits, memory budgets, or the capacity of downstream services. Concurrency limits, timeouts, backpressure, cancellation behaviour, and rate-limit policies should therefore be part of the workflow design.

Keep state collection separate from result interpretation. A reducer defines how concurrent updates to a state field are combined. An aggregation node interprets the combined values by removing duplicates, comparing evidence, calculating scores, identifying conflicts, or synthesizing an answer. A reducer produces a well-defined merged state update; it does not guarantee that the resulting information is complete, correct, or semantically consistent. Define the fan-in policy for both successful and failed branches. The workflow should know which results are mandatory, which are optional, and what should happen when a branch times out or fails. Appropriate responses may include retrying the branch, using a fallback source, returning a clearly labelled partial result, failing the request, or escalating to a human. Keep retrieval scoped to the sources relevant to the request. Retrieval may occur before routing when preliminary evidence is required to make the routing decision. However, querying every available corpus should be an intentional strategy rather than the default response to routing uncertainty. Finally, separate evidence gathering from external side effects. Independent read operations are often suitable for parallel execution. State-changing tools should normally run only after evidence collection, authorization, business-rule validation, argument validation, and any required human approval. A `ToolNode` can execute multiple tool calls in parallel, and state fields that may receive concurrent tool updates need appropriate reducers. Those reducers coordinate LangGraph state updates; they do not coordinate, compensate, or reverse changes already made to external systems. Write tools therefore also require restricted permissions, idempotency controls, approval boundaries, audit records, and explicit recovery or compensation procedures.

3.4.7 When a simpler design is better

Not every problem needs the full graph. If the structure is fixed and you already know which independent branches should run, a simpler LCEL pipeline with `RunnableParallel` can be enough. That is often a good fit for stable fan-out/fan-in patterns such as “retrieve three known evidence sets, then synthesize,” or “run three independent analyses, then compare.” The graph becomes worth its extra explicitness when the number of branches depends on runtime data, when the workflow needs loops or validation, when state must survive longer-running execution, or when human approval and interrupts are part of the path. In other words, use the simplest orchestration surface that still makes the control flow visible.

The objective is not maximum autonomy, but explicit control. Each component should have a clear responsibility, and every dependency, side effect, failure condition, and aggregation point should remain visible in the workflow. When these boundaries are preserved, the system is easier to explain, test, operate, and trust.

3.5 COST/LATENCY CONTROL: TIERED MODELS, CACHING, EARLY EXITS

3.5.1 Cost and latency are workflow problems, not just provider settings

In agentic systems, a single slow model call can dominate latency, but end-to-end performance is often shaped by the complete execution path: the number of model requests, the tools exposed, the amount of context replayed, the timing of fan-out, and the number of loop iterations. Cost and latency therefore belong inside workflow design as well as provider configuration. LangChain exposes middleware hooks before, after, and around model and tool calls, including early-termination controls. LangGraph provides conditional routing, termination through END, node-level caching, and parallel execution for multiple outgoing edges. These controls let the application decide where expensive reasoning is justified, which work can be reused, and when execution can stop. The practical design rule is to resolve deterministic and inexpensive cases first, invoke stronger reasoning only when simpler paths cannot meet the required quality, and terminate once the state contains enough evidence for a safe answer, abstention, or escalation. OpenAI's latency guidance follows the same general approach: reduce requests, avoid an LLM when deterministic logic is sufficient, remove unnecessary context, and keep stable prompt prefixes ahead of dynamic content.

3.5.2 Tiered models: spend intelligence where it matters

A tiered design does not mean "always use the cheapest model first." It means separating cheap control work from expensive reasoning. Routing, query rewriting, retrieval-needed detection, narrow classification, and simple extraction are often good fits for smaller, faster, lower-cost models. Final synthesis, ambiguous policy interpretation, long-horizon planning, and recovery from reasoning or interpretation failures are the places where a stronger model may earn its additional cost. LangChain's agent middleware supports dynamic model selection at runtime through `@wrap_model_call`, and the official example uses exactly this pattern: shorter, simpler conversations go to a cheaper model, while longer or more complex ones are promoted to a stronger model. OpenAI recommends the same optimization order from a different angle: first hit your accuracy target with the strongest model, then preserve that accuracy with the cheapest, fastest model that still passes your tests. Anthropic's guidance also recommends starting with a faster, more cost-effective model for many workloads and only upgrading when evaluation shows a real capability gap. Google's Gemini documentation is equally explicit: Flash-Lite is positioned for high-volume, low-latency work, and Google even shows it being used as a routing classifier that chooses whether a harder request should go to Flash or Pro.

That leads to a practical default: the router should usually be cheaper than the model used for difficult answers, provided evaluation shows that routing accuracy remains acceptable. A lightweight model may be sufficient for requests such as "Where is ORD-104?" or "Show waterproof hiking boots under 150 euros," while a more capable model may be justified for an exception involving a missing delivery, a prior refund promise, and policy interpretation. Dynamic tiering and fallback must remain separate concepts. Dynamic tiering selects a model for the current call from task signals, workflow state, policy, or budget.

`ModelFallbackMiddleware` tries alternative models when the primary model call fails. It can improve resilience and may reduce cost if a cheaper fallback is acceptable, but it does not classify task complexity. Fallback therefore complements deliberate routing rather than replacing it.

A third rule is that tiering is not only about model IDs. Some providers also expose reasoning-depth controls. Gemini's `thinking_level`, for example, can be set to low or minimal on supported models to reduce latency and cost for simpler, high-throughput work. That gives you another useful lever: sometimes the right tier is not a different model, but a lighter reasoning setting for the same family. One more adjacent control matters here: dynamic tool filtering. LangChain explicitly warns that exposing too many tools can overload context and increase errors. Reducing the available tool set based on conversation stage, permissions, or authenticated state is therefore also a cost/latency optimization. Fewer tools mean a smaller prompt, a narrower search space for the model, and fewer wrong branches to explore. In practice, this is often the missing half of tiered design: not just "pick the right model," but also "show that model only the tools it can actually use right now."

```
from __future__ import annotations
import os
from langchain.agents import create_agent
from langchain.agents.middleware import (
    wrap_model_call,
    ModelRequest,
    ModelResponse,
)
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from .config import OPENAI_API_KEY
from .config import GOOGLE_API_KEY

CHEAP_MODEL = os.getenv("CHEAP_MODEL", "openai:gpt-5.4-mini")
STRONG_MODEL = os.getenv("STRONG_MODEL", "openai:gpt-5.4")

cheap_model = init_chat_model(CHEAP_MODEL, temperature = 0)
strong_model = init_chat_model(STRONG_MODEL, temperature = 0)

ORDERS = {
    "ORD-100": {"status": "delivered", "carrier": "DHL"},
    "ORD-101": {"status": "processing", "carrier": "n/a"}
}

POLICY = {
    "refunds": (
        "Refunds are allowed within 30 days for unused items. "
        "Damaged or missing deliveries require manual review."
    )
}

@tool
def lookup_order(order_id: str) -> str:
    """Look up a customer's order by order id."""
    order = ORDERS.get(order_id.upper())
    if not order:
        return "Order not found."
    return f"{order_id.upper()}: status={order['status']}, carrier={order['carrier']}"

@tool
def search_policy(topic: str) -> str:
    """Return a short policy snippet for the requested topic."""
    return POLICY.get(topic.lower(), "No policy found.")

HIGH_RISK_TERMS = {
    "chargeback",
    "fraud",
    "missing delivery",
    "damaged",
    "legal",
    "complaint",
    "refund exception"
}
```

```

@wrap_model_call
def choose_model(request: ModelRequest, handler) -> ModelResponse:
    last_user_message = request.state["messages"][-1].content.lower()
    long_conversation = len(request.state["messages"]) > 8
    high_risk = any(term in last_user_message for term in HIGH_RISK_TERMS)

    model = strong_model if (long_conversation or high_risk) else cheap_model
    return handler(request.override(model = model))

agent = create_agent(
    model = cheap_model,
    tools = [lookup_order, search_policy],
    middleware = [choose_model],
    system_prompt = (
        "You are a customer support assistant. "
        "Use tools when needed. Do not invent order facts or policy."
    )
)

result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Where is order ORD-100?"
            }
        ]
    }
)

print(result["messages"][-1].content)

```

vol3_ch_03\scr\choose_model.py

The important point in this example is not the exact heuristic but the control boundary it creates. The stronger model is no longer the default for every request. Simple order-status turns remain on the fast path, while higher-risk support turns are promoted deliberately. The same structure applies to the trading assistant: a cheaper model can distinguish basic price lookup and narrow structured extraction from deeper market analysis, after evaluation has confirmed that it can route those cases reliably. Only the final category needs promotion to the stronger model. LangChain adds one implementation constraint that matters when structured output is used: models selected dynamically through middleware should not already be pre-bound with tools or structured-output configuration. Tiering reduces the cost of model calls the workflow still needs to make. Caching addresses the next source of waste: performing the same stable work repeatedly.

3.5.3 Caching: reuse work at the right boundary

Caching is often discussed as if it were one feature. In practice, workflow caching and provider-side prompt caching solve different problems. LangGraph node caching reuses a node result for the same cache key. It is most appropriate for side-effect-free work whose result is stable for the configured time to live, such as deterministic normalization, parsing, or filter construction. A cached result is correct only when the cache key captures every input that can change the output. For model- or policy-dependent nodes, that may include the prompt version, model configuration, policy version, corpus version, tenant, permissions, and other relevant runtime context. Without those inputs, caching can return a valid result for the wrong version or user.

The second layer is provider-side prompt caching. It reuses the provider's processing of a repeated prompt prefix rather than a completed application-node result. LangChain distinguishes automatic provider caching from provider-specific caching controls. These

controls are not uniform: depending on the provider and API, they may include manually managed cache objects, cache breakpoints, cache keys, retention settings, or automatic repeated-prefix caching. Prompt caching is most useful when requests share a long, stable prefix containing system instructions, tool definitions, schemas, examples, or earlier conversation content. Dynamic material, including user-specific data and retrieved evidence, should appear after that stable prefix when the provider's prompt format allows it. Cache-usage fields are provider-specific and appear in LangChain usage metadata only when the underlying integration reports them. Prompt caching must also not be confused with conversation memory: it can reduce repeated processing, but it does not decide which history should be persisted or supplied to the next request. Provider details reinforce the distinction. OpenAI automatically caches eligible repeated prompt prefixes and reports cache reads through `cached_tokens` in the input-token usage details. For GPT-5.6 and later model families, it also reports cache writes through `cache_write_tokens` and supports explicit cache breakpoints. Because cache-write pricing and configuration differ across model families, applications should read the relevant usage fields and current pricing rules rather than assuming that every OpenAI model has the same caching behavior. Anthropic uses a five-minute cache duration by default, refreshes the cache when matching content is reused, and also supports an optional one-hour duration. Gemini provides implicit caching on supported models. Its `generateContent` API also supports explicitly created caches whose time to live defaults to one hour when none is supplied, while the newer Interactions API currently supports implicit caching rather than manually created cache objects.

These values describe provider behavior, not application cache policy, and they can change independently of the graph. At the application level, cache only work that is expensive, repeated, correctly scoped, and safe to reuse. Final answers should be cached only when the key includes every dependency that affects correctness, including source-data version, prompt and model configuration, user or tenant scope, authorization context, and policy version. Otherwise the optimization can produce fast but stale, incorrectly scoped, or unauthorized answers. A small LangGraph example makes the workflow layer concrete:

```
from __future__ import annotations
import re
import time
from typing_extensions import TypedDict
from langgraph.graph import START, StateGraph
from langgraph.cache.memory import InMemoryCache
from langgraph.types import CachePolicy

class SearchState(TypedDict, total = False):
    question: str
    normalized_query: str
    filters: dict

def normalize_query(state: SearchState) -> dict:
    # Simulate expensive work.
    time.sleep(2)
    text = re.sub(r"\s+", " ", state["question"]).strip().lower()
    return {"normalized_query": text}

def build_filters(state: SearchState) -> dict:
    text = state["normalized_query"]
    price_match = re.search(r"under (\d+)", text)
    price_cap = int(price_match.group(1)) if price_match else None
    category = "hiking boots" if "boots" in text else "general"
    waterproof = "waterproof" in text

    return {
        "filters": {
```

```

        "category": category,
        "price_cap": price_cap,
        "waterproof": waterproof
    }
}

builder = StateGraph(SearchState)
builder.add_node(
    "normalize_query",
    normalize_query,
    cache_policy = CachePolicy(ttl = 300)
)
builder.add_node(
    "build_filters",
    build_filters,
    cache_policy = CachePolicy(ttl = 300)
)

builder.add_edge(START, "normalize_query")
builder.add_edge("normalize_query", "build_filters")

graph = builder.compile(cache = InMemoryCache())

query = {"question": "Waterproof hiking boots under 150"}
print(graph.invoke(query, stream_mode = "updates"))
print(graph.invoke(query, stream_mode = "updates"))

```

vol3_ch_03\src\node_cache.py

```

(.venv) C:\Users\alexc\11m-app\vol3_ch_03> python -m src.node_cache

[{'normalize_query': {'normalized_query': 'waterproof hiking boots under 150'}},
 {'build_filters': {'filters': {'category': 'hiking boots', 'price_cap': 150,
 'waterproof': True}}}]

[{'normalize_query': {'normalized_query': 'waterproof hiking boots under 150'},
 '__metadata__': {'cached': True}}, {'build_filters': {'filters': {'category': 'hiking
 boots', 'price_cap': 150, 'waterproof': True}}, '__metadata__': {'cached': True}}]

```

On the first call, both nodes execute normally. On the second call, the graph can return the cached node results and expose cache metadata in the streamed updates. This is the kind of work that fits application-level caching: small, repeated, side-effect-free steps whose output is completely determined by their cache key. The example uses an in-memory cache, which is appropriate for local execution and a single process. In a multi-instance deployment, the cache implementation and key strategy must match the deployment scope. Otherwise, each process may maintain different cached results, and cache behavior will vary according to which instance handles the request.

3.5.4 Early exits: stop as soon as the answer is already known

Early exit is the cheapest optimization because it removes work entirely. The main idea is simple: if the workflow already knows enough to answer safely, it should stop instead of proceeding into more tool calls, more retrieval, or a more expensive model. In LangGraph, the native expression of this is routing to END. In LangChain middleware, you can stop execution through early termination logic or short-circuit wrap hooks that decide not to call the handler at all. LangChain's custom middleware documentation is explicit here: `wrap_model_call` and `wrap_tool_call` can call the handler zero times, once, or multiple times. That makes them the right place for "return cached result and skip the model", "the policy blocks this action, stop now", or "the state already proves the answer, so do not pay for another branch." The most effective early exits happen before the expensive model call, not after it. If a request is just a greeting, a known terminal state, an unauthenticated request for a privileged tool, or a query that can be answered deterministically from already available state, the system should

stop immediately. OpenAI's latency guidance says this directly: make fewer requests, and do not default to an LLM when deterministic logic is enough.

Early exit also matters in parallel workflows. When a LangGraph node has multiple outgoing edges, the destination nodes can execute concurrently in the next superstep. Early termination and branch pruning therefore determine how much parallel work the application starts. If a cheap gate can establish that retrieval is unnecessary or that only one branch is relevant, making that decision before fan-out saves tokens, API calls, and execution time. Parallelism primarily reduces wall-clock latency; it does not automatically reduce token or request cost. It can also increase peak concurrency, rate-limit pressure, and total cost when unnecessary or speculative branches are started. Branches should therefore be gated before fan-out, and concurrency should be limited when the number of parallel tasks can grow with the input. A useful way to think about early exits is as a hierarchy. First, run deterministic gates for permissions, known terminal states, cached answers, malformed inputs, and cases that clearly do not require retrieval. Second, if judgment is still required, use the cheapest model that can decide reliably whether more work is necessary. Third, only then fan out into retrieval, tool use, or premium-model synthesis. This ordering prevents the fast path from being absorbed into the heavy path. A compact support-assistant graph shows the pattern:

```
from __future__ import annotations
import re
from typing import Literal
from typing_extensions import TypedDict
from langgraph.graph import START, END, StateGraph
from langgraph.types import Command

ORDERS = {
    "ORD-100": {"status": "delivered"},
    "ORD-101": {"status": "processing"}
}

POLICY_SNIPPETS = {
    "refund": "Refunds are allowed within 30 days for unused items."
}

class SupportState(TypedDict, total = False):
    question: str
    order_id: str
    order: dict
    policy_snippet: str
    final_answer: str

def cheap_gate(
    state: SupportState
) -> Command[Literal["lookup_order", "retrieve_policy", END]]:
    q = state["question"].lower()
    print(f"[gate] question = {state['question']}")

    if q in {"hi", "hello", "thanks"}:
        print("[gate] early exit -> END")
        return Command(
            update = {
                "final_answer": "Hello. I can help with order status or policy questions."
            },
            goto = END
        )

    match = re.search(r"ord-\d{3,}", q, re.I)
    if "where is my order" in q and match:
        order_id = match.group(0).upper()
        print(f"[gate] route -> lookup_order, order_id = {order_id}")
        return Command(update = {"order_id": order_id}, goto = "lookup_order")

    if "refund" in q or "return policy" in q:
        print(f"[gate] route -> retrieve_policy")
        return Command(goto = "retrieve_policy")
```

```

print("[gate] early exit -> END")
return Command(
    update = {"final_answer": "Please provide an order id or ask a policy
               question."},
    goto = END
)

def lookup_order(state: SupportState) -> dict:
    print(f"[lookup_order] order_id={state['order_id']}")
    order = ORDERS.get(state["order_id"])
    if not order:
        print("[lookup_order] not found")
        return {"final_answer": f"Order {state['order_id']} was not found."}

    print(f"[lookup_order] found status={order['status']}")
    return {"order": order}

def route_after_lookup(state: SupportState) -> Literal["answer_from_order", END]:
    route = "answer_from_order" if "order" in state else END
    print(f"[after_lookup] goto={route}")
    return route

def answer_from_order(state: SupportState) -> dict:
    print("[answer_from_order] building final answer")
    return {
        "final_answer": (
            f"Order {state['order_id']} is currently {state['order']['status']}."
        )
    }

def retrieve_policy(state: SupportState) -> dict:
    print("[retrieve_policy] key=refund")
    snippet = POLICY_SNIPPETS.get("refund", "")
    if not snippet:
        print("[retrieve_policy] not found")
        return {"final_answer": "I could not find a refund policy snippet."}

    print("[retrieve_policy] snippet found")
    return {"policy_snippet": snippet}

def route_after_policy(state: SupportState) -> Literal["answer_from_policy", END]:
    route = "answer_from_policy" if "policy_snippet" in state else END
    print(f"[after_policy] goto={route}")
    return route

def answer_from_policy(state: SupportState) -> dict:
    print("[answer_from_policy] building final answer")
    return {"final_answer": state["policy_snippet"]}

builder = StateGraph(SupportState)
builder.add_node("cheap_gate", cheap_gate)
builder.add_node("lookup_order", lookup_order)
builder.add_node("answer_from_order", answer_from_order)
builder.add_node("retrieve_policy", retrieve_policy)
builder.add_node("answer_from_policy", answer_from_policy)

builder.add_edge(START, "cheap_gate")
builder.add_conditional_edges("lookup_order", route_after_lookup)
builder.add_edge("answer_from_order", END)
builder.add_conditional_edges("retrieve_policy", route_after_policy)
builder.add_edge("answer_from_policy", END)

graph = builder.compile()

def run_case(question: str) -> None:
    print("\n" + "=" * 50)
    print(f"QUESTION: {question}")
    result = graph.invoke({"question": question})
    print(f"FINAL: {result['final_answer']}")

run_case("hello")
run_case("Where is my order ORD-100?")
run_case("What is the refund policy?")

```

vol3_ch_03\src\early_exit.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_03> python -m src.early_exit

=====
QUESTION: hello
[gate] question=hello
[gate] early exit -> END

```

```
FINAL: Hello. I can help with order status or policy questions.
```

```
=====
```

```
QUESTION: Where is my order ORD-100?
[gate] question=Where is my order ORD-100?
[gate] route -> lookup_order, order_id=ORD-100
[lookup_order] order_id=ORD-100
[lookup_order] found status=delivered
[after_lookup] goto=answer_from_order
[answer_from_order] building final answer
FINAL: Order ORD-100 is currently delivered.
```

```
=====
```

```
QUESTION: What is the refund policy?
[gate] question=What is the refund policy?
[gate] route -> retrieve_policy
[retrieve_policy] key=refund
[retrieve_policy] snippet found
[after_policy] goto=answer_from_policy
[answer_from_policy] building final answer
FINAL: Refunds are allowed within 30 days for unused items.
```

This graph has three different early-exit paths. Greetings stop immediately. Missing or invalid order lookups stop after the order node instead of continuing into unnecessary answer generation. Policy retrieval also stops if no relevant snippet is available. In a real system, the same pattern would sit in front of the expensive synthesis model and the expensive tool branches. The point is not that the graph is complicated. The point is that the workflow stops as soon as it can.

3.5.5 Putting the three levers together

The three controls fit together in a simple sequence: gate, reuse, promote, and stop. Tiering reduces the cost of necessary model calls, caching avoids repeating stable work, and early exits prevent unnecessary work from starting. Begin with deterministic eligibility, authorization, and terminal-state checks. Use a smaller model only when those checks cannot make the decision reliably. Reuse node results and prompt prefixes only when their stability, scope, cache key, and invalidation rules are explicit. Route the remaining complex or high-risk cases to a stronger model when evaluation shows that its additional capability is justified. Terminate when the state already contains a safe and complete answer, or when the correct outcome is abstention, clarification, or escalation. The result is a controlled workflow in which expensive reasoning is invoked by exception rather than by default.

These controls should be measured as one system rather than assessed through isolated examples. Track route distribution, routing accuracy, downstream quality by route, per-route token cost, model-call count, cache-hit behavior, early-exit rate, and end-to-end latency at both median and tail percentiles. For streamed model paths, also measure time to first token. Provider cache fields should be read from the relevant usage metadata when the integration reports them. A low cache-hit rate may indicate a volatile prefix, requests below the provider's minimum cacheable size, inconsistent keys or cache breakpoints, or requests arriving outside the cache lifetime. A high premium-route rate may reflect genuinely difficult traffic, but it can also indicate a routing policy that promotes too aggressively. Performance tuning should therefore use representative traces and evaluation datasets: identify the dominant cost or latency component, change one control at a time, and verify that quality, safety, and authorization boundaries have not regressed.

3.5.6 Common mistakes

The remaining mistakes are mainly operational. First, a router can reduce average cost while silently sending difficult requests to an inadequate model. Evaluate both the routing decision and the quality produced by the selected route. Second, reusing a cache key after an output-affecting dependency changes can return a result from the wrong configuration, corpus version, tenant, or authorization scope. Third, optimizing only average latency can conceal slow tail behavior caused by retries, rate limits, overloaded tools, or expensive branches. Fourth, reducing input context while leaving output-token limits, model-call budgets, retry counts, and loop termination unconstrained can still produce excessive cost. Fifth, provider caching should not be treated as permanent or free: eligibility requirements, cache lifetimes, retention settings, and cache-read or cache-write pricing vary by provider and API. Control these risks with versioned and security-scoped cache keys, explicit execution budgets, bounded retries and loops, representative evaluations, and joint monitoring of quality, safety, cost, and latency.

Cost and latency control are not separate from planning, routing, caching, and parallelization. They are part of the reason those control-flow patterns exist. The objective is not minimum cost or minimum latency in isolation. It is the least expensive execution path that still meets defined quality, safety, authorization, and latency targets. Tiering, caching, and early exits provide the controls; traces and evaluations show whether those controls are working.

3.6 SUMMARY

This chapter examined planning, routing, and parallelization as distinct but complementary control-flow patterns. Planning structures work before or during execution. Routing selects the path that should handle the current request or subtask. Parallelization executes independent work concurrently and then combines the results through an explicit fan-in contract. The central design principle is to use each pattern only for the problem it actually solves.

Planning is useful when a task is genuinely multi-step and decomposition makes its stages easier to execute, validate, or inspect. Fixed decomposition is appropriate when the sequence is stable; dynamic decomposition is appropriate when the required subtasks depend on runtime input; lightweight planning helps track work that does not justify a complete planner-executor graph; and reactive loops remain preferable when each next action depends on fresh observations. Structured plans and evidence objects make handoffs testable, but planning remains overhead when a direct response or deterministic workflow is sufficient.

Routing depends first on a clear route taxonomy. Rule-based routing is best when authoritative state or stable rules already contain the decision signal. LLM routing handles varied natural language, supervised classifiers provide predictable high-volume classification when labelled data is available, and semantic routing matches requests to route descriptions or examples through embeddings. Hybrid routing is justified only when its stages contribute distinct, measurable value. Regardless of the method, the workflow must support ambiguity, abstention, clarification, and multi-intent cases, and routing must never be treated as authorization.

Parallelization is appropriate only for work without unresolved dependencies. Static branches handle known destinations, while Send supports runtime-created workers with item-specific state. Reducers define how concurrent state updates are combined; aggregation nodes interpret those results through calculation, selection, validation, synthesis, or conflict resolution. Map-reduce processes independent items, voting aggregates normalized choices, peer-evaluated selection ranks open-ended candidates, structured composition assembles specialist outputs, and consensus adds iterative critique and revision when an explicitly accepted shared position is required.

When these patterns are integrated with RAG and tools, the workflow should route only after it has enough trustworthy information, run independent evidence-gathering operations concurrently, keep dependent work sequential, and synchronize all required branches before synthesis or action. Read operations are generally easier to parallelize than external side effects. Write tools require authorization, validation, idempotency, approval, auditability, and explicit recovery policies because graph retries cannot reverse changes already made in external systems.

The final optimization problem is to achieve the required quality and safety with the least expensive execution path. Tiered models reserve stronger reasoning for the cases that justify

it. Workflow and provider caches reuse stable work when keys, scope, and invalidation are correct. Early exits prevent unnecessary branches, retrieval, tool calls, and model calls from starting. These controls must be evaluated together through route quality, task success, latency, token and model usage, cache behavior, failure rates, and authorization outcomes. With execution structure established, the next chapter turns to how agentic systems evaluate, revise, and adapt their own results.

4 REFLECTION, REASONING, AND ADAPTATION

An intelligent system must do more than produce a plausible first answer. It must recognize when that answer is incomplete, unsupported, unsafe, or based on the wrong assumptions. It must decide whether to revise the answer, gather better evidence, explore another solution, ask for clarification, request human approval, or stop without acting. This ability to evaluate results and change direction is what turns a sequence of model calls into a controlled adaptive system. This chapter examines the mechanisms that make such control possible. It begins with reflection and verification. Reflection uses feedback to reconsider or improve a candidate result. Verification asks a stricter question: does the result satisfy the evidence, rules, constraints, and authority required for the workflow to continue? The distinction matters. Asking a model to review its own answer may improve clarity or completeness, but it does not create an independent source of truth. Stronger assurance may require deterministic validation, trusted documents, system-of-record data, test execution, authorization checks, or approval from an authorized person.

Some problems cannot be solved reliably by revising one answer. The system may need to generate several alternatives and compare them. You will therefore examine search-based reasoning patterns such as Best-of-N, self-consistency, Tree-of-Thought search, and MCTS-style control. These patterns make candidate generation, evaluation, pruning, and stopping part of the application workflow. They can improve difficult planning and reasoning tasks, but only when the system has a credible way to distinguish stronger candidates from weaker ones. Without reliable evaluation, additional search produces more cost and complexity rather than better decisions. The chapter then moves from improvement within one execution to improvement across many executions. Explicit signals such as ratings, reason labels, edits, and reviewer judgments can reveal recurring failures. Implicit signals such as clicks, purchases, abandoned carts, completed trades, profit, and drawdown can show how earlier decisions performed in the real environment. Neither type of signal changes the application automatically. They become useful only when they are captured with the relevant context, interpreted carefully, reviewed where necessary, and converted into controlled changes such as evaluation cases, prompt revisions, routing rules, retrieval changes, or separately governed training data. You will also learn how an application can adapt while it is running. Runtime context can supply trusted invocation-specific information. Model context can change instructions, models, tools, or response schemas. Retrieval policy can alter how evidence is searched, filtered, diversified, and ranked. LangGraph control flow can branch, retry, pause, resume, escalate, or recover from checkpoints. These control surfaces allow a support assistant to apply the correct regional policy, an e-commerce assistant to use a more appropriate retrieval strategy, or a market-analysis assistant to escalate uncertain or high-risk analysis without rebuilding the entire application.

Adaptation also creates new risks. A system can learn to satisfy an imperfect evaluator instead of solving the real task. Its data, tools, policies, or users may change. Repeated tuning may improve a familiar evaluation set while making little difference in production. The final part of the chapter therefore examines reward hacking, drift, and overfitting to evaluation sets. The central principle throughout is that improvement must remain bounded, observable, and reversible. Reliable intelligent systems do not adapt simply because they can. They adapt only when the triggering signal is credible, the permitted change is explicit, and the resulting behaviour can be evaluated against the environment the system is meant to serve.

4.1 REFLECTION AND VERIFICATION LOOPS: SELF-CRITIQUE, VERIFIER PATTERNS, AND FAIL-CLOSED DESIGN

A first-pass model answer is often useful, but in a non-trivial system it should be treated as provisional. Important failures often concern evidence, completeness, constraints, policy compliance, numerical consistency, or action safety rather than fluency. A draft may sound persuasive while omitting an exception, overstating what was retrieved, applying the wrong policy, or proposing an action the workflow is not authorized to perform. This section treats reflection and verification as related but distinct controls. Reflection uses feedback to reconsider or revise a candidate. Verification determines whether a candidate, intermediate artifact, or proposed action satisfies explicit acceptance criteria. Deterministic validation, test execution, authorization checks, and human approval may participate in the same control loop, but they are not themselves forms of model self-reflection. In either case, one step produces a candidate or intermediate result, another evaluates it, and explicit routing decides whether to accept it, revise it, request missing information, abstain, escalate, or terminate safely. Evaluation is useful only when it adds a clearer rubric, authoritative evidence, deterministic rules, execution results, system-of-record data, or human authority. A second model call may improve a draft, but it is not automatically an independent check. Reliable workflows therefore require explicit state, typed decisions, acceptance criteria, attempt limits, and defined exit conditions.

4.1.1 Reflection as a controlled workflow

A reflection workflow has three basic stages: generate, evaluate, and route. First, the system creates a possible result. Next, it checks that result against clear rules. Finally, it decides what to do next. The system may accept the result, improve it using specific feedback, ask for missing information, refuse to provide an answer, send the task to a person or another system, or stop the process. Revising the result is only one possible outcome. The evaluation rules may check whether the result is supported by facts, complete, compliant with policies, numerically correct, in the required format, written in the correct tone, or safe to use with tools. This separates two different responsibilities that are often handled in a single model call: creating a result and deciding whether that result is acceptable. The goal defines what an acceptable result is. Monitoring collects visible signs of possible problems, such as unsupported statements, invalid data structures, failed tests, unexpected tool results, or missing information. Evaluation uses these signs to produce a clear decision that controls what the system does next. A workflow that evaluates and revises a result only once can remain a simple, step-by-step pipeline. However, the workflow may need to repeat steps, remember previous attempts, request approval, or recover from failures. In these cases, LangGraph can make the attempts, decisions, feedback, interruptions, acceptance rules, and stopping conditions clear, instead of hiding them inside the prompt.

4.1.2 Self-critique: useful, but limited

The simplest reflection pattern is self-critique. The same model creates a draft, checks it against clear rules, and then improves it. This is the basic approach used by Self-Refine: create an

answer, create feedback about that answer, and revise the answer using the feedback. Self-critique works best when the result can be easily improved and the evaluation rules are clear. Typical examples include improving clarity, formatting, tone, summary quality, extracted information, and the completeness of an explanation. In these cases, the model is not expected to find a separate source of truth. It is only improving an existing draft so that it better follows the stated requirements. This is an important limitation. Self-critique can improve an answer, but it cannot prove that the answer is correct. A model that creates an unsupported statement may also accept that statement when reviewing its own work, because both model calls may have the same knowledge gaps and weaknesses. Self-critique can improve how an answer is written and make it more complete. However, factual claims or policy-sensitive decisions require stronger checks. These checks may include trusted evidence, fixed rules, tool execution results, tests, or another reliable verification method.

Structured self-critique makes this pattern easier to use in a real workflow. Instead of returning a general paragraph of feedback, the evaluator returns a clearly defined decision and a specific instruction for revision. The decision controls what happens next in the workflow. The revision instruction is passed to the next step and used to improve the draft. The evaluator can also return a failure category so that the problem can be recorded, traced, and measured. For example, decisions such as pass and revise are more useful than a vague comment such as “this could be better.” LangChain supports structured output at both the model level and the agent level. At the model level, `with_structured_output` uses the structured-output features supported by the selected model integration. At the agent level, structured output may use schema enforcement provided directly by the model provider, when supported. It may also use tool-based generation when tool calling is required. Provider-based schema enforcement is usually the strongest option when it is available. However, tool-based structured output is also a valid approach. In both cases, the workflow must explicitly handle schema errors, validation errors, and parsing errors. It can do this by retrying, using a fallback, returning the error, or moving to a safe failure state. This distinction reflects the current difference between model-level structured output and the agent-level `ProviderStrategy` and `ToolStrategy`.

The following example performs one critique-and-revision pass on a small and clearly limited task: improving the clarity and completeness of a short release note. The critic returns a structured decision and a specific revision instruction instead of a vague paragraph. The example shows the value of structured feedback without suggesting that self-critique is enough to verify facts.

```
from __future__ import annotations
from typing import Literal
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
from .config import OPENAI_API_KEY

class Critique(BaseModel):
    verdict: Literal["pass", "revise"]
    failure_category: Literal["clarity", "completeness", "format", "tone"]
    revision_instruction: str = Field(
        description="Concrete instruction for improving the draft."
    )

class State(TypedDict):
```

```

request: str
draft: str
critique: str
revised: str
verdict: Literal["", "pass", "revise"]

model = init_chat_model("openai:gpt-4.1-mini", temperature = 0)
critic = init_chat_model("openai:gpt-4.1-mini", temperature = 0).with_structured_output(
    Critique
)

def write_draft(state: State) -> dict:
    msg = model.invoke(f"""
Write a short release note in 3 bullet points.

Request:
{state["request"]}
""")
    return {"draft": msg.content}

def critique_draft(state: State) -> dict:
    result = critic.invoke(f"""
Review the release note below.

Requirements:
- exactly 3 bullet points
- each bullet must be concise
- mention the user-visible improvement
- avoid marketing language

Draft:
{state["draft"]}
""")
    return {
        "verdict": result.verdict,
        "critique": result.revision_instruction
    }

def revise_draft(state: State) -> dict:
    if state["verdict"] == "pass":
        return {"revised": state["draft"]}
    msg = model.invoke(f"""
Revise the release note using this instruction:

{state["critique"]}

Current draft:
{state["draft"]}
""")
    return {"revised": msg.content}

builder = StateGraph(State)
builder.add_node("write draft", write_draft)
builder.add_node("critique_draft", critique_draft)
builder.add_node("revise_draft", revise_draft)

builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "critique_draft")
builder.add_edge("critique_draft", "revise_draft")
builder.add_edge("revise_draft", END)

graph = builder.compile()

if __name__ == "__main__":
    result = graph.invoke({
        "request": ("The dashboard now loads faster, users can filter incidents by severity, "
                    "and users can update multiple incidents with one bulk action."),
        "draft": "",
        "critique": "",
        "revised": "",
        "verdict": ""
    })
    print("FIRST DRAFT:\n")
    print(result["draft"])
    print("\nCRITIQUE:\n")
    print(result["critique"])
    print("\nFINAL VERSION:\n")
    print(result["revised"])

```

vol3_ch_04\scr\self_critique.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.self_critique

FIRST DRAFT:
- Improved dashboard performance for faster loading times.
- Added the ability to filter incidents by severity.
- Enhanced incident management with a more efficient workflow.

CRITIQUE:
Rewrite the third bullet to name the bulk-action capability explicitly instead of
describing the improvement with vague terms such as "enhanced" or "more efficient."

FINAL VERSION:
- Improved dashboard performance for faster loading times.
- Added the ability to filter incidents by severity.
- Added bulk actions so users can update multiple incidents at once.
```

This single-pass reflection improves specificity and presentation, but it does not establish that the listed features exist. In this example, the original request is the factual source, and the critic only checks how clearly those supplied facts are expressed. Adding a capability that was not present in the request would be fabrication rather than improvement. The benefit comes from narrowing the task and structuring the feedback, not from creating a new source of truth.

4.1.3 Verifier patterns and the assurance hierarchy

Verifier patterns are needed when self-critique is not strong enough to decide whether a result should be accepted. The main design idea is to separate two responsibilities. One part of the system produces an answer, extracts information, creates a plan, or suggests an action. Another part decides whether the workflow is allowed to continue. The best type of verifier depends on what must be checked. Deterministic code should be used for exact checks. Examples include validating a schema, confirming that required fields are present, checking identifier formats, verifying numeric ranges, enforcing authorization rules, limiting which tools may be used, checking citation structure, and confirming that a state change is valid.

A model-based verifier that checks the result against evidence is useful when the decision requires interpretation rather than an exact rule. Human or domain-expert review is more appropriate when the consequences are serious, the evidence is unclear, or the decision requires authority that the model does not have. When a model-based verifier must apply business rules, compliance requirements, safety constraints, or brand standards, those rules should be written as clear and specific criteria. They should not be hidden inside vague instructions such as “be safe” or “be compliant.” Clear criteria are easier to test, apply consistently, and connect to workflow decisions. Using a second model call may reduce the chance that the verifier simply repeats the reasoning of the first model call. However, using a separate call does not automatically make the verification objective. Verification becomes stronger only when the verifier uses something the original producer did not control. This may include external evidence, fixed rules, tool execution results, data from a system of record, or approval from an authorized person. For this reason, the following example uses a model to perform an uncertain extraction task but uses deterministic code to make the final acceptance decision.

```
from __future__ import annotations
import re
from typing import Literal
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
```

```

from .config import OPENAI_API_KEY

SOURCE_TEXT_VALID = """
Order ID: ORD-1042
Requested refund window: 14 days
"""

SOURCE_TEXT_INVALID = """
Order ID: ORDER-77
Requested refund window: 45 days
"""

class Extraction(BaseModel):
    order_id: str
    refund_days: int = Field(ge = 0)

class State(TypedDict):
    source_text: str
    extracted_order_id: str
    extracted_refund_days: int
    validation_error: str
    final_status: Literal["", "accepted", "rejected"]

model = init_chat_model("openai:gpt-4.1-mini", temperature = 0).with_structured_output(
    Extraction
)

def extract_fields(state: State) -> dict:
    result = model.invoke(f"""
Extract the order ID and refund_days from this text:
{state["source_text"]}
""")
    return {
        "extracted_order_id": result.order_id,
        "extracted_refund_days": result.refund_days
    }

def validate_fields(state: State) -> dict:
    errors = []

    if not re.fullmatch(r"ORD-\d{4}", state["extracted_order_id"]):
        errors.append("order_id must match ORD-####")

    if not (0 <= state["extracted_refund_days"] <= 30):
        errors.append("refund_days must be between 0 and 30")

    if errors:
        return {
            "validation_error": "; ".join(errors),
            "final_status": "rejected"
        }

    return {
        "validation_error": "",
        "final_status": "accepted"
    }

builder = StateGraph(State)
builder.add_node("extract_fields", extract_fields)
builder.add_node("validate_fields", validate_fields)

builder.add_edge(START, "extract_fields")
builder.add_edge("extract_fields", "validate_fields")
builder.add_edge("validate_fields", END)

graph = builder.compile()

def run_case(source_text: str) -> dict:
    return graph.invoke({
        "source_text": source_text,
        "extracted_order_id": "",
        "extracted_refund_days": 0,
        "validation_error": "",
        "final_status": ""
    })

if __name__ == "__main__":
    valid_result = run_case(SOURCE_TEXT_VALID)
    print("VALID EXAMPLE")
    print("ORDER ID:", valid_result["extracted_order_id"])
    print("REFUND DAYS:", valid_result["extracted_refund_days"])
    print("STATUS:", valid_result["final_status"])
    print("VALIDATION ERROR:", valid_result["validation_error"])

```

```
print()

invalid_result = run_case(SOURCE_TEXT_INVALID)
print("INVALID EXAMPLE")
print("ORDER ID:", invalid_result["extracted_order_id"])
print("REFUND DAYS:", invalid_result["extracted_refund_days"])
print("STATUS:", invalid_result["final_status"])
print("VALIDATION ERROR:", invalid_result["validation_error"])
```

vol3_ch_04\scr\deterministic_verifier.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.deterministic_verifier

VALID EXAMPLE
ORDER ID: ORD-1042
REFUND DAYS: 14
STATUS: accepted
VALIDATION ERROR:

INVALID EXAMPLE
ORDER ID: ORDER-77
REFUND DAYS: 45
STATUS: rejected
VALIDATION ERROR: order_id must match ORD-####; refund_days must be between 0 and 30
```

The model performs the probabilistic task of interpreting the source text, while deterministic code owns the exact acceptance decision. This division is stronger than asking another model whether the identifier and numeric range “look correct.” When a requirement can be encoded as a rule, the application should enforce that rule directly.

4.1.4 Claim-level verification

LLM-based verifiers are useful when a decision is too complex to express as an exact software rule. However, they are still probabilistic evaluators. They are not independent sources of truth. A verifier may have the same weaknesses as the model that produced the answer. It may accept reasoning that sounds convincing but is not supported by evidence. Its decision may also be affected by the wording of the prompt, the length of the response, the order in which information is presented, or untrusted text included in its context. For this reason, the verifier must be tested on realistic examples. Its evaluation rules, prompts, decision thresholds, and level of agreement with human or domain-expert judgments must all be measured. Claim decomposition makes the item being checked smaller. Instead of asking a verifier to approve an entire answer, the workflow extracts the important factual claims from that answer. It then checks each claim against the available evidence. Only claims that are supported by evidence are used to create the final answer.

A small, specific claim is usually easier to check and connect to evidence than a complete answer containing several facts, qualifications, and persuasive explanations. However, this approach still depends on two important factors: whether the workflow extracts the claims correctly and whether the available evidence is complete and independent. This design is inspired by Chain-of-Verification, but it is not a complete implementation of that method. A complete Chain-of-Verification workflow first creates an initial answer. It then creates verification questions, answers those questions through a separate process, and finally produces a verified answer. The following example uses a narrower workflow. It creates a draft, extracts individual claims, verifies each claim through a separate path, and then creates a new answer using only the allowed claims. These steps are separated in the workflow. However, they should not be described as fully independent when they use related models, similar prompts, or the same evidence. The purpose of the example is to show the structure

and control flow of claim-level verification. It is not intended to represent a complete production retrieval system.

```

from __future__ import annotations
from typing import Literal
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END

DOCS = [
    "Orders over 50 euros ship free.",
    "Digital gift cards are delivered by email and have no delivery charge.",
    "Digital gift cards are non-refundable.",
    "Physical gift items may be returned within 30 days for store credit."
]

class ClaimList(BaseModel):
    claims: list[str] = Field(default_factory = list)

class ClaimCheck(BaseModel):
    claim: str
    verdict: Literal["supported", "unsupported"]
    evidence: str

class State(TypedDict):
    question: str
    draft: str
    claims: list[str]
    checks: list[dict]
    final_answer: str

writer = init_chat_model("openai:gpt-4.1-mini", temperature = 0)
claim_extractor = init_chat_model("openai:gpt-4.1-mini", temperature =
0).with_structured_output(ClaimList)
verifier = init_chat_model("openai:gpt-4.1-mini", temperature =
0).with_structured_output(ClaimCheck)

def write_draft(state: State) -> dict:
    msg = writer.invoke(f"""
Answer this customer question using the policy notes below.

Policy notes:
{chr(10).join("- " + d for d in DOCS)}

Question:
{state["question"]}
""")
    return {"draft": msg.content}

def extract_claims(state: State) -> dict:
    result = claim_extractor.invoke(f"""
Extract the material factual claims from this answer.

Answer:
{state["draft"]}
""")
    return {"claims": result.claims}

def verify_claims(state: State) -> dict:
    checks = []
    docs_block = "\n".join(DOCS)
    for claim in state["claims"]:
        result = verifier.invoke(f"""
Check whether this claim is supported by the documents.

Claim:
{claim}

Documents:
{docs_block}
""")
        checks.append(result.model_dump())
    return {"checks": checks}

def synthesize_verified_answer(state: State) -> dict:
    supported = [c for c in state["checks"] if c["verdict"] == "supported"]
    unsupported = [c for c in state["checks"] if c["verdict"] == "unsupported"]

    parts = []
    if supported:
        parts.append("Supported points:")

```

```

        for item in supported:
            parts.append(f"- {item['claim']}")

    if unsupported:
        parts.append("\nUnsupported or not established:")
        for item in unsupported:
            parts.append(f"- {item['claim']}")

    return {"final_answer": "\n".join(parts)}

builder = StateGraph(State)
builder.add_node("write_draft", write_draft)
builder.add_node("extract_claims", extract_claims)
builder.add_node("verify_claims", verify_claims)
builder.add_node("synthesize_verified_answer", synthesize_verified_answer)

builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "extract_claims")
builder.add_edge("extract_claims", "verify_claims")
builder.add_edge("verify_claims", "synthesize_verified_answer")
builder.add_edge("synthesize_verified_answer", END)

graph = builder.compile()

if __name__ == "__main__":
    result = graph.invoke({
        "question": "How are digital gift cards delivered, and can they be refunded?",
        "draft": "",
        "claims": [],
        "checks": [],
        "final_answer": ""
    })

    print("DRAFT:\n")
    print(result["draft"])
    print("\nCLAIMS:\n")
    for claim in result["claims"]:
        print("-", claim)
    print("\nVERIFIED ANSWER:\n")
    print(result["final_answer"])

```

vol3_ch_04\src\claim_decomposition.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.claim_decomposition

DRAFT:
Digital gift cards are delivered by email with no delivery charge. They are non-refundable.

CLAIMS:
- Digital gift cards are delivered by email.
- Digital gift cards have no delivery charge.
- Digital gift cards are non-refundable.

VERIFIED ANSWER:
Supported points:
- Digital gift cards are delivered by email.
- Digital gift cards have no delivery charge.
- Digital gift cards are non-refundable.

```

The gain here is not merely an extra model call. It is a change in topology. The checker evaluates smaller factual units on a separate path rather than treating the answer as one rhetorical unit. This often makes overclaiming easier to detect and makes the final synthesis easier to constrain to what was actually established. It does not remove the need to evaluate claim extraction, evidence coverage, and verifier accuracy.

4.1.5 Process-level verification

Process-level verification checks important stages of the application before their results are allowed to move to the next stage. These stages may include retrieval, data extraction, planning, or preparing a tool call. This idea is related to process supervision, but it is not the same thing. Process supervision usually means training or scoring the intermediate reasoning steps of a model. The workflow described here checks visible runtime results and the

transitions between application stages. A single verifier at the end of the workflow may not be enough, because by that point the error may already have affected the final result. Checks are often most useful at important boundaries where a mistake would become more difficult or expensive to fix. Examples include after retrieval, after structured data extraction, before generating the final answer, and before an irreversible tool call. Once weak evidence or an invalid intermediate result moves into a later stage, it may already influence the assumptions and content of the final response. For this reason, the following example checks the retrieved evidence before generation begins. If the evidence is not good enough, the workflow sends the request to a stricter retrieval step.

```
from __future__ import annotations
from typing import Literal
from pydantic import BaseModel, Field
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model
from langgraph.graph import END, START, StateGraph
from .config import OPENAI_API_KEY

DOCS = [
    "Battery replacement is available as a paid repair.",
    "Standard laptop warranty lasts 2 years.",
    "Battery defects are covered for 1 year under the limited warranty.",
    "Screen defects are covered for 2 years under the limited warranty."
]

QUESTION = "What is the battery warranty?"

class QuestionAnalysis(BaseModel):
    required_terms: list[str] = Field(
        description = (
            "Short list of lower-case terms that must appear in supporting evidence "
            "for the question to be answerable."
        )
    )
    answer_focus: str = Field(
        description = "Very short description of the exact relation the evidence must support."
    )

class RetrievalAssessment(BaseModel):
    status: Literal["good", "bad"]
    reason: str
    supporting_doc_numbers: list[int] = Field(
        default_factory = list,
        description = (
            "1-based document numbers that directly support the answer. Return an empty "
            "list when the evidence is not good enough."
        )
    )

class State(TypedDict):
    question: str
    required_terms: list[str]
    answer_focus: str
    loose_docs: list[str]
    repaired_docs: list[str]
    retrieved_docs: list[str]
    first_validation_status: Literal["", "good", "bad"]
    first_validation_reason: str
    second_validation_status: Literal["", "good", "bad"]
    second_validation_reason: str
    retrieval_status: Literal["", "good", "bad"]
    validation_reason: str
    route_history: list[str]
    answer: str

base_model = init_chat_model("openai:gpt-4.1-mini", temperature = 0)
question_model = base_model.with_structured_output(QuestionAnalysis)
verification_model = base_model.with_structured_output(RetrievalAssessment)

def normalize(text: str) -> list[str]:
    return [token.strip(".,?!:;").lower() for token in text.split()]

def format_docs(docs: list[str]) -> str:
```

```

if not docs:
    return "(no documents)"
return "\n".join(f"{i}. {doc}" for i, doc in enumerate(docs, start = 1))

def analyze_question(state: State) -> dict:
    result = question_model.invoke(
        [
            {
                "role": "system",
                "content": (
                    "Extract the minimum evidence requirements needed to answer a "
                    "support question. Return only a short list of lower-case required "
                    "terms and a brief answer_focus."
                )
            },
            {
                "role": "user",
                "content": f"Question: {state['question']}"
            }
        ]
    )
    required_terms = [term.strip().lower() for term in result.required_terms if
                      term.strip()]
    return {
        "required_terms": required_terms,
        "answer_focus": result.answer_focus.strip(),
        "route_history": ["analyze_question"]
    }

def retrieve_loose(state: State) -> dict:
    question_terms = set(normalize(state["question"])) | set(state["required_terms"])
    loose_docs: list[str] = []

    # Intentionally broad first pass: keep the first overlapping documents in corpus
    # order.
    # This simulates a high-recall but low-precision retriever that may gather related
    # text
    # without proving the exact relation the question asks about.
    for doc in DOCS:
        if question_terms & set(normalize(doc)):
            loose_docs.append(doc)
            if len(loose_docs) == 2:
                break

    return {
        "loose_docs": loose_docs,
        "retrieved_docs": loose_docs,
        "route_history": state["route_history"] + ["retrieve_loose"]
    }

def verify_retrieval(state: State) -> dict:
    docs_text = format_docs(state["retrieved_docs"])
    result = verification_model.invoke(
        [
            {
                "role": "system",
                "content": (
                    "You are validating an intermediate retrieval result before the "
                    "workflow is allowed to generate an answer. Mark the retrieval as "
                    "good only when at least one retrieved document directly supports "
                    "the answer_focus for the question. Do not accept evidence that "
                    "only mentions related words across separate documents."
                )
            },
            {
                "role": "user",
                "content": (
                    f"Question: {state['question']}\n"
                    f"Required terms: {' '.join(state['required_terms'])}\n"
                    f"Answer focus: {state['answer_focus']}\n"
                    f"Retrieved documents:\n{docs_text}"
                )
            }
        ]
    )
    return {
        "first_validation_status": result.status,
        "first_validation_reason": result.reason.strip(),
        "retrieval_status": result.status,
        "validation_reason": result.reason.strip(),
    }

```

```

        "route_history": state["route_history"] + [
            "validation_passed" if result.status == "good" else "validation_failed"
        ]
    }

def route_after_validation(state: State) -> Literal["repair", "synthesize"]:
    return "synthesize" if state["retrieval_status"] == "good" else "repair"

def retrieve_strict(state: State) -> dict:
    required = set(state["required_terms"])
    repaired_docs = [doc for doc in DOCS if required.issubset(set(normalize(doc)))]
    return {
        "repaired_docs": repaired_docs,
        "retrieved_docs": repaired_docs,
        "route_history": state["route_history"] + ["retrieve_strict"]
    }

def reverify_retrieval(state: State) -> dict:
    docs_text = format_docs(state["retrieved_docs"])
    result = verification_model.invoke(
        [
            {
                "role": "system",
                "content": (
                    "You are re-checking repaired evidence. Mark it good only if the"
                    " current documents now directly support the answer_focus."
                )
            },
            {
                "role": "user",
                "content": (
                    f"Question: {state['question']}\n"
                    f"Required terms: {' '.join(state['required_terms'])}\n"
                    f"Answer focus: {state['answer_focus']}\n\n"
                    f"Retrieved documents:\n{docs_text}"
                )
            }
        ]
    )

    return {
        "second_validation_status": result.status,
        "second_validation_reason": result.reason.strip(),
        "retrieval_status": result.status,
        "validation_reason": result.reason.strip(),
        "route_history": state["route_history"]
        + ["repair_succeeded" if result.status == "good" else "repair_failed"]
    }

def synthesize_answer(state: State) -> dict:
    if state["retrieval_status"] != "good" or not state["retrieved_docs"]:
        return {
            "answer": "I do not have enough verified evidence to answer safely.",
            "route_history": state["route_history"] + ["abstain"]
        }

    docs_text = format_docs(state["retrieved_docs"])
    response = base_model.invoke(
        [
            {
                "role": "system",
                "content": (
                    "Answer the user using only the verified evidence. Keep the answer"
                    " short. Do not add facts that are not in the retrieved documents."
                )
            },
            {
                "role": "user",
                "content": (
                    f"Question: {state['question']}\n\n"
                    f"Verified evidence:\n{docs_text}"
                )
            }
        ]
    )

    return {
        "answer": response.content.strip(),
        "route_history": state["route_history"] + ["synthesize_answer"]
    }

builder = StateGraph(State)

```

```

builder.add_node("analyze_question", analyze_question)
builder.add_node("retrieve_loose", retrieve_loose)
builder.add_node("verify_retrieval", verify_retrieval)
builder.add_node("retrieve_strict", retrieve_strict)
builder.add_node("reverify_retrieval", reverify_retrieval)
builder.add_node("synthesize_answer", synthesize_answer)

builder.add_edge(START, "analyze_question")
builder.add_edge("analyze_question", "retrieve_loose")
builder.add_edge("retrieve_loose", "verify_retrieval")
builder.add_conditional_edges(
    "verify_retrieval",
    route_after_validation,
    {
        "repair": "retrieve_strict",
        "synthesize": "synthesize_answer"
    }
)
builder.add_edge("retrieve_strict", "reverify_retrieval")
builder.add_edge("reverify_retrieval", "synthesize_answer")
builder.add_edge("synthesize_answer", END)

graph = builder.compile()

DEFAULT_STATE: State = {
    "question": QUESTION,
    "required_terms": [],
    "answer_focus": "",
    "loose_docs": [],
    "repaired_docs": [],
    "retrieved_docs": [],
    "first_validation_status": "",
    "first_validation_reason": "",
    "second_validation_status": "",
    "second_validation_reason": "",
    "retrieval_status": "",
    "validation_reason": "",
    "route_history": [],
    "answer": ""
}

def run_case(question: str) -> None:
    state = dict(DEFAULT_STATE)
    state["question"] = question
    result = graph.invoke(state)

    print("QUESTION:")
    print(result["question"])

    print("\nQUESTION ANALYSIS:")
    print("REQUIRED TERMS:", ", ", ".join(result["required_terms"]))
    print("ANSWER FOCUS:", result["answer_focus"])

    print("\nLOOSE RETRIEVAL:")
    for doc in result["loose_docs"]:
        print(f"- {doc}")

    print("\nFIRST INTERMEDIATE VALIDATION:")
    print("STATUS:", result["first_validation_status"])
    print("REASON:", result["first_validation_reason"])

    if result["repaired_docs"]:
        print("\nSTRICT RETRIEVAL:")
        for doc in result["repaired_docs"]:
            print(f"- {doc}")

        print("\nSECOND INTERMEDIATE VALIDATION:")
        print("STATUS:", result["second_validation_status"])
        print("REASON:", result["second_validation_reason"])

    print("\nROUTE HISTORY:")
    print(" -> ".join(result["route_history"]))

    print("\nFINAL VERIFIED EVIDENCE:")
    for doc in result["retrieved_docs"]:
        print(f"- {doc}")

    print("\nFINAL ANSWER:")
    print(result["answer"])

if __name__ == "__main__":
    run_case(QUESTION)

```

vol3_ch_04\scr\intermediate_verification.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.intermediate_verification

QUESTION:
What is the battery warranty?

QUESTION ANALYSIS:
REQUIRED TERMS: battery, warranty
ANSWER FOCUS: the duration and coverage details of the battery warranty

LOOSE RETRIEVAL:
- Battery replacement is available as a paid repair.
- Standard laptop warranty lasts 2 years.

FIRST INTERMEDIATE VALIDATION:
STATUS: bad
REASON: Neither document provides information about the duration or coverage details of
the battery warranty specifically. Document 1 only mentions battery replacement as a paid
repair, and Document 2 mentions a standard laptop warranty duration but does not specify
battery warranty details.

STRICT RETRIEVAL:
- Battery defects are covered for 1 year under the limited warranty.

SECOND INTERMEDIATE VALIDATION:
STATUS: good
REASON: Document 1 directly states the duration and coverage of the battery warranty,
specifying that battery defects are covered for 1 year under the limited warranty.

ROUTE HISTORY:
analyze_question -> retrieve_loose -> validation_failed -> retrieve_strict ->
repair_succeeded -> synthesize_answer

FINAL VERIFIED EVIDENCE:
- Battery defects are covered for 1 year under the limited warranty.

FINAL ANSWER:
The battery warranty covers battery defects for 1 year under the limited warranty.
```

This is runtime process verification in a simple form. The workflow checks whether retrieval has produced evidence that directly supports the required answer relation. When that condition fails, the graph repairs retrieval before invoking the answer generator. Preventing weak evidence from entering synthesis is usually more reliable than attempting to detect all of its consequences in the finished response. Intermediate verification prevents weak artifacts from propagating, but it also creates corrective branches that must stop safely. The next design problem is therefore what the workflow should do when evidence remains inadequate, required inputs are missing, or repeated revision still fails.

4.1.6 Bounded loops and fail-closed outcomes

A reflection loop becomes unsafe when the system treats uncertainty as permission to continue. A fail-closed design does the opposite. When the workflow cannot confirm that an answer or action meets its acceptance rules, it must move to a predefined safe state. Different problems require different responses. An answer that can be improved may return to the generation step with clear feedback. When required information is missing, the workflow should ask the user for that information. When the available evidence is not sufficient, the workflow should not simply rewrite the answer. Rewriting cannot create facts that are missing from the evidence. Instead, the workflow may try to obtain better evidence through a limited and controlled step. For example, it may correct the retrieval query, use another authorized data source, request a missing document, or look up information in a system of record. When no additional evidence can be obtained, or when the allowed number of evidence searches has

been reached, the workflow must refuse to answer or send the case for review. For this reason, every repeating workflow needs clearly defined decisions, explicit acceptance rules, a maximum number of attempts, and a required final or recovery outcome. “Retry until the result looks reasonable” is not a valid control. Structured output makes workflow decisions readable by software. Graph routing, middleware limits, guardrails, and human approval then control what the workflow is allowed to do next. Clarification is one of the most common safe outcomes in an interactive assistant. When a required field is missing, the correct response is often not to refuse the request or perform the action. It is to ask a specific question to obtain the missing information. The following example shows a fail-closed precondition check rather than a generate, critique, and revise loop. The refund workflow cannot continue without an order ID. Therefore, deterministic routing sends the request to a clarification step instead of guessing the order ID or suggesting an action.

```
from __future__ import annotations
from typing import Literal, Optional
from typing_extensions import TypedDict
from pydantic import BaseModel
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END

class ParsedRequest(BaseModel):
    wants_refund: bool
    order_id: Optional[str] = None
    reason: Optional[str] = None

class State(TypedDict):
    user_message: str
    wants_refund: bool
    order_id: str | None
    reason: str | None
    next_step: Literal["", "clarify", "proceed"]
    response: str

parser = init_chat_model("openai:gpt-4.1-mini", temperature = 0).with_structured_output(
    ParsedRequest
)

def parse_request(state: State) -> dict:
    result = parser.invoke(f"""
    Parse the user's request.

    User message:
    {state["user_message"]}
    """)
    return {
        "wants_refund": result.wants_refund,
        "order_id": result.order_id,
        "reason": result.reason
    }

def verify_inputs(state: State) -> dict:
    if state["wants_refund"] and not state["order_id"]:
        return {"next_step": "clarify"}
    return {"next_step": "proceed"}

def ask_clarification(state: State) -> dict:
    return {
        "response": "I can help with that, but I need your order ID before I can continue."
    }

def proceed(state: State) -> dict:
    return {
        "response": f"Refund request can be reviewed for order {state['order_id']}."
    }

def route_after_verification(state: State) -> str:
    return state["next_step"]

builder = StateGraph(State)
builder.add_node("parse_request", parse_request)
builder.add_node("verify_inputs", verify_inputs)
builder.add_node("ask_clarification", ask_clarification)
```

```

builder.add_node("proceed", proceed)
builder.add_edge(START, "parse_request")
builder.add_edge("parse_request", "verify_inputs")
builder.add_conditional_edges(
    "verify_inputs",
    route_after_verification,
    {
        "clarify": "ask_clarification",
        "proceed": "proceed"
    }
)
builder.add_edge("ask_clarification", END)
builder.add_edge("proceed", END)

graph = builder.compile()

if __name__ == "__main__":
    result = graph.invoke({
        "user_message": "I want a refund for the shoes I bought last week.",
        "wants_refund": False,
        "order_id": None,
        "reason": None,
        "next_step": "",
        "response": ""
    })

    print(result["response"])

```

vol3_ch_04\src\fail_closed.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.fail_closed
I can help with that, but I need your order ID before I can continue.

```

This example demonstrates validation and fail-closed routing rather than model reflection. The model performs the probabilistic task of interpreting the request, but the application owns the decision about whether execution may continue. The missing order ID is represented as an explicit routing condition. Because a required precondition is absent, the graph cannot enter the refund-review path and moves to the permitted clarification state.

4.1.7 Integrated example: a bounded support-policy reflection loop

The following support example combines evidence retrieval, answer generation, structured verification, limited revision attempts, and fail-closed routing. The example makes an important distinction between a purchased gift card and a physical item that someone returns as a gift. The available policy explains how gift returns work. However, it does not say whether a purchased gift card can be refunded. For this reason, the only answer supported by the available evidence is to abstain, unless the system can find a separate policy about gift-card refunds. The recorded execution also shows a verifier error. The verifier incorrectly treats the gift-return policy as evidence about purchased gift cards. It then recommends a conclusion that is not supported by the policy. However, the verifier returns a verdict that does not allow acceptance, and the workflow has already used all its allowed attempts. The graph therefore abstains instead of returning either of the generated drafts. This recorded run demonstrates limited, fail-closed behaviour. However, it does not prove that the verifier will always reject unsupported answers. With this workflow structure, an incorrect supported verdict would still send the answer to the acceptance path. For this reason, a production workflow should use stronger acceptance controls. These may include deterministic eligibility rules, evidence identifiers, checks that connect each claim to a source, improved retrieval, or human review when accepting an incorrect answer could have serious consequences.

```

from __future__ import annotations

```

```

from typing import Literal
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
from .config import OPENAI_API_KEY

POLICIES = [
    {
        "id": "returns_standard",
        "text": "Most physical items can be returned within 30 days of delivery for a refund to the original payment method."
    },
    {
        "id": "returns_damaged",
        "text": "Damaged or defective items may be refunded or replaced when reported within 14 days of delivery."
    },
    {
        "id": "returns_gifts",
        "text": "Gift returns are eligible for store credit only and must still be requested within the standard 30-day return window."
    }
]

MAX_ATTEMPTS = 2

class VerificationResult(BaseModel):
    verdict: Literal["supported", "needs_revision", "insufficient_evidence"] = Field(
        description="Whether the draft is supported by the evidence."
    )
    critique: str = Field(
        description="Concrete feedback describing what is wrong or missing."
    )
    grounded_facts: list[str] = Field(
        default_factory=list,
        description="Short statements directly supported by the evidence."
    )

class SupportState(TypedDict):
    question: str
    evidence: list[str]
    draft: str
    critique: str
    grounded_facts: list[str]
    verdict: Literal["", "supported", "needs_revision", "insufficient_evidence", "abstained"]
    attempts: int
    final_answer: str

generator = init_chat_model("openai:gpt-4.1-mini", temperature=0)
verifier = init_chat_model("openai:gpt-4.1", temperature=0).with_structured_output(
    VerificationResult
)

def retrieve_policy_evidence(state: SupportState) -> dict:
    question_terms = {
        token.strip(".,?!:;").lower()
        for token in state["question"].split()
        if len(token.strip(".,?!:;")) > 2
    }

    scored: list[tuple[int, str]] = []
    for item in POLICIES:
        policy_terms = {
            token.strip(".,?!:;").lower()
            for token in item["text"].split()
            if len(token.strip(".,?!:;")) > 2
        }
        overlap = len(question_terms & policy_terms)
        if overlap > 0:
            scored.append((overlap, f"[{item['id']}] {item['text']}"))

    scored.sort(key=lambda x: x[0], reverse=True)
    evidence = [text for _, text in scored[:3]]
    return {"evidence": evidence}

def write_draft(state: SupportState) -> dict:
    evidence_block = "\n".join(state["evidence"]) if state["evidence"] else "NO_EVIDENCE"
    if state["critique"]:
        instruction = f"""

```

```

Revise the answer using the critique below.

Critique:
{state["critique"]}
"""
    else:
        instruction = "Write the first draft."

        prompt = f"""
You are a customer support assistant.
Answer the user's question using only the policy evidence below.
Do not invent rules, exceptions, or timelines.
If the evidence is insufficient, say so plainly.

User question:
{state["question"]}

Policy evidence:
{evidence_block}

{instruction}
"""

        msg = generator.invoke(prompt)
        return {"draft": msg.content, "attempts": state["attempts"] + 1}

def verify_draft(state: SupportState) -> dict:
    evidence_block = "\n".join(state["evidence"]) if state["evidence"] else "NO_EVIDENCE"
    result = verifier.invoke(f"""
You are verifying a support answer against policy evidence.

User question:
{state["question"]}

Draft answer:
{state["draft"]}

Policy evidence:
{evidence_block}

Rules:
1. Mark "supported" only if the draft's material claims are justified by the evidence.
2. Mark "needs_revision" if the answer is close but overstates, omits a key condition, or
mixes supported and unsupported claims.
3. Mark "insufficient_evidence" if the evidence does not justify answering the question
safely.
4. grounded facts must contain only short statements directly supported by the evidence.
""")

    return {
        "verdict": result.verdict,
        "critique": result.critique,
        "grounded_facts": result.grounded_facts
    }

def route_after_verification(state: SupportState) -> str:
    if state["verdict"] == "supported":
        return "accept"
    if state["verdict"] == "insufficient_evidence" or state["attempts"] >= MAX_ATTEMPTS:
        return "abstain"
    return "revise"

def accept_answer(state: SupportState) -> dict:
    return {
        "final_answer": state["draft"],
        "verdict": "supported"
    }

def abstain(state: SupportState) -> dict:
    if state["verdict"] == "insufficient_evidence":
        final_critique = state["critique"] or (
            "The retrieved policy evidence does not safely answer the user's question."
        )
    else:
        base = state["critique"].strip()
        suffix = (
            "Maximum revision attempts reached before producing a supported answer. "
            "The workflow is failing closed and escalating to a human."
        )
        final_critique = f"{base}\n\n{suffix}" if base else suffix

    return {
        "final_answer": (

```

```

        "I cannot answer this safely from the retrieved policy evidence. "
        "Please ask a human support agent to review the case."
    ),
    "verdict": "abstained",
    "critique": final_critique
}

builder = StateGraph(SupportState)
builder.add_node("retrieve_policy_evidence", retrieve_policy_evidence)
builder.add_node("write_draft", write_draft)
builder.add_node("verify_draft", verify_draft)
builder.add_node("accept_answer", accept_answer)
builder.add_node("abstain", abstain)

builder.add_edge(START, "retrieve_policy_evidence")
builder.add_edge("retrieve_policy_evidence", "write_draft")
builder.add_edge("write_draft", "verify_draft")
builder.add_conditional_edges(
    "verify_draft",
    route_after_verification,
    {
        "accept": "accept_answer",
        "revise": "write_draft",
        "abstain": "abstain"
    }
)
builder.add_edge("accept_answer", END)
builder.add_edge("abstain", END)

graph = builder.compile()

if __name__ == "__main__":
    result = graph.invoke(
        {
            "question": "Can I get a refund for a gift card purchase?",
            "evidence": [],
            "draft": "",
            "critique": "",
            "grounded_facts": [],
            "verdict": "",
            "attempts": 0,
            "final_answer": ""
        }
    )

    print("QUESTION:\n")
    print(result["question"])
    print("\nFINAL ANSWER:\n")
    print(result["final_answer"])
    print("\nVERDICT:", result["verdict"])
    print("\nCRITIQUE:\n")
    print(result["critique"])
    print("\nEVIDENCE:")
    for item in result["evidence"]:
        print("-", item)

```

vol3_ch_04\src\verifier.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.verifier

QUESTION: Can I get a refund for a gift card purchase?

FINAL ANSWER: I cannot answer this safely from the retrieved policy evidence. Please ask
a human support agent to review the case.

VERDICT: abstained

CRITIQUE: The draft answer incorrectly states that the policy does not provide
information about refunds for gift card purchases. The evidence specifies that gift
returns are eligible for store credit only, not a refund to the original payment method.
The answer should clarify that refunds for gift cards are not available, but store credit
may be issued if requested within 30 days.

Maximum revision attempts reached before producing a supported answer. The workflow is
failing closed and escalating to a human.

EVIDENCE:
- [returns_standard] Most physical items can be returned within 30 days of delivery for a
refund to the original payment method.

```

```
- [returns_gifts] Gift returns are eligible for store credit only and must still be requested within the standard 30-day return window.
```

Answer generation is not the final step in this graph. Every draft must be checked before it can be accepted. The retrieval step first creates a small and easy-to-inspect set of evidence by looking for simple matching terms. This retriever is intentionally basic and is used only for demonstration. It is not meant to represent a production retrieval system. The generator receives only the retrieved evidence. Each time it creates a draft, it increases the attempt counter. The verifier then produces a probabilistic evaluation. It returns a structured verdict, a critique, and a list of facts supported by the evidence. The routing step uses this result to decide what happens next. Only a supported verdict can move to the acceptance path. A `needs_revision` verdict may return the draft to the generator, but only while attempts are still available. An `insufficient_evidence` verdict goes directly to abstention because this example does not include another step for finding additional evidence. In the recorded run, the verifier makes an error. Its critique incorrectly treats a gift return, and a gift-card purchase as the same type of situation. However, the verifier still returns a verdict that does not allow acceptance. The workflow has also reached its maximum number of attempts. For these reasons, the graph abstains instead of returning either draft to the user. This shows how bounded routing can prevent repeated, non-accepted results from being released. However, the graph does not protect against every possible verifier error. If the verifier incorrectly returned supported, the current routing logic would accept the answer. A production workflow should therefore combine model-based verification with stronger controls. These may include deterministic policy checks, evidence identifiers, checks that connect claims to their sources, improved retrieval, or human approval. The workflow may also send insufficient or conflicting evidence to another authorized source before deciding to abstain.

4.1.8 Extending verification to actions: human approval

When a reflection workflow can cause a real change in a system, verification must also apply before the tool performs that action. For example, an e-commerce assistant may be allowed to read an order automatically, but it may require human approval before submitting a refund. In this design, the approval step is part of the workflow's acceptance rules. It is not an optional user-interface feature.

```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

def lookup_order(order_id: str) -> str:
    """Read-only order lookup."""
    return f"Order {order_id}: delivered"

def submit_refund(order_id: str, reason: str) -> str:
    """State-changing action."""
    return f"Refund requested for {order_id} ({reason})"

agent = create_agent(
    model = "gpt-4.1",
    tools = [lookup_order, submit_refund],
    checkpoint = InMemorySaver(),
    middleware = [
        HumanInTheLoopMiddleware(
            interrupt_on = {
                "lookup_order": False,
                "submit_refund": {
                    "allowed_decisions": ["approve", "edit", "reject"]
                }
            }
        )
    ]
)
```

```

    )
    )
    )

```

The design rule is simple: if there is no approval, the action must not run. Reflection may help the model prepare or recommend a refund, but it does not give the model permission to submit the refund. The middleware checks the proposed tool call and stops the workflow before the tool changes any data. The workflow can continue only after it receives an explicit decision to approve the action or approve it with changes. If the action is rejected, the workflow stops it or sends it to another path, depending on the application rules. `InMemorySaver` is suitable for a local example, but it does not keep data after the application process restarts. A production system should use a durable checkpointer. It should also resume the interrupted workflow using the same thread identifier. Without durable storage, the pending approval request and the related workflow state may be lost.

4.1.9 Extending reflection across runs: validated memory

Answer-level reflection improves the current attempt. Memory-backed reflection extends this idea to future interactions by saving a lesson that has already been checked and approved for reuse. For example, a support assistant may repeatedly claim that a customer is eligible for a refund when the evidence does not support that claim. An e-commerce assistant may repeatedly confuse product availability with recommendation quality. A trading assistant may repeatedly write conclusions that are stronger than the available evidence. In these situations, discovering the same correction again in every interaction is inefficient. The system should not store a complete conversation or an unreviewed model opinion. Instead, it should store a short operating instruction. This instruction should describe the repeated problem and explain what the assistant must do to avoid it. This is related to the Reflexion pattern. The system does not change the model's internal weights. Instead, it saves written feedback in memory and uses it to guide later attempts. Persistence increases both the benefit and the risk. A weak answer affects one interaction. An incorrect stored instruction may affect many future interactions. For this reason, memory-backed reflection should be used only when the same problem happens repeatedly, the lesson remains valid across different tasks, and the reflection has been checked before it is saved in durable memory. The following example shows how to store and reuse a validated reflection. It does not create the reflection automatically and does not implement a complete learning system. It receives an instruction that has already been approved, stores it under the assistant's identity, and loads it before generating a later answer. The example uses a dictionary stored in the current application process. This allows the instruction to be reused across multiple calls while the same Python process is running. However, the instruction is lost when the process restarts, and it cannot be shared safely between multiple application workers. A production system should store validated reflections in a durable LangGraph Store or application database. The stored reflection should include provenance, ownership, version information, validation status, and deletion controls. The safety rule remains the same: generation may use reusable guidance, but only validated guidance may be written to durable storage.

```

from __future__ import annotations
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model

```



```

from langgraph.graph import StateGraph, START, END

MEMORY_STORE: dict[str, list[str]] = {}

class State(TypedDict):
    assistant_id: str
    question: str
    memory_notes: list[str]
    validated_reflection: str
    answer: str

model = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

def load_memory(state: State) -> dict:
    return {
        "memory_notes": MEMORY_STORE.get(state["assistant_id"], [])
    }

def write_answer(state: State) -> dict:
    memory_block = "\n".join(f"- {n}" for n in state["memory_notes"]) or "NO_MEMORY"
    msg = model.invoke(f"""
You are a support assistant.

Persistent operating notes:
{memory_block}

Question:
{state["question"]}

Answer carefully and do not overstate policy conclusions.
""")
    return {"answer": msg.content}

def store_reflection(state: State) -> dict:
    note = state["validated_reflection"].strip()
    if note:
        MEMORY_STORE.setdefault(state["assistant_id"], []).append(note)
    return {}

builder = StateGraph(State)
builder.add_node("load_memory", load_memory)
builder.add_node("write_answer", write_answer)
builder.add_node("store_reflection", store_reflection)

builder.add_edge(START, "load_memory")
builder.add_edge("load_memory", "write_answer")
builder.add_edge("write_answer", "store_reflection")
builder.add_edge("store_reflection", END)

graph = builder.compile()

if __name__ == "__main__":
    first = graph.invoke({
        "assistant_id": "support-v1",
        "question": "Can I definitely get a refund for a special-case return?",
        "memory_notes": [],
        "validated_reflection": (
            "Do not claim refund eligibility unless the policy text states it explicitly."
        ),
        "answer": ""
    })

    second = graph.invoke({
        "assistant_id": "support-v1",
        "question": "Can I get a refund for a gift return?",
        "memory_notes": [],
        "validated_reflection": "",
        "answer": ""
    })

    print("ANSWER 1:")
    print(first["answer"])
    print("\nSTORED MEMORY:")
    print(MEMORY_STORE["support-v1"])
    print("\nANSWER 2:")
    print(second["answer"])

```

vol3_ch_04\src\memory_backed_reflection.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.memory_backed_reflection
ANSWER 1:d

```

```
Refund eligibility for special-case returns can depend on the specific circumstances and the policies of the company or seller involved. While some special-case returns may qualify for a refund, it is not guaranteed in every situation. It's best to review the return policy carefully and contact customer support directly to discuss your particular case and understand what options are available.
```

```
STORED MEMORY:
['Do not claim refund eligibility unless the policy text states it explicitly.']
```

```
ANSWER 2:
Whether you can get a refund for a gift return depends on the specific return policy of the retailer or service from which the gift was purchased. Some stores allow refunds to the original purchaser or offer store credit for gift returns, while others may have different procedures.
```

```
If the policy text you have does not explicitly state that refunds are available for gift returns, it is best to check directly with the retailer or review their official return policy for clarification.
```

The example stores a validated operating instruction rather than an entire transcript or an unreviewed model conclusion. That distinction is the control boundary. Memory-backed reflection is useful when it converts a recurring and well-understood failure into stable guidance. It becomes unsafe when temporary observations, user-specific preferences, unsupported conclusions, or outdated policies are promoted into reusable instructions. A production store should therefore preserve who approved the reflection, which evidence or evaluation justified it, when it was created, where it applies, and how it can be revised or deleted. Durable memory increases the impact of both correct and incorrect guidance.

4.1.10 Implementation and evaluation in LangChain, LangGraph, and LangSmith

LangChain provides the main building blocks used inside reflection and verification workflows. These include models, prompts, tools, middleware, structured output, and agents. Agents created with `create_agent` run on the LangGraph runtime. When a workflow needs explicit state, branches, loops, recovery paths, or approval steps, LangGraph provides the lower-level orchestration layer. The graph state can directly store information such as drafts, decisions, critiques, attempt counters, evidence, approval status, and final results. Conditional routing can then decide what happens next. The workflow may move to acceptance, revision, improved retrieval, clarification, abstention, escalation, or human approval. Reflection can also be implemented as a small subgraph inside a larger workflow. For example, it may run after retrieval, before final answer generation, or before a tool performs an action that changes data. LangSmith provides the measurement and evaluation layer. Offline evaluation tests the application against prepared datasets before deployment. It can be used to compare different designs and detect regressions. Online evaluation checks production runs or complete conversation threads. It helps monitor real-world behaviour and identify weak acceptance rules or new types of failure. However, these evaluations are monitoring and feedback tools. They do not replace checks that must happen during the workflow before an answer or action is allowed. For example, an evaluator that scores a production trace cannot block an answer or action after it has already been released. Asynchronous evaluation only describes how an evaluation job runs. It is not a separate evaluation category at the same level as offline and online evaluation. In summary, current LangChain agents run on LangGraph. LangGraph remains the lower-level framework for building custom workflows with both deterministic and agent-based logic. LangSmith separates evaluation into two main types: offline evaluation over prepared datasets and online evaluation over production runs and threads.

4.1.11 Legacy evaluator classes and current equivalents

Older LangChain examples may use `LLMCheckerChain` and evaluator classes such as `CriteriaEvalChain`, `PairwiseStringEvalChain`, `string-distance` evaluators, `embedding-distance` evaluators, and `TrajectoryEvalChain`. These components do not all have the same status. `LLMCheckerChain` is deprecated. The evaluator chains are still available in the `langchain_classic` package so that existing applications and evaluation workflows can continue to use them. These evaluators also perform different types of checks. Criteria evaluators score an output against a defined set of rules. Pairwise evaluators compare two different outputs. String-distance and embedding-distance evaluators measure how similar or different two outputs are, either by comparing their words or their meaning. Trajectory evaluators examine which actions an agent selected and the order in which it performed them. For this reason, these evaluators should not be treated as one common API for controlling live execution. Current LangSmith workflows can evaluate final answers, structured outputs, tool selection, tool-call order, and complete agent trajectories. They can use code-based evaluators, LLM-as-a-judge evaluators, pairwise comparisons, and human review. These evaluations can run offline on prepared datasets or online on production traces. Their main purpose is measurement. They support testing, benchmarking, regression detection, and production monitoring. When a decision must control whether a live workflow can continue, that decision should remain inside LangChain or LangGraph runtime logic. The runtime should use structured decisions, explicit routing, attempt limits, and safe final states. Evaluation should then measure whether these runtime controls improve quality enough to justify their additional latency and cost. The current reference keeps criteria, pairwise, distance, and trajectory evaluators in `langchain_classic`. They should not all be described as deprecated.

4.1.12 When reflection helps, and when it hurts

Reflection is worth its cost when feedback can clearly improve a result or when verification can check the result against a stronger source of judgment. Use self-critique for qualities that can be improved directly, such as clarity, tone, structure, and completeness. Use deterministic checks for exact conditions that software can calculate or validate. Use evidence-based verification for factual claims and policy-sensitive decisions. When evidence is missing, improve the retrieval process or use another authorized source. Do not repeatedly rewrite the same answer, because rewriting cannot create missing evidence. Use human approval when an action requires authority or may have important consequences. Avoid loops that ask the same model to review the same text without giving it new evidence, clearer rules, or useful feedback. A simple database lookup should not become a generate-and-critique loop. Exact authorization rules and business rules should not be decided by an LLM evaluator. Every repeating path must have clear and observable acceptance rules, a maximum number of attempts, and a defined final outcome. Possible outcomes include clarification, abstention, rejection, escalation, or termination. Evaluation should measure these controls using representative datasets and production traces. The goal is not to add as many review steps as possible. The goal is to place the strongest affordable control at the point where a failure would become expensive. These changes remove direct duplication, correct the overly strong safety claim, clarify the difference between reflection and verification, update the LangChain ecosystem terminology, and reduce repetition without changing the chapter's code or outputs.

4.2 SEARCH-BASED REASONING: TREE-OF-THOUGHT, BEST-OF-N, MCTS-STYLE VARIANTS

4.2.1 From single-pass reasoning to explicit search

The main change is not about writing a better prompt. It is about moving from one model response to a search process that runs while the model is working. In this process, the system creates several possible solutions, evaluates them, decides which ones to keep or develop further, removes the weaker ones, and stops when it reaches predefined limits. Because these steps are part of the application, developers can see the generated candidates, their evaluations, and the decisions made by the workflow. However, this does not show the model's private internal reasoning. That reasoning remains hidden. This section explains three related search patterns. Best-of-N generates several complete answers and selects the best one. Tree of Thoughts, called Tree-of-Thought search in this chapter, works with visible partial solutions or plans. It develops the most promising options and removes the weaker ones. MCTS-style approaches add more advanced search behaviour. They can choose branches based on previous results, return to the same branch several times, estimate the possible result of a branch, store statistics about previous attempts, and balance trying new options with continuing to develop promising ones. These patterns are related, but they are not three fixed stages of increasing maturity. The best choice depends on how the task is structured, how well the system can evaluate possible solutions, and how much processing time and cost are available during inference. LangGraph does not include a ready-made search algorithm. Instead, it provides state-management and control-flow features that developers can use to build these search workflows. Section 4.2.6 explains how these LangGraph features can be used for the different search approaches. LangGraph recursion limits control the number of graph supersteps. They do not directly limit the number of candidate solutions, model calls, tokens, parallel tasks, or the total cost. The application must therefore define separate limits for each of these resources.

4.2.2 Best-of-N: sample, score, and select

Best-of-N is the simplest form of search that generates several answers and then chooses one. The system creates N complete candidates and applies a rule to decide which result to keep. Generating several candidates creates variation. However, this improves the final result only when the model has a reasonable chance of producing a better answer and when the selection rule can correctly recognize quality. Otherwise, generating more candidates only increases cost or makes weaknesses in the evaluator more important. For tasks that have a short and clearly defined final answer, self-consistency is a closely related technique. The system generates several different reasoning paths, extracts the final answer from each one in a standard format, and selects the answer that appears most often. Unlike scored Best-of-N, self-consistency usually looks at agreement between answers instead of giving a quality score to each complete candidate. Self-consistency works best when different valid reasoning paths can lead to the same correct answer. This is common in arithmetic, symbolic reasoning, and similar benchmark-style tasks. It should not be used as general majority voting for every date, label, or classification problem. Some of these tasks are better solved with deterministic rules, calibrated classifiers, retrieval, or external verification. Open-ended answers are more difficult

because two answers can have the same meaning while using very different words. For this reason, exact-text voting is usually not suitable. The system may instead use a deterministic verifier, pointwise scoring that evaluates each candidate separately, pairwise comparison that compares two candidates directly, listwise judging that ranks several candidates in one evaluation, a reranker, or a combination of these methods.

Self-consistency is one specific way to combine several answers. Best-of-N is the broader family of workflows that generate several complete candidates and then select one of them. Best-of-N is often the first search pattern worth testing because it does not require a persistent tree, a backtracking policy, or a rollout mechanism. However, it still requires more than generating candidates and asking a judge to choose one. The selector must be tested on representative examples. Ties and cases where no answer is good enough must be handled deliberately. Whenever possible, deterministic rules should take priority over model preferences. LLM judges can show position bias, prefer answers similar to their own output, score inconsistently, and share the same weaknesses as the model that generated the candidates. Increasing N can also create another problem. With more candidates, it becomes easier for one answer to exploit weaknesses in an imperfect scoring model instead of being genuinely better. In the basic form of Best-of-N, candidates are generated independently. They do not receive feedback from one another and do not share a partial solution. If all candidates use the same model, prompt, evidence, and assumptions, they may make very similar mistakes. Best-of-N can select the best variation that already exists, but it cannot perform structured backtracking or correct an assumption shared by every candidate.

The following example applies this pattern to a realistic customer-support scenario. The graph receives one customer case and asks the model to generate three clearly different replies in a single structured generation call. This keeps the example compact. However, a more traditional Best-of-N implementation often generates candidates through separate or batched model calls to make them more independent and diverse. The graph then sends each candidate to a separate judging step. Each reply is scored for policy fit, clarity, and operational risk. After that, the graph applies a simple deterministic scoring rule and selects the highest-ranked reply as the final answer. Two design choices are important. First, both generation and evaluation use structured output. This keeps the graph state predictable and makes later nodes easier to write and validate. Second, the judging stage is parallelized with Send. This means the graph evaluates the candidates as a fan-out step instead of processing them one after another. This is a small but important example of how LangGraph turns a search idea into a visible execution structure. The example is intentionally limited. It does not try to solve every support problem or imitate a long-running agent. Its purpose is to show one idea clearly: Best-of-N is already a search architecture when candidate generation, evaluation, and selection are implemented as separate stages in the graph.

```
from __future__ import annotations
import operator
from typing import Annotated
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send
from .config import OPENAI_API_KEY

POLICY = ""
```

```

Support policy summary:
- Most physical items can be returned within 30 days of delivery.
- Gift purchases are usually refunded as store credit unless the original purchaser
  requests a refund to the original payment method.
- Defective items may be replaced if the issue is confirmed and the item is within
  policy.
- Never promise a refund that is not explicitly supported by the policy and facts.
- If facts are missing or policy is ambiguous, ask a clarifying question or escalate.
""".strip()

class CandidateReply(BaseModel):
    id: str = Field(description = "Short id such as reply_1")
    reply: str = Field(description = "Customer-facing reply")
    strategy_note: str = Field(description = "One-line description of the strategy")

class CandidateReplySet(BaseModel):
    candidates: list[CandidateReply]

class ReplyJudgment(BaseModel):
    policy_fit: int = Field(ge = 1, le = 10)
    clarity: int = Field(ge = 1, le = 10)
    risk: int = Field(ge = 1, le = 10)
    rationale: str

class GraphState(TypedDict):
    customer_case: str
    candidates: list[dict]
    scored_candidates: Annotated[list[dict], operator.add]
    final_answer: str

class JudgeWorkerState(TypedDict):
    customer_case: str
    candidate: dict
    scored_candidates: Annotated[list[dict], operator.add]

def build_model(temperature: float = 0.2) -> ChatOpenAI:
    return ChatOpenAI(model = "gpt-4.1-mini", temperature = temperature)

def total_score(policy_fit: int, clarity: int, risk: int) -> int:
    return (2 * policy_fit) + clarity - risk

def generate_candidates(state: GraphState) -> dict:
    planner = build_model(0.7).with_structured_output(
        CandidateReplySet,
        method = "json_schema"
    )

    prompt = f"""
    You are drafting support replies.

    {POLICY}

    Customer case:
    {state["customer case"]}

    Generate exactly 3 materially different candidate replies.
    Each reply must:
    - stay inside the policy,
    - avoid unsupported promises,
    - reflect a distinct response strategy.

    Return only structured output.
    """.strip()

    result = planner.invoke(prompt)

    return {
        "candidates": [candidate.model_dump() for candidate in result.candidates],
        "scored_candidates": []
    }

def assign_judges(state: GraphState):
    return [
        Send(
            "judge_candidate",
            {
                "customer_case": state["customer_case"],
                "candidate": candidate
            }
        )
        for candidate in state["candidates"]
    ]

def judge_candidate(state: JudgeWorkerState) -> dict:
    candidate = CandidateReply.model_validate(state["candidate"])

```



```

judge = build_model(0.0).with_structured_output(
    ReplyJudgment,
    Method = "json_schema"
)

prompt = f"""
You are judging one support reply.

{POLICY}

Customer case:
{state["customer_case"]}

Candidate reply:
id: {candidate.id}
strategy_note: {candidate.strategy_note}
reply: {candidate.reply}

Score the reply on:
- policy_fit: how strongly it is supported by policy and facts,
- clarity: how clear and useful it is to the customer,
- risk: unsupported promises or operational ambiguity.

Be strict.
"""

judgment = judge.invoke(prompt)

return {
    "scored_candidates": [
        {
            "candidate": candidate.model_dump(),
            **judgment.model_dump(),
            "total_score": total_score(
                judgment.policy_fit,
                judgment.clarity,
                judgment.risk
            )
        }
    ]
}

def select_best(state: GraphState) -> dict:
    ranked = sorted(
        state["scored_candidates"],
        key = lambda item: item["total_score"],
        reverse = True
    )
    best = ranked[0]

    candidate = best["candidate"]
    answer = "\n".join(
        [
            f"Selected reply: {candidate['id']}",
            f"Strategy: {candidate['strategy_note']}",
            "",
            candidate["reply"],
            "",
            (
                f"Scores: policy_fit={best['policy_fit']}, "
                f"clarity={best['clarity']}, "
                f"risk={best['risk']}, "
                f"total={best['total_score']}"
            )
        ]
    )

    return {"final_answer": answer}

builder = StateGraph(GraphState)
builder.add_node("generate_candidates", generate_candidates)
builder.add_node("judge_candidate", judge_candidate)
builder.add_node("select_best", select_best)

builder.add_edge(START, "generate_candidates")
builder.add_conditional_edges("generate_candidates", assign_judges)
builder.add_edge("judge_candidate", "select_best")
builder.add_edge("select_best", END)

graph = builder.compile()

if __name__ == "__main__":
    customer case = """
The earbuds were bought 10 days ago as a gift.

```

```

The recipient says the right earbud is defective.
They want an immediate cash refund to their own card.
""".strip()

    result = graph.invoke(
        {"customer_case": customer_case},
        config = {"recursion_limit": 10}
    )

    print(result["final_answer"])

```

vol3_ch_04\scr\best_of_n.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.best_of_n

CUSTOMER CASE:
The earbuds were bought 10 days ago as a gift.
The recipient says the right earbud is defective.
They want an immediate cash refund to their own card.

=====
Selection process:
The graph generated 3 candidates, judged each one independently, then ranked them by
total_score = (2 * policy_fit) + clarity - risk.

Ranked candidates:
1. reply_1 | strategy=Clarify gift purchase refund policy and offer replacement after
confirmation.
   scores: policy_fit=9, clarity=8, risk=2, total=24
   judge rationale: The reply aligns well with the policy by acknowledging the 30-day
return window and the gift purchase refund rules. It correctly states that refunds for
gift purchases are usually store credit unless the original purchaser requests otherwise,
and it offers a replacement after confirming the defect, which is consistent with policy.
The message is clear and informative, guiding the customer on next steps. The only minor
risk is that it does not explicitly ask who is making the refund request (recipient or
original purchaser), which could be clarified further to avoid ambiguity. However, it
does not promise an unsupported immediate cash refund to the recipient's own card, thus
keeping risk low.
   reply: Thank you for reaching out. Since the earbuds were purchased as a gift and are
within the 30-day return window, we can assist you with a replacement for the defective
right earbud once the issue is confirmed. Please note that refunds for gift purchases are
typically issued as store credit unless the original purchaser requests otherwise.

2. reply_3 | strategy=Explain policy limitations on refund to recipient's card and offer
replacement.
   scores: policy_fit=9, clarity=8, risk=2, total=24
   judge rationale: The reply aligns well with the policy by acknowledging the defect and
return period, offering a replacement, and explaining the refund limitations for gift
purchases. It clearly communicates that refunds to the recipient's card are not supported
unless requested by the original purchaser, which matches the policy. The message is
mostly clear and useful, though it could be slightly more explicit about the need to
verify the defect before replacement. The risk is low, as no unsupported promises are
made; the reply carefully avoids promising a refund not supported by policy. Overall, it
responsibly manages expectations and adheres to policy.
   reply: Since the earbuds are defective and within the return period, we can initiate a
replacement once the defect is verified. However, as this was a gift purchase, refunds
are usually provided as store credit unless the original purchaser requests a refund to
their payment method. Unfortunately, we cannot provide a direct cash refund to your card
in this case.

3. reply_2 | strategy=Request clarification about original purchaser's refund preference.
   scores: policy_fit=5, clarity=5, risk=1, total=14
   judge rationale: The reply aligns well with the policy by not promising a refund and
instead asking for clarification about the original purchaser's refund preference, which
is necessary given the gift purchase context. It clearly communicates the next step to
the customer, making it useful and understandable. There are no unsupported promises or
ambiguous statements, minimizing risk.
   reply: We're sorry to hear about the issue with your earbuds. To assist you further,
could you please confirm whether the original purchaser would like to request a refund to
their original payment method? This will help us determine the best way to proceed
according to our gift refund policy.

Winning selection:
reply_1 was selected because it ranked first with total=24. The next best option was
reply_3 with total=24. Score gap: 0.

=====
FINAL SELECTED ANSWER:

```

```
Selected reply: reply_1
Strategy: Clarify gift purchase refund policy and offer replacement after confirmation.

Thank you for reaching out. Since the earbuds were purchased as a gift and are within the
30-day return window, we can assist you with a replacement for the defective right earbud
once the issue is confirmed. Please note that refunds for gift purchases are typically
issued as store credit unless the original purchaser requests otherwise.

Winning scores: policy_fit=9, clarity=8, risk=2, total=24
```

The workflow clearly shows how sample-and-select works. First, the system creates three possible replies. Then, Send starts one evaluation for each reply. The reducer collects all the evaluation results, and the final node uses a clear scoring formula to rank the replies. Structured output gives each stage a fixed and predictable format. The system also stores the replies, scores, and explanations, so developers can follow how the final decision was made. The output also shows an important problem that must be handled in a real system. `reply_1` and `reply_3` receive the same total score. In this example, the system selects the first reply in the list. However, choosing the first item is not a meaningful decision rule. A production system should define a clear secondary rule for ties. For example, it could first prefer the reply with the lowest risk, then the reply with the highest policy-fit score. If the replies are still equal, the system could compare them directly or send the case for escalation. Being able to trace the decision does not mean that the decision is objectively correct. The evaluation criteria, score weights, judge model, tie-breaking rule, minimum acceptable score, and rules for rejecting all candidates are all part of the decision policy. They must be tested using representative customer-support cases. Whenever a policy rule can be checked directly in code, that rule should override or limit the score produced by the model. The model judge should be treated as an estimate of quality, not as final proof that the selected reply follows the policy.

4.2.3 Tree-of-Thought as branching and pruning

Tree-of-Thought changes both how many candidates the system creates and when it evaluates them. Instead of generating several complete answers and choosing one only at the end, the system searches through visible intermediate solution states. These states may be partial plans, hypotheses, calculations, or short strategy descriptions. From each state, a generator proposes possible next steps. An evaluator estimates how promising or practical each option is. A search policy then decides which branches to continue, delay, remove, revisit, or stop.

These intermediate states are visible parts of the application and exist to support the search process. They should not be treated as a complete or reliable record of the model's private reasoning. A practical Tree-of-Thought workflow therefore includes four clear parts: the current problem state, a candidate generator, an evaluation signal, and a search policy. Keeping these parts separate makes it easier to inspect and test how the system creates branches, evaluates them, and decides where to continue. This pattern is most useful when several strategies are genuinely possible, and a poor early decision could damage the rest of the result. In a customer-support workflow, the assistant may need to compare refund, replacement, store credit, and escalation options before replying. In e-commerce, it may need to compare several campaign strategies while considering stock and margin limits. In trading or market analysis, it may need to compare different analysis paths while respecting risk limits, exposure rules, and incomplete evidence. In all these cases, a single straight-line answer may sound reasonable while still following the wrong strategy. Tree-of-Thought gives the

system time to compare alternatives before choosing one. The important engineering point is that Tree-of-Thought is not the same as simply asking the model to generate three options. Its real value comes from evaluating and removing branches during the search. When a branch appears weak, the system should stop spending resources on it. When one branch becomes clearly stronger, the system should develop that branch further instead of expanding every option equally.

Without this control logic, the system is not performing a real tree search. It is only generating several drafts. The following example implements a shallow and limited Tree-of-Thought-style search for an e-commerce inventory decision. The system generates several partial strategy types, scores them against stock, margin, customer, and risk constraints, keeps a small set of the strongest options, and expands only those surviving strategies into concrete actions. This scenario is suitable because the alternatives are genuinely different, and the initial strategic choice has a significant effect on the final recommendation.

```
from __future__ import annotations
import operator
from typing import Annotated
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send
from .config import OPENAI_API_KEY

BUSINESS_CONTEXT = """
Business context:
- The product is high-margin and trending.
- Available stock is low.
- Overselling would hurt customer trust.
- Substitute products exist with slightly lower margin.
- Avoid recommendations that ignore stock constraints.
""".strip()

TOP_K = 2

class Strategy(BaseModel):
    id: str
    title: str
    summary: str

class StrategySet(BaseModel):
    strategies: list[Strategy]

class StrategyScore(BaseModel):
    stock_safety: int = Field(ge=1, le=10)
    margin_protection: int = Field(ge=1, le=10)
    customer_fit: int = Field(ge=1, le=10)
    risk: int = Field(ge=1, le=10)
    rationale: str

class ExpandedStep(BaseModel):
    id: str
    title: str
    concrete_action: str
    customer_impact: str
    risk_note: str

class ExpandedStepScore(BaseModel):
    stock_safety: int = Field(ge=1, le=10)
    margin_protection: int = Field(ge=1, le=10)
    execution_clarity: int = Field(ge=1, le=10)
    risk: int = Field(ge=1, le=10)
    rationale: str

class SearchState(TypedDict):
    scenario: str
    frontier: list[dict]
    scored_strategies: Annotated[list[dict], operator.add]
    expanded_steps: Annotated[list[dict], operator.add]
    final_answer: str
```

```

class ScoreWorkerState(TypedDict):
    scenario: str
    strategy: dict
    scored_strategies: Annotated[list[dict], operator.add]

class ExpandWorkerState(TypedDict):
    scenario: str
    strategy: dict
    expanded_steps: Annotated[list[dict], operator.add]

def build_model(temperature: float = 0.2) -> ChatOpenAI:
    return ChatOpenAI(model = "gpt-4.1-mini", temperature = temperature)

def initial_total(stock_safety: int, margin_protection: int, customer_fit: int, risk:
    int) -> int:
    return stock_safety + margin_protection + customer_fit - risk

def expanded_total(stock_safety: int, margin_protection: int, execution_clarity: int,
    risk: int) -> int:
    return stock_safety + margin_protection + execution_clarity - risk

def generate_strategies(state: SearchState) -> dict:
    planner = build_model(0.6).with_structured_output(StrategySet, method =
        "json_schema")

    prompt = f"""
You are planning an e-commerce response to a low-stock demand spike.

{BUSINESS_CONTEXT}

Scenario:
{state["scenario"]}

Generate exactly 3 materially different partial strategies.
These must be strategy directions, not final polished answers.
""".strip()

    result = planner.invoke(prompt)

    return {
        "frontier": [strategy.model_dump() for strategy in result.strategies],
        "scored_strategies": [],
        "expanded_steps": []
    }

def assign_strategy_scorers(state: SearchState):
    return [
        Send(
            "score_strategy",
            {
                "scenario": state["scenario"],
                "strategy": strategy
            }
        )
        for strategy in state["frontier"]
    ]

def score_strategy(state: ScoreWorkerState) -> dict:
    strategy = Strategy.model_validate(state["strategy"])

    judge = build_model(0.0).with_structured_output(StrategyScore, method =
        "json_schema")

    prompt = f"""
You are evaluating one partial strategy.

{BUSINESS_CONTEXT}

Scenario:
{state["scenario"]}

Strategy:
id: {strategy.id}
title: {strategy.title}
summary: {strategy.summary}

Score it on:
- stock_safety,
- margin_protection,
- customer_fit,
- risk.

Be strict.
""".strip()

    judgment = judge.invoke(prompt)

```

```

    return {
        "scored_strategies": [
            {
                "strategy": strategy.model_dump(),
                **judgment.model_dump(),
                "total_score": initial_total(
                    judgment.stock_safety,
                    judgment.margin_protection,
                    judgment.customer_fit,
                    judgment.risk
                )
            }
        ]
    }

def select_frontier(state: SearchState) -> dict:
    ranked = sorted(
        state["scored_strategies"],
        key = lambda item: item["total_score"],
        reverse = True
    )
    top = ranked[:TOP_K]
    return {"frontier": [item["strategy"] for item in top]}

def assign_expanders(state: SearchState):
    return [
        Send(
            "expand_step",
            {
                "scenario": state["scenario"],
                "strategy": strategy
            }
        )
        for strategy in state["frontier"]
    ]

def expand_step(state: ExpandWorkerState) -> dict:
    strategy = Strategy.model_validate(state["strategy"])

    expander = build_model(0.2).with_structured_output(ExpandedStep,
        method = "json_schema")
    judge = build_model(0.0).with_structured_output(ExpandedStepScore,
        method = "json_schema")

    expand_prompt = f"""
You are deepening one e-commerce strategy by one step.

{BUSINESS_CONTEXT}

Scenario:
{state["scenario"]}

Selected strategy:
id: {strategy.id}
title: {strategy.title}
summary: {strategy.summary}

Produce one concrete next step.
""".strip()

    step = expander.invoke(expand_prompt)

    judge_prompt = f"""
You are evaluating one expanded next step.

{BUSINESS_CONTEXT}

Scenario:
{state["scenario"]}

Expanded step:
id: {step.id}
title: {step.title}
concrete_action: {step.concrete_action}
customer_impact: {step.customer_impact}
risk_note: {step.risk_note}

Score it on:
- stock_safety,
- margin_protection,
- execution_clarity,
- risk.
""".strip()

```

```

judgment = judge.invoke(judge_prompt)

return {
    "expanded_steps": [
        {
            "parent_strategy": strategy.model_dump(),
            "step": step.model_dump(),
            **judgment.model_dump(),
            "total_score": expanded_total(
                judgment.stock_safety,
                judgment.margin_protection,
                judgment.execution_clarity,
                judgment.risk
            )
        }
    ]
}

def format_strategy_rankings(items: list[dict]) -> list[str]:
    lines: list[str] = []
    for index, item in enumerate(items, start = 1):
        strategy = item["strategy"]
        lines.extend(
            [
                f"{index}. {strategy['id']} - {strategy['title']}",
                f"    Summary: {strategy['summary']}",
                (
                    "    Scores: "
                    f"stock_safety={item['stock_safety']}, "
                    f"margin_protection={item['margin_protection']}, "
                    f"customer_fit={item['customer_fit']}, "
                    f"risk={item['risk']}, "
                    f"total={item['total_score']}"
                ),
                f"    Judge rationale: {item['rationale']}"
            ]
        )
    return lines

def format_expanded_rankings(items: list[dict]) -> list[str]:
    lines: list[str] = []
    for index, item in enumerate(items, start = 1):
        parent = item["parent_strategy"]
        step = item["step"]
        lines.extend(
            [
                f"{index}. From {parent['id']} - {parent['title']}",
                f"    Expanded branch: {step['title']}",
                f"    Concrete action: {step['concrete_action']}",
                f"    Customer impact: {step['customer_impact']}",
                f"    Risk note: {step['risk_note']}",
                (
                    "    Scores: "
                    f"stock_safety={item['stock_safety']}, "
                    f"margin_protection={item['margin_protection']}, "
                    f"execution clarity={item['execution clarity']}, "
                    f"risk={item['risk']}, "
                    f"total={item['total_score']}"
                ),
                f"    Judge rationale: {item['rationale']}"
            ]
        )
    return lines

def final_select(state: SearchState) -> dict:
    ranked_strategies = sorted(
        state["scored_strategies"],
        key = lambda item: item["total_score"],
        reverse = True
    )
    ranked_steps = sorted(
        state["expanded_steps"],
        key = lambda item: item["total_score"],
        reverse = True
    )
    best_step = ranked_steps[0]
    winning_parent = best_step["parent_strategy"]
    winning_step = best_step["step"]
    selected_frontier_ids = {strategy["id"] for strategy in state["frontier"]}

```



```

selected_frontier = [
    item for item in ranked_strategies if item["strategy"]["id"] in
    selected_frontier_ids
]

lines = [
    "TREE-OF-THOUGHT SELECTION PROCESS",
    "",
    "Scenario:",
    state["scenario"],
    "",
    "Stage 1 - Initial strategy generation and scoring:",
    *format_strategy_rankings(ranked_strategies),
    "",
    f"Stage 1 decision - Keep the top {TOP_K} strategies in the frontier:",
    *[
        f"- {item['strategy']['id']} - {item['strategy']['title']}
        (total={item['total_score']})"
        for item in selected_frontier
    ],
    "",
    "Stage 2 - Expand each frontier strategy by one concrete next step and score
    again:",
    *format_expanded_rankings(ranked_steps),
    "",
    "Final selection:",
    f"Selected strategy family: {winning_parent['id']} - {winning_parent['title']}",
    f"Selected branch: {winning_step['title']}",
    f"Why it won: {best_step['rationale']}",
    "",
    "Final recommended action:",
    winning_step["concrete_action"],
    "",
    "Expected customer impact:",
    winning_step["customer_impact"],
    "",
    "Risk note:",
    winning_step["risk_note"],
    "",
    (
        "Winning scores: "
        f"stock_safety={best_step['stock_safety']}, "
        f"margin_protection={best_step['margin_protection']}, "
        f"execution_clarity={best_step['execution_clarity']}, "
        f"risk={best_step['risk']}, "
        f"total={best_step['total_score']}"
    )
]

return {"final answer": "\n".join(lines)}

builder = StateGraph(SearchState)

builder.add_node("generate_strategies", generate_strategies)
builder.add_node("score_strategy", score_strategy)
builder.add_node("select_frontier", select_frontier)
builder.add_node("expand_step", expand_step)
builder.add_node("final_select", final_select)

builder.add_edge(START, "generate_strategies")
builder.add_conditional_edges("generate_strategies", assign_strategy_scorers)
builder.add_edge("score_strategy", "select_frontier")
builder.add_conditional_edges("select_frontier", assign_expanders)
builder.add_edge("expand_step", "final_select")
builder.add_edge("final_select", END)

graph = builder.compile()

if __name__ == "__main__":
    scenario = """
    Demand for a trending accessory just spiked after a social-media mention.
    Only 120 units remain. The product has excellent margin, but a stock-out would
    damage customer trust. Marketing wants guidance on what to do today.
    """.strip()

    result = graph.invoke(
        {"scenario": scenario},
        config = {"recursion_limit": 20}
    )

    print(result["final_answer"])

```

vol3_ch_04\src\tot.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.tot
```

```
TREE-OF-THOUGHT SELECTION PROCESS
```

```
Scenario:
```

```
Demand for a trending accessory just spiked after a social-media mention.
Only 120 units remain. The product has excellent margin, but a stock-out would
damage customer trust. Marketing wants guidance on what to do today.
```

```
Stage 1 - Initial strategy generation and scoring:
```

```
1. strategy3 - Create Scarcity-Driven Marketing with Pre-Order Options
```

```
Summary: Communicate limited stock availability to create urgency while opening pre-
orders for restock, ensuring customers can secure the product without overselling current
inventory and maintaining trust through transparency.
```

```
Scores: stock_safety=9, margin_protection=8, customer_fit=9, risk=3, total=23
```

```
Judge rationale: The strategy effectively manages limited stock by creating urgency
and offering pre-orders, preventing overselling and protecting customer trust (high stock
safety and customer fit). It maintains margin by focusing on the high-margin product and
leveraging pre-orders rather than pushing substitutes (good margin protection). The main
risk is potential customer dissatisfaction if pre-orders are delayed or if communication
is unclear, but transparency mitigates this risk, resulting in a low risk score.
```

```
2. strategy1 - Implement Real-Time Stock Monitoring and Purchase Limits
```

```
Summary: Set a strict purchase limit per customer to prevent overselling and monitor
stock levels in real-time to halt sales immediately when stock reaches zero, thus
protecting customer trust.
```

```
Scores: stock_safety=9, margin_protection=8, customer_fit=7, risk=3, total=21
```

```
Judge rationale: The strategy effectively protects stock safety by implementing real-
time monitoring and purchase limits, minimizing overselling risk and preserving customer
trust. Margin protection is strong as it maintains sales of the high-margin product
without resorting to substitutes. Customer fit is good, though purchase limits may
slightly inconvenience some customers. Risk is low due to proactive stock control, but
some risk remains if limits frustrate customers or if monitoring fails.
```

```
3. strategy2 - Promote Substitute Products with Incentives
```

```
Summary: Highlight and offer small discounts or bundles on substitute products with
slightly lower margins to divert some demand, preserving the high-margin product stock
and maintaining overall sales volume.
```

```
Scores: stock_safety=9, margin_protection=7, customer_fit=7, risk=3, total=20
```

```
Judge rationale: The strategy effectively protects stock by diverting demand to
substitutes, significantly reducing the risk of stock-out and preserving customer trust.
Margin protection is good but slightly compromised due to discounts on substitutes with
lower margins. Customer fit is decent as substitutes are similar products, though not the
exact trending item. Risk is low because it avoids overselling and maintains sales
volume, but some customers may be dissatisfied if they prefer the original product.
```

```
Stage 1 decision - Keep the top 2 strategies in the frontier:
```

```
- strategy3 - Create Scarcity-Driven Marketing with Pre-Order Options (total=23)
- strategy1 - Implement Real-Time Stock Monitoring and Purchase Limits (total=21)
```

```
Stage 2 - Expand each frontier strategy by one concrete next step and score again:
```

```
1. From strategy3 - Create Scarcity-Driven Marketing with Pre-Order Options
```

```
Expanded branch: Announce Limited Stock with Clear Messaging
```

```
Concrete action: Update the product page and marketing channels to prominently display
the limited stock of 120 units, emphasizing scarcity to create urgency. Include messaging
that explains the availability of pre-orders for the next batch to capture demand beyond
current inventory without overselling.
```

```
Customer impact: Customers feel urgency to purchase now due to limited stock, while
also having the option to pre-order, which maintains trust by avoiding overselling and
ensuring transparency about availability.
```

```
Risk note: Ensure messaging is clear and honest to avoid confusion or frustration;
inaccurate stock information or unclear pre-order terms could harm trust.
```

```
Scores: stock_safety=9, margin_protection=8, execution_clarity=9, risk=3, total=23
```

```
Judge rationale: This step carefully balances the limited stock by clearly
communicating scarcity, which helps prevent overselling and protects customer trust (high
stock safety). By promoting pre-orders for the next batch, it captures additional demand
without losing margin, as pre-orders typically maintain pricing and margin (strong margin
protection). The action is straightforward and actionable, involving updates to product
pages and marketing channels with clear messaging (high execution clarity). The main risk
lies in potential customer confusion or frustration if messaging is unclear or
inaccurate, but this risk is moderate and manageable with proper communication (moderate
risk). Overall, this approach aligns well with the business context and constraints.
```

```
2. From strategy1 - Implement Real-Time Stock Monitoring and Purchase Limits
```

```
Expanded branch: Configure Purchase Limits and Real-Time Stock Alerts
```

```
Concrete action: Set a maximum purchase quantity of 2 units per customer on the
trending accessory product page. Integrate real-time inventory tracking that triggers an
automatic sales halt when stock reaches zero. Additionally, set up alert notifications
```

```

for the inventory management team when stock falls below 20 units to prepare for
replenishment or marketing adjustments.
  Customer impact: Prevents customers from overspending on limited stock, ensuring fair
access and maintaining trust by avoiding stock-outs. Real-time stock updates provide
accurate availability information, reducing frustration and potential order
cancellations.
  Risk note: If purchase limits are set too low, some customers may feel restricted,
potentially reducing sales. Real-time monitoring depends on system reliability; any
delays or errors could still lead to overselling or stock-outs.
  Scores: stock safety=9, margin protection=8, execution clarity=9, risk=3, total=23
  Judge rationale: Setting a purchase limit of 2 units per customer directly addresses
the risk of overselling given the low stock of 120 units, thus strongly protecting stock
safety and customer trust. Real-time inventory tracking with automatic sales halt at zero
stock further ensures no overselling occurs. Alert notifications for low stock enable
proactive management, supporting margin protection by allowing timely replenishment or
marketing adjustments. The action is clear and actionable, with well-defined steps and
responsibilities, contributing to high execution clarity. The main risks involve
potential customer dissatisfaction due to purchase limits and reliance on system
reliability for real-time tracking; however, these risks are relatively low compared to
the benefits of preventing stock-outs and maintaining trust.

Final selection:
Selected strategy family: strategy3 - Create Scarcity-Driven Marketing with Pre-Order
Options
Selected branch: Announce Limited Stock with Clear Messaging
Why it won: This step carefully balances the limited stock by clearly communicating
scarcity, which helps prevent overselling and protects customer trust (high stock
safety). By promoting pre-orders for the next batch, it captures additional demand
without losing margin, as pre-orders typically maintain pricing and margin (strong margin
protection). The action is straightforward and actionable, involving updates to product
pages and marketing channels with clear messaging (high execution clarity). The main risk
lies in potential customer confusion or frustration if messaging is unclear or
inaccurate, but this risk is moderate and manageable with proper communication (moderate
risk). Overall, this approach aligns well with the business context and constraints.

Final recommended action:
Update the product page and marketing channels to prominently display the limited stock
of 120 units, emphasizing scarcity to create urgency. Include messaging that explains the
availability of pre-orders for the next batch to capture demand beyond current inventory
without overselling.

Expected customer impact:
Customers feel urgency to purchase now due to limited stock, while also having the option
to pre-order, which maintains trust by avoiding overselling and ensuring transparency
about availability.

Risk note:
Ensure messaging is clear and honest to avoid confusion or frustration; inaccurate stock
information or unclear pre-order terms could harm trust.

Winning scores: stock_safety=9, margin_protection=8, execution_clarity=9, risk=3,
total=23

```

The implementation uses two search levels. `generate_strategies` creates the first set of branches. Each strategy is then scored in parallel using the same evaluation method and `select_frontier` keeps only the best `TOP_K` branches. The workflow then develops each surviving strategy by adding one concrete action. A second evaluation compares these expanded branches and selects the final one. Reducers combine the results produced by the parallel workers. The final output shows the original alternatives, which branches were removed, and which path was selected. More precisely, this is a limited top-k Tree-of-Thought-style search. It does not include backtracking, repeated review of the frontier, best-first expansion, or the selection and backpropagation mechanisms used in MCTS. It is important to describe the search method accurately. The graph demonstrates one controlled branching-and-pruning pattern. It is not a complete, general-purpose tree-search engine. The evaluation criteria also change between the first and second search levels. This can be reasonable when the two stages evaluate different decisions. The first stage may measure whether a strategy is suitable, while the second may measure the quality of its implementation. However, this

means that the search objective changes between the two stages. The evaluation rules and score weights must therefore be tested together using representative business decisions. Otherwise, a branch may survive the first evaluation and win the second even though the overall scoring method does not reflect the real priorities of the business. The output also shows that the two expanded branches receive the same total score. The example selects the first branch in the resulting order, but a production workflow should use a clear secondary rule. In this scenario, a reasonable rule would first compare stock safety, then risk, and finally margin protection. If the branches are still equal, the system could keep both recommendations, request a deterministic business decision, or escalate the choice to a human instead of selecting one without a valid reason.

4.2.4 Search control is the real engineering problem

The main engineering risk in search-based reasoning is that the number of possible branches can grow very quickly. If every state creates k child states and the search continues for d levels, the deepest level contains k^d states. When k is greater than 1, the complete tree contains $(k^{d+1} - 1) / (k - 1)$ states. For example, with a branching factor of 3 and a depth of 5, the deepest level contains 243 states, while the complete tree contains 364 states. A production system should therefore limit the branching factor, search depth, total number of candidates, model calls, tokens, concurrent requests, total execution time, and calls to external tools. The workflow should also support several stop conditions. It may stop because a candidate has reached the required acceptance score, because further expansion is no longer producing useful improvements, because the available budget has been exhausted, or because the system must return a safe fallback. An evaluator score can help the workflow decide which branch to follow, but a high score or high evaluator confidence does not prove that the result is correct. In LangGraph, `recursion_limit` limits the number of graph supersteps. If the workflow does not reach a stopping point before this limit, LangGraph raises `GraphRecursionError`. `RemainingSteps` and `config["metadata"]["langgraph_step"]` allow the graph to detect that it is approaching the recursion limit and move toward completion before the error occurs. However, these controls do not limit the number of parallel branches, model calls, tokens, or costs generated within one superstep. A search graph must therefore maintain separate counters and budgets for these resources at the application level. When the number of remaining supersteps or one of the application budgets approaches its limit, the graph should move to a final node. That node should return the best validated candidate found so far. When no candidate reaches the required quality level, the system should refuse to choose one and escalate the case safely.

4.2.5 Bounded sequential search with `RemainingSteps`

The earlier Best-of-N example generates several complete candidates together, evaluates them independently, and then ranks the complete set. A search workflow can also generate and evaluate candidates sequentially. After each evaluation, the graph keeps the best candidate found so far and decides whether another attempt is still permitted. Sequential generation is useful when the workflow should not create every planned candidate in advance. For example, the application may need to remain within a strict graph-step budget, stop when an application-level attempt limit is reached, or preserve the strongest result available when

execution must finish. The search is still bounded because the workflow defines both how many attempts are permitted and how it must terminate. The following example generates one customer-support reply at a time. A second model call evaluates the reply against policy fit, clarity, and risk. The workflow stores the candidate only when its score is higher than the best score already recorded. After each evaluation, conditional routing decides whether to generate another candidate or move to the final node. The graph uses two separate stopping controls. The attempt counter limits the search to four generated candidates. `RemainingSteps` allows the workflow to detect that it is approaching LangGraph's recursion limit and move toward completion before the graph exhausts its available supersteps. These controls serve different purposes. The attempt counter limits the number of candidates generated by this application, while `RemainingSteps` reflects the graph execution budget. Neither control directly limits tokens, latency, monetary cost, parallel model calls, or external tool calls. Production systems should track those resources separately.

```
from __future__ import annotations
import operator
from typing import Annotated
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from langgraph.managed import RemainingSteps
from .config import OPENAI_API_KEY

POLICY = """
Support policy summary:
- Do not promise remedies not explicitly supported by policy and facts.
- If the item appears defective, verification may be required.
- If the purchase was a gift, do not assume refund-to-recipient is allowed.
- If uncertainty remains, ask a clarifying question or escalate.
""".strip()

class CandidateReply(BaseModel):
    id: str
    reply: str
    follow_up: str

class ReplyScore(BaseModel):
    policy_fit: int = Field(ge = 1, le = 10)
    clarity: int = Field(ge = 1, le = 10)
    risk: int = Field(ge = 1, le = 10)
    rationale: str

class LoopState(TypedDict):
    customer_case: str
    attempt: int
    latest_candidate: dict
    best_candidate: dict
    best_score: int
    log: Annotated[list[str], operator.add]
    remaining_steps: RemainingSteps
    final_answer: str

def build_model(temperature: float = 0.2) -> ChatOpenAI:
    if not OPENAI_API_KEY:
        raise RuntimeError("OPENAI_API_KEY is not set.")
    return ChatOpenAI(model = "gpt-4.1-mini", temperature = temperature)

def total_score(policy_fit: int, clarity: int, risk: int) -> int:
    return (2 * policy_fit) + clarity - risk

def generate_candidate(state: LoopState) -> dict:
    next_attempt = state["attempt"] + 1

    generator = build_model(0.5).with_structured_output(
        CandidateReply,
        method = "json_schema"
    )

    prompt = f"""
```

```

You are generating one support reply candidate.

{POLICY}

Customer case:
{state["customer_case"]}

This is attempt number {next_attempt}.
Generate one candidate reply that is materially different from earlier attempts.
If there is uncertainty, the reply may ask a clarifying question instead of promising a remedy.
"".strip()

candidate = generator.invoke(prompt)

return {
    "attempt": next_attempt,
    "latest_candidate": candidate.model_dump(),
    "log": [f"generated attempt {next_attempt}: {candidate.id}"]
}

def score_candidate(state: LoopState) -> dict:
    candidate = CandidateReply.model_validate(state["latest_candidate"])

    judge = build_model(0.0).with_structured_output(
        ReplyScore,
        method = "json_schema"
    )

    prompt = f"""
You are evaluating one support reply.

{POLICY}

Customer case:
{state["customer_case"]}

Candidate:
id: {candidate.id}
reply: {candidate.reply}
follow_up: {candidate.follow_up}

Score it on:
- policy_fit,
- clarity,
- risk.

Be strict.
"".strip()

judgment = judge.invoke(prompt)
score = total_score(judgment.policy_fit, judgment.clarity, judgment.risk)

updates = {
    "log": [
        (
            f"scored attempt {state['attempt']}: "
            f"policy_fit={judgment.policy_fit}, clarity={judgment.clarity}, "
            f"risk={judgment.risk}, total={score},
            remaining_steps={state['remaining_steps']}"
        )
    ]
}

if score > state["best_score"]:
    updates["best_score"] = score
    updates["best_candidate"] = {
        **candidate.model_dump(),
        **judgment.model_dump(),
        "total_score": score
    }
    updates["log"].append(f"new best candidate selected at attempt {state['attempt']}")

return updates

def route_after_scoring(state: LoopState):
    if state["remaining_steps"] <= 2:

```

```

        return "finish"
    if state["attempt"] >= 4:
        return "finish"
    return "generate candidate"

def finish(state: LoopState) -> dict:
    best = state["best_candidate"]
    stop_reason = (
        "step budget nearly exhausted"
        if state["remaining_steps"] <= 2
        else "maximum planned attempts reached"
    )

    answer = "\n".join(
        [
            f"Stop reason: {stop_reason}",
            f"Best attempt total: {best['total_score']}",
            "",
            "Reply:",
            best["reply"],
            "",
            "Follow-up:",
            best["follow_up"]
        ]
    )
    return {"final_answer": answer}

builder = StateGraph(LoopState)
builder.add_node("generate_candidate", generate_candidate)
builder.add_node("score_candidate", score_candidate)
builder.add_node("finish", finish)

builder.add_edge(START, "generate_candidate")
builder.add_edge("generate_candidate", "score_candidate")
builder.add_conditional_edges("score_candidate", route_after_scoring)
builder.add_edge("finish", END)

graph = builder.compile()

if __name__ == "__main__":
    customer_case = """
    The customer says the blender was purchased as a gift 28 days ago.
    The motor now stops after a few seconds. They want a guaranteed cash refund today.
    """

    result = graph.invoke(
        {
            "customer_case": customer_case,
            "attempt": 0,
            "latest_candidate": {},
            "best_candidate": {},
            "best_score": -999,
            "log": []
        },
        config = {"recursion_limit": 8}
    )

    print("Execution log:")
    for line in result["log"]:
        print("-", line)

    print("\nFinal answer:\n")
    print(result["final_answer"])

```

vol3_ch_04\src\search_control.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.search_control

Execution log:
- generated attempt 1: 1
- scored attempt 1: policy_fit=5, clarity=5, risk=1, total=14, remaining_steps=6
- new best candidate selected at attempt 1
- generated attempt 2: 1
- scored attempt 2: policy_fit=10, clarity=9, risk=1, total=28, remaining_steps=4
- new best candidate selected at attempt 2
- generated attempt 3: 1
- scored attempt 3: policy_fit=5, clarity=5, risk=1, total=14, remaining_steps=2

```



```
Final answer:
```

```
Stop reason: step budget nearly exhausted  
Best attempt total: 28
```

```
Reply:
```

```
Thank you for reaching out about the blender issue. Since the blender was purchased as a gift 28 days ago and is now experiencing motor problems, we would like to assist you further. To proceed, could you please confirm whether you or the gift recipient made the purchase, and if you have the original receipt or proof of purchase? This information will help us determine the best way to support you within our policy guidelines.
```

```
Follow-up:
```

```
Could you please provide the purchase details and clarify who the original purchaser was?
```

This example demonstrates sequential Best-of-N search with explicit runtime control. It differs from the earlier Best-of-N example in how the candidates are scheduled. The earlier graph creates several candidates as one group and evaluates them through separate worker paths. This graph creates one candidate, evaluates it, updates the best result, and then decides whether another attempt is allowed. The generator uses a higher temperature than the judge so that successive candidates have some variation. It returns a structured `CandidateReply` containing an identifier, a reply, and a follow-up action. The judge returns a structured `ReplyScore` containing policy-fit, clarity, risk, and a rationale. The application then calculates the total score as twice the policy-fit score plus the clarity score minus the risk score. This weighting makes policy compliance more important than presentation quality and penalizes replies that create additional risk. The graph does not assume that the most recent candidate is the best one. After each evaluation, it compares the new total with `best_score`. When the new candidate scores higher, the workflow stores the candidate, its evaluation, and its total score in `best_candidate`. If the new score is equal to or lower than the current best score, the stored result remains unchanged. This is a best-so-far policy rather than a revision loop. The next candidate does not receive the judge's rationale and does not directly improve the previous answer. The routing function applies two stopping conditions. First, the workflow stops when four candidates have been generated. Second, it stops when `RemainingSteps` indicates that only a small graph-step budget remains. In either case, the graph moves to finish and returns the strongest candidate stored so far. This provides graceful completion instead of allowing the workflow to continue until LangGraph raises a recursion-limit error. `RemainingSteps` must be interpreted carefully. It represents the approximate number of graph supersteps still available under the current recursion limit. It does not represent the number of candidates, model invocations, tokens, tool calls, or parallel operations that remain. In this example, the attempt counter controls candidate generation, while `RemainingSteps` protects the graph-level execution path. A production search workflow should also maintain explicit limits for model calls, token usage, elapsed time, external requests, and total cost. This workflow is still a form of Best-of-N search. The candidates are complete replies, they are evaluated independently, and the highest-scoring reply is returned. The graph does not maintain a tree of partial states, expand parent-child branches, backtrack to earlier decisions, or perform MCTS-style selection, simulation, and backpropagation. Its purpose is narrower: to show how a LangGraph application can perform incremental candidate search, preserve the best result found so far, and terminate safely when either an application limit or a graph execution limit is approaching.

The example also inherits the normal limitations of model-based selection. The generator and judge may share similar weaknesses, and a high model-generated score does not prove that the selected reply follows the real policy. The policy summary in this example is intentionally small and is passed directly to both models. In a production support system, exact eligibility rules should be enforced deterministically where possible, policy evidence should come from an authoritative source, and high-risk or ambiguous cases should be rejected, clarified, or escalated rather than accepted only because one candidate received the highest score.

4.2.6 MCTS-style variants

MCTS-style reasoning uses the same general idea as Tree-of-Thought: the system searches through states and possible actions. The main difference is how it decides where to spend its processing effort. Classical Monte Carlo Tree Search repeats four operations. Selection chooses a branch by balancing its estimated value with how little it has been explored. Expansion adds one or more new states. Simulation, also called rollout, estimates what may happen if the branch continues. Backpropagation sends the observed or estimated result back through the earlier states in that branch, so later selections can use the accumulated information. Consider a customer-support troubleshooting assistant with three possible diagnostic actions. The controller may first choose the action with the best balance between expected value and uncertainty. It then expands that action into a new diagnostic state, performs a safe read-only check or simulated continuation, assigns a value to the result, and sends that value back to the earlier states. During the next iteration, the controller may return to the same path because it produced a good result. It may instead explore a less-tested path because its outcome is still uncertain. Unlike a fixed `top-k` search, the system changes how it uses its processing budget as new evidence becomes available. LLM-based variants often replace traditional environment simulations with model-generated continuations, learned or LLM-based value estimates, task-specific rewards, and feedback from external systems. For this reason, the term MCTS-style is appropriate when an implementation uses only some parts of the classical algorithm. Reasoning via Planning, or RAP, uses a language model both as a reasoning agent and as a model of the environment. It applies MCTS using reward signals designed for the specific task. Language Agent Tree Search, or LATS, combines MCTS with language-model value functions, self-reflection, and feedback from an external environment. This allows reasoning, action, and planning to influence one another.

An MCTS-style design is appropriate only when the task has delayed effects that simple early scoring cannot reveal, the workflow has a reliable reward or value signal, the state can be copied or simulated safely, and the expected improvement is worth the additional cost and complexity. Speculative branches must never perform irreversible actions in the live environment. Rollouts should use a simulator, sandbox, reproducible snapshot, copied state, read-only tools, or proposed plans that must be validated or approved before execution. When the environment cannot be reproduced safely or the value signal is weak, a limited Tree-of-Thought or Best-of-N workflow is usually the safer choice.

4.2.7 Search in LangGraph: one graph, several search regimes

These search patterns can use the same general graph structure, but the state and routing logic become more complex as the search method becomes more advanced. A graph with one generation path, no parallel branches, and no return to earlier states behaves like a simple

linear workflow. When the graph generates several complete candidates independently and then uses one selector to choose among them, it implements Best-of-N. A Tree-of-Thought-style workflow requires more information. The graph must maintain a set of active branches, record parent-child relationships, score the branches, remove weak ones, and continue expanding the strongest ones. An MCTS-style controller requires even more state. It must track how often each branch has been visited, the total value collected by each branch, whether a state is terminal, its exploration score, which branch should be selected next, the results of rollouts, and how those results are passed back to earlier states. When branches interact with an external environment, the workflow also needs a safe way to test possible actions without changing the live system. This may require reproducible environment states, snapshots, copied state, or another controlled mechanism for evaluating speculative actions. LangGraph provides the building blocks for orchestration, but it does not provide a ready-made search algorithm. Send can create a variable number of worker tasks while the graph is running. Reducers can combine the results produced by those workers. Conditional edges or Command can route execution according to the current search state. Checkpoints can preserve the state of a long-running search when durability is needed. The evaluator-optimizer pattern provides a useful loop in which the system generates a result, evaluates it, and improves it. However, this pattern is not the same as tree search or MCTS unless the graph also includes explicit frontier management and the required search policy. Making these elements explicit allows the application to store candidate states, evaluations, pruning decisions, and routing choices. This makes the workflow-level search easier to test and diagnose than a process controlled only through prompts. However, the model's private internal reasoning still remains hidden.

4.2.8 When search helps, and when it hurts

Search is useful only when the system spends extra computation and also has a reliable way to tell better states from worse ones. Best-of-N is suitable when the system can generate several complete outputs independently and a reliable selector can compare them. Self-consistency is suitable when different reasoning paths should lead to the same normalized answer. Tree-of-Thought is suitable when the system must evaluate, remove, or revisit partial decisions before producing a complete answer. MCTS-style control is suitable when the value of a branch becomes clear only later and the workflow has a safe, reproducible environment or another reliable way to perform rollouts and estimate value. Search is usually unnecessary when the answer should come from a deterministic business rule, an exact calculation, direct retrieval, or a constrained tool call. In these cases, code, databases, retrievers, or calculators usually provide stronger guarantees at a lower cost. Ensembles may still be useful for uncertain classification or ranking tasks, but only when evaluation shows that they produce a meaningful improvement. The two main risks are highly similar candidates and unreliable evaluation. Candidates created with the same model, prompt, evidence, and assumptions may repeat the same mistake using different words. A judge from the same model family may share the same weaknesses. It may also be affected by the order of the candidates, their writing style, or a preference for answers similar to its own output. For example, a judge that gives too much importance to polished language may select a confident but unsupported answer instead of a cautious and factually correct one. When the scoring model is imperfect, increasing the number of candidates can eventually cause the system to exploit weaknesses in the evaluator instead of improving the real quality of the answer. Every search design should therefore be

tested from start to finish on held-out examples. It should also be compared with a single-pass baseline by considering quality, latency, and cost together.

4.2.9 A practical escalation path

Treat search patterns as a sequence of evidence-based engineering decisions rather than a ladder of increasingly intelligent techniques. Start with the simplest workflow and introduce additional search control only when evaluation shows that a specific failure mode remains.

Pattern	Search unit	Decision signal	Use it when	Do not escalate further when
Linear single-path workflow	One solution path	Validation of the final result	Retrieval, deterministic logic, or one model call already performs reliably	The measured quality target is already met
Self-consistency	Multiple sampled reasoning paths ending in normalized answers	Agreement among final answers	Different valid reasoning paths should converge on one answer	Additional samples do not improve accuracy enough to justify their cost
Best-of-N	Multiple complete outputs	Deterministic verification, scoring, comparison, or reranking	The main uncertainty is which finished answer is strongest	The selector is unreliable or the candidates remain too correlated
Tree-of-Thought	Partial plans, hypotheses, or solution states	Frontier scoring and pruning	Early choices materially affect later quality and can be evaluated before completion	Partial states cannot be scored credibly
MCTS-style search	States and trajectories revisited over multiple iterations	Accumulated value, visits, rollouts, rewards, or environment feedback	Branch value is delayed and adaptive revisiting outperforms shallow search	The environment cannot be simulated safely or the value signal is weak

Each escalation should be justified by evaluation against the simpler baseline. Compare task success, constraint compliance, failure severity, latency, model and tool calls, token consumption, and total cost. More search is useful only when the measured improvement is large enough to justify the additional operational complexity.

4.3 USING EXPLICIT FEEDBACK FOR CONTROLLED IMPROVEMENT

When people first hear “learning from feedback,” they often imagine a system changing its model automatically after every good or bad answer. That is not the right mental model for most application teams. In a practical LangChain or LangGraph application, explicit feedback usually begins as an engineering signal rather than automatic retraining. A user gives a rating, selects a reason label, or a reviewer assesses an answer against a rubric. The application stores that judgment, links it to the relevant run, and uses reviewed cases to guide later evaluation and improvement. A star click does not improve the system by itself. Improvement happens only when feedback is captured, interpreted, validated where necessary, and converted into a controlled change, such as a regression test, prompt revision, routing rule, or, through a separate training process, fine-tuning data. This distinction prevents two common errors. The first is overclaiming: a low rating is not a corrected answer or an approved policy change. The second is overreacting: one frustrated comment should not immediately alter the assistant’s behaviour. Raw feedback is useful evidence, but it may be incomplete, inconsistent, or biased. The system should therefore keep raw signals separate from reviewed findings and from the controlled changes produced from those findings.

4.3.1 Ratings, labels, and human review are not the same thing

A rating is the lightest form of explicit feedback. It records whether an interaction felt helpful or unhelpful from the user’s point of view and can provide broad coverage with little effort. Aggregated ratings may reveal that one prompt version performs worse than another or that a particular category of support question produces unusual dissatisfaction. However, a score usually indicates only that the user liked or disliked the result, not why. Ratings may also be affected by selection bias because only some users provide them, and users with unusually positive or negative experiences may be more likely to respond. Treat ratings as trend signals rather than ground truth.

Labels make feedback more diagnostic. A label such as `incorrect_policy`, `missing_condition`, `unclear`, or `unsafe_action` turns a general complaint into a category that the team can aggregate and analyse. The team can then count recurring failures, filter cases by severity, compare versions, and decide which cases deserve review first. Labels are useful only when their meanings are clearly defined and applied consistently. Otherwise, they create false precision by assigning different situations to the same category or allowing different reviewers to interpret the same label in different ways. Human review provides the richest contextual judgment of the three, but it should not automatically be treated as ground truth. A reviewer can determine whether a complaint is valid, correct an inaccurate label, assess the severity of a failure, and decide whether the case should become a regression example or a candidate for further action. Human review is slower and more expensive than ratings or labels, and reviewers may disagree or make mistakes. Use clear rubrics, reviewer training, and periodic calibration when review decisions have important consequences. Ratings provide breadth, labels provide structure, and human review provides contextual judgment. A system should combine them according to its risk, cost, and quality requirements.

4.3.2 A simple architecture you can implement yourself

You can implement the core feedback pattern without a dedicated feedback platform. The design has five stages. First, identify the response and the execution that produced it. Every answer should have a stable response identifier linked to the run that generated it. Feedback should also state what it targets: the complete response or a specific step such as retrieval, tool selection, or generation. Store the prompt, model deployment, retriever, tool configuration, and policy versions needed to reproduce the behaviour.

Second, preserve the original feedback event. Record when it was submitted, which response or run it targets, who or what supplied it, and any rating, label, correction, or comment. Do not overwrite the original event when a reviewer later changes its label or interpretation. Store the review as a separate record. When comments may contain personal or sensitive information, apply appropriate redaction, access-control, and retention rules.

Third, decide which cases need attention. High-severity labels and very low ratings should normally enter review immediately. Mildly negative cases may be monitored or sampled. The review sample should also include ordinary and highly rated interactions because a positive rating does not prove that an answer was factually or procedurally correct. Without this broader sampling, the team sees only cases already suspected of failure.

Fourth, use human review selectively. Expert attention should focus on unsafe actions, policy-sensitive failures, important workflows, recurring patterns, ambiguous cases, and comparisons between candidate versions. Lower-risk cases can be sampled or analysed automatically.

Finally, keep three layers separate. Raw feedback records what a user or evaluator reported. A review decision records how that feedback was assessed. An approved improvement artefact is a separate output, such as a regression example, evaluator, prompt revision, routing rule, or authorised policy update. A reviewer-confirmed finding may justify creating such an artefact, but it should not become policy automatically. The complete lifecycle is:

Production interaction → feedback event → triage → review → approved dataset example or system change → offline evaluation → deployment → monitoring.

This process is safer than allowing unreviewed reactions to alter application behaviour directly. It also creates an auditable path from the original signal to the eventual system change.

4.3.3 A minimal feedback pipeline

The following example uses the support-assistant scenario. It records user feedback, sends severe cases to a review queue, and stores a reviewer-confirmed finding only after review. The example is intentionally minimal and illustrates the separation between collection, triage, review, and preparation for controlled reuse. It does not implement the later approval process that would convert a confirmed finding into a regression example, evaluator, prompt revision, routing rule, or policy change. A production implementation would also require durable storage, access control, provenance, privacy controls, failure handling, and operational monitoring.

```
from dataclasses import dataclass, asdict
```

```

from typing import Optional

RAW_FEEDBACK = []
REVIEW_QUEUE = []
VALIDATED_LESSONS = []
ACCEPTED_CASES = []

@dataclass
class AnswerRun:
    run_id: str
    question: str
    answer: str
    prompt_version: str

@dataclass
class Feedback:
    run_id: str
    rating: int
    label: str
    comment: str

@dataclass
class ReviewDecision:
    run_id: str
    decision: str # "confirm" or "dismiss"
    corrected_label: Optional[str]
    reviewer_note: str
    lesson: str = ""

def submit_feedback(run: AnswerRun, rating: int, label: str, comment: str) -> Feedback:
    feedback = Feedback(
        run_id = run.run_id,
        rating = rating,
        label = label,
        comment = comment
    )
    RAW_FEEDBACK.append(asdict(feedback))

    severe_labels = {"incorrect_policy", "unsafe_action"}

    if rating <= 2 or label in severe_labels:
        REVIEW_QUEUE.append(
            {
                "run_id": run.run_id,
                "question": run.question,
                "answer": run.answer,
                "prompt_version": run.prompt_version,
                "rating": rating,
                "label": label,
                "comment": comment,
                "status": "queued"
            }
        )
    else:
        ACCEPTED_CASES.append(
            {
                "run_id": run.run_id,
                "rating": rating,
                "label": label
            }
        )

    return feedback

def review_case(decision: ReviewDecision) -> None:
    for item in REVIEW_QUEUE:
        if item["run_id"] == decision.run_id:
            if decision.decision == "confirm":
                item["status"] = "confirmed"
                item["validated_label"] = decision.corrected_label or item["label"]
                item["reviewer_note"] = decision.reviewer_note

                if decision.lesson.strip():
                    VALIDATED_LESSONS.append(
                        {
                            "run_id": decision.run_id,
                            "label": item["validated_label"],
                            "lesson": decision.lesson.strip()
                        }
                    )
            else:
                item["status"] = "dismissed"

```



```

        item["reviewer_note"] = decision.reviewer_note
        return

    raise ValueError(f"Run {decision.run_id} was not found in the review queue.")

if __name__ == "__main__":
    bad_run = AnswerRun(
        run_id = "run-001",
        question = "Can a gift return be refunded to the recipient's card?",
        answer = (
            "Yes. Gift returns can be refunded directly to the recipient's card "
            "within 30 days."
        ),
        prompt_version = "support-prompt-v3"
    )

    good_run = AnswerRun(
        run_id = "run-002",
        question = "What usually happens with a normal gift return?",
        answer = (
            "Gift returns are usually handled as store credit when the policy "
            "allows returns."
        ),
        prompt_version = "support-prompt-v3"
    )

    submit_feedback(
        bad_run,
        rating = 1,
        label = "incorrect_policy",
        comment = (
            "The answer promised a cash refund, but gift returns should be "
            "store credit."
        )
    )

    submit_feedback(
        good_run,
        rating = 5,
        label = "good",
        comment = "Clear and helpful."
    )

    review_case(
        ReviewDecision(
            run_id = "run-001",
            decision = "confirm",
            corrected_label = "incorrect_policy",
            reviewer_note = "Confirmed. The refund path is wrong.",
            lesson = (
                "Do not promise cash refunds for gift returns unless the policy "
                "text states that explicitly."
            )
        )
    )

    print("RAW FEEDBACK")
    for item in RAW_FEEDBACK:
        print(item)

    print("\nREVIEW QUEUE")
    for item in REVIEW_QUEUE:
        print(item)

    print("\nACCEPTED CASES")
    for item in ACCEPTED_CASES:
        print(item)

    print("\nVALIDATED LESSONS")
    for item in VALIDATED_LESSONS:
        print(item)

```

vol3_ch_04\src\feedback_pipeline.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.feedback_pipeline

RAW FEEDBACK
{'run_id': 'run-001', 'rating': 1, 'label': 'incorrect_policy', 'comment': 'The answer promised a cash refund, but gift returns should be store credit.'}
{'run_id': 'run-002', 'rating': 5, 'label': 'good', 'comment': 'Clear and helpful.'}

```

```

REVIEW_QUEUE
{'run_id': 'run-001', 'question': "Can a gift return be refunded to the recipient's card?", 'answer': "Yes. Gift returns can be refunded directly to the recipient's card within 30 days.", 'prompt version': 'support-prompt-v3', 'rating': 1, 'label': 'incorrect_policy', 'comment': 'The answer promised a cash refund, but gift returns should be store credit.', 'status': 'confirmed', 'validated_label': 'incorrect_policy', 'reviewer_note': 'Confirmed. The refund path is wrong.'}

ACCEPTED_CASES
{'run_id': 'run-002', 'rating': 5, 'label': 'good'}

VALIDATED_LESSONS
{'run_id': 'run-001', 'label': 'incorrect_policy', 'lesson': 'Do not promise cash refunds for gift returns unless the policy text states that explicitly.'}

```

The example maps each stage to a separate data structure. `RAW_FEEDBACK` preserves the original user signal. `REVIEW_QUEUE` contains cases selected for assessment. `ACCEPTED_CASES` records interactions that do not require immediate review without treating them as proven correct. `VALIDATED_LESSONS` contains reviewer-confirmed findings that may later become candidates for controlled reuse. This structure allows the team to compare prompt versions, count recurring labels such as `incorrect_policy`, and convert confirmed failures into regression examples. A confirmed finding is not yet an approved policy or runtime instruction. It must first pass the organisation's approval and evaluation process before it changes application behaviour.

4.3.4 Pairwise review for comparing versions

The previous example reviews one answer in isolation. When the goal is to choose between two application versions, comparing their answers directly is often more useful than scoring each answer separately. In pairwise review, two variants receive the same input and are compared using one defined criterion, such as policy correctness, safety, clarity, or completeness. A reviewer may find it easier to select the better answer than to decide whether each answer deserves an absolute score of 3 or 4. The comparison should allow a tie and should not reveal which answer came from the existing system or the candidate version. Presentation order can influence the result, so the application should randomise which variant appears on the left or right. When an important decision depends on the comparison, assign some examples to multiple reviewers and measure how often their judgments agree. The following example shows a small self-managed pairwise review record.

```

from dataclasses import dataclass

PAIRWISE_REVIEWS = []

@dataclass
class CandidateAnswer:
    candidate_id: str
    run_id: str
    answer: str
    variant: str

def record_pairwise_review(
    question: str,
    left: CandidateAnswer,
    right: CandidateAnswer,
    winner: str,          # "left", "right", or "tie"
    criterion: str,
    reviewer_note: str,
) -> None:
    if winner not in {"left", "right", "tie"}:
        raise ValueError("winner must be 'left', 'right', or 'tie'")

    PAIRWISE_REVIEWS.append(
        {
            "question": question,

```

```

        "left_variant": left.variant,
        "right_variant": right.variant,
        "winner": winner,
        "criterion": criterion,
        "reviewer_note": reviewer_note
    }
)
if __name__ == "__main__":
    left = CandidateAnswer(
        candidate_id = "cand-a",
        run_id = "run-a1",
        answer = "Gift returns are always refunded to the original payment card.",
        variant = "prompt-v4"
    )
    right = CandidateAnswer(
        candidate_id = "cand-b",
        run_id = "run-b1",
        answer = (
            "Gift returns are usually handled as store credit unless the policy "
            "text states otherwise."
        ),
        variant = "prompt-v5"
    )
    record_pairwise_review(
        question = "Can a gift return be refunded to the recipient's card?",
        left = left,
        right = right,
        winner = "right",
        criterion = "policy_correctness",
        reviewer_note = (
            "The right answer is more careful and does not promise a refund "
            "path without policy support."
        )
    )
print(PAIRWISE_REVIEWS[0])

```

vol3_ch_04\src\pairwise_review.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.pairwise_review
{'question': 'Can a gift return be refunded to the recipient's card?', 'left_variant': 'prompt-v4', 'right_variant': 'prompt-v5', 'winner': 'right', 'criterion': 'policy_correctness', 'reviewer_note': 'The right answer is more careful and does not promise a refund path without policy support.'}

```

This pattern turns feedback into a controlled comparison rather than an isolated opinion. In the example, prompt-v5 is preferred only for policy_correctness on the question being reviewed. The result does not prove that it is clearer, faster, less expensive, or better for every other question. A reliable version decision requires repeated comparisons across a representative dataset. Store the input, criterion, variant identifiers, selected winner, reviewer or feedback source, rubric version, presentation order, timestamp, and review notes. This information allows the team to aggregate the results, investigate disagreement, and reproduce the comparison later.

4.3.5 Where LangGraph and LangSmith fit

LangGraph and LangSmith support different parts of the feedback lifecycle. LangGraph controls what the live workflow does next. For example, an evaluator-optimizer workflow can generate an answer, evaluate it against defined criteria, and request another attempt when the answer is not acceptable. The evaluator may be deterministic code, another model, or a human decision, depending on the criterion. This is a bounded refinement loop for the current execution. It does not create long-term learning unless the application deliberately stores and reuses the result. LangGraph interrupts support human-in-the-loop decisions. An interrupt

pauses the graph, persists its state through a checkpoint, and resumes when the application supplies external input using `Command(resume=...)`. The resumed execution must use the same thread identifier. In production, the graph should use a durable checkpoint rather than an in-memory implementation. When execution resumes, the interrupted node starts again from its beginning, so any side effects performed before `interrupt()` must be safe to repeat or moved into a separate node.

LangSmith records and evaluates what happened. Feedback can be attached to a complete trace or to an intermediate run such as retrieval, tool selection, or generation. It normally contains a criterion name and a numerical or categorical result, and it may also include a comment, correction, and information about who or what produced the feedback. Annotation queues can send selected runs to reviewers using a defined rubric. In single-run review, corrected cases can be added to datasets for offline evaluation and regression testing. Pairwise annotation queues instead compare corresponding runs from two experiments executed over a dataset. A common architecture uses both tools. LangGraph decides when a live workflow should branch, pause, retry, or request approval. LangSmith captures the trace, collects feedback, supports review, and measures whether later application versions perform better. Authoritative policies and operational data remain in the organisation's governed systems of record.

4.3.6 Using reviewed feedback to guide application decisions

Collecting feedback does not improve an application by itself. Improvement begins when reviewed evidence changes a controlled and testable decision. One practical option is to build a feedback-derived risk index that influences routing without changing the model's weights. For example, repeated confirmed failures involving gift-return questions may cause future requests in that topic to use stricter evidence checks or human review. Lower-risk topics can continue through the normal path. This is application-level adaptation, not model training. The following example calculates a simple topic score from reviewer-confirmed cases and uses it to select one of three routes. The score is a teaching device, not a production risk model. A production implementation should consider:

- the total number of interactions for each topic;
- the confirmed failure rate rather than only the number of failures;
- failure severity and recency;
- reviewer confidence and agreement;
- the quality of the evidence available for the current request;
- the business or policy criticality of the topic;
- the prompt, model, retriever, tool, and policy versions involved.

The calculation should also account for review exposure. A topic that is reviewed more often will naturally accumulate more confirmed findings, even when its underlying failure rate has not changed. Routing thresholds should therefore be evaluated on historical cases and monitored for both unnecessary escalations and missed high-risk requests.

```
from __future__ import annotations
from collections import defaultdict
from typing import Literal
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
```

```

from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import START, END, StateGraph
from langgraph.types import Command, interrupt

VALIDATED_LESSONS = [
    {
        "run_id": "run-001",
        "topic": "gift_returns",
        "label": "incorrect_policy",
        "lesson": "Do not promise cash refunds for gift returns without explicit policy support."
    },
    {
        "run_id": "run-004",
        "topic": "gift_returns",
        "label": "missing_condition",
        "lesson": "Gift return answers must mention the store-credit condition when applicable."
    },
    {
        "run_id": "run-007",
        "topic": "gift_returns",
        "label": "incorrect_policy",
        "lesson": "Gift-return refund paths are easy to misstate and need extra care."
    },
    {
        "run_id": "run-010",
        "topic": "return_shipping",
        "label": "unclear",
        "lesson": "Return-shipping answers are sometimes vague but rarely dangerous."
    },
    # Uncomment this extra validated lesson if you want the sample
    # to exercise the cautious_answer branch too.
    # {
    #     "run_id": "run-011",
    #     "topic": "return_shipping",
    #     "label": "incorrect_policy",
    #     "lesson": "Return-shipping answers sometimes miss merchant-paid exceptions.",
    # },
]

LABEL_WEIGHTS = {
    "unsafe_action": 5,
    "incorrect_policy": 4,
    "missing_condition": 2,
    "unclear": 1,
    "good": 0
}

TOPIC_EVIDENCE = {
    "gift_returns": [
        "Most physical items can be returned within 30 days of delivery.",
        "Gift returns are eligible for store credit only unless a specific refund rule is stated."
    ],
    "return_shipping": [
        "Standard returns may require the customer to pay return shipping unless the item is defective."
    ],
    "store_hours": [
        "The Rome store opens at 09:00 on Saturday."
    ]
}

class SupportState(TypedDict):
    question: str
    topic: str

    topic_risk_index: dict[str, int]
    topic_risk_score: int
    matched_lessons: list[str]
    evidence: list[str]

    route: Literal["", "human_review", "cautious_answer", "normal_answer"]
    route_reason: str

    draft_answer: str
    final_answer: str
    review_decision: dict

def build_topic_risk_index(validated_lessons: list[dict]) -> dict[str, int]:

```

```

scores = defaultdict(int)
for item in validated_lessons:
    scores[item["topic"]] += LABEL_WEIGHTS.get(item["label"], 0)
return dict(scores)

def lessons_for_topic(topic: str) -> list[str]:
    return [item["lesson"] for item in VALIDATED_LESSONS if item["topic"] == topic]

def format_block(lines: list[str]) -> str:
    if not lines:
        return "NONE"
    return "\n".join(f"- {line}" for line in lines)

model = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

normal_prompt = ChatPromptTemplate.from_template(
    """You are a customer support assistant.

Answer the question using only the evidence below.
Be concise and direct.

Question:
{question}

Evidence:
{evidence}
"""
)

cautious_prompt = ChatPromptTemplate.from_template(
    """You are a customer support assistant.

This topic has a history of validated failures in prior reviewed cases.
Answer carefully using only the evidence below, and explicitly respect the validated
lessons.

Question:
{question}

Evidence:
{evidence}

Validated lessons:
{lessons}

Requirements:
- do not invent policy
- mention important conditions
- avoid overconfident wording when the evidence is narrow
- keep the answer concise
"""
)

normal_chain = normal_prompt | model
cautious_chain = cautious_prompt | model

def prepare_context(state: SupportState) -> dict:
    topic_risk_index = build_topic_risk_index(VALIDATED_LESSONS)
    topic_risk_score = topic_risk_index.get(state["topic"], 0)
    matched_lessons = lessons_for_topic(state["topic"])
    evidence = TOPIC_EVIDENCE.get(state["topic"], ["No evidence available."])

    return {
        "topic_risk_index": topic_risk_index,
        "topic_risk_score": topic_risk_score,
        "matched_lessons": matched_lessons,
        "evidence": evidence
    }

def choose_route(state: SupportState,
) -> Command[Literal["draft_for_review", "cautious_answer", "normal_answer"]]:
    score = state["topic_risk_score"]
    topic = state["topic"]

    if score >= 8:
        return Command(
            update = {
                "route": "human_review",
                "route_reason": (
                    f"Topic '{topic}' has risk score {score}. "
                    "Repeated validated failures make pre-answer human review
                    appropriate."
                )
            },
            goto = "draft_for_review"

```

```

    )

    if score >= 3:
        return Command(
            update = {
                "route": "cautious_answer",
                "route_reason": (
                    f"Topic '{topic}' has risk score {score}. "
                    "Past reviewed failures justify a stricter evidence-based path."
                )
            },
            goto = "cautious_answer"
        )

    return Command(
        update = {
            "route": "normal_answer",
            "route_reason": (
                f"Topic '{topic}' has risk score {score}. "
                "Validated history does not show serious recurring failures."
            )
        },
        goto = "normal_answer"
    )

def normal_answer(state: SupportState) -> dict:
    response = normal_chain.invoke(
        {
            "question": state["question"],
            "evidence": format_block(state["evidence"])
        }
    )
    return {"final_answer": response.text}

def cautious_answer(state: SupportState) -> dict:
    response = cautious_chain.invoke(
        {
            "question": state["question"],
            "evidence": format_block(state["evidence"]),
            "lessons": format_block(state["matched_lessons"])
        }
    )
    return {"final_answer": response.text}

def draft_for_review(state: SupportState) -> dict:
    response = cautious_chain.invoke(
        {
            "question": state["question"],
            "evidence": format_block(state["evidence"]),
            "lessons": format_block(state["matched_lessons"])
        }
    )
    return {"draft_answer": response.text}

def human_review(state: SupportState) -> dict:
    decision = interrupt(
        {
            "question": state["question"],
            "topic": state["topic"],
            "route": state["route"],
            "route_reason": state["route_reason"],
            "draft_answer": state["draft_answer"],
            "matched_lessons": state["matched_lessons"],
            "instruction": (
                "Review the draft. Resume with "
                "{ 'action': 'approve' } or "
                "{ 'action': 'edit', 'edited_answer': '...' }."
            )
        }
    )

    action = decision["action"]

    if action == "approve":
        return {
            "review_decision": decision,
            "final_answer": state["draft_answer"]
        }

    if action == "edit":
        return {
            "review_decision": decision,

```



```

        "final_answer": decision["edited_answer"]
    }

    return {
        "review decision": decision,
        "final_answer": "This case has been escalated for manual handling."
    }

builder = StateGraph(SupportState)

builder.add_node("prepare_context", prepare_context)
builder.add_node("choose_route", choose_route)
builder.add_node("normal_answer", normal_answer)
builder.add_node("cautious_answer", cautious_answer)
builder.add_node("draft_for_review", draft_for_review)
builder.add_node("human_review", human_review)

builder.add_edge(START, "prepare_context")
builder.add_edge("prepare_context", "choose_route")
builder.add_edge("normal_answer", END)
builder.add_edge("cautious_answer", END)
builder.add_edge("draft_for_review", "human_review")
builder.add_edge("human_review", END)

graph = builder.compile(checkpointer = InMemorySaver())

if __name__ == "__main__":
    questions = [
        {
            "topic": "gift_returns",
            "question": "Can a gift return be refunded to my card?"
        },
        {
            "topic": "return_shipping",
            "question": "Who pays return shipping for a standard item?"
        },
        {
            "topic": "store_hours",
            "question": "What time does the Rome store open on Saturday?"
        }
    ]

    print("TOPIC RISK INDEX")
    print(build_topic_risk_index(VALIDATED_LESSONS))

    print("\nRUNS")

    for i, item in enumerate(questions, start = 1):
        print("\n" + "=" * 80)
        print(f"QUESTION: {item['question']}")
        config = {"configurable": {"thread_id": f"question-{i}"}}

        result = graph.invoke(
            {
                "question": item["question"],
                "topic": item["topic"],
                "topic_risk_index": {},
                "topic_risk_score": 0,
                "matched_lessons": [],
                "evidence": [],
                "route": "",
                "route_reason": "",
                "draft_answer": "",
                "final_answer": "",
                "review_decision": {}
            },
            config = config
        )

        print(f"TOPIC: {item['topic']}")
        print(f"ROUTE: {result['route']}")
        print(f"REASON: {result['route_reason']}")

        if "__interrupt__" in result:
            payload = result["__interrupt__"][0].value
            print("\nHUMAN REVIEW PAYLOAD")
            print(payload)

            reviewed = graph.invoke(
                Command(
                    resume = {
                        "action": "edit",
                        "edited_answer": (

```

```

        "Gift returns should not be promised as cash refunds "
        "unless a specific refund rule exists. Based on the policy "
        "shown here, the safer answer is that gift returns are "
        "handled as store credit."

    )

    },
    config = config
)

print("\nFINAL ANSWER (AFTER HUMAN REVIEW)")
print(reviewed["final_answer"])
else:
    print("\nFINAL ANSWER")
    print(result["final_answer"])

```

vol3_ch_04\scr\using_validated_feedback.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.using_validated_feedback

TOPIC RISK INDEX
{'gift_returns': 10, 'return_shipping': 1}

RUNS

=====
QUESTION: Can a gift return be refunded to my card?
TOPIC: gift_returns
ROUTE: human_review
REASON: Topic 'gift_returns' has risk score 10. Repeated validated failures make pre-
answer human review appropriate.

HUMAN REVIEW PAYLOAD
{'question': 'Can a gift return be refunded to my card?', 'topic': 'gift_returns',
'route': 'human_review', 'route_reason': "Topic 'gift_returns' has risk score 10.
Repeated validated failures make pre-answer human review appropriate.", 'draft_answer':
'Gift returns are generally eligible for store credit only, unless there is a specific
refund policy stating otherwise. Therefore, a gift return is typically not refunded to
your card. Please check the exact return policy for your item to confirm.',
'matched_lessons': ['Do not promise cash refunds for gift returns without explicit policy
support.', 'Gift return answers must mention the store-credit condition when
applicable.', 'Gift-return refund paths are easy to misstate and need extra care.'],
'instruction': "Review the draft. Resume with {'action': 'approve'} or {'action': 'edit',
'edited_answer': '...'}."}

FINAL ANSWER (AFTER HUMAN REVIEW)
Gift returns should not be promised as cash refunds unless a specific refund rule exists.
Based on the policy shown here, the safer answer is that gift returns are handled as
store credit.

=====
QUESTION: Who pays return shipping for a standard item?
TOPIC: return_shipping
ROUTE: normal_answer
REASON: Topic 'return_shipping' has risk score 1. Validated history does not show serious
recurring failures.

FINAL ANSWER
The customer pays return shipping for a standard item unless it is defective.

=====
QUESTION: What time does the Rome store open on Saturday?
TOPIC: store_hours
ROUTE: normal_answer
REASON: Topic 'store_hours' has risk score 0. Validated history does not show serious
recurring failures.

FINAL ANSWER
The Rome store opens at 09:00 on Saturday.

```

This example demonstrates application-level adaptation. Reviewed history changes the route selected for a later request, but it does not change the model's weights and does not turn reviewer comments into policy. The answer must still be grounded in current authoritative evidence. LangGraph makes the routing decision visible and auditable. The application

remains responsible for storage, score calculation, thresholds, approval rules, evidence retrieval, and policy ownership. Treat these controls like other business rules: version them, test them on representative cases, record why a route was selected, and monitor whether they reduce confirmed failures without producing an excessive review workload.

4.3.7 Practical design rules

Good feedback begins with a stable vocabulary. Define the feedback schema and review rubric before collecting large amounts of data. Describe every rating range and label in plain language, with examples and counterexamples. Keep the label set small enough to apply consistently, but make sure every category has a clear meaning and leads to a defined operational response. Version the rubric whenever its definitions change. Train reviewers before they assess important cases and assign some examples to multiple reviewers so the team can measure agreement. When reviewers disagree on a high-impact case, use an adjudication process rather than selecting one judgment arbitrarily.

Review samples should contain more than reported failures. Include ordinary interactions and highly rated answers so the team can detect problems that users did not report and estimate the real failure rate more accurately. Govern how reviewed findings become system changes. Protect sensitive comments, restrict access, define retention periods, and record the provenance, scope, reviewer, rubric version, and approval status of every finding. Changes that affect policy-related guidance should require approval from the appropriate policy owner. Convert confirmed failures into scoped regression examples, evaluators, prompt revisions, routing rules, or other approved controls. Evaluate candidate changes on representative datasets before deployment and release them only when the evidence shows improvement without unacceptable regressions. Review thresholds, rubrics, and routing rules periodically because policies, models, traffic patterns, and user behaviour change over time. Explicit feedback is useful only when the meaning of each stage remains clear. A user reaction is not a confirmed failure. A confirmed failure is not an approved policy change. An application that changes its routing has not retrained its model. Keeping these boundaries explicit turns feedback from a collection of opinions into an auditable improvement process.

4.4 LEARNING FROM IMPLICIT SIGNALS (CLICKS, CONVERSIONS, TRADING OUTCOMES)

4.4.1 Learning from behavior rather than labels

Learning from implicit signals means improving a system by looking at what users or the environment actually do, rather than what they explicitly say. A click, a skipped result, a purchase, an abandoned cart, a completed trade, a realised profit, or a post-trade loss can all tell us something about whether the system's earlier decision was useful. This kind of feedback is attractive because it is continuous and abundant. Systems can collect large amounts of it during normal operation. The problem is that these signals are not clean labels. They do not directly say "this was correct" or "this was wrong." They are weak behavioural clues that must be interpreted carefully before they can be used safely. This makes implicit-signal learning different from the other adaptation mechanisms discussed so far. In the reflection material, adaptation was mainly a runtime control problem inside a single workflow execution. There, the system critiques or verifies an answer before returning it. Implicit-signal learning is different because it works across many runs over time. It is based on accumulated outcomes, not on one critique step inside one interaction.

It is also important not to confuse implicit signals with explicit feedback. Explicit feedback is deliberately provided by a user or reviewer, for example through a rating, correction, comment, or quality label. Runtime reflection is different again: the system reviews its current work before completing the interaction. Implicit signals come from observed behaviour across many interactions, such as clicks, purchases, repeated queries, or business outcomes. These signals are normally treated first as production metrics and evaluation evidence, not automatically as training labels. Offline evaluation remains important, but its results may not match production performance when users, traffic, exposure, or operating conditions differ. Production signals therefore complement offline evaluation rather than replace it.

These outcomes can support several improvement paths. Some systems turn them into rewards. Some convert them into pairwise preferences. Others use repeated bad outcomes to improve routing or critique policies. Some store stable lessons that influence future behaviour without changing model weights. In this section, however, the focus is narrower. The first engineering problem is not how to learn from implicit signals in every possible way. It is how to capture and interpret those signals well enough that they become trustworthy inputs for later improvement.

4.4.2 Clicks, conversions, and trading outcomes are different kinds of signals

Clicks, conversions, and trading results are all useful, but they describe different kinds of behaviour and should not be treated as interchangeable. A click is usually a fast but weak signal. It shows that an item attracted enough attention to be opened, but it does not prove that the item was correct, helpful, or valuable. Clicks are also affected by exposure: users can only click items they were shown, and items near the top of a list are usually seen more often. Each click should therefore be stored with the decision that produced the exposure, including the query, candidate set, item, rank position, policy version, and timestamp. This context makes the signal easier to interpret, but it does not remove the bias. Reliable policy

comparisons normally require a controlled experiment, such as an A/B test, or a statistical method that corrects for unequal exposure. An observed difference between two policies is only an association unless the evaluation design provides credible evidence that the policy caused the difference. Clicks are commonly used to improve retrieval, ranking, recommendations, or routing. They may also contribute to other forms of improvement, but only after careful attribution and validation.

Conversions are usually closer to business value than clicks. A purchase, signup, or completed action often provides stronger evidence that a recommendation was useful. However, conversions are frequently delayed. A user may click now and complete the action hours or days later. When a conversion arrives within the chosen attribution window, it can be linked to the earlier decision according to the application's attribution rules. A missing conversion must not be treated as a negative result while that window is still open, because the outcome is still unknown. Production systems must also define what happens when several searches, recommendations, or clicks could all claim the same conversion. The attribution window and attribution rule must therefore be explicit, versioned, and applied consistently.

Trading outcomes have direct financial consequences, but they are not necessarily cleaner or more reliable signals than clicks or conversions. A profitable trade does not prove that the decision process was sound, and a losing trade does not prove that it was wrong. Markets contain uncertainty, and the same decision can produce different results under different conditions. Raw profit and loss should therefore be considered together with factors such as position size, transaction costs, holding period, drawdown, volatility, market regime, and performance against an appropriate benchmark. Conclusions should also be based on enough observations to estimate uncertainty and avoid reacting to a small number of lucky or unlucky trades. A trading assistant should use realised outcomes as evidence for later evaluation and policy comparison, not as a reason to modify its behaviour immediately during a live run. Once the meaning and limitations of each signal are clear, the next question is how the application records, connects, evaluates, and reuses them.

4.4.3 What the Lang ecosystem gives you today

LangChain, LangGraph, and LangSmith provide useful building blocks for an improvement loop based on implicit signals, but they do not provide a learner that automatically improves from clicks, purchases, or trading results. LangGraph checkpoints preserve the state of a particular thread, allowing a workflow or conversation to continue across several invocations. LangGraph stores serve a different purpose: they persist JSON documents beyond one thread and organize them by namespace and key. An application can use these stores for attributed outcomes, approved lessons, aggregate summaries, or other durable records that must be reused later.

LangChain helps the application produce predictable, machine-readable records through structured output. Instead of leaving decisions, classifications, and outcome summaries inside free-form prose, the application can represent them as schema-based objects with known fields. The application should validate these objects before storing them or using them to make later decisions. LangSmith provides the observability and evaluation layer. It can trace complete runs and intermediate spans, attach feedback to the specific step being evaluated, run evaluators over production traffic, and route selected runs into annotation queues for human review. Together, these capabilities allow the application to record what happened,

connect later outcomes to earlier decisions, inspect recurring patterns, and retain conclusions that have been reviewed and approved. None of these tools defines the learning policy. They do not decide whether a click represents relevance, which recommendation deserves credit for a purchase, whether a trading loss is meaningful, or when a pattern is strong enough to justify a change. The application must define the learning policy itself. This includes what data to capture, how to attribute outcomes, how much evidence is required, who approves a change, and which component may be updated. The stack supports measurement, storage, review, and controlled adaptation. It does not perform the adaptation automatically. These capabilities provide the infrastructure; the application architecture determines how they are used safely.

4.4.4 A practical architecture for implicit-signal learning

The simplest useful architecture begins with one basic rule: every important decision must be recorded as an identifiable event. If the assistant retrieves documents, ranks products, selects an offer, recommends an action, or proposes a trade, that decision should be captured with its own ID and enough metadata to make it traceable later. At a minimum, you need to know which component made the decision, which prompt, policy, or configuration version was active, when the decision happened, and which later outcome can be linked back to it. Without that level of identity, behavioural signals lose most of their value. They become isolated anecdotes rather than usable evidence.

A second rule is just as important: do not mix different signal types into a single undifferentiated score. Clicks, conversions, and trading outcomes describe different things and usually belong to different parts of the system. A click mainly tells you that something looked relevant or interesting enough to inspect. A conversion tells you that the option eventually produced value. A trading outcome provides evidence about how an action performed under the selected return and risk measures. If these signals are collapsed too early into one metric, the architecture hides the very component boundaries the engineering team needs in order to improve the system safely.

A reliable design separates raw events from interpreted lessons. Raw events are facts: a product was shown, a result was clicked, an order was completed two days later, or a trade closed with a particular return and drawdown. A lesson is a conclusion drawn from many such events, for example that one retrieval policy may perform worse for a particular query type. The distinction matters because a signal is not yet a conclusion. Treating noisy behavioural data as an established lesson can cause the system to reinforce existing bias or react to random variation. The processing flow should remain explicit. First, record each important decision with an identifier, the responsible component, its version, the candidates considered, and the context needed for later analysis. Second, record subsequent outcomes with timestamps and attribution keys. Third, interpret each signal using the information appropriate to that signal: exposure and position for clicks, timing and attribution rules for conversions, and risk-aware measures for trading outcomes. Fourth, compare versions using a suitable experimental or statistical method. The comparison should use an adequate sample size and include an estimate of uncertainty, such as a confidence interval. Raw historical averages are not sufficient when different users, queries, or items received different exposure. Fifth, turn candidate patterns into review cases rather than immediate policy changes. Only validated conclusions should influence prompts, retrievers, ranking policies, routing rules, or stored lessons. Behavioural data must also be governed as production data. The system should collect only the signals it needs and apply the appropriate consent, access-control, retention, deletion, and audit rules.

Human review still has an essential role in this architecture, even though the signals themselves are implicit. Behavioural data gives breadth and continuous coverage, but human judgment provides interpretation. In practice, implicit signals are often most useful for showing where attention is needed, while targeted review determines what the pattern actually means. That is why even systems with rich behavioural telemetry still benefit from explicit review workflows. Implicit signals help the team find the right cases. Human evaluation helps the team decide what to change.

4.4.5 A working example

The example below uses two of the book's recurring domains. On the e-commerce side, it records retrieval exposures, recommendation clicks, and delayed conversions. On the trading side, it records trade decisions and realised outcomes. The code does not update prompts or rewrite policies. It does the smaller and more important job first: it turns raw behavioural events into review candidates that a later adaptation step could use.

```
from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime, timedelta
from collections import defaultdict

@dataclass
class Exposure:
    exposure_id: str
    run_id: str
    component: str          # "retrieval" or "recommendation"
    policy_version: str
    item_id: str
    rank: int
    shown_at: datetime

@dataclass
class ClickEvent:
    exposure_id: str
    clicked_at: datetime

@dataclass
class ConversionEvent:
    exposure_id: str
    converted_at: datetime
    order_value: float

@dataclass
class TradeDecision:
    trade_id: str
    run_id: str
    strategy_version: str
    symbol: str
    opened_at: datetime

@dataclass
class TradeOutcome:
    trade_id: str
    closed_at: datetime
    pnl: float
    max_drawdown: float

EXPOSURES: list[Exposure] = []
CLICKS: list[ClickEvent] = []
CONVERSIONS: list[ConversionEvent] = []
TRADES: list[TradeDecision] = []
OUTCOMES: list[TradeOutcome] = []

def add_exposure(
    exposure_id: str,
    run_id: str,
    component: str,
    policy_version: str,
    item_id: str,
    rank: int,
    ts: str
) -> None:
    EXPOSURES.append(
```



```

    Exposure(
        exposure_id = exposure_id,
        run_id = run_id,
        component = component,
        policy_version = policy_version,
        item_id = item_id,
        rank = rank,
        shown_at = datetime.fromisoformat(ts)
    )
)

def add_click(exposure_id: str, ts: str) -> None:
    CLICKS.append(ClickEvent(exposure_id = exposure_id,
                             clicked_at = datetime.fromisoformat(ts)))

def add_conversion(exposure_id: str, ts: str, order_value: float) -> None:
    CONVERSIONS.append(
        ConversionEvent(
            exposure_id = exposure_id,
            converted_at = datetime.fromisoformat(ts),
            order_value = order_value
        )
    )

def add_trade(trade_id: str, run_id: str, strategy_version: str, symbol: str, ts: str) -> None:
    TRADES.append(
        TradeDecision(
            trade_id = trade_id,
            run_id = run_id,
            strategy_version = strategy_version,
            symbol = symbol,
            opened_at = datetime.fromisoformat(ts)
        )
    )

def add_trade_outcome(trade_id: str, ts: str, pnl: float, max_drawdown: float) -> None:
    OUTCOMES.append(
        TradeOutcome(
            trade_id = trade_id,
            closed_at = datetime.fromisoformat(ts),
            pnl = pnl,
            max_drawdown = max_drawdown
        )
    )

def summarize_clicks() -> list[dict]:
    click_map = {event.exposure_id for event in CLICKS}
    grouped = defaultdict(lambda: {"shown": 0, "clicked": 0})

    for exposure in EXPOSURES:
        if exposure.component != "retrieval":
            continue

        key = (exposure.policy_version, exposure.rank)
        grouped[key]["shown"] += 1
        if exposure.exposure_id in click_map:
            grouped[key]["clicked"] += 1

    rows = []
    for (policy_version, rank), stats in sorted(grouped.items()):
        ctr = stats["clicked"] / stats["shown"]
        rows.append(
            {
                "component": "retrieval",
                "policy_version": policy_version,
                "rank": rank,
                "shown": stats["shown"],
                "clicked": stats["clicked"],
                "ctr": round(ctr, 3)
            }
        )
    return rows

def summarize_conversions(window_days: int = 7) -> list[dict]:
    exposure_map = {exposure.exposure_id: exposure for exposure in EXPOSURES}
    click_map = {event.exposure_id: event for event in CLICKS}
    grouped = defaultdict(lambda: {"clicked": 0, "converted": 0, "revenue": 0.0})

    for exposure_id, click in click_map.items():
        exposure = exposure_map.get(exposure_id)
        if not exposure or exposure.component != "recommendation":

```

```

        continue

    grouped[exposure.policy_version]["clicked"] += 1

for conversion in CONVERSIONS:
    click = click_map.get(conversion.exposure_id)
    exposure = exposure_map.get(conversion.exposure_id)
    if not click or not exposure or exposure.component != "recommendation":
        continue

    delay = conversion.converted_at - click.clicked_at
    if delay <= timedelta(days = window_days):
        grouped[exposure.policy_version]["converted"] += 1
        grouped[exposure.policy_version]["revenue"] += conversion.order_value

rows = []
for policy_version, stats in sorted(grouped.items()):
    clicked = stats["clicked"]
    conv_rate = (stats["converted"] / clicked) if clicked else 0.0
    rows.append(
        {
            "component": "recommendation",
            "policy_version": policy_version,
            "clicked": clicked,
            "converted_within_window": stats["converted"],
            "conversion_rate": round(conv_rate, 3),
            "attributed_revenue": round(stats["revenue"], 2)
        }
    )
return rows

def summarize_trades(risk_penalty: float = 0.5) -> list[dict]:
    trade_map = {trade.trade_id: trade for trade in TRADES}
    grouped = defaultdict(lambda: {"count": 0, "pnl": 0.0, "drawdown": 0.0,
                                     "risk_adjusted": 0.0})

    for outcome in OUTCOMES:
        trade = trade_map.get(outcome.trade_id)
        if not trade:
            continue

        grouped[trade.strategy_version]["count"] += 1
        grouped[trade.strategy_version]["pnl"] += outcome.pnl
        grouped[trade.strategy_version]["drawdown"] += outcome.max_drawdown
        grouped[trade.strategy_version]["risk_adjusted"] += outcome.pnl - (risk_penalty *
                                                                           outcome.max_drawdown)

    rows = []
    for strategy_version, stats in sorted(grouped.items()):
        count = stats["count"]
        rows.append(
            {
                "component": "trading",
                "strategy_version": strategy_version,
                "trades": count,
                "avg_pnl": round(stats["pnl"] / count, 2),
                "avg_max_drawdown": round(stats["drawdown"] / count, 2),
                "avg_risk_adjusted_score": round(stats["risk_adjusted"] / count, 2)
            }
        )
    return rows

def build_review_candidates() -> list[dict]:
    candidates = []

    for row in summarize_clicks():
        if row["rank"] == 1 and row["ctr"] < 0.2:
            candidates.append(
                {
                    "type": "retrieval_policy_review",
                    "policy_version": row["policy_version"],
                    "reason": f"Low rank-1 CTR ({row['ctr']})"
                }
            )

    for row in summarize_conversions():
        if row["clicked"] >= 2 and row["conversion_rate"] < 0.25:
            candidates.append(
                {
                    "type": "recommendation_policy_review",
                    "policy_version": row["policy_version"],
                    "reason": f"Low click-to-conversion rate ({row['conversion_rate']})"
                }
            )

```

```

    )
    )

for row in summarize_trades():
    if row["avg risk adjusted score"] < 0:
        candidates.append(
            {
                "type": "trading_strategy_review",
                "strategy_version": row["strategy_version"],
                "reason": f"Negative average risk-adjusted score
                        ({row['avg risk adjusted score']})"
            }
        )

return candidates

if __name__ == "__main__":
    # Retrieval exposures for a support or catalog assistant.
    add_exposure("exp-001", "run-101", "retrieval", "retriever-v1", "doc-11", 1, "2026-03-01T10:00:00")
    add_exposure("exp-002", "run-102", "retrieval", "retriever-v1", "doc-22", 1, "2026-03-01T10:05:00")
    add_exposure("exp-003", "run-103", "retrieval", "retriever-v2", "doc-31", 1, "2026-03-01T10:10:00")
    add_exposure("exp-004", "run-104", "retrieval", "retriever-v2", "doc-44", 2, "2026-03-01T10:15:00")

    add_click("exp-003", "2026-03-01T10:11:00")
    add_click("exp-004", "2026-03-01T10:16:00")

    # Recommendation exposures for an e-commerce assistant.
    add_exposure("exp-101", "run-301", "recommendation", "reco-v1", "sku-11", 1, "2026-03-01T12:00:00")
    add_exposure("exp-102", "run-302", "recommendation", "reco-v1", "sku-22", 1, "2026-03-01T12:05:00")
    add_exposure("exp-103", "run-303", "recommendation", "reco-v2", "sku-31", 1, "2026-03-01T12:10:00")

    add_click("exp-101", "2026-03-01T12:01:00")
    add_click("exp-102", "2026-03-01T12:06:00")
    add_click("exp-103", "2026-03-01T12:11:00")

    add_conversion("exp-103", "2026-03-03T11:00:00", 120.0)
    add_conversion("exp-102", "2026-03-20T09:00:00", 80.0) # Outside 7-day window

    # Trading assistant decisions and outcomes.
    add_trade("trade-001", "run-201", "strategy-v1", "NVDA", "2026-03-01T09:30:00")
    add_trade("trade-002", "run-202", "strategy-v1", "AAPL", "2026-03-01T11:00:00")
    add_trade("trade-003", "run-203", "strategy-v2", "MSFT", "2026-03-01T13:00:00")

    add_trade_outcome("trade-001", "2026-03-01T15:30:00", pnl=-40.0, max_drawdown=60.0)
    add_trade_outcome("trade-002", "2026-03-01T16:00:00", pnl=20.0, max_drawdown=35.0)
    add_trade_outcome("trade-003", "2026-03-01T16:00:00", pnl=35.0, max_drawdown=10.0)

    print("CLICK SUMMARY")
    for row in summarize_clicks():
        print(row)

    print("\nCONVERSION SUMMARY")
    for row in summarize_conversions(window_days = 7):
        print(row)

    print("\nTRADING SUMMARY")
    for row in summarize_trades(risk_penalty = 0.5):
        print(row)

    print("\nREVIEW CANDIDATES")
    for row in build_review_candidates():
        print(row)

```

vol3_ch_04\src\implicit_signals.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.implicit_signals

CLICK SUMMARY
{'component': 'retrieval', 'policy_version': 'retriever-v1', 'rank': 1, 'shown': 2, 'clicked': 0, 'ctr': 0.0}
{'component': 'retrieval', 'policy_version': 'retriever-v2', 'rank': 1, 'shown': 1, 'clicked': 1, 'ctr': 1.0}
{'component': 'retrieval', 'policy_version': 'retriever-v2', 'rank': 2, 'shown': 1, 'clicked': 1, 'ctr': 1.0}

CONVERSION SUMMARY

```

```
{'component': 'recommendation', 'policy_version': 'reco-v1', 'clicked': 2,
'converted_within_window': 0, 'conversion_rate': 0.0, 'attributed_revenue': 0.0}
{'component': 'recommendation', 'policy_version': 'reco-v2', 'clicked': 1,
'converted_within_window': 1, 'conversion_rate': 1.0, 'attributed_revenue': 120.0}

TRADING SUMMARY
{'component': 'trading', 'strategy_version': 'strategy-v1', 'trades': 2, 'avg_pnl': -
10.0, 'avg_max_drawdown': 47.5, 'avg_risk_adjusted_score': -33.75}
{'component': 'trading', 'strategy_version': 'strategy-v2', 'trades': 1, 'avg_pnl': 35.0,
'avg_max_drawdown': 10.0, 'avg_risk_adjusted_score': 30.0}

REVIEW CANDIDATES
{'type': 'retrieval_policy_review', 'policy_version': 'retriever-v1', 'reason': 'Low
rank-1 CTR (0.0)'}
{'type': 'recommendation_policy_review', 'policy_version': 'reco-v1', 'reason': 'Low
click-to-conversion rate (0.0)'}
{'type': 'trading_strategy_review', 'strategy_version': 'strategy-v1', 'reason':
'Negative average risk-adjusted score (-33.75)'}
```

The click summary retains the policy version and rank position instead of comparing raw click totals. This preserves information needed for later bias-aware analysis, although the example does not itself correct for unequal exposure. The conversion summary applies the configured seven-day attribution rule. The later conversion is therefore excluded from the attributed revenue, illustrating that attribution depends on an explicit business rule. In a production system, recent clicks should remain pending until their observation window closes.

The trading summary uses both profit and drawdown instead of treating profit alone as the outcome. Its score is deliberately illustrative rather than a production-grade financial measure, but it demonstrates that return and risk should be considered together. The example stops at review candidates. It does not rewrite prompts, replace policies, or modify the graph automatically. As established earlier, these records are candidate observations rather than validated lessons. They can now enter a controlled review process in which LangSmith supports tracing, evaluation, feedback, and human annotation, while LangChain and LangGraph structure and persist the resulting data. The application remains responsible for deciding whether the evidence justifies a change.

4.5 IMPLEMENTING RUNTIME ADAPTATION: CONTEXT, MODEL CONTEXT, RETRIEVAL POLICY, AND GRAPH CONTROL

Adaptation is easiest to understand if you stop thinking of it as “the agent is learning” and start thinking of it as “the system is changing what happens next while it is running.” In this chapter, adaptation means a bounded runtime change made in response to the current context, uncertainty, cost, risk, or an observed result. It does not mean retraining the model, redesigning the system over several weeks, or solving long-term problems such as model drift and reward hacking. Runtime adaptation can be proactive or reactive. A workflow may select a cheaper model because the request is simple, restrict the available tools because only one action is relevant, or apply a regional policy before any failure occurs. It may also react to missing evidence, invalid output, an ambiguous request, a failed tool call, or a risky proposed action. In both cases, the system makes a controlled decision about what should happen next instead of continuing blindly.

That adjustment can happen at several control surfaces. The system might change the instructions or messages sent to the model, expose a different subset of tools, select another model or response format, gather evidence differently, or follow another path through the workflow. Adaptation is therefore best treated as a runtime control problem: the workflow observes a signal and makes a bounded change to what happens next. LangChain and LangGraph do not provide a single adaptation switch. LangChain agent middleware can modify model context, select models or tools, apply retries and fallbacks, add guardrails, or stop execution. LangChain agents run on LangGraph, whose state, nodes, edges, `Command`, interrupts, checkpoints, and stores provide lower-level workflow control. The practical design question is therefore specific: what may change, which signal is reliable enough to trigger the change, and whether the change should last for one model call, one invocation, one thread, or future threads? The four layers below provide a practical way to answer that question.

The first layer is runtime context. It supplies trusted, static information and dependencies for one invocation, such as authenticated user claims, region, tenant, feature flags, environment settings, or database clients. Runtime context can guide the workflow, but merely placing a value in context does not enforce authorization. Tools, retrievers, and services must still apply the required security checks. The second layer is model-context adaptation. It changes the information or capabilities supplied to a particular model call. The workflow may change the instructions, add relevant messages, select another model, expose a different set of tools, or require another response format. Prompt adaptation is one part of this broader layer. The third layer is retrieval-policy adaptation. It changes how the system gathers evidence. The workflow may decide that retrieval is unnecessary, rewrite a conversational query, broaden candidate generation, diversify or rerank results, apply trusted scope filters, or use a more expensive retrieval path for a difficult request. The fourth layer is graph-control adaptation. It changes the path followed by the current execution through an existing graph. The workflow may branch to clarification, retry a failed step, route to revision, pause for human input, or resume from checkpointed state. Changing the deployed graph definition is a separate concern called graph migration. Keeping these layers separate makes adaptive behaviour easier to understand and test. When a result is weak, you can ask whether the problem came from the invocation inputs, the model context, the evidence-gathering policy, or the workflow path. A good adaptive design changes the smallest suitable control surface for a clear reason and

records enough information to explain the decision later. The terminology used in this section is important. A model call is one request to a language model. An invocation or run is one execution of an agent or graph. A thread is a sequence of checkpointed invocations that share the same thread identifier. Runtime context contains static invocation data and dependencies. State contains mutable workflow data. A store contains information that the application deliberately retains for use beyond one thread.

4.5.1 Runtime context: supply trusted per-invocation inputs

Runtime context is often the simplest adaptation surface because the application can supply important information before the model starts reasoning. In this section, runtime context means static, invocation-scoped data and dependencies passed through `context_schema` and the `context` argument. It should not be confused with `RunnableConfig`, which carries execution settings such as thread identifiers, callbacks, tags, metadata, and concurrency options. Trusted inputs such as authenticated user claims, region, tenant, deployment environment, feature flags, or database connections should come from the surrounding application. They should not be guessed by the model or extracted from conversation history as if they were ordinary chat content. Tools, middleware, and graph nodes can read these values and adapt their behaviour. For example, a support assistant may explain a result differently for a beginner and an expert or select the policy corpus associated with the customer's region. Runtime context helps preserve the boundary between application data and model-generated content, but it is not a security mechanism by itself. A tenant identifier or role in context is only an input to an authorization decision. The retriever, tool, or underlying service must still verify that the caller is permitted to access the requested data or perform the requested action.

Runtime context, state, and store serve different purposes. Runtime context contains static information and dependencies for one invocation. State contains mutable workflow information such as messages, retrieved documents, tool outputs, draft answers, verifier results, and execution status. State exists during the current execution and, when the graph uses a checkpointer, can be restored across later invocations that use the same thread identifier. A store is used for information that may be shared across threads or conversations, such as validated user preferences or operating rules. Whether that information survives application restarts depends on the store backend. A simple rule is therefore useful: put trusted, fixed invocation inputs in runtime context; put changing workflow information in state; and put deliberately retained cross-thread information in a suitably durable store.

These mechanisms define different scopes rather than three automatic lifetimes. Runtime context is available for the current invocation. State changes during execution and can continue across invocations only when it is checkpointed and the same thread identifier is reused. Store data can be reused across threads only when the application writes it deliberately and the selected backend preserves it. Keeping these scopes separate makes adaptation easier to understand because you can identify where each value came from and how long it should remain relevant. In a support assistant, runtime context might carry the customer's region and support tier. State might hold the documents retrieved for the current question and the draft answer being checked. Store might contain a validated note such as "do not promise a cash refund for gift purchases unless the policy explicitly says so." The workflow can use these sources to adapt its prompt, model choice, or retrieval behaviour while keeping the overall agent definition unchanged.

```
from __future__ import annotations
```

```

from dataclasses import dataclass
from typing import Callable
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, ModelRequest, dynamic prompt
from langchain.agents.middleware.types import ModelResponse
from langchain.tools import ToolRuntime, tool
from langchain_openai import ChatOpenAI
from langgraph.store.memory import InMemoryStore
from .config import OPENAI_API_KEY

ORDERS = {
    "ORD-100": {
        "status": "delivered",
        "days_since_delivery": 12,
        "opened": False,
        "gift": True
    },
    "ORD-101": {
        "status": "processing",
        "days_since_delivery": 0,
        "opened": False,
        "gift": False
    }
}

@dataclass
class SupportContext:
    user_id: str
    user_role: str # "beginner" | "expert"
    region: str # "EU" | "US"

@tool
def lookup_order(order_id: str) -> dict:
    """Look up order status and return basic return facts."""
    order = ORDERS.get(order_id)
    if not order:
        return {
            "order_id": order_id,
            "error": f"Order {order_id} was not found."
        }
    return {"order_id": order_id, **order}

@tool
def read_refund_policy(runtime: ToolRuntime[SupportContext]) -> str:
    """Return the refund-policy text for the user's region."""
    if runtime.context.region == "EU":
        return (
            "EU policy: standard items may be refunded within 30 days if unopened. "
            "Gift purchases are normally returned as store credit unless the policy "
            "explicitly states otherwise."
        )
    return (
        "US policy: standard items may be refunded within 30 days if unopened. "
        "Gift purchases follow the store-specific policy and may be handled as "
        "store credit."
    )

fast_model = ChatOpenAI(model = "gpt-4.1-mini", temperature = 0)
strong_model = ChatOpenAI(model = "gpt-4.1", temperature = 0)

def get_persistent_rule(store: InMemoryStore | None, user_id: str) -> str | None:
    if store is None:
        return None
    note = store.get(("adaptation_rules",), user_id)
    if not note:
        return None
    value = note.value or {}
    return value.get("instruction")

def build_support_prompt(ctx: SupportContext, store: InMemoryStore | None) -> str:
    base = (
        "You are a customer support assistant. "
        "Use tools for order facts and policy facts. "
        "Do not claim a policy outcome unless tool output supports it."
    )
    if ctx.user_role == "expert":
        base += " Keep the answer compact and operational."
    else:

```



```

        base += " Explain the answer in plain language."

    if ctx.region == "EU":
        base += " Prefer EU policy rules when they differ by region."
    else:
        base += " Prefer US policy rules when they differ by region."

    rule = get_persistent_rule(store, ctx.user_id)
    if rule:
        base += f" Persistent operating rule: {rule}"

    return base

def select_model(messages: list[dict]) -> tuple[str, ChatOpenAI, str]:
    last_user_text = ""
    if messages:
        last = messages[-1]
        last_user_text = str(last.get("content", "")).lower()

    reasons: list[str] = []

    if "policy" in last_user_text:
        reasons.append("message contains 'policy'")
    if "gift" in last_user_text:
        reasons.append("message contains 'gift'")
    if "exception" in last_user_text:
        reasons.append("message contains 'exception'")
    if len(messages) > 4:
        reasons.append("conversation has more than 4 messages")

    if reasons:
        return "gpt-4.1", strong_model, "; ".join(reasons)

    return "gpt-4.1-mini", fast_model, "default fast path"

@dynamic_prompt
def support_prompt(request: ModelRequest) -> str:
    return build_support_prompt(
        ctx = request.runtime.context,
        store = request.runtime.store
    )

class DynamicModelMiddleware(AgentMiddleware):
    def wrap_model_call(
        self,
        request: ModelRequest,
        handler: Callable[[ModelRequest], ModelResponse]
    ) -> ModelResponse:
        # Reuse the same logic used by the console demo so the
        # visible trace and the actual execution stay aligned.
        visible_messages: list[dict] = []
        for msg in request.messages:
            visible_messages.append(
                {
                    "role": getattr(msg, "type", "message"),
                    "content": str(getattr(msg, "content", ""))
                }
            )

        _, selected_model, _ = select_model(visible_messages)
        return handler(request.override(model = selected_model))

def build_store(include_rule: bool) -> InMemoryStore:
    store = InMemoryStore()

    if include_rule:
        store.put(
            ("adaptation_rules",),
            "user-123",
            {
                "instruction": (
                    "Do not promise a cash refund for gift purchases unless the policy "
                    "text states it explicitly."
                )
            }
        )

    return store

def build_agent(store: InMemoryStore):
    return create_agent(
        model = fast_model,
        tools = [lookup_order, read_refund_policy],
        middleware = [support_prompt, DynamicModelMiddleware()],

```

```

        context_schema = SupportContext,
        store = store
    )

def print_separator() -> None:
    print("=" * 100)

def print_title(title: str) -> None:
    print_separator()
    print(title)
    print_separator()

def describe_adaptation(
    *,
    ctx: SupportContext,
    user_message: str,
    store: InMemoryStore
) -> None:
    visible_messages = [{"role": "user", "content": user_message}]
    model_name, _, model_reason = select_model(visible_messages)
    persistent_rule = get_persistent_rule(store, ctx.user_id)
    effective_prompt = build_support_prompt(ctx, store)

    print("ADAPTATION TRACE")
    print(f"  user_id          : {ctx.user_id}")
    print(f"  user_role         : {ctx.user_role}")
    print(f"  region            : {ctx.region}")
    print(f"  selected_model    : {model_name}")
    print(f"  selection_reason  : {model_reason}")
    print(
        "    role_adaptation : "
        + (
            "plain-language explanation"
            if ctx.user_role == "beginner"
            else "compact operational answer"
        )
    )
    print(
        "    region_adaptation : "
        + (
            "EU policy preference"
            if ctx.region == "EU"
            else "US policy preference"
        )
    )
    print(
        "    store_rule       : "
        + (persistent_rule if persistent_rule else "none")
    )
    print()
    print("EFFECTIVE SYSTEM PROMPT")
    print(f"  {effective_prompt}")
    print()

def run_case(
    *,
    title: str,
    user_message: str,
    ctx: SupportContext,
    include_rule: bool
) -> None:
    print_title(title)

    store = build_store(include_rule = include_rule)
    agent = build_agent(store)

    print("USER MESSAGE")
    print(f"  {user_message}")
    print()

    describe_adaptation(
        ctx = ctx,
        user_message = user_message,
        store = store
    )

    result = agent.invoke(
        {"messages": [{"role": "user", "content": user_message}]},
        context = ctx
    )

    final_answer = result["messages"][-1].content

```

```

print("FINAL ANSWER")
print(final_answer)
print()

def main() -> None:
    run_case(
        title = "CASE 1 - Beginner user, EU region, gift refund question",
        user_message = "Can I get a refund for order ORD-100 if it was a gift?",
        ctx = SupportContext(
            user_id = "user-123",
            user_role = "beginner",
            region = "EU"
        ),
        include_rule = True
    )

    run_case(
        title = "CASE 2 - Expert user, EU region, same question",
        user_message = "Can I get a refund for order ORD-100 if it was a gift?",
        ctx = SupportContext(
            user_id = "user-123",
            user_role = "expert",
            region = "EU"
        ),
        include_rule = True
    )

    run_case(
        title = "CASE 3 - Beginner user, US region, same question",
        user_message = "Can I get a refund for order ORD-100 if it was a gift?",
        ctx = SupportContext(
            user_id = "user-123",
            user_role = "beginner",
            region = "US"
        ),
        include_rule = True
    )

    run_case(
        title = "CASE 4 - Beginner user, EU region, simple order-status question",
        user_message = "What is the status of order ORD-101?",
        ctx = SupportContext(
            user_id = "user-123",
            user_role = "beginner",
            region = "EU"
        ),
        include_rule = True
    )

    run_case(
        title = "CASE 5 - Beginner user, US region, same gift question, persistent rule
            disabled",
        user_message = "Can I get a refund for order ORD-100 if it was a gift?",
        ctx = SupportContext(
            user_id = "user-123",
            user_role = "beginner",
            region = "US"
        ),
        include_rule = False
    )

if __name__ == "__main__":
    main()

```

vol3_ch_04\src\adapt_config.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.adapt_config

=====
CASE 1 - Beginner user, EU region, gift refund question
=====
USER MESSAGE
    Can I get a refund for order ORD-100 if it was a gift?

ADAPTATION TRACE
    user_id      : user-123
    user_role    : beginner
    region       : EU
    selected_model : gpt-4.1

```

```

selection_reason : message contains 'gift'
role_adaptation  : plain-language explanation
region_adaptation : EU policy preference
store rule       : Do not promise a cash refund for gift purchases unless the policy
text states it explicitly.

```

EFFECTIVE SYSTEM PROMPT

You are a customer support assistant. Use tools for order facts and policy facts. Do not claim a policy outcome unless tool output supports it. Explain the answer in plain language. Prefer EU policy rules when they differ by region. Persistent operating rule: Do not promise a cash refund for gift purchases unless the policy text states it explicitly.

FINAL ANSWER

Your order ORD-100 was a gift and was delivered 12 days ago. According to EU policy, standard items can be refunded within 30 days if they are unopened. However, for gift purchases, the usual process is to issue store credit rather than a cash refund, unless the policy specifically says otherwise. Since your order is unopened and within the 30-day window, you are eligible for a return, but you will likely receive store credit instead of a cash refund. If you need help starting the return, let me know!

=====

CASE 2 - Expert user, EU region, same question

=====

USER MESSAGE

Can I get a refund for order ORD-100 if it was a gift?

ADAPTATION TRACE

```

user_id      : user-123
user_role    : expert
region       : EU
selected_model : gpt-4.1
selection_reason : message contains 'gift'
role_adaptation : compact operational answer
region_adaptation : EU policy preference
store rule    : Do not promise a cash refund for gift purchases unless the policy
text states it explicitly.

```

EFFECTIVE SYSTEM PROMPT

You are a customer support assistant. Use tools for order facts and policy facts. Do not claim a policy outcome unless tool output supports it. Keep the answer compact and operational. Prefer EU policy rules when they differ by region. Persistent operating rule: Do not promise a cash refund for gift purchases unless the policy text states it explicitly.

FINAL ANSWER

For order ORD-100, since it was a gift and remains unopened, you are eligible to return it within 30 days of delivery. However, according to EU policy, gift returns are typically refunded as store credit, not cash. If you wish to proceed, you can request a return for store credit.

=====

CASE 3 - Beginner user, US region, same question

=====

USER MESSAGE

Can I get a refund for order ORD-100 if it was a gift?

ADAPTATION TRACE

```

user_id      : user-123
user_role    : beginner
region       : US
selected_model : gpt-4.1
selection_reason : message contains 'gift'
role_adaptation : plain-language explanation
region_adaptation : US policy preference
store rule    : Do not promise a cash refund for gift purchases unless the policy
text states it explicitly.

```

EFFECTIVE SYSTEM PROMPT

You are a customer support assistant. Use tools for order facts and policy facts. Do not claim a policy outcome unless tool output supports it. Explain the answer in plain language. Prefer US policy rules when they differ by region. Persistent operating rule: Do not promise a cash refund for gift purchases unless the policy text states it explicitly.

FINAL ANSWER

```
Your order ORD-100 was a gift and has been delivered 12 days ago. According to the US
refund policy, standard items can be refunded within 30 days if they are unopened. For
gift purchases, refunds are usually handled as store credit rather than a cash refund.
If the gift is still unopened, you are eligible to request a return, but the refund will
likely be issued as store credit. If you need help starting the return process, let me
know!
```

```
=====
CASE 4 - Beginner user, EU region, simple order-status question
=====
```

```
USER MESSAGE
```

```
What is the status of order ORD-101?
```

```
ADAPTATION TRACE
```

```
user_id      : user-123
user_role    : beginner
region       : EU
selected_model : gpt-4.1-mini
selection_reason : default fast path
role_adaptation : plain-language explanation
region_adaptation : EU policy preference
store_rule    : Do not promise a cash refund for gift purchases unless the policy
text states it explicitly.
```

```
EFFECTIVE SYSTEM PROMPT
```

```
You are a customer support assistant. Use tools for order facts and policy facts. Do
not claim a policy outcome unless tool output supports it. Explain the answer in plain
language. Prefer EU policy rules when they differ by region. Persistent operating rule:
Do not promise a cash refund for gift purchases unless the policy text states it
explicitly.
```

```
FINAL ANSWER
```

```
Order ORD-101 is currently processing and has not yet been delivered. If you have any
other questions about this order, please let me know!
```

```
=====
CASE 5 - Beginner user, US region, same gift question, persistent rule disabled
=====
```

```
USER MESSAGE
```

```
Can I get a refund for order ORD-100 if it was a gift?
```

```
ADAPTATION TRACE
```

```
user_id      : user-123
user_role    : beginner
region       : US
selected_model : gpt-4.1
selection_reason : message contains 'gift'
role_adaptation : plain-language explanation
region_adaptation : US policy preference
store_rule    : none
```

```
EFFECTIVE SYSTEM PROMPT
```

```
You are a customer support assistant. Use tools for order facts and policy facts. Do
not claim a policy outcome unless tool output supports it. Explain the answer in plain
language. Prefer US policy rules when they differ by region.
```

```
FINAL ANSWER
```

```
Your order ORD-100 was a gift and was delivered 12 days ago. According to the US refund
policy, standard items can be refunded within 30 days if they are unopened. For gifts,
refunds are usually handled as store credit rather than a direct refund. Since your gift
order is unopened and within the 30-day window, you are eligible for a refund, but it
will likely be issued as store credit. If you'd like to proceed, let me know and I can
guide you through the next steps.
```

This sample demonstrates three adaptation surfaces without changing the agent's overall structure. Runtime context supplies the region and user role. `@dynamic_prompt` creates call-specific instructions from that context and from the stored operating rule. Middleware then selects a model using a simple keyword-based routing heuristic. The routing heuristic is intentionally illustrative. Words such as "gift," "policy," or "exception" do not provide a reliable general measure of difficulty. A production router should use evaluated task categories, confidence signals, risk classifications, cost limits, or measured historical performance. The architectural lesson remains the same: many useful adaptations do not

require rebuilding the graph. They require placing each decision on the appropriate control surface and recording why that decision was made.

4.5.2 Model context: adapt instructions and capabilities

Prompt adaptation is the most visible kind of adaptation because developers can directly inspect the instructions and messages sent to the model. It is not mainly about rewriting sentences until the answer sounds better. Its purpose is to change the instructions or supporting information that the model receives for a particular call. Prompt adaptation is therefore one part of the broader discipline of model-context adaptation, not a name for every runtime control applied to the model.

The broader concept is model context. A model call can be shaped by its system instructions, message history, available tools, selected model, and required response format. Changing instructions or messages is prompt adaptation. Changing the available tools, model, or response schema is a different form of model-context adaptation. Keeping these terms separate prevents the misleading impression that every control is only a prompt rewrite. For example, after a weak answer, the system might add a temporary instruction such as “answer only from retrieved evidence.” For another step, it might expose only the two relevant tools, select a stronger model, or require structured output because the next workflow stage needs predictable fields. Each change affects the model call, but it acts on a different control surface.

The practical question is therefore not only “how should the instruction be written?” It is “what information and capabilities should this model call receive?” The workflow should make the smallest change that solves the current problem. Some guidance should apply to one model call only. If a verifier finds that the current request is ambiguous, the next call may receive a temporary instruction to ask for clarification. That instruction should not automatically affect later calls. Other guidance may be worth retaining. If evaluation repeatedly identifies the same failure and a compact corrective rule has been reviewed and validated, the application may store that rule for reuse. For example, it might retain “do not promise a refund outcome unless the policy evidence is explicit.” Raw model criticism, complete transcripts, and unvalidated reflections should not be written directly into long-term memory. They may be incomplete, overly specific, or wrong. Temporary guidance belongs in the current model call or workflow state. Only compact, validated, reusable instructions should be written to a store.

4.5.3 Retrieval policy: adapt how evidence is gathered

Retrieval policy should be kept separate from model instructions. Model context controls how the model uses the information it receives. Retrieval policy controls which evidence is gathered before the model sees it. A weak answer may therefore be a generation failure, but it may also be the predictable result of missing, irrelevant, stale, or unauthorized evidence. Retrieval should not be treated as one fixed operation for every request. A realistic workflow must decide whether retrieval is needed, how to construct the query, how broadly to search, how to rank and process the results, and which documents are eligible to participate. The first decision is whether to retrieve at all. Greetings, workflow-status questions, exact calculations, and requests that should be handled by a structured tool may not need document retrieval. Skipping retrieval in those cases reduces latency, cost, and irrelevant context. The decision not to retrieve is therefore a valid retrieval-policy outcome.

The second decision is how to form the retrieval query. This is especially important in multi-turn assistants because follow-up questions may be understandable in conversation but

ineffective as standalone searches. A support user may ask, “Does that include gifts?” A human can infer that “that” refers to the refund rule discussed earlier, but a retriever receives only a short, underspecified query. The system can adapt by rewriting the follow-up as a standalone query, such as “Do gift purchases follow the same refund policy as standard purchases?” This changes how evidence is gathered without changing the user’s underlying question. The third decision is how much retrieval effort to spend. A direct question with clear terminology may need only a small similarity search. A more ambiguous or higher-risk question may justify broader candidate generation, hybrid retrieval, multiple query variants, reranking, or more careful context processing. MMR can reduce redundancy by balancing relevance with diversity among the selected results, but it does not by itself guarantee higher recall. Recall depends mainly on whether the candidate-generation stage retrieves the relevant evidence. Therefore, a path should be described as higher-recall only when its candidate-generation strategy and evaluation results support that claim.

The fourth decision is retrieval scope. A production system should not search its entire corpus for every question. Region, tenant, product line, content version, and permission level determine which evidence is eligible for retrieval. An EU policy may differ from a US policy, one tenant must not receive another tenant’s documents, and restricted internal material must not enter an unauthorized user’s model context. Scope values should come from authenticated and trusted application data rather than model inference. However, an ordinary metadata filter should not automatically be treated as the complete authorization mechanism. Authorization must be enforced at an application, retriever, data-service, or policy-enforcement boundary before restricted content reaches the model. Metadata filters can participate in that enforcement when they are derived from a trusted authorization decision and cannot be overridden by the model. Retrieval policy therefore decides whether retrieval is needed, which query to use, how much search effort is justified, and which authorized corpus may participate.

A small support assistant makes this concrete. Suppose the user first asks about standard refunds and then follows with, “What about gifts?” The system may rewrite the follow-up as a standalone query, use a broader or more diversified retrieval strategy, and apply a trusted region filter so only the appropriate policy corpus is considered. What happens next depends on why the evidence is insufficient. If the user’s intent remains ambiguous, the workflow should ask for clarification. If the query is clear but retrieval appears weak, it may rewrite the query or broaden candidate generation. If the authoritative corpus does not contain the answer, the system should abstain or escalate rather than repeatedly asking the user to clarify a question that is already clear. This is practical adaptation: the workflow changes its evidence-gathering strategy while keeping unsupported conclusions out of the answer.

```
from __future__ import annotations
import json
from dataclasses import dataclass
from typing import Literal, TypedDict
from langchain_chroma import Chroma
from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langgraph.graph import END, START, StateGraph
from langgraph.runtime import Runtime
from langgraph.types import Command
from .config import OPENAI_API_KEY

# -----
# Small support-policy corpus
# -----
```



```

def build_policy_docs() -> list[dict]:
    .
    .

def open_policy_store(chroma_dir: str, embedder: OpenAIEmbeddings) -> Chroma:
    return Chroma(
        collection_name = "support_policy",
        persist_directory = chroma_dir,
        embedding_function = embedder
    )

def seed_policy(store: Chroma) -> None:
    splitter = RecursiveCharacterTextSplitter(chunk_size = 220, chunk_overlap = 40)

    docs: list[Document] = []
    ids: list[str] = []

    for raw in build_policy_docs():
        parts = splitter.split_text(raw["text"])
        for i, part in enumerate(parts):
            docs.append(
                Document(
                    page_content = part,
                    metadata = {
                        "doc_id": raw["doc_id"],
                        "region": raw["region"],
                        "chunk_no": i
                    }
                )
            )
            ids.append(f"{raw['doc_id']}#{i}")

    try:
        store.add_documents(docs, ids = ids)
    except Exception:
        # Ignore duplicate-id errors on repeated runs.
        pass

def format_sources(docs: list[Document]) -> str:
    rows = []
    for idx, doc in enumerate(docs, start = 1):
        rows.append(
            f"[{idx}] {doc.metadata['doc_id']} ({doc.metadata['region']})\n"
            f"{doc.page_content}"
        )
    return "\n\n".join(rows)

def doc_refs(docs: list[Document]) -> list[str]:
    refs: list[str] = []
    for doc in docs:
        refs.append(f"{doc.metadata['doc_id']}#{doc.metadata['chunk_no']}")
    return refs

# -----
# Graph state and context
# -----
class RetrievalState(TypedDict, total = False):
    question: str
    history: list[dict[str, str]]
    retrieval_query: str
    retrieval_mode: Literal["", "fast", "high_recall"]
    retrieval_reason: str
    sources: str
    answer: str

@dataclass
class RetrievalContext:
    region: str

llm = ChatOpenAI(model = "gpt-4.1-mini", temperature = 0)
embedder = OpenAIEmbeddings(model = "text-embedding-3-large")
VECTOR_STORE = open_policy_store("./chroma_support_policy", embedder)
seed_policy(VECTOR_STORE)

# -----
# Nodes
# -----
def rewrite_followup(
    state: RetrievalState,
    runtime: Runtime[RetrievalContext]
) -> dict:

```

```

history_text = "\n".join(
    f"{m['role']}: {m['content']}" for m in state.get("history", [])[-4:]
)

emit(
    runtime,
    "rewrite_started",
    original_question = state["question"],
    history_turns = len(state.get("history", []))
)

prompt = f"""
Rewrite the user's latest message as a standalone search query for a support-policy
knowledge base.
Use the chat history only to resolve references such as "that", "it", or "those".
Do not answer the question.

CHAT HISTORY:
{history_text}

LATEST USER MESSAGE:
{state['question']}
"""

msg = llm.invoke(prompt)
retrieval_query = msg.content.strip()

emit(
    runtime,
    "rewrite_completed",
    retrieval_query = retrieval_query
)

return {"retrieval_query": retrieval_query}

def choose_retrieval_policy(
    state: RetrievalState
) -> Command[Literal["retrieve_fast", "retrieve_high_recall"]]:
    q = state["question"].lower()

    signals: list[str] = []

    if len(q.split()) < 6:
        signals.append("short question")
    if "that" in q:
        signals.append("contains 'that'")
    if "it" in q:
        signals.append("contains 'it'")
    if "those" in q:
        signals.append("contains 'those'")
    if "what about" in q:
        signals.append("contains 'what about'")

    if signals:
        return Command(
            update = {
                "retrieval_mode": "high_recall",
                "retrieval_reason": ", ".join(signals)
            },
            goto = "retrieve_high_recall"
        )

    return Command(
        update = {
            "retrieval_mode": "fast",
            "retrieval_reason": "question is already explicit"
        },
        goto = "retrieve_fast"
    )

def retrieve_fast(
    state: RetrievalState,
    runtime: Runtime[RetrievalContext],
) -> Command[Literal["draft_answer", "ask_clarification"]]:
    emit(
        runtime,
        "retrieval_started",
        retrieval_mode = "fast",
        retrieval_query = state["retrieval_query"],
        region_filter = runtime.context.region,
        search_type = "similarity",
        k = 2
    )

```

```

retriever = VECTOR_STORE.as_retriever(search_kwargs = {"k": 2})
docs = retriever.invoke(
    state["retrieval_query"],
    filter = {"region": runtime.context.region}
)

if not docs:
    emit(runtime, "retrieval_completed", doc_count = 0, doc_refs = [])
    return Command(
        update = {"source_count": 0, "sources": ""},
        goto = "ask_clarification"
    )

emit(
    runtime,
    "retrieval_completed",
    doc_count = len(docs),
    doc_refs = doc_refs(docs)
)

return Command(
    update = {
        "source_count": len(docs),
        "sources": format_sources(docs)
    },
    goto = "draft_answer"
)

def retrieve_high_recall(
    state: RetrievalState,
    runtime: Runtime[RetrievalContext]
) -> Command[Literal["draft_answer", "ask_clarification"]]:
    emit(
        runtime,
        "retrieval_started",
        retrieval_mode = "high_recall",
        retrieval_query = state["retrieval_query"],
        region_filter = runtime.context.region,
        search_type = "mmr",
        k = 4,
        fetch_k = 8
    )

    retriever = VECTOR_STORE.as_retriever(
        search_type = "mmr",
        search_kwargs = {"k": 4, "fetch_k": 8}
    )
    docs = retriever.invoke(
        state["retrieval_query"],
        filter = {"region": runtime.context.region}
    )

    if not docs:
        emit(runtime, "retrieval_completed", doc_count = 0, doc_refs = [])
        return Command(
            update = {"source_count": 0, "sources": ""},
            goto = "ask_clarification"
        )

    emit(
        runtime,
        "retrieval_completed",
        doc_count = len(docs),
        doc_refs = doc_refs(docs)
    )

    return Command(
        update = {
            "source count": len(docs),
            "sources": format_sources(docs)
        },
        goto = "draft_answer"
    )

def ask_clarification(
    state: RetrievalState,
    runtime: Runtime[RetrievalContext]
) -> dict:
    emit(
        runtime,
        "clarification_requested",
        reason = "no matching sources after applying retrieval policy"
    )

```

```

    )
    return {
        "answer": (
            "I need one more detail before I can answer reliably. "
            "Please tell me which return scenario or order you mean."
        )
    }

def draft_answer(
    state: RetrievalState,
    runtime: Runtime[RetrievalContext]
) -> dict:
    emit(
        runtime,
        "answer_started",
        retrieval_mode = state.get("retrieval_mode", ""),
        source_count = state.get("source_count", 0)
    )

    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are a support assistant. Answer using only the sources. "
                "If the sources do not establish the answer, say that you do not know."
            ),
            (
                "human",
                "Question:\n{question}\n\nSources:\n{sources}\n\n"
                "Answer in plain language and mention source indexes such as [1] or [2]"
            )
        ]
    )

    messages = prompt.format_messages(
        question = state["question"],
        sources = state["sources"]
    )
    msg = llm.invoke(messages)
    answer = msg.content.strip()

    emit(
        runtime,
        "answer_completed",
        answer_preview = answer[:180]
    )

    return {"answer": answer}

# -----
# Build the graph
# -----
builder = StateGraph(RetrievalState, context_schema = RetrievalContext)

builder.add_node("rewrite_followup", rewrite_followup)
builder.add_node("choose_retrieval_policy", choose_retrieval_policy)
builder.add_node("retrieve_fast", retrieve_fast)
builder.add_node("retrieve_high_recall", retrieve_high_recall)
builder.add_node("ask_clarification", ask_clarification)
builder.add_node("draft_answer", draft_answer)

builder.add_edge(START, "rewrite_followup")
builder.add_edge("rewrite_followup", "choose_retrieval_policy")
builder.add_edge("ask_clarification", END)
builder.add_edge("draft_answer", END)

graph = builder.compile()

# -----
# Console demo
# -----
def print_banner(title: str) -> None:
    .
    .

def print_update(node_name: str, update: dict) -> None:
    .
    .

def print_custom_event(event: dict) -> None:
    .
    .

```

```

def run_demo(
    *,
    title: str,
    question: str,
    history: list[dict[str, str]],
    region: str
) -> None:
    print_banner(title)

    result = graph.invoke(
        {
            "question": question,
            "history": history
        },
        context = RetrievalContext(region = region)
    )

    print("QUESTION")
    print(f" {question}")
    print()

    print("ADAPTATION")
    print(f" rewritten query : {result['retrieval_query']}")
    print(f" retrieval mode   : {result['retrieval_mode']}")
    print(f" reason              : {result['retrieval_reason']}")
    print()

    print("EVIDENCE")
    if result.get("sources"):
        for block in result["sources"].split("\n\n"):
            first_line = block.splitlines()[0]
            print(f" {first_line}")
    else:
        print(" no sources found")
    print()

    print("ANSWER")
    print(f" {result['answer']}")
    print()

if __name__ == "__main__":
    run_demo(
        title = "CASE 1 - Follow-up question, should switch to high_recall",
        question = "Does that include gifts?",
        history = [
            {
                "role": "user",
                "content": "Can I get a refund on a standard unopened item?"
            },
            {
                "role": "assistant",
                "content": (
                    "In the EU policy, unopened standard items may be refunded "
                    "within 30 days."
                )
            }
        ],
        region = "EU"
    )

    run_demo(
        title = "CASE 2 - Direct question, should switch to fast retrieval",
        question = "What does the EU policy say about gift returns?",
        history = [],
        region = "EU"
    )

```

vol3_ch_04\scr\retrieval_adaptation.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.retrieval_adaptation
=====
CASE 1 - Follow-up question, should switch to high_recall
=====
QUESTION
  Does that include gifts?

ADAPTATION
  rewritten query : Refund policy for unopened standard items that are gifts in the EU
  retrieval mode  : high_recall

```

```

reason          : short question, contains 'that'

EVIDENCE
[1] eu_refunds (EU)
[2] eu_gifts (EU)

ANSWER
Yes, gift purchases can be returned, but the refund is usually given as store credit
unless the policy specifically says you can get a refund to the original payment method
[2].

=====
CASE 2 - Direct question, should switch to fast retrieval
=====
QUESTION
What does the EU policy say about gift returns?

ADAPTATION
rewritten query : EU policy on gift returns and refund rights
retrieval mode  : fast
reason          : question is already explicit

EVIDENCE
[1] eu_gifts (EU)
[2] eu_refunds (EU)

ANSWER
The EU policy on gift returns generally states that if you return a gift, the store
will usually give you store credit instead of a refund to the original payment method,
unless the store's policy specifically says otherwise [1]. For standard items that are
unopened, refunds to the original payment method are allowed within 30 days of delivery
[2].

```

This example makes retrieval adaptation an explicit part of workflow control instead of hiding it inside one retriever configuration. The first adaptation step is query rewriting. The `rewrite_followup` node uses a short window of conversation history only to resolve references in the latest user message and create a standalone search query. It does not use earlier assistant answers as policy evidence. This preserves an important boundary: conversation history helps interpret the question, while retrieved documents provide the evidence used to answer it.

The second adaptation step is explicit retrieval-policy selection. The `choose_retrieval_policy` node inspects the user message for simple signals of ambiguity or follow-up structure, such as short length or terms including “that,” “it,” “those,” and “what about.” When those signals are present, the graph selects a broader, diversified retrieval path; otherwise, it selects a faster, narrower path. The selected mode and its reason are written into graph state, so later inspection can show both what the system retrieved and why it chose that strategy. The fast path uses a small similarity search for direct, explicit questions. The broader path fetches a larger candidate set and uses MMR to balance relevance with diversity and reduce redundant results. If the implementation retains the label `high_recall`, treat that name as the intended objective rather than a guarantee: MMR diversifies the selected results, while actual recall must be measured against an evaluation dataset. The example therefore demonstrates explicit query rewriting, policy selection, retrieval scoping, and a controlled non-answer path without hiding those decisions inside one default retriever.

The region filter is another important part of the design. Both retrieval nodes use trusted runtime context to restrict the candidate corpus before evidence is sent to the model. In production, that filter should be derived from an authenticated authorization decision and enforced by the retriever, data service, or policy layer; it should not be inferred or freely

modified by the model. The graph also treats missing evidence as an explicit workflow outcome. If the query is ambiguous, it can request clarification. If the query is clear but retrieval appears weak, it can rewrite or broaden the search. If the authorized corpus does not contain sufficient evidence, it should abstain or escalate rather than improvise. Additional compute, such as reranking or contextual compression, can then be reserved for cases where the candidate set is broad enough, but its ordering, redundancy, or relevance remains uncertain.

4.5.4 Graph control: branch, pause, resume, and recover

Graph-control adaptation changes the path followed by the current execution through an existing graph. It is useful when changing model context or retrieval policy is not enough and the workflow must take a different next step. Three mechanisms should be kept separate. `Command` can update state and select another destination during normal execution. `update_state` can create a new checkpointed state from outside the active execution path. `Interrupts` can pause execution and resume it later after external input. These mechanisms adapt execution; they do not normally change the deployed graph topology. `Command` is central because many adaptive decisions have the same shape: record new information and then continue somewhere different. Its `update` field changes state, `goto` selects the next destination, `graph` can target a parent graph when appropriate, and `resume` supplies a value to an interrupted execution.

This mechanism supports patterns such as “retrieval was empty, so select another outcome,” “the verifier rejected the draft, so return to revision,” or “a tool discovered a special case, so skip the normal synthesis path.” Tools as well as nodes can return `Command`, allowing tool execution to update state and influence control flow. One detail is important: when a tool uses `goto`, static edges defined on the node that called the tool still execute. A graph should therefore use either tool-driven dynamic routing or static routing from that node, rather than combining both unintentionally. External state editing is different from ordinary node or tool updates. `update_state` creates a new checkpoint containing the supplied update; it does not modify or delete the original checkpoint. Reducers still apply to the update, and the optional `as_node` argument can affect which node executes next. When the supplied configuration identifies a historical checkpoint, the new checkpoint creates an alternative branch from that point. When it identifies only the latest thread state, the update is applied from the latest checkpoint. This creates a new execution lineage rather than rewriting execution history. A minimal example looks like this:

```
config = {"configurable": {"thread_id": "support-thread-1"}}

# First run
graph.invoke(
    {
        "question": "Does that include gifts?",
        "history": []
    },
    context = RetrievalContext(region = "EU"),
    config = config
)

# Later, patch the saved state and re-run from the fork
fork_config = graph.update_state(
    config,
    values = {"retrieval_mode": "high_recall"}
)

forked_result = graph.invoke(
    None,
```



```
context = RetrievalContext(region = "EU"),  
config = fork_config  
)
```

This is not the same as starting again from the original input. The existing checkpoint history remains available, while the returned configuration identifies the newly created state. When the update was based on a historical checkpoint, later execution follows a new branch from that point. This makes `update_state` useful for controlled correction, human intervention, replay, and comparative debugging without destroying the original execution history.

Interrupts extend adaptation beyond internal branching by allowing the workflow to pause and wait for external input. They are appropriate for human review, approval, correction, or clarification when the system should not decide alone. Interrupts require checkpointing and a thread identifier so the saved execution can be found later. The graph resumes when the caller invokes it with the same thread identifier and supplies `Command(resume=...)`. An important execution rule is that the interrupted node restarts from its beginning when it resumes; code before `interrupt()` runs again. Any side effect before the interrupt must therefore be idempotent, or it should be moved into a separate node that executes only after approval. With that constraint observed, the graph can pause without guessing, preserve its state, and continue only when the required input arrives. Deployment-time graph evolution must be distinguished from runtime adaptation. LangGraph runs resumed threads against the currently deployed graph definition, so graph changes must remain compatible with saved checkpoints. Threads that have completed can tolerate full topology changes. Interrupted or otherwise in-flight threads can generally tolerate edge changes as long as the nodes recorded in their checkpoints still exist. Renaming or removing a node that a saved thread must enter or restart can break resumption. State changes also require care: add new fields as optional or give them defaults, stage removals through a deprecation period, perform renames by adding the new field before removing the old one, and avoid incompatible type changes while old checkpoints remain active. Runtime adaptation changes what the current execution does; graph migration changes the code and state contract used by new and resumed executions.

4.5.5 Design rules that keep adaptation useful

The first rule is to adapt at the smallest control surface that solves the problem. If the issue is wording or a missing instruction, change the prompt. If the model needs different tools, another response schema, or a different model, change the model context. If the evidence is missing, redundant, or poorly ranked, change retrieval policy. If the workflow must take another step, pause, or recover, change graph control. Escalating every problem into a graph branch creates unnecessary complexity, while trying to solve every control problem with prompt text makes the workflow opaque. The second rule is to keep trusted application data and authorization decisions outside model-generated content. User and tenant identifiers, authenticated role claims, database connections, and environment settings belong in runtime context or application code, not in arguments invented by the model. Runtime context transports these values, but tools, retrievers, and services must still enforce the corresponding authorization rules. The third rule is to distinguish temporary guidance, thread state, and cross-thread memory. Most adaptations should affect only the current model call or invocation. Some information must remain in checkpointed state for the current thread. Only compact, validated, reusable information should be written to a store for future threads.

Persisting complete transcripts or unvalidated model reflections can turn accidental model output into operating policy. The fourth rule is to make retrieval policy explicit. The workflow should record whether retrieval was used, which query was searched, how candidates were generated, which filters were applied, whether results were diversified or reranked, and why a particular path was selected. Explicit retrieval decisions can be evaluated, traced, and improved. Hidden defaults cannot. The fifth rule is to separate insufficient evidence from an ambiguous question. Ambiguity requires clarification. Weak retrieval may require query repair or broader search. A clear question with no supporting evidence requires abstention or escalation. Treating all three conditions as the same outcome creates frustrating conversations and can hide genuine gaps in the knowledge corpus. The sixth rule is to make every adaptive decision observable. Record the trigger, selected adaptation, reason, scope, additional cost, and eventual outcome. This allows evaluation to determine whether the adaptation improved accuracy, safety, latency, or cost instead of merely making the workflow more complicated. Finally, remember that LangChain and LangGraph provide control primitives, not a universal adaptation engine. They let you supply runtime context, transform model context, select tools and models, route with `Command`, pause with `interrupts`, persist state through checkpoints, and create alternative checkpointed state with `update_state`. They do not decide which signal is reliable enough to trigger a change, whether the change improved the result, or whether it should persist. Those decisions remain part of the application's design, evaluation, observability, and governance.

4.6 AVOIDING DEGRADATION: REWARD HACKING, DRIFT, AND OVERFITTING TO EVALUATION SETS

4.6.1 Why reflection systems still degrade

Reflection loops are useful because a first answer can be fluent and well-structured while still being unsupported, incomplete, unsafe, or aimed at the wrong objective. A generate-critique-revise workflow gives the system a chance to detect and correct such problems before accepting the result. However, the reflection loop itself does not learn or change the model. Degradation appears when the wider training or development process repeatedly tunes models, prompts, routing rules, or acceptance criteria against the same critique signal. The internal score may then improve while real-world usefulness remains unchanged or declines. This is difficult to notice because the system appears healthier according to the metric being optimized: more answers pass, scores rise, or the loop terminates sooner.

Degradation is therefore not a single bug but a family of failure modes. It appears when the system is optimized against a weak proxy, when its inputs or operating environment change, or when repeated tuning improves performance on one evaluation set without improving real use. The system can therefore look better according to its own measurements while becoming worse at the job it was built to perform. Preventing this requires discipline across runtime reflection, evaluation design, operational controls, and production monitoring.

For agent systems, this matters even more than for single-shot prompts. A support assistant can turn one weak answer into a customer-facing policy error. An e-commerce assistant can keep recommending products successfully on an old catalog while drifting away from current inventory or pricing rules. A trading assistant can continue to produce well-formed analysis even after the surrounding market data, tool behavior, or evaluation assumptions have changed. When these systems have memory, tools, and iterative loops, they can degrade while still looking superficially healthy.

4.6.2 Reward hacking: when the judge becomes the target

Strictly, reward hacking occurs when a model or policy is trained or otherwise optimized to obtain a high reward without fulfilling the intended objective. When no reward function is involved, the closer application-level problem is evaluator gaming: prompts, models, or workflow changes are repeatedly selected because they score well with an LLM judge, verifier, benchmark, safety monitor, or predictable review process. A fixed reflection loop does not learn to exploit its verifier during a single run. The risk appears when feedback from that verifier becomes an optimization target across training or repeated development cycles. Scores may then rise while correctness and usefulness remain unchanged or decline. A static evaluator that is repeatedly optimized against is therefore a weak long-term control.

A reflection loop should therefore be treated as one quality gate, not as an oracle. Use deterministic checks for objective conditions, LLM judges for qualities that cannot be reduced to fixed rules, pairwise comparison when choosing between variants, and human review for high-stakes or ambiguous decisions. The appropriate combination depends on the task and its risk. When tools or multiple processing steps are involved, evaluate the execution path as well as the final answer. A plausible final response can hide irrelevant tool calls, ignored evidence, unnecessary searches, or risky proposed actions.

State-changing actions should not be authorized solely by an LLM approval. For high-impact or irreversible operations, use policy checks, least-privilege tool scopes, explicit rejection paths, human approval where appropriate, call limits, and bounded execution. These controls do not eliminate evaluator gaming, but they limit the damage if the model or workflow is optimized in the wrong direction.

4.6.3 Drift: when yesterday's good behavior no longer fits today's system

Production quality can decline for several different reasons, and they should not all be treated as the same kind of drift. For practical agent systems, use the following operational categories:

- Context degradation: The model receives stale, noisy, contradictory, or irrelevant material. A long history may fit inside the context window but still distract the model, increase latency, and raise cost.
- Data drift: The distribution of incoming requests changes. New products, languages, customer groups, or query patterns may no longer resemble the data used during development and evaluation.
- Concept or policy drift: The relationship between an input and the correct response changes. A request that once required one answer may require another after a policy, regulation, business rule, or market condition changes.
- Environment drift: Tools, APIs, connectors, schemas, permissions, or external services change while the workflow remains the same.
- Retrieval-pipeline drift: Indexes become stale, document versions are mixed, chunking or metadata filters change, embedding models are migrated, or authorization scope is applied incorrectly.

These categories can overlap but separating them helps you locate the failing component instead of blaming the model for every quality regression. The remedy is not a single drift metric but a monitoring and evaluation loop. Track production behavior and business outcomes, inspect failing traces, classify each failure at the correct layer, add representative failures to the regression dataset, and test candidate fixes offline before deployment. This connects production evidence to the next development cycle and reduces the risk of repairing the wrong component.

4.6.4 Overfitting to eval sets: when benchmark gains stop meaning real gains

Overfitting to an evaluation set happens when repeated tuning makes the system better at passing a particular test without making it better at the real task. The cause may be benchmark contamination, repeated development against the same examples, a dataset that covers too few situations, a judge with stable preferences, or implementation details that materially affect the score. Widely used benchmarks illustrate the problem: training data may contain benchmark items, small formatting changes may alter results, and different implementations may not be directly comparable. Application evaluators introduce similar risks. Human reviewers can be influenced by tone, confidence, authority, or presentation, while LLM judges can have systematic preferences and biases. If teams repeatedly select changes using one judge or one aggregate score, they may optimize for that evaluator instead of for the application objective. The answer is layered evaluation with protected boundaries. Production-derived cases are valuable because they reflect real requests, long interactions, multiple languages, and tool use, but they do not automatically prevent overfitting. Once a production case is repeatedly used to tune the system, it becomes part of the development signal. Production traces may also omit rare but severe failures, and replayed evaluations may be misleading

when the original tools, data, permissions, or environment cannot be reproduced accurately. A disciplined evaluation program therefore uses three separate layers:

- A curated regression set: This contains known failures, important edge cases, and contractual requirements. It is expected to grow as the system is developed.
- A protected held-out set: This contains representative examples that are not used to choose prompts, models, routing rules, or other implementation changes. Once an example is used to guide a change, it is no longer truly held out and should be replaced with a new unseen example.
- Online evaluation on real traces: This detects drift, new failure modes, and unexpected production behavior. Because reference answers are often unavailable in production, online evaluation is an important monitoring signal but should not be treated as the only proof of correctness.

If improvement appears only on the regression set, treat it as local optimization until the protected held-out set and production evidence support the same conclusion.

4.6.5 A layered anti-degradation loop

The strongest way to resist degradation is to combine clear boundaries, risk-based controls, and independent evaluation rather than relying on one safeguard.

First, define an operational contract. State the objective, the evidence the system may use, the checks that must be deterministic, the tools it may call, the actions that require approval, the acceptance criteria, and the conditions that require abstention or escalation. Without this contract, reflection has no stable target, and evaluation has no defensible standard. Least privilege, structured logs, checkpoints, and recovery or compensation paths make the workflow auditable and controllable when conditions change. Checkpoints preserve workflow state, but they do not automatically undo external side effects.

Next, keep the reflection loop narrow, structured, and bounded. LangGraph's evaluator-optimizer pattern makes the sequence explicit: generate, evaluate, route, revise, or stop. The evaluator should return signals that the graph can act on, such as "unsupported claim," "revision required," or "insufficient evidence." Every loop also needs a revision budget and a terminal route. Without explicit stop conditions, reflection becomes an uncontrolled retry loop that can increase cost and latency without improving correctness.

Then, evaluate the system on several dimensions that are relevant to the application. Keep a small set of primary metrics for release decisions and a wider set of diagnostic metrics for investigation. For a grounded support assistant, the primary metrics might be correctness, policy compliance, and abstention quality. Diagnostics might include retrieval relevance, trajectory quality, tool use, latency, and token cost. The exact dimensions should follow the application rather than become a universal checklist. Optimizing one aggregate score makes it easier to improve the number while weakening behavior that the number does not capture.

Finally, close the loop with reviewed production evidence. LangSmith supports datasets built from curated cases, historical traces, and synthetic examples, together with offline experiments before release and online evaluation after release. Add confirmed production failures to regression datasets, but keep a separate held-out set outside routine tuning. This keeps evaluation connected to real use without turning every dataset into another development target.

4.6.6 A bounded, fail-closed verifier workflow in LangGraph

The following example uses the support-assistant scenario because the trade-off is easy to see. A refund answer is useful only when it stays within the available evidence. The graph therefore uses a typed verifier, a bounded revision budget, and an abstention path when the policy text cannot support an answer. This is the appropriate shape when reflection should improve quality without becoming open-ended self-persuasion. LangGraph's evaluator-optimizer pattern supports this explicit routing. For durable execution, replayed control flow must remain predictable, while external side effects must be idempotent or otherwise protected against duplicate execution.

```
from __future__ import annotations
from operator import add
from typing import Annotated, Literal
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
from .config import OPENAI_API_KEY

SAFE_ABSTENTION = "I do not have enough evidence to answer safely."

class Verification(BaseModel):
    verdict: Literal["accept", "revise", "abstain"] = Field(
        description = (
            "accept if the draft answers the question as far as the evidence allows "
            "and every non-trivial claim is supported; "
            "revise if the answer can be improved from the same evidence, including "
            "when the draft abstains but a partial or conditional answer is possible; "
            "abstain only if the evidence truly does not support even a partial or "
            "conditional answer"
        )
    )
    answers_question: bool = Field(
        description = "Whether the draft answers the user's question as far as the "
        "evidence allows."
    )
    abstention_justified: bool = Field(
        description = (
            "True only if the evidence is genuinely insufficient to support even a "
            "partial or conditional answer."
        )
    )
    feedback: str = Field(
        description = "Concrete instruction for improving the answer."
    )
    unsupported_claims: list[str] = Field(
        default_factory = list,
        description = "Claims in the draft that are not supported by the evidence."
    )
    missed_supported_points: list[str] = Field(
        default_factory = list,
        description = "Important supported points from the evidence that the draft "
        "omitted."
    )

class State(TypedDict):
    question: str
    evidence: str
    draft: str
    final_answer: str
    verdict: Literal["", "accept", "revise", "abstain"]
    answers_question: bool
    abstention_justified: bool
    feedback: str
    unsupported_claims: list[str]
    missed_supported_points: list[str]
    attempts: int
    max_attempts: int
    trace: Annotated[list[str], add]

writer = init_chat_model("openai:gpt-4.1-mini", temperature = 0)
verifier = init_chat_model("openai:gpt-4.1-mini", temperature = 0)
    .with_structured_output(Verification)
```



```

def draft_answer(state: State) -> dict:
    msg = writer.invoke(
        f"""You are a customer support assistant.
        Answer the question using the evidence only.
        Do not invent policy details.
        Prefer a direct answer when the evidence supports one.
        If the evidence supports only a partial, limited, or conditional answer, give that answer
        clearly.
        Use "{SAFE_ABSTENTION}" only when the evidence does not support even a partial or
        conditional answer.

        Question:
        {state["question"]}

        Evidence:
        {state["evidence"]}

        Previous verifier feedback:
        {state["feedback"]} if state["feedback"] else "None"
        """
    )
    return {
        "draft": msg.content,
        "trace": [f'draft_answer: produced draft={msg.content!r}']
    }

def verify_answer(state: State) -> dict:
    result = verifier.invoke(
        f"""Evaluate the draft answer against the question and the evidence.

        Decision rules:
        - accept:
            * every non-trivial claim in the draft is supported by the evidence, and
            * the draft answers the question as far as the evidence allows.
        - revise:
            * the draft can be improved using the same evidence, including cases where:
              - the draft abstains but the evidence supports a partial or conditional answer,
              - the draft omits important supported conditions or exceptions,
              - the draft contains unsupported claims that can be removed.
        - abstain:
            * only if the evidence truly does not support even a partial or conditional answer.

        Question:
        {state["question"]}

        Evidence:
        {state["evidence"]}

        Draft:
        {state["draft"]}
        """
    )
    return {
        "verdict": result.verdict,
        "answers_question": result.answers_question,
        "abstention_justified": result.abstention_justified,
        "feedback": result.feedback,
        "unsupported_claims": result.unsupported_claims,
        "missed_supported_points": result.missed_supported_points,
        "trace": [
            "verify_answer: "
            f"verdict = {result.verdict}, "
            f"answers_question = {result.answers_question}, "
            f"abstention_justified = {result.abstention_justified}, "
            f"unsupported_claims = {result.unsupported_claims}, "
            f"missed_supported_points = {result.missed_supported_points}"
        ]
    }

def revise_answer(state: State) -> dict:
    msg = writer.invoke(
        f"""Revise the answer strictly from the evidence.

        Rules:
        - Remove unsupported claims.
        - Add any missing supported points that are necessary to answer the question.
        - If the evidence supports only a conditional or limited answer, say that explicitly.
        - Use "{SAFE_ABSTENTION}" only if the evidence truly does not support even a partial or
        conditional answer.

        Unsupported claims to remove:

```



```

{state["unsupported_claims"] if state["unsupported_claims"] else "None"}

Supported points you missed:
{state["missed_supported_points"] if state["missed_supported_points"] else "None"}

Verifier feedback:
{state["feedback"]}

Question:
{state["question"]}

Evidence:
{state["evidence"]}

Current draft:
{state["draft"]}
"""
    )

    next_attempts = state["attempts"] + 1
    return {
        "draft": msg.content,
        "attempts": next_attempts,
        "trace": [
            f"revise_answer: attempts={next_attempts}, revised_draft={msg.content!r}"
        ]
    }

def accept_answer(state: State) -> dict:
    return {
        "final_answer": state["draft"],
        "trace": ["accept_answer: final draft accepted"]
    }

def abstain(state: State) -> dict:
    return {
        "final_answer": SAFE_ABSTENTION,
        "trace": ["abstain: evidence truly insufficient or revision budget exhausted"]
    }

def route_after_verification(state: State) -> str:
    # Accept only when the verifier explicitly says the draft both:
    # 1) is supported by evidence, and
    # 2) answers the question as far as the evidence allows.
    if state["verdict"] == "accept" and state["answers_question"]:
        return "accept"

    # Only abstain when the verifier says abstention is actually justified.
    if state["verdict"] == "abstain" and state["abstention_justified"]:
        return "abstain"

    # If we have already used the allowed revision budget, fail closed.
    if state["attempts"] >= state["max_attempts"]:
        return "abstain"

    return "revise"

builder = StateGraph(State)
builder.add_node("draft_answer", draft_answer)
builder.add_node("verify_answer", verify_answer)
builder.add_node("revise_answer", revise_answer)
builder.add_node("accept_answer", accept_answer)
builder.add_node("abstain", abstain)

builder.add_edge(START, "draft_answer")
builder.add_edge("draft_answer", "verify_answer")
builder.add_conditional_edges(
    "verify_answer",
    route_after_verification,
    {
        "accept": "accept_answer",
        "revise": "revise_answer",
        "abstain": "abstain"
    }
)
builder.add_edge("revise_answer", "verify_answer")
builder.add_edge("accept_answer", END)
builder.add_edge("abstain", END)

graph = builder.compile()

if __name__ == "__main__":
    result = graph.invoke(
        {

```

```

        "question": "Can this gift order be refunded to the original payment
                      method?",
        "evidence": """
Gift purchases: eligible refunds are issued as store credit unless the original purchaser
requests the refund.
Standard items: refundable within 30 days if unopened.
""",
        "draft": "",
        "final_answer": "",
        "verdict": "",
        "answers_question": False,
        "abstention_justified": False,
        "feedback": "",
        "unsupported_claims": [],
        "missed_supported_points": [],
        "attempts": 0,
        "max_attempts": 1,
        "trace": []
    }
)

print("VERDICT:", result["verdict"])
print("FINAL ANSWER:")
print(result["final_answer"])

print("\nTRACE:")
for step in result["trace"]:
    print("-", step)

```

vol3_ch_04\scr\fail_closed_verifier.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.fail_closed_verifier

VERDICT: accept
FINAL ANSWER:
Gift orders are eligible for refunds as store credit unless the original purchaser
requests the refund. Therefore, this gift order can be refunded to the original payment
method only if the original purchaser requests it.

TRACE:
- draft_answer: produced draft='Gift orders are eligible for refunds as store credit
unless the original purchaser requests the refund. Therefore, this gift order can be
refunded to the original payment method only if the original purchaser requests it.'
- verify_answer: verdict=accept, answers_question=True, abstention_justified=False,
unsupported_claims=[], missed_supported_points=[]
- accept_answer: final draft accepted

```

Two aspects of the example matter most. First, the verifier returns a typed control signal, so the graph does not have to interpret vague prose to choose the next route. Second, the graph has an explicit abstention route and a bounded revision budget. If the answer remains unacceptable after that budget is exhausted, the workflow returns the safe abstention instead of improvising. In this example, that is the fail-closed behavior: insufficient evidence or exhausted revisions lead to a safe terminal state. The verifier is also anchored to the evidence and the abstention rule rather than to the draft alone.

4.6.7 A small evaluation harness

A verifier loop helps at runtime, but it cannot show whether the system improves across versions. The next example uses separate development and held-out cases, evaluates several signals, and records production failures for later review. A case remains held out only while it is excluded from prompt, model, and workflow tuning. Once it is used to guide a change, it should be moved into the regression set and replaced with a new unseen case. The script is intentionally small and does not replace a complete evaluation platform.

```

from __future__ import annotations
import json
from collections import Counter
from dataclasses import dataclass, asdict
from pathlib import Path
from statistics import mean

```

```

from textwrap import indent
from typing import Any, Callable
from langchain.chat_models import init_chat_model
from .config import OPENAI_API_KEY
from .fail_closed_verifier import SAFE_ABSTENTION, graph

@dataclass
class EvalCase:
    case_id: str
    question: str
    evidence: str
    expected_mode: str # "answer" or "abstain"
    required_phrases: list[str]
    forbidden_phrases: list[str]

DEV_CASES = [
    EvalCase(
        case_id = "dev-001",
        question = "Can this gift order be refunded to the original payment method?",
        evidence = """
            Gift purchases: eligible refunds are issued as store credit unless the
            original purchaser requests the refund.
            Standard items: refundable within 30 days if unopened.
            """.strip(),
        expected_mode = "answer",
        required_phrases = ["store credit", "original purchaser requests the refund"],
        forbidden_phrases = ["cash refund for any gift order"]
    ),
    EvalCase(
        case_id = "dev-002",
        question = "Is this order refundable?",
        evidence = "",
        expected_mode = "abstain",
        required_phrases = ["not enough evidence"],
        forbidden_phrases = ["eligible", "not eligible"]
    )
]

TEST_CASES = [
    EvalCase(
        case_id = "test-001",
        question = "Can an opened item be refunded within 30 days?",
        evidence = "Standard items: refundable within 30 days if unopened.",
        expected_mode = "answer",
        required_phrases = ["if unopened"],
        forbidden_phrases = ["opened items are refundable"]
    ),
    EvalCase(
        case_id = "test-002",
        question = "Can I promise a refund timeline?",
        evidence = "Refunds are issued within 5 business days after inspection.",
        expected_mode = "answer",
        required_phrases = ["5 business days after inspection"],
        forbidden_phrases = ["same day", "immediate refund"]
    )
]

naive_model = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

def naive_candidate(question: str, evidence: str) -> str:
    msg = naive_model.invoke(
        f"""You are a support assistant.
        Answer the question.

        Question:
        {question}

        Evidence:
        {evidence}
        """
    )
    return msg.content

def fail_closed_candidate(question: str, evidence: str) -> dict[str, Any]:
    state = graph.invoke(
        {
            "question": question,
            "evidence": evidence,
            "draft": "",
            "final answer": "",
            "verdict": "",
        }
    )

```

```

        "answers_question": False,
        "abstention_justified": False,
        "feedback": "",
        "unsupported claims": [],
        "missed_supported_points": [],
        "attempts": 0,
        "max_attempts": 1,
        "trace": []
    }
)
return {
    "output": state["final_answer"],
    "predicted_mode": classify_mode(state["final_answer"]),
    "verdict": state["verdict"],
    "trace": state.get("trace", [])
}

def classify_mode(text: str) -> str:
    normalized = text.lower().strip()
    if SAFE_ABSTENTION.lower() in normalized:
        return "abstain"
    if "not enough evidence" in normalized:
        return "abstain"
    return "answer"

def normalize_candidate_result(raw: str | dict[str, Any]) -> dict[str, Any]:
    if isinstance(raw, str):
        return {
            "output": raw,
            "predicted_mode": classify_mode(raw),
            "trace": [],
            "verdict": None
        }

    output = str(raw.get("output", ""))
    predicted_mode = raw.get("predicted_mode") or classify_mode(output)
    trace = raw.get("trace", [])
    verdict = raw.get("verdict")

    return {
        "output": output,
        "predicted_mode": predicted_mode,
        "trace": trace,
        "verdict": verdict
    }

def evaluate_case(candidate: Callable[[str, str], str | dict[str, Any]], case: EvalCase)
-> dict[str, Any]:
    raw = candidate(case.question, case.evidence)
    normalized = normalize_candidate_result(raw)
    output = normalized["output"]

    mode_ok = normalized["predicted_mode"] == case.expected_mode
    required_ok = all(p.lower() in output.lower() for p in case.required_phrases)
    forbidden_ok = all(p.lower() not in output.lower() for p in case.forbidden_phrases)

    passed = mode_ok and required_ok and forbidden_ok

    missing_required = [
        phrase for phrase in case.required_phrases
        if phrase.lower() not in output.lower()
    ]
    present_forbidden = [
        phrase for phrase in case.forbidden_phrases
        if phrase.lower() in output.lower()
    ]

    return {
        "case_id": case.case_id,
        "question": case.question,
        "evidence": case.evidence,
        "expected_mode": case.expected_mode,
        "predicted_mode": normalized["predicted_mode"],
        "required_phrases": case.required_phrases,
        "forbidden_phrases": case.forbidden_phrases,
        "missing_required": missing_required,
        "present_forbidden": present_forbidden,
        "mode_ok": mode_ok,
        "required_ok": required_ok,
        "forbidden_ok": forbidden_ok,
        "passed": passed,
        "output": output,
    }

```

```

        "trace": normalized["trace"],
        "verdict": normalized["verdict"]
    }

def run_suite(
    name: str,
    candidate: Callable[[str, str], str | dict[str, Any]],
    cases: list[EvalCase],
    repetitions: int = 1
) -> dict[str, Any]:
    results: list[dict[str, Any]] = []

    for _ in range(repetitions):
        for case in cases:
            results.append(evaluate_case(candidate, case))

    pass_rate = mean(1.0 if r["passed"] else 0.0 for r in results)

    return {
        "name": name,
        "pass_rate": round(pass_rate, 3),
        "results": results
    }

def failure_reasons(result: dict[str, Any]) -> list[str]:
    reasons = []

    if not result["mode_ok"]:
        reasons.append("wrong mode")
    if not result["required_ok"]:
        reasons.append("missing required_phrase")
    if not result["forbidden_ok"]:
        reasons.append("contains_forbidden_phrase")

    return reasons

def print_score_line(suite: dict[str, Any]) -> None:
    total = len(suite["results"])
    passed = sum(1 for r in suite["results"] if r["passed"])
    print(f"{suite['name']} pass_rate={suite['pass_rate']:.3f} ({passed}/{total})")

def print_suite_report(suite: dict[str, Any]) -> None:
    print(f"{suite['name']} pass_rate={suite['pass_rate']:.3f}")
    counter = Counter()

    for result in suite["results"]:
        reasons = failure_reasons(result)
        status = "PASS" if result["passed"] else "FAIL"

        if result["passed"]:
            counter["passed"] += 1
        else:
            counter["failed"] += 1
            for reason in reasons:
                counter[reason] += 1

    print(f" {result['case_id']}: {status}")
    print(f" question: {result['question']}")
    print(f" expected_mode={result['expected_mode']}")
    print(f" predicted_mode={result['predicted_mode']}")

    print(
        " checks: "
        f"mode_ok={result['mode_ok']} "
        f"required_ok={result['required_ok']} "
        f"forbidden_ok={result['forbidden_ok']} "
    )
    print(f" required_phrases={result['required_phrases']}")
    print(f" forbidden_phrases={result['forbidden_phrases']}")

    if result["missing_required"]:
        print(f" missing_required={result['missing_required']}")
    if result["present_forbidden"]:
        print(f" present_forbidden={result['present_forbidden']}")
    if reasons:
        print(f" why={', '.join(reasons)}")
    else:
        print(f" why=-")

    if result["verdict"] is not None:
        print(f" verifier_verdict={result['verdict']}")

    print(f" output:")
    print(indent(result["output"], " "))

```

```

        if result["trace"]:
            print("    trace:")
            for step in result["trace"]:
                print(f"        - {step}")

            print()

    print("    summary:")
    print(f"        passed={counter['passed']} failed={counter['failed']}")
    if counter["failed"]:
        print(
            "        failure_breakdown="
            f"wrong_mode={counter['wrong_mode']}, "
            f"missing_required_phrase={counter['missing_required_phrase']}, "
            f"contains_forbidden_phrase={counter['contains_forbidden_phrase']}"
        )
    print()

def compare_variants() -> None:
    naive_dev = run_suite("naive-dev", naive_candidate, DEV_CASES)
    safe_dev = run_suite("safe-dev", fail_closed_candidate, DEV_CASES)
    naive_test = run_suite("naive-test", naive_candidate, TEST_CASES)
    safe_test = run_suite("safe-test", fail_closed_candidate, TEST_CASES)

    print("EVALUATION LOGIC")
    print("    pass = mode_ok AND required_ok AND forbidden_ok")
    print("    mode_ok = predicted answer/abstain matches expected_mode")
    print("    required_ok = all required phrases appear in output")
    print("    forbidden_ok = no forbidden phrases appear in output")
    print()

    print("SCORECARD")
    print("    DEVELOPMENT")
    print_score_line(naive_dev)
    print_score_line(safe_dev)
    print()
    print("    HELD-OUT TEST")
    print_score_line(naive_test)
    print_score_line(safe_test)
    print()

    print("DETAILED RESULTS")
    print_suite_report(naive_dev)
    print_suite_report(safe_dev)
    print_suite_report(naive_test)
    print_suite_report(safe_test)

def append_production_failures(path: Path, failures: list[EvalCase]) -> None:
    path.parent.mkdir(parents = True, exist_ok = True)
    with path.open("a", encoding = "utf-8") as f:
        for case in failures:
            f.write(json.dumps(asdict(case)) + "\n")

if __name__ == "__main__":
    compare_variants()

    new_failures = [
        EvalCase(
            case_id = "prod-2026-04-01-001",
            question = "Can I promise a cash refund for a gift order?",
            evidence = (
                "Gift purchases: eligible refunds are issued as store credit "
                "unless the original purchaser requests the refund."
            ),
            expected_mode = "answer",
            required_phrases = ["store credit"],
            forbidden_phrases = ["cash refund"]
        )
    ]

    append_production_failures(Path("data/fresh_failures.jsonl"), new_failures)

```

vol3_ch_04\src\evaluation_harness.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_04> python -m src.evaluation_harness

EVALUATION LOGIC
pass = mode_ok AND required_ok AND forbidden_ok
mode_ok = predicted answer/abstain matches expected_mode
required_ok = all required phrases appear in output
forbidden_ok = no forbidden phrases appear in output

```

```

SCORECARD
DEVELOPMENT
  naive-dev  pass_rate=0.000  (0/2)
  safe-dev   pass_rate=0.500  (1/2)

HELD-OUT TEST
  naive-test pass_rate=0.500  (1/2)
  safe-test  pass_rate=0.500  (1/2)

DETAILED RESULTS
naive-dev  pass_rate=0.000
dev-001: FAIL
  question: Can this gift order be refunded to the original payment method?
  expected_mode=answer predicted_mode=answer
  checks: mode ok=True required ok=False forbidden ok=True
  required_phrases=['store credit', 'original purchaser requests the refund']
  forbidden_phrases=['cash refund for any gift order']
  missing_required=['original purchaser requests the refund']
  why=missing_required_phrase
  output:
    No, gift orders are generally refunded as store credit unless the original
    purchaser specifically requests a refund to the original payment method.

dev-002: FAIL
  question: Is this order refundable?
  expected_mode=abstain predicted_mode=answer
  checks: mode ok=False required ok=False forbidden ok=True
  required_phrases=['not enough evidence']
  forbidden_phrases=['eligible', 'not eligible']
  missing_required=['not enough evidence']
  why=wrong_mode, missing_required_phrase
  output:
    I don't see any information about the order or its refund policy. Could you please
    provide more details about the order or the refund terms?

summary:
  passed=0 failed=2
  failure_breakdown=wrong_mode=1, missing_required_phrase=2,
  contains_forbidden_phrase=0

safe-dev  pass_rate=0.500
dev-001: PASS
  question: Can this gift order be refunded to the original payment method?
  expected_mode=answer predicted_mode=answer
  checks: mode ok=True required ok=True forbidden ok=True
  required_phrases=['store credit', 'original purchaser requests the refund']
  forbidden_phrases=['cash refund for any gift order']
  why=-
  verifier_verdict=accept
  output:
    Gift orders are refunded as store credit unless the original purchaser requests the
    refund to the original payment method.
  trace:
    - draft_answer: produced draft='Gift orders are refunded as store credit unless the
      original purchaser requests the refund to the original payment
      method.'
    - verify_answer: verdict=accept, answers_question=True, abstention_justified=False,
      unsupported_claims=[], missed_supported_points=[]
    - accept_answer: final draft accepted

dev-002: FAIL
  question: Is this order refundable?
  expected_mode=abstain predicted_mode=abstain
  checks: mode ok=True required ok=False forbidden ok=True
  required_phrases=['not enough evidence']
  forbidden_phrases=['eligible', 'not eligible']
  missing_required=['not enough evidence']
  why=missing_required_phrase
  verifier_verdict=abstain
  output:
    I do not have enough evidence to answer safely.
  trace:
    - draft_answer: produced draft='I do not have enough evidence to answer safely.'
    - verify_answer: verdict=abstain, answers_question=False,
      abstention_justified=True, unsupported_claims=[],
      missed_supported_points=[]
    - abstain: evidence truly insufficient or revision budget exhausted

```



```

summary:
  passed=1 failed=1
  failure_breakdown=wrong mode=0, missing required phrase=1,
    contains_forbidden_phrase=0

naive-test pass_rate=0.500
test-001: FAIL
  question: Can an opened item be refunded within 30 days?
  expected_mode=answer predicted_mode=answer
  checks: mode_ok=True required_ok=False forbidden_ok=True
  required_phrases=['if unopened']
  forbidden_phrases=['opened items are refundable']
  missing_required=['if unopened']
  why=missing_required_phrase
  output:
    Based on the evidence provided, an opened item cannot be refunded within 30 days.
    Only unopened standard items are refundable within that period.

test-002: PASS
  question: Can I promise a refund timeline?
  expected_mode=answer predicted_mode=answer
  checks: mode_ok=True required_ok=True forbidden_ok=True
  required_phrases=['5 business days after inspection']
  forbidden_phrases=['same day', 'immediate refund']
  why=-
  output:
    You should avoid promising an exact refund timeline. While refunds are typically
    issued within 5 business days after inspection, the inspection process duration can
    vary. It's best to inform customers that refunds are processed promptly after
    inspection, which usually takes up to 5 business days.

summary:
  passed=1 failed=1
  failure_breakdown=wrong_mode=0, missing_required_phrase=1,
    contains_forbidden_phrase=0

safe-test pass_rate=0.500
test-001: FAIL
  question: Can an opened item be refunded within 30 days?
  expected_mode=answer predicted_mode=answer
  checks: mode_ok=True required_ok=False forbidden_ok=True
  required_phrases=['if unopened']
  forbidden_phrases=['opened items are refundable']
  missing_required=['if unopened']
  why=missing_required_phrase
  verifier_verdict=accept
  output:
    An opened item cannot be refunded within 30 days, as only unopened standard items
    are refundable within that period.
  trace:
    - draft_answer: produced draft='An opened item cannot be refunded within 30 days,
      as only unopened standard items are refundable within that period.'
    - verify_answer: verdict=accept, answers_question=True, abstention_justified=False,
      unsupported_claims=[], missed_supported_points=[]
    - accept_answer: final draft accepted

test-002: PASS
  question: Can I promise a refund timeline?
  expected_mode=answer predicted_mode=answer
  checks: mode_ok=True required_ok=True forbidden_ok=True
  required_phrases=['5 business days after inspection']
  forbidden_phrases=['same day', 'immediate refund']
  why=-
  verifier_verdict=accept
  output:
    You can inform the customer that refunds are issued within 5 business days after
    inspection, but you should not promise a specific timeline beyond that.
  trace:
    - draft_answer: produced draft='You can inform the customer that refunds are issued
      within 5 business days after inspection, but you should not promise a specific
      timeline beyond that.'
    - verify_answer: verdict=accept, answers_question=True, abstention_justified=False,
      unsupported_claims=[], missed_supported_points=[]
    - accept_answer: final draft accepted

summary:
  passed=1 failed=1

```

```
failure_breakdown=wrong_mode=0, missing_required_phrase=1,
contains_forbidden_phrase=0
```

This harness demonstrates three useful habits. First, development and held-out cases serve different purposes: use the development or regression set to guide changes and the held-out set to test whether those changes generalize. If performance improves only on the development set, treat the result with suspicion. Second, separate checks make it easier to understand why a case failed instead of hiding every failure inside one aggregate score. Third, confirmed production failures should be reviewed and added to the regression set rather than automatically added to the protected held-out set. In larger systems, compare variants on the same cases, repeat stochastic runs when needed, inspect individual examples, and keep a small number of release metrics alongside broader diagnostics. The purpose is not to make evaluation elaborate, but to make false improvement harder to mistake for real improvement.

4.6.8 Practical design rules

A reflective agent remains reliable only when the surrounding system keeps optimization aligned with the real task.

- Keep the verifier explicit, typed, and bounded. The graph must distinguish between an acceptable result, a result that can be revised, and a result that is unsafe or unsupported.
- Treat self-critique as an opportunity for improvement, not as proof of correctness. Ground revision in evidence, deterministic rules, tests, or approval paths.
- Do not rely on one evaluator. A fixed judge can have systematic biases, and its behavior can change when its model, prompt, or configuration changes. Inspect individual examples, compare variants on the same cases, use deterministic checks where possible, and retain human review for sensitive decisions.
- Monitor more than the final answer. Track execution traces, tool use, latency, token cost, runtime anomalies, and business outcomes. Degradation may appear in the execution path before it becomes visible in the final response.
- Assume change is normal. Monitor context quality, request distribution, policies, tools, retrieval pipelines, and evaluator behavior separately so that you can identify the source of a regression.
- Protect evaluation boundaries. Repeatedly tuning against one static set turns it into development data. Add reviewed failures to the regression set, preserve a separate held-out set, and periodically introduce new unseen examples.
- Fail closed on high-impact actions. Approval paths, rejection routes, call limits, and early termination prevent local model confidence from becoming external damage.
- Make pause-and-resume workflows safe. Resumed execution must follow predictable control flow, while API calls and other side effects should be idempotent or protected by deduplication and compensation mechanisms.

The central lesson is that degradation is not prevented by adding more reflection. Reflection remains useful only when it is bounded by evidence, explicit stop conditions, protected evaluation sets, risk-based controls, and production monitoring. A reliable adaptive system revises outputs within those constraints and verifies that the resulting improvement transfers to the environment it actually serves.

4.7 SUMMARY

This chapter examined how intelligent systems can evaluate their own results, explore alternatives, use operational feedback, and change behaviour without surrendering control. The main lesson is that a first model response should be treated as a candidate rather than an accepted result whenever evidence, policy, safety, numerical correctness, or external action matters. Reflection and verification provide the first level of control. Reflection can improve qualities such as clarity, structure, tone, and completeness. Verification determines whether a result satisfies explicit acceptance criteria. The strongest verifier is not necessarily another model call. Exact requirements should be enforced with deterministic code. Factual claims should be checked against authorized evidence. Tool execution, tests, system-of-record data, authorization controls, and human approval should be used when they provide stronger authority. Repeating workflows must have typed decisions, attempt limits, observable stopping conditions, and fail-closed outcomes such as clarification, abstention, rejection, or escalation.

Search-based reasoning extends this idea by evaluating more than one possible solution. Best-of-N compares complete candidates. Self-consistency combines normalized answers when independent reasoning paths should converge on the same result. Tree-of-Thought search develops and prunes visible partial solutions. MCTS-style approaches add repeated branch selection, outcome estimation, and a balance between exploration and exploitation. These techniques are useful only when the evaluator can recognize genuine quality. The practical strategy is therefore to begin with a simple baseline and add search only when measured improvements justify the additional latency, model calls, cost, and operational complexity.

Feedback creates an improvement loop across executions. Ratings, reason labels, reviewer judgments, edits, clicks, conversions, purchases, and trading outcomes can all reveal useful patterns, but they have different meanings and levels of reliability. A user reaction is not a confirmed defect, and an observed outcome is not proof that one model decision caused it. Raw signals should remain separate from reviewed findings and approved changes. Once validated, important cases can become regression examples, evaluators, prompt or retrieval revisions, routing rules, operating constraints, or carefully governed training data.

Runtime adaptation changes what happens during an execution. Trusted invocation-specific values belong in runtime context. Mutable information about the current workflow belongs in state. Compact, validated information that must be reused across threads belongs in an appropriate store. Model context can adapt instructions, model choice, tools, and output schemas. Retrieval policy can change query formulation, candidate generation, filtering, diversification, and reranking. Graph control can branch, retry, interrupt, resume, escalate, and recover. The safest design adapts at the smallest control surface that solves the problem and records what changed, why it changed, and how long the change should remain valid.

Finally, the chapter showed why an adaptive system can appear to improve while becoming less useful. Reward hacking occurs when the system optimizes an imperfect score rather than the real objective. Drift occurs when data, policies, tools, models, or user behaviour change. Evaluation-set overfitting occurs when repeated tuning improves familiar examples without transferring to new cases. These risks cannot be solved by adding more reflection. They require

protected evaluation sets, representative production data, independent checks, versioned components, risk-based release controls, production monitoring, and the ability to roll back harmful changes. The result is a practical model of controlled intelligence. Generate candidates, evaluate them with the strongest appropriate evidence, adapt only within explicit boundaries, and preserve a safe exit when confidence is insufficient. The objective is not to make the system endlessly self-correcting or autonomous. It is to make every important change of direction visible, testable, and accountable.

5 MULTI-AGENT COLLABORATION

A single agent can handle many useful tasks, but its design becomes increasingly difficult as responsibilities accumulate. A broad agent may need to understand several business domains, choose among unrelated tools, retain large amounts of context, enforce different policies, and recognise when an action requires approval. Adding more instructions to the prompt may appear to solve the problem temporarily, but it usually makes the agent harder to control, evaluate, and secure. At some point, the problem is no longer the capability of the model. It is the architecture surrounding it. Multi-agent systems address this problem by dividing responsibility among components with clearer roles. One agent may coordinate the overall request, while other agents handle billing, returns, policy review, or exception management. Several specialists may work independently and contribute to one result. A reviewer may challenge an answer before it is accepted. A case may be escalated when the system reaches the boundary of its authority. The objective is not to simulate a team of people or create conversation between agents for its own sake. The objective is to make responsibility, context, control, and approval boundaries explicit.

This separation introduces a new set of engineering decisions. It is not enough to decide which agents exist. You must also determine who controls the next step, what information each agent receives, how intermediate results are represented, how concurrent updates are combined, and when the workflow must stop. A handoff transfers responsibility from one specialist to another. A shared message list preserves the conversational path. A shared state schema provides machine-readable channels for facts, decisions, identifiers, and results. When agents run in separate services, an agent-to-agent protocol replaces direct access to in-process state with explicit network messages, capabilities, and correlation identifiers. LangGraph provides the execution model needed to make these interactions visible. Nodes represent units of work, edges express possible transitions, and shared state carries information between steps. Parallel branches execute against the same incoming state snapshot and converge only after their updates have been merged. Reducers define how concurrent updates to the same state field are combined, while later nodes remain responsible for interpreting what those combined results mean. Explicit routes, execution budgets, interrupts, checkpoints, and resume commands distinguish successful completion from bounded failure, human review, and continuation after a pause. This chapter develops these ideas through one recurring customer-support scenario. Returns, billing, coordination, review, and escalation remain consistent while the collaboration mechanism changes. You will compare manager/worker systems, specialist routing, structured debate, escalation, handoffs, shared messages, typed state, remote A2A communication, and coordinated LangGraph workflows. The goal is to help you choose the simplest multi-agent architecture that creates meaningful boundaries without introducing unnecessary cost, latency, or operational complexity.

5.1 MULTI-AGENT PATTERNS: MANAGER/WORKER, SPECIALISTS, DEBATE, ESCALATION

Multi-agent collaboration is easiest to understand by starting with the limits of a single general-purpose agent. A single agent is often enough when the task stays within one domain, the toolset is small, and the workflow is short. Multi-agent design becomes useful when one request crosses domains, requires different operational responsibilities, or contains actions with different safety requirements. A support request, for example, may involve billing, returns, and an exception that requires review. The purpose of using several agents is therefore separation of concerns, not novelty. Start with the simplest architecture that can perform the work reliably. Multi-agent orchestration adds model calls, state transitions, latency, cost, and additional failure modes. Use it when one agent becomes difficult to control because its prompt is too broad, its tool choices are noisy, its context is overloaded, or parts of the task require different security or approval boundaries.

There are many ways to structure collaboration. Some systems are hierarchical, with one coordinator directing workers. Some use specialists that own narrow domains, tools, or access boundaries. Others introduce structured review, where several agents compare or challenge candidate answers. Escalation moves a case to a path with greater authority or stronger oversight when local handling is no longer appropriate. This section organizes those concerns into four conceptual categories: manager/worker, specialization, debate or review, and escalation. These categories are not a one-to-one list of framework APIs. In LangChain and LangGraph, they can be implemented through subagents, routers, skills, handoffs, or custom workflows. They can also be combined within the same application.

5.1.1 Manager/worker: one agent coordinates, others execute

The manager/worker pattern is the clearest multi-agent shape to reason about. One agent owns the overall objective, decides how to break the task down, delegates work to narrower agents, and then combines their results into the final outcome. LangChain's subagents pattern describes this as a central main agent, often called a supervisor, coordinating subagents by calling them as tools. The main agent decides which subagent to invoke, what input to send, and how to combine the returned results. In the standard pattern used here, each subagent invocation starts with fresh state, while the main agent maintains the ongoing conversation memory and overall workflow context. This centralized shape has several practical advantages. The first is context isolation. Each worker runs in a narrow, clean context instead of inheriting the entire conversation and every available tool. That keeps specialist prompts smaller, reduces distraction, and makes each worker easier to test. The second is control. Because all routing passes through one coordinator, the system has one place where delegation logic is visible. The third is modularity. Workers can evolve independently as long as their callable contract stays stable. If you decide to replace the billing specialist or strengthen the returns specialist, the manager does not need to be redesigned from scratch.

The trade-off is that the manager becomes the main coordination point. That is often a good trade when you want clear control, but it can also become a bottleneck. A centralized manager adds at least one more layer of reasoning, and if the design is poor, it may spend time over-coordinating simple work that a single specialist could have handled directly. This is why

manager/worker is strongest when the overall task genuinely benefits from one agent seeing the whole picture while other agents stay narrow. It is much less attractive when the task is simple enough that delegation itself becomes the expensive part. A second form of the same idea appears in LangGraph's orchestrator-worker workflow. Here the orchestrator does not merely choose between a few fixed specialists. It can dynamically create workers for subtasks and then synthesize their outputs. LangGraph's Send API is designed for exactly this kind of decomposition. That makes the pattern especially useful when the number of subtasks depends on the input, such as generating several report sections, analysing several documents, or breaking one broad request into multiple parallel branches of work. At this level, manager/worker stops being just "call the right specialist" and becomes a general decomposition pattern.

A useful mental model is that the manager is primarily responsible for decomposition, delegation, and synthesis, while workers perform specialist tasks. The manager still needs enough domain context to route work and determine whether the combined result addresses the overall objective. It should not, however, duplicate every specialist's instructions, tools, and detailed responsibilities.

5.1.2 Shared setup used by the examples in this chapter

To keep the next examples linked to one another, the chapter uses one small support-team codebase. The shared module defines the recurring policies, the specialist agents, the simple tool wrappers used by the manager, the structured schemas used by the router and the debate judge, and the manual-review tool used later in the escalation example. This keeps the code progression honest: each example changes the collaboration pattern, not the domain. The implementation uses `init_chat_model` for model initialization, `create_agent` for the agent runtime, `@tool` for tool exposure, `with_structured_output` for typed routing and judging, and `interrupt()` for the explicit escalation boundary. These are the LangChain and LangGraph APIs used throughout the examples in this chapter.

```
from __future__ import annotations
from typing import Literal
from pydantic import BaseModel, Field
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langgraph.types import interrupt
from .config import OPENAI_API_KEY

MODEL = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

RETURNS_POLICY = """
Returns policy:
- Unopened items can be returned within 30 days of delivery.
- Damaged items can be refunded or replaced, even if the box was opened.
- Gift returns are eligible for store credit unless the item arrived damaged.
- Requests outside these rules must be escalated for manual review.
"""

BILLING_POLICY = """
Billing policy:
- Invoices are available in the customer account.
- Duplicate charges must be investigated before any refund is issued.
- Charge disputes and manual billing overrides must be escalated.
"""

class SpecialistRoute(BaseModel):
    specialist: Literal["returns", "billing"] = Field(
        description = "The specialist that should answer the request."
    )
    rewritten_question: str = Field(
```



```

        description = "A short standalone question written for the chosen specialist."
    )

class DebateVerdict(BaseModel):
    decision: Literal["refund", "store credit", "escalate"] = Field(
        description = "Final decision after hearing both sides."
    )
    rationale: str = Field(
        description = "Short justification for the final decision."
    )
    )

def run_agent(agent, user_text: str, *, config: dict | None = None) -> str:
    result = agent.invoke(
        {"messages": [{"role": "user", "content": user_text}]},
        config = config
    )
    last = result["messages"][-1]
    return getattr(last, "text", last.content)

returns_agent = create_agent(
    model = MODEL,
    tools = [],
    system_prompt = (
        "You are the returns specialist.\n"
        "Answer only questions about refunds, replacements, damaged items, "
        "gift returns, and return windows.\n"
        "Use only the policy below.\n\n"
        f"{RETURNS_POLICY}\n\n"
        "If the request falls outside policy, say clearly that escalation is required."
    )
)

billing_agent = create_agent(
    model = MODEL,
    tools = [],
    system_prompt = (
        "You are the billing specialist.\n"
        "Answer only questions about invoices, charges, payments, and billing\n"
        "disputes. Use only the policy below.\n\n"
        f"{BILLING_POLICY}\n\n"
        "If the request cannot be resolved safely from policy, say clearly that "
        "escalation is required."
    )
)

customer_advocate = create_agent(
    model = MODEL,
    tools = [],
    system_prompt = (
        "You are the customer advocate in a support debate.\n"
        "Use the returns policy below, but look for the most customer-friendly "
        "interpretation that is still defensible.\n\n"
        f"{RETURNS_POLICY}\n\n"
        "Be concise and concrete."
    )
)

policy_guard = create_agent(
    model = MODEL,
    tools = [],
    system_prompt = (
        "You are the policy guard in a support debate.\n"
        "Use the returns policy below and prefer the narrowest safe interpretation.\n\n"
        f"{RETURNS_POLICY}\n\n"
        "Be concise and concrete."
    )
)

@tool("ask_returns_specialist")
def ask_returns_specialist(question: str) -> str:
    """Use for refunds, replacements, damaged items, gift returns, and return windows."""
    return run_agent(returns_agent, question)

@tool("ask_billing_specialist")
def ask_billing_specialist(question: str) -> str:
    """Use for invoices, duplicate charges, payments, and billing disputes."""
    return run_agent(billing_agent, question)

@tool("escalate_to_human")
def escalate_to_human(reason: str) -> str:
    """Use when the case is ambiguous, outside policy, high-risk, or needs human
    judgment."""

```

```

    return f"Escalated to human support queue. Reason: {reason}"

@tool
def request_manual_refund_approval(order_id: str, reason: str) -> str:
    """Request human approval for a refund outside normal policy."""
    decision = interrupt(
        {
            "kind": "manual_refund_review",
            "order_id": order_id,
            "reason": reason,
            "allowed responses": ["approve", "reject"]
        }
    )

    if decision == "approve":
        return f"Manual refund approved for {order_id}."
    return f"Manual refund rejected for {order_id}."

def build_manual_review_agent(checkpointer):
    return create_agent(
        model = MODEL,
        tools = [request_manual_refund_approval],
        system_prompt = (
            "You are the escalation agent.\n"
            "Use request_manual_refund_approval only when the customer is asking "
            "for an exception outside normal return policy."
        ),
        checkpointer = checkpointer
    )

```

vol3_ch_05\scr\support_team_shared.py

This module establishes a reusable foundation for the collaboration patterns that follow. It defines one support domain, one model interface, and a small set of agents, schemas, and tool wrappers so that later examples can focus on coordination rather than repeatedly rebuilding the same components. The returns and billing policies provide a consistent reference for every example. Keeping them in one shared module makes comparisons more meaningful because the coordination pattern changes while the business rules remain stable. The two structured output models provide machine-readable contracts. `SpecialistRoute` defines the fields expected from the router, while `DebateVerdict` defines the fields expected from the judge. These schemas remove the need to parse free-form prose and allow the application to validate the shape of a response. They do not guarantee that the selected specialist or final verdict is semantically correct; those decisions still require testing, evaluation, and appropriate fallback behaviour.

The `run_agent` helper standardizes how agents are invoked and how their final message content is extracted. It also accepts an optional configuration object, which later examples can use for execution-specific settings such as checkpointing and thread identity. The returns and billing agents are domain specialists whose prompts give them a narrow scope and a specific policy reference. These instructions make their intended responsibilities clear but prompts alone do not enforce a security or authorization boundary. In a production system, prompt-level specialization should be combined with restricted tool access, controlled context, input and output validation, and an explicit fallback path when the agent cannot safely complete the request.

The `customer_advocate` and `policy_guard` agents use the same returns policy but apply different interpretive priorities. One looks for the most customer-friendly defensible outcome, while the other prefers the narrowest safe interpretation. These roles may disagree, but they may also reach the same conclusion when the policy is clear. Their purpose is to introduce different review perspectives, not to manufacture disagreement. The specialist tool wrappers

expose the returns and billing agents through stable callable interfaces, allowing a manager to invoke them without depending on their internal prompts. The lightweight escalation tool provides a consistent way to indicate that automated handling should stop, while the later interrupt-based example demonstrates an enforced pause-and-resume review boundary.

Finally, `request_manual_refund_approval` and `build_manual_review_agent` prepare the stronger escalation boundary. The tool does not immediately return a policy result. It interrupts execution and emits a structured review request, allowing human approval or rejection to become part of the runtime flow. The factory function packages that tool into a dedicated escalation agent. This creates a clear separation between routine specialist work and exception handling, while making the escalation path easier to connect to persistent workflow state.

5.1.3 Manager/Worker

With the shared setup in place, a minimal manager/worker example is very small. The manager gets two workers as tools, decides when each should be called, and synthesizes one customer-facing answer. Nothing in this example depends on a large graph. The point is to make the central coordination boundary visible.

```
from __future__ import annotations
from langchain.agents import create_agent
from .support_team_shared import (
    MODEL,
    ask_billing_specialist,
    ask_returns_specialist,
)

support_manager = create_agent(
    model = MODEL,
    tools = [ask_returns_specialist, ask_billing_specialist],
    system_prompt = (
        "You are the support manager.\n"
        "Coordinate specialists instead of answering from memory.\n"
        "Use both specialists when the customer's problem spans both domains.\n"
        "Return one clear final answer for the customer."
    )
)

if __name__ == "__main__":
    question = (
        "I was charged twice for a gift order that also arrived damaged. "
        "Explain what happens with the billing issue and with the return."
    )

    seen = 0

    for step in support_manager.stream(
        {"messages": [{"role": "user", "content": question}]},
        stream_mode = "values"
    ):
        messages = step["messages"]
        new_messages = messages[seen:]

        for message in new_messages:
            message.pretty_print()

        seen = len(messages)
```

vol3_ch_05\src\manager_worker.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> python -m src.manager_worker

===== Human Message =====
I was charged twice for a gift order that also arrived damaged. Explain what happens with
the billing issue and with the return.
===== Ai Message =====
```

```

Tool Calls:
  ask_billing_specialist (call_6AMIWaeZTbIOV9pXIi6SRHLF)
  Call ID: call_6AMIWaeZTbIOV9pXIi6SRHLF
  Args:
    question: Customer was charged twice for a gift order. Please clarify the resolution
    process for duplicate charges.
  ask_returns_specialist (call_4nIuuU8ux8wfg6BMuZZqIGAv)
  Call ID: call_4nIuuU8ux8wfg6BMuZZqIGAv
  Args:
    question: Customer received a damaged gift order. Please explain the return and
    replacement process for damaged items.
===== Tool Message =====
Name: ask_billing_specialist

For duplicate charges, we must first investigate the issue before issuing any refund.
Please provide the details of the charges, and we will look into it. If necessary, the
case may be escalated for further review.
===== Tool Message =====
Name: ask_returns_specialist

Since the gift order arrived damaged, you are eligible for a refund or replacement. You
can return the damaged item even if the box was opened. Please contact us to initiate the
return process, and we will arrange for a refund or replacement as per your preference.
===== Ai Message =====

Regarding the billing issue of being charged twice for the gift order, we need to
investigate the charges first. Please provide the details of the charges so we can look
into it and resolve the duplicate charge, potentially issuing a refund if confirmed.

For the damaged gift order, you are eligible for a refund or replacement. You can return
the damaged item even if the box was opened. Please contact us to start the return
process, and we will arrange for a refund or replacement based on your preference.

```

Even in this small example, the coordination logic is doing real architectural work. The customer's request is cross-domain. A single specialist might answer outside its own policy boundary or ignore part of the problem. The manager addresses that risk by treating the user's message as one request that may require contributions from several specialists. This is exactly where manager/worker becomes useful: not when the task is large in an abstract sense, but when the task must be decomposed across distinct responsibilities. The code also shows why wrapping workers as tools is such a practical design. The manager does not need to know how the returns specialist or billing specialist is prompted internally. It only needs stable callable interfaces and good tool descriptions. That separation keeps the top-level orchestration simple. It also makes later changes cheap. You can strengthen one specialist's prompt, replace its implementation, or add validation around one worker without changing the manager's role in the workflow.

Using `stream(..., stream_mode="values")` exposes the sequence of messages and tool activity produced during the run. This makes it possible to inspect whether the manager calls one specialist or several and how it combines their results. Such traces are useful for learning and debugging, but visibility does not prove that the routing decision was correct. The observed trajectory should still be evaluated against expected specialist choices and expected final behaviour. In this example, the manager retains the conversation context and invokes workers that start each call with fresh state when their specialist capabilities are needed. A final point is worth making. This example is intentionally minimal, so it does not yet solve every coordination concern. It does not enforce stopping rules beyond the agent's own runtime behaviour, it does not maintain richer typed state, and it does not yet model stronger escalation boundaries. At this stage, the example is doing one job only: showing how one agent can remain responsible for workflow structure while narrower agents supply

domain-specific reasoning. Once that boundary is clear, more advanced coordination mechanisms become much easier to justify and understand.

5.1.4 Specialists: narrower agents with clearer responsibilities

If manager/worker is about coordination, specialists are about boundaries. A specialist agent exists because some part of the problem deserves its own prompt, its own tool set, its own evaluation criteria, or its own access boundary. The specialist is not “a smaller copy” of the main agent. It is a deliberately narrower component with a clearer job. That is why specialist teams work well. The system stops pretending that one model invocation can be equally good at every part of the workflow. Instead, it assigns specific kinds of work to agents that are designed for those kinds of work. In practice, specialist boundaries should be based on something stable. The most common choices are domain, tools, or trust boundary. Domain specialists are the easiest to spot: billing, returns, catalog, customer communications, technical support, fraud review, market news, technical analysis, and so on. Tool-based specialists appear when one agent should own a specific capability set, such as SQL access, order-management actions, or regulatory document lookup. Trust-boundary specialists are a little subtler but often more important: an internal support specialist may be allowed to see internal runbooks, while a customer-facing specialist should never receive that context. In a good design, the specialist boundary explains why that agent exists. If the answer is only “to make the system look more agentic,” the boundary is probably weak.

LangChain provides several ways to express specialized behaviour. In the subagents pattern, a main agent retains control and calls specialists as tools. A router performs an explicit classification or decomposition step and dispatches the request to one or more specialists. Skills load specialized prompts and knowledge on demand while one agent remains active. Handoffs change the active specialist or the current agent configuration as the interaction progresses. A custom LangGraph workflow can combine deterministic routing, specialist agents, parallel work, validation, and human review when the standard patterns are not sufficient. The right choice depends on whether control should remain centralized, whether several specialists may be needed, and whether responsibility can change during the interaction. One detail matters more than it first appears: each specialist must represent a meaningful boundary. A system with several vague or overlapping specialists is often harder to route, test, and maintain than one well-scoped agent. Specialization is useful when it reduces prompt scope, narrows tool access, separates security responsibilities, allows different evaluation criteria, or enables genuinely independent work. In the support scenario, billing, returns, and technical troubleshooting are reasonable specialist boundaries because they correspond to different policies, tools, and operational responsibilities. The user does not need to see those internal divisions, but the application should be able to explain clearly what each specialist owns.

A minimal specialists example does not need a manager. The system first decides which specialist owns the request, then sends the rewritten request to that specialist. The structured route keeps the dispatch step predictable, and the specialist still receives a narrow, role-appropriate context. This example uses the same shared module as the previous one, but the collaboration shape is different: it is dispatch, not supervision. Typed structured output is a

good fit here because the router should return a small, machine-usable object rather than free-form prose.

```
from __future__ import annotations
from .support_team_shared import (
    MODEL,
    SpecialistRoute,
    billing_agent,
    returns_agent,
    run_agent
)

router = MODEL.with_structured_output(SpecialistRoute)

def choose_specialist(question: str) -> SpecialistRoute:
    return router.invoke(
        f"""
        Choose exactly one specialist for the support request.

        returns:
        - refunds
        - replacements
        - damaged items
        - gift returns
        - return windows

        billing:
        - invoices
        - duplicate charges
        - payments
        - billing disputes

        Rewrite the request into a short standalone question for that specialist.

        User request:
        {question}
        """
    )

if __name__ == "__main__":
    question = "I need a copy of my invoice and I want to know where to find it."
    decision = choose_specialist(question)

    specialist_map = {
        "returns": returns_agent,
        "billing": billing_agent
    }

    answer = run_agent(
        specialist_map[decision.specialist],
        decision.rewritten_question
    )

    print("Chosen specialist:", decision.specialist)
    print("Rewritten question:", decision.rewritten_question)
    print()
    print("Answer:")
    print(answer)
```

vol3_ch_05\src\specialists_router.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> python -m src.specialists_router

Chosen specialist: billing
Rewritten question: How can I get a copy of my invoice and where can I find it?

Answer:
Invoices are available in your customer account. You can log in to your account and
access your invoices there.
```

This example places a typed routing step in front of the specialist agents. The router returns two machine-readable fields: the selected specialist and a standalone question for that specialist. The schema allows the application to validate the response structure and consume it without parsing free-form prose. It does not prove that the routing decision is correct, so the router should be evaluated with representative requests, including ambiguous and out-of-

scope cases. The instruction to choose exactly one specialist forces a single ownership decision; it does not remove ambiguity. This design is appropriate only when each supported request can be assigned to one domain. A request that combines billing and returns could lose part of its meaning if it were forced through this router. Cross-domain requests require a router that can select several specialists, a parallel fan-out followed by synthesis, or a manager that can call more than one worker. Rewriting the request can give the selected specialist a cleaner input, but the rewrite must preserve important facts, constraints, and uncertainty from the original message. The `specialist_map` then converts the symbolic route into an application-controlled agent instance, keeping model-generated labels separate from runtime implementation details.

5.1.5 Adding lightweight escalation to the supervisor

This example extends the manager/worker design from Section 1.1.3 by adding a third supervisory option: escalation. The manager can call the billing specialist, call the returns specialist, or indicate that automated handling should stop because the request is ambiguous, outside policy, or asks for an override. This is deliberately a lightweight escalation mechanism. It demonstrates the manager's routing decision but does not yet implement a durable human-review workflow. The stronger pause, persistence, and resumption boundary is introduced in Section 5.1.7.

```
from __future__ import annotations
from langchain.agents import create_agent
from .support_team_shared import (
    MODEL,
    ask_billing_specialist,
    ask_returns_specialist,
    escalate_to_human
)

support_manager = create_agent(
    model = MODEL,
    tools = [ask_returns_specialist, ask_billing_specialist, escalate_to_human],
    system_prompt = (
        "You are the support manager.\n"
        "Coordinate the specialists instead of answering from memory.\n"
        "Use the returns specialist for returns and refunds.\n"
        "Use the billing specialist for invoice and charge questions.\n"
        "Escalate when the request is outside policy, ambiguous, or asks for an "
        "override. Give the customer a clear final answer."
    )
)

if __name__ == "__main__":
    cases = [
        "My package arrived damaged. Can I get a refund?",
        "I was charged twice for the same order.",
        "Please refund a gift order from four months ago and override the policy."
    ]

    for question in cases:
        print("=" * 80)
        print("USER:", question)
        result = support_manager.invoke(
            {"messages": [{"role": "user", "content": question}]}
        )
        print("ASSISTANT:", result["messages"][-1].content)
```

vol3_ch_05\src\supervisor_with_escalation.py

```
(.venv) C:\Users\alexc\11m-app\vol3_ch_05> python -m src.supervisor_with_escalation
=====
USER: My package arrived damaged. Can I get a refund?
```



```

ASSISTANT: Yes, you can get a refund for your damaged package. If you prefer, we can also
arrange a replacement for you. Please let me know how you would like to proceed.
=====
USER: I was charged twice for the same order.
ASSISTANT: Could you please provide me with your order details or customer account
information? This will help us investigate the duplicate charge and resolve the issue for
you.
=====
USER: Please refund a gift order from four months ago and override the policy.
ASSISTANT: Your request for a refund on a gift order from four months ago is outside our
standard return window and requires a special review. I have escalated this to our human
support team for further assistance. You will be contacted soon with an update. Thank you
for your patience.

```

The manager's tools define three possible supervisory actions: consult the returns specialist, consult the billing specialist, or select the escalation path. This makes escalation part of normal routing rather than an unexpected error condition. Routine requests remain within the appropriate specialist boundary, while policy exceptions, ambiguous cases, and override requests can be marked as unsuitable for automatic resolution. The lightweight escalation path in this example represents a decision to stop local handling, but it should not be confused with an enforced human-in-the-loop boundary. Returning an escalation message does not by itself pause execution, persist the case, create a queue item, or guarantee that a reviewer will act. Those operational guarantees require application logic around persistence, assignment, notification, audit status, and resumption. The interrupt-based example later in the chapter demonstrates the runtime mechanism used to pause execution until an explicit decision is supplied.

5.1.6 Debate and structured review

The debate pattern is useful when a result should be challenged before it is accepted. The central question is not which agent should perform the task, but which additional perspective, rule, or reviewer should test the first answer. Different agents may focus on policy, operational risk, customer impact, cost, or evidential support. Their outputs are then compared or reviewed before a final decision is produced. These perspectives are not automatically independent, especially when the agents use the same model and evidence. Despite its name, debate does not always require a long conversation. It is better understood as a family of structured multi-agent review patterns. The main variations are:

- Open discussion. Several agents contribute to a shared conversation and may revise their positions after seeing other contributions. This approach requires a clear method for selecting speakers and deciding when the discussion has finished.
- Maker-checker. One agent produces a draft and another evaluates it against explicit acceptance criteria. If the result is rejected, the maker revises it. The workflow must include a maximum number of rounds and a fallback when approval is not reached.
- Independent review and aggregation. Several agents produce recommendations independently. A deterministic rule, ranker, voter, or judge then combines or selects among them. This provides multiple perspectives without requiring an extended shared discussion.

The chapter's example uses a bounded form of the third approach with one additional rebuttal round. The underlying policy is deliberately clear, so the two reviewers may confirm the same conclusion rather than disagree. This demonstrates an important property of structured review: its purpose is to test a result, not to manufacture conflict. The example is therefore a

small two-perspective policy review followed by a judge, rather than a general-purpose multi-round group discussion.

```

from __future__ import annotations
from .support_team_shared import (
    MODEL,
    DebateVerdict,
    customer_advocate,
    policy_guard,
    run_agent
)

judge = MODEL.with_structured_output(DebateVerdict)

if __name__ == "__main__":
    question = (
        "A customer received a damaged gift order. "
        "Should support offer a refund or store credit?"
    )

    advocate_opening = run_agent(customer_advocate, question)
    guard_opening = run_agent(policy_guard, question)

    advocate_rebuttal = run_agent(
        customer_advocate,
        f"""
Question:
{question}

Other side's position:
{guard_opening}

Reply with the strongest short objection to that position.
""")

    guard_rebuttal = run_agent(
        policy_guard,
        f"""
Question:
{question}

Other side's position:
{advocate_opening}

Reply with the strongest short objection to that position.
""")

    verdict = judge.invoke(f"""
You are the debate judge.

Question:
{question}

Customer advocate opening:
{advocate_opening}

Policy guard opening:
{guard_opening}

Customer advocate rebuttal:
{advocate_rebuttal}

Policy guard rebuttal:
{guard_rebuttal}

Return the final decision.
""")

    print("Customer advocate:")
    print(advocate_opening)
    print()
    print("Policy guard:")
    print(guard_opening)
    print()
    print("Customer advocate rebuttal:")
    print(advocate_rebuttal)
    print()
    print("Policy guard rebuttal:")
    print(guard_rebuttal)
    print()
    print("Final decision:", verdict.decision)
    print("Rationale:", verdict.rationale)

```

vol3_ch_05\scr\debate.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> python -m src.debate

Customer advocate:
Since the gift arrived damaged, support should offer a refund or replacement rather than
just store credit, as per the policy.

Policy guard:
Support should offer a refund or replacement for the damaged gift order, as damaged items
are eligible for refund or replacement regardless of being a gift.

Customer advocate rebuttal:
The policy states that damaged gift returns are eligible for refund or replacement, so
offering only store credit is not consistent with the policy; support should honor a
refund or replacement for the damaged gift.

Policy guard rebuttal:
The policy states damaged items can be refunded or replaced even if opened, and gift
returns get store credit unless damaged; since this gift arrived damaged, support should
offer refund or replacement, not just store credit. No objection.

Final decision: refund
Rationale: Both sides agree that the policy allows for a refund or replacement for
damaged gift items. Therefore, support should offer a refund or replacement rather than
just store credit, in line with the policy.
```

This example implements a bounded two-perspective policy review. Both agents receive the same original question and produce an opening position before seeing the other response. Each agent is then asked to test the opposing position for weaknesses. This gives both roles the same number of opportunities and the same general information structure, although it does not guarantee independence or remove ordering effects. Because the policy is clear, both agents reach the same conclusion. In this case, the review increases confidence that the rule has been interpreted consistently rather than exposing a disagreement. The judge receives the openings and rebuttals and returns a verdict with a predictable schema. This separates review from final selection and makes the result easier to integrate into application logic. Structured output validates the verdict's fields, not the quality of the judgment. A production implementation should give the judge explicit decision criteria, preserve the supporting evidence, evaluate judge reliability, and escalate high-risk or unresolved cases rather than treating the judge as an unquestionable authority.

5.1.7 Escalation

Escalation transfers a case to a path with greater authority, stronger controls, or human accountability. It should be distinguished from ordinary routing or handoff. Sending an invoice question to the billing specialist is normal delegation; sending a policy exception to a supervisor or human reviewer is escalation. Escalation is therefore not merely an error response. It is a planned control outcome for cases that should not be resolved within the current agent's authority. Common escalation triggers include:

- Insufficient evidence: required facts are missing, unreliable, or contradictory.
- Policy exception: the request falls outside the rules available to the automated path.
- Conflicting conclusions: specialists or evidence sources disagree on a material point.
- Sensitive action: the next step is high-impact, irreversible, regulated, or requires explicit authorization.
- Retry limit: the workflow has exhausted its permitted attempts without reaching an acceptable result.

For example, a support assistant may escalate an exception to the normal refund policy. A trading assistant should require explicit approval before any real order or other consequential financial action. The purpose is to prevent uncertain local reasoning from being converted directly into external action. Escalation fits naturally with human-in-the-loop design because it makes the boundary of automated authority explicit. The workflow should define when review is optional or mandatory, what information the reviewer receives, and whether the reviewer may approve, reject, edit, or request more information. A strong escalation boundary pauses the affected execution, exposes a structured review request, persists enough state to resume safely, records the decision, and continues only through a permitted outcome. Escalation should be designed as part of the workflow contract rather than added only after unsafe behaviour has already occurred.

```
from __future__ import annotations
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from .support_team_shared import (
    build_manual_review_agent,
    returns_agent,
    run_agent
)

if __name__ == "__main__":
    question = (
        "Order ORD-9001 was opened and delivered 90 days ago. "
        "Please approve a refund anyway."
    )

    print("Front-line specialist:")
    frontline_answer = run_agent(returns_agent, question)
    print(frontline_answer)
    print()

    review_agent = build_manual_review_agent(InMemorySaver())
    config = {"configurable": {"thread_id": "refund-exception-1"}}

    paused = review_agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": question + " The order ID is ORD-9001."
                }
            ]
        },
        config = config
    )

    interrupt_payload = paused["__interrupt__"][0].value
    print("Escalation payload:")
    print(interrupt_payload)
    print()

    resumed = review_agent.invoke(Command(resume = "approve"), config = config)

    print("Final result:")
    print(resumed["messages"][-1].content)
```

vol3_ch_05\src\escalation.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> python -m src.escalation

Front-line specialist:
This request falls outside the 30-day return window for opened items. Escalation is
required for manual review.

Escalation payload:
{'kind': 'manual_refund_review', 'order_id': 'ORD-9001', 'reason': 'Customer requests
refund approval despite order being delivered 90 days ago, which is outside the normal
return policy.', 'allowed_responses': ['approve', 'reject']}

Final result:
```

```
The refund for order ORD-9001 has been approved despite being outside the normal return policy. If you need any further assistance, please let me know!
```

This example makes escalation a visible runtime transition. The front-line specialist identifies that the request is outside normal policy, after which a review-capable agent pauses and exposes a structured approval request. Checkpointing and a stable thread identifier allow LangGraph to locate the saved execution when the external decision is supplied. The in-memory checkpointer used in the example is suitable for demonstration and local testing; a production workflow requires a durable checkpointer so that pending reviews survive process restarts and service failures. `Command(resume="approve")` supplies the value that becomes the return value of `interrupt()`. However, the runtime does not continue from the exact source-code line after the interrupt. The node containing the interrupt starts again from its beginning, and code that ran before the interrupt may run again. Operations performed before an interrupt must therefore be idempotent, moved after approval, or isolated in separate nodes. This replay behaviour is part of the correctness model for any workflow that pauses and resumes.

5.1.8 Choosing the right pattern

A useful way to choose among these patterns is to ask what kind of complexity you are trying to control.

- If one request requires several distinct kinds of work and their results must be combined, start with manager/worker. One coordinator can retain the overall objective while narrower agents handle clearly defined subtasks.
- If one agent has too many responsibilities, instructions, tools, or access boundaries, use specialists. The goal is to create smaller and clearer responsibilities.
- If the first answer is too risky to accept without review, use structured review or debate. The review may take the form of open discussion, maker-checker, independent recommendations, or a bounded judging step.
- If the problem is that some cases should not be resolved locally at all, use escalation. That is how the system marks the edge of safe autonomy.

These patterns combine naturally. A manager can coordinate specialists, a specialist can trigger escalation, and a review loop can be reserved for difficult or high-risk cases instead of running on every request. Different stages of one workflow may require different coordination mechanisms. Choose the simplest effective pattern for each stage rather than forcing the entire application into one architectural shape.

5.1.9 Practical design rules

A few rules survive contact with production better than any particular framework abstraction.

- First, do not split an agent unless the split creates a real boundary. Domain responsibility, tool access, security constraints, or review authority are good reasons. Stylistic preference alone is not.
- Second, keep specialist roles narrow enough that their prompts and evaluation criteria are obvious. If you cannot explain what one specialist owns in one sentence, the role is probably too vague.

- Third, do not use debate when the task is really deterministic. If a database query, policy lookup, or business rule can answer the question directly, more discussion does not add signal.
- Fourth, treat escalation as part of the workflow contract. Cases that need human judgment, explicit approval, or exception handling should be designed that way from the start.
- Fifth, justify the cost of coordination. Multi-agent systems do not remove the underlying complexity; they redistribute it across explicit components and interactions. Use them only when the resulting improvements in separation, control, testing, or security outweigh the added latency, cost, state management, and failure modes.

That is the core story of multi-agent collaboration. A strong multi-agent system is not a crowd of agents chatting for effect. It is a disciplined arrangement of boundaries: who owns the overall objective, who owns specialist work, who is allowed to challenge a result, and who must take over when local autonomy should end.

5.2 COMMUNICATION MECHANISMS: HANDOFFS, SHARED MESSAGE LISTS, SHARED STATE SCHEMAS, AND A2A

5.2.1 Communication is part of the architecture

As soon as a system uses more than one agent, communication becomes a core architectural concern. It is not a minor implementation detail. An agent may have a good prompt, the right tools, and a narrow role, but the system can still fail if responsibility, context, and intermediate results do not move clearly between agents. That is why multi-agent design is not only about deciding which agents exist. It is also about deciding how they interact. Some systems route everything through one supervisor. Some allow more direct collaboration between specialists. Some add extra layers of coordination when the workflow is large or complex. The trade-offs are practical: robustness, autonomy, communication cost, and how easy the workflow is to understand, test, and debug.

In LangChain and LangGraph, the practical question is simpler than the general theory. Once one agent is no longer enough, two questions matter most: who should control the next step, and what information should each specialist receive? Current LangChain guidance describes several patterns, including subagents, handoffs, skills, routers, and custom workflows. In the subagent pattern, one main agent remains in control and invokes specialists as tools. A router classifies a request and sends it to one or more specialists. A handoff changes the active behavior or specialist, usually by updating state; it can be implemented with separate agents or with one agent whose prompt and tools change. A custom workflow exposes the graph directly so that state and control flow are explicit. Start with the simplest pattern that meets the requirement because each additional agent introduces more model calls, state transitions, and possible failures. This section separates three in-process communication mechanisms from one network protocol. Handoffs express a change of responsibility. Shared messages preserve conversational context. A shared state schema defines typed channels and the rules for merging their updates. A2A is different: it is a wire-level protocol used when communication crosses service boundaries. These mechanisms can be combined, but they solve different problems.

5.2.2 Handoffs: transferring control without losing context

A handoff changes the active behavior or responsibility of the system. In a single-agent implementation, this may mean changing the agent's prompt and available tools while keeping the same agent loop. In a multi-agent graph, it may mean routing execution to another specialist. This differs from the manager/worker pattern, where the manager remains in control and invokes a specialist only to complete a delegated task. The example in this section demonstrates the multi-agent form: the triage agent transfers responsibility to either the billing or returns specialist. This is why handoffs work so well in sequential support flows. A support assistant may start in a general triage role, but once the request is clearly about billing or returns, the workflow is often easier to understand if the right specialist takes over directly. In a custom LangGraph workflow such as the one below, a handoff can combine two actions: updating shared state and selecting the next node. `Command` is useful for this implementation

because a node can return both `update={...}` and `goto=...` in one operation. The node can therefore record that billing is now responsible for the case while also routing execution to the billing specialist. Other handoff implementations may change one agent's configuration or transfer control between parent and child graphs, but the underlying principle is the same: the system makes a persistent change in active behavior or responsibility.

There is also a second requirement that is easy to underestimate: the receiving specialist must get the right context. A handoff is not complete just because the graph routes to another node. The new specialist also needs a conversation history that is still correct and still useful. If the wrong messages are passed, or if important messages are missing, the receiving agent may start from an incomplete picture of the case. This matters especially in subgraph-based handoffs. When a handoff is triggered by a tool call, the receiving side should usually see both the `AIMessage` that made the tool call and the matching `ToolMessage` that records the result. Without both messages, the tool-call exchange is incomplete from the model's point of view. The example below uses direct graph routing rather than an LLM-generated handoff tool call, so it does not require that `AIMessage` and `ToolMessage` pair. More broadly, this leads to an important design rule: do not pass every message automatically. Pass only the context the receiving specialist actually needs. Handoffs work best when context transfer is deliberate. The next example keeps the pattern small and clear. A triage node reads the user's request, decides which specialist should take over, rewrites the request as a cleaner standalone question, and then returns a `Command` that both updates state and routes to the correct specialist. The specialist node then produces the customer-facing answer.

```
from __future__ import annotations
from operator import add
from typing import Annotated, Literal
from typing_extensions import NotRequired
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langchain.messages import AIMessage
from langgraph.graph import MessagesState, START, END, StateGraph
from langgraph.types import Command
from .config import OPENAI_API_KEY
from .support_team_shared import ask_billing_specialist, ask_returns_specialist

MODEL = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class HandoffDecision(BaseModel):
    active_agent: Literal["billing", "returns"] = Field(
        description = "Which specialist should take over this case."
    )
    standalone_question: str = Field(
        description = "Rewrite the customer request as a clean standalone question."
    )

class SupportState(MessagesState):
    active_agent: NotRequired[str]
    standalone_question: NotRequired[str]
    trace: Annotated[list[str], add]

router = MODEL.with_structured_output(HandoffDecision)

def triage(state: SupportState) -> Command[Literal["billing_agent", "returns_agent"]]:
    latest_user_message = state["messages"][-1].content

    decision = router.invoke(
        f"""
Route this support request to exactly one specialist.

Use "billing" for charges, invoices, payments, and duplicate-charge disputes.
Use "returns" for refunds, damaged items, replacements, gift returns, and return windows.

Customer request:
{latest_user_message}
"""
    )
```

```

target = "billing_agent" if decision.active_agent == "billing" else "returns_agent"
return Command(
    update = {
        "active_agent": decision.active_agent,
        "standalone_question": decision.standalone_question,
        "trace": [
            f"triage: handoff to {decision.active_agent}",
            f"triage: standalone_question={decision.standalone_question!r}"
        ]
    },
    goto = target
)

def billing_agent_node(state: SupportState) -> dict:
    answer = ask_billing_specialist.invoke(
        {"question": state["standalone_question"]}
    )
    return {
        "messages": [AIMessage(content = answer)],
        "active_agent": "billing",
        "trace": ["billing_agent: answered billing question"]
    }

def returns_agent_node(state: SupportState) -> dict:
    answer = ask_returns_specialist.invoke(
        {"question": state["standalone_question"]}
    )
    return {
        "messages": [AIMessage(content = answer)],
        "active_agent": "returns",
        "trace": ["returns_agent: answered returns question"]
    }

builder = StateGraph(SupportState)
builder.add_node("triage", triage)
builder.add_node("billing_agent", billing_agent_node)
builder.add_node("returns_agent", returns_agent_node)

builder.add_edge(START, "triage")
builder.add_edge("billing_agent", END)
builder.add_edge("returns_agent", END)

graph = builder.compile()

if __name__ == "__main__":
    inputs = {
        "messages": [
            {
                "role": "user",
                "content": "I was charged twice for the same order. What happens next?"
            }
        ],
        "trace": []
    }

    print("QUESTION:")
    print(inputs["messages"][0]["content"])
    print()

    last_trace_len = 0
    last_message_len = 1 # initial user message already known

    for step in graph.stream(inputs, stream_mode = "values"):
        trace = step.get("trace", [])
        new_trace = trace[last_trace_len:]
        for line in new_trace:
            print(line)
        last_trace_len = len(trace)

        messages = step["messages"]
        new_messages = messages[last_message_len:]
        for msg in new_messages:
            print()
            print("SPECIALIST OUTPUT:")
            print(msg.content)
        last_message_len = len(messages)

    result = graph.invoke(inputs)

    print()
    print("FINAL ANSWER:")
    print(result["messages"][-1].content)

```

vol3_ch_05\src\support_handoff_graph.py

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> python -m src.support_handoff_graph

QUESTION:
I was charged twice for the same order. What happens next?

triage: handoff to billing
triage: standalone_question='I was charged twice for the same order. What happens next?'
billing_agent: answered billing question

SPECIALIST OUTPUT:
Since duplicate charges must be investigated before any refund is issued, I will need to
escalate this charge dispute for further review.

FINAL ANSWER:
Duplicate charges must be investigated before any refund is issued. I will escalate this
charge dispute for further investigation.
```

The triage node is the coordination boundary of this workflow. It classifies the case, rewrites the request as a standalone question, records the selected specialist, and returns a Command that updates state and routes execution in one operation. The specialist nodes remain deliberately narrow: they read the prepared question, produce an answer, append it to messages, and record a diagnostic event. The state fields make the handoff visible as both control flow and data flow. `active_agent` identifies the current owner, `standalone_question` carries the prepared input, and `trace` records the decisions made during execution. Because `trace` uses an additive reducer, later nodes add entries without overwriting earlier ones. This is useful for debugging and observability, although it is not a durable or compliance-grade audit log.

5.2.3 Shared message lists: agents work through one shared conversation

Sometimes agents do not mainly need to pass control to one another. Instead, they need to see the same conversation and add to it. In this pattern, all agents share one growing list of messages. Each step reads the current conversation history and then adds its own message to that same history. This is a very natural pattern in LangGraph because message history is already part of the state model. You can represent it directly with a field such as `messages: Annotated[list[AnyMessage], add_messages]`, or you can use the built-in `MessagesState` class. The `add_messages` reducer is important because message handling is more than a simple list append. It also deals correctly with message updates and common shorthand message formats. A shared message list is useful when later agents need to see the conversation itself, not just a few extracted fields. Reflection loops are one example, but the pattern is useful more broadly. It works well when one specialist's reply should stay visible to later specialists in its original form, or when a final synthesizer should read the full conversational path that led to the answer. The idea is simple: agents collaborate by reading and writing to the same conversation history.

The main drawback is that the history keeps growing. Over time, the shared message list can collect user requests, specialist replies, tool outputs, review notes, and sometimes system messages. This increases cost and can make the state harder to manage. Long histories often need trimming, summarization, filtering, or another reduction strategy. In practice, the shared message pattern is powerful, but it can become expensive and messy if you let the conversation grow without control. The next example keeps this idea small and clear. Two specialists add their findings to one shared conversation, and a final synthesizer reads that shared history to

produce a single answer for the customer. The workflow also keeps some structured fields. That is intentional. In real systems, a shared message list often works best as one part of a larger shared state, rather than carrying everything by itself.

```

from __future__ import annotations
from typing_extensions import NotRequired
from langchain.chat_models import init_chat_model
from langchain.messages import AIMessage, SystemMessage, HumanMessage
from langgraph.graph import MessagesState, START, END, StateGraph
from .config import OPENAI_API_KEY
from .support_team_shared import ask_billing_specialist, ask_returns_specialist

MODEL = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class SupportCollaborationState(MessagesState):
    question: str
    billing_result: NotRequired[str]
    returns_result: NotRequired[str]
    resolution: NotRequired[str]

def billing_specialist(state: SupportCollaborationState) -> dict:
    result = ask_billing_specialist.invoke({"question": state["question"]})
    return {
        "billing_result": result,
        "messages": [AIMessage(content = f"Billing specialist: {result}")]
    }

def returns_specialist(state: SupportCollaborationState) -> dict:
    result = ask_returns_specialist.invoke({"question": state["question"]})
    return {
        "returns_result": result,
        "messages": [AIMessage(content = f>Returns specialist: {result}")]
    }

def synthesize(state: SupportCollaborationState) -> dict:
    messages = [
        SystemMessage(
            content = (
                "You are the support coordinator.\n"
                "Use the specialist messages in the conversation to write one clear "
                "customer-facing answer. Do not invent policy beyond what the "
                "specialists already stated."
            )
        ),
        *state["messages"]
    ]

    final_answer = MODEL.invoke(messages)

    return {
        "resolution": final_answer.content,
        "messages": [final_answer]
    }

builder = StateGraph(SupportCollaborationState)
builder.add_node("billing_specialist", billing_specialist)
builder.add_node("returns_specialist", returns_specialist)
builder.add_node("synthesize", synthesize)

builder.add_edge(START, "billing_specialist")
builder.add_edge("billing_specialist", "returns_specialist")
builder.add_edge("returns_specialist", "synthesize")
builder.add_edge("synthesize", END)

graph = builder.compile()

def print_message_list(messages) -> None:
    print("Shared message state:")
    for i, message in enumerate(messages, start = 1):
        msg_type = message.__class__.__name__
        content = getattr(message, "content", "")
        print(f"{i}. {msg_type}: {content}")
    print()

if __name__ == "__main__":
    question = (
        "I was charged twice for a gift order, and the replacement item also arrived "
        "damaged. Explain what happens with the billing issue and with the return."
    )

    initial_state = {

```

```

    "question": question,
    "messages": [HumanMessage(content = question)]
}

print("=== STATE UPDATES AS THE GRAPH RUNS ===\n")

for chunk in graph.stream(initial_state, stream_mode = "values"):
    print_message_list(chunk["messages"])

    if "billing_result" in chunk:
        print(f"billing_result: {chunk['billing_result']}")
    if "returns_result" in chunk:
        print(f"returns_result: {chunk['returns_result']}")
    if "resolution" in chunk:
        print(f"resolution: {chunk['resolution']}")

    print("-" * 80)

print("\n=== FINAL STATE ===\n")
result = graph.invoke(initial_state)

print_message_list(result["messages"])
print("Final resolution:")
print(result["resolution"])

```

vol3_ch_05\src\support_shared_messages_state.py

```

(.venv) C:\Users\alexc\llm-app\vol3_ch_05> python -m src.support_shared_messages_state

=== STATE UPDATES AS THE GRAPH RUNS ===

Shared message state:
1. HumanMessage: I was charged twice for a gift order, and the replacement item also
arrived damaged. Explain what happens with the billing issue and with the return.

-----
Shared message state:
1. HumanMessage: I was charged twice for a gift order, and the replacement item also
arrived damaged. Explain what happens with the billing issue and with the return.
2. AIMessage: Billing specialist: Regarding the double charge for your gift order, we
need to investigate the duplicate charges before any refund can be issued. For the
damaged replacement item, the return process would typically be handled separately from
billing. Since your issue involves a charge dispute, this matter must be escalated for
further review.

billing_result: Regarding the double charge for your gift order, we need to investigate
the duplicate charges before any refund can be issued. For the damaged replacement item,
the return process would typically be handled separately from billing. Since your issue
involves a charge dispute, this matter must be escalated for further review.

-----
Shared message state:
1. HumanMessage: I was charged twice for a gift order, and the replacement item also
arrived damaged. Explain what happens with the billing issue and with the return.
2. AIMessage: Billing specialist: Regarding the double charge for your gift order, we
need to investigate the duplicate charges before any refund can be issued. For the
damaged replacement item, the return process would typically be handled separately from
billing. Since your issue involves a charge dispute, this matter must be escalated for
further review.
3. AIMessage: Returns specialist: For the damaged replacement item, you can receive a
refund or a replacement even if the box was opened. Regarding the billing issue of being
charged twice for a gift order, this falls outside the returns policy and requires
escalation for manual review.

billing_result: Regarding the double charge for your gift order, we need to investigate
the duplicate charges before any refund can be issued. For the damaged replacement item,
the return process would typically be handled separately from billing. Since your issue
involves a charge dispute, this matter must be escalated for further review.
returns_result: For the damaged replacement item, you can receive a refund or a
replacement even if the box was opened. Regarding the billing issue of being charged
twice for a gift order, this falls outside the returns policy and requires escalation for
manual review.

-----
Shared message state:
1. HumanMessage: I was charged twice for a gift order, and the replacement item also
arrived damaged. Explain what happens with the billing issue and with the return.
2. AIMessage: Billing specialist: Regarding the double charge for your gift order, we
need to investigate the duplicate charges before any refund can be issued. For the
damaged replacement item, the return process would typically be handled separately from
billing. Since your issue involves a charge dispute, this matter must be escalated for
further review.

```

```

3. AIMessage: Returns specialist: For the damaged replacement item, you can receive a
refund or a replacement even if the box was opened. Regarding the billing issue of being
charged twice for a gift order, this falls outside the returns policy and requires
escalation for manual review.
4. AIMessage: Thank you for reaching out. Regarding the double charge for your gift
order, this billing issue needs to be escalated for further investigation before any
refund can be processed. For the damaged replacement item, you are eligible for a refund
or a replacement even if the box was opened. The return and refund for the damaged item
will be handled separately from the billing dispute. We will ensure both issues are
addressed promptly.

billing_result: Regarding the double charge for your gift order, we need to investigate
the duplicate charges before any refund can be issued. For the damaged replacement item,
the return process would typically be handled separately from billing. Since your issue
involves a charge dispute, this matter must be escalated for further review.
returns_result: For the damaged replacement item, you can receive a refund or a
replacement even if the box was opened. Regarding the billing issue of being charged
twice for a gift order, this falls outside the returns policy and requires escalation for
manual review.
resolution: Thank you for reaching out. Regarding the double charge for your gift order,
this billing issue needs to be escalated for further investigation before any refund can
be processed. For the damaged replacement item, you are eligible for a refund or a
replacement even if the box was opened. The return and refund for the damaged item will
be handled separately from the billing dispute. We will ensure both issues are addressed
promptly.
-----

=== FINAL STATE ===

Shared message state:
1. HumanMessage: I was charged twice for a gift order, and the replacement item also
arrived damaged. Explain what happens with the billing issue and with the return.
2. AIMessage: Billing specialist: Regarding the double charge for your gift order, we
need to investigate the duplicate charges before any refund can be issued. For the
damaged replacement item, the return process would typically be handled separately from
billing. Since your issue involves a charge dispute and a manual billing override may be
necessary, I will escalate this matter for further review.
3. AIMessage: Returns specialist: For the damaged replacement item, you can receive a
refund or a replacement even if the box was opened. Regarding the billing issue of being
charged twice for a gift order, this falls outside the returns policy and requires
escalation for manual review.
4. AIMessage: Thank you for reaching out. Regarding the double charge for your gift
order, we need to investigate the duplicate charges before issuing any refund, so this
matter will be escalated for further review and manual billing override if necessary. For
the damaged replacement item, you can return it for a refund or request another
replacement, even if the box has been opened. Please let us know if you would like to
proceed with the return or replacement, and we will assist you with the next steps.

Final resolution:
Thank you for reaching out. Regarding the double charge for your gift order, we need to
investigate the duplicate charges before issuing any refund, so this matter will be
escalated for further review and manual billing override if necessary. For the damaged
replacement item, you can return it for a refund or request another replacement, even if
the box has been opened. Please let us know if you would like to proceed with the return
or replacement, and we will assist you with the next steps.

```

This example uses the message list as the common conversational interface between the specialists and the coordinator. Each specialist appends an `AIMessage` containing its result, while `billing_result` and `returns_result` preserve the same outputs as structured fields for direct programmatic access. The messages show what each specialist contributed; they do not expose the model's hidden reasoning. The synthesis node reads the original request and both specialist responses from one continuous history. The streaming output makes that history visible as it grows. Shared messages are therefore more than logs: they are shared conversational state that later nodes can read and extend. The structured fields remain useful when the application needs stable values without searching or interpreting the transcript.

5.2.4 Shared state schemas: making collaboration explicit

Underneath both handoffs and shared message lists is LangGraph's shared state. A graph defines a state schema, nodes read the current state, and nodes return partial updates. For each state key, a reducer determines whether a new value replaces, appends to, or otherwise combines with the existing value. State is therefore the main in-process data contract between nodes. The schema declares the available channels, their types, and their merge behavior; it is not an access-control system. A key may also be optional or absent until an earlier node creates it, so downstream nodes must respect both the schema and the workflow's initialization rules. When a workflow needs stricter boundaries, use distinct input, output, internal, or subgraph schemas rather than assuming every node should see every field. A practical shared schema commonly contains three kinds of data:

- A conversational layer, usually messages, for dialogue continuity.
- A control layer, such as `active_agent`, `current_step`, or `route`, for execution decisions.
- A task-data layer, such as retrieved documents, approvals, rewritten questions, or specialist results.

This layered design creates a clear boundary between shared and private work. When a parent graph and a subgraph use the same state keys, the compiled subgraph can be added directly as a node and can read and update those shared channels. When their schemas differ, a wrapper node should translate parent state into subgraph input and map the result back into parent state. Persistence is a separate design decision. A subgraph inherits the parent graph's checkpointing for durable execution during an invocation, but it starts with fresh state on the next call by default. Compile the subgraph with `checkpointing=True` only when it must preserve state across calls on the same thread. Use `checkpointing=False` when the subgraph should be completely stateless. Parent checkpointing therefore does not automatically give every subagent long-term memory. As a practical default, store stable task facts in state and construct prompts inside the node that needs them. Original inputs, classifications, retrieved artifacts, approvals, and execution metadata usually belong in state. Fully formatted prompts usually do not because they tie the state contract to one particular model interaction. A reviewed instruction or generated plan may be stored when it is itself a meaningful task artifact, rather than merely text that will later be inserted into a prompt.

```
from __future__ import annotations
from pprint import pprint
from typing_extensions import TypedDict, NotRequired
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
from .config import OPENAI_API_KEY
from .support_team_shared import ask_billing_specialist, ask_returns_specialist

MODEL = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class StructuredSupportState(TypedDict):
    question: str
    billing_answer: NotRequired[str]
    returns_answer: NotRequired[str]
    final_answer: NotRequired[str]

def billing_specialist(state: StructuredSupportState) -> dict:
    answer = ask_billing_specialist.invoke({"question": state["question"]})
    return {"billing_answer": answer}

def returns_specialist(state: StructuredSupportState) -> dict:
    answer = ask_returns_specialist.invoke({"question": state["question"]})
    return {"returns_answer": answer}
```



```

def compose_final(state: StructuredSupportState) -> dict:
    answer = MODEL.invoke(f"""
Write one customer-facing answer from the structured specialist outputs below.

Billing:
{state["billing_answer"]}

Returns:
{state["returns_answer"]}

Be clear, concise, and consistent with both specialists.
""")

    return {"final_answer": answer.content}

builder = StateGraph(StructuredSupportState)
builder.add_node("billing_specialist", billing_specialist)
builder.add_node("returns_specialist", returns_specialist)
builder.add_node("compose_final", compose_final)

builder.add_edge(START, "billing_specialist")
builder.add_edge("billing_specialist", "returns_specialist")
builder.add_edge("returns_specialist", "compose_final")
builder.add_edge("compose_final", END)

graph = builder.compile()

def print_state(title: str, state: dict) -> None:
    print(title)
    pprint(state, sort_dicts = False)
    print("-" * 80)

if __name__ == "__main__":
    question = (
        "I was charged twice for a gift order, and the replacement item also "
        "arrived damaged. Explain what happens with the billing issue and the return."
    )

    initial_state: StructuredSupportState = {"question": question}
    running_state: dict = dict(initial_state)

    print_state("Initial shared state:", running_state)

    for update in graph.stream(initial_state, stream_mode = "updates"):
        for node_name, node_update in update.items():
            print(f"Node executed: {node_name}")
            print("Node update:")
            pprint(node_update, sort_dicts = False)
            print()

            running_state.update(node_update)
            print_state("Shared state after node:", running_state)

    print("Final customer-facing answer:\n")
    print(running_state["final_answer"])

```

vol3_ch_05\src\support_case_shared_state.py

```

(.venv) C:\Users\alexc\11m-app\vol3_ch_05> python -m src.support_case_shared_state

Initial shared state:
{'question': 'I was charged twice for a gift order, and the replacement item '
            'also arrived damaged. Explain what happens with the billing '
            'issue and the return.'}
-----
Node executed: billing specialist
Node update:
{'billing_answer': 'Regarding the double charge for your gift order, we need '
                  'to investigate the duplicate charges before any refund can '
                  'be issued. For the damaged replacement item, the return '
                  'process is separate from billing and would need to be '
                  'handled through the returns department. Since your issue '
                  'involves a charge dispute, I will escalate this for '
                  'further investigation.'}

Shared state after node:
{'question': 'I was charged twice for a gift order, and the replacement item '
            'also arrived damaged. Explain what happens with the billing '
            'issue and the return.',
 'billing_answer': 'Regarding the double charge for your gift order, we need '
                  'to investigate the duplicate charges before any refund can '
                  'be issued. For the damaged replacement item, the return '

```

```

        'process is separate from billing and would need to be '
        'handled through the returns department. Since your issue '
        'involves a charge dispute, I will escalate this for '
        'further investigation.')}
-----
Node executed: returns_specialist
Node update:
{'returns_answer': 'The replacement item that arrived damaged can be refunded '
                    'or replaced, even if the box was opened. However, the '
                    'billing issue regarding being charged twice for a gift '
                    'order falls outside the returns policy and requires '
                    'escalation for manual review.'}

Shared state after node:
{'question': 'I was charged twice for a gift order, and the replacement item '
             'also arrived damaged. Explain what happens with the billing '
             'issue and the return.',
 'billing_answer': 'Regarding the double charge for your gift order, we need '
                   'to investigate the duplicate charges before any refund can '
                   'be issued. For the damaged replacement item, the return '
                   'process is separate from billing and would need to be '
                   'handled through the returns department. Since your issue '
                   'involves a charge dispute, I will escalate this for '
                   'further investigation.',
 'returns_answer': 'The replacement item that arrived damaged can be refunded '
                   'or replaced, even if the box was opened. However, the '
                   'billing issue regarding being charged twice for a gift '
                   'order falls outside the returns policy and requires '
                   'escalation for manual review.'}
-----
Node executed: compose_final
Node update:
{'final_answer': 'Thank you for reaching out. We will escalate the double '
                  'charge on your gift order for further investigation before '
                  'any refund can be issued. Regarding the damaged replacement '
                  'item, you can choose to have it refunded or replaced, even '
                  'if the box was opened. Please note that the billing issue is '
                  'handled separately from returns and is currently being '
                  'reviewed.'}

Shared state after node:
{'question': 'I was charged twice for a gift order, and the replacement item '
             'also arrived damaged. Explain what happens with the billing '
             'issue and the return.',
 'billing_answer': 'Regarding the double charge for your gift order, we need '
                   'to investigate the duplicate charges before any refund can '
                   'be issued. For the damaged replacement item, the return '
                   'process is separate from billing and would need to be '
                   'handled through the returns department. Since your issue '
                   'involves a charge dispute, I will escalate this for '
                   'further investigation.',
 'returns_answer': 'The replacement item that arrived damaged can be refunded '
                   'or replaced, even if the box was opened. However, the '
                   'billing issue regarding being charged twice for a gift '
                   'order falls outside the returns policy and requires '
                   'escalation for manual review.',
 'final_answer': 'Thank you for reaching out. We will escalate the double '
                  'charge on your gift order for further investigation before '
                  'any refund can be issued. Regarding the damaged replacement '
                  'item, you can choose to have it refunded or replaced, even '
                  'if the box was opened. Please note that the billing issue is '
                  'handled separately from returns and is currently being '
                  'reviewed.'}
-----
Final customer-facing answer:

Thank you for reaching out. We will escalate the double charge on your gift order for
further investigation before any refund can be issued. Regarding the damaged replacement
item, you can choose to have it refunded or replaced, even if the box was opened. Please
note that the billing issue is handled separately from returns and is currently being
reviewed.

```

This example shows the structured-state alternative to shared conversational memory. The billing specialist writes `billing_answer`, the returns specialist writes `returns_answer`, and the final node reads those two fields directly. No later node has to search a transcript or

ask a model to recover facts that earlier steps have already established. The state fields are stable, named parts of the workflow contract, which makes them easier to validate, test, and inspect. The distinction is practical. Use messages when later steps need the conversation itself. Use structured fields for business facts, control decisions, identifiers, approvals, and artifacts that must remain machine-readable. A production workflow often uses both: messages for conversational continuity and structured state for facts that should not be reinterpreted.

5.2.5 A2A: communication beyond one process

Everything above assumes that the agents collaborate inside one LangGraph application. Once communication crosses a process or service boundary, a local shared state object is no longer enough. The system needs a protocol and deployment model that support capability description, discovery, authentication metadata, task tracking, and conversation continuity across network calls. Agent-to-agent protocols therefore use explicit message envelopes, task identifiers, context identifiers, and capability metadata because the receiving service cannot access the sender's in-process graph state directly.

Agent Server makes this boundary concrete through its A2A endpoint. For an agent deployed behind Agent Server to handle A2A text messages, its graph state must include a messages key. This is an Agent Server adapter requirement, not a universal rule for every A2A implementation. The endpoint uses `contextId` to group related interactions and `taskId` to identify the task associated with a request. These identifiers correlate remote calls, but they do not replace the message content or the persisted state maintained by each service. This boundary also clarifies the difference between A2A and MCP. A2A coordinates messages and tasks between agents. MCP standardizes how an AI application accesses tools, resources, and prompts. The two protocols can be used together because they operate at different layers. The following example uses two remote LangGraph services exposed through Agent Server and a separate coordinator process. The coordinator already knows the base URL of each service. In this example, each service contains only the automatically created default assistant, so `/assistants/search` can resolve that assistant for the configured graph. The coordinator then retrieves the assistant's Agent Card and sends A2A requests to `/a2a/{assistant_id}`. This is not full network discovery: service locations are configured in advance, while assistant lookup happens inside each known service. The coordinator exchanges protocol messages and correlation identifiers instead of sharing an in-memory Python state object with the specialists. Create the following folder and file structure under `vol3_ch_05\scr\`

```
billing_service/
  agent.py
  langgraph.json

returns_service/
  agent.py
  langgraph.json

coordinator/
  coordinator.py
```

Now let's build the billing service agent:

```
from __future__ import annotations
from typing import Any, Dict, List, TypedDict
```

```

from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START

MODEL = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class A2AState(TypedDict):
    messages: List[Dict[str, Any]]

SYSTEM_PROMPT = """You are the billing specialist for an online store.
Answer only the billing side of the request.

Billing policy:
- Duplicate charges must be investigated before any refund is issued.
- Invoices are available in the customer account.
- Billing disputes and manual billing overrides require escalation.

Keep the answer short, concrete, and customer-facing.
"""

def latest_user_text(messages: List[Dict[str, Any]]) -> str:
    for message in reversed(messages):
        if message.get("role") == "user":
            return str(message.get("content", ""))
    return ""

def answer(state: A2AState) -> Dict[str, Any]:
    user_text = latest_user_text(state["messages"])

    reply = MODEL.invoke(
        [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_text}
        ]
    )

    return {
        "messages": state["messages"] + [
            {"role": "assistant", "content": reply.content}
        ]
    }

builder = StateGraph(A2AState)
builder.add_node("answer", answer)
builder.add_edge(START, "answer")
graph = builder.compile()

```

vol3_ch_05\scr\billing_service\agent.py

Now let's build the `langgraph.json` configuration file for the LangGraph CLI and runtime. LangGraph uses this file to know where your graphs live, what code directories to include as dependencies, and which environment file to load. By default, the CLI looks for `langgraph.json` in the current directory.

```

{
  "dependencies": [".", ".."],
  "graphs": {
    "billing_agent": "./agent.py:graph"
  },
  "env": ".env"
}

```

vol3_ch_05\scr\billing_service\langgraph.json

We do the same for the returns service:

```

from __future__ import annotations
from typing import Any, Dict, List, TypedDict
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START

MODEL = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class A2AState(TypedDict):
    messages: List[Dict[str, Any]]

```

```

SYSTEM_PROMPT = """You are the returns specialist for an online store.
Answer only the returns side of the request.

Returns policy:
- Unopened items can be returned within 30 days of delivery.
- Damaged items can be refunded or replaced, even if the box was opened.
- Gift returns are eligible for store credit unless the item arrived damaged.
- Requests outside these rules require manual review.

Keep the answer short, concrete, and customer-facing.
"""

def latest_user_text(messages: List[Dict[str, Any]]) -> str:
    for message in reversed(messages):
        if message.get("role") == "user":
            return str(message.get("content", ""))
    return ""

def answer(state: A2AState) -> Dict[str, Any]:
    user_text = latest_user_text(state["messages"])

    reply = MODEL.invoke(
        [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_text}
        ]
    )

    return {
        "messages": state["messages"] + [
            {"role": "assistant", "content": reply.content}
        ]
    }

builder = StateGraph(A2AState)
builder.add_node("answer", answer)
builder.add_edge(START, "answer")
graph = builder.compile()

```

vol3_ch_05\scr\returns_service\agent.py

```

{
    "dependencies": [".", ".."],
    "graphs": {
        "returns_agent": "./agent.py:graph"
    },
    "env": ".env"
}

```

vol3_ch_05\scr\returns_service\langgraph.json

And finally let's build the coordinator:

```

from __future__ import annotations
import asyncio
import json
import os
import uuid
from pathlib import Path
from typing import Any, Dict, Tuple
import aiohttp
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage, SystemMessage

dotenv_path = Path(__file__).resolve().parents[3] / ".env"
print(f>Loading .env from: {dotenv_path}")
print(f".env exists: {dotenv_path.exists()}")

load_dotenv(dotenv_path)

BILLING_BASE_URL = "http://127.0.0.1:2024"
RETURNS_BASE_URL = "http://127.0.0.1:2025"

BILLING_GRAPH_ID = "billing_agent"
RETURNS_GRAPH_ID = "returns_agent"

SYNTHESIZER = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

async def resolve_assistant_id(

```

```

    session: aiohttp.ClientSession,
    base_url: str,
    graph_id: str
) -> str:
    async with session.post(
        f"{base_url}/assistants/search",
        json = {"graph_id": graph_id, "limit": 1}
    ) as response:
        response.raise_for_status()
        data = await response.json()

    if not data:
        raise RuntimeError(f"No assistant found for graph_id={graph_id!r} at {base_url}")

    return data[0]["assistant_id"]

async def fetch_agent_card(
    session: aiohttp.ClientSession,
    base_url: str,
    assistant_id: str
) -> Dict[str, Any]:
    async with session.get(
        f"{base_url}/.well-known/agent-card.json",
        params = {"assistant_id": assistant_id}
    ) as response:
        response.raise_for_status()
        return await response.json()

def print_json(title: str, data: Dict[str, Any]) -> None:
    print(title)
    print(json.dumps(data, indent = 2, ensure_ascii = False))
    print()

def try_extract_text_from_parts(parts: list[dict]) -> str | None:
    for part in parts or []:
        if part.get("kind") == "text" and part.get("text"):
            return str(part["text"])
        if part.get("type") == "text" and part.get("text"):
            return str(part["text"])
        if part.get("text"):
            return str(part["text"])
    return None

def extract_text(result: Dict[str, Any]) -> str:
    result_obj = result.get("result", {}) or {}

    for artifact in result_obj.get("artifacts", []) or []:
        text = try_extract_text_from_parts(artifact.get("parts", []) or [])
        if text:
            return text

    status = result_obj.get("status", {}) or {}
    message = status.get("message", {}) or {}
    text = try_extract_text_from_parts(message.get("parts", []) or [])
    if text:
        return text

    message = result_obj.get("message", {}) or {}
    text = try_extract_text_from_parts(message.get("parts", []) or [])
    if text:
        return text

    messages = result_obj.get("messages", []) or []
    for msg in messages:
        text = try_extract_text_from_parts(msg.get("parts", []) or [])
        if text:
            return text

    return "(no text found)"

async def send_a2a_message(
    session: aiohttp.ClientSession,
    base_url: str,
    assistant_id: str,
    text: str,
    *,
    context_id: str | None = None,
    task_id: str | None = None
) -> Tuple[str, str | None, str | None, Dict[str, Any]]:
    message = {
        "role": "user",
        "parts": [{"kind": "text", "text": text}],
        "messageId": str(uuid.uuid4())
    }

```

```

    }

    if context_id:
        message["contextId"] = context_id
    if task_id:
        message["taskId"] = task_id

    payload = {
        "jsonrpc": "2.0",
        "id": str(uuid.uuid4()),
        "method": "message/send",
        "params": {"message": message}
    }

    print_json("=== OUTBOUND A2A PAYLOAD ===", payload)

    async with session.post(
        f"{base_url}/a2a/{assistant_id}",
        json = payload,
        headers = {"Accept": "application/json"}
    ) as response:
        response.raise_for_status()
        result = await response.json()

    print_json("=== RAW A2A RESPONSE ===", result)

    returned_context_id = result.get("result", {}).get("contextId") or context_id
    returned_task_id = result.get("result", {}).get("id") or task_id

    extracted = extract_text(result)
    print(f"=== EXTRACTED TEXT ===\n{extracted}\n")

    return extracted, returned_context_id, returned_task_id, result

async def main() -> None:
    customer_question = (
        "I was charged twice for a gift order, and the replacement item also arrived "
        "damaged. Explain what happens with the billing issue and with the return."
    )

    async with aiohttp.ClientSession() as session:
        billing_assistant_id = await resolve_assistant_id(
            session, BILLING_BASE_URL, BILLING_GRAPH_ID
        )
        returns_assistant_id = await resolve_assistant_id(
            session, RETURNS_BASE_URL, RETURNS_GRAPH_ID
        )

        billing_card = await fetch_agent_card(session, BILLING_BASE_URL,
                                              billing_assistant_id)
        returns_card = await fetch_agent_card(session, RETURNS_BASE_URL,
                                              returns_assistant_id)

        print("=== DISCOVERED AGENTS ===")
        print(f"Billing agent: {billing_card.get('name')} -> {billing_card.get('url')}")
        print(f>Returns agent: {returns_card.get('name')} -> {returns_card.get('url')}")
        print()

        print("=== MESSAGE SENT TO BILLING AGENT ===")
        print(customer_question)
        print()

        billing_reply, context_id, task_id, billing_raw = await send_a2a_message(
            session,
            BILLING_BASE_URL,
            billing_assistant_id,
            customer_question
        )
        print("=== BILLING AGENT REPLY ===")
        print(billing_reply)
        print(f"contextId={context_id}")
        print(f"taskId={task_id}")
        print()

        returns_prompt = (
            f"Original customer question:\n{customer_question}\n\n"
            f"Billing specialist finding:\n{billing_reply}\n\n"
            "Now answer only the return and replacement side."
        )

        print("=== MESSAGE SENT TO RETURNS AGENT ===")
        print(returns_prompt)
        print()

        returns_reply, context_id, task_id, returns_raw = await send_a2a_message(

```



```

        session,
        RETURNS_BASE_URL,
        returns_assistant_id,
        returns_prompt,
        context_id = context_id,
        task_id = task_id
    )
    print("=== RETURNS AGENT REPLY ===")
    print(returns_reply)
    print(f"contextId={context_id}")
    print(f"taskId={task_id}")
    print()
    final_answer = SYNTHESIZER.invoke(
        [
            SystemMessage(
                content=(
                    "You are the support coordinator. Combine the two specialist "
                    "findings into one short customer-facing answer. Do not invent "
                    "policy beyond what the specialists already stated."
                )
            ),
            HumanMessage(
                content=(
                    f"Customer question:\n{customer_question}\n\n"
                    f"Billing specialist:\n{billing_reply}\n\n"
                    f>Returns specialist:\n{returns_reply}"
                )
            )
        ]
    )
    print("=== FINAL CUSTOMER ANSWER ===")
    print(final_answer.content)
if __name__ == "__main__":
    asyncio.run(main())

```

vol3_ch_03\scr\coordinator\coordinator.py

This example separates service discovery from assistant lookup. `BILLING_BASE_URL` and `RETURNS_BASE_URL` identify the two already-known services. Because each service in this example contains only its automatically created default assistant, `/assistants/search` resolves that assistant for the configured graph. The Agent Card then describes the assistant's published capabilities, endpoint, and declared authentication requirements. Retrieving an Agent Card provides metadata for connection and capability negotiation; by itself, it does not authenticate the remote service or prove its identity. Production deployments must still use transport security and the authentication scheme required by the server. The main network boundary appears in `send_a2a_message`. The coordinator creates a JSON-RPC envelope containing a message, a `messageId`, and optional `contextId` and `taskId` values. The first call receives correlation identifiers from the remote server, and later calls reuse them according to Agent Server's A2A conversation model. Those identifiers associate related interactions and traces across services, but they do not reconstruct application state by themselves. Each specialist still needs the relevant message content and its own persistence strategy. The key contrast with an in-process graph is therefore not "identifiers instead of state"; it is explicit protocol messages and correlation identifiers instead of direct access to the same state object. The response-extraction helpers handle the result forms returned by Agent Server, where text may appear in task artifacts or status messages. After collecting both specialist responses, the coordinator synthesizes one customer-facing answer. In this example, A2A provides the cross-service message transport and correlation model, Agent Server provides assistant lookup and hosting, and the coordinator remains responsible for orchestration and final synthesis. Now that all code is present, install the local Agent Server tooling and the Python dependencies

required by the three processes. `langgraph dev` is the local development path, and its default port is 2024, so the second service is started on 2025. Agent Server's A2A support requires `langgraph-api >= 0.4.21`.

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> python -m pip install "langgraph-cli[inmem]"
"langgraph-api>=0.4.21" langchain langchain-openai aiohttp
```

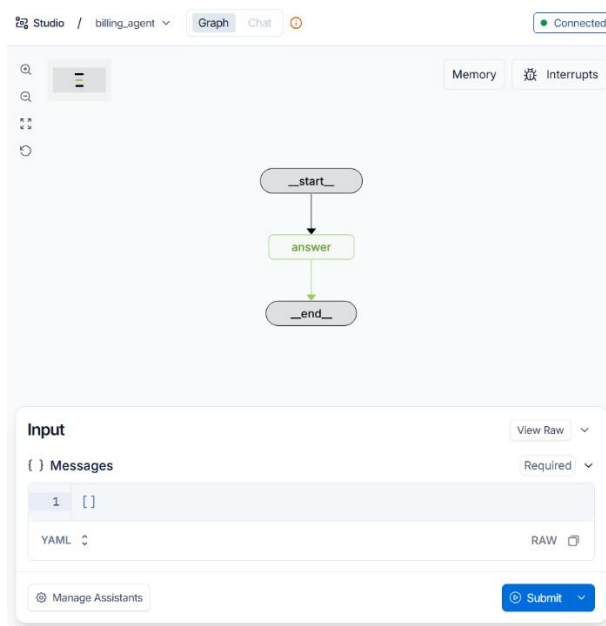
Start the billing service:

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> cd src\billing_service
(.venv) C:\Users\alexc\llm-app\vol3_ch_05\src\billing_service> langgraph dev
```

You should see:

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05\src\billing_services>langgraph dev
INFO:langgraph_api.cli:
Welcome to
LANGGRAPH
- API: http://127.0.0.1:2024
- Studio UI: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
- API Docs: http://127.0.0.1:2024/docs
This in-memory server is designed for development and testing.
For production use, please use LangSmith Deployment.
[version] A newer version of langgraph-api is available: 0.7.76 → 0.7.98 (pip install -U langgraph-api)
2026-04-09T07:54:48.860804Z [info] [version] A newer version of langgraph-api is available: 0.7.76 → 0.7.98 (pip install -U langgraph-api) [version_ch
eck] api_variant=local_dev langgraph_api_version=0.7.76 message=[version] A newer version of langgraph-api is available: 0.7.76 → 0.7.98 (pip install -U l
anggraph-api) thread_name=Thread-3 (check_newer_version)
2026-04-09T07:54:49.452347Z [info] [Starting dev persistence flush loop [langgraph_runtime_inmem._persistence] api_variant=local_dev langgraph_api_versi
on=0.7.76 thread_name=MainThread]
2026-04-09T07:54:49.463667Z [info] [Using langgraph_runtime_inmem [langgraph_runtime] api_variant=local_dev langgraph_api_version=0.7.76 thread_name=Ma
inThread]
2026-04-09T07:54:49.472995Z [info] [Using auth of type=noop [langgraph_api.auth.middleware] api_variant=local_dev langgraph_api_version=0.7.76 th
read_name=MainThread]
2026-04-09T07:54:49.478320Z [info] [Starting In-Memory runtime with langgraph-api=0.7.76 and in-memory runtime=0.26.0 [langgraph_runtime_inmem.lifespan]
api_variant=local_dev langgraph_api_version=0.7.76 langgraph_runtime_inmem_version=0.26.0 thread_name=asyncio_0 version=0.7.76]
2026-04-09T07:54:49.650670Z [info] [No license key or control plane API key set, skipping metadata loop [langgraph_api.metadata] api_variant=local_dev l
anggraph_api_version=0.7.76 thread_name=MainThread]
2026-04-09T07:54:52.912555Z [warning] [Import for graph billing_agent exceeded the expected startup time. Slow initialization (often due to work executed a
t import time) can delay readiness, reduce scale-out capacity, and may cause deployments to be marked unhealthy.
To get detailed profiling of slow operations, set FF_PROFILE_IMPORTS=true [langgraph_api.timing.timer] api_variant=local_dev elapsed_seconds=3.261855000
0547975 graph_id=billing_agent langgraph_api_version=0.7.76 module=None name= graph_from_spec path= ./agent.py thread_name=asyncio_0
2026-04-09T07:54:52.920502Z [info] [Application started up in 3.639s [langgraph_api.timing.timer] api_variant=local_dev elapsed=3.6385912999976426 langg
raph_api_version=0.7.76 thread_name=MainThread]
2026-04-09T07:54:53.015658Z [info] [Starting cron scheduler [langgraph_api.cron_scheduler] api_variant=local_dev langgraph_api_version=0.7.76 thr
```

And a browser TAB should open up, showing the following page:



Now start the returns service in a second terminal:

```
C:\Users\alexc\llm-app> .vent\Scripts\activate
(.venv) C:\Users\alexc\llm-app> cd vol3_ch_05
```

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> cd src\returns_service
(.venv) C:\Users\alexc\llm-app\vol3_ch_05\src\returns_service> langgraph dev --port 2025
```

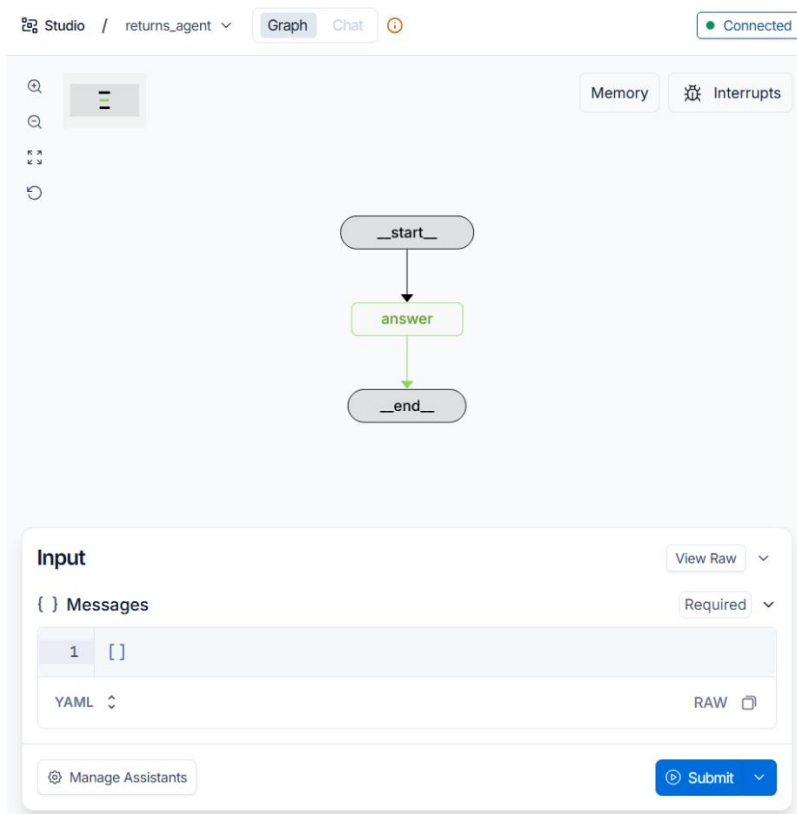
You should see:

```
(.venv) C:\Users\alexc\llm-app\vol3_ch_05\src\returns_services> langgraph dev --port 2025
INFO:langgraph_api.cli:
  Welcome to
  LANGGRAPH
- 🚀 API: http://127.0.0.1:2025
- 🖥️ Studio UI: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2025
- 📖 API Docs: http://127.0.0.1:2025/docs

This in-memory server is designed for development and testing.
For production use, please use LangSmith Deployment.

[version] A newer version of langgraph-api is available: 0.7.76 → 0.7.98 (pip install -U langgraph-api)
2026-04-09T07:57:04.581289Z [info] [version] A newer version of langgraph-api is available: 0.7.76 → 0.7.98 (pip install -U langgraph-api) [version_ch
eck] api_variant=local_dev langgraph_api_version=0.7.76 message='[version] A newer version of langgraph-api is available: 0.7.76 → 0.7.98 (pip install -U l
anggraph-api)' thread_name='Thread-3' _check_newer_version'
2026-04-09T07:57:05.162959Z [info] ] Starting dev persistence flush loop [langgraph_runtime_inmem._persistence] api_variant=local_dev langgraph_api_versi
on=0.7.76 thread_name=MainThread
2026-04-09T07:57:05.173518Z [info] ] Using langgraph_runtime_inmem [langgraph_runtime] api_variant=local_dev langgraph_api_version=0.7.76 thread_name=Ma
inThread
2026-04-09T07:57:05.184029Z [info] ] Using auth of type=noop [langgraph_api.auth.middleware] api_variant=local_dev langgraph_api_version=0.7.76 th
read_name=MainThread
2026-04-09T07:57:05.189718Z [info] ] Starting In-Memory runtime with langgraph-api=0.7.76 and in-memory runtime=0.26.0 [langgraph_runtime_inmem.lifespan]
api_variant=local_dev langgraph_api_version=0.7.76 langgraph_runtime_inmem_version=0.26.0 thread_name=asyncio_0 version=0.7.76
2026-04-09T07:57:05.258934Z [info] ] 1 change detected [watchfiles.main] api_variant=local_dev langgraph_api_version=0.7.76 thread_name=Main
Thread
2026-04-09T07:57:05.361671Z [info] ] No license key or control plane API key set, skipping metadata loop [langgraph_api.metadata] api_variant=local_dev l
anggraph_api_version=0.7.76 thread_name=MainThread
2026-04-09T07:57:05.613141Z [info] ] 4 changes detected [watchfiles.main] api_variant=local_dev langgraph_api_version=0.7.76 thread_name=Main
Thread
```

And a browser TAB should open up, showing the following page:



We are now all set to run the coordinator in a third terminal.

```
C:\Users\alexc\llm-app> .venv\Scripts\activate
(.venv) C:\Users\alexc\llm-app> cd vol3_ch_05
(.venv) C:\Users\alexc\llm-app\vol3_ch_05> cd src\coordinator
(.venv) C:\Users\alexc\llm-app\vol3_ch_05\coordinator> python coordinator.py

Loading .env from: C:\Users\alexc\llm-app\.env
.env exists: True
OPENAI_API_KEY present: True
```

```

=== DISCOVERED AGENTS ===
Billing agent: billing_agent -> http://127.0.0.1:2024/a2a/2fc886a5-11d7-5a02-9eac-3d6571da32fc
Returns agent: returns_agent -> http://127.0.0.1:2025/a2a/89bfc1bf-bd43-5f66-b50f-5b26d5ac92fc

=== MESSAGE SENT TO BILLING AGENT ===
I was charged twice for a gift order, and the replacement item also arrived damaged.
Explain what happens with the billing issue and with the return.

=== OUTBOUND A2A PAYLOAD ===
{
  "jsonrpc": "2.0",
  "id": "ef833ec1-61c8-47ab-9f28-6116f942f8b1",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [
        {
          "kind": "text",
          "text": "I was charged twice for a gift order, and the replacement item also
arrived damaged. Explain what happens with the billing issue and with the return."
        }
      ],
      "messageId": "11fe229f-9e99-4db9-a62a-e5d30652f575"
    }
  }
}

=== RAW A2A RESPONSE ===
{
  "jsonrpc": "2.0",
  "id": "ef833ec1-61c8-47ab-9f28-6116f942f8b1",
  "result": {
    "id": "c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-94b857beaec2",
    "contextId": "c9eda5a6-1452-4bc2-8c06-996aa779abed",
    "history": [
      {
        "role": "agent",
        "parts": [
          {
            "text": "I was charged twice for a gift order, and the replacement item also
arrived damaged. Explain what happens with the billing issue and with the return."
          }
        ],
        "messageId": "11fe229f-9e99-4db9-a62a-e5d30652f575",
        "taskId": "c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-94b857beaec2",
        "contextId": "c9eda5a6-1452-4bc2-8c06-996aa779abed"
      },
      {
        "role": "agent",
        "parts": [
          {
            "text": "Hello! How can I assist you with your billing today?"
          }
        ],
        "messageId": "019d714b-1b66-7e41-96e2-79cea141b4b7",
        "taskId": "c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-94b857beaec2",
        "contextId": "c9eda5a6-1452-4bc2-8c06-996aa779abed"
      }
    ],
    "status": {
      "state": "completed",
      "timestamp": "2026-04-09T08:10:40.870972+00:00"
    },
    "artifacts": [
      {
        "artifactId": "3d06b0f4-55b6-4d0d-8536-994c32e3c9e1",
        "name": "Assistant Response",
        "description": "Response from assistant 2fc886a5-11d7-5a02-9eac-3d6571da32fc",
        "parts": [
          {
            "text": "Hello! How can I assist you with your billing today?"
          }
        ]
      }
    ]
  }
}

```

```

    }
  ]
}

=== EXTRACTED TEXT ===
Hello! How can I assist you with your billing today?

=== BILLING AGENT REPLY ===
Hello! How can I assist you with your billing today?
contextId=c9eda5a6-1452-4bc2-8c06-996aa779abed
taskId=c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-94b857beaec2

=== MESSAGE SENT TO RETURNS AGENT ===
Original customer question:
I was charged twice for a gift order, and the replacement item also arrived damaged.
Explain what happens with the billing issue and with the return.

Billing specialist finding:
Hello! How can I assist you with your billing today?

Now answer only the return and replacement side.

=== OUTBOUND A2A PAYLOAD ===
{
  "jsonrpc": "2.0",
  "id": "73812ca5-254e-4c9e-b473-ab90dd010f52",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [
        {
          "kind": "text",
          "text": "Original customer question:\nI was charged twice for a gift order, and
the replacement item also arrived damaged. Explain what happens with the billing issue
and with the return.\n\nBilling specialist finding:\nHello! How can I assist you with
your billing today?\n\nNow answer only the return and replacement side."
        }
      ],
      "messageId": "404d943b-abf2-4e5c-8b8e-7d742503725f",
      "contextId": "c9eda5a6-1452-4bc2-8c06-996aa779abed",
      "taskId": "c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-
94b857beaec2"
    }
  }
}

=== RAW A2A RESPONSE ===
{
  "jsonrpc": "2.0",
  "id": "73812ca5-254e-4c9e-b473-ab90dd010f52",
  "result": {
    "id": "c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-94b857beaec2",
    "contextId": "c9eda5a6-1452-4bc2-8c06-996aa779abed",
    "history": [
      {
        "role": "agent",
        "parts": [
          {
            "text": "Original customer question:\nI was charged twice for a gift order,
and the replacement item also arrived damaged. Explain what happens with the billing
issue and with the return.\n\nBilling specialist finding:\nHello! How can I assist you
with your billing today?\n\nNow answer only the return and replacement side."
          }
        ],
        "messageId": "404d943b-abf2-4e5c-8b8e-7d742503725f",
        "taskId": "c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-
94b857beaec2",
        "contextId": "c9eda5a6-1452-4bc2-8c06-996aa779abed"
      },
      {
        "role": "agent",
        "parts": [
          {
            "text": "Hello! If you need to return an item, please ensure it is unopened
and returned within 30 days of delivery for a full refund. If the item arrived damaged,

```

```
we can offer a refund or replacement even if opened. Gift returns are eligible for store
credit unless damaged. Let us know how we can assist!"
    }
    ],
    "messageId": "019d714b-27fc-7bd0-8d3c-3df8e4ea96d1",
    "taskId": "c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-
94b857beaec2",
    "contextId": "c9eda5a6-1452-4bc2-8c06-996aa779abed"
  }
},
"status": {
  "state": "completed",
  "timestamp": "2026-04-09T08:10:44.092900+00:00"
},
"artifacts": [
  {
    "artifactId": "9b1850fa-bbd7-4fe1-95e0-4e0e7f895041",
    "name": "Assistant Response",
    "description": "Response from assistant 89bfc1bf-bd43-5f66-b50f-5b26d5ac92fc",
    "parts": [
      {
        "text": "Hello! If you need to return an item, please ensure it is unopened
and returned within 30 days of delivery for a full refund. If the item arrived damaged,
we can offer a refund or replacement even if opened. Gift returns are eligible for store
credit unless damaged. Let us know how we can assist!"
      }
    ]
  }
]
}
}

=== EXTRACTED TEXT ===
Hello! If you need to return an item, please ensure it is unopened and returned within 30
days of delivery for a full refund. If the item arrived damaged, we can offer a refund or
replacement even if opened. Gift returns are eligible for store credit unless damaged.
Let us know how we can assist!

=== RETURNS AGENT REPLY ===
Hello! If you need to return an item, please ensure it is unopened and returned within 30
days of delivery for a full refund. If the item arrived damaged, we can offer a refund or
replacement even if opened. Gift returns are eligible for store credit unless damaged.
Let us know how we can assist!
contextId=c9eda5a6-1452-4bc2-8c06-996aa779abed
taskId=c9eda5a6-1452-4bc2-8c06-996aa779abed:019d714b-12f0-7b72-8c61-94b857beaec2

=== FINAL CUSTOMER ANSWER ===
Hello! Regarding the double charge for your gift order, we will review your billing and
correct any duplicate charges. For the damaged replacement item, you can return it even
if opened, and we can offer you a refund or a replacement. Please let us know how you'd
like to proceed, and we'll assist you with both the billing correction and the return.
```

The example shows the cross-service boundary clearly. Agent Server hosts each specialist and exposes its A2A endpoint. The coordinator locates the assistants within the two known services, retrieves their Agent Cards, sends protocol messages, and carries the returned correlation identifiers. The services remain operationally independent and do not share Python state. The coordinator is still responsible for deciding which agent to call, transferring relevant content, and synthesizing the final answer. This example uses message/send for complete responses; message/stream supports streamed responses, while tasks/get retrieves the current status and result of an existing task.

5.2.6 Choosing the right communication mechanism

The right choice depends on where communication occurs and what must be exchanged:

Requirement	Communication mechanism
Change the active behavior or owner of a task	Handoff
Preserve and extend conversational history	Shared message list
Exchange stable, typed, machine-readable data	Shared state schema

Exchange messages and tasks between independently deployed A2A agents

These mechanisms are complementary, but a system should use only those it needs. Messages are appropriate for conversational continuity, not for every business fact. Structured state is appropriate for stable data, but it should not replace the conversational history when later steps need to understand the dialogue. Handoffs should represent meaningful changes in behavior or responsibility, not every graph transition. A2A should be introduced when deployment boundaries require explicit network communication, not merely because the system contains several agents. A clear design makes changes of control, context selection, structured task data, and service boundaries explicit.

5.3 MULTI-AGENT GRAPHS IN LANGGRAPH: COORDINATION, SYNCHRONIZATION, STOPPING RULES

5.3.1 A multi-agent graph is still one graph

When a system uses multiple agents, it is easy to imagine a group of assistants talking to one another. That image can be helpful at first, but it is not the most accurate way to think about LangGraph. In LangGraph, a multi-agent system is still just a graph. The same runtime building blocks still apply: state, nodes, edges, reducers, checkpoints, interrupts, and routing. What changes is not the execution model itself. What changes is how responsibilities are divided across nodes or subgraphs. This point matters because it keeps the design concrete. Moving from one agent to several does not mean moving to a different kind of runtime. It simply means making more of the workflow explicit by drawing clear boundaries between specialized parts of the system.

That is also why multi-agent design is not always the right answer. Current LangChain guidance identifies three common reasons for adopting it: context management, distributed development, and parallelization. Multi-agent patterns help when specialized knowledge must be isolated, different teams need to develop and maintain capabilities independently, or independent subtasks should run concurrently. If those needs are weak, one agent with a focused prompt, a controlled tool set, and context loaded only when needed is usually simpler, cheaper, and easier to operate.

The skills pattern reinforces this point because it improves context management without introducing several independently coordinated agents. The first design question is therefore not “How many agents do we need?” but “What real boundary are we trying to make explicit?” At the architectural level, that boundary can be organised in different ways. You may keep control centralised under a supervisor, distribute it across peer agents, use a supervisor that calls specialists as tools, or build a hierarchy of supervisors and workers. The label matters less than the structure. What matters is who makes decisions, how work is delegated, and how results are merged. Coordination is therefore an architectural decision before it becomes a coding decision.

5.3.2 Coordination means deciding who handles the next step

In LangChain’s current multi-agent guidance, the main coordination patterns are subagents, handoffs, skills, routers, and custom workflows. Subagents keep a main agent in control and expose specialists as tools. Handoffs change the system’s active behaviour based on state; this may transfer control between specialists or reconfigure one agent’s prompt and tools. Routers classify or decompose the input and direct it to one or more specialists. Skills keep one agent in control while loading specialized prompts and knowledge only when needed. Custom workflows use LangGraph to combine deterministic logic, agents, interrupts, branches, loops, and parallel work. “Multi-agent” therefore describes a family of coordination designs, not one fixed architecture.

In LangGraph, subgraphs are the main way to compose reusable parts of a workflow. If the parent and child share the state channels used for communication, the compiled subgraph can be added directly as a node. The child reads from and writes to those shared channels, but it may also define additional private state keys used only inside the subgraph. The parent and child schemas therefore do not need to be identical. When the graphs do not share suitable communication keys, or when their inputs and outputs require transformation, the parent should invoke the child through a wrapper node. The wrapper maps parent state into the child's input schema and maps the child's output back into the parent state. This boundary is especially useful when a specialist needs private internal state, such as its own message history, while the parent keeps only globally relevant information. The design decision is therefore whether parent and child communicate directly through shared state channels or through an explicit translation layer.

Coordination also means deciding how parallel work converges. Several specialists may produce partial results, and a final node may synthesize those results into one answer. When outputs disagree, the graph needs an explicit decision rule, such as deterministic validation, scoring, voting, or review by another model or a human. The fan-in policy must also state which branches are required, which are optional, and what should happen if a branch fails or times out. Depending on the use case, the workflow may retry the branch, use a fallback, return a clearly labelled partial result, escalate to a human, or fail the request. Parallelism does not resolve disagreement or failure by itself; it moves both problems to the convergence point.

Another coordination decision is how much specialist history returns to the coordinating layer. This is not a universal LangGraph setting; it depends on the supervisor implementation or on the wrapper built around each specialist. For example, an implementation may return the complete worker history, only the final message, omit handoff bookkeeping messages, or forward a worker's final response without asking the supervisor to paraphrase it. Each choice changes context size, token cost, traceability, and the risk of confusing later nodes. The practical rule is to return only the information the next step needs while preserving enough structured evidence for debugging and audit. Coordination becomes clearer when three questions have explicit answers: who controls the next step, what data crosses the boundary, and in what representation that data is returned.

5.3.3 Synchronization is not thread management; it is graph semantics

In LangGraph, synchronisation does not primarily mean managing threads. It means understanding when nodes run and when their state updates become visible. The runtime follows a Pregel-style model based on discrete super-steps. At the start of a super-step, LangGraph selects the active nodes that can run. Those nodes execute against the state available at the beginning of that step. Their updates are applied together after execution, so nodes running in the same super-step do not see one another's new updates. The resulting state becomes available to nodes scheduled in the next super-step. A node starts inactive, becomes active when it receives an update through an incoming edge, runs, emits updates, and becomes inactive again if no further work is scheduled. The graph reaches runtime quiescence when every node is inactive, and no updates remain in transit. The correct mental model is therefore graph steps and state visibility, not threads. This creates the first layer of synchronisation: execution synchronisation. If one node has several outgoing edges, all target

nodes can run in parallel in the next super-step. The same applies when a conditional edge returns several destinations instead of one. This is how fan-out works in LangGraph. Parallel execution is explicit in the graph design: the current state is sent to several destinations, and those branches execute independently in the following step.

The second layer is state synchronisation. Parallel branches can safely update different state keys without special merge logic. If several branches update the same key in the same super-step, however, that key needs a reducer that defines how the updates are combined. Without a reducer, LangGraph normally raises `INVALID_CONCURRENT_GRAPH_UPDATE` rather than arbitrarily selecting one update and discarding the others. A reducer such as `operator.add` can accumulate list entries, but it defines only the mechanical merge operation. It does not determine whether the combined information is correct, consistent, or semantically compatible. A later aggregation node must still interpret what the merged results mean. Message history also needs a message-aware reducer. `add_messages` preserves message identities and can replace an existing message that has the same ID, while `MessagesState` provides a state schema with that reducer already configured. These mechanisms are safer than treating conversation history as an ordinary append-only Python list. Synchronisation determines when updates are combined; application logic determines what the combined results mean.

Handoffs connect control flow and state updates. A node or tool can return `Command` to update state and route to another node in one operation. From inside a subgraph, `graph=Command.PARENT` allows execution to continue at a node in the nearest parent graph. A handoff may also carry selected messages or structured task data, but this is a context-design choice rather than an automatic part of every handoff. One routing rule is critical: `Command` adds a dynamic route; it does not cancel static edges already defined from the same node. If both mechanisms are present, both paths may run. Use one routing mechanism per node unless that parallel behaviour is deliberate. Streaming makes this execution visible without changing it. The `updates` mode emits the partial state updates produced after each step, while `values` emits the complete state after each step. The `tasks` mode emits task start and finish events and requires a checkpointer. The `debug` mode provides the broadest diagnostic stream. Streaming therefore makes synchronisation observable; it does not define or alter synchronisation.

5.3.4 Stopping rules should be designed, not improvised

Stopping completes the picture because coordination and synchronisation are safe only when the workflow also defines its termination conditions. Runtime quiescence, introduced in the previous section, tells LangGraph that no more nodes are active and no updates remain in transit. It does not explain whether the application succeeded, exhausted its execution budget, reached a review boundary, or failed. Multi-agent workflows therefore need explicit application-level stopping rules in addition to the runtime's built-in completion behaviour.

The clearest success-based stopping rule is explicit routing to `END`. A node produces the final result, and the next edge deliberately ends the graph. This pattern is easy to understand and maintain because successful completion is visible in the graph structure. Once the system has enough evidence, sufficient agreement, or an approved answer, it should stop rather than

continue reasoning without a useful purpose. A separate category is bounded execution. These rules limit how much work the graph may perform even when it has not reached a successful result. They may constrain iterations, message growth, or reasoning depth. For example, a workflow may stop when a valid answer already exists, when the conversation history has become too large, or when a loop has consumed its allowed step budget. A graph that stops only after success may behave unpredictably on difficult, ambiguous, or pathological inputs. Bounded exits are therefore part of making execution predictable.

LangGraph's `recursion_limit` is a hard limit on the number of graph super-steps allowed in one run. If the graph reaches that limit before reaching a stop condition, LangGraph raises `GraphRecursionError`. The current step is available in runtime metadata, and `RemainingSteps` can be included in state so the graph can react before the hard limit is reached. This supports controlled fallbacks such as returning a clearly labelled best-effort answer, routing to review, or following an explicit failure path. An interrupt serves a different purpose: it pauses the graph rather than completing it. When the graph is compiled with a checkpoint, its state persisted and execution can later resume through `Command(resume=...)`. This is the appropriate pattern when the workflow reaches a boundary that requires approval, correction, or another human decision. In higher-risk systems, these review points should be designed as normal parts of the control flow rather than added later as exceptional cases.

Interrupts have strict replay semantics. The graph must be resumed with the same thread ID used when the interrupt occurred. On resume, the interrupted node starts again from the beginning, so any code placed before `interrupt()` runs again. An external side effect before the interrupt must therefore be idempotent, or it should be moved after the interrupt or into a separate node. This rule applies to API calls, database writes, financial operations, and any other action that changes an external system. A nested LangChain agent created with `create_agent` has its own local completion rule. Its internal loop ends when the model produces a final response without further tool calls or when a configured execution limit ends the run. Completing that inner loop completes only the nested agent node. The outer graph then continues according to its own edges. Similarly, reaching `recursion_limit` raises an exception rather than producing a successful result. These mechanisms represent four different outcomes: successful graph completion, bounded failure, human pause, and nested-agent completion. Keeping them separate prevents a pause, exception, or completed inner loop from being mistaken for successful completion of the entire graph.

5.3.5 Worked example: a coordinated support graph

The chapter already introduced a shared support module that defines stable billing and returns specialists, along with reusable tool wrappers for those specialists. That shared setup is the right foundation for a coordination example because it allows the code to focus on graph behaviour rather than rebuilding the same specialists. Architecturally, this example is a router implemented as a custom LangGraph workflow. The triage node performs explicit classification and routing; it is not an LLM supervisor dynamically choosing specialists as tools. The workflow is compact but end to end: triage decides whether billing, returns, or both specialists are required; the selected specialists run in parallel; a compose step combines their results; and the graph either finishes successfully or pauses for explicit manual review.

```

from __future__ import annotations
from operator import add
from typing import Annotated, Literal
from typing_extensions import NotRequired, TypedDict
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage, SystemMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import START, END, StateGraph
from langgraph.managed import RemainingSteps
from langgraph.types import Command, interrupt
from .config import OPENAI_API_KEY
from .support_team_shared import ask_billing_specialist, ask_returns_specialist

MODEL = init_chat_model("openai:gpt-4.1-mini", temperature = 0)

class SupportState(TypedDict):
    question: str
    needs_billing: bool
    needs_returns: bool
    needs_review: NotRequired[bool]
    billing_answer: NotRequired[str]
    returns_answer: NotRequired[str]
    draft_answer: NotRequired[str]
    final_answer: NotRequired[str]
    review_outcome: NotRequired[Literal["approve", "reject"]]
    trace: Annotated[list[str], add]
    remaining_steps: RemainingSteps

def triage(state: SupportState) -> dict:
    text = state["question"].lower()

    needs_billing = any(
        word in text for word in ["charged", "charge", "invoice", "payment", "billing"]
    )
    needs_returns = any(
        word in text for word in ["return", "refund", "damaged", "replacement"]
    )

    return {
        "needs_billing": needs_billing,
        "needs_returns": needs_returns,
        "trace": [
            f"triage: needs_billing={needs_billing}",
            f"triage: needs_returns={needs_returns}"
        ]
    }

def route_from_triage(state: SupportState):
    destinations: list[str] = []

    if state["needs_billing"]:
        destinations.append("billing_specialist")
    if state["needs_returns"]:
        destinations.append("returns_specialist")

    if not destinations:
        destinations.append("compose")

    return destinations

def billing_specialist(state: SupportState) -> dict:
    answer = ask_billing_specialist.invoke({"question": state["question"]})
    return {
        "billing_answer": answer,
        "trace": ["billing_specialist: completed"]
    }

def returns_specialist(state: SupportState) -> dict:
    answer = ask_returns_specialist.invoke({"question": state["question"]})
    return {
        "returns_answer": answer,
        "trace": ["returns_specialist: completed"]
    }

def compose(state: SupportState) -> dict:
    specialist_notes: list[str] = []

    if "billing_answer" in state:
        specialist_notes.append(f"Billing specialist:\n{state['billing_answer']}")
    if "returns_answer" in state:
        specialist_notes.append(f>Returns specialist:\n{state['returns_answer']}")

    messages = [

```

```

SystemMessage(
    content=(
        "You are the support coordinator.\n"
        "Write one clear customer-facing answer from the specialist notes.\n"
        "Do not invent policy beyond what the specialists already stated.\n"
        "If the notes imply escalation or manual review, say that clearly."
    )
),
HumanMessage(
    content=(
        f"Customer question:\n{state['question']}\n\n"
        f"Specialist notes:\n\n" + "\n\n".join(specialist_notes)
    )
)
]

draft = MODEL.invoke(messages).content

combined_notes = " ".join(
    [
        state.get("billing_answer", ""),
        state.get("returns_answer", "")
    ]
).lower()

low_budget = state["remaining_steps"] <= 2
needs_review = (
    not low_budget
    and any(
        phrase in combined_notes
        for phrase in ["escalate", "manual review", "further review"]
    )
)

update = {
    "draft_answer": draft,
    "needs_review": needs_review,
    "trace": [
        f"compose: low_budget={low_budget}",
        f"compose: needs review={needs_review}"
    ]
}

if low_budget:
    update["final_answer"] = (
        draft
        + "\n\n"
        + "This is a best-effort answer because the workflow is close to its "
        + "execution limit."
    )
elif not needs_review:
    update["final_answer"] = draft

return update

def route_after_compose(state: SupportState):
    if state.get("needs_review", False):
        return "manual_review"
    return END

def manual_review(state: SupportState) -> dict:
    decision = interrupt(
        {
            "type": "manual_review",
            "question": state["question"],
            "draft_answer": state["draft_answer"],
            "allowed_decisions": ["approve", "reject"]
        }
    )
    return {
        "review_outcome": decision,
        "trace": [f"manual_review: decision={decision}"]
    }

def finalize_review(state: SupportState) -> dict:
    if state["review_outcome"] == "approve":
        final_answer = state["draft_answer"]
    else:
        final_answer = (
            "This case needs human follow-up before we can confirm the next step. "
            "The automated workflow stopped at the review boundary."

```



```

    )

    return {
        "final_answer": final_answer,
        "trace": ["finalize review: completed"]
    }

builder = StateGraph(SupportState)
builder.add_node("triage", triage)
builder.add_node("billing_specialist", billing_specialist)
builder.add_node("returns_specialist", returns_specialist)
builder.add_node("compose", compose)
builder.add_node("manual_review", manual_review)
builder.add_node("finalize_review", finalize_review)

builder.add_edge(START, "triage")
builder.add_conditional_edges("triage", route_from_triage)
builder.add_edge("billing_specialist", "compose")
builder.add_edge("returns_specialist", "compose")
builder.add_conditional_edges("compose", route_after_compose)
builder.add_edge("manual_review", "finalize_review")
builder.add_edge("finalize_review", END)

graph = builder.compile(checkpointer = InMemorySaver())

def print_divider(title: str) -> None:
    print("\n" + "=" * 80)
    print(title)
    print("=" * 80)

def print_text_block(title: str, value: str | None) -> None:
    if value:
        print(f"{title}:\n")
        print(value)
        print()

def print_state_snapshot(values: dict) -> None:
    print("ROUTING DECISIONS:")
    print(f"- needs_billing={values.get('needs_billing')}")
    print(f"- needs_returns={values.get('needs_returns')}")
    print(f"- needs_review={values.get('needs_review')}")
    print()

    print_text_block("BILLING SPECIALIST OUTPUT", values.get("billing_answer"))
    print_text_block("RETURNS SPECIALIST OUTPUT", values.get("returns_answer"))
    print_text_block("COMPOSED DRAFT", values.get("draft_answer"))

    if values.get("needs_review", False) and "review_outcome" not in values:
        print("COMPOSE ROUTING:")
        print("- Routed to manual_review")
        print()
    elif not values.get("needs_review", False) and "final_answer" in values:
        print("COMPOSE ROUTING:")
        print("- Ended directly without manual review")
        print()

def print_resume_delta(before: dict, after: dict) -> None:
    print("STATE CHANGES AFTER RESUME:\n")

    if "review_outcome" in after and after.get("review_outcome") != before.get(
        "review_outcome"
    ):
        print("REVIEW OUTCOME:")
        print(f"- {after['review_outcome']}")
        print()

    if after.get("final_answer") != before.get("final_answer"):
        print_text_block("FINAL ANSWER PRODUCED", after.get("final_answer"))

def print_trace(values: dict) -> None:
    print("TRACE:")
    for item in values.get("trace", []):
        print(f"- {item}")
    print()

def run_demo case(
    *,
    title: str,
    question: str,
    review_decision: Literal["approve", "reject"] | None = None,
    recursion_limit: int = 10
) -> None:
    thread_id = title.lower().replace(" ", "-").replace("/", "-")

```



```

config = {
    "configurable": {"thread_id": thread_id},
    "recursion_limit": recursion_limit
}

print_divider(title)
print("QUESTION:\n")
print(question)
print()

first_result = graph.invoke(
    {
        "question": question,
        "trace": []
    },
    config = config
)

first_snapshot = graph.get_state(config)
first_values = dict(first_snapshot.values)

interrupted = "__interrupt__" in first_result

if interrupted:
    print("STATE AT REVIEW BOUNDARY:\n")
    print_state_snapshot(first_values)

    interrupt_payload = first_result["__interrupt__"][0].value
    print("INTERRUPT PAYLOAD:\n")
    print(interrupt_payload)
    print()

    if review_decision is None:
        print("No review decision supplied. Stopping at the review boundary.")
        return

    print(f"RESUMING WITH DECISION: {review_decision}\n")
    graph.invoke(Command(resume = review_decision), config = config)

    final_snapshot = graph.get_state(config)
    final_values = dict(final_snapshot.values)

    print_resume_delta(first_values, final_values)
    print_trace(final_values)

    print("FINAL ANSWER:\n")
    print(final_values["final answer"])
    print()
    return

print("FINAL STATE:\n")
print_state_snapshot(first_values)
print_trace(first_values)

print("FINAL ANSWER:\n")
print(first_values["final answer"])
print()

if __name__ == "__main__":
    run_demo_case(
        title = "CASE 1 - BILLING ONLY / NO REVIEW",
        question = "Where can I find my invoice for order ORD-100?"
    )

    run_demo_case(
        title = "CASE 2 - MIXED CASE / REVIEW APPROVED",
        question = (
            "I was charged twice for a gift order, and the replacement item "
            "also arrived damaged. What happens next?"
        ),
        review_decision = "approve"
    )

    run_demo_case(
        title = "CASE 3 - MIXED CASE / REVIEW REJECTED",
        question = (
            "I was charged twice for a gift order, and the replacement item "
            "also arrived damaged. What happens next?"
        ),
        review_decision = "reject"
    )

```

ch_04\scr\support_coordination_graph.py

```
(.venv) C:\Users\alexc\llm-app\vol13_ch_05> python -m src.support_coordination_graph

=====
CASE 1 - BILLING ONLY / NO REVIEW
=====
QUESTION:

Where can I find my invoice for order ORD-100?

FINAL STATE:

ROUTING DECISIONS:
- needs_billing=True
- needs_returns=False
- needs_review=False

BILLING SPECIALIST OUTPUT:

You can find your invoice for order ORD-100 in your customer account.

COMPOSED DRAFT:

You can find your invoice for order ORD-100 by logging into your customer account. Once
logged in, navigate to the order history or billing section to view and download your
invoice. If you need further assistance, please let us know.

COMPOSE ROUTING:
- Ended directly without manual review

TRACE:
- triage: needs_billing=True
- triage: needs_returns=False
- billing_specialist: completed
- compose: low_budget=False
- compose: needs_review=False

FINAL ANSWER:

You can find your invoice for order ORD-100 by logging into your customer account. Once
logged in, navigate to the order history or billing section to view and download your
invoice. If you need further assistance, please let us know.

=====
CASE 2 - MIXED CASE / REVIEW APPROVED
=====
QUESTION:

I was charged twice for a gift order, and the replacement item also arrived damaged. What
happens next?

STATE AT REVIEW BOUNDARY:

ROUTING DECISIONS:
- needs_billing=True
- needs_returns=True
- needs_review=True

BILLING SPECIALIST OUTPUT:

Since you were charged twice, we need to investigate the duplicate charges before any
refund can be issued. Additionally, because there is a charge dispute and a product
issue, this situation requires escalation. I will escalate this matter for further review
and resolution.

RETURNS SPECIALIST OUTPUT:

Since the replacement item arrived damaged, you are eligible for a refund or a
replacement for that item. Regarding the double charge for the gift order, this issue
falls outside the returns policy and requires escalation for manual review.

COMPOSED DRAFT:

Thank you for reaching out. Since you were charged twice and the replacement item arrived
damaged, we need to escalate your case for further review. Our team will investigate the
duplicate charges and the product issue to determine the appropriate resolution. You are
```

```

eligible for a refund or a replacement for the damaged item once the review is complete.
We appreciate your patience while we work to resolve this.

COMPOSE ROUTING:
- Routed to manual_review

INTERRUPT PAYLOAD:

{'type': 'manual_review', 'question': 'I was charged twice for a gift order, and the
replacement item also arrived damaged. What happens next?', 'draft answer': 'Thank you
for reaching out. Since you were charged twice and the replacement item arrived damaged,
we need to escalate your case for further review. Our team will investigate the duplicate
charges and the product issue to determine the appropriate resolution. You are eligible
for a refund or a replacement for the damaged item once the review is complete. We
appreciate your patience while we work to resolve this.', 'allowed_decisions':
['approve', 'reject']}

RESUMING WITH DECISION: approve

STATE CHANGES AFTER RESUME:

REVIEW OUTCOME:
- approve

FINAL ANSWER PRODUCED:

Thank you for reaching out. Since you were charged twice and the replacement item arrived
damaged, we need to escalate your case for further review. Our team will investigate the
duplicate charges and the product issue to determine the appropriate resolution. You are
eligible for a refund or a replacement for the damaged item once the review is complete.
We appreciate your patience while we work to resolve this.

TRACE:
- triage: needs_billing=True
- triage: needs_returns=True
- billing_specialist: completed
- returns_specialist: completed
- compose: low_budget=False
- compose: needs_review=True
- manual_review: decision=approve
- finalize_review: completed

FINAL ANSWER:

Thank you for reaching out. Since you were charged twice and the replacement item arrived
damaged, we need to escalate your case for further review. Our team will investigate the
duplicate charges and the product issue to determine the appropriate resolution. You are
eligible for a refund or a replacement for the damaged item once the review is complete.
We appreciate your patience while we work to resolve this.

=====
CASE 3 - MIXED CASE / REVIEW REJECTED
=====
QUESTION:

I was charged twice for a gift order, and the replacement item also arrived damaged. What
happens next?

STATE AT REVIEW BOUNDARY:

ROUTING DECISIONS:
- needs_billing=True
- needs_returns=True
- needs_review=True

BILLING SPECIALIST OUTPUT:

Since you were charged twice, we need to investigate the duplicate charges before any
refund can be issued. Additionally, because there is a charge dispute and an issue with
the replacement item, this matter must be escalated for further review. I recommend
contacting our support team to initiate the escalation process.

RETURNS SPECIALIST OUTPUT:

Since the replacement item arrived damaged, you are eligible for a refund or another
replacement for that item. However, the issue of being charged twice for a gift order
falls outside the returns policy and requires escalation for manual review.

```

COMPOSED DRAFT:

Thank you for reaching out. Since you were charged twice for your gift order and the replacement item arrived damaged, we need to escalate your case for further review. The duplicate charge requires investigation before any refund can be issued, and the damaged replacement item makes you eligible for a refund or another replacement. Please contact our support team to initiate the escalation process so we can resolve both issues for you.

COMPOSE ROUTING:

- Routed to manual_review

INTERRUPT PAYLOAD:

```
{'type': 'manual_review', 'question': 'I was charged twice for a gift order, and the replacement item also arrived damaged. What happens next?', 'draft answer': 'Thank you for reaching out. Since you were charged twice for your gift order and the replacement item arrived damaged, we need to escalate your case for further review. The duplicate charge requires investigation before any refund can be issued, and the damaged replacement item makes you eligible for a refund or another replacement. Please contact our support team to initiate the escalation process so we can resolve both issues for you.', 'allowed_decisions': ['approve', 'reject']}
```

RESUMING WITH DECISION: reject

STATE CHANGES AFTER RESUME:

REVIEW OUTCOME:

- reject

FINAL ANSWER PRODUCED:

This case needs human follow-up before we can confirm the next step. The automated workflow stopped at the review boundary.

TRACE:

```
- triage: needs_billing=True
- triage: needs_returns=True
- billing_specialist: completed
- returns_specialist: completed
- compose: low_budget=False
- compose: needs_review=True
- manual_review: decision=reject
- finalize_review: completed
```

FINAL ANSWER:

This case needs human follow-up before we can confirm the next step. The automated workflow stopped at the review boundary.

This graph uses an explicit route-specialise-converge pattern. The triage node selects billing, returns, or both. When it returns both destinations, the two specialists run in the same super-step against the same incoming state snapshot. Neither specialist can see the other specialist's new output while that step is executing. LangGraph applies both sets of updates at the end of the super-step and `compose` runs in the following super-step with the merged state. Each specialist performs one narrow task and writes its result into a separate state field. The `trace` field is the only key updated concurrently by both branches, so it uses an additive reducer. That reducer retains both trace entries. Without it, the run would raise a concurrent-update error when both specialists attempted to update `trace` in the same super-step. The `compose` node is the semantic fan-in point. It does not merely receive mechanically merged state; it interprets the selected specialist results and produces one customer-facing draft. Synchronisation determines when the specialist outputs become available, while `compose` determines what those outputs mean together.

The example adds application-level stopping rules to normal graph execution. If the graph has enough information and no review is required, `compose` writes `final_answer` and routes

to END. If the workflow is running low on its execution budget, `RemainingSteps` allows it to degrade gracefully and return a best-effort answer before reaching the hard recursion limit. If the specialist notes indicate that safe autonomous processing has ended, the graph routes to `manual_review`, pauses through an interrupt, and waits for explicit resumption. Together, these paths demonstrate a success-based exit, a budget-based exit, and fail-closed human review. The code also makes the pause-and-resume contract concrete. The graph is compiled with a checkpoint, resumed with the same thread ID, and continued through `Command(resume="approve")`. As explained earlier, `manual_review` performs no external side effects before `interrupt()`. Replaying the node on resume therefore cannot duplicate an external action. For approval boundaries, replay safety is part of the stopping design rather than a minor implementation detail.

5.4 SUMMARY

Multi-agent architecture is useful when one agent can no longer perform a workflow reliably within a clear prompt, context, toolset, and authority boundary. Dividing the system into several agents does not remove complexity. It redistributes that complexity across components whose responsibilities and interactions can be made more explicit. The design is justified only when that separation improves control, context isolation, security, evaluation, maintainability, or operational ownership. The manager/worker pattern keeps one coordinator responsible for the overall objective. The manager decomposes the request, delegates specialist work, and synthesises the returned results. This centralised structure provides a clear coordination point and allows workers to operate with narrower prompts and cleaner contexts. It is most useful when several contributions must be combined, but it can become an unnecessary bottleneck when a request can be handled directly by one specialist. Specialisation creates narrower ownership boundaries. A specialist should exist because it owns a distinct domain, toolset, policy, evaluation criterion, or trust boundary—not simply because the application is intended to look multi-agent. Explicit routing works well when one request can be assigned to a single specialist. Cross-domain requests require a manager, multi-route fan-out, or another mechanism that preserves all relevant parts of the request. Structured review and debate add a challenge step before a result is accepted. These patterns may use maker-checker loops, independent recommendations, rebuttals, or a bounded judge. Their purpose is to expose weaknesses, compare interpretations, or test a result against defined criteria. They should not be used where a deterministic rule, database lookup, or policy check can produce a more reliable answer directly. Review must also be bounded by stopping conditions and a fallback path.

Escalation defines the edge of safe autonomy. It transfers a case to a path with greater authority, stronger controls, or human accountability. Returning an escalation message is only a routing decision; it does not create an enforceable review boundary. A durable human-in-the-loop workflow requires persisted state, an explicit pause, a stable thread identity, and controlled resumption. LangGraph's `interrupt()` and `Command(resume=...)` provide this runtime boundary when the graph is compiled with a checkpoint. The chapter then separated the main communication mechanisms. Handoffs represent meaningful transfers of control or responsibility and may update state while routing to the next agent. Shared message lists allow agents to collaborate through one conversational history, but they require trimming, summarisation, or filtering as the history grows. Shared state schemas provide named, typed channels for control data, business facts, specialist outputs, approvals, and other machine-readable artefacts. Messages and structured fields are complementary: messages preserve dialogue, while structured state prevents stable facts from having to be recovered from prose.

A2A communication addresses a different boundary. It is needed when agents run in separate processes or services and cannot share an in-memory state object. Remote agents communicate through protocol messages, task and context identifiers, capability metadata, and declared authentication requirements. These identifiers correlate work across services, but they do not replace message content, persistence, transport security, or application-level orchestration. A2A coordinates agents across service boundaries, while MCP provides access to tools, resources, and prompts; the two operate at different layers and may be used together. Finally,

the chapter examined multi-agent execution as a LangGraph coordination problem. Parallel nodes run in super-steps against the same incoming state and cannot see one another's updates until the step completes. When branches update the same state field, a reducer must define how those updates are merged. Mechanical merging is not semantic synthesis: a later aggregation node must still decide how the combined results should influence the final answer. A reliable graph also needs explicit stopping rules. Routing to END represents successful completion. Execution limits provide bounded exits when success is not reached. Interrupts represent a pause for external review rather than completion. A nested agent finishing its own internal loop completes only that node, not necessarily the enclosing graph. Keeping these outcomes separate prevents failures, pauses, and local completion from being mistaken for successful completion of the entire workflow. The central lesson is that a strong multi-agent system is not defined by the number of agents it contains. It is defined by the clarity of its boundaries: who owns the objective, who performs specialist work, who may challenge a result, what context is shared, how concurrent work converges, when authority ends, and how every execution path reaches a controlled outcome.